

PDS2-2026-PF-grupo13

1.0

Generated by Doxygen 1.17.0

1 Sistema de Tolerância de Falha em Fábricas	1
1.1 Motivação	1
1.2 Funcionalidades	1
1.3 Hierarquia do Sistema	2
1.4 Cargos de Usuário	2
1.5 Estrutura de Diretórios	2
1.6 Estado de Desenvolvimento	2
1.7 Programação Defensiva e Tratamento de Exceções	3
1.8 Compilação e Testes	3
1.8.1 Instalação do gcovr	3
1.9 Execução da Aplicação	4
1.10 Documentação	4
1.11 Equipe	4
2 notas-C6	5
3 notas-C7	7
4 Directory Hierarchy	9
4.1 Directories	9
5 Namespace Index	11
5.1 Namespace List	11
6 Hierarchical Index	13
6.1 Class Hierarchy	13
7 Class Index	17
7.1 Class List	17
8 File Index	21
8.1 File List	21
9 Directory Documentation	23
9.1 design Directory Reference	23
9.2 include Directory Reference	23
9.3 src Directory Reference	23
9.4 tests Directory Reference	24
10 Namespace Documentation	25
10.1 doctest Namespace Reference	25
10.1.1 Typedef Documentation	27
10.1.1.1 size_t	27
10.1.2 Function Documentation	27
10.1.2.1 assertString()	27
10.1.2.2 DOCTEST_DEFINE_DECORATOR() [1/9]	27

10.1.2.3 DOCTEST_DEFINE_DECORATOR() [2/9]	27
10.1.2.4 DOCTEST_DEFINE_DECORATOR() [3/9]	27
10.1.2.5 DOCTEST_DEFINE_DECORATOR() [4/9]	28
10.1.2.6 DOCTEST_DEFINE_DECORATOR() [5/9]	28
10.1.2.7 DOCTEST_DEFINE_DECORATOR() [6/9]	28
10.1.2.8 DOCTEST_DEFINE_DECORATOR() [7/9]	28
10.1.2.9 DOCTEST_DEFINE_DECORATOR() [8/9]	28
10.1.2.10 DOCTEST_DEFINE_DECORATOR() [9/9]	28
10.1.2.11 failureString()	28
10.1.2.12 getContextOptions()	29
10.1.2.13 operator"!=() [1/3]	29
10.1.2.14 operator"!=() [2/3]	29
10.1.2.15 operator"!=() [3/3]	29
10.1.2.16 operator+()	29
10.1.2.17 operator<()	29
10.1.2.18 operator<=()	29
10.1.2.19 operator==([1/3]	29
10.1.2.20 operator==([2/3]	30
10.1.2.21 operator==([3/3]	30
10.1.2.22 operator>()	30
10.1.2.23 operator>=()	30
10.1.2.24 registerExceptionTranslator()	30
10.1.2.25 registerReporter()	30
10.1.2.26 skipPathFromFilename()	30
10.1.2.27 toString() [1/25]	31
10.1.2.28 toString() [2/25]	31
10.1.2.29 toString() [3/25]	31
10.1.2.30 toString() [4/25]	31
10.1.2.31 toString() [5/25]	31
10.1.2.32 toString() [6/25]	31
10.1.2.33 toString() [7/25]	31
10.1.2.34 toString() [8/25]	31
10.1.2.35 toString() [9/25]	32
10.1.2.36 toString() [10/25]	32
10.1.2.37 toString() [11/25]	32
10.1.2.38 toString() [12/25]	32
10.1.2.39 toString() [13/25]	32
10.1.2.40 toString() [14/25]	32
10.1.2.41 toString() [15/25]	32
10.1.2.42 toString() [16/25]	32
10.1.2.43 toString() [17/25]	32
10.1.2.44 toString() [18/25]	33

10.1.2.45 toString() [19/25]	33
10.1.2.46 toString() [20/25]	33
10.1.2.47 toString() [21/25]	33
10.1.2.48 toString() [22/25]	33
10.1.2.49 toString() [23/25]	33
10.1.2.50 toString() [24/25]	33
10.1.2.51 toString() [25/25]	33
10.1.3 Variable Documentation	34
10.1.3.1 is_running_in_test	34
10.1.3.2 IsNaN< double >	34
10.1.3.3 IsNaN< float >	34
10.1.3.4 IsNaN< long double >	34
10.1.3.5 TestCaseData	34
10.2 doctest::assertType Namespace Reference	34
10.2.1 Enumeration Type Documentation	35
10.2.1.1 Enum	35
10.3 doctest::Color Namespace Reference	37
10.3.1 Enumeration Type Documentation	37
10.3.1.1 Enum	37
10.3.2 Function Documentation	37
10.3.2.1 operator<<()	37
10.4 doctest::detail Namespace Reference	38
10.4.1 Typedef Documentation	40
10.4.1.1 assert_handler	40
10.4.1.2 funcType	40
10.4.1.3 reporterCreatorFunc	40
10.4.2 Function Documentation	40
10.4.2.1 acquireGeneratorDecisionIndex()	40
10.4.2.2 acquireGeneratorValue()	40
10.4.2.3 binary_assert()	41
10.4.2.4 checkIfShouldThrow()	41
10.4.2.5 declval()	41
10.4.2.6 decomp_assert()	41
10.4.2.7 failed_out_of_a_testing_context()	41
10.4.2.8 filloss() [1/2]	41
10.4.2.9 filloss() [2/2]	42
10.4.2.10 forward() [1/2]	42
10.4.2.11 forward() [2/2]	42
10.4.2.12 instantiationHelper()	42
10.4.2.13 isDebuggerActive()	42
10.4.2.14 MakeContextScope()	42
10.4.2.15 registerExceptionTranslatorImpl()	42

10.4.2.16 registerReporterImpl()	43
10.4.2.17 regTest()	43
10.4.2.18 reporterCreator()	43
10.4.2.19 setTestSuite()	43
10.4.2.20 stringifyBinaryExpr()	43
10.4.2.21 throwException()	43
10.4.2.22 tlssPop()	43
10.4.2.23 tlssPush()	44
10.4.2.24 toStream()	44
10.4.2.25 unary_assert()	44
10.4.3 Variable Documentation	44
10.4.3.1 TestCase	44
10.5 doctest::detail::binaryAssertComparison Namespace Reference	44
10.5.1 Enumeration Type Documentation	45
10.5.1.1 Enum	45
10.6 doctest::detail::types Namespace Reference	45
10.6.1 Typedef Documentation	46
10.6.1.1 void_t	46
10.7 doctest::TestCaseFailureReason Namespace Reference	46
10.7.1 Enumeration Type Documentation	46
10.7.1.1 Enum	46
10.8 doctest_detail_test_suite_ns Namespace Reference	46
10.8.1 Function Documentation	47
10.8.1.1 getCurrentTestSuite()	47
10.9 std Namespace Reference	47
10.9.1 Typedef Documentation	47
10.9.1.1 istream	47
10.9.1.2 nullptr_t	47
10.9.1.3 ostream	47
10.9.1.4 size_t	48
10.9.2 Function Documentation	48
10.9.2.1 operator<<()	48
11 Class Documentation	49
11.1 AcessoNegado Class Reference	49
11.1.1 Detailed Description	49
11.2 Alarme Class Reference	49
11.2.1 Detailed Description	50
11.2.2 Constructor & Destructor Documentation	50
11.2.2.1 Alarme()	50
11.2.3 Member Function Documentation	51
11.2.3.1 AtualizarAlarme()	51

11.2.3.2 GetEstado()	51
11.2.3.3 SetDesc()	51
11.2.3.4 SetEstado()	52
11.3 doctest::Approx Struct Reference	52
11.3.1 Detailed Description	53
11.3.2 Constructor & Destructor Documentation	53
11.3.2.1 Approx()	53
11.3.3 Member Function Documentation	53
11.3.3.1 epsilon()	53
11.3.3.2 operator>()	53
11.3.3.3 scale()	53
11.3.4 Friends And Related Symbol Documentation	53
11.3.4.1 operator"!= [1/2]	53
11.3.4.2 operator"!= [2/2]	53
11.3.4.3 operator< [1/2]	54
11.3.4.4 operator< [2/2]	54
11.3.4.5 operator<= [1/2]	54
11.3.4.6 operator<= [2/2]	54
11.3.4.7 operator== [1/2]	54
11.3.4.8 operator== [2/2]	54
11.3.4.9 operator> [1/2]	54
11.3.4.10 operator> [2/2]	54
11.3.4.11 operator>= [1/2]	55
11.3.4.12 operator>= [2/2]	55
11.3.5 Member Data Documentation	55
11.3.5.1 m_epsilon	55
11.3.5.2 m_scale	55
11.3.5.3 m_value	55
11.4 doctest::AssertData Struct Reference	55
11.4.1 Detailed Description	56
11.4.2 Constructor & Destructor Documentation	56
11.4.2.1 AssertData()	56
11.4.3 Member Data Documentation	56
11.4.3.1 m_at	56
11.4.3.2 m_decomp	57
11.4.3.3 m_exception	57
11.4.3.4 m_exception_string	57
11.4.3.5 m_exception_type	57
11.4.3.6 m_expr	57
11.4.3.7 m_failed	57
11.4.3.8 m_file	57
11.4.3.9 m_line	57

11.4.3.10 m_test_case	58
11.4.3.11 m_threw	58
11.4.3.12 m_threw_as	58
11.5 Atuator Class Reference	58
11.5.1 Detailed Description	59
11.5.2 Constructor & Destructor Documentation	59
11.5.2.1 Atuator()	59
11.5.2.2 ~Atuator()	60
11.5.3 Member Function Documentation	60
11.5.3.1 Tipo()	60
11.6 std::basic_istream< charT, traits > Class Template Reference	60
11.6.1 Detailed Description	60
11.7 std::basic_ostream< charT, traits > Class Template Reference	60
11.7.1 Detailed Description	60
11.8 std::char_traits< charT > Struct Template Reference	61
11.8.1 Detailed Description	61
11.9 Conjunto Class Reference	61
11.9.1 Detailed Description	62
11.9.2 Constructor & Destructor Documentation	62
11.9.2.1 Conjunto()	62
11.9.3 Member Function Documentation	62
11.9.3.1 AcessarEquipamento()	62
11.9.3.2 AdicionarEquipamento() [1/2]	63
11.9.3.3 AdicionarEquipamento() [2/2]	63
11.9.3.4 AtualizarAlarmes()	63
11.9.3.5 DiagnosticarConjunto()	64
11.9.3.6 ExibirAlarmes()	64
11.9.3.7 ExibirEquipamentos()	64
11.9.3.8 ExibirTudo()	64
11.9.3.9 GetDesc()	65
11.9.3.10 GetEquipamentos()	65
11.9.3.11 GetId()	65
11.9.3.12 GetNome()	65
11.9.3.13 ProximoldEquipamento()	65
11.9.3.14 QuantidadeAlarmes()	66
11.9.3.15 QuantidadeEquipamentos()	66
11.9.3.16 RemoverEquipamento()	66
11.9.3.17 SetDesc()	67
11.9.3.18 SetNome()	67
11.10 doctest::Contains Class Reference	67
11.10.1 Detailed Description	67
11.10.2 Constructor & Destructor Documentation	68

11.10.2.1 Contains()	68
11.10.3 Member Function Documentation	68
11.10.3.1 checkWith()	68
11.10.4 Member Data Documentation	68
11.10.4.1 string	68
11.11 doctest::Context Class Reference	68
11.11.1 Detailed Description	69
11.11.2 Constructor & Destructor Documentation	69
11.11.2.1 Context() [1/3]	69
11.11.2.2 Context() [2/3]	69
11.11.2.3 Context() [3/3]	69
11.11.2.4 ~Context()	69
11.11.3 Member Function Documentation	69
11.11.3.1 addFilter()	69
11.11.3.2 applyCommandLine()	69
11.11.3.3 clearFilters()	69
11.11.3.4 operator=() [1/2]	70
11.11.3.5 operator=() [2/2]	70
11.11.3.6 run()	70
11.11.3.7 setAsDefaultForAssertsOutOfTestCases()	70
11.11.3.8 setAssertHandler()	70
11.11.3.9 setCout()	70
11.11.3.10 setOption() [1/3]	70
11.11.3.11 setOption() [2/3]	70
11.11.3.12 setOption() [3/3]	70
11.11.3.13 shouldExit()	71
11.12 doctest::ContextOptions Struct Reference	71
11.12.1 Detailed Description	72
11.12.2 Member Data Documentation	72
11.12.2.1 abort_after	72
11.12.2.2 binary_name	72
11.12.2.3 case_sensitive	72
11.12.2.4 count	72
11.12.2.5 cout	72
11.12.2.6 currentTest	72
11.12.2.7 duration	73
11.12.2.8 exit	73
11.12.2.9 first	73
11.12.2.10 force_colors	73
11.12.2.11 gnu_file_line	73
11.12.2.12 help	73
11.12.2.13 last	73

11.12.2.14 list_reporters	73
11.12.2.15 list_test_cases	74
11.12.2.16 list_test_suites	74
11.12.2.17 minimal	74
11.12.2.18 no_breaks	74
11.12.2.19 no_colors	74
11.12.2.20 no_debug_output	74
11.12.2.21 no_exitcode	74
11.12.2.22 no_intro	74
11.12.2.23 no_line_numbers	75
11.12.2.24 no_path_in_filenames	75
11.12.2.25 no_run	75
11.12.2.26 no_skip	75
11.12.2.27 no_skipped_summary	75
11.12.2.28 no_throw	75
11.12.2.29 no_time_in_output	75
11.12.2.30 no_version	75
11.12.2.31 order_by	76
11.12.2.32 out	76
11.12.2.33 quiet	76
11.12.2.34 rand_seed	76
11.12.2.35 strip_file_prefixes	76
11.12.2.36 subcase_filter_levels	76
11.12.2.37 success	76
11.12.2.38 version	77
11.13 doctest::detail::ContextScope< L > Class Template Reference	77
11.13.1 Detailed Description	78
11.13.2 Constructor & Destructor Documentation	78
11.13.2.1 ContextScope() [1/4]	78
11.13.2.2 ContextScope() [2/4]	78
11.13.2.3 ContextScope() [3/4]	78
11.13.2.4 ContextScope() [4/4]	78
11.13.2.5 ~ContextScope()	78
11.13.3 Member Function Documentation	79
11.13.3.1 operator=() [1/2]	79
11.13.3.2 operator=() [2/2]	79
11.13.3.3 stringify()	79
11.14 doctest::detail::ContextScopeBase Struct Reference	79
11.14.1 Detailed Description	80
11.14.2 Constructor & Destructor Documentation	80
11.14.2.1 ContextScopeBase() [1/3]	80
11.14.2.2 ~ContextScopeBase()	80

11.14.2.3 ContextScopeBase() [2/3]	80
11.14.2.4 ContextScopeBase() [3/3]	80
11.14.3 Member Function Documentation	80
11.14.3.1 destroy()	80
11.14.3.2 operator=() [1/2]	81
11.14.3.3 operator=() [2/2]	81
11.14.4 Member Data Documentation	81
11.14.4.1 need_to_destroy	81
11.15 doctest::CurrentTestCaseStats Struct Reference	81
11.15.1 Detailed Description	81
11.15.2 Member Data Documentation	81
11.15.2.1 failure_flags	81
11.15.2.2 numAssertsCurrentTest	82
11.15.2.3 numAssertsFailedCurrentTest	82
11.15.2.4 seconds	82
11.15.2.5 testCaseSuccess	82
11.16 doctest::detail::deferred_false< T > Struct Template Reference	82
11.16.1 Detailed Description	83
11.17 Dependencia Class Reference	83
11.17.1 Detailed Description	83
11.17.2 Constructor & Destructor Documentation	83
11.17.2.1 Dependencia()	83
11.17.3 Member Function Documentation	83
11.17.3.1 Get_Condicao()	83
11.17.3.2 Get_Id()	84
11.17.3.3 Get_Resultado()	84
11.18 doctest::detail::types::enable_if< COND, T > Struct Template Reference	84
11.18.1 Detailed Description	84
11.19 doctest::detail::types::enable_if< true, T > Struct Template Reference	84
11.19.1 Detailed Description	85
11.19.2 Member Typedef Documentation	85
11.19.2.1 type	85
11.20 EntradaInvalida Class Reference	85
11.20.1 Detailed Description	85
11.21 Equipamento Class Reference	86
11.21.1 Detailed Description	87
11.21.2 Constructor & Destructor Documentation	87
11.21.2.1 Equipamento()	87
11.21.2.2 ~Equipamento()	87
11.21.3 Member Function Documentation	87
11.21.3.1 AdicionarParametro() [1/4]	87
11.21.3.2 AdicionarParametro() [2/4]	88

11.21.3.3 AdicionarParametro() [3/4]	88
11.21.3.4 AdicionarParametro() [4/4]	88
11.21.3.5 AtualizarAlarmes()	89
11.21.3.6 DiagnosticarEquipamento()	89
11.21.3.7 ExibirAlarmes()	89
11.21.3.8 ExibirParametros()	90
11.21.3.9 GetAlarmes()	90
11.21.3.10 GetDesc()	90
11.21.3.11 GetId()	90
11.21.3.12 GetNome()	90
11.21.3.13 GetParametros()	91
11.21.3.14 ProximoldParametro()	91
11.21.3.15 QuantidadeAlarmes()	91
11.21.3.16 QuantidadeParametros()	91
11.21.3.17 RemoverParametro()	92
11.21.3.18 SetDesc()	92
11.21.3.19 Tipo()	92
11.22 ExcecaoSistema Class Reference	93
11.22.1 Detailed Description	93
11.23 doctest::detail::ExceptionTranslator< T > Class Template Reference	93
11.23.1 Detailed Description	94
11.23.2 Constructor & Destructor Documentation	94
11.23.2.1 ExceptionTranslator()	94
11.23.3 Member Function Documentation	94
11.23.3.1 translate()	94
11.24 doctest::detail::Expression_lhs< L > Struct Template Reference	94
11.24.1 Detailed Description	95
11.24.2 Constructor & Destructor Documentation	95
11.24.2.1 Expression_lhs()	95
11.24.3 Member Function Documentation	95
11.24.3.1 operator L()	95
11.24.3.2 operator Result()	95
11.24.4 Member Data Documentation	95
11.24.4.1 lhs	95
11.24.4.2 m_at	96
11.25 doctest::detail::ExpressionDecomposer Struct Reference	96
11.25.1 Detailed Description	96
11.25.2 Constructor & Destructor Documentation	96
11.25.2.1 ExpressionDecomposer()	96
11.25.3 Member Function Documentation	96
11.25.3.1 operator<<() [1/2]	96
11.25.3.2 operator<<() [2/2]	97

11.25.4 Member Data Documentation	97
11.25.4.1 m_at	97
11.26 doctest::detail::types::false_type Struct Reference	97
11.26.1 Detailed Description	97
11.26.2 Member Data Documentation	98
11.26.2.1 value	98
11.27 doctest::detail::filldata< T, Enable > Struct Template Reference	98
11.27.1 Detailed Description	98
11.27.2 Member Function Documentation	98
11.27.2.1 fill()	98
11.28 doctest::detail::filldata< const char[N]> Struct Template Reference	98
11.28.1 Detailed Description	99
11.28.2 Member Function Documentation	99
11.28.2.1 fill()	99
11.29 doctest::detail::filldata< const void * > Struct Reference	99
11.29.1 Detailed Description	99
11.29.2 Member Function Documentation	99
11.29.2.1 fill()	99
11.30 doctest::detail::filldata< const volatile void * > Struct Reference	100
11.30.1 Detailed Description	100
11.30.2 Member Function Documentation	100
11.30.2.1 fill()	100
11.31 doctest::detail::filldata< T * > Struct Template Reference	100
11.31.1 Detailed Description	100
11.31.2 Member Function Documentation	101
11.31.2.1 fill()	101
11.32 doctest::detail::filldata< T, typename detail::types::enable_if< !detail::has_insertion_operator< T >::value &&detail::is_container< T >::value >::type > Struct Template Reference	101
11.32.1 Detailed Description	101
11.32.2 Member Function Documentation	101
11.32.2.1 fill()	101
11.33 doctest::detail::filldata< T, typename detail::types::enable_if< !detail::has_insertion_operator< T >::value &&detail::is_pair< T >::value >::type > Struct Template Reference	102
11.33.1 Detailed Description	102
11.33.2 Member Function Documentation	102
11.33.2.1 fill()	102
11.34 doctest::detail::filldata< T[N]> Struct Template Reference	102
11.34.1 Detailed Description	103
11.34.2 Member Function Documentation	103
11.34.2.1 fill()	103
11.35 doctest::detail::has_insertion_operator< T, typename > Struct Template Reference	103
11.35.1 Detailed Description	103

11.36 doctest::detail::has_insertion_operator< T, decltype(operator<<(declval< std::ostream & >()), declval< const T & >()), void())> Struct Template Reference	104
11.36.1 Detailed Description	104
11.37 doctest::IContextScope Struct Reference	104
11.37.1 Detailed Description	105
11.37.2 Member Function Documentation	105
11.37.2.1 stringify()	105
11.38 IdDuplicado Class Reference	105
11.38.1 Detailed Description	105
11.39 IdInexistente Class Reference	106
11.39.1 Detailed Description	106
11.40 IdInvalido Class Reference	106
11.40.1 Detailed Description	106
11.41 doctest::detail::IExceptionTranslator Struct Reference	107
11.41.1 Detailed Description	107
11.41.2 Member Function Documentation	107
11.41.2.1 translate()	107
11.42 doctest::IReporter Struct Reference	107
11.42.1 Detailed Description	108
11.42.2 Member Function Documentation	108
11.42.2.1 get_active_contexts()	108
11.42.2.2 get_num_active_contexts()	108
11.42.2.3 get_num_stringified_contexts()	108
11.42.2.4 get_stringified_contexts()	108
11.42.2.5 log_assert()	108
11.42.2.6 log_message()	108
11.42.2.7 report_query()	108
11.42.2.8 subcase_end()	109
11.42.2.9 subcase_start()	109
11.42.2.10 test_case_end()	109
11.42.2.11 test_case_exception()	109
11.42.2.12 test_case_reenter()	109
11.42.2.13 test_case_skipped()	109
11.42.2.14 test_case_start()	109
11.42.2.15 test_run_end()	109
11.42.2.16 test_run_start()	109
11.43 doctest::detail::types::is_array< T > Struct Template Reference	110
11.43.1 Detailed Description	110
11.44 doctest::detail::types::is_array< T[SIZE]> Struct Template Reference	110
11.44.1 Detailed Description	111
11.45 doctest::detail::is_container< T, typename > Struct Template Reference	111
11.45.1 Detailed Description	111

11.46 doctest::detail::is_container< T, types::void_t< typename T::value_type, typename T::iterator, decltype(declval< T >().begin()), decltype(declval< T >().end())> > Struct Template Reference	112
11.46.1 Detailed Description	112
11.47 doctest::detail::types::is_enum< T > Struct Template Reference	112
11.47.1 Detailed Description	113
11.47.2 Member Data Documentation	113
11.47.2.1 value	113
11.48 doctest::detail::is_pair< T, typename > Struct Template Reference	113
11.48.1 Detailed Description	113
11.49 doctest::detail::is_pair< T, types::void_t< typename T::first_type, typename T::second_type, decltype(declval< T >().first), decltype(declval< T >().second)> > Struct Template Reference	114
11.49.1 Detailed Description	114
11.50 doctest::detail::types::is_pointer< T > Struct Template Reference	114
11.50.1 Detailed Description	115
11.51 doctest::detail::types::is_pointer< T * > Struct Template Reference	115
11.51.1 Detailed Description	115
11.52 doctest::detail::types::is_rvalue_reference< T > Struct Template Reference	116
11.52.1 Detailed Description	116
11.53 doctest::detail::types::is_rvalue_reference< T && > Struct Template Reference	116
11.53.1 Detailed Description	117
11.54 doctest::detail::types::is_same< T, U > Struct Template Reference	117
11.54.1 Detailed Description	117
11.55 doctest::detail::types::is_same< T, T > Struct Template Reference	118
11.55.1 Detailed Description	118
11.56 doctest::detail::is_std_string< T, Enable > Struct Template Reference	118
11.56.1 Detailed Description	119
11.57 doctest::detail::is_std_string< T, typename types::enable_if< types::is_same< decltype(declval< const T & >().c_str()), const char * >::value &&types::is_same< decltype(declval< const T & >().size()), size_t >::value >::type > Struct Template Reference	119
11.57.1 Detailed Description	120
11.58 doctest::IsNaN< F > Struct Template Reference	120
11.58.1 Detailed Description	120
11.58.2 Constructor & Destructor Documentation	120
11.58.2.1 IsNaN()	120
11.58.3 Member Function Documentation	121
11.58.3.1 operator bool()	121
11.58.3.2 operator"!()	121
11.58.4 Member Data Documentation	121
11.58.4.1 flipped	121
11.58.4.2 value	121
11.59 LimitesInvalidos Class Reference	121
11.59.1 Detailed Description	122
11.60 LinhaDeProducao Class Reference	122

11.60.1 Detailed Description	123
11.60.2 Constructor & Destructor Documentation	123
11.60.2.1 LinhaDeProducao()	123
11.60.3 Member Function Documentation	123
11.60.3.1 AcessarConjunto()	123
11.60.3.2 AdicionarConjunto() [1/2]	124
11.60.3.3 AdicionarConjunto() [2/2]	124
11.60.3.4 AtualizarAlarmes()	124
11.60.3.5 DiagnosticarLinha()	125
11.60.3.6 ExibirAlarmes()	125
11.60.3.7 ExibirConjuntos()	125
11.60.3.8 ExibirTudo()	125
11.60.3.9 GetConjuntos()	126
11.60.3.10 GetDesc()	126
11.60.3.11 GetId()	126
11.60.3.12 GetLocal()	126
11.60.3.13 GetNome()	126
11.60.3.14 ProximoidConjunto()	127
11.60.3.15 QuantidadeAlarmes()	127
11.60.3.16 QuantidadeConjuntos()	127
11.60.3.17 RemoverConjunto()	127
11.60.3.18 SetDesc()	128
11.60.3.19 SetLocal()	128
11.60.3.20 SetNome()	128
11.61 Log Class Reference	128
11.61.1 Detailed Description	129
11.61.2 Constructor & Destructor Documentation	129
11.61.2.1 Log()	129
11.61.3 Member Function Documentation	129
11.61.3.1 GetArquivoLog()	129
11.61.3.2 Registrar()	130
11.61.3.3 SetArquivoLog()	130
11.62 doctest::detail::MessageBuilder Struct Reference	130
11.62.1 Detailed Description	131
11.62.2 Constructor & Destructor Documentation	131
11.62.2.1 MessageBuilder() [1/3]	131
11.62.2.2 MessageBuilder() [2/3]	131
11.62.2.3 MessageBuilder() [3/3]	132
11.62.2.4 ~MessageBuilder()	132
11.62.3 Member Function Documentation	132
11.62.3.1 log()	132
11.62.3.2 operator*()	132

11.62.3.3 operator,()	132
11.62.3.4 operator<<()	132
11.62.3.5 operator=() [1/2]	132
11.62.3.6 operator=() [2/2]	133
11.62.3.7 react()	133
11.62.4 Member Data Documentation	133
11.62.4.1 logged	133
11.62.4.2 m_stream	133
11.63 doctest::MessageData Struct Reference	133
11.63.1 Detailed Description	134
11.63.2 Member Data Documentation	134
11.63.2.1 m_file	134
11.63.2.2 m_line	134
11.63.2.3 m_severity	134
11.63.2.4 m_string	134
11.64 Motor Class Reference	134
11.64.1 Detailed Description	135
11.64.2 Constructor & Destructor Documentation	136
11.64.2.1 Motor()	136
11.64.2.2 ~Motor()	136
11.64.3 Member Function Documentation	136
11.64.3.1 Tipo()	136
11.65 Parametro Class Reference	136
11.65.1 Detailed Description	137
11.65.2 Constructor & Destructor Documentation	137
11.65.2.1 Parametro() [1/2]	137
11.65.2.2 Parametro() [2/2]	138
11.65.3 Member Function Documentation	138
11.65.3.1 DiagnosticarParametro()	138
11.65.3.2 GetDesc()	139
11.65.3.3 GetEstado()	139
11.65.3.4 GetHist()	139
11.65.3.5 GetId()	139
11.65.3.6 GetMax()	140
11.65.3.7 GetMin()	140
11.65.3.8 GetValorAtual()	140
11.65.3.9 ResetarHistorico()	140
11.65.3.10 ResetarLimites()	140
11.65.3.11 SetDesc()	141
11.65.3.12 SetMax()	141
11.65.3.13 SetMin()	141
11.65.3.14 SetValorAtual()	142

11.65.3.15 ValorValido()	142
11.66 doctest::QueryData Struct Reference	142
11.66.1 Detailed Description	143
11.66.2 Member Data Documentation	143
11.66.2.1 data	143
11.66.2.2 num_data	143
11.66.2.3 run_stats	143
11.67 doctest::detail::RelationalComparator< int, L, R > Struct Template Reference	143
11.67.1 Detailed Description	143
11.67.2 Member Function Documentation	144
11.67.2.1 operator()	144
11.68 doctest::detail::types::remove_const< T > Struct Template Reference	144
11.68.1 Detailed Description	144
11.68.2 Member Typedef Documentation	144
11.68.2.1 type	144
11.69 doctest::detail::types::remove_const< const T > Struct Template Reference	145
11.69.1 Detailed Description	145
11.69.2 Member Typedef Documentation	145
11.69.2.1 type	145
11.70 doctest::detail::types::remove_reference< T > Struct Template Reference	145
11.70.1 Detailed Description	145
11.70.2 Member Typedef Documentation	146
11.70.2.1 type	146
11.71 doctest::detail::types::remove_reference< T & > Struct Template Reference	146
11.71.1 Detailed Description	146
11.71.2 Member Typedef Documentation	146
11.71.2.1 type	146
11.72 doctest::detail::types::remove_reference< T && > Struct Template Reference	146
11.72.1 Detailed Description	147
11.72.2 Member Typedef Documentation	147
11.72.2.1 type	147
11.73 doctest::detail::Result Struct Reference	147
11.73.1 Detailed Description	147
11.73.2 Constructor & Destructor Documentation	148
11.73.2.1 Result() [1/2]	148
11.73.2.2 Result() [2/2]	148
11.73.3 Member Data Documentation	148
11.73.3.1 m_decomp	148
11.73.3.2 m_passed	148
11.74 doctest::detail::ResultBuilder Struct Reference	148
11.74.1 Detailed Description	149
11.74.2 Constructor & Destructor Documentation	149

11.74.2.1 ResultBuilder() [1/2]	149
11.74.2.2 ResultBuilder() [2/2]	150
11.74.3 Member Function Documentation	150
11.74.3.1 binary_assert()	150
11.74.3.2 log()	150
11.74.3.3 react()	150
11.74.3.4 setResult()	150
11.74.3.5 translateException()	150
11.74.3.6 unary_assert()	150
11.75 Sensor Class Reference	151
11.75.1 Detailed Description	152
11.75.2 Constructor & Destructor Documentation	152
11.75.2.1 Sensor()	152
11.75.2.2 ~Sensor()	152
11.75.3 Member Function Documentation	152
11.75.3.1 Tipo()	152
11.76 SessaoInvalida Class Reference	153
11.76.1 Detailed Description	153
11.77 doctest::detail::should_stringify_as_underlying_type< T > Struct Template Reference	153
11.77.1 Detailed Description	153
11.77.2 Member Data Documentation	154
11.77.2.1 value	154
11.78 Sistema Class Reference	154
11.78.1 Detailed Description	155
11.78.2 Constructor & Destructor Documentation	155
11.78.2.1 Sistema()	155
11.78.3 Member Function Documentation	155
11.78.3.1 AcessarLinha()	155
11.78.3.2 AdicionarLinha() [1/2]	156
11.78.3.3 AdicionarLinha() [2/2]	156
11.78.3.4 AdicionarUsuario()	156
11.78.3.5 CarregarSave()	157
11.78.3.6 CarregarUltimoSave()	157
11.78.3.7 ExibirLinhas()	158
11.78.3.8 GetLinhas()	158
11.78.3.9 GetUsuarioLogado()	158
11.78.3.10 IdAtivo()	158
11.78.3.11 Login()	159
11.78.3.12 Logout()	159
11.78.3.13 ProximoldDisponivel()	159
11.78.3.14 RemoverLinha()	160
11.78.3.15 RemoverUsuario()	160

11.78.3.16 <code>SalvarAlteracoes()</code>	160
11.79 <code>doctest::String</code> Class Reference	161
11.79.1 Detailed Description	161
11.79.2 Member Typedef Documentation	162
11.79.2.1 <code>size_type</code>	162
11.79.3 Constructor & Destructor Documentation	162
11.79.3.1 <code>String()</code> [1/7]	162
11.79.3.2 <code>~String()</code>	162
11.79.3.3 <code>String()</code> [2/7]	162
11.79.3.4 <code>String()</code> [3/7]	162
11.79.3.5 <code>String()</code> [4/7]	162
11.79.3.6 <code>String()</code> [5/7]	162
11.79.3.7 <code>String()</code> [6/7]	163
11.79.3.8 <code>String()</code> [7/7]	163
11.79.4 Member Function Documentation	163
11.79.4.1 <code>c_str()</code> [1/2]	163
11.79.4.2 <code>c_str()</code> [2/2]	163
11.79.4.3 <code>capacity()</code>	163
11.79.4.4 <code>compare()</code> [1/2]	163
11.79.4.5 <code>compare()</code> [2/2]	163
11.79.4.6 <code>find()</code>	163
11.79.4.7 <code>operator+=(())</code>	164
11.79.4.8 <code>operator=()</code> [1/2]	164
11.79.4.9 <code>operator=()</code> [2/2]	164
11.79.4.10 <code>operator[]()</code> [1/2]	164
11.79.4.11 <code>operator[]()</code> [2/2]	164
11.79.4.12 <code>rfind()</code>	164
11.79.4.13 <code>size()</code>	164
11.79.4.14 <code>substr()</code> [1/2]	164
11.79.4.15 <code>substr()</code> [2/2]	165
11.79.5 Friends And Related Symbol Documentation	165
11.79.5.1 <code>operator<<</code>	165
11.79.6 Member Data Documentation	165
11.79.6.1 <code>buf</code>	165
11.79.6.2 <code>data</code>	165
11.79.6.3 <code>npos</code>	165
11.80 <code>doctest::AssertData::StringContains</code> Class Reference	165
11.80.1 Detailed Description	166
11.80.2 Constructor & Destructor Documentation	166
11.80.2.1 <code>StringContains()</code> [1/2]	166
11.80.2.2 <code>StringContains()</code> [2/2]	166
11.80.3 Member Function Documentation	166

11.80.3.1 <code>c_str()</code>	166
11.80.3.2 <code>check()</code>	166
11.80.3.3 <code>operator const String &()</code>	167
11.81 <code>doctest::StringMaker< T > Struct Template Reference</code>	167
11.81.1 Detailed Description	167
11.82 <code>doctest::detail::StringMakerBase< C > Struct Template Reference</code>	167
11.82.1 Detailed Description	168
11.82.2 Member Function Documentation	168
11.82.2.1 <code>convert()</code>	168
11.83 <code>doctest::detail::StringMakerBase< true > Struct Reference</code>	168
11.83.1 Detailed Description	168
11.83.2 Member Function Documentation	169
11.83.2.1 <code>convert()</code>	169
11.84 <code>doctest::detail::Subcase Struct Reference</code>	169
11.84.1 Detailed Description	169
11.84.2 Constructor & Destructor Documentation	169
11.84.2.1 <code>Subcase()</code> [1/3]	169
11.84.2.2 <code>Subcase()</code> [2/3]	170
11.84.2.3 <code>Subcase()</code> [3/3]	170
11.84.2.4 <code>~Subcase()</code>	170
11.84.3 Member Function Documentation	170
11.84.3.1 <code>operator bool()</code>	170
11.84.3.2 <code>operator=()</code> [1/2]	170
11.84.3.3 <code>operator=()</code> [2/2]	170
11.84.4 Member Data Documentation	170
11.84.4.1 <code>m_entered</code>	170
11.84.4.2 <code>m_signature</code>	170
11.85 <code>doctest::SubcaseSignature Struct Reference</code>	171
11.85.1 Detailed Description	171
11.85.2 Member Function Documentation	171
11.85.2.1 <code>operator<()</code>	171
11.85.2.2 <code>operator==(())</code>	171
11.85.3 Member Data Documentation	171
11.85.3.1 <code>m_file</code>	171
11.85.3.2 <code>m_line</code>	171
11.85.3.3 <code>m_name</code>	172
11.86 <code>doctest::detail::TestCase Struct Reference</code>	172
11.86.1 Detailed Description	173
11.86.2 Constructor & Destructor Documentation	173
11.86.2.1 <code>TestCase()</code> [1/3]	173
11.86.2.2 <code>TestCase()</code> [2/3]	173
11.86.2.3 <code>TestCase()</code> [3/3]	173

11.86.2.4 ~TestCase()	173
11.86.3 Member Function Documentation	174
11.86.3.1 operator*() [1/2]	174
11.86.3.2 operator*() [2/2]	174
11.86.3.3 operator<()	174
11.86.3.4 operator=() [1/2]	174
11.86.3.5 operator=() [2/2]	174
11.86.4 Member Data Documentation	174
11.86.4.1 m_full_name	174
11.86.4.2 m_template_id	174
11.86.4.3 m_test	175
11.86.4.4 m_type	175
11.87 doctest::TestCaseData Struct Reference	175
11.87.1 Detailed Description	175
11.87.2 Member Data Documentation	176
11.87.2.1 m_description	176
11.87.2.2 m_expected_failures	176
11.87.2.3 m_file	176
11.87.2.4 m_line	176
11.87.2.5 m_may_fail	176
11.87.2.6 m_name	176
11.87.2.7 m_no_breaks	176
11.87.2.8 m_no_output	177
11.87.2.9 m_should_fail	177
11.87.2.10 m_skip	177
11.87.2.11 m_test_suite	177
11.87.2.12 m_timeout	177
11.88 doctest::TestCaseException Struct Reference	177
11.88.1 Detailed Description	178
11.88.2 Member Data Documentation	178
11.88.2.1 error_string	178
11.88.2.2 is_crash	178
11.89 doctest::detail::TestFailureException Struct Reference	178
11.89.1 Detailed Description	178
11.90 doctest::TestRunStats Struct Reference	178
11.90.1 Detailed Description	179
11.90.2 Member Data Documentation	179
11.90.2.1 numAsserts	179
11.90.2.2 numAssertsFailed	179
11.90.2.3 numTestCases	179
11.90.2.4 numTestCasesFailed	179
11.90.2.5 numTestCasesPassingFilters	179

11.90.2.6 numTestSuitesPassingFilters	179
11.91 doctest::detail::TestSuite Struct Reference	180
11.91.1 Detailed Description	180
11.91.2 Member Function Documentation	180
11.91.2.1 operator*() [1/2]	180
11.91.2.2 operator*() [2/2]	180
11.91.3 Member Data Documentation	180
11.91.3.1 m_description	180
11.91.3.2 m_expected_failures	181
11.91.3.3 m_may_fail	181
11.91.3.4 m_no_breaks	181
11.91.3.5 m_no_output	181
11.91.3.6 m_should_fail	181
11.91.3.7 m_skip	181
11.91.3.8 m_test_suite	181
11.91.3.9 m_timeout	182
11.92 doctest::detail::types::true_type Struct Reference	182
11.92.1 Detailed Description	182
11.92.2 Member Data Documentation	182
11.92.2.1 value	182
11.93 std::tuple< Types > Class Template Reference	183
11.93.1 Detailed Description	183
11.94 doctest::detail::types::underlying_type< T > Struct Template Reference	183
11.94.1 Detailed Description	183
11.94.2 Member Typedef Documentation	183
11.94.2.1 type	183
11.95 Usuario Class Reference	184
11.95.1 Detailed Description	184
11.95.2 Constructor & Destructor Documentation	184
11.95.2.1 Usuario()	184
11.95.3 Member Function Documentation	185
11.95.3.1 GetCargo() [1/2]	185
11.95.3.2 GetCargo() [2/2]	185
11.95.3.3 GetLogin()	185
11.95.3.4 GetSenha()	185
11.95.3.5 ValidarSenha()	186
11.96 Valvula Class Reference	186
11.96.1 Detailed Description	187
11.96.2 Constructor & Destructor Documentation	187
11.96.2.1 Valvula()	187
11.96.2.2 ~Valvula()	188
11.96.3 Member Function Documentation	188

11.96.3.1 Tipo()	188
12 File Documentation	189
12.1 design/notas-C6.md File Reference	189
12.2 design/notas-C7.md File Reference	189
12.3 include/Alarme.hpp File Reference	189
12.4 Alarme.hpp	189
12.5 include/Atuador.hpp File Reference	190
12.6 Atuador.hpp	190
12.7 include/Cargo.hpp File Reference	190
12.7.1 Enumeration Type Documentation	190
12.7.1.1 Cargo	190
12.8 Cargo.hpp	191
12.9 include/Conjunto.hpp File Reference	191
12.10 Conjunto.hpp	191
12.11 include/Dependencia.hpp File Reference	192
12.12 Dependencia.hpp	192
12.13 include/Equipamento.hpp File Reference	193
12.14 Equipamento.hpp	193
12.15 include/EstadoOperacional.hpp File Reference	194
12.15.1 Enumeration Type Documentation	194
12.15.1.1 EstadoOperacional	194
12.16 EstadoOperacional.hpp	194
12.17 include/Excecoes.hpp File Reference	194
12.18 Excecoes.hpp	195
12.19 include/LinhaDeProducao.hpp File Reference	196
12.20 LinhaDeProducao.hpp	196
12.21 include/Log.hpp File Reference	197
12.22 Log.hpp	197
12.23 include/Motor.hpp File Reference	197
12.24 Motor.hpp	198
12.25 include/Parametro.hpp File Reference	198
12.26 Parametro.hpp	198
12.27 include/Sensor.hpp File Reference	199
12.28 Sensor.hpp	199
12.29 include/Sistema.hpp File Reference	199
12.30 Sistema.hpp	200
12.31 include/Usuario.hpp File Reference	200
12.32 Usuario.hpp	201
12.33 include/Valvula.hpp File Reference	201
12.34 Valvula.hpp	201
12.35 README.md File Reference	202

12.36 src/Alarme.cpp File Reference	202
12.37 Alarme.cpp	202
12.38 src/Atuador.cpp File Reference	202
12.39 Atuador.cpp	202
12.40 src/Conjunto.cpp File Reference	203
12.41 Conjunto.cpp	203
12.42 src/Dependencia.cpp File Reference	204
12.43 Dependencia.cpp	205
12.44 src/Equipamento.cpp File Reference	205
12.45 Equipamento.cpp	205
12.46 src/LinhaDeProducao.cpp File Reference	207
12.47 LinhaDeProducao.cpp	207
12.48 src/Log.cpp File Reference	208
12.49 Log.cpp	208
12.50 src/main.cpp File Reference	209
12.50.1 Function Documentation	209
12.50.1.1 main()	209
12.51 main.cpp	210
12.52 src/Motor.cpp File Reference	213
12.53 Motor.cpp	213
12.54 src/Parametro.cpp File Reference	214
12.55 Parametro.cpp	214
12.56 src/Sensor.cpp File Reference	215
12.57 Sensor.cpp	215
12.58 src/Sistema.cpp File Reference	216
12.59 Sistema.cpp	216
12.60 src/Usuario.cpp File Reference	219
12.61 Usuario.cpp	219
12.62 src/Valvula.cpp File Reference	220
12.63 Valvula.cpp	220
12.64 tests/doctest.h File Reference	220
12.64.1 Macro Definition Documentation	233
12.64.1.1 ADD_FAIL_AT	233
12.64.1.2 ADD_FAIL_CHECK_AT	233
12.64.1.3 ADD_MESSAGE_AT	233
12.64.1.4 AND_GIVEN	234
12.64.1.5 AND_THEN	234
12.64.1.6 AND_WHEN	234
12.64.1.7 CAPTURE	234
12.64.1.8 CHECK	234
12.64.1.9 CHECK_EQ	235
12.64.1.10 CHECK_FALSE	235

12.64.1.11 CHECK_FALSE_MESSAGE	235
12.64.1.12 CHECK_GE	235
12.64.1.13 CHECK_GT	235
12.64.1.14 CHECK_LE	236
12.64.1.15 CHECK_LT	236
12.64.1.16 CHECK_MESSAGE	236
12.64.1.17 CHECK_NE	236
12.64.1.18 CHECK_NOTHROW	236
12.64.1.19 CHECK_NOTHROW_MESSAGE	237
12.64.1.20 CHECK_THROWS	237
12.64.1.21 CHECK_THROWS_AS	237
12.64.1.22 CHECK_THROWS_AS_MESSAGE	237
12.64.1.23 CHECK_THROWS_MESSAGE	237
12.64.1.24 CHECK_THROWS_WITH	238
12.64.1.25 CHECK_THROWS_WITH_AS	238
12.64.1.26 CHECK_THROWS_WITH_AS_MESSAGE	238
12.64.1.27 CHECK_THROWS_WITH_MESSAGE	238
12.64.1.28 CHECK_UNARY	239
12.64.1.29 CHECK_UNARY_FALSE	239
12.64.1.30 DOCTEST_ADD_AT_IMPL	239
12.64.1.31 DOCTEST_ADD_FAIL_AT	239
12.64.1.32 DOCTEST_ADD_FAIL_CHECK_AT	240
12.64.1.33 DOCTEST_ADD_MESSAGE_AT	240
12.64.1.34 DOCTEST_ALIGNMENT	240
12.64.1.35 DOCTEST_AND_GIVEN	240
12.64.1.36 DOCTEST_AND_THEN	240
12.64.1.37 DOCTEST_AND_WHEN	241
12.64.1.38 DOCTEST_ANONYMOUS	241
12.64.1.39 DOCTEST_ASSERT_IMPLEMENT_1	241
12.64.1.40 DOCTEST_ASSERT_IMPLEMENT_2	241
12.64.1.41 DOCTEST_ASSERT_IN_TESTS	242
12.64.1.42 DOCTEST_ASSERT_LOG_REACT_RETURN	242
12.64.1.43 DOCTEST_ASSERT_OUT_OF_TESTS	242
12.64.1.44 DOCTEST_BINARY_ASSERT	243
12.64.1.45 DOCTEST_BINARY_RELATIONAL_OP	243
12.64.1.46 DOCTEST_BREAK_INTO_DEBUGGER	243
12.64.1.47 DOCTEST_CAPTURE	243
12.64.1.48 DOCTEST_CAST_TO_VOID	244
12.64.1.49 DOCTEST_CAT	244
12.64.1.50 DOCTEST_CAT_IMPL	244
12.64.1.51 DOCTEST_CHECK	244
12.64.1.52 DOCTEST_CHECK_EQ	244

12.64.1.53 DOCTEST_CHECK_FALSE	245
12.64.1.54 DOCTEST_CHECK_FALSE_MESSAGE	245
12.64.1.55 DOCTEST_CHECK_GE	245
12.64.1.56 DOCTEST_CHECK_GT	245
12.64.1.57 DOCTEST_CHECK_LE	245
12.64.1.58 DOCTEST_CHECK_LT	246
12.64.1.59 DOCTEST_CHECK_MESSAGE	246
12.64.1.60 DOCTEST_CHECK_NE	246
12.64.1.61 DOCTEST_CHECK_NOTHROW	246
12.64.1.62 DOCTEST_CHECK_NOTHROW_MESSAGE	246
12.64.1.63 DOCTEST_CHECK_THROWS	247
12.64.1.64 DOCTEST_CHECK_THROWS_AS	247
12.64.1.65 DOCTEST_CHECK_THROWS_AS_MESSAGE	247
12.64.1.66 DOCTEST_CHECK_THROWS_MESSAGE	247
12.64.1.67 DOCTEST_CHECK_THROWS_WITH	247
12.64.1.68 DOCTEST_CHECK_THROWS_WITH_AS	248
12.64.1.69 DOCTEST_CHECK_THROWS_WITH_AS_MESSAGE	248
12.64.1.70 DOCTEST_CHECK_THROWS_WITH_MESSAGE	248
12.64.1.71 DOCTEST_CHECK_UNARY	248
12.64.1.72 DOCTEST_CHECK_UNARY_FALSE	249
12.64.1.73 DOCTEST_CLANG	249
12.64.1.74 DOCTEST_CLANG_SUPPRESS_WARNING	249
12.64.1.75 DOCTEST_CLANG_SUPPRESS_WARNING_POP	249
12.64.1.76 DOCTEST_CLANG_SUPPRESS_WARNING_PUSH	249
12.64.1.77 DOCTEST_CLANG_SUPPRESS_WARNING_WITH_PUSH	249
12.64.1.78 DOCTEST_CMP_EQ	249
12.64.1.79 DOCTEST_CMP_GE	250
12.64.1.80 DOCTEST_CMP_GT	250
12.64.1.81 DOCTEST_CMP_LE	250
12.64.1.82 DOCTEST_CMP_LT	250
12.64.1.83 DOCTEST_CMP_NE	250
12.64.1.84 DOCTEST_COMPARISON_RETURN_TYPE	251
12.64.1.85 DOCTEST_COMPILER	251
12.64.1.86 DOCTEST_CONFIG_NO_EXCEPTIONS	251
12.64.1.87 DOCTEST_CONFIG_NO_TRY_CATCH_IN_ASSERTS	251
12.64.1.88 DOCTEST_CONFIG_POSIX_SIGNALS	251
12.64.1.89 DOCTEST_CONFIG_STRING_SIZE_TYPE	251
12.64.1.90 DOCTEST_CONSTEXPR	251
12.64.1.91 DOCTEST_CONSTEXPR_FUNC	252
12.64.1.92 DOCTEST_COUNTER	252
12.64.1.93 DOCTEST_CPLUSPLUS	252
12.64.1.94 DOCTEST_CREATE_AND_REGISTER_FUNCTION	252

12.64.1.95 DOCTEST_CREATE_AND_REGISTER_FUNCTION_IN_CLASS	252
12.64.1.96 DOCTEST_DECLARE_INTERFACE	253
12.64.1.97 DOCTEST_DEFINE_DECORATOR	253
12.64.1.98 DOCTEST_DEFINE_INTERFACE	253
12.64.1.99 DOCTEST_DO_BINARY_EXPRESSION_COMPARISON	254
12.64.1.100 DOCTEST_EMPTY	254
12.64.1.101 DOCTEST_EXCEPTION_EMPTY_FUNC	254
12.64.1.102 DOCTEST_FAIL	255
12.64.1.103 DOCTEST_FAIL_CHECK	255
12.64.1.104 DOCTEST_FAST_CHECK_EQ	255
12.64.1.105 DOCTEST_FAST_CHECK_GE	255
12.64.1.106 DOCTEST_FAST_CHECK_GT	255
12.64.1.107 DOCTEST_FAST_CHECK_LE	255
12.64.1.108 DOCTEST_FAST_CHECK_LT	255
12.64.1.109 DOCTEST_FAST_CHECK_NE	256
12.64.1.110 DOCTEST_FAST_CHECK_UNARY	256
12.64.1.111 DOCTEST_FAST_CHECK_UNARY_FALSE	256
12.64.1.112 DOCTEST_FAST_REQUIRE_EQ	256
12.64.1.113 DOCTEST_FAST_REQUIRE_GE	256
12.64.1.114 DOCTEST_FAST_REQUIRE_GT	256
12.64.1.115 DOCTEST_FAST_REQUIRE_LE	256
12.64.1.116 DOCTEST_FAST_REQUIRE_LT	256
12.64.1.117 DOCTEST_FAST_REQUIRE_NE	257
12.64.1.118 DOCTEST_FAST_REQUIRE_UNARY	257
12.64.1.119 DOCTEST_FAST_REQUIRE_UNARY_FALSE	257
12.64.1.120 DOCTEST_FAST_WARN_EQ	257
12.64.1.121 DOCTEST_FAST_WARN_GE	257
12.64.1.122 DOCTEST_FAST_WARN_GT	257
12.64.1.123 DOCTEST_FAST_WARN_LE	257
12.64.1.124 DOCTEST_FAST_WARN_LT	257
12.64.1.125 DOCTEST_FAST_WARN_NE	258
12.64.1.126 DOCTEST_FAST_WARN_UNARY	258
12.64.1.127 DOCTEST_FAST_WARN_UNARY_FALSE	258
12.64.1.128 DOCTEST_FORBIT_EXPRESSION	258
12.64.1.129 DOCTEST_FUNC_EMPTY	258
12.64.1.130 DOCTEST_FUNC_SCOPE_BEGIN	258
12.64.1.131 DOCTEST_FUNC_SCOPE_END	259
12.64.1.132 DOCTEST_FUNC_SCOPE_RET	259
12.64.1.133 DOCTEST_GCC	259
12.64.1.134 DOCTEST_GCC_SUPPRESS_WARNING	259
12.64.1.135 DOCTEST_GCC_SUPPRESS_WARNING_POP	259
12.64.1.136 DOCTEST_GCC_SUPPRESS_WARNING_PUSH	259

12.64.1.137 DOCTEST_GCC_SUPPRESS_WARNING_WITH_PUSH	259
12.64.1.138 DOCTEST_GENERATE	260
12.64.1.139 DOCTEST_GIVEN	260
12.64.1.140 DOCTEST_GLOBAL_NO_WARNINGS	260
12.64.1.141 DOCTEST_HAS_BUILTIN	260
12.64.1.142 DOCTEST_ICC	260
12.64.1.143 DOCTEST_IMPLEMENT_FIXTURE	261
12.64.1.144 DOCTEST_INFO	261
12.64.1.145 DOCTEST_INFO_IMPL	261
12.64.1.146 DOCTEST_INLINE_NOINLINE	262
12.64.1.147 DOCTEST_INTERFACE	262
12.64.1.148 DOCTEST_INTERFACE_DECL	262
12.64.1.149 DOCTEST_INTERFACE_DEF	262
12.64.1.150 DOCTEST_MAKE_STD_HEADERS_CLEAN_FROM_WARNINGS_ON_WALL_BEGIN	262
12.64.1.151 DOCTEST_MAKE_STD_HEADERS_CLEAN_FROM_WARNINGS_ON_WALL_END	263
12.64.1.152 DOCTEST_MESSAGE	263
12.64.1.153 DOCTEST_MSVC	263
12.64.1.154 DOCTEST_MSVC_SUPPRESS_WARNING	263
12.64.1.155 DOCTEST_MSVC_SUPPRESS_WARNING_POP	263
12.64.1.156 DOCTEST_MSVC_SUPPRESS_WARNING_PUSH	263
12.64.1.157 DOCTEST_MSVC_SUPPRESS_WARNING_WITH_PUSH	263
12.64.1.158 DOCTEST_NO_SANITIZE_INTEGER	264
12.64.1.159 DOCTEST_NOEXCEPT	264
12.64.1.160 DOCTEST_NOINLINE	264
12.64.1.161 DOCTEST_NORETURN	264
12.64.1.162 DOCTEST_PARTS_PUBLIC_ASSERT_COMPARATOR	264
12.64.1.163 DOCTEST_PARTS_PUBLIC_ASSERT_DATA	264
12.64.1.164 DOCTEST_PARTS_PUBLIC_ASSERT_EXPRESSION	264
12.64.1.165 DOCTEST_PARTS_PUBLIC_ASSERT_HANDLER	264
12.64.1.166 DOCTEST_PARTS_PUBLIC_ASSERT_MESSAGE	265
12.64.1.167 DOCTEST_PARTS_PUBLIC_ASSERT_RESULT	265
12.64.1.168 DOCTEST_PARTS_PUBLIC_ASSERT_TYPE	265
12.64.1.169 DOCTEST_PARTS_PUBLIC_COLOR	265
12.64.1.170 DOCTEST_PARTS_PUBLIC_COMPILER	265
12.64.1.171 DOCTEST_PARTS_PUBLIC_CONFIG	265
12.64.1.172 DOCTEST_PARTS_PUBLIC_CONTEXT	265
12.64.1.173 DOCTEST_PARTS_PUBLIC_CONTEXT_OPTIONS	265
12.64.1.174 DOCTEST_PARTS_PUBLIC_CONTEXT_SCOPE	266
12.64.1.175 DOCTEST_PARTS_PUBLIC_DEBUGGER	266
12.64.1.176 DOCTEST_PARTS_PUBLIC_DECORATORS	266
12.64.1.177 DOCTEST_PARTS_PUBLIC_EXCEPTION_TRANSLATOR	266
12.64.1.178 DOCTEST_PARTS_PUBLIC_EXCEPTIONS	266

12.64.1.179 DOCTEST_PARTS_PUBLIC_GENERATOR	266
12.64.1.180 DOCTEST_PARTS_PUBLIC_MACROS	266
12.64.1.181 DOCTEST_PARTS_PUBLIC_MATCHERS_APPROX	266
12.64.1.182 DOCTEST_PARTS_PUBLIC_MATCHERS_CONTAINS	267
12.64.1.183 DOCTEST_PARTS_PUBLIC_MATCHERS_IS_NAN	267
12.64.1.184 DOCTEST_PARTS_PUBLIC_PATH	267
12.64.1.185 DOCTEST_PARTS_PUBLIC_PLATFORM	267
12.64.1.186 DOCTEST_PARTS_PUBLIC_REPORTER	267
12.64.1.187 DOCTEST_PARTS_PUBLIC_STD_FWD	267
12.64.1.188 DOCTEST_PARTS_PUBLIC_STD_TYPE_TRAITS	267
12.64.1.189 DOCTEST_PARTS_PUBLIC_STD_UTILITY	267
12.64.1.190 DOCTEST_PARTS_PUBLIC_STRING	268
12.64.1.191 DOCTEST_PARTS_PUBLIC_SUBCASE	268
12.64.1.192 DOCTEST_PARTS_PUBLIC_TEST_CASE	268
12.64.1.193 DOCTEST_PARTS_PUBLIC_TEST_SUITE	268
12.64.1.194 DOCTEST_PARTS_PUBLIC_UTILITY	268
12.64.1.195 DOCTEST_PARTS_PUBLIC_VERSION	268
12.64.1.196 DOCTEST_PARTS_PUBLIC_WARNINGS	268
12.64.1.197 DOCTEST_PLATFORM_LINUX	268
12.64.1.198 DOCTEST_REF_WRAP	269
12.64.1.199 DOCTEST_REGISTER_EXCEPTION_TRANSLATOR	269
12.64.1.200 DOCTEST_REGISTER_EXCEPTION_TRANSLATOR_IMPL	269
12.64.1.201 DOCTEST_REGISTER_FUNCTION	269
12.64.1.202 DOCTEST_REGISTER_LISTENER	270
12.64.1.203 DOCTEST_REGISTER_REPORTER	270
12.64.1.204 DOCTEST_RELATIONAL_OP	270
12.64.1.205 DOCTEST_REQUIRE [1/2]	270
12.64.1.206 DOCTEST_REQUIRE [2/2]	271
12.64.1.207 DOCTEST_REQUIRE_EQ [1/2]	271
12.64.1.208 DOCTEST_REQUIRE_EQ [2/2]	271
12.64.1.209 DOCTEST_REQUIRE_FALSE [1/2]	271
12.64.1.210 DOCTEST_REQUIRE_FALSE [2/2]	271
12.64.1.211 DOCTEST_REQUIRE_FALSE_MESSAGE [1/2]	271
12.64.1.212 DOCTEST_REQUIRE_FALSE_MESSAGE [2/2]	272
12.64.1.213 DOCTEST_REQUIRE_GE [1/2]	272
12.64.1.214 DOCTEST_REQUIRE_GE [2/2]	272
12.64.1.215 DOCTEST_REQUIRE_GT [1/2]	272
12.64.1.216 DOCTEST_REQUIRE_GT [2/2]	272
12.64.1.217 DOCTEST_REQUIRE_LE [1/2]	272
12.64.1.218 DOCTEST_REQUIRE_LE [2/2]	273
12.64.1.219 DOCTEST_REQUIRE_LT [1/2]	273
12.64.1.220 DOCTEST_REQUIRE_LT [2/2]	273

12.64.1.221 DOCTEST_REQUIRE_MESSAGE [1/2]	273
12.64.1.222 DOCTEST_REQUIRE_MESSAGE [2/2]	273
12.64.1.223 DOCTEST_REQUIRE_NE [1/2]	273
12.64.1.224 DOCTEST_REQUIRE_NE [2/2]	274
12.64.1.225 DOCTEST_REQUIRE_NOTHROW	274
12.64.1.226 DOCTEST_REQUIRE_NOTHROW_MESSAGE	274
12.64.1.227 DOCTEST_REQUIRE_THROWS	274
12.64.1.228 DOCTEST_REQUIRE_THROWS_AS	274
12.64.1.229 DOCTEST_REQUIRE_THROWS_AS_MESSAGE	275
12.64.1.230 DOCTEST_REQUIRE_THROWS_MESSAGE	275
12.64.1.231 DOCTEST_REQUIRE_THROWS_WITH	275
12.64.1.232 DOCTEST_REQUIRE_THROWS_WITH_AS	275
12.64.1.233 DOCTEST_REQUIRE_THROWS_WITH_AS_MESSAGE	276
12.64.1.234 DOCTEST_REQUIRE_THROWS_WITH_MESSAGE	276
12.64.1.235 DOCTEST_REQUIRE_UNARY [1/2]	276
12.64.1.236 DOCTEST_REQUIRE_UNARY [2/2]	276
12.64.1.237 DOCTEST_REQUIRE_UNARY_FALSE [1/2]	276
12.64.1.238 DOCTEST_REQUIRE_UNARY_FALSE [2/2]	277
12.64.1.239 DOCTEST_SCENARIO	277
12.64.1.240 DOCTEST_SCENARIO_CLASS	277
12.64.1.241 DOCTEST_SCENARIO_METHOD	277
12.64.1.242 DOCTEST_SCENARIO_TEMPLATE	277
12.64.1.243 DOCTEST_SCENARIO_TEMPLATE_DEFINE	278
12.64.1.244 DOCTEST_STRINGIFY	278
12.64.1.245 DOCTEST_SUBCASE	278
12.64.1.246 DOCTEST_SUPPRESS_COMMON_WARNINGS_POP	278
12.64.1.247 DOCTEST_SUPPRESS_COMMON_WARNINGS_PUSH	278
12.64.1.248 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP	279
12.64.1.249 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH	279
12.64.1.250 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_POP	279
12.64.1.251 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_PUSH	279
12.64.1.252 DOCTEST_SYMBOL_EXPORT	279
12.64.1.253 DOCTEST_SYMBOL_IMPORT	279
12.64.1.254 DOCTEST_TEST_CASE	280
12.64.1.255 DOCTEST_TEST_CASE_CLASS	280
12.64.1.256 DOCTEST_TEST_CASE_FIXTURE	280
12.64.1.257 DOCTEST_TEST_CASE_TEMPLATE	280
12.64.1.258 DOCTEST_TEST_CASE_TEMPLATE_APPLY	280
12.64.1.259 DOCTEST_TEST_CASE_TEMPLATE_DEFINE	281
12.64.1.260 DOCTEST_TEST_CASE_TEMPLATE_DEFINE_IMPL	281
12.64.1.261 DOCTEST_TEST_CASE_TEMPLATE_IMPL	282
12.64.1.262 DOCTEST_TEST_CASE_TEMPLATE_INSTANTIATE	282

12.64.1.263 DOCTEST_TEST_CASE_TEMPLATE_INSTANTIATE_IMPL	282
12.64.1.264 DOCTEST_TEST_CASE_TEMPLATE_INVOKE	283
12.64.1.265 DOCTEST_TEST_SUITE	283
12.64.1.266 DOCTEST_TEST_SUITE_BEGIN	283
12.64.1.267 DOCTEST_TEST_SUITE_END	283
12.64.1.268 DOCTEST_TEST_SUITE_IMPL	284
12.64.1.269 DOCTEST_THEN	284
12.64.1.270 DOCTEST_TO_LVALUE	284
12.64.1.271 DOCTEST_TOSTR	285
12.64.1.272 DOCTEST_TOSTR_IMPL	285
12.64.1.273 DOCTEST_TYPE_TO_STRING	285
12.64.1.274 DOCTEST_TYPE_TO_STRING_AS	285
12.64.1.275 DOCTEST_UNARY_ASSERT	286
12.64.1.276 DOCTEST_UNUSED	286
12.64.1.277 DOCTEST_VERSION	286
12.64.1.278 DOCTEST_VERSION_MAJOR	286
12.64.1.279 DOCTEST_VERSION_MINOR	286
12.64.1.280 DOCTEST_VERSION_PATCH	286
12.64.1.281 DOCTEST_VERSION_STR	287
12.64.1.282 DOCTEST_WARN	287
12.64.1.283 DOCTEST_WARN_EQ	287
12.64.1.284 DOCTEST_WARN_FALSE	287
12.64.1.285 DOCTEST_WARN_FALSE_MESSAGE	287
12.64.1.286 DOCTEST_WARN_GE	288
12.64.1.287 DOCTEST_WARN_GT	288
12.64.1.288 DOCTEST_WARN_LE	288
12.64.1.289 DOCTEST_WARN_LT	288
12.64.1.290 DOCTEST_WARN_MESSAGE	288
12.64.1.291 DOCTEST_WARN_NE	289
12.64.1.292 DOCTEST_WARN_NOTHROW	289
12.64.1.293 DOCTEST_WARN_NOTHROW_MESSAGE	289
12.64.1.294 DOCTEST_WARN_THROWS	289
12.64.1.295 DOCTEST_WARN_THROWS_AS	289
12.64.1.296 DOCTEST_WARN_THROWS_AS_MESSAGE	290
12.64.1.297 DOCTEST_WARN_THROWS_MESSAGE	290
12.64.1.298 DOCTEST_WARN_THROWS_WITH	290
12.64.1.299 DOCTEST_WARN_THROWS_WITH_AS	290
12.64.1.300 DOCTEST_WARN_THROWS_WITH_AS_MESSAGE	291
12.64.1.301 DOCTEST_WARN_THROWS_WITH_MESSAGE	291
12.64.1.302 DOCTEST_WARN_UNARY	291
12.64.1.303 DOCTEST_WARN_UNARY_FALSE	291
12.64.1.304 DOCTEST_WHEN	291

12.64.1.305 DOCTEST_WRAP_IN_TRY	292
12.64.1.306 FAIL	292
12.64.1.307 FAIL_CHECK	292
12.64.1.308 FAST_CHECK_EQ	292
12.64.1.309 FAST_CHECK_GE	292
12.64.1.310 FAST_CHECK_GT	293
12.64.1.311 FAST_CHECK_LE	293
12.64.1.312 FAST_CHECK_LT	293
12.64.1.313 FAST_CHECK_NE	293
12.64.1.314 FAST_CHECK_UNARY	293
12.64.1.315 FAST_CHECK_UNARY_FALSE	294
12.64.1.316 FAST_REQUIRE_EQ	294
12.64.1.317 FAST_REQUIRE_GE	294
12.64.1.318 FAST_REQUIRE_GT	294
12.64.1.319 FAST_REQUIRE_LE	294
12.64.1.320 FAST_REQUIRE_LT	295
12.64.1.321 FAST_REQUIRE_NE	295
12.64.1.322 FAST_REQUIRE_UNARY	295
12.64.1.323 FAST_REQUIRE_UNARY_FALSE	295
12.64.1.324 FAST_WARN_EQ	295
12.64.1.325 FAST_WARN_GE	296
12.64.1.326 FAST_WARN_GT	296
12.64.1.327 FAST_WARN_LE	296
12.64.1.328 FAST_WARN_LT	296
12.64.1.329 FAST_WARN_NE	296
12.64.1.330 FAST_WARN_UNARY	297
12.64.1.331 FAST_WARN_UNARY_FALSE	297
12.64.1.332 GENERATE	297
12.64.1.333 GIVEN	297
12.64.1.334 INFO	297
12.64.1.335 MESSAGE	298
12.64.1.336 REGISTER_EXCEPTION_TRANSLATOR	298
12.64.1.337 REGISTER_LISTENER	298
12.64.1.338 REGISTER_REPORTER	298
12.64.1.339 REQUIRE	298
12.64.1.340 REQUIRE_EQ	299
12.64.1.341 REQUIRE_FALSE	299
12.64.1.342 REQUIRE_FALSE_MESSAGE	299
12.64.1.343 REQUIRE_GE	299
12.64.1.344 REQUIRE_GT	299
12.64.1.345 REQUIRE_LE	300
12.64.1.346 REQUIRE_LT	300

12.64.1.347 REQUIRE_MESSAGE	300
12.64.1.348 REQUIRE_NE	300
12.64.1.349 REQUIRE_NOTHROW	300
12.64.1.350 REQUIRE_NOTHROW_MESSAGE	301
12.64.1.351 REQUIRE_THROWS	301
12.64.1.352 REQUIRE_THROWS_AS	301
12.64.1.353 REQUIRE_THROWS_AS_MESSAGE	301
12.64.1.354 REQUIRE_THROWS_MESSAGE	301
12.64.1.355 REQUIRE_THROWS_WITH	302
12.64.1.356 REQUIRE_THROWS_WITH_AS	302
12.64.1.357 REQUIRE_THROWS_WITH_AS_MESSAGE	302
12.64.1.358 REQUIRE_THROWS_WITH_MESSAGE	302
12.64.1.359 REQUIRE_UNARY	303
12.64.1.360 REQUIRE_UNARY_FALSE	303
12.64.1.361 SCENARIO	303
12.64.1.362 SCENARIO_CLASS	303
12.64.1.363 SCENARIO_METHOD	303
12.64.1.364 SCENARIO_TEMPLATE	304
12.64.1.365 SCENARIO_TEMPLATE_DEFINE	304
12.64.1.366 SFINAE_OP	304
12.64.1.367 SUBCASE	304
12.64.1.368 TEST_CASE	304
12.64.1.369 TEST_CASE_CLASS	305
12.64.1.370 TEST_CASE_FIXTURE	305
12.64.1.371 TEST_CASE_TEMPLATE	305
12.64.1.372 TEST_CASE_TEMPLATE_APPLY	305
12.64.1.373 TEST_CASE_TEMPLATE_DEFINE	305
12.64.1.374 TEST_CASE_TEMPLATE_INSTANTIATE	306
12.64.1.375 TEST_CASE_TEMPLATE_INVOKE	306
12.64.1.376 TEST_SUITE	306
12.64.1.377 TEST_SUITE_BEGIN	306
12.64.1.378 TEST_SUITE_END	306
12.64.1.379 THEN	307
12.64.1.380 TO_LVALUE	307
12.64.1.381 TYPE_TO_STRING	307
12.64.1.382 TYPE_TO_STRING_AS	307
12.64.1.383 WARN	307
12.64.1.384 WARN_EQ	308
12.64.1.385 WARN_FALSE	308
12.64.1.386 WARN_FALSE_MESSAGE	308
12.64.1.387 WARN_GE	308
12.64.1.388 WARN_GT	308

12.64.1.389 WARN_LE	309
12.64.1.390 WARN_LT	309
12.64.1.391 WARN_MESSAGE	309
12.64.1.392 WARN_NE	309
12.64.1.393 WARN_NOTHROW	309
12.64.1.394 WARN_NOTHROW_MESSAGE	310
12.64.1.395 WARN_THROWS	310
12.64.1.396 WARN_THROWS_AS	310
12.64.1.397 WARN_THROWS_AS_MESSAGE	310
12.64.1.398 WARN_THROWS_MESSAGE	310
12.64.1.399 WARN_THROWS_WITH	311
12.64.1.400 WARN_THROWS_WITH_AS	311
12.64.1.401 WARN_THROWS_WITH_AS_MESSAGE	311
12.64.1.402 WARN_THROWS_WITH_MESSAGE	311
12.64.1.403 WARN_UNARY	312
12.64.1.404 WARN_UNARY_FALSE	312
12.64.1.405 WHEN	312
12.65 doctest.h	312
12.66 tests/test_Alarme.cpp File Reference	426
12.66.1 Function Documentation	426
12.66.1.1 TEST_CASE() [1/7]	426
12.66.1.2 TEST_CASE() [2/7]	426
12.66.1.3 TEST_CASE() [3/7]	426
12.66.1.4 TEST_CASE() [4/7]	427
12.66.1.5 TEST_CASE() [5/7]	427
12.66.1.6 TEST_CASE() [6/7]	427
12.66.1.7 TEST_CASE() [7/7]	427
12.67 test_Alarme.cpp	427
12.68 tests/test_Conjunto.cpp File Reference	428
12.68.1 Function Documentation	429
12.68.1.1 TEST_CASE() [1/25]	429
12.68.1.2 TEST_CASE() [2/25]	429
12.68.1.3 TEST_CASE() [3/25]	429
12.68.1.4 TEST_CASE() [4/25]	429
12.68.1.5 TEST_CASE() [5/25]	429
12.68.1.6 TEST_CASE() [6/25]	429
12.68.1.7 TEST_CASE() [7/25]	430
12.68.1.8 TEST_CASE() [8/25]	430
12.68.1.9 TEST_CASE() [9/25]	430
12.68.1.10 TEST_CASE() [10/25]	430
12.68.1.11 TEST_CASE() [11/25]	430
12.68.1.12 TEST_CASE() [12/25]	430

12.68.1.13 TEST_CASE() [13/25]	430
12.68.1.14 TEST_CASE() [14/25]	431
12.68.1.15 TEST_CASE() [15/25]	431
12.68.1.16 TEST_CASE() [16/25]	431
12.68.1.17 TEST_CASE() [17/25]	431
12.68.1.18 TEST_CASE() [18/25]	431
12.68.1.19 TEST_CASE() [19/25]	431
12.68.1.20 TEST_CASE() [20/25]	431
12.68.1.21 TEST_CASE() [21/25]	432
12.68.1.22 TEST_CASE() [22/25]	432
12.68.1.23 TEST_CASE() [23/25]	432
12.68.1.24 TEST_CASE() [24/25]	432
12.68.1.25 TEST_CASE() [25/25]	432
12.69 test_Conjunto.cpp	433
12.70 tests/test_Equipamento.cpp File Reference	435
12.70.1 Function Documentation	435
12.70.1.1 TEST_CASE() [1/20]	435
12.70.1.2 TEST_CASE() [2/20]	436
12.70.1.3 TEST_CASE() [3/20]	436
12.70.1.4 TEST_CASE() [4/20]	436
12.70.1.5 TEST_CASE() [5/20]	436
12.70.1.6 TEST_CASE() [6/20]	436
12.70.1.7 TEST_CASE() [7/20]	436
12.70.1.8 TEST_CASE() [8/20]	436
12.70.1.9 TEST_CASE() [9/20]	437
12.70.1.10 TEST_CASE() [10/20]	437
12.70.1.11 TEST_CASE() [11/20]	437
12.70.1.12 TEST_CASE() [12/20]	437
12.70.1.13 TEST_CASE() [13/20]	437
12.70.1.14 TEST_CASE() [14/20]	437
12.70.1.15 TEST_CASE() [15/20]	437
12.70.1.16 TEST_CASE() [16/20]	438
12.70.1.17 TEST_CASE() [17/20]	438
12.70.1.18 TEST_CASE() [18/20]	438
12.70.1.19 TEST_CASE() [19/20]	438
12.70.1.20 TEST_CASE() [20/20]	438
12.71 test_Equipamento.cpp	439
12.72 tests/test_Excecoes.cpp File Reference	440
12.72.1 Function Documentation	441
12.72.1.1 TEST_CASE() [1/20]	441
12.72.1.2 TEST_CASE() [2/20]	441
12.72.1.3 TEST_CASE() [3/20]	441

12.72.1.4 TEST_CASE() [4/20]	442
12.72.1.5 TEST_CASE() [5/20]	442
12.72.1.6 TEST_CASE() [6/20]	442
12.72.1.7 TEST_CASE() [7/20]	442
12.72.1.8 TEST_CASE() [8/20]	442
12.72.1.9 TEST_CASE() [9/20]	442
12.72.1.10 TEST_CASE() [10/20]	442
12.72.1.11 TEST_CASE() [11/20]	443
12.72.1.12 TEST_CASE() [12/20]	443
12.72.1.13 TEST_CASE() [13/20]	443
12.72.1.14 TEST_CASE() [14/20]	443
12.72.1.15 TEST_CASE() [15/20]	443
12.72.1.16 TEST_CASE() [16/20]	443
12.72.1.17 TEST_CASE() [17/20]	443
12.72.1.18 TEST_CASE() [18/20]	444
12.72.1.19 TEST_CASE() [19/20]	444
12.72.1.20 TEST_CASE() [20/20]	444
12.73 test_Excecoes.cpp	444
12.74 tests/test_LinhaDeProducao.cpp File Reference	446
12.74.1 Function Documentation	446
12.74.1.1 TEST_CASE() [1/23]	446
12.74.1.2 TEST_CASE() [2/23]	447
12.74.1.3 TEST_CASE() [3/23]	447
12.74.1.4 TEST_CASE() [4/23]	447
12.74.1.5 TEST_CASE() [5/23]	447
12.74.1.6 TEST_CASE() [6/23]	447
12.74.1.7 TEST_CASE() [7/23]	447
12.74.1.8 TEST_CASE() [8/23]	447
12.74.1.9 TEST_CASE() [9/23]	448
12.74.1.10 TEST_CASE() [10/23]	448
12.74.1.11 TEST_CASE() [11/23]	448
12.74.1.12 TEST_CASE() [12/23]	448
12.74.1.13 TEST_CASE() [13/23]	448
12.74.1.14 TEST_CASE() [14/23]	448
12.74.1.15 TEST_CASE() [15/23]	448
12.74.1.16 TEST_CASE() [16/23]	449
12.74.1.17 TEST_CASE() [17/23]	449
12.74.1.18 TEST_CASE() [18/23]	449
12.74.1.19 TEST_CASE() [19/23]	449
12.74.1.20 TEST_CASE() [20/23]	449
12.74.1.21 TEST_CASE() [21/23]	449
12.74.1.22 TEST_CASE() [22/23]	449

12.74.1.23 TEST_CASE() [23/23]	450
12.75 test_LinhaDeProducao.cpp	450
12.76 tests/test_Log.cpp File Reference	452
12.76.1 Function Documentation	452
12.76.1.1 TEST_CASE() [1/6]	452
12.76.1.2 TEST_CASE() [2/6]	452
12.76.1.3 TEST_CASE() [3/6]	452
12.76.1.4 TEST_CASE() [4/6]	453
12.76.1.5 TEST_CASE() [5/6]	453
12.76.1.6 TEST_CASE() [6/6]	453
12.77 test_Log.cpp	453
12.78 tests/test_main.cpp File Reference	454
12.78.1 Macro Definition Documentation	454
12.78.1.1 DOCTEST_CONFIG_IMPLEMENT_WITH_MAIN	454
12.79 test_main.cpp	454
12.80 tests/test_Parametro.cpp File Reference	454
12.80.1 Function Documentation	455
12.80.1.1 TEST_CASE() [1/14]	455
12.80.1.2 TEST_CASE() [2/14]	455
12.80.1.3 TEST_CASE() [3/14]	455
12.80.1.4 TEST_CASE() [4/14]	455
12.80.1.5 TEST_CASE() [5/14]	455
12.80.1.6 TEST_CASE() [6/14]	455
12.80.1.7 TEST_CASE() [7/14]	456
12.80.1.8 TEST_CASE() [8/14]	456
12.80.1.9 TEST_CASE() [9/14]	456
12.80.1.10 TEST_CASE() [10/14]	456
12.80.1.11 TEST_CASE() [11/14]	456
12.80.1.12 TEST_CASE() [12/14]	456
12.80.1.13 TEST_CASE() [13/14]	456
12.80.1.14 TEST_CASE() [14/14]	457
12.81 test_Parametro.cpp	457
12.82 tests/test_Sistema.cpp File Reference	458
12.82.1 Function Documentation	459
12.82.1.1 TEST_CASE() [1/24]	459
12.82.1.2 TEST_CASE() [2/24]	459
12.82.1.3 TEST_CASE() [3/24]	459
12.82.1.4 TEST_CASE() [4/24]	459
12.82.1.5 TEST_CASE() [5/24]	459
12.82.1.6 TEST_CASE() [6/24]	459
12.82.1.7 TEST_CASE() [7/24]	460
12.82.1.8 TEST_CASE() [8/24]	460

12.82.1.9 TEST_CASE() [9/24]	460
12.82.1.10 TEST_CASE() [10/24]	460
12.82.1.11 TEST_CASE() [11/24]	460
12.82.1.12 TEST_CASE() [12/24]	460
12.82.1.13 TEST_CASE() [13/24]	460
12.82.1.14 TEST_CASE() [14/24]	461
12.82.1.15 TEST_CASE() [15/24]	461
12.82.1.16 TEST_CASE() [16/24]	461
12.82.1.17 TEST_CASE() [17/24]	461
12.82.1.18 TEST_CASE() [18/24]	461
12.82.1.19 TEST_CASE() [19/24]	461
12.82.1.20 TEST_CASE() [20/24]	461
12.82.1.21 TEST_CASE() [21/24]	462
12.82.1.22 TEST_CASE() [22/24]	462
12.82.1.23 TEST_CASE() [23/24]	462
12.82.1.24 TEST_CASE() [24/24]	462
12.83 test_Sistema.cpp	462
12.84 tests/test_Usuario.cpp File Reference	464
12.84.1 Function Documentation	465
12.84.1.1 TEST_CASE() [1/7]	465
12.84.1.2 TEST_CASE() [2/7]	465
12.84.1.3 TEST_CASE() [3/7]	465
12.84.1.4 TEST_CASE() [4/7]	465
12.84.1.5 TEST_CASE() [5/7]	465
12.84.1.6 TEST_CASE() [6/7]	465
12.84.1.7 TEST_CASE() [7/7]	466
12.85 test_Usuario.cpp	466
Index	467

Chapter 1

Sistema de Tolerância de Falha em Fábricas

Sistema de monitoramento industrial orientado a objetos desenvolvido em C++. Permite o acompanhamento em tempo real de parâmetros de equipamentos organizados em linhas de produção, detectando automaticamente condições de falha e disparando alarmes.

1.1 Motivação

A indústria depende de equipamentos que operam continuamente sob limites bem definidos de temperatura, pressão, vibração e outros parâmetros. Uma falha não detectada em um único equipamento pode se propagar pela linha de produção e gerar paradas custosas ou riscos de segurança. Escolhemos este tema porque ele exercita de forma natural os pilares da disciplina: uma **hierarquia clara de composição** (**Sistema** → Linha → **Conjunto** → **Equipamento** → Parâmetro), **herança** entre tipos de equipamento (**Motor**, **Sensor**, Válvula, **Atuador**), **encapsulamento** dos limites e do histórico de cada parâmetro, e **propagação de diagnóstico** por toda a estrutura. Além de ser um domínio rico em regras de negócio, é um problema concreto e fácil de visualizar, o que facilita a modelagem e a escrita de testes significativos.

1.2 Funcionalidades

- Gerenciamento hierárquico de **Linhas de Produção** → **Conjuntos** → **Equipamentos** → **Parâmetros**
- Monitoramento de parâmetros com limites configuráveis (Min/Max)
- Disparo automático de **alarmes** quando parâmetros violam seus limites
- **Diagnóstico** propagado por toda a hierarquia, identificando os pontos de falha
- Controle de acesso por usuários com três níveis de cargo
- Persistência de dados com save/load de estado
- Registro de eventos via **Log** em arquivo

1.3 Hierarquia do Sistema

Composição (relação "tem-um"):

```
Sistema
  \-- LinhaDeProducao
      \-- Conjunto
          \-- Equipamento
              +-- Parametro
              \-- Alarme
```

Herança (relação "é-um") — especializações de [Equipamento](#):

```
Equipamento
  +-- Motor      (velocidade, torque)
  +-- Sensor     (sinal)
  +-- Valvula    (vazao)
  \-- Atuador    (retorno)
```

1.4 Cargos de Usuário

Cargo	Permissões
ADMIN	Gerencia usuários e toda a estrutura do sistema
ENGENHEIRO	Configura linhas, conjuntos e equipamentos
TECNICO	Visualiza dados e alarmes

1.5 Estrutura de Diretórios

```
.
+-- include/      # Contratos das classes (.hpp), incl. Excecoes.hpp
+-- src/          # Implementacoes (.cpp) e main.cpp (interface de terminal)
+-- design/       # User Stories, Cartoes CRC e documentacao gerada
+-- tests/        # Testes automatizados (doctest)
+-- dados/        # Arquivos de inicializacao (ex.: inicial.txt)
+-- build/        # Artefatos de compilacao
+-- Makefile      # Automacao de build e testes
+-- Doxyfile      # Configuracao do Doxygen
\-- README.md
```

1.6 Estado de Desenvolvimento

Comportamentos implementados e validados:

- **Contratos (.hpp) e documentação Doxygen** para todas as classes principais.
- **Implementação (.cpp)** completa das entidades e do [Sistema](#), incluindo diagnóstico propagado, alarmes e persistência (save/load) de estado.
- **Especializações** de equipamento ([Motor](#), [Sensor](#), [Válvula](#), [Atuador](#)) integradas via **polimorfismo** com `std::unique_ptr<Equipamento>`.
- **Programação defensiva**: validação de entradas com **exceções próprias** (hierarquia [ExcecaoSistema](#) em `include/Excecoes.hpp`).
- **RAII**: nenhuma gestão manual de memória (`new/delete`); alocação dinâmica apenas via smart pointers.
- **Interface de terminal** interativa (`src/main.cpp`) com login, controle de acesso por cargo e tratamento de exceções.
- **Testes de unidade** (doctest): uma suíte por classe principal + suíte de exceções/polimorfismo/persistência. Atualmente **98 testes** passando com **~74% de cobertura** (`gcovr`).

1.7 Programação Defensiva e Tratamento de Exceções

As entradas inválidas lançam exceções específicas do domínio, todas derivadas de `ExcecaoSistema` (logo, captráveis por um único `catch` (`const ExcecaoSistema&`)):

Exceção	Quando
<code>EntradaInvalida</code>	string vazia (descrição, login, senha)
<code>LimitesInvalidos</code>	<code>Max < Min</code> em um parâmetro
<code>IdInvalido</code>	identificador negativo
<code>IdDuplicado</code>	inserir um ID já existente
<code>IdInexistente</code>	acessar/remover um ID ausente
<code>AcessoNegado</code>	operação acima do cargo do usuário (interface)
<code>SessaoInvalida</code>	operação que exige sessão sem usuário logado (interface)

Observação: uma **leitura fora dos limites** de um parâmetro **não** é erro — é justamente a condição de falha que o sistema detecta, então é armazenada e sinalizada via diagnóstico/alarme.

1.8 Compilação e Testes

O `Makefile` automatiza a compilação e a verificação por testes de unidade (framework `doctest`):

```
make          # compila os módulos e gera build/test_runner
make test     # executa os testes e gera os relatórios de cobertura (gcovr)
make app      # compila a aplicação interativa em build/sistema
make run      # compila (se preciso) e executa a aplicação
make clean    # remove a pasta build/
```

- `make test` executa todos os casos de teste e, em seguida, gera a cobertura via `gcovr` (saída em texto e em `build/coverage.html`).
- O `make clean` é portátil: usa `rmdir /S /Q` no Windows e `rm -rf` em Linux/macOS (detecção automática de SO no `Makefile`).

1.8.1 Instalação do gcovr

O `gcovr` é uma ferramenta Python (wrapper do `gcov`) e precisa ser instalado à parte. Caso o `pip` não esteja disponível, primeiro habilite-o com:

```
python -m ensurepip --upgrade
```

Em seguida instale o `gcovr` via `pip`:

SO	Comando
Linux	<code>python3 -m pip install --user gcovr</code>
macOS	<code>python3 -m pip install --user gcovr</code>
Windows	<code>python -m pip install --user gcovr</code>

Alternativas pelo gerenciador de pacotes do sistema:

- **Fedora/RHEL:** `sudo dnf install gcovr`
- **Debian/Ubuntu:** `sudo apt install gcovr`
- **macOS (Homebrew):** `brew install gcovr`

Verifique a instalação com `python3 -m gcovr --version` (use `python` no Windows). O `Makefile` invoca `python3 -m gcovr` em Linux/macOS e `python -m gcovr` em Windows.

1.9 Execução da Aplicação

```
make run      # ou: make app && ./build/sistema
```

Ao iniciar, o sistema carrega o arquivo de inicialização `dados/inicial.txt` (fornecido no repositório), que cadastra três usuários de exemplo:

Login	Senha	Cargo
admin	admin123	ADMIN
eng	eng123	ENGENHEIRO
tec	tec123	TECNICO

Após o login, um menu por cargo permite criar/listar/remover linhas, conjuntos, equipamentos (Motor/Sensor/Válvula/Atuador) e parâmetros, executar diagnóstico, e salvar/carregar o estado (`save.txt`). Entradas inválidas são tratadas por exceções e exibidas sem encerrar o programa.

1.10 Documentação

A documentação das classes pode ser gerada com o Doxygen a partir da raiz do projeto:

```
doxygen Doxyfile
```

A saída HTML estará disponível em `docs/html/index.html`. Uma versão, atualiza, em PDF pode ser encontrada na raiz do projeto como `doxygen_doc.pdf`.

1.11 Equipe

Nº Matrícula	Nome
2024010002	Felipe Hildegardes Jorge
2022028400	Wennedes de Oliveira Nogueira Junior
2020057144	Gustavo Coelho Santos

Chapter 2

notas-C6

- Excelente uso das notas de use cases.
- Cuidado com os nomes de métodos/funções. Por convenção usamos capitalização apenas para nome de Tipos/Structs/Classes. Métodos/funções começam com letra minúscula.
- Não encontrei nenhuma hierarquia de classes. Seria bom ter ao menos uma hierarquia significativa.
- Documentação incompleta: cadê a classe [Cargo](#)?
- Cuidado com descrições óbvias. Em [Usuario::Usuario\(\)](#): "Constrói um usuário.", sim, a classe é Usuário e é isso o que construtores fazem. Uma descrição mais legal foi de [Sistema::GetUsuarioLocado\(\)](#): "Ponteiro para o usuário atualmente logado, ou nullptr se não houver sessão.". Aqui vocês mencionam que existe uma sessão de usuário que persiste em execução, e que ela pode não estar carregada. Apenas neste caso retorna nullptr.
- A parte de [Log](#) não ajuda muito. Basicamente ela recebe mensagens e escreve em um arquivo. Embora seja bom centralizar isso, [Log](#) parece meio anêmica. Será que não tem uma sobrecarga de [Log::Registrar](#) para cada classe que pode ser logada?
- No README: não foi justificada a escolha do tema. Por causa do uso de notas de use cases irei considerar a entrega, mas leiam com mais cuidado as descrições dos checkpoints. Esse requisito estava em negrito, inclusive.

Chapter 3

notas-C7

- projeto bem limpo e organizado. diretório build e .gitignore corretos.
- o makeclean só funciona para windows: não tem powershell para linux :)
- makefile bem escrito: só compila o que precisa, e apenas quando alteram arquivos
- interessante usar o python para rodar o gcovr no windows

Chapter 4

Directory Hierarchy

4.1 Directories

design	23
include	23
Alarme.hpp	189
Atuador.hpp	190
Cargo.hpp	190
Conjunto.hpp	191
Dependencia.hpp	192
Equipamento.hpp	193
EstadoOperacional.hpp	194
Excecoes.hpp	194
LinhaDeProducao.hpp	196
Log.hpp	197
Motor.hpp	197
Parametro.hpp	198
Sensor.hpp	199
Sistema.hpp	199
Usuario.hpp	200
Valvula.hpp	201
src	23
Alarme.cpp	202
Atuador.cpp	202
Conjunto.cpp	203
Dependencia.cpp	204
Equipamento.cpp	205
LinhaDeProducao.cpp	207
Log.cpp	208
main.cpp	209
Motor.cpp	213
Parametro.cpp	214
Sensor.cpp	215
Sistema.cpp	216
Usuario.cpp	219
Valvula.cpp	220
tests	24
doctest.h	220
test_Alarme.cpp	426
test_Conjunto.cpp	428

test_Equipamento.cpp	435
test_Excecoes.cpp	440
test_LinhaDeProducao.cpp	446
test_Log.cpp	452
test_main.cpp	454
test_Parametro.cpp	454
test_Sistema.cpp	458
test_Usuario.cpp	464

Chapter 5

Namespace Index

5.1 Namespace List

Here is a list of all namespaces with brief descriptions:

doctest	25
doctest::assertType	34
doctest::Color	37
doctest::detail	38
doctest::detail::binaryAssertComparison	44
doctest::detail::types	45
doctest::TestCaseFailureReason	46
doctest_detail_test_suite_ns	46
std	47

Chapter 6

Hierarchical Index

6.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

Alarme	49
doctest::Approx	52
doctest::AssertData	55
doctest::detail::ResultBuilder	148
std::basic_istream< charT, traits >	60
std::basic_ostream< charT, traits >	60
std::char_traits< charT >	61
Conjunto	61
doctest::Contains	67
doctest::Context	68
doctest::ContextOptions	71
doctest::CurrentTestCaseStats	81
Dependencia	83
doctest::detail::types::enable_if< COND, T >	84
doctest::detail::types::enable_if< true, T >	84
Equipamento	86
Atuador	58
Motor	134
Sensor	151
Valvula	186
doctest::detail::Expression_lhs< L >	94
doctest::detail::ExpressionDecomposer	96
doctest::detail::types::false_type	97
doctest::detail::has_insertion_operator< T, decltype(operator<<(declval< std::ostream & >(), declval< const T & >()), void())>	104
doctest::detail::is_container< T, types::void_t< typename T::value_type, typename T::iterator, decltype(declval< T >().begin()), decltype(declval< T >().end())> >	112
doctest::detail::is_pair< T, types::void_t< typename T::first_type, typename T::second_type, decltype(declval< T >().first), decltype(declval< T >().second)> >	114
doctest::detail::is_std_string< T, typename types::enable_if< types::is_same< decltype(declval< const T & >().c_str()), const char * >value &&types::is_same< decltype(declval< const T & >().size()), size_t >value >type >	119
doctest::detail::types::is_array< T[SIZE]>	110
doctest::detail::types::is_pointer< T * >	115
doctest::detail::types::is_rvalue_reference< T && >	116

doctest::detail::types::is_same< T, T >	118
doctest::detail::deferred_false< T >	82
doctest::detail::has_insertion_operator< T, typename >	103
doctest::detail::is_container< T, typename >	111
doctest::detail::is_pair< T, typename >	113
doctest::detail::is_std_string< T, Enable >	118
doctest::detail::types::is_array< T >	110
doctest::detail::types::is_pointer< T >	114
doctest::detail::types::is_rvalue_reference< T >	116
doctest::detail::types::is_same< T, U >	117
doctest::detail::filldata< T, Enable >	98
doctest::detail::filldata< const char[N]>	98
doctest::detail::filldata< const void * >	99
doctest::detail::filldata< const volatile void * >	100
doctest::detail::filldata< T * >	100
doctest::detail::filldata< T, typename detail::types::enable_if< !detailhas_insertion_operator< T >value &&detail::is_container< T >value >type >	101
doctest::detail::filldata< T, typename detail::types::enable_if< !detailhas_insertion_operator< T >value &&detail::is_pair< T >value >type >	102
doctest::detail::filldata< T[N]>	102
doctest::IContextScope	104
doctest::detail::ContextScopeBase	79
doctest::detail::ContextScope< L >	77
doctest::detail::IExceptionTranslator	107
doctest::detail::ExceptionTranslator< T >	93
doctest::IReporter	107
doctest::detail::types::is_enum< T >	112
doctest::IsNaN< F >	120
LinhaDeProducao	122
Log	128
doctest::MessageData	133
doctest::detail::MessageBuilder	130
Parametro	136
doctest::QueryData	142
doctest::detail::RelationalComparator< int, L, R >	143
doctest::detail::types::remove_const< T >	144
doctest::detail::types::remove_const< const T >	145
doctest::detail::types::remove_reference< T >	145
doctest::detail::types::remove_reference< T & >	146
doctest::detail::types::remove_reference< T && >	146
doctest::detail::Result	147
std::runtime_error	
ExcecaoSistema	93
AcessoNegado	49
EntradaInvalida	85
IdDuplicado	105
IdInexistente	106
IdInvalido	106
LimitesInvalidos	121
SessaoInvalida	153
doctest::detail::should_stringify_as_underlying_type< T >	153
Sistema	154
doctest::String	161
doctest::AssertData::StringContains	165
doctest::detail::StringMakerBase< C >	167
doctest::detail::StringMakerBase< detail::has_insertion_operator< T >value detailtypes::is_pointer< T >value detailtypes::is_array< T >value detailis_pair< T >value detailis_container< T >value >	167

doctest::StringMaker< T >	167
doctest::detail::StringMakerBase< true >	168
doctest::detail::Subcase	169
doctest::SubcaseSignature	171
doctest::TestCaseData	175
doctest::detail::TestCase	172
doctest::TestCaseException	177
doctest::detail::TestFailureException	178
doctest::TestRunStats	178
doctest::detail::TestSuite	180
doctest::detail::types::true_type	182
doctest::detail::has_insertion_operator< T, decltype(operator<<(declval< std::ostream & >(), declval< const T & >()), void())>	104
doctest::detail::is_container< T, types::void_t< typename T::value_type, typename T::iterator, decltype(declval< T >().begin()), decltype(declval< T >().end())> >	112
doctest::detail::is_pair< T, types::void_t< typename T::first_type, typename T::second_type, decltype(declval< T >().first), decltype(declval< T >().second)> >	114
doctest::detail::is_std_string< T, typename types::enable_if< types::is_same< decltype(declval< const T & >().c_str()), const char * >value &&types::is_same< decltype(declval< const T & >().size()), size_t >value >type >	119
doctest::detail::types::is_array< T[SIZE]>	110
doctest::detail::types::is_pointer< T * >	115
doctest::detail::types::is_rvalue_reference< T && >	116
doctest::detail::types::is_same< T, T >	118
std::tuple< Types >	183
doctest::detail::types::underlying_type< T >	183
Usuario	184

Chapter 7

Class Index

7.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

AcessoNegado	
Operação sem permissão para o cargo do usuário logado	49
Alarme	
Alarme associado a um Parametro de um equipamento	49
doctest::Approx	52
doctest::AssertData	55
Atuador	58
std::basic_istream< charT, traits >	60
std::basic_ostream< charT, traits >	60
std::char_traits< charT >	61
Conjunto	
Agrupamento lógico de equipamentos dentro de uma LinhaDeProducao	61
doctest::Contains	67
doctest::Context	68
doctest::ContextOptions	71
doctest::detail::ContextScope< L >	77
doctest::detail::ContextScopeBase	79
doctest::CurrentTestCaseStats	81
doctest::detail::deferred_false< T >	82
Dependencia	83
doctest::detail::types::enable_if< COND, T >	84
doctest::detail::types::enable_if< true, T >	84
EntradaInvalida	
Entrada inválida (string vazia, argumento sem sentido)	85
Equipamento	
Representa um equipamento físico dentro de um Conjunto	86
ExcecaoSistema	
Exceção base do sistema de monitoramento	93
doctest::detail::ExceptionTranslator< T >	93
doctest::detail::Expression_lhs< L >	94
doctest::detail::ExpressionDecomposer	96
doctest::detail::types::false_type	97
doctest::detail::filldata< T, Enable >	98
doctest::detail::filldata< const char[N]>	98
doctest::detail::filldata< const void * >	99

doctest::detail::filldata< const volatile void * >	100
doctest::detail::filldata< T * >	100
doctest::detail::filldata< T, typename detail::types::enable_if< !detail::has_insertion_operator< T >::value &&detail::is_container< T >::value >::type >	101
doctest::detail::filldata< T, typename detail::types::enable_if< !detail::has_insertion_operator< T >::value &&detail::is_pair< T >::value >::type >	102
doctest::detail::filldata< T[N]>	102
doctest::detail::has_insertion_operator< T, typename >	103
doctest::detail::has_insertion_operator< T, decltype(operator<<(declval< std::ostream & >(), declval< const T & >()), void())>	104
doctest::IContextScope	104
IdDuplicado	
Tentativa de inserir um identificador já existente	105
IdInexistente	
Acesso a um identificador inexistente	106
IdInvalido	
Identificador inválido (ex.: negativo)	106
doctest::detail::IExceptionTranslator	107
doctest::IReporter	107
doctest::detail::types::is_array< T >	110
doctest::detail::types::is_array< T[SIZE]>	110
doctest::detail::is_container< T, typename >	111
doctest::detail::is_container< T, types::void_t< typename T::value_type, typename T::iterator, decltype(declval< T >().begin()), decltype(declval< T >().end())> >	112
doctest::detail::types::is_enum< T >	112
doctest::detail::is_pair< T, typename >	113
doctest::detail::is_pair< T, types::void_t< typename T::first_type, typename T::second_type, decltype(declval< T >().first), decltype(declval< T >().second)> >	114
doctest::detail::types::is_pointer< T >	114
doctest::detail::types::is_pointer< T * >	115
doctest::detail::types::is_rvalue_reference< T >	116
doctest::detail::types::is_rvalue_reference< T && >	116
doctest::detail::types::is_same< T, U >	117
doctest::detail::types::is_same< T, T >	118
doctest::detail::is_std_string< T, Enable >	118
doctest::detail::is_std_string< T, typename types::enable_if< types::is_same< decltype(declval< const T & >().c_str()), const char * >::value &&types::is_same< decltype(declval< const T & >().size()), size_t >::value >::type >	119
doctest::IsNaN< F >	120
LimitesInvalidos	
Limites incoerentes (Max < Min)	121
LinhaDeProducao	
Representa uma linha de produção industrial	122
Log	
Registra eventos e operações do sistema em arquivo de texto	128
doctest::detail::MessageBuilder	130
doctest::MessageData	133
Motor	134
Parametro	
Representa um parâmetro monitorado de um equipamento	136
doctest::QueryData	142
doctest::detail::RelationalComparator< int, L, R >	143
doctest::detail::types::remove_const< T >	144
doctest::detail::types::remove_const< const T >	145
doctest::detail::types::remove_reference< T >	145
doctest::detail::types::remove_reference< T & >	146
doctest::detail::types::remove_reference< T && >	146
doctest::detail::Result	147

doctest::detail::ResultBuilder	148
Sensor	151
SessaoInvalida	
Operação que exige sessão ativa sem usuário logado	153
doctest::detail::should_stringify_as_underlying_type< T >	153
Sistema	
Ponto de entrada central do sistema de monitoramento	154
doctest::String	161
doctest::AssertData::StringContains	165
doctest::StringMaker< T >	167
doctest::detail::StringMakerBase< C >	167
doctest::detail::StringMakerBase< true >	168
doctest::detail::Subcase	169
doctest::SubcaseSignature	171
doctest::detail::TestCase	172
doctest::TestCaseData	175
doctest::TestCaseException	177
doctest::detail::TestFailureException	178
doctest::TestRunStats	178
doctest::detail::TestSuite	180
doctest::detail::types::true_type	182
std::tuple< Types >	183
doctest::detail::types::underlying_type< T >	183
Usuario	
Representa um usuário do sistema com credenciais e cargo	184
Valvula	186

Chapter 8

File Index

8.1 File List

Here is a list of all files with brief descriptions:

include/Alarme.hpp	189
include/Atuador.hpp	190
include/Cargo.hpp	190
include/Conjunto.hpp	191
include/Dependencia.hpp	192
include/Equipamento.hpp	193
include/EstadoOperacional.hpp	194
include/Excecoes.hpp	194
include/LinhaDeProducao.hpp	196
include/Log.hpp	197
include/Motor.hpp	197
include/Parametro.hpp	198
include/Sensor.hpp	199
include/Sistema.hpp	199
include/Usuario.hpp	200
include/Valvula.hpp	201
src/Alarme.cpp	202
src/Atuador.cpp	202
src/Conjunto.cpp	203
src/Dependencia.cpp	204
src/Equipamento.cpp	205
src/LinhaDeProducao.cpp	207
src/Log.cpp	208
src/main.cpp	209
src/Motor.cpp	213
src/Parametro.cpp	214
src/Sensor.cpp	215
src/Sistema.cpp	216
src/Usuario.cpp	219
src/Valvula.cpp	220
tests/doctest.h	220
tests/test_Alarme.cpp	426
tests/test_Conjunto.cpp	428
tests/test_Equipamento.cpp	435
tests/test_Excecoes.cpp	440

tests/ test_LinhaDeProducao.cpp	446
tests/ test_Log.cpp	452
tests/ test_main.cpp	454
tests/ test_Parametro.cpp	454
tests/ test_Sistema.cpp	458
tests/ test_Usuario.cpp	464

Chapter 9

Directory Documentation

9.1 design Directory Reference

9.2 include Directory Reference

Files

- file [Alarme.hpp](#)
- file [Atuador.hpp](#)
- file [Cargo.hpp](#)
- file [Conjunto.hpp](#)
- file [Dependencia.hpp](#)
- file [Equipamento.hpp](#)
- file [EstadoOperacional.hpp](#)
- file [Excecoes.hpp](#)
- file [LinhaDeProducao.hpp](#)
- file [Log.hpp](#)
- file [Motor.hpp](#)
- file [Parametro.hpp](#)
- file [Sensor.hpp](#)
- file [Sistema.hpp](#)
- file [Usuario.hpp](#)
- file [Valvula.hpp](#)

9.3 src Directory Reference

Files

- file [Alarme.cpp](#)
- file [Atuador.cpp](#)
- file [Conjunto.cpp](#)
- file [Dependencia.cpp](#)
- file [Equipamento.cpp](#)
- file [LinhaDeProducao.cpp](#)
- file [Log.cpp](#)
- file [main.cpp](#)
- file [Motor.cpp](#)
- file [Parametro.cpp](#)
- file [Sensor.cpp](#)
- file [Sistema.cpp](#)
- file [Usuario.cpp](#)
- file [Valvula.cpp](#)

9.4 tests Directory Reference

Files

- file [doctest.h](#)
- file [test_Alarme.cpp](#)
- file [test_Conjunto.cpp](#)
- file [test_Equipamento.cpp](#)
- file [test_Excecoes.cpp](#)
- file [test_LinhaDeProducao.cpp](#)
- file [test_Log.cpp](#)
- file [test_main.cpp](#)
- file [test_Parametro.cpp](#)
- file [test_Sistema.cpp](#)
- file [test_Usuario.cpp](#)

Chapter 10

Namespace Documentation

10.1 doctest Namespace Reference

Namespaces

- namespace [detail](#)
- namespace [assertType](#)
- namespace [Color](#)
- namespace [TestCaseFailureReason](#)

Classes

- class [String](#)
- struct [StringMaker](#)
- class [Contains](#)
- struct [Approx](#)
- struct [IsNaN](#)
- struct [ContextOptions](#)
- struct [AssertData](#)
- struct [SubcaseSignature](#)
- struct [TestCaseData](#)
- struct [IContextScope](#)
- struct [MessageData](#)
- class [Context](#)
- struct [CurrentTestCaseStats](#)
- struct [TestCaseException](#)
- struct [TestRunStats](#)
- struct [QueryData](#)
- struct [IReporter](#)

Typedefs

- typedef decltype(sizeof(void*)) [size_t](#)

Functions

- [DOCTEST_INTERFACE](#) [String](#) [operator+](#) (const [String](#) &lhs, const [String](#) &rhs)
- [DOCTEST_INTERFACE](#) bool [operator==](#) (const [String](#) &lhs, const [String](#) &rhs)
- [DOCTEST_INTERFACE](#) bool [operator!=](#) (const [String](#) &lhs, const [String](#) &rhs)
- [DOCTEST_INTERFACE](#) bool [operator<](#) (const [String](#) &lhs, const [String](#) &rhs)
- [DOCTEST_INTERFACE](#) bool [operator>](#) (const [String](#) &lhs, const [String](#) &rhs)
- [DOCTEST_INTERFACE](#) bool [operator<=](#) (const [String](#) &lhs, const [String](#) &rhs)
- [DOCTEST_INTERFACE](#) bool [operator>=](#) (const [String](#) &lhs, const [String](#) &rhs)
- [template](#)<typename T>
[String](#) [toString](#) ()
- [template](#)<typename T, typename [detail::types::enable_if](#)<![detail::should_stringify_as_underlying_type](#)< T >::value, bool >::type = true>
[String](#) [toString](#) (const [DOCTEST_REF_WRAP](#)(T) value)
- [String](#) && [toString](#) ([String](#) &&in)
- [DOCTEST_INTERFACE](#) [String](#) [toString](#) (const [String](#) &in)
- [DOCTEST_INTERFACE](#) [String](#) [toString](#) ([std::nullptr_t](#))
- [DOCTEST_INTERFACE](#) [String](#) [toString](#) (bool in)
- [DOCTEST_INTERFACE](#) [String](#) [toString](#) (float in)
- [DOCTEST_INTERFACE](#) [String](#) [toString](#) (double in)
- [DOCTEST_INTERFACE](#) [String](#) [toString](#) (double long in)
- [DOCTEST_INTERFACE](#) [String](#) [toString](#) (char in)
- [DOCTEST_INTERFACE](#) [String](#) [toString](#) (char signed in)
- [DOCTEST_INTERFACE](#) [String](#) [toString](#) (char unsigned in)
- [DOCTEST_INTERFACE](#) [String](#) [toString](#) (short in)
- [DOCTEST_INTERFACE](#) [String](#) [toString](#) (short unsigned in)
- [DOCTEST_INTERFACE](#) [String](#) [toString](#) (signed in)
- [DOCTEST_INTERFACE](#) [String](#) [toString](#) (unsigned in)
- [DOCTEST_INTERFACE](#) [String](#) [toString](#) (long in)
- [DOCTEST_INTERFACE](#) [String](#) [toString](#) (long unsigned in)
- [DOCTEST_INTERFACE](#) [String](#) [toString](#) (long long in)
- [DOCTEST_INTERFACE](#) [String](#) [toString](#) (long long unsigned in)
- [DOCTEST_INTERFACE](#) [String](#) [toString](#) (const [Contains](#) &in)
- [DOCTEST_INTERFACE](#) bool [operator==](#) (const [String](#) &lhs, const [Contains](#) &rhs)
- [DOCTEST_INTERFACE](#) bool [operator==](#) (const [Contains](#) &lhs, const [String](#) &rhs)
- [DOCTEST_INTERFACE](#) bool [operator!=](#) (const [String](#) &lhs, const [Contains](#) &rhs)
- [DOCTEST_INTERFACE](#) bool [operator!=](#) (const [Contains](#) &lhs, const [String](#) &rhs)
- [DOCTEST_INTERFACE](#) [String](#) [toString](#) (const [Approx](#) &in)
- [DOCTEST_INTERFACE](#) [String](#) [toString](#) ([IsNaN](#)< float > in)
- [DOCTEST_INTERFACE](#) [String](#) [toString](#) ([IsNaN](#)< double > in)
- [DOCTEST_INTERFACE](#) [String](#) [toString](#) ([IsNaN](#)< double long > in)
- [DOCTEST_INTERFACE](#) const [ContextOptions](#) * [getContextOptions](#) ()
- [DOCTEST_INTERFACE](#) const char * [assertString](#) ([assertType::Enum](#) at)
- [DOCTEST_INTERFACE](#) const char * [failureString](#) ([assertType::Enum](#) at)
- [DOCTEST_DEFINE_DECORATOR](#) (test_suite, const char *, "")
- [DOCTEST_DEFINE_DECORATOR](#) (description, const char *, "")
- [DOCTEST_DEFINE_DECORATOR](#) (skip, bool, true)
- [DOCTEST_DEFINE_DECORATOR](#) (no_breaks, bool, true)
- [DOCTEST_DEFINE_DECORATOR](#) (no_output, bool, true)
- [DOCTEST_DEFINE_DECORATOR](#) (timeout, double, 0)
- [DOCTEST_DEFINE_DECORATOR](#) (may_fail, bool, true)
- [DOCTEST_DEFINE_DECORATOR](#) (should_fail, bool, true)
- [DOCTEST_DEFINE_DECORATOR](#) (expected_failures, int, 0)
- [template](#)<typename T>
int [registerExceptionTranslator](#) ([String](#)(*translateFunction)(T)) noexcept
- [DOCTEST_INTERFACE](#) const char * [skipPathFromFilename](#) (const char *file)
- [template](#)<typename Reporter>
int [registerReporter](#) (const char *name, int priority, bool isReporter) noexcept

Variables

- template struct [DOCTEST_INTERFACE_DECL IsNaN< float >](#)
- template struct [DOCTEST_INTERFACE_DECL IsNaN< double >](#)
- template struct [DOCTEST_INTERFACE_DECL IsNaN< long double >](#)
- struct [DOCTEST_INTERFACE TestCaseData](#)
- [DOCTEST_INTERFACE](#) bool [is_running_in_test](#)

10.1.1 Typedef Documentation

10.1.1.1 `size_t`

```
typedef decltype(sizeof(void*)) std::size\_t
```

Definition at line [671](#) of file [doctest.h](#).

10.1.2 Function Documentation

10.1.2.1 `assertString()`

```
DOCTEST\_INTERFACE const char * doctest::assertString (  
    assertType::Enum at)
```

10.1.2.2 `DOCTEST_DEFINE_DECORATOR()` [1/9]

```
doctest::DOCTEST_DEFINE_DECORATOR (  
    description ,  
    const char * ,  
    "" )
```

10.1.2.3 `DOCTEST_DEFINE_DECORATOR()` [2/9]

```
doctest::DOCTEST_DEFINE_DECORATOR (  
    expected_failures ,  
    int ,  
    0 )
```

10.1.2.4 `DOCTEST_DEFINE_DECORATOR()` [3/9]

```
doctest::DOCTEST_DEFINE_DECORATOR (  
    may_fail ,  
    bool ,  
    true )
```

10.1.2.5 DOCTEST_DEFINE_DECORATOR() [4/9]

```
doctest::DOCTEST_DEFINE_DECORATOR (
    no_breaks ,
    bool ,
    true )
```

10.1.2.6 DOCTEST_DEFINE_DECORATOR() [5/9]

```
doctest::DOCTEST_DEFINE_DECORATOR (
    no_output ,
    bool ,
    true )
```

10.1.2.7 DOCTEST_DEFINE_DECORATOR() [6/9]

```
doctest::DOCTEST_DEFINE_DECORATOR (
    should_fail ,
    bool ,
    true )
```

10.1.2.8 DOCTEST_DEFINE_DECORATOR() [7/9]

```
doctest::DOCTEST_DEFINE_DECORATOR (
    skip ,
    bool ,
    true )
```

10.1.2.9 DOCTEST_DEFINE_DECORATOR() [8/9]

```
doctest::DOCTEST_DEFINE_DECORATOR (
    test_suite ,
    const char * ,
    "" )
```

10.1.2.10 DOCTEST_DEFINE_DECORATOR() [9/9]

```
doctest::DOCTEST_DEFINE_DECORATOR (
    timeout ,
    double ,
    0 )
```

10.1.2.11 failureString()

```
DOCTEST_INTERFACE const char * doctest::failureString (
    assertType::Enum at)
```

10.1.2.12 getContextOptions()

```
DOCTEST_INTERFACE const ContextOptions * doctest::getContextOptions ()
```

10.1.2.13 operator"!==() [1/3]

```
DOCTEST_INTERFACE bool doctest::operator!= (
    const Contains & lhs,
    const String & rhs)
```

10.1.2.14 operator"!==() [2/3]

```
DOCTEST_INTERFACE bool doctest::operator!= (
    const String & lhs,
    const Contains & rhs)
```

10.1.2.15 operator"!==() [3/3]

```
DOCTEST_INTERFACE bool doctest::operator!= (
    const String & lhs,
    const String & rhs)
```

10.1.2.16 operator+()

```
DOCTEST_INTERFACE String doctest::operator+ (
    const String & lhs,
    const String & rhs)
```

10.1.2.17 operator<()

```
DOCTEST_INTERFACE bool doctest::operator< (
    const String & lhs,
    const String & rhs)
```

10.1.2.18 operator<=()

```
DOCTEST_INTERFACE bool doctest::operator<= (
    const String & lhs,
    const String & rhs)
```

10.1.2.19 operator==() [1/3]

```
DOCTEST_INTERFACE bool doctest::operator== (
    const Contains & lhs,
    const String & rhs)
```

10.1.2.20 operator==() [2/3]

```
DOCTEST_INTERFACE bool doctest::operator== (
    const String & lhs,
    const Contains & rhs)
```

10.1.2.21 operator==() [3/3]

```
DOCTEST_INTERFACE bool doctest::operator== (
    const String & lhs,
    const String & rhs)
```

10.1.2.22 operator>()

```
DOCTEST_INTERFACE bool doctest::operator> (
    const String & lhs,
    const String & rhs)
```

10.1.2.23 operator>=()

```
DOCTEST_INTERFACE bool doctest::operator>= (
    const String & lhs,
    const String & rhs)
```

10.1.2.24 registerExceptionTranslator()

```
template<typename T>
int doctest::registerExceptionTranslator (
    String(* translateFunction ) (T)) [noexcept]
```

Definition at line 2371 of file [doctest.h](#).

10.1.2.25 registerReporter()

```
template<typename Reporter>
int doctest::registerReporter (
    const char * name,
    int priority,
    bool isReporter) [noexcept]
```

Definition at line 2837 of file [doctest.h](#).

10.1.2.26 skipPathFromFilename()

```
DOCTEST_INTERFACE const char * doctest::skipPathFromFilename (
    const char * file)
```

10.1.2.27 toString() [1/25]

```
template<typename T>
String doctest::toString ()
```

Definition at line 1128 of file [doctest.h](#).

10.1.2.28 toString() [2/25]

```
DOctest_INTERFACE String doctest::toString (
    bool in)
```

10.1.2.29 toString() [3/25]

```
DOctest_INTERFACE String doctest::toString (
    char in)
```

10.1.2.30 toString() [4/25]

```
DOctest_INTERFACE String doctest::toString (
    char signed in)
```

10.1.2.31 toString() [5/25]

```
DOctest_INTERFACE String doctest::toString (
    char unsigned in)
```

10.1.2.32 toString() [6/25]

```
DOctest_INTERFACE String doctest::toString (
    const Approx & in)
```

10.1.2.33 toString() [7/25]

```
DOctest_INTERFACE String doctest::toString (
    const Contains & in)
```

10.1.2.34 toString() [8/25]

```
template<typename T, typename detail::types::enable_if<!detail::should_stringify_as_underlying_type<
T >::value, bool >::type = true>
String doctest::toString (
    const DOctest_REF_WRAP(T) value)
```

Definition at line 1143 of file [doctest.h](#).

10.1.2.35 toString() [9/25]

```
DOCTEST_INTERFACE String doctest::toString (  
    const String & in)
```

10.1.2.36 toString() [10/25]

```
DOCTEST_INTERFACE String doctest::toString (  
    double in)
```

10.1.2.37 toString() [11/25]

```
DOCTEST_INTERFACE String doctest::toString (  
    double long in)
```

10.1.2.38 toString() [12/25]

```
DOCTEST_INTERFACE String doctest::toString (  
    float in)
```

10.1.2.39 toString() [13/25]

```
DOCTEST_INTERFACE String doctest::toString (  
    IsNaN< double > in)
```

10.1.2.40 toString() [14/25]

```
DOCTEST_INTERFACE String doctest::toString (  
    IsNaN< double long > in)
```

10.1.2.41 toString() [15/25]

```
DOCTEST_INTERFACE String doctest::toString (  
    IsNaN< float > in)
```

10.1.2.42 toString() [16/25]

```
DOCTEST_INTERFACE String doctest::toString (  
    long in)
```

10.1.2.43 toString() [17/25]

```
DOCTEST_INTERFACE String doctest::toString (  
    long long in)
```


10.1.2.44 toString() [18/25]

```
DOCTEST_INTERFACE String doctest::toString (
    long long unsigned in)
```

10.1.2.45 toString() [19/25]

```
DOCTEST_INTERFACE String doctest::toString (
    long unsigned in)
```

10.1.2.46 toString() [20/25]

```
DOCTEST_INTERFACE String doctest::toString (
    short in)
```

10.1.2.47 toString() [21/25]

```
DOCTEST_INTERFACE String doctest::toString (
    short unsigned in)
```

10.1.2.48 toString() [22/25]

```
DOCTEST_INTERFACE String doctest::toString (
    signed in)
```

10.1.2.49 toString() [23/25]

```
DOCTEST_INTERFACE String doctest::toString (
    std::nullptr_t )
```

10.1.2.50 toString() [24/25]

```
String && doctest::toString (
    String && in) [inline]
```

Definition at line 1148 of file [doctest.h](#).

10.1.2.51 toString() [25/25]

```
DOCTEST_INTERFACE String doctest::toString (
    unsigned in)
```

10.1.3 Variable Documentation

10.1.3.1 `is_running_in_test`

`DOCTEST_INTERFACE` `bool doctest::is_running_in_test [extern]`

10.1.3.2 `IsNaN< double >`

`template struct DOCTEST_INTERFACE_DECL doctest::IsNaN< double > [extern]`

10.1.3.3 `IsNaN< float >`

`template struct DOCTEST_INTERFACE_DECL doctest::IsNaN< float > [extern]`

10.1.3.4 `IsNaN< long double >`

`template struct DOCTEST_INTERFACE_DECL doctest::IsNaN< long double > [extern]`

10.1.3.5 `TestCaseData`

`struct DOCTEST_INTERFACE doctest::TestCaseData`

Definition at line 1637 of file [doctest.h](#).

10.2 `doctest::assertType` Namespace Reference

Enumerations

- enum `Enum` {
 - `is_warn` = 1 , `is_check` = 2 * `is_warn` , `is_require` = 2 * `is_check` , `is_normal` = 2 * `is_require` ,
 - `is_throws` = 2 * `is_normal` , `is_throws_as` = 2 * `is_throws` , `is_throws_with` = 2 * `is_throws_as` , `is_nothrow` = 2 * `is_throws_with` ,
 - `is_false` = 2 * `is_nothrow` , `is_unary` = 2 * `is_false` , `is_eq` = 2 * `is_unary` , `is_ne` = 2 * `is_eq` ,
 - `is_lt` = 2 * `is_ne` , `is_gt` = 2 * `is_lt` , `is_ge` = 2 * `is_gt` , `is_le` = 2 * `is_ge` ,
 - `DT_WARN` = `is_normal` | `is_warn` , `DT_CHECK` = `is_normal` | `is_check` , `DT_REQUIRE` = `is_normal` | `is_↵` require , `DT_WARN_FALSE` = `is_normal` | `is_false` | `is_warn` ,
 - `DT_CHECK_FALSE` = `is_normal` | `is_false` | `is_check` , `DT_REQUIRE_FALSE` = `is_normal` | `is_false` | `is_↵` require , `DT_WARN_THROWS` = `is_throws` | `is_warn` , `DT_CHECK_THROWS` = `is_throws` | `is_check` ,
 - `DT_REQUIRE_THROWS` = `is_throws` | `is_require` , `DT_WARN_THROWS_AS` = `is_throws_as` | `is_warn` ,
 - `DT_CHECK_THROWS_AS` = `is_throws_as` | `is_check` , `DT_REQUIRE_THROWS_AS` = `is_throws_as` | `is_require` ,
 - `DT_WARN_THROWS_WITH` = `is_throws_with` | `is_warn` , `DT_CHECK_THROWS_WITH` = `is_throws_with` | `is_check` ,
 - `DT_REQUIRE_THROWS_WITH` = `is_throws_with` | `is_require` , `DT_WARN_THROWS_WITH_AS` = `is_throws_with` | `is_throws_as` | `is_warn` ,
 - `DT_CHECK_THROWS_WITH_AS` = `is_throws_with` | `is_throws_as` | `is_check` , `DT_REQUIRE_THROWS_WITH_AS` = `is_throws_with` | `is_throws_as` | `is_require` ,
 - `DT_WARN_NOTHROW` = `is_nothrow` | `is_warn` ,
 - `DT_CHECK_NOTHROW` = `is_nothrow` | `is_check` ,
 - `DT_REQUIRE_NOTHROW` = `is_nothrow` | `is_require` , `DT_WARN_EQ` = `is_normal` | `is_eq` | `is_warn` ,

```

DT_CHECK_EQ = is_normal | is_eq | is_check , DT_REQUIRE_EQ = is_normal | is_eq | is_require ,
DT_WARN_NE = is_normal | is_ne | is_warn , DT_CHECK_NE = is_normal | is_ne | is_check ,
DT_REQUIRE_NE = is_normal | is_ne | is_require , DT_WARN_GT = is_normal | is_gt | is_warn ,
DT_CHECK_GT = is_normal | is_gt | is_check , DT_REQUIRE_GT = is_normal | is_gt | is_require ,
DT_WARN_LT = is_normal | is_lt | is_warn , DT_CHECK_LT = is_normal | is_lt | is_check ,
DT_REQUIRE_LT = is_normal | is_lt | is_require , DT_WARN_GE = is_normal | is_ge | is_warn ,
DT_CHECK_GE = is_normal | is_ge | is_check , DT_REQUIRE_GE = is_normal | is_ge | is_require
,
DT_WARN_LE = is_normal | is_le | is_warn , DT_CHECK_LE = is_normal | is_le | is_check ,
DT_REQUIRE_LE = is_normal | is_le | is_require , DT_WARN_UNARY = is_normal | is_unary | is_↵
warn ,
DT_CHECK_UNARY = is_normal | is_unary | is_check , DT_REQUIRE_UNARY = is_normal | is_↵
_unary | is_require , DT_WARN_UNARY_FALSE = is_normal | is_false | is_unary | is_warn ,
DT_CHECK_UNARY_FALSE = is_normal | is_false | is_unary | is_check ,
DT_REQUIRE_UNARY_FALSE = is_normal | is_false | is_unary | is_require }

```

10.2.1 Enumeration Type Documentation

10.2.1.1 Enum

```
enum doctest::assertType::Enum
```

Enumerator

is_warn	
is_check	
is_require	
is_normal	
is_throws	
is_throws_as	
is_throws_with	
is_nothrow	
is_false	
is_unary	
is_eq	
is_ne	
is_lt	
is_gt	
is_ge	
is_le	
DT_WARN	
DT_CHECK	
DT_REQUIRE	
DT_WARN_FALSE	

DT_CHECK_FALSE	
DT_REQUIRE_FALSE	
DT_WARN_THROWS	
DT_CHECK_THROWS	
DT_REQUIRE_THROWS	
DT_WARN_THROWS_AS	
DT_CHECK_THROWS_AS	
DT_REQUIRE_THROWS_AS	
DT_WARN_THROWS_WITH	
DT_CHECK_THROWS_WITH	
DT_REQUIRE_THROWS_WITH	
DT_WARN_THROWS_WITH_AS	
DT_CHECK_THROWS_WITH_AS	
DT_REQUIRE_THROWS_WITH_AS	
DT_WARN_NOTHROW	
DT_CHECK_NOTHROW	
DT_REQUIRE_NOTHROW	
DT_WARN_EQ	
DT_CHECK_EQ	
DT_REQUIRE_EQ	
DT_WARN_NE	
DT_CHECK_NE	
DT_REQUIRE_NE	
DT_WARN_GT	
DT_CHECK_GT	
DT_REQUIRE_GT	
DT_WARN_LT	
DT_CHECK_LT	
DT_REQUIRE_LT	
DT_WARN_GE	
DT_CHECK_GE	
DT_REQUIRE_GE	
DT_WARN_LE	
DT_CHECK_LE	
DT_REQUIRE_LE	
DT_WARN_UNARY	
DT_CHECK_UNARY	
DT_REQUIRE_UNARY	
DT_WARN_UNARY_FALSE	
DT_CHECK_UNARY_FALSE	
DT_REQUIRE_UNARY_FALSE	

Definition at line 1532 of file [doctest.h](#).

10.3 doctest::Color Namespace Reference

Enumerations

- enum [Enum](#) {
[None](#) = 0 , [White](#) , [Red](#) , [Green](#) ,
[Blue](#) , [Cyan](#) , [Yellow](#) , [Grey](#) ,
[Bright](#) = 0x10 , [BrightRed](#) = Bright | Red , [BrightGreen](#) = Bright | Green , [LightGrey](#) = Bright | Grey ,
[BrightWhite](#) = Bright | White }

Functions

- [DOCTEST_INTERFACE](#) [std::ostream](#) & [operator<<](#) ([std::ostream](#) &s, [Color::Enum](#) code)

10.3.1 Enumeration Type Documentation

10.3.1.1 Enum

```
enum doctest::Color::Enum
```

Enumerator

None	
White	
Red	
Green	
Blue	
Cyan	
Yellow	
Grey	
Bright	
BrightRed	
BrightGreen	
LightGrey	
BrightWhite	

Definition at line [2085](#) of file [doctest.h](#).

10.3.2 Function Documentation

10.3.2.1 operator<<()

```
DOCTEST\_INTERFACE std::ostream & doctest::Color::operator<< (  

    std::ostream & s,  

    Color::Enum code)
```

10.4 doctest::detail Namespace Reference

Namespaces

- namespace [types](#)
- namespace [binaryAssertComparison](#)

Classes

- struct [deferred_false](#)
- struct [is_std_string](#)
- struct [is_std_string< T, typename types::enable_if< types::is_same< decltype\(declval< const T &>\(\).c_str\(\)\), const char * >::value &&types::is_same< decltype\(declval< const T &>\(\).size\(\)\), size_t >::value >::type >](#)
- struct [has_insertion_operator](#)
- struct [has_insertion_operator< T, decltype\(operator<<\(declval< std::ostream &>\(\)\), declval< const T &>\(\)\), void\(\)>](#)
- struct [should_stringify_as_underlying_type](#)
- struct [is_pair](#)
- struct [is_pair< T, types::void_t< typename T::first_type, typename T::second_type, decltype\(declval< T >\(\).first\), decltype\(declval< T >\(\).second\)> >](#)
- struct [is_container](#)
- struct [is_container< T, types::void_t< typename T::value_type, typename T::iterator, decltype\(declval< T >\(\).begin\(\)\), decltype\(declval< T >\(\).end\(\)\)> >](#)
- struct [StringMakerBase](#)
- struct [filldata](#)
- struct [StringMakerBase< true >](#)
- struct [filldata< T\[N\]>](#)
- struct [filldata< const char\[N\]>](#)
- struct [filldata< const void * >](#)
- struct [filldata< const volatile void * >](#)
- struct [filldata< T * >](#)
- struct [filldata< T, typename detail::types::enable_if< !detail::has_insertion_operator< T >::value &&detail::is_pair< T >::value >::type >](#)
- struct [filldata< T, typename detail::types::enable_if< !detail::has_insertion_operator< T >::value &&detail::is_container< T >::value >::type >](#)
- struct [RelationalComparator](#)
- struct [Result](#)
- struct [ResultBuilder](#)
- struct [Expression_lhs](#)
- struct [ExpressionDecomposer](#)
- struct [Subcase](#)
- struct [TestSuite](#)
- struct [TestCase](#)
- struct [IExceptionTranslator](#)
- class [ExceptionTranslator](#)
- struct [ContextScopeBase](#)
- class [ContextScope](#)
- struct [MessageBuilder](#)
- struct [TestFailureException](#)

Typedefs

- using [funcType](#) = void (*)()
- using [assert_handler](#) = void (*)(const [AssertData](#) &)
- using [reporterCreatorFunc](#) = [IReporter](#) (*)(const [ContextOptions](#) &)

Functions

- [DOCTEST_INTERFACE](#) bool [isDebuggerActive](#) ()
- template<typename T>
T && [declval](#) ()
- template<class T>
[DOCTEST_CONSTEXPR_FUNC](#) T && [forward](#) (typename [types::remove_reference](#)< T >::type &t)
[DOCTEST_NOEXCEPT](#)
- template<class T>
[DOCTEST_CONSTEXPR_FUNC](#) T && [forward](#) (typename [types::remove_reference](#)< T >::type &&t)
[DOCTEST_NOEXCEPT](#)
- [DOCTEST_INTERFACE](#) std::ostream * [tlssPush](#) ()
- [DOCTEST_INTERFACE](#) String [tlssPop](#) ()
- template<typename T>
void [filloss](#) (std::ostream *stream, const T &in)
- template<typename T, size_t N>
void [filloss](#) (std::ostream *stream, const T(&in)[N])
- template<typename T>
String [toString](#) (const T &in)
- template<typename L, typename R>
String [stringifyBinaryExpr](#) (const [DOCTEST_REF_WRAP](#)(L) lhs, const char *op, const [DOCTEST_REF_WRAP](#)(R) rhs)
- [DOCTEST_INTERFACE](#) int [setTestSuite](#) (const [TestSuite](#) &ts) noexcept
- [DOCTEST_INTERFACE](#) int [regTest](#) (const [TestCase](#) &tc) noexcept
- [DOCTEST_INTERFACE](#) void [registerExceptionTranslatorImpl](#) (const [IExceptionTranslator](#) *et) noexcept
- template<typename L>
[ContextScope](#)< L > [MakeContextScope](#) (const L &lambda)
- [DOCTEST_INTERFACE](#) bool [checkIfShouldThrow](#) (assertType::Enum at)
- [DOCTEST_INTERFACE](#) void [throwException](#) ()
- [DOCTEST_INTERFACE](#) void [failed_out_of_a_testing_context](#) (const [AssertData](#) &ad)
- [DOCTEST_INTERFACE](#) bool [decomp_assert](#) (assertType::Enum at, const char *file, int line, const char *expr, const [Result](#) &result)
- template<int comparison, typename L, typename R>
[DOCTEST_NOINLINE](#) bool [binary_assert](#) (assertType::Enum at, const char *file, int line, const char *expr, const [DOCTEST_REF_WRAP](#)(L) lhs, const [DOCTEST_REF_WRAP](#)(R) rhs)
- template<typename L>
[DOCTEST_NOINLINE](#) bool [unary_assert](#) (assertType::Enum at, const char *file, int line, const char *expr, const [DOCTEST_REF_WRAP](#)(L) val)
- [DOCTEST_INTERFACE](#) void [registerReporterImpl](#) (const char *name, int prio, [reporterCreatorFunc](#) c, bool isReporter) noexcept
- template<typename Reporter>
[IReporter](#) * [reporterCreator](#) (const [ContextOptions](#) &o)
- [DOCTEST_INTERFACE](#) size_t [acquireGeneratorDecisionIndex](#) (size_t count)
- template<typename T, typename... Rest>
T [acquireGeneratorValue](#) (T first, Rest... rest)
- template<typename T>
int [instantiationHelper](#) (const T &) noexcept

Variables

- struct [DOCTEST_INTERFACE](#) [TestCase](#)

10.4.1 Typedef Documentation

10.4.1.1 `assert_handler`

using [doctest::detail::assert_handler](#) = void (*) (const [AssertData](#) &)

Definition at line [2591](#) of file [doctest.h](#).

10.4.1.2 `funcType`

using [doctest::detail::funcType](#) = void (*) ()

Definition at line [2235](#) of file [doctest.h](#).

10.4.1.3 `reporterCreatorFunc`

using [doctest::detail::reporterCreatorFunc](#) = [IReporter](#) * (*) (const [ContextOptions](#) &)

Definition at line [2825](#) of file [doctest.h](#).

10.4.2 Function Documentation

10.4.2.1 `acquireGeneratorDecisionIndex()`

```
DOCTEST\_INTERFACE size\_t doctest::detail::acquireGeneratorDecisionIndex (  
    size\_t count)
```

10.4.2.2 `acquireGeneratorValue()`

```
template<typename T, typename... Rest>  
T doctest::detail::acquireGeneratorValue (  
    T first,  
    Rest... rest)
```

Definition at line [2859](#) of file [doctest.h](#).

10.4.2.3 binary_assert()

```
template<int comparison, typename L, typename R>
DOCTEST_NOINLINE bool doctest::detail::binary_assert (
    assertType::Enum at,
    const char * file,
    int line,
    const char * expr,
    const DOCTEST_REF_WRAP(L) lhs,
    const DOCTEST_REF_WRAP(R) rhs)
```

Definition at line 2678 of file [doctest.h](#).

10.4.2.4 checkIfShouldThrow()

```
DOCTEST_INTERFACE bool doctest::detail::checkIfShouldThrow (
    assertType::Enum at)
```

10.4.2.5 declval()

```
template<typename T>
T && doctest::detail::declval ()
```

10.4.2.6 decomp_assert()

```
DOCTEST_INTERFACE bool doctest::detail::decomp_assert (
    assertType::Enum at,
    const char * file,
    int line,
    const char * expr,
    const Result & result)
```

10.4.2.7 failed_out_of_a_testing_context()

```
DOCTEST_INTERFACE void doctest::detail::failed_out_of_a_testing_context (
    const AssertData & ad)
```

10.4.2.8 filloss() [1/2]

```
template<typename T>
void doctest::detail::filloss (
    std::ostream * stream,
    const T & in)
```

Definition at line 1085 of file [doctest.h](#).

10.4.2.9 filloss() [2/2]

```
template<typename T, size\_t N>
void doctest::detail::filloss (
    std::ostream * stream,
    const T(&) in[N])
```

Definition at line [1090](#) of file [doctest.h](#).

10.4.2.10 forward() [1/2]

```
template<class T>
DOCTEST\_CONSTEXPR\_FUNC T && doctest::detail::forward (
    typename types::remove\_reference< T >::type && t)
```

Definition at line [837](#) of file [doctest.h](#).

10.4.2.11 forward() [2/2]

```
template<class T>
DOCTEST\_CONSTEXPR\_FUNC T && doctest::detail::forward (
    typename types::remove\_reference< T >::type & t)
```

Definition at line [831](#) of file [doctest.h](#).

10.4.2.12 instantiationHelper()

```
template<typename T>
int doctest::detail::instantiationHelper (
    const T & ) [noexcept]
```

Definition at line [2882](#) of file [doctest.h](#).

10.4.2.13 isDebuggerActive()

```
DOCTEST\_INTERFACE bool doctest::detail::isDebuggerActive ()
```

10.4.2.14 MakeContextScope()

```
template<typename L>
ContextScope< L > doctest::detail::MakeContextScope (
    const L & lambda)
```

Definition at line [2457](#) of file [doctest.h](#).

10.4.2.15 registerExceptionTranslatorImpl()

```
DOCTEST\_INTERFACE void doctest::detail::registerExceptionTranslatorImpl (
    const IExceptionTranslator * et) [noexcept]
```

10.4.2.16 registerReporterImpl()

```
DOCTEST_INTERFACE void doctest::detail::registerReporterImpl (
    const char * name,
    int prio,
    reporterCreatorFunc c,
    bool isReporter) [noexcept]
```

10.4.2.17 regTest()

```
DOCTEST_INTERFACE int doctest::detail::regTest (
    const TestCase & tc) [noexcept]
```

10.4.2.18 reporterCreator()

```
template<typename Reporter>
IReporter * doctest::detail::reporterCreator (
    const ContextOptions & o)
```

Definition at line 2831 of file [doctest.h](#).

10.4.2.19 setTestSuite()

```
DOCTEST_INTERFACE int doctest::detail::setTestSuite (
    const TestSuite & ts) [noexcept]
```

10.4.2.20 stringifyBinaryExpr()

```
template<typename L, typename R>
String doctest::detail::stringifyBinaryExpr (
    const DOCTEST_REF_WRAP(L) lhs,
    const char * op,
    const DOCTEST_REF_WRAP(R) rhs)
```

Definition at line 1289 of file [doctest.h](#).

10.4.2.21 throwException()

```
DOCTEST_INTERFACE void doctest::detail::throwException ()
```

10.4.2.22 tlssPop()

```
DOCTEST_INTERFACE String doctest::detail::tlssPop ()
```

10.4.2.23 tlssPush()

```
DOCTEST_INTERFACE std::ostream * doctest::detail::tlssPush ()
```

10.4.2.24 toStream()

```
template<typename T>  
String doctest::detail::toStream (  
    const T & in)
```

Definition at line 1097 of file [doctest.h](#).

10.4.2.25 unary_assert()

```
template<typename L>  
DOCTEST_NOINLINE bool doctest::detail::unary_assert (  
    assertType::Enum at,  
    const char * file,  
    int line,  
    const char * expr,  
    const DOCTEST_REF_WRAP(L) val)
```

Definition at line 2699 of file [doctest.h](#).

10.4.3 Variable Documentation

10.4.3.1 TestCase

```
struct DOCTEST_INTERFACE doctest::detail::TestCase
```

Definition at line 1466 of file [doctest.h](#).

10.5 doctest::detail::binaryAssertComparison Namespace Reference

Enumerations

- enum Enum {
 eq = 0 , ne , gt , lt ,
 ge , le }

10.5.1 Enumeration Type Documentation

10.5.1.1 Enum

```
enum doctest::detail::binaryAssertComparison::Enum
```

Enumerator

eq	
ne	
gt	
lt	
ge	
le	

Definition at line 1768 of file [doctest.h](#).

10.6 doctest::detail::types Namespace Reference

Classes

- struct [enable_if](#)
- struct [enable_if< true, T >](#)
- struct [true_type](#)
- struct [false_type](#)
- struct [remove_reference](#)
- struct [remove_reference< T & >](#)
- struct [remove_reference< T && >](#)
- struct [is_same](#)
- struct [is_same< T, T >](#)
- struct [is_rvalue_reference](#)
- struct [is_rvalue_reference< T && >](#)
- struct [remove_const](#)
- struct [remove_const< const T >](#)
- struct [is_enum](#)
- struct [underlying_type](#)
- struct [is_pointer](#)
- struct [is_pointer< T * >](#)
- struct [is_array](#)
- struct [is_array< T\[SIZE\]>](#)

Typedefs

- template<typename... Unused>
using [void_t](#) = void

10.6.1 Typedef Documentation

10.6.1.1 void_t

```
template<typename... Unused>
using doctest::detail::types::void_t = void
```

Definition at line 807 of file [doctest.h](#).

10.7 doctest::TestCaseFailureReason Namespace Reference

Enumerations

- enum [Enum](#) {
[None](#) = 0 , [AssertFailure](#) = 1 , [Exception](#) = 2 , [Crash](#) = 4 ,
[TooManyFailedAsserts](#) = 8 , [Timeout](#) = 16 , [ShouldHaveFailedButDidnt](#) = 32 , [ShouldHaveFailedAndDid](#) = 64
, [DidntFailExactlyNumTimes](#) = 128 , [FailedExactlyNumTimes](#) = 256 , [CouldHaveFailedAndDid](#) = 512 }

10.7.1 Enumeration Type Documentation

10.7.1.1 Enum

```
enum doctest::TestCaseFailureReason::Enum
```

Enumerator

None	
AssertFailure	
Exception	
Crash	
TooManyFailedAsserts	
Timeout	
ShouldHaveFailedButDidnt	
ShouldHaveFailedAndDid	
DidntFailExactlyNumTimes	
FailedExactlyNumTimes	
CouldHaveFailedAndDid	

Definition at line 2731 of file [doctest.h](#).

10.8 doctest_detail_test_suite_ns Namespace Reference

Functions

- [DOCTEST_INTERFACE doctest::detail::TestSuite & getCurrentTestSuite \(\)](#) noexcept

10.8.1 Function Documentation

10.8.1.1 getCurrentTestSuite()

```
DOCTEST_INTERFACE doctest::detail::TestSuite & doctest_detail_test_suite_ns::getCurrentTestSuite() [noexcept]
```

10.9 std Namespace Reference

Classes

- struct [char_traits](#)
- class [basic_ostream](#)
- class [basic_istream](#)
- class [tuple](#)

Typedefs

- typedef decltype(nullptr) [nullptr_t](#)
- typedef decltype(sizeof(void*)) [size_t](#)
- typedef [basic_ostream](#)< char, [char_traits](#)< char > > [ostream](#)
- typedef [basic_istream](#)< char, [char_traits](#)< char > > [istream](#)

Functions

- template<class traits>
[basic_ostream](#)< char, traits > & [operator<<](#) ([basic_ostream](#)< char, traits > &, const char *)

10.9.1 Typedef Documentation

10.9.1.1 istream

```
typedef basic\_istream<char, char\_traits<char> > std::istream
```

Definition at line [684](#) of file [doctest.h](#).

10.9.1.2 nullptr_t

```
typedef decltype(nullptr) std::nullptr\_t
```

Definition at line [670](#) of file [doctest.h](#).

10.9.1.3 ostream

```
typedef basic\_ostream<char, char\_traits<char> > std::ostream
```

Definition at line [678](#) of file [doctest.h](#).

10.9.1.4 `size_t`

```
typedef decltype(sizeof(void *)) std::size\_t
```

Definition at line [671](#) of file [doctest.h](#).

10.9.2 Function Documentation

10.9.2.1 `operator<<()`

```
template<class traits>  
basic\_ostream< char, traits > & std::operator<< (  
    basic\_ostream< char, traits > & ,  
    const char * )
```


Chapter 11

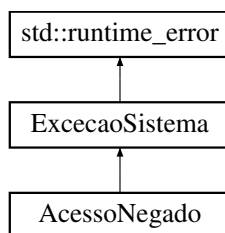
Class Documentation

11.1 AcessoNegado Class Reference

Operação sem permissão para o cargo do usuário logado.

```
#include <Excecoes.hpp>
```

Inheritance diagram for AcessoNegado:



11.1.1 Detailed Description

Operação sem permissão para o cargo do usuário logado.

Definition at line 49 of file [Excecoes.hpp](#).

The documentation for this class was generated from the following file:

- [include/Excecoes.hpp](#)

11.2 Alarme Class Reference

[Alarme](#) associado a um [Parametro](#) de um equipamento.

```
#include <Alarme.hpp>
```

Public Member Functions

- [Alarme](#) (std::string Desc, int Id, [Parametro](#) &p)
Constrói um alarme vinculado a um parâmetro.
- bool [AtualizarAlarme](#) ()
Consulta o parâmetro associado e atualiza o estado do alarme.
- bool [GetEstado](#) () const
- void [SetEstado](#) (bool Estado)
Força o estado do alarme manualmente.
- void [SetDesc](#) (std::string desc)
Atualiza a descrição do alarme.

11.2.1 Detailed Description

[Alarme](#) associado a um [Parametro](#) de um equipamento.

Monitora um único parâmetro e mantém um estado booleano que indica se o valor atual viola os limites configurados naquele parâmetro.

Definition at line 14 of file [Alarme.hpp](#).

11.2.2 Constructor & Destructor Documentation

11.2.2.1 Alarme()

```
Alarme::Alarme (  
    std::string Desc,  
    int Id,  
    Parametro & p)
```

Constrói um alarme vinculado a um parâmetro.

Parameters

<i>Desc</i>	Descrição do alarme.
<i>Id</i>	Identificador único do alarme.
<i>p</i>	Parâmetro monitorado por este alarme.

Note

Implementa US-014 (Criar [Alarme](#)).

Definition at line 3 of file [Alarme.cpp](#).

11.2.3 Member Function Documentation

11.2.3.1 AtualizarAlarme()

```
bool Alarme::AtualizarAlarme ()
```

Consulta o parâmetro associado e atualiza o estado do alarme.

Returns

true se o alarme estiver ativo (parâmetro em falha).

Note

Implementa US-014 (Criar [Alarme](#)).

Definition at line 10 of file [Alarme.cpp](#).

11.2.3.2 GetEstado()

```
bool Alarme::GetEstado () const
```

Returns

true se o alarme estiver ativo.

Definition at line 16 of file [Alarme.cpp](#).

11.2.3.3 SetDesc()

```
void Alarme::SetDesc (  
    std::string desc)
```

Atualiza a descrição do alarme.

Parameters

<i>desc</i>	Nova descrição.
-------------	-----------------

Definition at line 24 of file [Alarme.cpp](#).

11.2.3.4 SetEstado()

```
void Alarme::SetEstado (
    bool Estado)
```

Força o estado do alarme manualmente.

Parameters

<i>Estado</i>	Novo estado.
---------------	--------------

Definition at line 20 of file [Alarme.cpp](#).

The documentation for this class was generated from the following files:

- [include/Alarme.hpp](#)
- [src/Alarme.cpp](#)

11.3 doctest::Approx Struct Reference

```
#include <doctest.h>
```

Public Member Functions

- [Approx](#) (double value)
- [Approx operator\(\)](#) (double value) const
- [Approx & epsilon](#) (double newEpsilon)
- [Approx & scale](#) (double newScale)

Public Attributes

- double [m_epsilon](#)
- double [m_scale](#)
- double [m_value](#)

Friends

- [DOCTEST_INTERFACE](#) friend bool [operator==](#) (double lhs, const [Approx](#) &rhs)
- [DOCTEST_INTERFACE](#) friend bool [operator==](#) (const [Approx](#) &lhs, double rhs)
- [DOCTEST_INTERFACE](#) friend bool [operator!=](#) (double lhs, const [Approx](#) &rhs)
- [DOCTEST_INTERFACE](#) friend bool [operator!=](#) (const [Approx](#) &lhs, double rhs)
- [DOCTEST_INTERFACE](#) friend bool [operator<=](#) (double lhs, const [Approx](#) &rhs)
- [DOCTEST_INTERFACE](#) friend bool [operator<=](#) (const [Approx](#) &lhs, double rhs)
- [DOCTEST_INTERFACE](#) friend bool [operator>=](#) (double lhs, const [Approx](#) &rhs)
- [DOCTEST_INTERFACE](#) friend bool [operator>=](#) (const [Approx](#) &lhs, double rhs)
- [DOCTEST_INTERFACE](#) friend bool [operator<](#) (double lhs, const [Approx](#) &rhs)
- [DOCTEST_INTERFACE](#) friend bool [operator<](#) (const [Approx](#) &lhs, double rhs)
- [DOCTEST_INTERFACE](#) friend bool [operator>](#) (double lhs, const [Approx](#) &rhs)
- [DOCTEST_INTERFACE](#) friend bool [operator>](#) (const [Approx](#) &lhs, double rhs)

11.3.1 Detailed Description

Definition at line 1337 of file [doctest.h](#).

11.3.2 Constructor & Destructor Documentation

11.3.2.1 Approx()

```
doctest::Approx::Approx (  
    double value)
```

11.3.3 Member Function Documentation

11.3.3.1 epsilon()

```
Approx & doctest::Approx::epsilon (  
    double newEpsilon)
```

11.3.3.2 operator>()

```
Approx doctest::Approx::operator() (  
    double value) const
```

11.3.3.3 scale()

```
Approx & doctest::Approx::scale (  
    double newScale)
```

11.3.4 Friends And Related Symbol Documentation

11.3.4.1 operator"!= [1/2]

```
DOCTEST_INTERFACE friend bool operator!= (  
    const Approx & lhs,  
    double rhs) [friend]
```

11.3.4.2 operator"!= [2/2]

```
DOCTEST_INTERFACE friend bool operator!= (  
    double lhs,  
    const Approx & rhs) [friend]
```

11.3.4.3 operator< [1/2]

```
DOCTEST_INTERFACE friend bool operator< (  
    const Approx & lhs,  
    double rhs) [friend]
```

11.3.4.4 operator< [2/2]

```
DOCTEST_INTERFACE friend bool operator< (  
    double lhs,  
    const Approx & rhs) [friend]
```

11.3.4.5 operator<= [1/2]

```
DOCTEST_INTERFACE friend bool operator<= (  
    const Approx & lhs,  
    double rhs) [friend]
```

11.3.4.6 operator<= [2/2]

```
DOCTEST_INTERFACE friend bool operator<= (  
    double lhs,  
    const Approx & rhs) [friend]
```

11.3.4.7 operator== [1/2]

```
DOCTEST_INTERFACE friend bool operator== (  
    const Approx & lhs,  
    double rhs) [friend]
```

11.3.4.8 operator== [2/2]

```
DOCTEST_INTERFACE friend bool operator== (  
    double lhs,  
    const Approx & rhs) [friend]
```

11.3.4.9 operator> [1/2]

```
DOCTEST_INTERFACE friend bool operator> (  
    const Approx & lhs,  
    double rhs) [friend]
```

11.3.4.10 operator> [2/2]

```
DOCTEST_INTERFACE friend bool operator> (  
    double lhs,  
    const Approx & rhs) [friend]
```

11.3.4.11 operator>= [1/2]

```
DOCTEST_INTERFACE friend bool operator>= (
    const Approx & lhs,
    double rhs) [friend]
```

11.3.4.12 operator>= [2/2]

```
DOCTEST_INTERFACE friend bool operator>= (
    double lhs,
    const Approx & rhs) [friend]
```

11.3.5 Member Data Documentation**11.3.5.1 m_epsilon**

```
double doctest::Approx::m_epsilon
```

Definition at line 1409 of file [doctest.h](#).

11.3.5.2 m_scale

```
double doctest::Approx::m_scale
```

Definition at line 1410 of file [doctest.h](#).

11.3.5.3 m_value

```
double doctest::Approx::m_value
```

Definition at line 1411 of file [doctest.h](#).

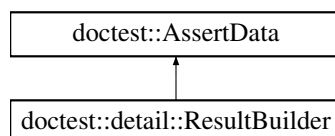
The documentation for this struct was generated from the following file:

- [tests/doctest.h](#)

11.4 doctest::AssertData Struct Reference

```
#include <doctest.h>
```

Inheritance diagram for doctest::AssertData:



Classes

- class [StringContains](#)

Public Member Functions

- [AssertData](#) ([assertType::Enum](#) at, const char *file, int line, const char *expr, const char *exception_type, const [StringContains](#) &exception_string)

Public Attributes

- const [TestCaseData](#) * [m_test_case](#)
- [assertType::Enum](#) [m_at](#)
- const char * [m_file](#)
- int [m_line](#)
- const char * [m_expr](#)
- bool [m_failed](#)
- bool [m_threw](#)
- [String](#) [m_exception](#)
- [String](#) [m_decomp](#)
- bool [m_threw_as](#)
- const char * [m_exception_type](#)
- class [DOCTEST_INTERFACE](#) [doctest::AssertData::StringContains](#) [m_exception_string](#)

11.4.1 Detailed Description

Definition at line 1639 of file [doctest.h](#).

11.4.2 Constructor & Destructor Documentation

11.4.2.1 AssertData()

```
doctest::AssertData::AssertData (  
    assertType::Enum at,  
    const char * file,  
    int line,  
    const char * expr,  
    const char * exception_type,  
    const StringContains & exception_string)
```

11.4.3 Member Data Documentation

11.4.3.1 m_at

[assertType::Enum](#) [doctest::AssertData::m_at](#)

Definition at line 1642 of file [doctest.h](#).

11.4.3.2 m_decomp

`String doctest::AssertData::m_decomp`

Definition at line 1653 of file [doctest.h](#).

11.4.3.3 m_exception

`String doctest::AssertData::m_exception`

Definition at line 1650 of file [doctest.h](#).

11.4.3.4 m_exception_string

```
class DOCTEST_INTERFACE doctest::AssertData::StringContains doctest::AssertData::m_exception←  
_string
```

11.4.3.5 m_exception_type

`const char* doctest::AssertData::m_exception_type`

Definition at line 1657 of file [doctest.h](#).

11.4.3.6 m_expr

`const char* doctest::AssertData::m_expr`

Definition at line 1645 of file [doctest.h](#).

11.4.3.7 m_failed

`bool doctest::AssertData::m_failed`

Definition at line 1646 of file [doctest.h](#).

11.4.3.8 m_file

`const char* doctest::AssertData::m_file`

Definition at line 1643 of file [doctest.h](#).

11.4.3.9 m_line

`int doctest::AssertData::m_line`

Definition at line 1644 of file [doctest.h](#).

11.4.3.10 m_test_case

```
const TestCaseData* doctest::AssertData::m_test_case
```

Definition at line 1641 of file [doctest.h](#).

11.4.3.11 m_threw

```
bool doctest::AssertData::m_threw
```

Definition at line 1649 of file [doctest.h](#).

11.4.3.12 m_threw_as

```
bool doctest::AssertData::m_threw_as
```

Definition at line 1656 of file [doctest.h](#).

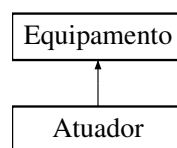
The documentation for this struct was generated from the following file:

- [tests/doctest.h](#)

11.5 Atuador Class Reference

```
#include <Atuador.hpp>
```

Inheritance diagram for Atuador:



Public Member Functions

- [Atuador](#) (int id, std::string desc, double retorno)
- [~Atuador](#) () override=default
- std::string [Tipo](#) () const override

Tipo concreto do equipamento (sobrescrito pelas especializações).

Public Member Functions inherited from [Equipamento](#)

- [Equipamento](#) (int id, std::string desc, std::string nome)
Constrói um equipamento.
- virtual [~Equipamento](#) ()=default
Destrutor virtual: permite uso polimórfico via ponteiro de base.
- void [AdicionarParametro](#) (double ValorAtual, std::string Desc, int id)
Adiciona um parâmetro sem limites definidos.
- int [AdicionarParametro](#) (double ValorAtual, std::string Desc)
Adiciona um parâmetro sem limites com ID gerado automaticamente.
- void [AdicionarParametro](#) (double ValorAtual, double Max, double Min, std::string Desc, int id)
Adiciona um parâmetro com limites Min/Max definidos.
- int [AdicionarParametro](#) (double ValorAtual, double Max, double Min, std::string Desc)
Adiciona um parâmetro com limites Min/Max e ID gerado automaticamente.
- int [ProximoldParametro](#) () const
- void [RemoverParametro](#) (int id)
Remove o parâmetro com o ID fornecido.
- std::vector< int > [DiagnosticarEquipamento](#) () const
Diagnostica todos os parâmetros do equipamento.
- void [AtualizarAlarmes](#) ()
Atualiza o estado de todos os alarmes do equipamento.
- void [ExibirParametros](#) () const
Exibe no console os parâmetros e seus valores atuais.
- void [ExibirAlarmes](#) () const
Exibe no console os alarmes ativos do equipamento.
- int [QuantidadeParametros](#) () const
- int [QuantidadeAlarmes](#) () const
- int [GetId](#) () const
- std::string [GetDesc](#) () const
- std::string [GetNome](#) () const
- const std::map< int, [Parametro](#) > & [GetParametros](#) () const
- const std::vector< [Alarme](#) > & [GetAlarmes](#) () const
- void [SetDesc](#) (std::string desc)
Atualiza a descrição do equipamento.

11.5.1 Detailed Description

Definition at line 6 of file [Atuador.hpp](#).

11.5.2 Constructor & Destructor Documentation

11.5.2.1 Atuador()

```
Atuador::Atuador (
    int id,
    std::string desc,
    double retorno)
```

Definition at line 3 of file [Atuador.cpp](#).

11.5.2.2 ~Atuador()

```
Atuador::~~Atuador () [override], [default]
```

11.5.3 Member Function Documentation

11.5.3.1 Tipo()

```
std::string Atuador::Tipo () const [override], [virtual]
```

Tipo concreto do equipamento (sobrescrito pelas especializações).

Returns

Nome do tipo (ex.: "Equipamento", "Motor", "Sensor").

Reimplemented from [Equipamento](#).

Definition at line 8 of file [Atuador.cpp](#).

The documentation for this class was generated from the following files:

- [include/Atuador.hpp](#)
- [src/Atuador.cpp](#)

11.6 std::basic_istream< charT, traits > Class Template Reference

11.6.1 Detailed Description

```
template<class charT, class traits>  
class std::basic_istream< charT, traits >
```

Definition at line 683 of file [doctest.h](#).

The documentation for this class was generated from the following file:

- [tests/doctest.h](#)

11.7 std::basic_ostream< charT, traits > Class Template Reference

11.7.1 Detailed Description

```
template<class charT, class traits>  
class std::basic_ostream< charT, traits >
```

Definition at line 677 of file [doctest.h](#).

The documentation for this class was generated from the following file:

- [tests/doctest.h](#)

11.8 std::char_traits< charT > Struct Template Reference

11.8.1 Detailed Description

```
template<class charT>
struct std::char_traits< charT >
```

Definition at line 673 of file [doctest.h](#).

The documentation for this struct was generated from the following file:

- [tests/doctest.h](#)

11.9 Conjunto Class Reference

Agrupamento lógico de equipamentos dentro de uma [LinhaDeProducao](#).

```
#include <Conjunto.hpp>
```

Public Member Functions

- [Conjunto](#) (int id, std::string nome, std::string desc)
Constrói um conjunto.
- void [AdicionarEquipamento](#) (int id, std::string desc)
Adiciona um equipamento ao conjunto.
- void [AdicionarEquipamento](#) (std::unique_ptr< [Equipamento](#) > equip)
Adiciona um equipamento já construído (permite tipos derivados).
- int [ProximoldEquipamento](#) () const
- void [RemoverEquipamento](#) (int id)
Remove o equipamento com o ID fornecido.
- void [SetDesc](#) (std::string desc)
Atualiza a descrição do conjunto.
- void [SetNome](#) (std::string nome)
Atualiza o nome do conjunto.
- void [ExibirEquipamentos](#) () const
Exibe no console os equipamentos do conjunto.
- void [ExibirAlarmes](#) () const
Exibe no console os alarmes ativos de todos os equipamentos.
- void [ExibirTudo](#) () const
Exibe equipamentos e alarmes de forma consolidada.
- int [QuantidadeEquipamentos](#) () const
- int [QuantidadeAlarmes](#) () const
- std::vector< int > [DiagnosticarConjunto](#) () const
Diagnostica todos os equipamentos do conjunto.
- void [AtualizarAlarmes](#) ()
Propaga a atualização de alarmes para todos os equipamentos.
- const std::map< int, std::unique_ptr< [Equipamento](#) > > & [GetEquipamentos](#) () const
- [Equipamento](#) & [AcessarEquipamento](#) (int id)
Acesso mutável a um equipamento pelo ID.
- int [GetId](#) () const
- std::string [GetDesc](#) () const
- std::string [GetNome](#) () const

11.9.1 Detailed Description

Agrupamento lógico de equipamentos dentro de uma [LinhaDeProducao](#).

Um conjunto reúne equipamentos correlacionados e propaga operações de diagnóstico e atualização de alarmes para todos eles.

Definition at line 16 of file [Conjunto.hpp](#).

11.9.2 Constructor & Destructor Documentation

11.9.2.1 Conjunto()

```
Conjunto::Conjunto (
    int id,
    std::string nome,
    std::string desc)
```

Constrói um conjunto.

Parameters

<i>id</i>	Identificador único.
<i>desc</i>	Descrição do conjunto.

Definition at line 7 of file [Conjunto.cpp](#).

11.9.3 Member Function Documentation

11.9.3.1 AcessarEquipamento()

```
Equipamento & Conjunto::AcessarEquipamento (
    int id)
```

Acesso mutável a um equipamento pelo ID.

Parameters

<i>id</i>	Identificador do equipamento.
-----------	-------------------------------

Returns

Referência ao equipamento.

Exceptions

<i>IdInexistente</i>	se não houver equipamento com o ID.
----------------------	-------------------------------------

Definition at line 128 of file [Conjunto.cpp](#).

11.9.3.2 AdicionarEquipamento() [1/2]

```
void Conjunto::AdicionarEquipamento (
    int id,
    std::string desc)
```

Adiciona um equipamento ao conjunto.

Parameters

<i>id</i>	Identificador único do equipamento.
<i>desc</i>	Descrição do equipamento.

Note

Implementa US-007 (Criar Equipamentos).

Definition at line 19 of file [Conjunto.cpp](#).

11.9.3.3 AdicionarEquipamento() [2/2]

```
void Conjunto::AdicionarEquipamento (
    std::unique_ptr< Equipamento > equip)
```

Adiciona um equipamento já construído (permite tipos derivados).

Parameters

<i>equip</i>	Equipamento (posse transferida ao conjunto via <code>unique_ptr</code>).
--------------	---

Note

Suporta polimorfismo: [Motor](#), [Sensor](#), [Valvula](#), [Atuador](#).

Definition at line 29 of file [Conjunto.cpp](#).

11.9.3.4 AtualizarAlarmes()

```
void Conjunto::AtualizarAlarmes ()
```

Propaga a atualização de alarmes para todos os equipamentos.

Note

Implementa US-014 (Criar [Alarme](#)).

Definition at line 118 of file [Conjunto.cpp](#).

11.9.3.5 DiagnosticarConjunto()

```
std::vector< int > Conjunto::DiagnosticarConjunto () const
```

Diagnostica todos os equipamentos do conjunto.

Returns

Vetor com IDs dos parâmetros em estado de falha.

Note

Implementa US-012 (Diagnóstico de [Conjunto](#)).

Definition at line 109 of file [Conjunto.cpp](#).

11.9.3.6 ExibirAlarmes()

```
void Conjunto::ExibirAlarmes () const
```

Exibe no console os alarmes ativos de todos os equipamentos.

Definition at line 80 of file [Conjunto.cpp](#).

11.9.3.7 ExibirEquipamentos()

```
void Conjunto::ExibirEquipamentos () const
```

Exibe no console os equipamentos do conjunto.

Note

Implementa US-006 (Acessar [Conjunto](#)).

Definition at line 65 of file [Conjunto.cpp](#).

11.9.3.8 ExibirTudo()

```
void Conjunto::ExibirTudo () const
```

Exibe equipamentos e alarmes de forma consolidada.

Note

Implementa US-006 (Acessar [Conjunto](#)).

Definition at line 92 of file [Conjunto.cpp](#).

11.9.3.9 GetDesc()

```
std::string Conjunto::GetDesc () const
```

Returns

Descrição do conjunto.

Definition at line 138 of file [Conjunto.cpp](#).

11.9.3.10 GetEquipamentos()

```
const std::map< int, std::unique_ptr< Equipamento > > & Conjunto::GetEquipamentos () const
```

Returns

Referência constante ao mapa de equipamentos (posse via unique_ptr).

Definition at line 124 of file [Conjunto.cpp](#).

11.9.3.11 GetId()

```
int Conjunto::GetId () const
```

Returns

Identificador único do conjunto.

Definition at line 136 of file [Conjunto.cpp](#).

11.9.3.12 GetNome()

```
std::string Conjunto::GetNome () const
```

Returns

Nome do conjunto.

Definition at line 140 of file [Conjunto.cpp](#).

11.9.3.13 ProximoIdEquipamento()

```
int Conjunto::ProximoIdEquipamento () const
```

Returns

Próximo ID disponível para equipamento (monotônico).

Definition at line 43 of file [Conjunto.cpp](#).

11.9.3.14 QuantidadeAlarmes()

```
int Conjunto::QuantidadeAlarmes () const
```

Returns

Número total de alarmes ativos no conjunto.

Definition at line 101 of file [Conjunto.cpp](#).

11.9.3.15 QuantidadeEquipamentos()

```
int Conjunto::QuantidadeEquipamentos () const
```

Returns

Número de equipamentos cadastrados no conjunto.

Definition at line 97 of file [Conjunto.cpp](#).

11.9.3.16 RemoverEquipamento()

```
void Conjunto::RemoverEquipamento (
    int id)
```

Remove o equipamento com o ID fornecido.

Parameters

<i>id</i>	Identificador do equipamento a remover.
-----------	---

Note

Implementa US-008 (Remover [Equipamento](#)).

Definition at line 47 of file [Conjunto.cpp](#).

11.9.3.17 SetDesc()

```
void Conjunto::SetDesc (
    std::string desc)
```

Atualiza a descrição do conjunto.

Parameters

<i>desc</i>	Nova descrição.
-------------	-----------------

Definition at line 54 of file [Conjunto.cpp](#).

11.9.3.18 SetNome()

```
void Conjunto::SetNome (
    std::string nome)
```

Atualiza o nome do conjunto.

Parameters

<i>nome</i>	Novo nome.
-------------	------------

Definition at line 61 of file [Conjunto.cpp](#).

The documentation for this class was generated from the following files:

- [include/Conjunto.hpp](#)
- [src/Conjunto.cpp](#)

11.10 doctest::Contains Class Reference

```
#include <doctest.h>
```

Public Member Functions

- [Contains](#) (const [String](#) &string)
- bool [checkWith](#) (const [String](#) &other) const

Public Attributes

- [String](#) string

11.10.1 Detailed Description

Definition at line 1308 of file [doctest.h](#).

11.10.2 Constructor & Destructor Documentation

11.10.2.1 Contains()

```
doctest::Contains::Contains (
    const String & string) [explicit]
```

11.10.3 Member Function Documentation

11.10.3.1 checkWith()

```
bool doctest::Contains::checkWith (
    const String & other) const
```

11.10.4 Member Data Documentation

11.10.4.1 string

[String](#) doctest::Contains::string

Definition at line [1314](#) of file [doctest.h](#).

The documentation for this class was generated from the following file:

- [tests/doctest.h](#)

11.11 doctest::Context Class Reference

```
#include <doctest.h>
```

Public Member Functions

- [Context](#) (int argc=0, const char *const *argv=nullptr)
- [Context](#) (const Context &)=delete
- [Context](#) (Context &&)=delete
- [Context](#) & operator= (const [Context](#) &)=delete
- [Context](#) & operator= ([Context](#) &&)=delete
- [~Context](#) ()
- void [applyCommandLine](#) (int argc, const char *const *argv)
- void [addFilter](#) (const char *filter, const char *value)
- void [clearFilters](#) ()
- void [setOption](#) (const char *option, bool value)
- void [setOption](#) (const char *option, int value)
- void [setOption](#) (const char *option, const char *value)
- bool [shouldExit](#) ()
- void [setAsDefaultForAssertsOutOfTestCases](#) ()
- void [setAssertHandler](#) ([detail::assert_handler](#) ah)
- void [setCout](#) ([std::ostream](#) *out)
- int [run](#) ()

11.11.1 Detailed Description

Definition at line 2595 of file [doctest.h](#).

11.11.2 Constructor & Destructor Documentation

11.11.2.1 Context() [1/3]

```
doctest::Context::Context (
    int argc = 0,
    const char *const * argv = nullptr) [explicit]
```

11.11.2.2 Context() [2/3]

```
doctest::Context::Context (
    const Context & ) [delete]
```

11.11.2.3 Context() [3/3]

```
doctest::Context::Context (
    Context && ) [delete]
```

11.11.2.4 ~Context()

```
doctest::Context::~Context ()
```

11.11.3 Member Function Documentation

11.11.3.1 addFilter()

```
void doctest::Context::addFilter (
    const char * filter,
    const char * value)
```

11.11.3.2 applyCommandLine()

```
void doctest::Context::applyCommandLine (
    int argc,
    const char *const * argv)
```

11.11.3.3 clearFilters()

```
void doctest::Context::clearFilters ()
```

11.11.3.4 operator=() [1/2]

```
Context & doctest::Context::operator= (
    const Context & ) [delete]
```

11.11.3.5 operator=() [2/2]

```
Context & doctest::Context::operator= (
    Context && ) [delete]
```

11.11.3.6 run()

```
int doctest::Context::run ()
```

11.11.3.7 setAsDefaultForAssertsOutOfTestCases()

```
void doctest::Context::setAsDefaultForAssertsOutOfTestCases ()
```

11.11.3.8 setAssertHandler()

```
void doctest::Context::setAssertHandler (
    detail::assert_handler ah)
```

11.11.3.9 setCout()

```
void doctest::Context::setCout (
    std::ostream * out)
```

11.11.3.10 setOption() [1/3]

```
void doctest::Context::setOption (
    const char * option,
    bool value)
```

11.11.3.11 setOption() [2/3]

```
void doctest::Context::setOption (
    const char * option,
    const char * value)
```

11.11.3.12 setOption() [3/3]

```
void doctest::Context::setOption (
    const char * option,
    int value)
```

11.11.3.13 shouldExit()

```
bool doctest::Context::shouldExit ()
```

The documentation for this class was generated from the following file:

- tests/[doctest.h](#)

11.12 doctest::ContextOptions Struct Reference

```
#include <doctest.h>
```

Public Attributes

- [std::ostream](#) * [cout](#) = nullptr
- [String](#) [binary_name](#)
- const [detail::TestCase](#) * [currentTest](#) = nullptr
- [String](#) [out](#)
- [String](#) [order_by](#)
- unsigned [rand_seed](#)
- unsigned [first](#)
- unsigned [last](#)
- int [abort_after](#)
- int [subcase_filter_levels](#)
- bool [success](#)
- bool [case_sensitive](#)
- bool [exit](#)
- bool [duration](#)
- bool [minimal](#)
- bool [quiet](#)
- bool [no_throw](#)
- bool [no_exitcode](#)
- bool [no_run](#)
- bool [no_intro](#)
- bool [no_version](#)
- bool [no_colors](#)
- bool [force_colors](#)
- bool [no_breaks](#)
- bool [no_skip](#)
- bool [gnu_file_line](#)
- bool [no_path_in_filenames](#)
- [String](#) [strip_file_prefixes](#)
- bool [no_line_numbers](#)
- bool [no_debug_output](#)
- bool [no_skipped_summary](#)
- bool [no_time_in_output](#)
- bool [help](#)
- bool [version](#)
- bool [count](#)
- bool [list_test_cases](#)
- bool [list_test_suites](#)
- bool [list_reporters](#)

11.12.1 Detailed Description

Definition at line 1469 of file [doctest.h](#).

11.12.2 Member Data Documentation

11.12.2.1 abort_after

```
int doctest::ContextOptions::abort_after
```

Definition at line 1483 of file [doctest.h](#).

11.12.2.2 binary_name

```
String doctest::ContextOptions::binary_name
```

Definition at line 1471 of file [doctest.h](#).

11.12.2.3 case_sensitive

```
bool doctest::ContextOptions::case_sensitive
```

Definition at line 1487 of file [doctest.h](#).

11.12.2.4 count

```
bool doctest::ContextOptions::count
```

Definition at line 1511 of file [doctest.h](#).

11.12.2.5 cout

```
std::ostream* doctest::ContextOptions::cout = nullptr
```

Definition at line 1470 of file [doctest.h](#).

11.12.2.6 currentTest

```
const detail::TestCase* doctest::ContextOptions::currentTest = nullptr
```

Definition at line 1473 of file [doctest.h](#).

11.12.2.7 duration

```
bool doctest::ContextOptions::duration
```

Definition at line 1489 of file [doctest.h](#).

11.12.2.8 exit

```
bool doctest::ContextOptions::exit
```

Definition at line 1488 of file [doctest.h](#).

11.12.2.9 first

```
unsigned doctest::ContextOptions::first
```

Definition at line 1480 of file [doctest.h](#).

11.12.2.10 force_colors

```
bool doctest::ContextOptions::force_colors
```

Definition at line 1498 of file [doctest.h](#).

11.12.2.11 gnu_file_line

```
bool doctest::ContextOptions::gnu_file_line
```

Definition at line 1501 of file [doctest.h](#).

11.12.2.12 help

```
bool doctest::ContextOptions::help
```

Definition at line 1509 of file [doctest.h](#).

11.12.2.13 last

```
unsigned doctest::ContextOptions::last
```

Definition at line 1481 of file [doctest.h](#).

11.12.2.14 list_reporters

```
bool doctest::ContextOptions::list_reporters
```

Definition at line 1514 of file [doctest.h](#).

11.12.2.15 list_test_cases

```
bool doctest::ContextOptions::list_test_cases
```

Definition at line [1512](#) of file [doctest.h](#).

11.12.2.16 list_test_suites

```
bool doctest::ContextOptions::list_test_suites
```

Definition at line [1513](#) of file [doctest.h](#).

11.12.2.17 minimal

```
bool doctest::ContextOptions::minimal
```

Definition at line [1490](#) of file [doctest.h](#).

11.12.2.18 no_breaks

```
bool doctest::ContextOptions::no_breaks
```

Definition at line [1499](#) of file [doctest.h](#).

11.12.2.19 no_colors

```
bool doctest::ContextOptions::no_colors
```

Definition at line [1497](#) of file [doctest.h](#).

11.12.2.20 no_debug_output

```
bool doctest::ContextOptions::no_debug_output
```

Definition at line [1505](#) of file [doctest.h](#).

11.12.2.21 no_exitcode

```
bool doctest::ContextOptions::no_exitcode
```

Definition at line [1493](#) of file [doctest.h](#).

11.12.2.22 no_intro

```
bool doctest::ContextOptions::no_intro
```

Definition at line [1495](#) of file [doctest.h](#).

11.12.2.23 no_line_numbers

```
bool doctest::ContextOptions::no_line_numbers
```

Definition at line 1504 of file [doctest.h](#).

11.12.2.24 no_path_in_filenames

```
bool doctest::ContextOptions::no_path_in_filenames
```

Definition at line 1502 of file [doctest.h](#).

11.12.2.25 no_run

```
bool doctest::ContextOptions::no_run
```

Definition at line 1494 of file [doctest.h](#).

11.12.2.26 no_skip

```
bool doctest::ContextOptions::no_skip
```

Definition at line 1500 of file [doctest.h](#).

11.12.2.27 no_skipped_summary

```
bool doctest::ContextOptions::no_skipped_summary
```

Definition at line 1506 of file [doctest.h](#).

11.12.2.28 no_throw

```
bool doctest::ContextOptions::no_throw
```

Definition at line 1492 of file [doctest.h](#).

11.12.2.29 no_time_in_output

```
bool doctest::ContextOptions::no_time_in_output
```

Definition at line 1507 of file [doctest.h](#).

11.12.2.30 no_version

```
bool doctest::ContextOptions::no_version
```

Definition at line 1496 of file [doctest.h](#).

11.12.2.31 order_by

`String doctest::ContextOptions::order_by`

Definition at line 1477 of file [doctest.h](#).

11.12.2.32 out

`String doctest::ContextOptions::out`

Definition at line 1476 of file [doctest.h](#).

11.12.2.33 quiet

`bool doctest::ContextOptions::quiet`

Definition at line 1491 of file [doctest.h](#).

11.12.2.34 rand_seed

`unsigned doctest::ContextOptions::rand_seed`

Definition at line 1478 of file [doctest.h](#).

11.12.2.35 strip_file_prefixes

`String doctest::ContextOptions::strip_file_prefixes`

Definition at line 1503 of file [doctest.h](#).

11.12.2.36 subcase_filter_levels

`int doctest::ContextOptions::subcase_filter_levels`

Definition at line 1484 of file [doctest.h](#).

11.12.2.37 success

`bool doctest::ContextOptions::success`

Definition at line 1486 of file [doctest.h](#).

11.12.2.38 version

```
bool doctest::ContextOptions::version
```

Definition at line 1510 of file [doctest.h](#).

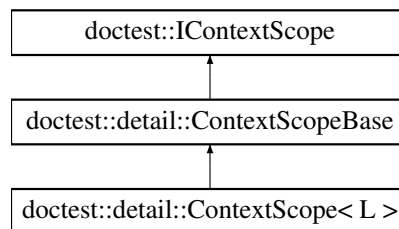
The documentation for this struct was generated from the following file:

- [tests/doctest.h](#)

11.13 doctest::detail::ContextScope< L > Class Template Reference

```
#include <doctest.h>
```

Inheritance diagram for doctest::detail::ContextScope< L >:



Public Member Functions

- [ContextScope](#) (const L &lambda)
- [ContextScope](#) (L &&lambda)
- [ContextScope](#) (const ContextScope &)=delete
- [ContextScope](#) (ContextScope &&) noexcept=default
- [ContextScope](#) & [operator=](#) (const [ContextScope](#) &)=delete
- [ContextScope](#) & [operator=](#) ([ContextScope](#) &&)=delete
- void [stringify](#) ([std::ostream](#) *s) const override
- [~ContextScope](#) () override

Public Member Functions inherited from [doctest::detail::ContextScopeBase](#)

- [ContextScopeBase](#) (const ContextScopeBase &)=delete
- [ContextScopeBase](#) & [operator=](#) (const [ContextScopeBase](#) &)=delete
- [ContextScopeBase](#) & [operator=](#) ([ContextScopeBase](#) &&)=delete
- [~ContextScopeBase](#) () override=default

Additional Inherited Members

Protected Member Functions inherited from [doctest::detail::ContextScopeBase](#)

- [ContextScopeBase](#) ()
- [ContextScopeBase](#) (ContextScopeBase &&other) noexcept
- void [destroy](#) ()

Protected Attributes inherited from [doctest::detail::ContextScopeBase](#)

- bool [need_to_destroy](#) {true}

11.13.1 Detailed Description

```
template<typename L>
class doctest::detail::ContextScope< L >
```

Definition at line 2428 of file [doctest.h](#).

11.13.2 Constructor & Destructor Documentation

11.13.2.1 ContextScope() [1/4]

```
template<typename L>
doctest::detail::ContextScope< L >::ContextScope (
    const L & lambda) [inline], [explicit]
```

Definition at line 2432 of file [doctest.h](#).

11.13.2.2 ContextScope() [2/4]

```
template<typename L>
doctest::detail::ContextScope< L >::ContextScope (
    L && lambda) [inline], [explicit]
```

Definition at line 2436 of file [doctest.h](#).

11.13.2.3 ContextScope() [3/4]

```
template<typename L>
doctest::detail::ContextScope< L >::ContextScope (
    const ContextScope< L > & ) [delete]
```

11.13.2.4 ContextScope() [4/4]

```
template<typename L>
doctest::detail::ContextScope< L >::ContextScope (
    ContextScope< L > && ) [default], [noexcept]
```

11.13.2.5 ~ContextScope()

```
template<typename L>
doctest::detail::ContextScope< L >::~~ContextScope () [inline], [override]
```

Definition at line 2449 of file [doctest.h](#).

11.13.3 Member Function Documentation

11.13.3.1 operator=() [1/2]

```
template<typename L>
ContextScope & doctest::detail::ContextScope< L >::operator= (
    const ContextScope< L > & ) [delete]
```

11.13.3.2 operator=() [2/2]

```
template<typename L>
ContextScope & doctest::detail::ContextScope< L >::operator= (
    ContextScope< L > && ) [delete]
```

11.13.3.3 stringify()

```
template<typename L>
void doctest::detail::ContextScope< L >::stringify (
    std::ostream * s) const [inline], [override], [virtual]
```

Implements [doctest::IContextScope](#).

Definition at line 2445 of file [doctest.h](#).

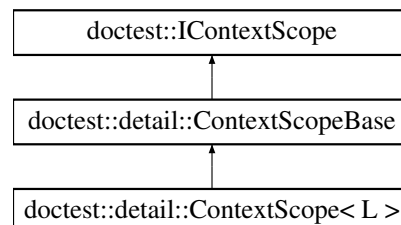
The documentation for this class was generated from the following file:

- [tests/doctest.h](#)

11.14 doctest::detail::ContextScopeBase Struct Reference

```
#include <doctest.h>
```

Inheritance diagram for doctest::detail::ContextScopeBase:



Public Member Functions

- [ContextScopeBase](#) (const ContextScopeBase &)=delete
- [ContextScopeBase](#) & operator= (const [ContextScopeBase](#) &)=delete
- [ContextScopeBase](#) & operator= ([ContextScopeBase](#) &&)=delete
- [~ContextScopeBase](#) () override=default

Public Member Functions inherited from [doctest::IContextScope](#)

- virtual void [stringify](#) ([std::ostream](#) *) const =0

Protected Member Functions

- [ContextScopeBase](#) ()
- [ContextScopeBase](#) ([ContextScopeBase](#) &&*other*) noexcept
- void [destroy](#) ()

Protected Attributes

- bool [need_to_destroy](#) {true}

11.14.1 Detailed Description

Definition at line [2411](#) of file [doctest.h](#).

11.14.2 Constructor & Destructor Documentation

11.14.2.1 ContextScopeBase() [1/3]

```
doctest::detail::ContextScopeBase::ContextScopeBase (
    const ContextScopeBase & )    [delete]
```

11.14.2.2 ~ContextScopeBase()

```
doctest::detail::ContextScopeBase::~~ContextScopeBase ()    [override], [default]
```

11.14.2.3 ContextScopeBase() [2/3]

```
doctest::detail::ContextScopeBase::ContextScopeBase ()    [protected]
```

11.14.2.4 ContextScopeBase() [3/3]

```
doctest::detail::ContextScopeBase::ContextScopeBase (
    ContextScopeBase && other)    [protected], [noexcept]
```

11.14.3 Member Function Documentation

11.14.3.1 destroy()

```
void doctest::detail::ContextScopeBase::destroy ()    [protected]
```


11.14.3.2 operator=() [1/2]

```
ContextScopeBase & doctest::detail::ContextScopeBase::operator= (
    const ContextScopeBase & ) [delete]
```

11.14.3.3 operator=() [2/2]

```
ContextScopeBase & doctest::detail::ContextScopeBase::operator= (
    ContextScopeBase && ) [delete]
```

11.14.4 Member Data Documentation

11.14.4.1 need_to_destroy

```
bool doctest::detail::ContextScopeBase::need_to_destroy {true} [protected]
```

Definition at line 2424 of file [doctest.h](#).

The documentation for this struct was generated from the following file:

- [tests/doctest.h](#)

11.15 doctest::CurrentTestCaseStats Struct Reference

```
#include <doctest.h>
```

Public Attributes

- int [numAssertsCurrentTest](#)
- int [numAssertsFailedCurrentTest](#)
- double [seconds](#)
- int [failure_flags](#)
- bool [testCaseSuccess](#)

11.15.1 Detailed Description

Definition at line 2746 of file [doctest.h](#).

11.15.2 Member Data Documentation

11.15.2.1 failure_flags

```
int doctest::CurrentTestCaseStats::failure_flags
```

Definition at line 2750 of file [doctest.h](#).

11.15.2.2 numAssertsCurrentTest

```
int doctest::CurrentTestCaseStats::numAssertsCurrentTest
```

Definition at line 2747 of file [doctest.h](#).

11.15.2.3 numAssertsFailedCurrentTest

```
int doctest::CurrentTestCaseStats::numAssertsFailedCurrentTest
```

Definition at line 2748 of file [doctest.h](#).

11.15.2.4 seconds

```
double doctest::CurrentTestCaseStats::seconds
```

Definition at line 2749 of file [doctest.h](#).

11.15.2.5 testCaseSuccess

```
bool doctest::CurrentTestCaseStats::testCaseSuccess
```

Definition at line 2751 of file [doctest.h](#).

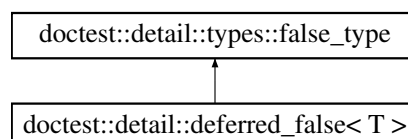
The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

11.16 doctest::detail::deferred_false< T > Struct Template Reference

```
#include <doctest.h>
```

Inheritance diagram for doctest::detail::deferred_false< T >:



Additional Inherited Members

Static Public Attributes inherited from [doctest::detail::types::false_type](#)

- static [DOCTEST_CONSTEXPR](#) bool [value](#) = false

11.16.1 Detailed Description

```
template<typename T>
struct doctest::detail::deferred_false< T >
```

Definition at line 842 of file [doctest.h](#).

The documentation for this struct was generated from the following file:

- [tests/doctest.h](#)

11.17 Dependencia Class Reference

```
#include <Dependencia.hpp>
```

Public Member Functions

- [Dependencia](#) (int id, [EstadoOperacional](#) condicao, [EstadoOperacional](#) resultado)
- int [Get_Id](#) ()
- [EstadoOperacional](#) [Get_Condicao](#) ()
- [EstadoOperacional](#) [Get_Resultado](#) ()

11.17.1 Detailed Description

Definition at line 6 of file [Dependencia.hpp](#).

11.17.2 Constructor & Destructor Documentation

11.17.2.1 Dependencia()

```
Dependencia::Dependencia (
    int id,
    EstadoOperacional condicao,
    EstadoOperacional resultado)
```

Definition at line 3 of file [Dependencia.cpp](#).

11.17.3 Member Function Documentation

11.17.3.1 Get_Condicao()

```
EstadoOperacional Dependencia::Get_Condicao ()
```

Definition at line 13 of file [Dependencia.cpp](#).

11.17.3.2 Get_Id()

```
int Dependencia::Get_Id ()
```

Definition at line 17 of file [Dependencia.cpp](#).

11.17.3.3 Get_Resultado()

```
EstadoOperacional Dependencia::Get_Resultado ()
```

Definition at line 9 of file [Dependencia.cpp](#).

The documentation for this class was generated from the following files:

- include/[Dependencia.hpp](#)
- src/[Dependencia.cpp](#)

11.18 doctest::detail::types::enable_if< COND, T > Struct Template Reference

```
#include <doctest.h>
```

11.18.1 Detailed Description

```
template<bool COND, typename T = void>
struct doctest::detail::types::enable_if< COND, T >
```

Definition at line 729 of file [doctest.h](#).

The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

11.19 doctest::detail::types::enable_if< true, T > Struct Template Reference

```
#include <doctest.h>
```

Public Types

- using [type](#) = T

11.19.1 Detailed Description

```
template<typename T>
struct doctest::detail::types::enable_if< true, T >
```

Definition at line 732 of file [doctest.h](#).

11.19.2 Member Typedef Documentation

11.19.2.1 type

```
template<typename T>
using doctest::detail::types::enable_if< true, T >::type = T
```

Definition at line 733 of file [doctest.h](#).

The documentation for this struct was generated from the following file:

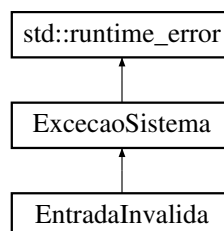
- [tests/doctest.h](#)

11.20 EntradaInvalida Class Reference

Entrada inválida (string vazia, argumento sem sentido).

```
#include <Excecoes.hpp>
```

Inheritance diagram for EntradaInvalida:



11.20.1 Detailed Description

Entrada inválida (string vazia, argumento sem sentido).

Definition at line 19 of file [Excecoes.hpp](#).

The documentation for this class was generated from the following file:

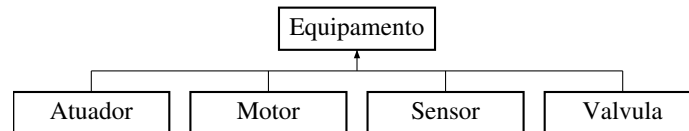
- [include/Excecoes.hpp](#)

11.21 Equipamento Class Reference

Representa um equipamento físico dentro de um [Conjunto](#).

```
#include <Equipamento.hpp>
```

Inheritance diagram for Equipamento:



Public Member Functions

- [Equipamento](#) (int id, std::string desc, std::string nome)
Constrói um equipamento.
- virtual [~Equipamento](#) ()=default
Destrutor virtual: permite uso polimórfico via ponteiro de base.
- virtual std::string [Tipo](#) () const
Tipo concreto do equipamento (sobrescrito pelas especializações).
- void [AdicionarParametro](#) (double ValorAtual, std::string Desc, int id)
Adiciona um parâmetro sem limites definidos.
- int [AdicionarParametro](#) (double ValorAtual, std::string Desc)
Adiciona um parâmetro sem limites com ID gerado automaticamente.
- void [AdicionarParametro](#) (double ValorAtual, double Max, double Min, std::string Desc, int id)
Adiciona um parâmetro com limites Min/Max definidos.
- int [AdicionarParametro](#) (double ValorAtual, double Max, double Min, std::string Desc)
Adiciona um parâmetro com limites Min/Max e ID gerado automaticamente.
- int [ProximoldParametro](#) () const
- void [RemoverParametro](#) (int id)
Remove o parâmetro com o ID fornecido.
- std::vector< int > [DiagnosticarEquipamento](#) () const
Diagnostica todos os parâmetros do equipamento.
- void [AtualizarAlarmes](#) ()
Atualiza o estado de todos os alarmes do equipamento.
- void [ExibirParametros](#) () const
Exibe no console os parâmetros e seus valores atuais.
- void [ExibirAlarmes](#) () const
Exibe no console os alarmes ativos do equipamento.
- int [QuantidadeParametros](#) () const
- int [QuantidadeAlarmes](#) () const
- int [GetId](#) () const
- std::string [GetDesc](#) () const
- std::string [GetNome](#) () const
- const std::map< int, [Parametro](#) > & [GetParametros](#) () const
- const std::vector< [Alarme](#) > & [GetAlarmes](#) () const
- void [SetDesc](#) (std::string desc)
Atualiza a descrição do equipamento.

11.21.1 Detailed Description

Representa um equipamento físico dentro de um [Conjunto](#).

Gerencia uma coleção de parâmetros monitorados e os alarmes derivados deles. Oferece diagnóstico consolidado do estado do equipamento.

Definition at line 16 of file [Equipamento.hpp](#).

11.21.2 Constructor & Destructor Documentation

11.21.2.1 Equipamento()

```
Equipamento::Equipamento (
    int id,
    std::string desc,
    std::string nome)
```

Constrói um equipamento.

Parameters

<i>id</i>	Identificador único.
<i>desc</i>	Descrição do equipamento.

Definition at line 6 of file [Equipamento.cpp](#).

11.21.2.2 ~Equipamento()

```
virtual Equipamento::~~Equipamento () [virtual], [default]
```

Destrutor virtual: permite uso polimórfico via ponteiro de base.

11.21.3 Member Function Documentation

11.21.3.1 AdicionarParametro() [1/4]

```
int Equipamento::AdicionarParametro (
    double ValorAtual,
    double Max,
    double Min,
    std::string Desc)
```

Adiciona um parâmetro com limites Min/Max e ID gerado automaticamente.

Returns

ID atribuído ao parâmetro criado.

Definition at line 50 of file [Equipamento.cpp](#).

11.21.3.2 AdicionarParametro() [2/4]

```
void Equipamento::AdicionarParametro (
    double ValorAtual,
    double Max,
    double Min,
    std::string Desc,
    int id)
```

Adiciona um parâmetro com limites Min/Max definidos.

Parameters

<i>ValorAtual</i>	Leitura inicial.
<i>Max</i>	Limite superior aceitável.
<i>Min</i>	Limite inferior aceitável.
<i>Desc</i>	Descrição do parâmetro.

Note

Implementa US-018 (Configurar parâmetros de equipamentos).

Definition at line 38 of file [Equipamento.cpp](#).

11.21.3.3 AdicionarParametro() [3/4]

```
int Equipamento::AdicionarParametro (
    double ValorAtual,
    std::string Desc)
```

Adiciona um parâmetro sem limites com ID gerado automaticamente.

Returns

ID atribuído ao parâmetro criado.

Definition at line 32 of file [Equipamento.cpp](#).

11.21.3.4 AdicionarParametro() [4/4]

```
void Equipamento::AdicionarParametro (
    double ValorAtual,
    std::string Desc,
    int id)
```

Adiciona um parâmetro sem limites definidos.

Parameters

<i>ValorAtual</i>	Leitura inicial.
<i>Desc</i>	Descrição do parâmetro.
<i>id</i>	Identificador único do parâmetro.

Note

Implementa US-018 (Configurar parâmetros de equipamentos).

Definition at line 20 of file [Equipamento.cpp](#).

11.21.3.5 AtualizarAlarmes()

```
void Equipamento::AtualizarAlarmes ()
```

Atualiza o estado de todos os alarmes do equipamento.

Note

Implementa US-014 (Criar [Alarme](#)).

Definition at line 77 of file [Equipamento.cpp](#).

11.21.3.6 DiagnosticarEquipamento()

```
std::vector< int > Equipamento::DiagnosticarEquipamento () const
```

Diagnostica todos os parâmetros do equipamento.

Returns

Vetor com IDs dos parâmetros em estado de falha.

Note

Implementa US-011 (Diagnóstico de [Equipamento](#)).

Definition at line 67 of file [Equipamento.cpp](#).

11.21.3.7 ExibirAlarmes()

```
void Equipamento::ExibirAlarmes () const
```

Exibe no console os alarmes ativos do equipamento.

Note

Implementa US-009 (Acessar [Equipamento](#)).

Definition at line 93 of file [Equipamento.cpp](#).

11.21.3.8 ExibirParametros()

```
void Equipamento::ExibirParametros () const
```

Exibe no console os parâmetros e seus valores atuais.

Note

Implementa US-009 (Acessar [Equipamento](#)).

Definition at line 86 of file [Equipamento.cpp](#).

11.21.3.9 GetAlarmes()

```
const std::vector< Alarme > & Equipamento::GetAlarmes () const
```

Returns

Referência constante ao vetor de alarmes.

Definition at line 116 of file [Equipamento.cpp](#).

11.21.3.10 GetDesc()

```
std::string Equipamento::GetDesc () const
```

Returns

Descrição do equipamento.

Definition at line 108 of file [Equipamento.cpp](#).

11.21.3.11 GetId()

```
int Equipamento::GetId () const
```

Returns

Identificador único do equipamento.

Definition at line 104 of file [Equipamento.cpp](#).

11.21.3.12 GetNome()

```
std::string Equipamento::GetNome () const
```

Returns

Nome do equipamento.

Definition at line 110 of file [Equipamento.cpp](#).

11.21.3.13 GetParametros()

```
const std::map< int, Parametro > & Equipamento::GetParametros () const
```

Returns

Referência constante ao mapa de parâmetros.

Definition at line 112 of file [Equipamento.cpp](#).

11.21.3.14 ProximoldParametro()

```
int Equipamento::ProximoIdParametro () const
```

Returns

Próximo ID disponível para parâmetro (monotônico).

Definition at line 56 of file [Equipamento.cpp](#).

11.21.3.15 QuantidadeAlarmes()

```
int Equipamento::QuantidadeAlarmes () const
```

Returns

Número de alarmes ativos.

Definition at line 102 of file [Equipamento.cpp](#).

11.21.3.16 QuantidadeParametros()

```
int Equipamento::QuantidadeParametros () const
```

Returns

Número de parâmetros cadastrados.

Definition at line 100 of file [Equipamento.cpp](#).

11.21.3.17 RemoveParametro()

```
void Equipamento::RemoveParametro (
    int id)
```

Remove o parâmetro com o ID fornecido.

Parameters

<i>id</i>	Identificador do parâmetro a remover.
-----------	---------------------------------------

Definition at line 60 of file [Equipamento.cpp](#).

11.21.3.18 SetDesc()

```
void Equipamento::SetDesc (
    std::string desc)
```

Atualiza a descrição do equipamento.

Parameters

<i>desc</i>	Nova descrição.
-------------	-----------------

Definition at line 120 of file [Equipamento.cpp](#).

11.21.3.19 Tipo()

```
std::string Equipamento::Tipo () const [virtual]
```

Tipo concreto do equipamento (sobrescrito pelas especializações).

Returns

Nome do tipo (ex.: "Equipamento", "Motor", "Sensor").

Reimplemented in [Atuador](#), [Motor](#), [Sensor](#), and [Valvula](#).

Definition at line 18 of file [Equipamento.cpp](#).

The documentation for this class was generated from the following files:

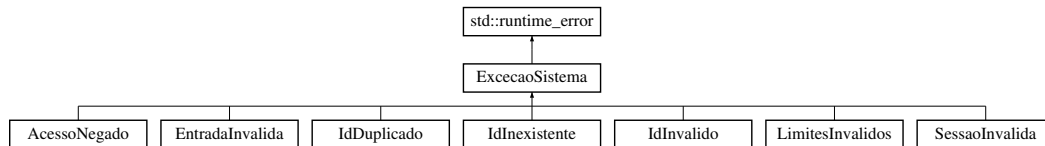
- [include/Equipamento.hpp](#)
- [src/Equipamento.cpp](#)

11.22 ExcecaoSistema Class Reference

Exceção base do sistema de monitoramento.

```
#include <Excecoes.hpp>
```

Inheritance diagram for ExcecaoSistema:



11.22.1 Detailed Description

Exceção base do sistema de monitoramento.

Todas as exceções específicas do domínio herdam desta classe, permitindo capturar qualquer erro do sistema com um único `catch` (`const ExcecaoSistema&`).

Definition at line 13 of file [Excecoes.hpp](#).

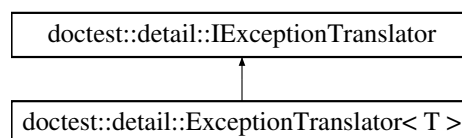
The documentation for this class was generated from the following file:

- [include/Excecoes.hpp](#)

11.23 doctest::detail::ExceptionTranslator< T > Class Template Reference

```
#include <doctest.h>
```

Inheritance diagram for doctest::detail::ExceptionTranslator< T >:



Public Member Functions

- [ExceptionTranslator](#) ([String](#)(*translateFunction)(T)) noexcept
- bool [translate](#) ([String](#) &res) const override

11.23.1 Detailed Description

```
template<typename T>
class doctest::detail::ExceptionTranslator< T >
```

Definition at line 2340 of file [doctest.h](#).

11.23.2 Constructor & Destructor Documentation

11.23.2.1 ExceptionTranslator()

```
template<typename T>
doctest::detail::ExceptionTranslator< T >::ExceptionTranslator (
    String(* translateFunction ) (T)) [inline], [explicit], [noexcept]
```

Definition at line 2342 of file [doctest.h](#).

11.23.3 Member Function Documentation

11.23.3.1 translate()

```
template<typename T>
bool doctest::detail::ExceptionTranslator< T >::translate (
    String & res) const [inline], [override], [virtual]
```

Implements [doctest::detail::IExceptionTranslator](#).

Definition at line 2345 of file [doctest.h](#).

The documentation for this class was generated from the following file:

- [tests/doctest.h](#)

11.24 doctest::detail::Expression_Lhs< L > Struct Template Reference

```
#include <doctest.h>
```

Public Member Functions

- [Expression_Lhs](#) (L &&in, [assertType::Enum](#) at)
- [DOCTEST_NOINLINE](#) operator Result ()
- [operator L](#) () const

Public Attributes

- [L lhs](#)
- [assertType::Enum m_at](#)

11.24.1 Detailed Description

```
template<typename L>
struct doctest::detail::Expression_lhs< L >
```

Definition at line 1971 of file [doctest.h](#).

11.24.2 Constructor & Destructor Documentation

11.24.2.1 Expression_lhs()

```
template<typename L>
doctest::detail::Expression_lhs< L >::Expression_lhs (
    L && in,
    assertType::Enum at) [inline], [explicit]
```

Definition at line 1976 of file [doctest.h](#).

11.24.3 Member Function Documentation

11.24.3.1 operator L()

```
template<typename L>
doctest::detail::Expression_lhs< L >::operator L () const [inline]
```

Definition at line 1995 of file [doctest.h](#).

11.24.3.2 operator Result()

```
template<typename L>
DOCTEST_NOINLINE doctest::detail::Expression_lhs< L >::operator Result () [inline]
```

Definition at line 1979 of file [doctest.h](#).

11.24.4 Member Data Documentation

11.24.4.1 lhs

```
template<typename L>
L doctest::detail::Expression_lhs< L >::lhs
```

Definition at line 1972 of file [doctest.h](#).

11.24.4.2 m_at

```
template<typename L>
assertType::Enum doctest::detail::Expression_lhs< L >::m_at
```

Definition at line 1973 of file [doctest.h](#).

The documentation for this struct was generated from the following file:

- [tests/doctest.h](#)

11.25 doctest::detail::ExpressionDecomposer Struct Reference

```
#include <doctest.h>
```

Public Member Functions

- [ExpressionDecomposer](#) ([assertType::Enum](#) at)
- [template<typename L> Expression_lhs< const L && > operator<< \(const L &&operand\)](#)
- [template<typename L, typename types::enable_if<!doctest::detail::types::is_rvalue_reference< L >::value, void >::type * = nullptr> Expression_lhs< const L & > operator<< \(const L &operand\)](#)

Public Attributes

- [assertType::Enum m_at](#)

11.25.1 Detailed Description

Definition at line 2045 of file [doctest.h](#).

11.25.2 Constructor & Destructor Documentation

11.25.2.1 ExpressionDecomposer()

```
doctest::detail::ExpressionDecomposer::ExpressionDecomposer (
    assertType::Enum at)
```

11.25.3 Member Function Documentation

11.25.3.1 operator<<() [1/2]

```
template<typename L>
Expression_lhs< const L && > doctest::detail::ExpressionDecomposer::operator<< (
    const L && operand) [inline]
```

Definition at line 2048 of file [doctest.h](#).

11.26.2 Member Data Documentation

11.26.2.1 value

`DOCTEST_CONSTEXPR` `bool doctest::detail::types::false_type::value = false [static]`

Definition at line 741 of file [doctest.h](#).

The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

11.27 doctest::detail::filldata< T, Enable > Struct Template Reference

```
#include <doctest.h>
```

Static Public Member Functions

- static void [fill](#) ([std::ostream](#) *stream, const T &in)

11.27.1 Detailed Description

```
template<typename T, typename Enable>
struct doctest::detail::filldata< T, Enable >
```

Definition at line 1193 of file [doctest.h](#).

11.27.2 Member Function Documentation

11.27.2.1 fill()

```
template<typename T, typename Enable>
void doctest::detail::filldata< T, Enable >::fill (
    std::ostream * stream,
    const T & in) [inline], [static]
```

Definition at line 1194 of file [doctest.h](#).

The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

11.28 doctest::detail::filldata< const char[N]> Struct Template Reference

```
#include <doctest.h>
```

Static Public Member Functions

- static void [fill](#) ([std::ostream](#) *stream, const char(&in)[N])

11.28.1 Detailed Description

```
template<size\_t N>
struct doctest::detail::filldata< const char[N]>
```

Definition at line [1224](#) of file [doctest.h](#).

11.28.2 Member Function Documentation

11.28.2.1 fill()

```
template<size\_t N>
void doctest::detail::filldata< const char[N]>::fill (
    std::ostream * stream,
    const char(&) in[N]) [inline], [static]
```

Definition at line [1225](#) of file [doctest.h](#).

The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

11.29 doctest::detail::filldata< const void * > Struct Reference

```
#include <doctest.h>
```

Static Public Member Functions

- static [DOCTEST_INTERFACE](#) void [fill](#) ([std::ostream](#) *stream, const void *in)

11.29.1 Detailed Description

Definition at line [1232](#) of file [doctest.h](#).

11.29.2 Member Function Documentation

11.29.2.1 fill()

```
DOCTEST\_INTERFACE void doctest::detail::filldata< const void * >::fill (
    std::ostream * stream,
    const void * in) [static]
```

The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

11.30 doctest::detail::filldata< const volatile void * > Struct Reference

```
#include <doctest.h>
```

Static Public Member Functions

- static [DOCTEST_INTERFACE](#) void [fill](#) ([std::ostream](#) *stream, const volatile void *in)

11.30.1 Detailed Description

Definition at line [1237](#) of file [doctest.h](#).

11.30.2 Member Function Documentation

11.30.2.1 fill()

```
DOCTEST\_INTERFACE void doctest::detail::filldata< const volatile void * >::fill (  
    std::ostream * stream,  
    const volatile void * in) [static]
```

The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

11.31 doctest::detail::filldata< T * > Struct Template Reference

```
#include <doctest.h>
```

Static Public Member Functions

- static void [fill](#) ([std::ostream](#) *stream, const T *in)

11.31.1 Detailed Description

```
template<typename T>  
struct doctest::detail::filldata< T * >
```

Definition at line [1242](#) of file [doctest.h](#).

11.31.2 Member Function Documentation

11.31.2.1 fill()

```
template<typename T>
void doctest::detail::filldata< T * >::fill (
    std::ostream * stream,
    const T * in) [inline], [static]
```

Definition at line 1244 of file [doctest.h](#).

The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

11.32 doctest::detail::filldata< T, typename detail::types::enable_if< !detail::has_insertion_operator< T >::value &&detail::is_container< T >::value >::type > Struct Template Reference

```
#include <doctest.h>
```

Static Public Member Functions

- static void fill (std::ostream *stream, const DOCTEST_REF_WRAP(T) in)

11.32.1 Detailed Description

```
template<typename T>
struct doctest::detail::filldata< T, typename detail::types::enable_if< !detail::has_insertion_operator< T >::value &&detail::is_container< T >::value >::type >
```

Definition at line 1271 of file [doctest.h](#).

11.32.2 Member Function Documentation

11.32.2.1 fill()

```
template<typename T>
void doctest::detail::filldata< T, typename detail::types::enable_if< !detail::has_insertion_operator< T >::value &&detail::is_container< T >::value >::type >::fill (
    std::ostream * stream,
    const DOCTEST_REF_WRAP(T) in) [inline], [static]
```

Definition at line 1275 of file [doctest.h](#).

The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

11.33 doctest::detail::filldata< T, typename detail::types::enable_if<!detail::has_insertion_operator< T >::value &&detail::is_pair< T >::value >::type > Struct Template Reference

```
#include <doctest.h>
```

Static Public Member Functions

- static void [fill](#) ([std::ostream](#) *stream, const T &in)

11.33.1 Detailed Description

```
template<typename T>
struct doctest::detail::filldata< T, typename detail::types::enable_if<!detail::has_insertion_operator< T
>::value &&detail::is_pair< T >::value >::type >
```

Definition at line [1262](#) of file [doctest.h](#).

11.33.2 Member Function Documentation

11.33.2.1 fill()

```
template<typename T>
void doctest::detail::filldata< T, typename detail::types::enable_if<!detail::has_insertion_operator<
T >::value &&detail::is_pair< T >::value >::type >::fill (
    std::ostream * stream,
    const T & in) [inline], [static]
```

Definition at line [1265](#) of file [doctest.h](#).

The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

11.34 doctest::detail::filldata< T[N]> Struct Template Reference

```
#include <doctest.h>
```

Static Public Member Functions

- static void [fill](#) ([std::ostream](#) *stream, const T(&in)[N])

11.34.1 Detailed Description

```
template<typename T, size\_t N>
struct doctest::detail::filldata< T[N]>
```

Definition at line [1206](#) of file [doctest.h](#).

11.34.2 Member Function Documentation

11.34.2.1 fill()

```
template<typename T, size\_t N>
void doctest::detail::filldata< T[N]>::fill (
    std::ostream * stream,
    const T(&) in[N]) [inline], [static]
```

Definition at line [1207](#) of file [doctest.h](#).

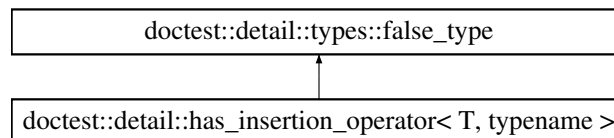
The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

11.35 doctest::detail::has_insertion_operator< T, typename > Struct Template Reference

```
#include <doctest.h>
```

Inheritance diagram for doctest::detail::has_insertion_operator< T, typename >:



Additional Inherited Members

Static Public Attributes inherited from [doctest::detail::types::false_type](#)

- static [DOCTEST_CONSTEXPR](#) bool [value](#) = false

11.35.1 Detailed Description

```
template<typename T, typename = void>
struct doctest::detail::has_insertion_operator< T, typename >
```

Definition at line [1030](#) of file [doctest.h](#).

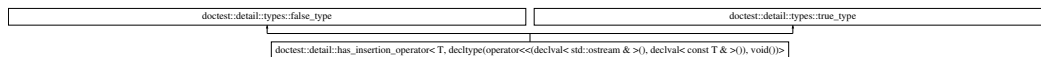
The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

11.36 doctest::detail::has_insertion_operator< T, decltype(operator<<(declval< std::ostream & >()), declval< const T & >()), void())> Struct Template Reference

```
#include <doctest.h>
```

Inheritance diagram for doctest::detail::has_insertion_operator< T, decltype(operator<<(declval< std::ostream & >()), declval< const T & >()), void())>:



Additional Inherited Members

Static Public Attributes inherited from doctest::detail::types::false_type

- static `DOCTEST_CONSTEXPR` bool `value` = false

Static Public Attributes inherited from doctest::detail::types::true_type

- static `DOCTEST_CONSTEXPR` bool `value` = true

11.36.1 Detailed Description

```
template<typename T>
struct doctest::detail::has_insertion_operator< T, decltype(operator<<(declval< std::ostream & >()),
declval< const T & >()), void())>
```

Definition at line 1034 of file [doctest.h](#).

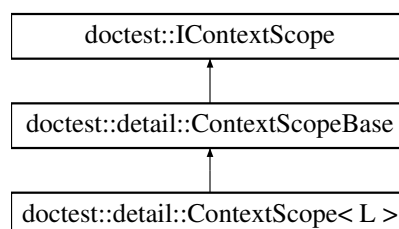
The documentation for this struct was generated from the following file:

- [tests/doctest.h](#)

11.37 doctest::IContextScope Struct Reference

```
#include <doctest.h>
```

Inheritance diagram for doctest::IContextScope:



Public Member Functions

- virtual void [stringify](#) ([std::ostream](#) *) const =0

11.37.1 Detailed Description

Definition at line [2401](#) of file [doctest.h](#).

11.37.2 Member Function Documentation

11.37.2.1 stringify()

```
virtual void doctest::IContextScope::stringify (  
    std::ostream * ) const    [pure virtual]
```

Implemented in [doctest::detail::ContextScope< L >](#).

The documentation for this struct was generated from the following file:

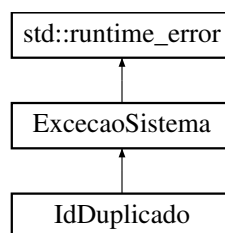
- tests/[doctest.h](#)

11.38 IdDuplicado Class Reference

Tentativa de inserir um identificador já existente.

```
#include <Excecoes.hpp>
```

Inheritance diagram for IdDuplicado:



11.38.1 Detailed Description

Tentativa de inserir um identificador já existente.

Definition at line [37](#) of file [Excecoes.hpp](#).

The documentation for this class was generated from the following file:

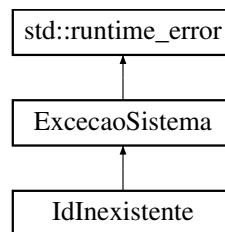
- include/[Excecoes.hpp](#)

11.39 IdInexistente Class Reference

Acesso a um identificador inexistente.

```
#include <Excecoes.hpp>
```

Inheritance diagram for IdInexistente:



11.39.1 Detailed Description

Acesso a um identificador inexistente.

Definition at line 43 of file [Excecoes.hpp](#).

The documentation for this class was generated from the following file:

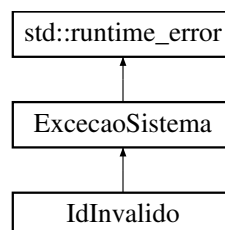
- include/[Excecoes.hpp](#)

11.40 IdInvalido Class Reference

Identificador inválido (ex.: negativo).

```
#include <Excecoes.hpp>
```

Inheritance diagram for IdInvalido:



11.40.1 Detailed Description

Identificador inválido (ex.: negativo).

Definition at line 31 of file [Excecoes.hpp](#).

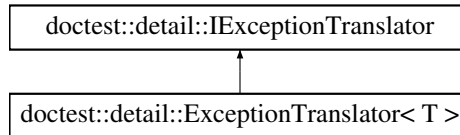
The documentation for this class was generated from the following file:

- include/[Excecoes.hpp](#)

11.41 doctest::detail::IExceptionTranslator Struct Reference

```
#include <doctest.h>
```

Inheritance diagram for doctest::detail::IExceptionTranslator:



Public Member Functions

- virtual bool [translate](#) ([String](#) &) const =0

11.41.1 Detailed Description

Definition at line [2334](#) of file [doctest.h](#).

11.41.2 Member Function Documentation

11.41.2.1 [translate\(\)](#)

```
virtual bool doctest::detail::IExceptionTranslator::translate (
    String & ) const [pure virtual]
```

Implemented in [doctest::detail::ExceptionTranslator< T >](#).

The documentation for this struct was generated from the following file:

- [tests/doctest.h](#)

11.42 doctest::IReporter Struct Reference

```
#include <doctest.h>
```

Public Member Functions

- virtual void [report_query](#) (const [QueryData](#) &)=0
- virtual void [test_run_start](#) ()=0
- virtual void [test_run_end](#) (const [TestRunStats](#) &)=0
- virtual void [test_case_start](#) (const [TestCaseData](#) &)=0
- virtual void [test_case_reenter](#) (const [TestCaseData](#) &)=0
- virtual void [test_case_end](#) (const [CurrentTestCaseStats](#) &)=0
- virtual void [test_case_exception](#) (const [TestCaseException](#) &)=0
- virtual void [subcase_start](#) (const [SubcaseSignature](#) &)=0
- virtual void [subcase_end](#) ()=0
- virtual void [log_assert](#) (const [AssertData](#) &)=0
- virtual void [log_message](#) (const [MessageData](#) &)=0
- virtual void [test_case_skipped](#) (const [TestCaseData](#) &)=0

Static Public Member Functions

- static int [get_num_active_contexts](#) ()
- static const [IContextScope](#) *const * [get_active_contexts](#) ()
- static int [get_num_stringified_contexts](#) ()
- static const [String](#) * [get_stringified_contexts](#) ()

11.42.1 Detailed Description

Definition at line 2774 of file [doctest.h](#).

11.42.2 Member Function Documentation

11.42.2.1 [get_active_contexts\(\)](#)

```
const IContextScope *const * doctest::IReporter::get_active_contexts () [static]
```

11.42.2.2 [get_num_active_contexts\(\)](#)

```
int doctest::IReporter::get_num_active_contexts () [static]
```

11.42.2.3 [get_num_stringified_contexts\(\)](#)

```
int doctest::IReporter::get_num_stringified_contexts () [static]
```

11.42.2.4 [get_stringified_contexts\(\)](#)

```
const String * doctest::IReporter::get_stringified_contexts () [static]
```

11.42.2.5 [log_assert\(\)](#)

```
virtual void doctest::IReporter::log_assert (  
    const AssertData & ) [pure virtual]
```

11.42.2.6 [log_message\(\)](#)

```
virtual void doctest::IReporter::log_message (  
    const MessageData & ) [pure virtual]
```

11.42.2.7 [report_query\(\)](#)

```
virtual void doctest::IReporter::report_query (  
    const QueryData & ) [pure virtual]
```

11.42.2.8 subcase_end()

```
virtual void doctest::IReporter::subcase_end () [pure virtual]
```

11.42.2.9 subcase_start()

```
virtual void doctest::IReporter::subcase_start (
    const SubcaseSignature & ) [pure virtual]
```

11.42.2.10 test_case_end()

```
virtual void doctest::IReporter::test_case_end (
    const CurrentTestCaseStats & ) [pure virtual]
```

11.42.2.11 test_case_exception()

```
virtual void doctest::IReporter::test_case_exception (
    const TestCaseException & ) [pure virtual]
```

11.42.2.12 test_case_reenter()

```
virtual void doctest::IReporter::test_case_reenter (
    const TestCaseData & ) [pure virtual]
```

11.42.2.13 test_case_skipped()

```
virtual void doctest::IReporter::test_case_skipped (
    const TestCaseData & ) [pure virtual]
```

11.42.2.14 test_case_start()

```
virtual void doctest::IReporter::test_case_start (
    const TestCaseData & ) [pure virtual]
```

11.42.2.15 test_run_end()

```
virtual void doctest::IReporter::test_run_end (
    const TestRunStats & ) [pure virtual]
```

11.42.2.16 test_run_start()

```
virtual void doctest::IReporter::test_run_start () [pure virtual]
```

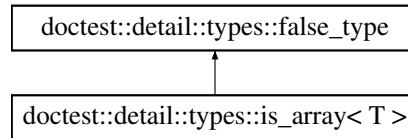
The documentation for this struct was generated from the following file:

- [tests/doctest.h](#)

11.43 doctest::detail::types::is_array< T > Struct Template Reference

```
#include <doctest.h>
```

Inheritance diagram for doctest::detail::types::is_array< T >:



Additional Inherited Members

Static Public Attributes inherited from doctest::detail::types::false_type

- static [DOCTEST_CONSTEXPR](#) bool [value](#) = false

11.43.1 Detailed Description

```
template<typename T>
struct doctest::detail::types::is_array< T >
```

Definition at line 799 of file [doctest.h](#).

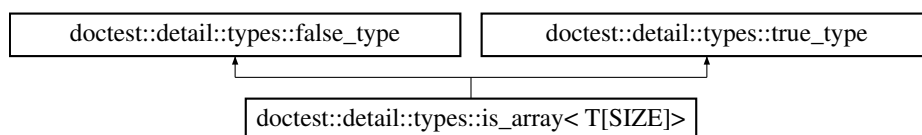
The documentation for this struct was generated from the following file:

- [tests/doctest.h](#)

11.44 doctest::detail::types::is_array< T[SIZE]> Struct Template Reference

```
#include <doctest.h>
```

Inheritance diagram for doctest::detail::types::is_array< T[SIZE]>:



Additional Inherited Members

Static Public Attributes inherited from doctest::detail::types::false_type

- static [DOCTEST_CONSTEXPR](#) bool [value](#) = false

Static Public Attributes inherited from [doctest::detail::types::true_type](#)

- static [DOCTEST_CONSTEXPR](#) bool [value](#) = true

11.44.1 Detailed Description

```
template<typename T, size\_t SIZE>
struct doctest::detail::types::is_array< T[SIZE]>
```

Definition at line [802](#) of file [doctest.h](#).

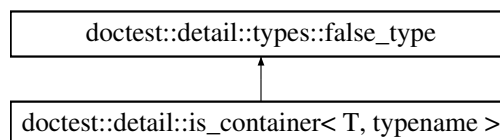
The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

11.45 doctest::detail::is_container< T, typename > Struct Template Reference

```
#include <doctest.h>
```

Inheritance diagram for doctest::detail::is_container< T, typename >:

**Additional Inherited Members****Static Public Attributes inherited from [doctest::detail::types::false_type](#)**

- static [DOCTEST_CONSTEXPR](#) bool [value](#) = false

11.45.1 Detailed Description

```
template<typename T, typename = void>
struct doctest::detail::is_container< T, typename >
```

Definition at line [1056](#) of file [doctest.h](#).

The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

11.46 doctest::detail::is_container< T, types::void_t< typename T::value_type, typename T::iterator, decltype(declval< T >().begin()), decltype(declval< T >().end())> > Struct Template Reference

```
#include <doctest.h>
```

Inheritance diagram for doctest::detail::is_container< T, types::void_t< typename T::value_type, typename T::iterator, decltype(declval< T >().begin()), decltype(declval< T >().end())> >:



Additional Inherited Members

Static Public Attributes inherited from [doctest::detail::types::false_type](#)

- static [DOCTEST_CONSTEXPR](#) bool [value](#) = false

Static Public Attributes inherited from [doctest::detail::types::true_type](#)

- static [DOCTEST_CONSTEXPR](#) bool [value](#) = true

11.46.1 Detailed Description

```
template<typename T>
struct doctest::detail::is_container< T, types::void_t< typename T::value_type, typename T::iterator,
decltype(declval< T >().begin()), decltype(declval< T >().end())> >
```

Definition at line 1059 of file [doctest.h](#).

The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

11.47 doctest::detail::types::is_enum< T > Struct Template Reference

```
#include <doctest.h>
```

Static Public Attributes

- static [DOCTEST_CONSTEXPR](#) bool [value](#) = __is_enum(T)

11.47.1 Detailed Description

```
template<typename T>
struct doctest::detail::types::is_enum< T >
```

Definition at line 783 of file [doctest.h](#).

11.47.2 Member Data Documentation

11.47.2.1 value

```
template<typename T>
DOCTEST_CONSTEXPR bool doctest::detail::types::is_enum< T >::value = __is_enum(T) [static]
```

Definition at line 784 of file [doctest.h](#).

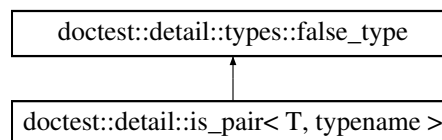
The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

11.48 doctest::detail::is_pair< T, typename > Struct Template Reference

```
#include <doctest.h>
```

Inheritance diagram for doctest::detail::is_pair< T, typename >:



Additional Inherited Members

Static Public Attributes inherited from [doctest::detail::types::false_type](#)

- static [DOCTEST_CONSTEXPR](#) bool [value](#) = false

11.48.1 Detailed Description

```
template<typename T, typename = void>
struct doctest::detail::is_pair< T, typename >
```

Definition at line 1044 of file [doctest.h](#).

The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

11.49 doctest::detail::is_pair< T, types::void_t< typename T::first_type, typename T::second_type, decltype(declval< T >().first), decltype(declval< T >().second)> > Struct Template Reference

```
#include <doctest.h>
```

Inheritance diagram for doctest::detail::is_pair< T, types::void_t< typename T::first_type, typename T::second_type, decltype(declval< T >().first), decltype(declval< T >().second)> >:



Additional Inherited Members

Static Public Attributes inherited from doctest::detail::types::false_type

- static `DOCTEST_CONSTEXPR` bool `value` = false

Static Public Attributes inherited from doctest::detail::types::true_type

- static `DOCTEST_CONSTEXPR` bool `value` = true

11.49.1 Detailed Description

```
template<typename T>
struct doctest::detail::is_pair< T, types::void_t< typename T::first_type, typename T::second_type,
decltype(declval< T >().first), decltype(declval< T >().second)> >
```

Definition at line 1047 of file [doctest.h](#).

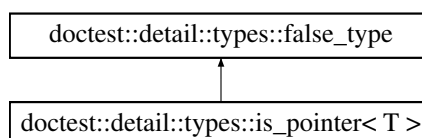
The documentation for this struct was generated from the following file:

- [tests/doctest.h](#)

11.50 doctest::detail::types::is_pointer< T > Struct Template Reference

```
#include <doctest.h>
```

Inheritance diagram for doctest::detail::types::is_pointer< T >:



Additional Inherited Members

Static Public Attributes inherited from [doctest::detail::types::false_type](#)

- static [DOCTEST_CONSTEXPR](#) bool [value](#) = false

11.50.1 Detailed Description

```
template<typename T>
struct doctest::detail::types::is_pointer< T * >
```

Definition at line 793 of file [doctest.h](#).

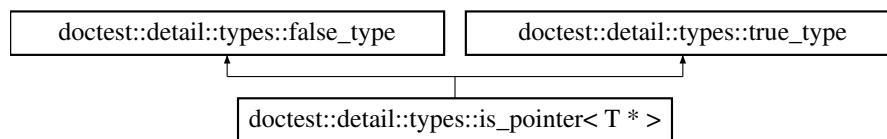
The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

11.51 doctest::detail::types::is_pointer< T * > Struct Template Reference

```
#include <doctest.h>
```

Inheritance diagram for doctest::detail::types::is_pointer< T * >:



Additional Inherited Members

Static Public Attributes inherited from [doctest::detail::types::false_type](#)

- static [DOCTEST_CONSTEXPR](#) bool [value](#) = false

Static Public Attributes inherited from [doctest::detail::types::true_type](#)

- static [DOCTEST_CONSTEXPR](#) bool [value](#) = true

11.51.1 Detailed Description

```
template<typename T>
struct doctest::detail::types::is_pointer< T * >
```

Definition at line 796 of file [doctest.h](#).

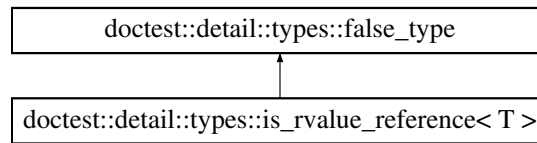
The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

11.52 doctest::detail::types::is_rvalue_reference< T > Struct Template Reference

```
#include <doctest.h>
```

Inheritance diagram for doctest::detail::types::is_rvalue_reference< T >:



Additional Inherited Members

Static Public Attributes inherited from [doctest::detail::types::false_type](#)

- static [DOCTEST_CONSTEXPR](#) bool [value](#) = false

11.52.1 Detailed Description

```
template<typename T>
struct doctest::detail::types::is_rvalue_reference< T >
```

Definition at line 766 of file [doctest.h](#).

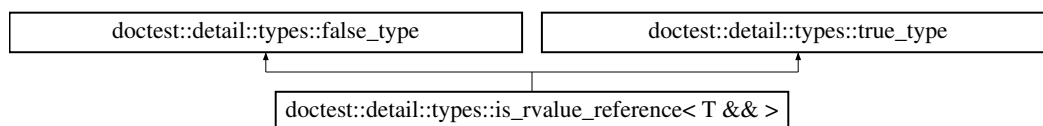
The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

11.53 doctest::detail::types::is_rvalue_reference< T && > Struct Template Reference

```
#include <doctest.h>
```

Inheritance diagram for doctest::detail::types::is_rvalue_reference< T && >:



Additional Inherited Members

Static Public Attributes inherited from [doctest::detail::types::false_type](#)

- static [DOCTEST_CONSTEXPR](#) bool [value](#) = false

Static Public Attributes inherited from [doctest::detail::types::true_type](#)

- static [DOCTEST_CONSTEXPR](#) bool [value](#) = true

11.53.1 Detailed Description

```
template<typename T>
struct doctest::detail::types::is_rvalue_reference< T && >
```

Definition at line 769 of file [doctest.h](#).

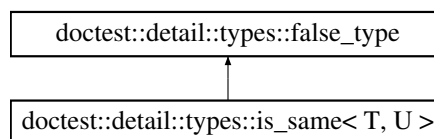
The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

11.54 doctest::detail::types::is_same< T, U > Struct Template Reference

```
#include <doctest.h>
```

Inheritance diagram for doctest::detail::types::is_same< T, U >:



Additional Inherited Members

Static Public Attributes inherited from [doctest::detail::types::false_type](#)

- static [DOCTEST_CONSTEXPR](#) bool [value](#) = false

11.54.1 Detailed Description

```
template<typename T, typename U>
struct doctest::detail::types::is_same< T, U >
```

Definition at line 760 of file [doctest.h](#).

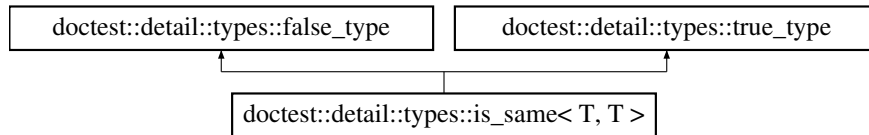
The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

11.55 doctest::detail::types::is_same< T, T > Struct Template Reference

```
#include <doctest.h>
```

Inheritance diagram for doctest::detail::types::is_same< T, T >:



Additional Inherited Members

Static Public Attributes inherited from [doctest::detail::types::false_type](#)

- static [DOCTEST_CONSTEXPR](#) bool [value](#) = false

Static Public Attributes inherited from [doctest::detail::types::true_type](#)

- static [DOCTEST_CONSTEXPR](#) bool [value](#) = true

11.55.1 Detailed Description

```
template<typename T>
struct doctest::detail::types::is_same< T, T >
```

Definition at line 763 of file [doctest.h](#).

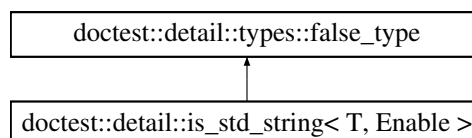
The documentation for this struct was generated from the following file:

- [tests/doctest.h](#)

11.56 doctest::detail::is_std_string< T, Enable > Struct Template Reference

```
#include <doctest.h>
```

Inheritance diagram for doctest::detail::is_std_string< T, Enable >:



Static Public Attributes inherited from [doctest::detail::types::false_type](#)

- static [DOCTEST_CONSTEXPR](#) bool [value](#) = false

11.56.1 Detailed Description

```
template<typename T, typename Enable = void>
struct doctest::detail::is_std_string< T, Enable >
```

Definition at line 864 of file [doctest.h](#).

The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

11.57 doctest::detail::is_std_string< T, typename types::enable_if< types::is_same< decltype(declval< const T & >().c_str()), const char * >::value &&types::is_same< decltype(declval< const T & >().size()), size_t >::value >::type > Struct Template Reference

```
#include <doctest.h>
```

Inheritance diagram for doctest::detail::is_std_string< T, typename types::enable_if< types::is_same< decltype(declval< const T & >().c_str()), const char * >::value &&types::is_same< decltype(declval< const T & >().size()), size_t >::value >::type >:



Additional Inherited Members

Static Public Attributes inherited from [doctest::detail::types::false_type](#)

- static [DOCTEST_CONSTEXPR](#) bool [value](#) = false

Static Public Attributes inherited from [doctest::detail::types::true_type](#)

- static [DOCTEST_CONSTEXPR](#) bool [value](#) = true

11.57.1 Detailed Description

```
template<typename T>
struct doctest::detail::is_std_string< T, typename types::enable_if< types::is_same< decltype(declval<
const T &>().c_str()), const char * >::value &&types::is_same< decltype(declval< const T &>().size()),
size_t >::value >::type >
```

Definition at line 867 of file [doctest.h](#).

The documentation for this struct was generated from the following file:

- [tests/doctest.h](#)

11.58 doctest::IsNaN< F > Struct Template Reference

```
#include <doctest.h>
```

Public Member Functions

- [IsNaN](#) (F f, bool flip=false)
- [IsNaN](#)< F > [operator!](#) () const
- [operator bool](#) () const

Public Attributes

- F [value](#)
- bool [flipped](#)

11.58.1 Detailed Description

```
template<typename F>
struct doctest::IsNaN< F >
```

Definition at line 1430 of file [doctest.h](#).

11.58.2 Constructor & Destructor Documentation

11.58.2.1 IsNaN()

```
template<typename F>
doctest::IsNaN< F >::IsNaN (
    F f,
    bool flip = false) [inline]
```

Definition at line 1433 of file [doctest.h](#).

11.58.3 Member Function Documentation

11.58.3.1 operator bool()

```
template<typename F>
doctest::IsNaN< F >::operator bool () const
```

11.58.3.2 operator"!()

```
template<typename F>
IsNaN< F > doctest::IsNaN< F >::operator! () const [inline]
```

Definition at line 1436 of file [doctest.h](#).

11.58.4 Member Data Documentation

11.58.4.1 flipped

```
template<typename F>
bool doctest::IsNaN< F >::flipped
```

Definition at line 1432 of file [doctest.h](#).

11.58.4.2 value

```
template<typename F>
F doctest::IsNaN< F >::value
```

Definition at line 1431 of file [doctest.h](#).

The documentation for this struct was generated from the following file:

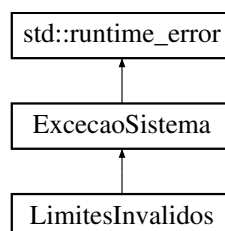
- [tests/doctest.h](#)

11.59 LimitesInvalidos Class Reference

Limites incoerentes (Max < Min).

```
#include <Excecoes.hpp>
```

Inheritance diagram for LimitesInvalidos:



11.59.1 Detailed Description

Limites incoerentes (Max < Min).

Definition at line 25 of file [Excecoes.hpp](#).

The documentation for this class was generated from the following file:

- include/[Excecoes.hpp](#)

11.60 LinhaDeProducao Class Reference

Representa uma linha de produção industrial.

```
#include <LinhaDeProducao.hpp>
```

Public Member Functions

- [LinhaDeProducao](#) (int id, std::string desc, std::string local, std::string nome)
Constrói uma linha de produção.
- void [AdicionarConjunto](#) (int id, std::string nome, std::string desc)
Adiciona um conjunto à linha.
- int [AdicionarConjunto](#) (std::string nome, std::string desc)
Adiciona um conjunto com ID gerado automaticamente (incremental).
- int [ProximoldConjunto](#) () const
- void [RemoverConjunto](#) (int id)
Remove o conjunto com o ID fornecido.
- void [ExibirConjuntos](#) () const
Exibe no console os conjuntos da linha.
- void [ExibirAlarmes](#) () const
Exibe no console os alarmes ativos de toda a linha.
- void [ExibirTudo](#) () const
Exibe conjuntos, equipamentos e alarmes de forma consolidada.
- int [QuantidadeConjuntos](#) () const
- int [QuantidadeAlarmes](#) () const
- std::vector< int > [DiagnosticarLinha](#) () const
Diagnostica todos os conjuntos da linha.
- void [AtualizarAlarmes](#) ()
Propaga a atualização de alarmes para todos os conjuntos.
- void [SetDesc](#) (std::string desc)
Atualiza a descrição da linha.
- void [SetLocal](#) (std::string local)
Atualiza a localização da linha.
- void [SetName](#) (std::string nome)
Atualiza o nome da linha.
- const std::map< int, [Conjunto](#) > & [GetConjuntos](#) () const
- [Conjunto](#) & [AcessarConjunto](#) (int id)
Acesso mutável a um conjunto pelo ID.
- int [GetId](#) () const
- std::string [GetDesc](#) () const
- std::string [GetLocal](#) () const
- std::string [GetName](#) () const

11.60.1 Detailed Description

Representa uma linha de produção industrial.

Agrega conjuntos de equipamentos e oferece operações de diagnóstico e visualização para toda a linha.

Definition at line 16 of file [LinhaDeProducao.hpp](#).

11.60.2 Constructor & Destructor Documentation

11.60.2.1 LinhaDeProducao()

```
LinhaDeProducao::LinhaDeProducao (
    int id,
    std::string desc,
    std::string local,
    std::string nome)
```

Constrói uma linha de produção.

Parameters

<i>id</i>	Identificador único.
<i>desc</i>	Descrição da linha.
<i>local</i>	Localização física.
<i>nome</i>	Nome da linha.

Definition at line 7 of file [LinhaDeProducao.cpp](#).

11.60.3 Member Function Documentation

11.60.3.1 AcessarConjunto()

```
Conjunto & LinhaDeProducao::AcessarConjunto (
    int id)
```

Acesso mutável a um conjunto pelo ID.

Parameters

<i>id</i>	Identificador do conjunto.
-----------	----------------------------

Returns

Referência ao conjunto.

Exceptions

<i>IdInexistente</i>	se não houver conjunto com o ID.
----------------------	----------------------------------

Definition at line 110 of file [LinhaDeProducao.cpp](#).

11.60.3.2 AdicionarConjunto() [1/2]

```
void LinhaDeProducao::AdicionarConjunto (
    int id,
    std::string nome,
    std::string desc)
```

Adiciona um conjunto à linha.

Parameters

<i>id</i>	Identificador único do conjunto.
<i>desc</i>	Descrição do conjunto.

Note

Implementa US-004 (Criar Conjuntos).

Definition at line 20 of file [LinhaDeProducao.cpp](#).

11.60.3.3 AdicionarConjunto() [2/2]

```
int LinhaDeProducao::AdicionarConjunto (
    std::string nome,
    std::string desc)
```

Adiciona um conjunto com ID gerado automaticamente (incremental).

Returns

ID atribuído ao conjunto criado.

Definition at line 31 of file [LinhaDeProducao.cpp](#).

11.60.3.4 AtualizarAlarmes()

```
void LinhaDeProducao::AtualizarAlarmes ()
```

Propaga a atualização de alarmes para todos os conjuntos.

Note

Implementa US-014 (Criar [Alarme](#)).

Definition at line 90 of file [LinhaDeProducao.cpp](#).

11.60.3.5 DiagnosticarLinha()

```
std::vector< int > LinhaDeProducao::DiagnosticarLinha () const
```

Diagnostica todos os conjuntos da linha.

Returns

Vetor com IDs dos parâmetros em estado de falha.

Note

Implementa US-013 (Diagnóstico de Linha de Produção).

Definition at line 81 of file [LinhaDeProducao.cpp](#).

11.60.3.6 ExibirAlarmes()

```
void LinhaDeProducao::ExibirAlarmes () const
```

Exibe no console os alarmes ativos de toda a linha.

Definition at line 55 of file [LinhaDeProducao.cpp](#).

11.60.3.7 ExibirConjuntos()

```
void LinhaDeProducao::ExibirConjuntos () const
```

Exibe no console os conjuntos da linha.

Note

Implementa US-003 (Acessar Linhas de Produção).

Definition at line 48 of file [LinhaDeProducao.cpp](#).

11.60.3.8 ExibirTudo()

```
void LinhaDeProducao::ExibirTudo () const
```

Exibe conjuntos, equipamentos e alarmes de forma consolidada.

Note

Implementa US-003 (Acessar Linhas de Produção).

Definition at line 61 of file [LinhaDeProducao.cpp](#).

11.60.3.9 GetConjuntos()

```
const std::map< int, Conjunto > & LinhaDeProducao::GetConjuntos () const
```

Returns

Referência constante ao mapa de conjuntos.

Definition at line 108 of file [LinhaDeProducao.cpp](#).

11.60.3.10 GetDesc()

```
std::string LinhaDeProducao::GetDesc () const
```

Returns

Descrição da linha.

Definition at line 122 of file [LinhaDeProducao.cpp](#).

11.60.3.11 GetId()

```
int LinhaDeProducao::GetId () const
```

Returns

Identificador único da linha.

Definition at line 118 of file [LinhaDeProducao.cpp](#).

11.60.3.12 GetLocal()

```
std::string LinhaDeProducao::GetLocal () const
```

Returns

Localização física da linha.

Definition at line 126 of file [LinhaDeProducao.cpp](#).

11.60.3.13 GetNome()

```
std::string LinhaDeProducao::GetNome () const
```

Returns

Nome da linha.

Definition at line 130 of file [LinhaDeProducao.cpp](#).

11.60.3.14 ProximoIdConjunto()

```
int LinhaDeProducao::ProximoIdConjunto () const
```

Returns

Próximo ID disponível para conjunto (monotônico).

Definition at line 37 of file [LinhaDeProducao.cpp](#).

11.60.3.15 QuantidadeAlarmes()

```
int LinhaDeProducao::QuantidadeAlarmes () const
```

Returns

Número total de alarmes ativos na linha.

Definition at line 73 of file [LinhaDeProducao.cpp](#).

11.60.3.16 QuantidadeConjuntos()

```
int LinhaDeProducao::QuantidadeConjuntos () const
```

Returns

Número de conjuntos cadastrados na linha.

Definition at line 69 of file [LinhaDeProducao.cpp](#).

11.60.3.17 RemoverConjunto()

```
void LinhaDeProducao::RemoverConjunto (
    int id)
```

Remove o conjunto com o ID fornecido.

Parameters

<i>id</i>	Identificador do conjunto a remover.
-----------	--------------------------------------

Note

Implementa US-005 (Remover [Conjunto](#)).

Definition at line 41 of file [LinhaDeProducao.cpp](#).

11.60.3.18 SetDesc()

```
void LinhaDeProducao::SetDesc (
    std::string desc)
```

Atualiza a descrição da linha.

Parameters

<i>desc</i>	Nova descrição.
-------------	-----------------

Definition at line 96 of file [LinhaDeProducao.cpp](#).

11.60.3.19 SetLocal()

```
void LinhaDeProducao::SetLocal (
    std::string local)
```

Atualiza a localização da linha.

Parameters

<i>local</i>	Nova localização.
--------------	-------------------

Definition at line 100 of file [LinhaDeProducao.cpp](#).

11.60.3.20 SetNome()

```
void LinhaDeProducao::SetNome (
    std::string nome)
```

Atualiza o nome da linha.

Parameters

<i>nome</i>	Novo nome.
-------------	------------

Definition at line 104 of file [LinhaDeProducao.cpp](#).

The documentation for this class was generated from the following files:

- include/[LinhaDeProducao.hpp](#)
- src/[LinhaDeProducao.cpp](#)

11.61 Log Class Reference

Registra eventos e operações do sistema em arquivo de texto.

```
#include <Log.hpp>
```


Public Member Functions

- [Log](#) (const std::string &arquivo)
Constrói o logger apontando para o arquivo especificado.
- bool [Registrar](#) (const std::string &mensagem) const
Acrescenta uma mensagem ao arquivo de log.
- std::string [GetArquivoLog](#) () const
- bool [SetArquivoLog](#) (const std::string &arquivo)
Redireciona o log para um novo arquivo.

11.61.1 Detailed Description

Registra eventos e operações do sistema em arquivo de texto.

Definition at line 9 of file [Log.hpp](#).

11.61.2 Constructor & Destructor Documentation

11.61.2.1 Log()

```
Log::Log (  
    const std::string & arquivo)
```

Constrói o logger apontando para o arquivo especificado.

Parameters

<i>arquivo</i>	Caminho do arquivo de log.
----------------	----------------------------

Definition at line 5 of file [Log.cpp](#).

11.61.3 Member Function Documentation

11.61.3.1 GetArquivoLog()

```
std::string Log::GetArquivoLog () const
```

Returns

Caminho do arquivo de log atual.

Definition at line 23 of file [Log.cpp](#).

11.61.3.2 Registrar()

```
bool Log::Registrar (
    const std::string & mensagem) const
```

Acrescenta uma mensagem ao arquivo de log.

Parameters

<i>mensagem</i>	Texto a registrar (limite de 500 caracteres).
-----------------	---

Returns

true se o registro foi bem-sucedido.

Note

Implementa US-019 (Registro de informações no log).

Definition at line 9 of file [Log.cpp](#).

11.61.3.3 SetArquivoLog()

```
bool Log::SetArquivoLog (
    const std::string & arquivo)
```

Redireciona o log para um novo arquivo.

Parameters

<i>arquivo</i>	Novo caminho de arquivo.
----------------	--------------------------

Returns

true se a alteração foi aceita.

Definition at line 27 of file [Log.cpp](#).

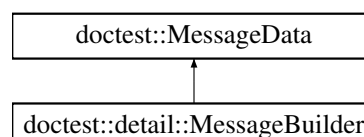
The documentation for this class was generated from the following files:

- [include/Log.hpp](#)
- [src/Log.cpp](#)

11.62 doctest::detail::MessageBuilder Struct Reference

```
#include <doctest.h>
```

Inheritance diagram for doctest::detail::MessageBuilder:



Public Member Functions

- [MessageBuilder](#) (const char *file, int line, [assertType::Enum](#) severity)
- [MessageBuilder](#) (const MessageBuilder &)=delete
- [MessageBuilder](#) (MessageBuilder &&)=delete
- [MessageBuilder](#) & [operator=](#) (const [MessageBuilder](#) &)=delete
- [MessageBuilder](#) & [operator=](#) ([MessageBuilder](#) &&)=delete
- [~MessageBuilder](#) () noexcept(false)
- template<typename T>
 [MessageBuilder](#) & [operator,](#) (const T &in)
- template<typename T>
 [DOctest_MSVC_SUPPRESS_WARNING_POP](#) [MessageBuilder](#) & [operator<<](#) (const T &in)
- template<typename T>
 [MessageBuilder](#) & [operator*](#) (const T &in)
- bool [log](#) ()
- void [react](#) ()

Public Attributes

- [std::ostream](#) * [m_stream](#)
- bool [logged](#) = false

Public Attributes inherited from [doctest::MessageData](#)

- [String](#) [m_string](#)
- const char * [m_file](#)
- int [m_line](#)
- [assertType::Enum](#) [m_severity](#)

11.62.1 Detailed Description

Definition at line 2487 of file [doctest.h](#).

11.62.2 Constructor & Destructor Documentation

11.62.2.1 MessageBuilder() [1/3]

```
doctest::detail::MessageBuilder::MessageBuilder (
    const char * file,
    int line,
    assertType::Enum severity)
```

11.62.2.2 MessageBuilder() [2/3]

```
doctest::detail::MessageBuilder::MessageBuilder (
    const MessageBuilder & ) [delete]
```

11.62.2.3 MessageBuilder() [3/3]

```
doctest::detail::MessageBuilder::MessageBuilder (
    MessageBuilder && ) [delete]
```

11.62.2.4 ~MessageBuilder()

```
doctest::detail::MessageBuilder::~~MessageBuilder ()
```

11.62.3 Member Function Documentation

11.62.3.1 log()

```
bool doctest::detail::MessageBuilder::log ()
```

11.62.3.2 operator*()

```
template<typename T>
MessageBuilder & doctest::detail::MessageBuilder::operator* (
    const T & in) [inline]
```

Definition at line 2521 of file [doctest.h](#).

11.62.3.3 operator,()

```
template<typename T>
MessageBuilder & doctest::detail::MessageBuilder::operator, (
    const T & in) [inline]
```

Definition at line 2504 of file [doctest.h](#).

11.62.3.4 operator<<()

```
template<typename T>
DOctest_MSVC_SUPPRESS_WARNING_POP MessageBuilder & doctest::detail::MessageBuilder::operator<<
(
    const T & in) [inline]
```

Definition at line 2512 of file [doctest.h](#).

11.62.3.5 operator=() [1/2]

```
MessageBuilder & doctest::detail::MessageBuilder::operator= (
    const MessageBuilder & ) [delete]
```

11.62.3.6 operator=() [2/2]

```
MessageBuilder & doctest::detail::MessageBuilder::operator= (
    MessageBuilder && ) [delete]
```

11.62.3.7 react()

```
void doctest::detail::MessageBuilder::react ()
```

11.62.4 Member Data Documentation

11.62.4.1 logged

```
bool doctest::detail::MessageBuilder::logged = false
```

Definition at line 2489 of file [doctest.h](#).

11.62.4.2 m_stream

```
std::ostream* doctest::detail::MessageBuilder::m_stream
```

Definition at line 2488 of file [doctest.h](#).

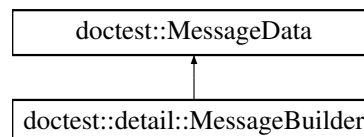
The documentation for this struct was generated from the following file:

- [tests/doctest.h](#)

11.63 doctest::MessageData Struct Reference

```
#include <doctest.h>
```

Inheritance diagram for doctest::MessageData:



Public Attributes

- [String m_string](#)
- [const char * m_file](#)
- [int m_line](#)
- [assertType::Enum m_severity](#)

11.63.1 Detailed Description

Definition at line 2477 of file [doctest.h](#).

11.63.2 Member Data Documentation

11.63.2.1 m_file

```
const char* doctest::MessageData::m_file
```

Definition at line 2479 of file [doctest.h](#).

11.63.2.2 m_line

```
int doctest::MessageData::m_line
```

Definition at line 2480 of file [doctest.h](#).

11.63.2.3 m_severity

```
assertType::Enum doctest::MessageData::m_severity
```

Definition at line 2481 of file [doctest.h](#).

11.63.2.4 m_string

```
String doctest::MessageData::m_string
```

Definition at line 2478 of file [doctest.h](#).

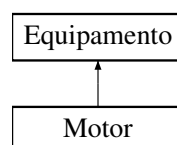
The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

11.64 Motor Class Reference

```
#include <Motor.hpp>
```

Inheritance diagram for Motor:



Public Member Functions

- [Motor](#) (int id, std::string desc, double velocidade, double torque)
- [~Motor](#) () override=default
- std::string [Tipo](#) () const override
Tipo concreto do equipamento (sobrescrito pelas especializações).

Public Member Functions inherited from [Equipamento](#)

- [Equipamento](#) (int id, std::string desc, std::string nome)
Constrói um equipamento.
- virtual [~Equipamento](#) ()=default
Destrutor virtual: permite uso polimórfico via ponteiro de base.
- void [AdicionarParametro](#) (double ValorAtual, std::string Desc, int id)
Adiciona um parâmetro sem limites definidos.
- int [AdicionarParametro](#) (double ValorAtual, std::string Desc)
Adiciona um parâmetro sem limites com ID gerado automaticamente.
- void [AdicionarParametro](#) (double ValorAtual, double Max, double Min, std::string Desc, int id)
Adiciona um parâmetro com limites Min/Max definidos.
- int [AdicionarParametro](#) (double ValorAtual, double Max, double Min, std::string Desc)
Adiciona um parâmetro com limites Min/Max e ID gerado automaticamente.
- int [ProximoldParametro](#) () const
- void [RemoverParametro](#) (int id)
Remove o parâmetro com o ID fornecido.
- std::vector< int > [DiagnosticarEquipamento](#) () const
Diagnostica todos os parâmetros do equipamento.
- void [AtualizarAlarmes](#) ()
Atualiza o estado de todos os alarmes do equipamento.
- void [ExibirParametros](#) () const
Exibe no console os parâmetros e seus valores atuais.
- void [ExibirAlarmes](#) () const
Exibe no console os alarmes ativos do equipamento.
- int [QuantidadeParametros](#) () const
- int [QuantidadeAlarmes](#) () const
- int [GetId](#) () const
- std::string [GetDesc](#) () const
- std::string [GetNome](#) () const
- const std::map< int, [Parametro](#) > & [GetParametros](#) () const
- const std::vector< [Alarme](#) > & [GetAlarmes](#) () const
- void [SetDesc](#) (std::string desc)
Atualiza a descrição do equipamento.

11.64.1 Detailed Description

Definition at line 6 of file [Motor.hpp](#).

11.64.2 Constructor & Destructor Documentation

11.64.2.1 Motor()

```
Motor::Motor (
    int id,
    std::string desc,
    double velocidade,
    double torque)
```

Definition at line 3 of file [Motor.cpp](#).

11.64.2.2 ~Motor()

```
Motor::~Motor () [override], [default]
```

11.64.3 Member Function Documentation

11.64.3.1 Tipo()

```
std::string Motor::Tipo () const [override], [virtual]
```

Tipo concreto do equipamento (sobrescrito pelas especializações).

Returns

Nome do tipo (ex.: "Equipamento", "Motor", "Sensor").

Reimplemented from [Equipamento](#).

Definition at line 9 of file [Motor.cpp](#).

The documentation for this class was generated from the following files:

- [include/Motor.hpp](#)
- [src/Motor.cpp](#)

11.65 Parametro Class Reference

Representa um parâmetro monitorado de um equipamento.

```
#include <Parametro.hpp>
```


Public Member Functions

- [Parametro](#) (double ValorAtual, std::string Desc, int id)
Constrói um parâmetro sem limites definidos.
- [Parametro](#) (double ValorAtual, double Max, double Min, std::string Desc, int id)
Constrói um parâmetro com limites Min/Max definidos.
- bool [ValorValido](#) (double Valor) const
Verifica se um valor está dentro dos limites configurados.
- void [SetValorAtual](#) (double Valor)
Atualiza o valor atual e registra no histórico.
- void [SetMax](#) (double Valor)
Define o limite superior.
- void [SetMin](#) (double Valor)
Define o limite inferior.
- void [SetDesc](#) (std::string Desc)
Atualiza a descrição do parâmetro.
- bool [DiagnosticarParametro](#) ()
Verifica se o valor atual viola os limites e atualiza o estado.
- double [GetValorAtual](#) () const
- double [GetMax](#) () const
- double [GetMin](#) () const
- std::string [GetDesc](#) () const
- bool [GetEstado](#) () const
- const std::vector< double > & [GetHist](#) () const
- int [GetId](#) ()
- void [ResetarLimites](#) ()
Restaura os limites Min/Max para os valores padrão (sem restrição).
- void [ResetarHistorico](#) ()
Limpa o histórico de leituras.

11.65.1 Detailed Description

Representa um parâmetro monitorado de um equipamento.

Armazena um valor numérico em tempo real, limites aceitáveis (Min/Max), histórico de leituras e um estado de diagnóstico.

Definition at line 13 of file [Parametro.hpp](#).

11.65.2 Constructor & Destructor Documentation

11.65.2.1 Parametro() [1/2]

```
Parametro::Parametro (  
    double ValorAtual,  
    std::string Desc,  
    int id)
```

Constrói um parâmetro sem limites definidos.

Parameters

<i>ValorAtual</i>	Leitura inicial do parâmetro.
<i>Desc</i>	Descrição do parâmetro.
<i>id</i>	Identificador único.

Definition at line 6 of file [Parametro.cpp](#).

11.65.2.2 Parametro() [2/2]

```
Parametro::Parametro (  
    double ValorAtual,  
    double Max,  
    double Min,  
    std::string Desc,  
    int id)
```

Constrói um parâmetro com limites Min/Max definidos.

Parameters

<i>ValorAtual</i>	Leitura inicial do parâmetro.
<i>Max</i>	Limite superior aceitável.
<i>Min</i>	Limite inferior aceitável.
<i>Desc</i>	Descrição do parâmetro.
<i>id</i>	Identificador único.

Note

Implementa US-018 (Configurar parâmetros de equipamentos).

Definition at line 26 of file [Parametro.cpp](#).

11.65.3 Member Function Documentation**11.65.3.1 DiagnosticarParametro()**

```
bool Parametro::DiagnosticarParametro ()
```

Verifica se o valor atual viola os limites e atualiza o estado.

Returns

true se o parâmetro estiver fora dos limites (em falha).

Note

Implementa US-011 (Diagnóstico de [Equipamento](#)).

Definition at line 79 of file [Parametro.cpp](#).

11.65.3.2 GetDesc()

```
std::string Parametro::GetDesc () const
```

Returns

Descrição do parâmetro.

Definition at line 96 of file [Parametro.cpp](#).

11.65.3.3 GetEstado()

```
bool Parametro::GetEstado () const
```

Returns

true se o parâmetro estiver em estado de falha.

Definition at line 100 of file [Parametro.cpp](#).

11.65.3.4 GetHist()

```
const std::vector< double > & Parametro::GetHist () const
```

Returns

Histórico de leituras anteriores.

Note

Relacionado a US-010 (Leitura do [Equipamento](#)).

Definition at line 104 of file [Parametro.cpp](#).

11.65.3.5 GetId()

```
int Parametro::GetId ()
```

Returns

Identificador único do parâmetro.

Definition at line 108 of file [Parametro.cpp](#).

11.65.3.6 GetMax()

```
double Parametro::GetMax () const
```

Returns

Limite máximo configurado.

Definition at line 88 of file [Parametro.cpp](#).

11.65.3.7 GetMin()

```
double Parametro::GetMin () const
```

Returns

Limite mínimo configurado.

Definition at line 92 of file [Parametro.cpp](#).

11.65.3.8 GetValorAtual()

```
double Parametro::GetValorAtual () const
```

Returns

Valor atual do parâmetro.

Definition at line 84 of file [Parametro.cpp](#).

11.65.3.9 ResetarHistorico()

```
void Parametro::ResetarHistorico ()
```

Limpa o histórico de leituras.

Definition at line 117 of file [Parametro.cpp](#).

11.65.3.10 ResetarLimites()

```
void Parametro::ResetarLimites ()
```

Restaura os limites Min/Max para os valores padrão (sem restrição).

Definition at line 112 of file [Parametro.cpp](#).

11.65.3.11 SetDesc()

```
void Parametro::SetDesc (  
    std::string Desc)
```

Atualiza a descrição do parâmetro.

Parameters

<i>Desc</i>	Nova descrição.
-------------	-----------------

Definition at line 72 of file [Parametro.cpp](#).

11.65.3.12 SetMax()

```
void Parametro::SetMax (  
    double Valor)
```

Define o limite superior.

Parameters

<i>Valor</i>	Novo limite máximo.
--------------	---------------------

Note

Implementa US-018 (Configurar parâmetros de equipamentos).

Definition at line 58 of file [Parametro.cpp](#).

11.65.3.13 SetMin()

```
void Parametro::SetMin (  
    double Valor)
```

Define o limite inferior.

Parameters

<i>Valor</i>	Novo limite mínimo.
--------------	---------------------

Note

Implementa US-018 (Configurar parâmetros de equipamentos).

Definition at line 65 of file [Parametro.cpp](#).

11.65.3.14 SetValorAtual()

```
void Parametro::SetValorAtual (
    double Valor)
```

Atualiza o valor atual e registra no histórico.

Parameters

<i>Valor</i>	Novo valor lido.
--------------	------------------

Note

Implementa US-010 (Leitura do [Equipamento](#)).

Definition at line 53 of file [Parametro.cpp](#).

11.65.3.15 ValorValido()

```
bool Parametro::ValorValido (
    double Valor) const
```

Verifica se um valor está dentro dos limites configurados.

Parameters

<i>Valor</i>	Valor a verificar.
--------------	--------------------

Returns

true se o valor estiver dentro dos limites.

Definition at line 49 of file [Parametro.cpp](#).

The documentation for this class was generated from the following files:

- [include/Parametro.hpp](#)
- [src/Parametro.cpp](#)

11.66 doctest::QueryData Struct Reference

```
#include <doctest.h>
```

Public Attributes

- const [TestRunStats](#) * [run_stats](#) = nullptr
- const [TestCaseData](#) ** [data](#) = nullptr
- unsigned [num_data](#) = 0

11.66.1 Detailed Description

Definition at line 2768 of file [doctest.h](#).

11.66.2 Member Data Documentation

11.66.2.1 data

```
const TestCaseData** doctest::QueryData::data = nullptr
```

Definition at line 2770 of file [doctest.h](#).

11.66.2.2 num_data

```
unsigned doctest::QueryData::num_data = 0
```

Definition at line 2771 of file [doctest.h](#).

11.66.2.3 run_stats

```
const TestRunStats* doctest::QueryData::run_stats = nullptr
```

Definition at line 2769 of file [doctest.h](#).

The documentation for this struct was generated from the following file:

- [tests/doctest.h](#)

11.67 doctest::detail::RelationalComparator< int, L, R > Struct Template Reference

```
#include <doctest.h>
```

Public Member Functions

- bool [operator\(\)](#) (const DOCTEST_REF_WRAP(L), const DOCTEST_REF_WRAP(R)) const

11.67.1 Detailed Description

```
template<int, class L, class R>  
struct doctest::detail::RelationalComparator< int, L, R >
```

Definition at line 1772 of file [doctest.h](#).

11.67.2 Member Function Documentation

11.67.2.1 operator>()

```
template<int, class L, class R>
bool doctest::detail::RelationalComparator< int, L, R >::operator() (
    const DOCTEST_REF_WRAPL,
    const DOCTEST_REF_WRAPR) const [inline]
```

Definition at line 1772 of file [doctest.h](#).

The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

11.68 doctest::detail::types::remove_const< T > Struct Template Reference

```
#include <doctest.h>
```

Public Types

- using [type](#) = T

11.68.1 Detailed Description

```
template<typename T>
struct doctest::detail::types::remove_const< T >
```

Definition at line 772 of file [doctest.h](#).

11.68.2 Member Typedef Documentation

11.68.2.1 type

```
template<typename T>
using doctest::detail::types::remove_const< T >::type = T
```

Definition at line 773 of file [doctest.h](#).

The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

11.69 doctest::detail::types::remove_const< const T > Struct Template Reference

```
#include <doctest.h>
```

Public Types

- using [type](#) = T

11.69.1 Detailed Description

```
template<typename T>  
struct doctest::detail::types::remove_const< const T >
```

Definition at line [777](#) of file [doctest.h](#).

11.69.2 Member Typedef Documentation

11.69.2.1 type

```
template<typename T>  
using doctest::detail::types::remove\_const< const T >::type = T
```

Definition at line [778](#) of file [doctest.h](#).

The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

11.70 doctest::detail::types::remove_reference< T > Struct Template Reference

```
#include <doctest.h>
```

Public Types

- using [type](#) = T

11.70.1 Detailed Description

```
template<typename T>  
struct doctest::detail::types::remove_reference< T >
```

Definition at line [745](#) of file [doctest.h](#).

11.70.2 Member Typedef Documentation

11.70.2.1 type

```
template<typename T>
using doctest::detail::types::remove_reference< T >::type = T
```

Definition at line 746 of file [doctest.h](#).

The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

11.71 doctest::detail::types::remove_reference< T & > Struct Template Reference

```
#include <doctest.h>
```

Public Types

- using [type](#) = T

11.71.1 Detailed Description

```
template<typename T>
struct doctest::detail::types::remove_reference< T & >
```

Definition at line 750 of file [doctest.h](#).

11.71.2 Member Typedef Documentation

11.71.2.1 type

```
template<typename T>
using doctest::detail::types::remove_reference< T & >::type = T
```

Definition at line 751 of file [doctest.h](#).

The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

11.72 doctest::detail::types::remove_reference< T && > Struct Template Reference

```
#include <doctest.h>
```

Public Types

- using [type](#) = T

11.72.1 Detailed Description

```
template<typename T>
struct doctest::detail::types::remove_reference< T && >
```

Definition at line [755](#) of file [doctest.h](#).

11.72.2 Member Typedef Documentation

11.72.2.1 type

```
template<typename T>
using doctest::detail::types::remove\_reference< T && >::type = T
```

Definition at line [756](#) of file [doctest.h](#).

The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

11.73 doctest::detail::Result Struct Reference

```
#include <doctest.h>
```

Public Member Functions

- [Result](#) ()=default
- [Result](#) (bool passed, const [String](#) &decomposition=[String](#)())

Public Attributes

- bool [m_passed](#)
- [String](#) [m_decomp](#)

11.73.1 Detailed Description

Definition at line [1814](#) of file [doctest.h](#).

11.73.2 Constructor & Destructor Documentation

11.73.2.1 Result() [1/2]

```
doctest::detail::Result::Result () [default]
```

11.73.2.2 Result() [2/2]

```
doctest::detail::Result::Result (
    bool passed,
    const String & decomposition = String())
```

11.73.3 Member Data Documentation

11.73.3.1 m_decomp

```
String doctest::detail::Result::m_decomp
```

Definition at line [1816](#) of file [doctest.h](#).

11.73.3.2 m_passed

```
bool doctest::detail::Result::m_passed
```

Definition at line [1815](#) of file [doctest.h](#).

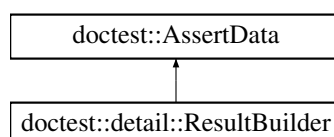
The documentation for this struct was generated from the following file:

- [tests/doctest.h](#)

11.74 doctest::detail::ResultBuilder Struct Reference

```
#include <doctest.h>
```

Inheritance diagram for doctest::detail::ResultBuilder:



Public Member Functions

- [ResultBuilder](#) ([assertType::Enum](#) at, const char *file, int line, const char *expr, const char *exception_type="", const [String](#) &exception_string="")
- [ResultBuilder](#) ([assertType::Enum](#) at, const char *file, int line, const char *expr, const char *exception_type, const [Contains](#) &exception_string)
- void [setResult](#) (const [Result](#) &res)
- template<int comparison, typename L, typename R>
[DOCTEST_NOINLINE](#) bool [binary_assert](#) (const [DOCTEST_REF_WRAP](#)(L) lhs, const [DOCTEST_REF_WRAP](#)(R) rhs)
- template<typename L>
[DOCTEST_NOINLINE](#) bool [unary_assert](#) (const [DOCTEST_REF_WRAP](#)(L) val)
- void [translateException](#) ()
- bool [log](#) ()
- void [react](#) () const

Public Member Functions inherited from [doctest::AssertData](#)

- [AssertData](#) ([assertType::Enum](#) at, const char *file, int line, const char *expr, const char *exception_type, const [StringContains](#) &exception_string)

Additional Inherited Members

Public Attributes inherited from [doctest::AssertData](#)

- const [TestCaseData](#) * [m_test_case](#)
- [assertType::Enum](#) [m_at](#)
- const char * [m_file](#)
- int [m_line](#)
- const char * [m_expr](#)
- bool [m_failed](#)
- bool [m_threw](#)
- [String](#) [m_exception](#)
- [String](#) [m_decomp](#)
- bool [m_threw_as](#)
- const char * [m_exception_type](#)
- class [DOCTEST_INTERFACE](#) [doctest::AssertData::StringContains](#) [m_exception_string](#)

11.74.1 Detailed Description

Definition at line 1847 of file [doctest.h](#).

11.74.2 Constructor & Destructor Documentation

11.74.2.1 ResultBuilder() [1/2]

```
doctest::detail::ResultBuilder::ResultBuilder (
    assertType::Enum at,
    const char * file,
    int line,
    const char * expr,
    const char * exception_type = "",
    const String & exception_string = "")
```

11.74.2.2 ResultBuilder() [2/2]

```
doctest::detail::ResultBuilder::ResultBuilder (
    assertType::Enum at,
    const char * file,
    int line,
    const char * expr,
    const char * exception_type,
    const Contains & exception_string)
```

11.74.3 Member Function Documentation

11.74.3.1 binary_assert()

```
template<int comparison, typename L, typename R>
DOCTEST_NOINLINE bool doctest::detail::ResultBuilder::binary_assert (
    const DOCTEST_REF_WRAP(L) lhs,
    const DOCTEST_REF_WRAP(R) rhs) [inline]
```

Definition at line 1869 of file [doctest.h](#).

11.74.3.2 log()

```
bool doctest::detail::ResultBuilder::log ()
```

11.74.3.3 react()

```
void doctest::detail::ResultBuilder::react () const
```

11.74.3.4 setResult()

```
void doctest::detail::ResultBuilder::setResult (
    const Result & res)
```

11.74.3.5 translateException()

```
void doctest::detail::ResultBuilder::translateException ()
```

11.74.3.6 unary_assert()

```
template<typename L>
DOCTEST_NOINLINE bool doctest::detail::ResultBuilder::unary_assert (
    const DOCTEST_REF_WRAP(L) val) [inline]
```

Definition at line 1878 of file [doctest.h](#).

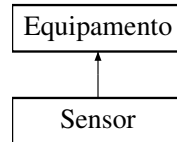
The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

11.75 Sensor Class Reference

```
#include <Sensor.hpp>
```

Inheritance diagram for Sensor:



Public Member Functions

- [Sensor](#) (int id, std::string desc, double sinal)
- [~Sensor](#) () override=default
- std::string [Tipo](#) () const override
Tipo concreto do equipamento (sobrescrito pelas especializações).

Public Member Functions inherited from [Equipamento](#)

- [Equipamento](#) (int id, std::string desc, std::string nome)
Constrói um equipamento.
- virtual [~Equipamento](#) ()=default
Destrutor virtual: permite uso polimórfico via ponteiro de base.
- void [AdicionarParametro](#) (double ValorAtual, std::string Desc, int id)
Adiciona um parâmetro sem limites definidos.
- int [AdicionarParametro](#) (double ValorAtual, std::string Desc)
Adiciona um parâmetro sem limites com ID gerado automaticamente.
- void [AdicionarParametro](#) (double ValorAtual, double Max, double Min, std::string Desc, int id)
Adiciona um parâmetro com limites Min/Max definidos.
- int [AdicionarParametro](#) (double ValorAtual, double Max, double Min, std::string Desc)
Adiciona um parâmetro com limites Min/Max e ID gerado automaticamente.
- int [ProximoldParametro](#) () const
- void [RemoverParametro](#) (int id)
Remove o parâmetro com o ID fornecido.
- std::vector< int > [DiagnosticarEquipamento](#) () const
Diagnostica todos os parâmetros do equipamento.
- void [AtualizarAlarmes](#) ()
Atualiza o estado de todos os alarmes do equipamento.
- void [ExibirParametros](#) () const
Exibe no console os parâmetros e seus valores atuais.
- void [ExibirAlarmes](#) () const
Exibe no console os alarmes ativos do equipamento.
- int [QuantidadeParametros](#) () const
- int [QuantidadeAlarmes](#) () const
- int [GetId](#) () const
- std::string [GetDesc](#) () const
- std::string [GetNome](#) () const
- const std::map< int, [Parametro](#) > & [GetParametros](#) () const
- const std::vector< [Alarme](#) > & [GetAlarmes](#) () const
- void [SetDesc](#) (std::string desc)
Atualiza a descrição do equipamento.

11.75.1 Detailed Description

Definition at line 6 of file [Sensor.hpp](#).

11.75.2 Constructor & Destructor Documentation

11.75.2.1 Sensor()

```
Sensor::Sensor (  
    int id,  
    std::string desc,  
    double sinal)
```

Definition at line 3 of file [Sensor.cpp](#).

11.75.2.2 ~Sensor()

```
Sensor::~Sensor () [override], [default]
```

11.75.3 Member Function Documentation

11.75.3.1 Tipo()

```
std::string Sensor::Tipo () const [override], [virtual]
```

Tipo concreto do equipamento (sobrescrito pelas especializações).

Returns

Nome do tipo (ex.: "Equipamento", "Motor", "Sensor").

Reimplemented from [Equipamento](#).

Definition at line 8 of file [Sensor.cpp](#).

The documentation for this class was generated from the following files:

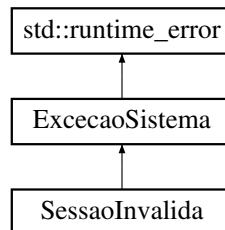
- include/[Sensor.hpp](#)
- src/[Sensor.cpp](#)

11.76 SessaoInvalida Class Reference

Operação que exige sessão ativa sem usuário logado.

```
#include <Excecoes.hpp>
```

Inheritance diagram for SessaoInvalida:



11.76.1 Detailed Description

Operação que exige sessão ativa sem usuário logado.

Definition at line 55 of file [Excecoes.hpp](#).

The documentation for this class was generated from the following file:

- include/[Excecoes.hpp](#)

11.77 doctest::detail::should_stringify_as_underlying_type< T > Struct Template Reference

```
#include <doctest.h>
```

Static Public Attributes

- static [DOCTEST_CONSTEXPR](#) bool [value](#)

11.77.1 Detailed Description

```
template<typename T>  
struct doctest::detail::should_stringify_as_underlying_type< T >
```

Definition at line 1038 of file [doctest.h](#).

11.77.2 Member Data Documentation

11.77.2.1 value

```
template<typename T>
DOCTEST_CONSTEXPR bool doctest::detail::should_stringify_as_underlying_type< T >::value [static]
```

Initial value:

```
=
detail::types::is_enum<T>::value && !doctest::detail::has_insertion_operator<T>::value
```

Definition at line 1039 of file [doctest.h](#).

The documentation for this struct was generated from the following file:

- [tests/doctest.h](#)

11.78 Sistema Class Reference

Ponto de entrada central do sistema de monitoramento.

```
#include <Sistema.hpp>
```

Public Member Functions

- [Sistema](#) ()
Inicializa o sistema sem linhas, usuários ou sessão ativa.
- void [AdicionarLinha](#) (int id, std::string desc, std::string local, std::string nome)
Cadastra uma nova linha de produção.
- int [AdicionarLinha](#) (std::string desc, std::string local, std::string nome)
Cadastra uma linha com ID gerado automaticamente (incremental).
- void [RemoverLinha](#) (int id)
Remove a linha de produção com o ID fornecido.
- const std::map< int, [LinhaDeProducao](#) > & [GetLinhas](#) () const
- [LinhaDeProducao](#) & [AcessarLinha](#) (int id)
Acesso mutável a uma linha pelo ID.
- void [ExibirLinhas](#) ()
Exibe no console todas as linhas de produção cadastradas.
- void [AdicionarUsuario](#) (std::string login, std::string senha, [Cargo](#) cargo)
Cadastra um novo usuário no sistema.
- void [RemoverUsuario](#) (std::string login)
Remove o usuário identificado pelo login.
- int [ProximoidDisponivel](#) () const
Retorna o próximo ID disponível para cadastro (monotônico).
- bool [IdAtivo](#) (int Id) const
Verifica se um ID já está em uso (existe uma linha cadastrada com ele).
- bool [Login](#) (std::string login, std::string senha)
Autentica um usuário e inicia a sessão.
- bool [Logout](#) ()
Encerra a sessão do usuário atual.
- [Usuario](#) * [GetUsuarioLogado](#) () const
- bool [SalvarAlteracoes](#) ()
Persiste o estado atual do sistema em arquivo.
- bool [CarregarUltimoSave](#) ()
Carrega o último estado salvo do sistema.
- bool [CarregarSave](#) (std::string arquivo)
Carrega o estado do sistema a partir de um arquivo específico.

11.78.1 Detailed Description

Ponto de entrada central do sistema de monitoramento.

Gerencia o conjunto de linhas de produção, os usuários cadastrados, o controle de sessão (login/logout) e a persistência dos dados.

Definition at line 18 of file [Sistema.hpp](#).

11.78.2 Constructor & Destructor Documentation

11.78.2.1 Sistema()

```
Sistema::Sistema ()
```

Inicializa o sistema sem linhas, usuários ou sessão ativa.

Definition at line 10 of file [Sistema.cpp](#).

11.78.3 Member Function Documentation

11.78.3.1 AcessarLinha()

```
LinhaDeProducao & Sistema::AcessarLinha (  
    int id)
```

Acesso mutável a uma linha pelo ID.

Parameters

<i>id</i>	Identificador da linha.
-----------	-------------------------

Returns

Referência à linha.

Exceptions

<i>IdInexistente</i>	se não houver linha com o ID.
----------------------	-------------------------------

Definition at line 43 of file [Sistema.cpp](#).

11.78.3.2 AdicionarLinha() [1/2]

```
void Sistema::AdicionarLinha (
    int id,
    std::string desc,
    std::string local,
    std::string nome)
```

Cadastra uma nova linha de produção.

Parameters

<i>id</i>	Identificador único da linha.
<i>desc</i>	Descrição da linha.
<i>local</i>	Localização física.
<i>nome</i>	Nome da linha.

Note

Implementa US-001 (Criar Linhas de Produção).

Definition at line 14 of file [Sistema.cpp](#).

11.78.3.3 AdicionarLinha() [2/2]

```
int Sistema::AdicionarLinha (
    std::string desc,
    std::string local,
    std::string nome)
```

Cadastra uma linha com ID gerado automaticamente (incremental).

Returns

ID atribuído à linha criada.

Definition at line 26 of file [Sistema.cpp](#).

11.78.3.4 AdicionarUsuario()

```
void Sistema::AdicionarUsuario (
    std::string login,
    std::string senha,
    Cargo cargo)
```

Cadastra um novo usuário no sistema.

Parameters

<i>login</i>	Login de acesso.
<i>senha</i>	Senha do usuário.
<i>cargo</i>	Cargo/nível de acesso.

Note

Implementa US-015 (Criar usuários).

Definition at line 60 of file [Sistema.cpp](#).

11.78.3.5 CarregarSave()

```
bool Sistema::CarregarSave (
    std::string arquivo)
```

Carrega o estado do sistema a partir de um arquivo específico.

Parameters

<i>arquivo</i>	Caminho do arquivo de save.
----------------	-----------------------------

Returns

true se o carregamento foi bem-sucedido.

Note

Implementa US-021 (Carregar dados).

Definition at line 203 of file [Sistema.cpp](#).

11.78.3.6 CarregarUltimoSave()

```
bool Sistema::CarregarUltimoSave ()
```

Carrega o último estado salvo do sistema.

Returns

true se o carregamento foi bem-sucedido.

Note

Implementa US-021 (Carregar dados).

Definition at line 199 of file [Sistema.cpp](#).

11.78.3.7 ExibirLinhas()

```
void Sistema::ExibirLinhas ()
```

Exibe no console todas as linhas de produção cadastradas.

Note

Implementa US-003 (Acessar Linhas de Produção).

Definition at line 51 of file [Sistema.cpp](#).

11.78.3.8 GetLinhas()

```
const std::map< int, LinhaDeProducao > & Sistema::GetLinhas () const
```

Returns

Referência constante ao mapa de linhas cadastradas.

Definition at line 39 of file [Sistema.cpp](#).

11.78.3.9 GetUsuarioLogado()

```
Usuario * Sistema::GetUsuarioLogado () const
```

Returns

Ponteiro para o usuário atualmente logado, ou nullptr se não houver sessão.

Definition at line 118 of file [Sistema.cpp](#).

11.78.3.10 IdAtivo()

```
bool Sistema::IdAtivo (  
    int Id) const
```

Verifica se um ID já está em uso (existe uma linha cadastrada com ele).

Parameters

<i>Id</i>	ID a verificar.
-----------	-----------------

Definition at line 89 of file [Sistema.cpp](#).

11.78.3.11 Login()

```
bool Sistema::Login (
    std::string login,
    std::string senha)
```

Autentica um usuário e inicia a sessão.

Parameters

<i>login</i>	Login do usuário.
<i>senha</i>	Senha do usuário.

Returns

true se as credenciais forem válidas.

Note

Implementa US-017 (Login de usuário).

Definition at line 93 of file [Sistema.cpp](#).

11.78.3.12 Logout()

```
bool Sistema::Logout ()
```

Encerra a sessão do usuário atual.

Returns

true se havia uma sessão ativa e foi encerrada com sucesso.

Note

Implementa US-022 (Logoff de usuário).

Definition at line 109 of file [Sistema.cpp](#).

11.78.3.13 ProximoIdDisponivel()

```
int Sistema::ProximoIdDisponivel () const
```

Retorna o próximo ID disponível para cadastro (monotônico).

Returns

Valor atual do contador interno (não consumido).

Definition at line 85 of file [Sistema.cpp](#).

11.78.3.14 RemoverLinha()

```
void Sistema::RemoverLinha (
    int id)
```

Remove a linha de produção com o ID fornecido.

Parameters

<i>id</i>	Identificador da linha a remover.
-----------	-----------------------------------

Note

Implementa US-002 (Remover Linhas de Produção).

Definition at line 32 of file [Sistema.cpp](#).

11.78.3.15 RemoverUsuario()

```
void Sistema::RemoverUsuario (
    std::string login)
```

Remove o usuário identificado pelo login.

Parameters

<i>login</i>	Login do usuário a remover.
--------------	-----------------------------

Note

Implementa US-016 (Remover usuários).

Definition at line 68 of file [Sistema.cpp](#).

11.78.3.16 SalvarAlteracoes()

```
bool Sistema::SalvarAlteracoes ()
```

Persiste o estado atual do sistema em arquivo.

Returns

true se o salvamento foi bem-sucedido.

Note

Implementa US-020 (Persistir dados).

Definition at line 156 of file [Sistema.cpp](#).

The documentation for this class was generated from the following files:

- [include/Sistema.hpp](#)
- [src/Sistema.cpp](#)

11.79 doctest::String Class Reference

```
#include <doctest.h>
```

Public Types

- using `size_type` = `DOCTEST_CONFIG_STRING_SIZE_TYPE`

Public Member Functions

- `String` () noexcept
- `~String` ()
- `String` (const char *in)
- `String` (const char *in, `size_type` in_size)
- `String` (std::istream &in, `size_type` in_size)
- template<typename T, typename `detail::types::enable_if`< `detail::is_std_string`< T >::value, bool >::type = true> `String` (const T &in)
- `String` (const String &other)
- `String` & `operator=` (const `String` &other)
- `String` & `operator+=` (const `String` &other)
- `String` (String &&other) noexcept
- `String` & `operator=` (`String` &&other) noexcept
- char `operator[]` (`size_type` i) const
- char & `operator[]` (`size_type` i)
- const char * `c_str` () const
- char * `c_str` ()
- `size_type` `size` () const
- `size_type` `capacity` () const
- `String` `substr` (`size_type` pos, `size_type` cnt=`npos`) &&
- `String` `substr` (`size_type` pos, `size_type` cnt=`npos`) const &
- `size_type` `find` (char ch, `size_type` pos=0) const
- `size_type` `rfind` (char ch, `size_type` pos=`npos`) const
- int `compare` (const char *other, bool no_case=false) const
- int `compare` (const `String` &other, bool no_case=false) const

Static Public Attributes

- static `DOCTEST_CONSTEXPR size_type` `npos` = `static_cast`<`size_type`>(-1)

Friends

- `DOCTEST_INTERFACE` std::ostream & `operator<<` (std::ostream &s, const `String` &in)

11.79.1 Detailed Description

Definition at line 895 of file `doctest.h`.

11.79.2 Member Typedef Documentation

11.79.2.1 size_type

using `doctest::String::size_type` = `DOCTEST_CONFIG_STRING_SIZE_TYPE`

Definition at line 897 of file `doctest.h`.

11.79.3 Constructor & Destructor Documentation

11.79.3.1 String() [1/7]

```
doctest::String::String () [noexcept]
```

11.79.3.2 ~String()

```
doctest::String::~String ()
```

11.79.3.3 String() [2/7]

```
doctest::String::String (  
    const char * in)
```

11.79.3.4 String() [3/7]

```
doctest::String::String (  
    const char * in,  
    size_type in_size)
```

11.79.3.5 String() [4/7]

```
doctest::String::String (  
    std::istream & in,  
    size_type in_size)
```

11.79.3.6 String() [5/7]

```
template<typename T, typename detail::types::enable_if< detail::is_std_string< T >::value,  
bool >::type = true>  
doctest::String::String (  
    const T & in) [inline]
```

Definition at line 940 of file `doctest.h`.

11.79.3.7 String() [6/7]

```
doctest::String::String (  
    const String & other)
```

11.79.3.8 String() [7/7]

```
doctest::String::String (  
    String && other) [noexcept]
```

11.79.4 Member Function Documentation

11.79.4.1 c_str() [1/2]

```
char * doctest::String::c_str () [inline]
```

Definition at line 960 of file [doctest.h](#).

11.79.4.2 c_str() [2/2]

```
const char * doctest::String::c_str () const [inline]
```

Definition at line 955 of file [doctest.h](#).

11.79.4.3 capacity()

```
size_type doctest::String::capacity () const
```

11.79.4.4 compare() [1/2]

```
int doctest::String::compare (  
    const char * other,  
    bool no_case = false) const
```

11.79.4.5 compare() [2/2]

```
int doctest::String::compare (  
    const String & other,  
    bool no_case = false) const
```

11.79.4.6 find()

```
size_type doctest::String::find (  
    char ch,  
    size_type pos = 0) const
```

11.79.4.7 operator+=()

```
String & doctest::String::operator+= (
    const String & other)
```

11.79.4.8 operator=() [1/2]

```
String & doctest::String::operator= (
    const String & other)
```

11.79.4.9 operator=() [2/2]

```
String & doctest::String::operator= (
    String && other) [noexcept]
```

11.79.4.10 operator[]() [1/2]

```
char & doctest::String::operator[] (
    size_type i)
```

11.79.4.11 operator[]() [2/2]

```
char doctest::String::operator[] (
    size_type i) const
```

11.79.4.12 rfind()

```
size_type doctest::String::rfind (
    char ch,
    size_type pos = npos) const
```

11.79.4.13 size()

```
size_type doctest::String::size () const
```

11.79.4.14 substr() [1/2]

```
String doctest::String::substr (
    size_type pos,
    size_type cnt = npos) &&
```

11.79.4.15 substr() [2/2]

```
String doctest::String::substr (
    size_type pos,
    size_type cnt = npos) const &
```

11.79.5 Friends And Related Symbol Documentation**11.79.5.1 operator<<**

```
DOCTEST_INTERFACE std::ostream & operator<< (
    std::ostream & s,
    const String & in) [friend]
```

11.79.6 Member Data Documentation**11.79.6.1 buf**

```
char doctest::String::buf[len]
```

Definition at line 911 of file [doctest.h](#).

11.79.6.2 data

```
view doctest::String::data
```

Definition at line 912 of file [doctest.h](#).

11.79.6.3 npos

```
DOCTEST_CONSTEXPR size_type doctest::String::npos = static_cast<size_type>(-1) [static]
```

Definition at line 929 of file [doctest.h](#).

The documentation for this class was generated from the following file:

- [tests/doctest.h](#)

11.80 doctest::AssertData::StringContains Class Reference

```
#include <doctest.h>
```

Public Member Functions

- [StringContains](#) (const [String](#) &str)
- [StringContains](#) ([Contains](#) cntn)
- bool [check](#) (const [String](#) &str)
- [operator const String & \(\)](#) const
- const char * [c_str](#) () const

11.80.1 Detailed Description

Definition at line 1659 of file [doctest.h](#).

11.80.2 Constructor & Destructor Documentation

11.80.2.1 StringContains() [1/2]

```
doctest::AssertData::StringContains::StringContains (  
    const String & str) [inline]
```

Definition at line 1665 of file [doctest.h](#).

11.80.2.2 StringContains() [2/2]

```
doctest::AssertData::StringContains::StringContains (  
    Contains cntn) [inline]
```

Definition at line 1668 of file [doctest.h](#).

11.80.3 Member Function Documentation

11.80.3.1 c_str()

```
const char * doctest::AssertData::StringContains::c_str () const [inline]
```

Definition at line 1679 of file [doctest.h](#).

11.80.3.2 check()

```
bool doctest::AssertData::StringContains::check (  
    const String & str) [inline]
```

Definition at line 1671 of file [doctest.h](#).

11.80.3.3 operator const String &()

doctest::AssertData::StringContains::operator const String & () const [inline]

Definition at line 1675 of file [doctest.h](#).

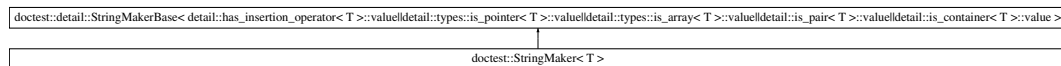
The documentation for this class was generated from the following file:

- tests/[doctest.h](#)

11.81 doctest::StringMaker< T > Struct Template Reference

```
#include <doctest.h>
```

Inheritance diagram for doctest::StringMaker< T >:



Additional Inherited Members

Static Public Member Functions inherited from [doctest::detail::StringMakerBase< detail::has_insertion_operator< T >::value|detail::types::is_pointer< T >::value|detail::types::is_array< T >::value|detail::is_pair< T >::value|detail::is_container< T >::value >](#)

- static [String convert](#) (const DOCTEST_REF_WRAP(T))

11.81.1 Detailed Description

```
template<typename T>
struct doctest::StringMaker< T >
```

Definition at line 1114 of file [doctest.h](#).

The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

11.82 doctest::detail::StringMakerBase< C > Struct Template Reference

```
#include <doctest.h>
```

Static Public Member Functions

- `template<typename T>`
static [String convert](#) (const `DOCTEST_REF_WRAP(T)`)

11.82.1 Detailed Description

```
template<bool C>
struct doctest::detail::StringMakerBase< C >
```

Definition at line 1071 of file [doctest.h](#).

11.82.2 Member Function Documentation

11.82.2.1 `convert()`

```
template<bool C>
template<typename T>
String doctest::detail::StringMakerBase< C >::convert (
    const DOCTEST_REF_WRAP(T)) [inline], [static]
```

Definition at line 1073 of file [doctest.h](#).

The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

11.83 `doctest::detail::StringMakerBase< true >` Struct Reference

```
#include <doctest.h>
```

Static Public Member Functions

- `template<typename T>`
static [String convert](#) (const `DOCTEST_REF_WRAP(T)` in)

11.83.1 Detailed Description

Definition at line 1104 of file [doctest.h](#).

11.83.2 Member Function Documentation

11.83.2.1 convert()

```
template<typename T>
String doctest::detail::StringMakerBase< true >::convert (
    const DOCTEST_REF_WRAP(T) in) [inline], [static]
```

Definition at line 1106 of file [doctest.h](#).

The documentation for this struct was generated from the following file:

- [tests/doctest.h](#)

11.84 doctest::detail::Subcase Struct Reference

```
#include <doctest.h>
```

Public Member Functions

- [Subcase](#) (const [String](#) &name, const char *file, int line)
- [Subcase](#) (const Subcase &)=delete
- [Subcase](#) (Subcase &&)=delete
- [Subcase](#) & [operator=](#) (const [Subcase](#) &)=delete
- [Subcase](#) & [operator=](#) ([Subcase](#) &&)=delete
- [~Subcase](#) ()
- [operator bool](#) () const

Public Attributes

- [SubcaseSignature](#) [m_signature](#)
- bool [m_entered](#) = false

11.84.1 Detailed Description

Definition at line 2129 of file [doctest.h](#).

11.84.2 Constructor & Destructor Documentation

11.84.2.1 Subcase() [1/3]

```
doctest::detail::Subcase::Subcase (
    const String & name,
    const char * file,
    int line)
```

11.84.2.2 Subcase() [2/3]

```
doctest::detail::Subcase::Subcase (
    const Subcase & ) [delete]
```

11.84.2.3 Subcase() [3/3]

```
doctest::detail::Subcase::Subcase (
    Subcase && ) [delete]
```

11.84.2.4 ~Subcase()

```
doctest::detail::Subcase::~~Subcase ()
```

11.84.3 Member Function Documentation

11.84.3.1 operator bool()

```
doctest::detail::Subcase::operator bool () const
```

11.84.3.2 operator=() [1/2]

```
Subcase & doctest::detail::Subcase::operator= (
    const Subcase & ) [delete]
```

11.84.3.3 operator=() [2/2]

```
Subcase & doctest::detail::Subcase::operator= (
    Subcase && ) [delete]
```

11.84.4 Member Data Documentation

11.84.4.1 m_entered

```
bool doctest::detail::Subcase::m_entered = false
```

Definition at line 2131 of file [doctest.h](#).

11.84.4.2 m_signature

```
SubcaseSignature doctest::detail::Subcase::m_signature
```

Definition at line 2130 of file [doctest.h](#).

The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

11.85 doctest::SubcaseSignature Struct Reference

```
#include <doctest.h>
```

Public Member Functions

- bool `operator==` (const [SubcaseSignature](#) &other) const
- bool `operator<` (const [SubcaseSignature](#) &other) const

Public Attributes

- [String](#) `m_name`
- const char * `m_file`
- int `m_line`

11.85.1 Detailed Description

Definition at line 2118 of file [doctest.h](#).

11.85.2 Member Function Documentation

11.85.2.1 `operator<()`

```
bool doctest::SubcaseSignature::operator< (  
    const SubcaseSignature & other) const
```

11.85.2.2 `operator==()`

```
bool doctest::SubcaseSignature::operator== (  
    const SubcaseSignature & other) const
```

11.85.3 Member Data Documentation

11.85.3.1 `m_file`

```
const char* doctest::SubcaseSignature::m_file
```

Definition at line 2120 of file [doctest.h](#).

11.85.3.2 `m_line`

```
int doctest::SubcaseSignature::m_line
```

Definition at line 2121 of file [doctest.h](#).

11.85.3.3 m_name

`String doctest::SubcaseSignature::m_name`

Definition at line 2119 of file [doctest.h](#).

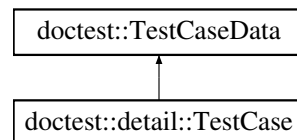
The documentation for this struct was generated from the following file:

- [tests/doctest.h](#)

11.86 doctest::detail::TestCase Struct Reference

```
#include <doctest.h>
```

Inheritance diagram for `doctest::detail::TestCase`:



Public Member Functions

- `TestCase` (`funcType` test, `const char *file`, `unsigned line`, `const TestSuite &test_suite`, `const String &type=String()`, `int template_id=-1`) `noexcept`
- `TestCase` (`const TestCase &other`) `noexcept`
- `TestCase` (`TestCase &&`)=`delete`
- `TestCase & operator=` (`const TestCase &other`) `noexcept`
- `DOctest_MSVC_SUPPRESS_WARNING_POP TestCase & operator=` (`TestCase &&`)=`delete`
- `TestCase & operator*` (`const char *in`) `noexcept`
- `template<typename T>`
`TestCase & operator*` (`const T &in`) `noexcept`
- `bool operator<` (`const TestCase &other`) `const noexcept`
- `~TestCase` ()=`default`

Public Attributes

- `funcType m_test`
- `String m_type`
- `int m_template_id`
- `String m_full_name`

Public Attributes inherited from [doctest::TestCaseData](#)

- [String](#) [m_file](#)
- unsigned [m_line](#)
- const char * [m_name](#)
- const char * [m_test_suite](#)
- const char * [m_description](#)
- bool [m_skip](#)
- bool [m_no_breaks](#)
- bool [m_no_output](#)
- bool [m_may_fail](#)
- bool [m_should_fail](#)
- int [m_expected_failures](#)
- double [m_timeout](#)

11.86.1 Detailed Description

Definition at line [2237](#) of file [doctest.h](#).

11.86.2 Constructor & Destructor Documentation

11.86.2.1 [TestCase\(\)](#) [1/3]

```
doctest::detail::TestCase::TestCase (
    funcType test,
    const char * file,
    unsigned line,
    const TestSuite & test_suite,
    const String & type = String(),
    int template_id = -1) [noexcept]
```

11.86.2.2 [TestCase\(\)](#) [2/3]

```
doctest::detail::TestCase::TestCase (
    const TestCase & other) [noexcept]
```

11.86.2.3 [TestCase\(\)](#) [3/3]

```
doctest::detail::TestCase::TestCase (
    TestCase && ) [delete]
```

11.86.2.4 [~TestCase\(\)](#)

```
doctest::detail::TestCase::~~TestCase () [default]
```

11.86.3 Member Function Documentation

11.86.3.1 `operator*()` [1/2]

```
TestCase & doctest::detail::TestCase::operator* (
    const char * in) [noexcept]
```

11.86.3.2 `operator*()` [2/2]

```
template<typename T>
TestCase & doctest::detail::TestCase::operator* (
    const T & in) [inline], [noexcept]
```

Definition at line 2266 of file [doctest.h](#).

11.86.3.3 `operator<()`

```
bool doctest::detail::TestCase::operator< (
    const TestCase & other) const [noexcept]
```

11.86.3.4 `operator=()` [1/2]

```
TestCase & doctest::detail::TestCase::operator= (
    const TestCase & other) [noexcept]
```

11.86.3.5 `operator=()` [2/2]

```
DOCTEST_MSVC_SUPPRESS_WARNING_POP TestCase & doctest::detail::TestCase::operator= (
    TestCase && ) [delete]
```

11.86.4 Member Data Documentation

11.86.4.1 `m_full_name`

```
String doctest::detail::TestCase::m_full_name
```

Definition at line 2243 of file [doctest.h](#).

11.86.4.2 `m_template_id`

```
int doctest::detail::TestCase::m_template_id
```

Definition at line 2241 of file [doctest.h](#).

11.86.4.3 m_test

`funcType doctest::detail::TestCase::m_test`

Definition at line 2238 of file [doctest.h](#).

11.86.4.4 m_type

`String doctest::detail::TestCase::m_type`

Definition at line 2240 of file [doctest.h](#).

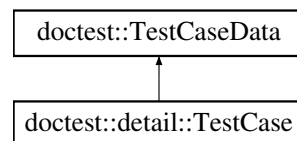
The documentation for this struct was generated from the following file:

- [tests/doctest.h](#)

11.87 doctest::TestCaseData Struct Reference

```
#include <doctest.h>
```

Inheritance diagram for doctest::TestCaseData:



Public Attributes

- [String m_file](#)
- unsigned [m_line](#)
- const char * [m_name](#)
- const char * [m_test_suite](#)
- const char * [m_description](#)
- bool [m_skip](#)
- bool [m_no_breaks](#)
- bool [m_no_output](#)
- bool [m_may_fail](#)
- bool [m_should_fail](#)
- int [m_expected_failures](#)
- double [m_timeout](#)

11.87.1 Detailed Description

Definition at line 2217 of file [doctest.h](#).

11.87.2 Member Data Documentation

11.87.2.1 m_description

```
const char* doctest::TestCaseData::m_description
```

Definition at line 2222 of file [doctest.h](#).

11.87.2.2 m_expected_failures

```
int doctest::TestCaseData::m_expected_failures
```

Definition at line 2228 of file [doctest.h](#).

11.87.2.3 m_file

```
String doctest::TestCaseData::m_file
```

Definition at line 2218 of file [doctest.h](#).

11.87.2.4 m_line

```
unsigned doctest::TestCaseData::m_line
```

Definition at line 2219 of file [doctest.h](#).

11.87.2.5 m_may_fail

```
bool doctest::TestCaseData::m_may_fail
```

Definition at line 2226 of file [doctest.h](#).

11.87.2.6 m_name

```
const char* doctest::TestCaseData::m_name
```

Definition at line 2220 of file [doctest.h](#).

11.87.2.7 m_no_breaks

```
bool doctest::TestCaseData::m_no_breaks
```

Definition at line 2224 of file [doctest.h](#).

11.87.2.8 m_no_output

```
bool doctest::TestCaseData::m_no_output
```

Definition at line 2225 of file [doctest.h](#).

11.87.2.9 m_should_fail

```
bool doctest::TestCaseData::m_should_fail
```

Definition at line 2227 of file [doctest.h](#).

11.87.2.10 m_skip

```
bool doctest::TestCaseData::m_skip
```

Definition at line 2223 of file [doctest.h](#).

11.87.2.11 m_test_suite

```
const char* doctest::TestCaseData::m_test_suite
```

Definition at line 2221 of file [doctest.h](#).

11.87.2.12 m_timeout

```
double doctest::TestCaseData::m_timeout
```

Definition at line 2229 of file [doctest.h](#).

The documentation for this struct was generated from the following file:

- [tests/doctest.h](#)

11.88 doctest::TestCaseException Struct Reference

```
#include <doctest.h>
```

Public Attributes

- [String error_string](#)
- [bool is_crash](#)

11.88.1 Detailed Description

Definition at line 2754 of file [doctest.h](#).

11.88.2 Member Data Documentation

11.88.2.1 error_string

```
String doctest::TestCaseException::error_string
```

Definition at line 2755 of file [doctest.h](#).

11.88.2.2 is_crash

```
bool doctest::TestCaseException::is_crash
```

Definition at line 2756 of file [doctest.h](#).

The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

11.89 doctest::detail::TestFailureException Struct Reference

```
#include <doctest.h>
```

11.89.1 Detailed Description

Definition at line 2563 of file [doctest.h](#).

The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

11.90 doctest::TestRunStats Struct Reference

```
#include <doctest.h>
```

Public Attributes

- unsigned [numTestCases](#)
- unsigned [numTestCasesPassingFilters](#)
- unsigned [numTestSuitesPassingFilters](#)
- unsigned [numTestCasesFailed](#)
- int [numAsserts](#)
- int [numAssertsFailed](#)

11.90.1 Detailed Description

Definition at line 2759 of file [doctest.h](#).

11.90.2 Member Data Documentation

11.90.2.1 numAsserts

```
int doctest::TestRunStats::numAsserts
```

Definition at line 2764 of file [doctest.h](#).

11.90.2.2 numAssertsFailed

```
int doctest::TestRunStats::numAssertsFailed
```

Definition at line 2765 of file [doctest.h](#).

11.90.2.3 numTestCases

```
unsigned doctest::TestRunStats::numTestCases
```

Definition at line 2760 of file [doctest.h](#).

11.90.2.4 numTestCasesFailed

```
unsigned doctest::TestRunStats::numTestCasesFailed
```

Definition at line 2763 of file [doctest.h](#).

11.90.2.5 numTestCasesPassingFilters

```
unsigned doctest::TestRunStats::numTestCasesPassingFilters
```

Definition at line 2761 of file [doctest.h](#).

11.90.2.6 numTestSuitesPassingFilters

```
unsigned doctest::TestRunStats::numTestSuitesPassingFilters
```

Definition at line 2762 of file [doctest.h](#).

The documentation for this struct was generated from the following file:

- [tests/doctest.h](#)

11.91 doctest::detail::TestSuite Struct Reference

```
#include <doctest.h>
```

Public Member Functions

- [TestSuite](#) & [operator*](#) (const char *in) noexcept
- template<typename T>
[TestSuite](#) & [operator*](#) (const T &in) noexcept

Public Attributes

- const char * [m_test_suite](#) = nullptr
- const char * [m_description](#) = nullptr
- bool [m_skip](#) = false
- bool [m_no_breaks](#) = false
- bool [m_no_output](#) = false
- bool [m_may_fail](#) = false
- bool [m_should_fail](#) = false
- int [m_expected_failures](#) = 0
- double [m_timeout](#) = 0

11.91.1 Detailed Description

Definition at line 2164 of file [doctest.h](#).

11.91.2 Member Function Documentation

11.91.2.1 [operator*\(\)](#) [1/2]

```
TestSuite & doctest::detail::TestSuite::operator* (  
    const char * in)    [noexcept]
```

11.91.2.2 [operator*\(\)](#) [2/2]

```
template<typename T>  
TestSuite & doctest::detail::TestSuite::operator* (  
    const T & in)    [inline], [noexcept]
```

Definition at line 2178 of file [doctest.h](#).

11.91.3 Member Data Documentation

11.91.3.1 [m_description](#)

```
const char* doctest::detail::TestSuite::m_description = nullptr
```

Definition at line 2166 of file [doctest.h](#).

11.91.3.2 m_expected_failures

```
int doctest::detail::TestSuite::m_expected_failures = 0
```

Definition at line 2172 of file [doctest.h](#).

11.91.3.3 m_may_fail

```
bool doctest::detail::TestSuite::m_may_fail = false
```

Definition at line 2170 of file [doctest.h](#).

11.91.3.4 m_no_breaks

```
bool doctest::detail::TestSuite::m_no_breaks = false
```

Definition at line 2168 of file [doctest.h](#).

11.91.3.5 m_no_output

```
bool doctest::detail::TestSuite::m_no_output = false
```

Definition at line 2169 of file [doctest.h](#).

11.91.3.6 m_should_fail

```
bool doctest::detail::TestSuite::m_should_fail = false
```

Definition at line 2171 of file [doctest.h](#).

11.91.3.7 m_skip

```
bool doctest::detail::TestSuite::m_skip = false
```

Definition at line 2167 of file [doctest.h](#).

11.91.3.8 m_test_suite

```
const char* doctest::detail::TestSuite::m_test_suite = nullptr
```

Definition at line 2165 of file [doctest.h](#).

11.93 std::tuple< Types > Class Template Reference

11.93.1 Detailed Description

```
template<class... Types>  
class std::tuple< Types >
```

Definition at line 686 of file [doctest.h](#).

The documentation for this class was generated from the following file:

- tests/[doctest.h](#)

11.94 doctest::detail::types::underlying_type< T > Struct Template Reference

```
#include <doctest.h>
```

Public Types

- using [type](#) = __underlying_type(T)

11.94.1 Detailed Description

```
template<typename T>  
struct doctest::detail::types::underlying_type< T >
```

Definition at line 788 of file [doctest.h](#).

11.94.2 Member Typedef Documentation

11.94.2.1 type

```
template<typename T>  
using doctest::detail::types::underlying\_type< T >::type = __underlying_type(T)
```

Definition at line 789 of file [doctest.h](#).

The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

11.95 Usuario Class Reference

Representa um usuário do sistema com credenciais e cargo.

```
#include <Usuario.hpp>
```

Public Member Functions

- [Usuario](#) (std::string login, std::string senha, [Cargo](#) cargo)
Constrói um usuário.
- bool [ValidarSenha](#) (std::string senha) const
Verifica se a senha fornecida corresponde à senha do usuário.
- std::string [GetLogin](#) () const
- [Cargo](#) [GetCargo](#) (std::string login)
Retorna o cargo do usuário identificado pelo login.
- [Cargo](#) [GetCargo](#) () const
- std::string [GetSenha](#) () const

11.95.1 Detailed Description

Representa um usuário do sistema com credenciais e cargo.

Definition at line 11 of file [Usuario.hpp](#).

11.95.2 Constructor & Destructor Documentation

11.95.2.1 Usuario()

```
Usuario::Usuario (
    std::string login,
    std::string senha,
    Cargo cargo)
```

Constrói um usuário.

Parameters

<i>login</i>	Login de acesso.
<i>senha</i>	Senha do usuário.
<i>cargo</i>	Cargo/nível de acesso.

Note

Implementa US-015 (Criar usuários).

Definition at line 4 of file [Usuario.cpp](#).

11.95.3 Member Function Documentation

11.95.3.1 GetCargo() [1/2]

```
Cargo Usuario::GetCargo () const
```

Returns

Cargo do usuário.

Definition at line 33 of file [Usuario.cpp](#).

11.95.3.2 GetCargo() [2/2]

```
Cargo Usuario::GetCargo (  
    std::string login)
```

Retorna o cargo do usuário identificado pelo login.

Parameters

<i>login</i>	Login do usuário.
--------------	-------------------

Returns

Cargo associado ao login.

Definition at line 28 of file [Usuario.cpp](#).

11.95.3.3 GetLogin()

```
std::string Usuario::GetLogin () const
```

Returns

Login do usuário.

Definition at line 24 of file [Usuario.cpp](#).

11.95.3.4 GetSenha()

```
std::string Usuario::GetSenha () const
```

Returns

Senha do usuário (uso interno: persistência).

Definition at line 37 of file [Usuario.cpp](#).

11.95.3.5 ValidarSenha()

```
bool Usuario::ValidarSenha (
    std::string senha) const
```

Verifica se a senha fornecida corresponde à senha do usuário.

Parameters

<i>senha</i>	Senha a validar.
--------------	------------------

Returns

true se a senha estiver correta.

Note

Implementa US-017 (Login de usuário).

Definition at line 16 of file [Usuario.cpp](#).

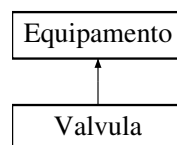
The documentation for this class was generated from the following files:

- [include/Usuario.hpp](#)
- [src/Usuario.cpp](#)

11.96 Valvula Class Reference

```
#include <Valvula.hpp>
```

Inheritance diagram for Valvula:



Public Member Functions

- [Valvula](#) (int id, std::string desc, double vazao)
 - [~Valvula](#) () override=default
 - std::string [Tipo](#) () const override
- Tipo concreto do equipamento (sobrescrito pelas especializações).*

Public Member Functions inherited from Equipamento

- [Equipamento](#) (int id, std::string desc, std::string nome)
Constrói um equipamento.
- virtual [~Equipamento](#) ()=default
Destrutor virtual: permite uso polimórfico via ponteiro de base.
- void [AdicionarParametro](#) (double ValorAtual, std::string Desc, int id)
Adiciona um parâmetro sem limites definidos.
- int [AdicionarParametro](#) (double ValorAtual, std::string Desc)
Adiciona um parâmetro sem limites com ID gerado automaticamente.
- void [AdicionarParametro](#) (double ValorAtual, double Max, double Min, std::string Desc, int id)
Adiciona um parâmetro com limites Min/Max definidos.
- int [AdicionarParametro](#) (double ValorAtual, double Max, double Min, std::string Desc)
Adiciona um parâmetro com limites Min/Max e ID gerado automaticamente.
- int [ProximoldParametro](#) () const
- void [RemoverParametro](#) (int id)
Remove o parâmetro com o ID fornecido.
- std::vector< int > [DiagnosticarEquipamento](#) () const
Diagnostica todos os parâmetros do equipamento.
- void [AtualizarAlarmes](#) ()
Atualiza o estado de todos os alarmes do equipamento.
- void [ExibirParametros](#) () const
Exibe no console os parâmetros e seus valores atuais.
- void [ExibirAlarmes](#) () const
Exibe no console os alarmes ativos do equipamento.
- int [QuantidadeParametros](#) () const
- int [QuantidadeAlarmes](#) () const
- int [GetId](#) () const
- std::string [GetDesc](#) () const
- std::string [GetNome](#) () const
- const std::map< int, [Parametro](#) > & [GetParametros](#) () const
- const std::vector< [Alarme](#) > & [GetAlarmes](#) () const
- void [SetDesc](#) (std::string desc)
Atualiza a descrição do equipamento.

11.96.1 Detailed Description

Definition at line 6 of file [Valvula.hpp](#).

11.96.2 Constructor & Destructor Documentation

11.96.2.1 Valvula()

```
Valvula::Valvula (
    int id,
    std::string desc,
    double vazao)
```

Definition at line 3 of file [Valvula.cpp](#).

11.96.2.2 ~Valvula()

```
Valvula::~Valvula () [override], [default]
```

11.96.3 Member Function Documentation

11.96.3.1 Tipo()

```
std::string Valvula::Tipo () const [override], [virtual]
```

Tipo concreto do equipamento (sobrescrito pelas especializações).

Returns

Nome do tipo (ex.: "Equipamento", "Motor", "Sensor").

Reimplemented from [Equipamento](#).

Definition at line 8 of file [Valvula.cpp](#).

The documentation for this class was generated from the following files:

- [include/Valvula.hpp](#)
- [src/Valvula.cpp](#)

Chapter 12

File Documentation

12.1 design/notas-C6.md File Reference

12.2 design/notas-C7.md File Reference

12.3 include/Alarme.hpp File Reference

```
#include <string>
#include <vector>
#include "Parametro.hpp"
```

Classes

- class [Alarme](#)
[Alarme](#) associado a um [Parametro](#) de um equipamento.

12.4 Alarme.hpp

[Go to the documentation of this file.](#)

```
00001 #ifndef ALARME_HPP
00002 #define ALARME_HPP
00003
00004 #include <string>
00005 #include <vector>
00006 #include "Parametro.hpp"
00007
00014 class Alarme {
00015 private:
00016     std::string _Desc;
00017     bool _Estado;
00018     int _Id;
00019     const Parametro* _Parametro;
00020 public:
00028     Alarme(std::string Desc, int Id, Parametro& p);
00029
00035     bool AtualizarAlarme();
00036
00038     bool GetEstado() const;
00039
00041     void SetEstado(bool Estado);
00042
00044     void SetDesc(std::string desc);
00045 };
00046
00047 #endif
```

12.5 include/Atuador.hpp File Reference

```
#include "Equipamento.hpp"
```

Classes

- class [Atuador](#)

12.6 Atuador.hpp

[Go to the documentation of this file.](#)

```
00001 #ifndef ATUADOR_HPP
00002 #define ATUADOR_HPP
00003
00004 #include "Equipamento.hpp"
00005
00006 class Atuador : public Equipamento {
00007 public:
00008     Atuador(int id, std::string desc, double retorno);
00009     ~Atuador() override = default;
00010
00011     std::string Tipo() const override;
00012 };
00013
00014 #endif
```

12.7 include/Cargo.hpp File Reference

Enumerations

- enum class [Cargo](#) { [ADMIN](#) , [ENGENHEIRO](#) , [TECNICO](#) }
Níveis de acesso dos usuários do sistema.

12.7.1 Enumeration Type Documentation

12.7.1.1 Cargo

```
enum class Cargo [strong]
```

Níveis de acesso dos usuários do sistema.

- ADMIN: Gerencia usuários e a estrutura completa do sistema.
- ENGENHEIRO: Configura linhas, conjuntos e equipamentos.
- TECNICO: Visualiza dados e alarmes.

Enumerator

ADMIN	
ENGENHEIRO	
TECNICO	

Definition at line 11 of file [Cargo.hpp](#).

12.8 Cargo.hpp

[Go to the documentation of this file.](#)

```
00001 #ifndef CARGO_HPP
00002 #define CARGO_HPP
00003
00011 enum class Cargo {
00012     ADMIN,
00013     ENGENHEIRO,
00014     TECNICO
00015 };
00016
00017 #endif
```

12.9 include/Conjunto.hpp File Reference

```
#include <string>
#include <vector>
#include <map>
#include <memory>
#include "Equipamento.hpp"
```

Classes

- class [Conjunto](#)

Agrupamento lógico de equipamentos dentro de uma [LinhaDeProducao](#).

12.10 Conjunto.hpp

[Go to the documentation of this file.](#)

```
00001 #ifndef CONJUNTO_HPP
00002 #define CONJUNTO_HPP
00003
00004 #include <string>
00005 #include <vector>
00006 #include <map>
00007 #include <memory>
00008 #include "Equipamento.hpp"
00009
00016 class Conjunto {
00017 private:
00018     int _id;
00019     std::string _desc;
00020     std::string _nome;
00021     int _ProximoIdEquipamento = 1;
00022
```

```

00023     std::map<int, std::unique_ptr<Equipamento> _Equipamentos;
00024 public:
00030     Conjunto(int id, std::string nome ,std::string desc);
00031
00038     void AdicionarEquipamento(int id, std::string desc);
00039
00045     void AdicionarEquipamento(std::unique_ptr<Equipamento> equip);
00046
00048     int ProximoIdEquipamento() const;
00049
00055     void RemoverEquipamento(int id);
00056
00058     void SetDesc(std::string desc);
00059
00061     void SetNome(std::string nome);
00062
00067     void ExibirEquipamentos() const;
00068
00070     void ExibirAlarmes() const;
00071
00076     void ExibirTudo() const;
00077
00079     int QuantidadeEquipamentos() const;
00080
00082     int QuantidadeAlarmes() const;
00083
00089     std::vector<int> DiagnosticarConjunto() const;
00090
00095     void AtualizarAlarmes();
00096
00098     const std::map<int, std::unique_ptr<Equipamento>>& GetEquipamentos() const;
00099
00106     Equipamento& AcessarEquipamento(int id);
00107
00109     int GetId() const;
00110
00112     std::string GetDesc() const;
00113
00115     std::string GetNome() const;
00116 };
00117
00118 #endif

```

12.11 include/Dependencia.hpp File Reference

```
#include "EstadoOperacional.hpp"
```

Classes

- class [Dependencia](#)

12.12 Dependencia.hpp

[Go to the documentation of this file.](#)

```

00001 #ifndef DEPENDENCIA_HPP
00002 #define DEPENDENCIA_HPP
00003
00004 #include "EstadoOperacional.hpp"
00005
00006 class Dependencia {
00007 private:
00008     int _id;
00009     EstadoOperacional _condicao;
00010     EstadoOperacional _resultado;
00011
00012 public:
00013     Dependencia(int id, EstadoOperacional condicao, EstadoOperacional resultado);
00014
00015     int Get_Id ();
00016     EstadoOperacional Get_Condicao ();
00017     EstadoOperacional Get_Resultado ();
00018
00019 };
00020
00021 #endif

```


12.13 include/Equipamento.hpp File Reference

```
#include <string>
#include <vector>
#include <map>
#include "Parametro.hpp"
#include "Alarme.hpp"
```

Classes

- class [Equipamento](#)
Representa um equipamento físico dentro de um [Conjunto](#).

12.14 Equipamento.hpp

[Go to the documentation of this file.](#)

```
00001 #ifndef EQUIPAMENTO_HPP
00002 #define EQUIPAMENTO_HPP
00003
00004 #include <string>
00005 #include <vector>
00006 #include <map>
00007 #include "Parametro.hpp"
00008 #include "Alarme.hpp"
00009
00016 class Equipamento {
00017 private:
00018     int _id;
00019     std::string _desc;
00020     std::string _nome;
00021     int _ProximoIdParametro = 1;
00022
00023     std::map<int, Parametro> _Parametros;
00024     std::vector<Alarme> _Alarmes;
00025 public:
00031     Equipamento(int id, std::string desc, std::string nome);
00032
00034     virtual ~Equipamento() = default;
00035
00040     virtual std::string Tipo() const;
00041
00049     void AdicionarParametro(double ValorAtual, std::string Desc, int id);
00050
00055     int AdicionarParametro(double ValorAtual, std::string Desc);
00056
00065     void AdicionarParametro(double ValorAtual, double Max, double Min, std::string Desc, int id);
00066
00071     int AdicionarParametro(double ValorAtual, double Max, double Min, std::string Desc);
00072
00074     int ProximoIdParametro() const;
00075
00080     void RemoverParametro(int id);
00081
00087     std::vector<int> DiagnosticarEquipamento() const;
00088
00093     void AtualizarAlarmes();
00094
00099     void ExibirParametros() const;
00100
00105     void ExibirAlarmes() const;
00106
00108     int QuantidadeParametros() const;
00109
00111     int QuantidadeAlarmes() const;
00112
00114     int GetId() const;
00115
00117     std::string GetDesc() const;
00118
00120     std::string GetNome() const;
```

```

00121
00123     const std::map<int, Parametro>& GetParametros() const;
00124
00126     const std::vector<Alarme>& GetAlarmes() const;
00127
00129     void SetDesc(std::string desc);
00130 };
00131
00132 #endif

```

12.15 include/EstadoOperacional.hpp File Reference

Enumerations

- enum class [EstadoOperacional](#) { [OPERACIONAL](#) , [FALHA_NA_ALIMENTACAO](#) , [FALHA_INTERNA](#) , [FALHA_NA_SAIDA](#) }

12.15.1 Enumeration Type Documentation

12.15.1.1 EstadoOperacional

```
enum class EstadoOperacional [strong]
```

Enumerator

OPERACIONAL	
FALHA_NA_ALIMENTACAO	
FALHA_INTERNA	
FALHA_NA_SAIDA	

Definition at line 4 of file [EstadoOperacional.hpp](#).

12.16 EstadoOperacional.hpp

[Go to the documentation of this file.](#)

```

00001 #ifndef ESTADOOPERACIONAL_HPP
00002 #define ESTADOOPERACIONAL_HPP
00003
00004 enum class EstadoOperacional {
00005     OPERACIONAL,
00006     FALHA_NA_ALIMENTACAO,
00007     FALHA_INTERNA,
00008     FALHA_NA_SAIDA
00009 };
00010
00011 #endif

```

12.17 include/Excecoes.hpp File Reference

```

#include <stdexcept>
#include <string>

```

Classes

- class [ExcecaoSistema](#)
Exceção base do sistema de monitoramento.
- class [EntradaInvalida](#)
Entrada inválida (string vazia, argumento sem sentido).
- class [LimitesInvalidos](#)
Limites incoerentes (Max < Min).
- class [IdInvalido](#)
Identificador inválido (ex.: negativo).
- class [IdDuplicado](#)
Tentativa de inserir um identificador já existente.
- class [IdInexistente](#)
Acesso a um identificador inexistente.
- class [AcessoNegado](#)
Operação sem permissão para o cargo do usuário logado.
- class [SessaoInvalida](#)
Operação que exige sessão ativa sem usuário logado.

12.18 Excecoes.hpp

[Go to the documentation of this file.](#)

```

00001 #ifndef EXCECOES_HPP
00002 #define EXCECOES_HPP
00003
00004 #include <stdexcept>
00005 #include <string>
00006
00013 class ExcecaoSistema : public std::runtime_error {
00014 public:
00015     using std::runtime_error::runtime_error;
00016 };
00017
00019 class EntradaInvalida : public ExcecaoSistema {
00020 public:
00021     using ExcecaoSistema::ExcecaoSistema;
00022 };
00023
00025 class LimitesInvalidos : public ExcecaoSistema {
00026 public:
00027     using ExcecaoSistema::ExcecaoSistema;
00028 };
00029
00031 class IdInvalido : public ExcecaoSistema {
00032 public:
00033     using ExcecaoSistema::ExcecaoSistema;
00034 };
00035
00037 class IdDuplicado : public ExcecaoSistema {
00038 public:
00039     using ExcecaoSistema::ExcecaoSistema;
00040 };
00041
00043 class IdInexistente : public ExcecaoSistema {
00044 public:
00045     using ExcecaoSistema::ExcecaoSistema;
00046 };
00047
00049 class AcessoNegado : public ExcecaoSistema {
00050 public:
00051     using ExcecaoSistema::ExcecaoSistema;
00052 };
00053
00055 class SessaoInvalida : public ExcecaoSistema {
00056 public:
00057     using ExcecaoSistema::ExcecaoSistema;
00058 };
00059
00060 #endif

```

12.19 include/LinhaDeProducao.hpp File Reference

```
#include <map>
#include <vector>
#include <string>
#include "Conjunto.hpp"
```

Classes

- class [LinhaDeProducao](#)
Representa uma linha de produção industrial.

12.20 LinhaDeProducao.hpp

[Go to the documentation of this file.](#)

```
00001 #ifndef LINHADEPRODUCAO_HPP
00002 #define LINHADEPRODUCAO_HPP
00003
00004 #include <map>
00005 #include <vector>
00006 #include <string>
00007
00008 #include "Conjunto.hpp"
00009
00016 class LinhaDeProducao {
00017 private:
00018     int _id;
00019     std::string _nome;
00020     std::string _local;
00021     std::string _desc;
00022     int _ProximoIdConjunto = 1;
00023
00024     std::map<int, Conjunto> _Conjuntos;
00025 public:
00033     LinhaDeProducao(int id, std::string desc, std::string local, std::string nome);
00034
00041     void AdicionarConjunto(int id, std::string nome, std::string desc);
00042
00047     int AdicionarConjunto(std::string nome, std::string desc);
00048
00050     int ProximoIdConjunto() const;
00051
00057     void RemoverConjunto(int id);
00058
00063     void ExibirConjuntos() const;
00064
00066     void ExibirAlarmes() const;
00067
00072     void ExibirTudo() const;
00073
00075     int QuantidadeConjuntos() const;
00076
00078     int QuantidadeAlarmes() const;
00079
00085     std::vector<int> DiagnosticarLinha() const;
00086
00091     void AtualizarAlarmes();
00092
00094     void SetDesc(std::string desc);
00095
00097     void SetLocal(std::string local);
00098
00100     void SetNome(std::string nome);
00101
00103     const std::map<int, Conjunto>& GetConjuntos() const;
00104
00111     Conjunto& AcessarConjunto(int id);
00112
00114     int GetId() const;
00115
00117     std::string GetDesc() const;
```

```
00118
00120     std::string GetLocal() const;
00121
00123     std::string GetNome() const;
00124 };
00125
00126 #endif
```

12.21 include/Log.hpp File Reference

```
#include <string>
```

Classes

- class [Log](#)

Registra eventos e operações do sistema em arquivo de texto.

12.22 Log.hpp

[Go to the documentation of this file.](#)

```
00001 #ifndef LOG_HPP
00002 #define LOG_HPP
00003
00004 #include <string>
00005
00009 class Log {
00010 private:
00011     std::string _arquivo;
00012 public:
00017     Log(const std::string& arquivo);
00018
00025     bool Registrar(const std::string& mensagem) const;
00026
00028     std::string GetArquivoLog() const;
00029
00035     bool SetArquivoLog(const std::string& arquivo);
00036 };
00037
00038 #endif
```

12.23 include/Motor.hpp File Reference

```
#include "Equipamento.hpp"
```

Classes

- class [Motor](#)

12.24 Motor.hpp

[Go to the documentation of this file.](#)

```
00001 #ifndef MOTOR_HPP
00002 #define MOTOR_HPP
00003
00004 #include "Equipamento.hpp"
00005
00006 class Motor : public Equipamento {
00007     public:
00008         Motor(int id, std::string desc, double velocidade, double torque);
00009         ~Motor() override = default;
00010
00011         std::string Tipo() const override;
00012 };
00013
00014 #endif
```

12.25 include/Parametro.hpp File Reference

```
#include <string>
#include <vector>
```

Classes

- class [Parametro](#)

Representa um parâmetro monitorado de um equipamento.

12.26 Parametro.hpp

[Go to the documentation of this file.](#)

```
00001 #ifndef PARAMETRO_HPP
00002 #define PARAMETRO_HPP
00003
00004 #include <string>
00005 #include <vector>
00006
00013 class Parametro {
00014     private:
00015         int _id;
00016         std::string _Desc;
00017
00018         double _ValorAtual;
00019         std::vector<double> _Hist;
00020
00021         double _Max;
00022         double _Min;
00023
00024         bool _Estado;
00025     public:
00032         Parametro(double ValorAtual, std::string Desc, int id);
00033
00043         Parametro(double ValorAtual, double Max, double Min, std::string Desc, int id);
00044
00050         bool ValorValido(double Valor) const;
00051
00057         void SetValorAtual(double Valor);
00058
00064         void SetMax(double Valor);
00065
00071         void SetMin(double Valor);
00072
00074         void SetDesc(std::string Desc);
00075
00081         bool DiagnosticarParametro();
```

```

00082
00084     double GetValorAtual() const;
00085
00087     double GetMax() const;
00088
00090     double GetMin() const;
00091
00093     std::string GetDesc() const;
00094
00096     bool GetEstado() const;
00097
00102     const std::vector<double>& GetHist() const;
00103
00105     int GetId();
00106
00108     void ResetarLimites();
00109
00111     void ResetarHistorico();
00112 };
00113
00114 #endif

```

12.27 include/Sensor.hpp File Reference

```
#include "Equipamento.hpp"
```

Classes

- class [Sensor](#)

12.28 Sensor.hpp

[Go to the documentation of this file.](#)

```

00001 #ifndef SENSOR_HPP
00002 #define SENSOR_HPP
00003
00004 #include "Equipamento.hpp"
00005
00006 class Sensor : public Equipamento {
00007 public:
00008     Sensor(int id, std::string desc, double sinal);
00009     ~Sensor() override = default;
00010
00011     std::string Tipo() const override;
00012 };
00013
00014 #endif

```

12.29 include/Sistema.hpp File Reference

```

#include <vector>
#include <map>
#include <string>
#include "LinhaDeProducao.hpp"
#include "Usuario.hpp"
#include "Cargo.hpp"

```

Classes

- class [Sistema](#)

Ponto de entrada central do sistema de monitoramento.

12.30 Sistema.hpp

[Go to the documentation of this file.](#)

```

00001 #ifndef SISTEMA_HPP
00002 #define SISTEMA_HPP
00003
00004 #include <vector>
00005 #include <map>
00006 #include <string>
00007
00008 #include "LinhaDeProducao.hpp"
00009 #include "Usuario.hpp"
00010 #include "Cargo.hpp"
00011
00012 class Sistema {
00013 private:
00014     int _ProximoId;
00015     std::map<int, LinhaDeProducao> _Linhas;
00016     std::vector<Usuario> _Usuarios;
00017
00018     int _UsuarioLogadoIdx;
00019     std::string _arquivoSave;
00020 public:
00021     Sistema();
00022
00023     void AdicionarLinha(int id, std::string desc, std::string local, std::string nome);
00024
00025     int AdicionarLinha(std::string desc, std::string local, std::string nome);
00026
00027     void RemoverLinha(int id);
00028
00029     const std::map<int, LinhaDeProducao>& GetLinhas() const;
00030
00031     LinhaDeProducao& AcessarLinha(int id);
00032
00033     void ExibirLinhas();
00034
00035     void AdicionarUsuario(std::string login, std::string senha, Cargo cargo);
00036
00037     void RemoverUsuario(std::string login);
00038
00039     int ProximoIdDisponivel() const;
00040
00041     bool IdAtivo(int Id) const;
00042
00043     bool Login(std::string login, std::string senha);
00044
00045     bool Logout();
00046
00047     Usuario* GetUsuarioLogado() const;
00048
00049     bool SalvarAlteracoes();
00050
00051     bool CarregarUltimoSave();
00052
00053     bool CarregarSave(std::string arquivo);
00054 };
00055 #endif

```

12.31 include/Usuario.hpp File Reference

```

#include <string>
#include "Cargo.hpp"

```


Classes

- class [Usuario](#)

Representa um usuário do sistema com credenciais e cargo.

12.32 Usuario.hpp

[Go to the documentation of this file.](#)

```
00001 #ifndef USUARIO_HPP
00002 #define USUARIO_HPP
00003
00004 #include <string>
00005
00006 #include "Cargo.hpp"
00007
00011 class Usuario {
00012 private:
00013     std::string _login;
00014     std::string _senha;
00015     Cargo _cargo;
00016 public:
00024     Usuario(std::string login, std::string senha, Cargo cargo);
00025
00032     bool ValidarSenha(std::string senha) const;
00033
00035     std::string GetLogin() const;
00036
00042     Cargo GetCargo(std::string login);
00043
00045     Cargo GetCargo() const;
00046
00048     std::string GetSenha() const;
00049 };
00050
00051 #endif
```

12.33 include/Valvula.hpp File Reference

```
#include "Equipamento.hpp"
```

Classes

- class [Valvula](#)

12.34 Valvula.hpp

[Go to the documentation of this file.](#)

```
00001 #ifndef VALVULA_HPP
00002 #define VALVULA_HPP
00003
00004 #include "Equipamento.hpp"
00005
00006 class Valvula : public Equipamento{
00007 public:
00008     Valvula(int id, std::string desc, double vazao);
00009     ~Valvula() override = default;
00010
00011     std::string Tipo() const override;
00012 };
00013
00014 #endif
```

12.35 README.md File Reference

12.36 src/Alarme.cpp File Reference

```
#include "Alarme.hpp"
```

12.37 Alarme.cpp

[Go to the documentation of this file.](#)

```
00001 #include "Alarme.hpp"
00002
00003 Alarme::Alarme(std::string Desc, int Id, Parametro& p) {
00004     this->_Desc = Desc;
00005     this->_Id = Id;
00006     this->_Parametro = &p;
00007     this->_Estado = false;
00008 }
00009
00010 bool Alarme::AtualizarAlarme() {
00011     bool falha = !this->_Parametro->ValorValido(this->_Parametro->GetValorAtual());
00012     this->_Estado = falha;
00013     return falha;
00014 }
00015
00016 bool Alarme::GetEstado() const {
00017     return this->_Estado;
00018 }
00019
00020 void Alarme::SetEstado(bool Estado) {
00021     this->_Estado = Estado;
00022 }
00023
00024 void Alarme::SetDesc(std::string desc) {
00025     this->_Desc = desc;
00026 }
```

12.38 src/Atuador.cpp File Reference

```
#include "Atuador.hpp"
```

12.39 Atuador.cpp

[Go to the documentation of this file.](#)

```
00001 #include "Atuador.hpp"
00002
00003 Atuador::Atuador(int id, std::string desc, double retorno)
00004     : Equipamento(id, desc, "Atuador") {
00005     this->AdicionarParametro(retorno, "Sinal de retorno do atuador", id * 100 + 1);
00006 }
00007
00008 std::string Atuador::Tipo() const { return "Atuador"; }
```

12.40 src/Conjunto.cpp File Reference

```
#include "Conjunto.hpp"
#include "Excecoes.hpp"
#include <iostream>
#include <utility>
```

12.41 Conjunto.cpp

[Go to the documentation of this file.](#)

```
00001 #include "Conjunto.hpp"
00002 #include "Excecoes.hpp"
00003
00004 #include <iostream>
00005 #include <utility>
00006
00007 Conjunto::Conjunto(int id, std::string nome, std::string desc) {
00008     if (id < 0) {
00009         throw IdInvalido("Conjunto: id nao pode ser negativo");
00010     }
00011     if (desc.empty()) {
00012         throw EntradaInvalida("Conjunto: descricao vazia");
00013     }
00014     this->_id = id;
00015     this->_nome = nome;
00016     this->_desc = desc;
00017 }
00018
00019 void Conjunto::AdicionarEquipamento(int id, std::string desc) {
00020     if (this->_Equipamentos.count(id) > 0) {
00021         throw IdDuplicado("Conjunto: equipamento com id ja existente");
00022     }
00023     this->_Equipamentos.emplace(id, std::make_unique<Equipamento>(id, desc, desc));
00024     if (id >= this->_ProximoIdEquipamento) {
00025         this->_ProximoIdEquipamento = id + 1;
00026     }
00027 }
00028
00029 void Conjunto::AdicionarEquipamento(std::unique_ptr<Equipamento> equip) {
00030     if (equip == nullptr) {
00031         throw EntradaInvalida("Conjunto: equipamento nulo");
00032     }
00033     int id = equip->GetId();
00034     if (this->_Equipamentos.count(id) > 0) {
00035         throw IdDuplicado("Conjunto: equipamento com id ja existente");
00036     }
00037     this->_Equipamentos.emplace(id, std::move(equip));
00038     if (id >= this->_ProximoIdEquipamento) {
00039         this->_ProximoIdEquipamento = id + 1;
00040     }
00041 }
00042
00043 int Conjunto::ProximoIdEquipamento() const {
00044     return this->_ProximoIdEquipamento;
00045 }
00046
00047 void Conjunto::RemoverEquipamento(int id) {
00048     if (this->_Equipamentos.count(id) == 0) {
00049         throw IdInexistente("Conjunto: equipamento inexistente");
00050     }
00051     this->_Equipamentos.erase(id);
00052 }
00053
00054 void Conjunto::SetDesc(std::string desc) {
00055     if (desc.empty()) {
00056         throw EntradaInvalida("Conjunto: descricao vazia");
00057     }
00058     this->_desc = desc;
00059 }
00060
00061 void Conjunto::SetName(std::string nome) {
00062     this->_nome = nome;
00063 }
00064
00065 void Conjunto::ExibirEquipamentos() const {
```

```

00066     if (this->_Equipamentos.size() > 0){
00067         std::cout << "Conjunto " << this->_nome << std::endl;
00068         for (const auto& e : this->_Equipamentos) {
00069             std::cout << "Equipamento ID: " << e.first
00070                 << " (" << e.second->Tipo() << ")" << std::endl;
00071             e.second->ExibirParametros();
00072         }
00073         return;
00074     }
00075 }
00076
00077 std::cout << "Nenhum equipamento presente" << std::endl;
00078 }
00079
00080 void Conjunto::ExibirAlarmes() const {
00081     if (this->_Equipamentos.size() > 0){
00082         for (const auto& e : this->_Equipamentos) {
00083             e.second->ExibirAlarmes();
00084         }
00085         return;
00086     }
00087 }
00088
00089 std::cout << "Nenhum alarme presente" << std::endl;
00090 }
00091
00092 void Conjunto::ExibirTudo() const {
00093     this->ExibirEquipamentos();
00094     this->ExibirAlarmes();
00095 }
00096
00097 int Conjunto::QuantidadeEquipamentos() const {
00098     return static_cast<int>(this->_Equipamentos.size());
00099 }
00100
00101 int Conjunto::QuantidadeAlarmes() const {
00102     int total = 0;
00103     for (const auto& e : this->_Equipamentos) {
00104         total += e.second->QuantidadeAlarmes();
00105     }
00106     return total;
00107 }
00108
00109 std::vector<int> Conjunto::DiagnosticarConjunto() const {
00110     std::vector<int> falhas;
00111     for (const auto& e : this->_Equipamentos) {
00112         std::vector<int> f = e.second->DiagnosticarEquipamento();
00113         falhas.insert(falhas.end(), f.begin(), f.end());
00114     }
00115     return falhas;
00116 }
00117
00118 void Conjunto::AtualizarAlarmes() {
00119     for (auto& e : this->_Equipamentos) {
00120         e.second->AtualizarAlarmes();
00121     }
00122 }
00123
00124 const std::map<int, std::unique_ptr<Equipamento>>& Conjunto::GetEquipamentos() const {
00125     return _Equipamentos;
00126 }
00127
00128 Equipamento& Conjunto::AcessarEquipamento(int id) {
00129     auto it = this->_Equipamentos.find(id);
00130     if (it == this->_Equipamentos.end()) {
00131         throw IdInexistente("Conjunto: equipamento inexistente");
00132     }
00133     return *it->second;
00134 }
00135
00136 int Conjunto::GetId() const { return this->_id; }
00137
00138 std::string Conjunto::GetDesc() const { return this->_desc; }
00139
00140 std::string Conjunto::GetNome() const { return this->_nome; }

```

12.42 src/Dependencia.cpp File Reference

```
#include "Dependencia.hpp"
```

12.43 Dependencia.cpp

[Go to the documentation of this file.](#)

```

00001 #include "Dependencia.hpp"
00002
00003 Dependencia::Dependencia(int id, EstadoOperacional condicao, EstadoOperacional resultado){
00004     this->_id = id;
00005     this->_condicao = entrada;
00006     this->_resultado = saida;
00007 }
00008
00009 EstadoOperacional Dependencia::Get_Resultado () {
00010     return this->_resultado;
00011 }
00012
00013 EstadoOperacional Dependencia::Get_Condicao () {
00014     return this->_condicao;
00015 }
00016
00017 int Dependencia::Get_Id () {
00018     return this->_id;
00019 }

```

12.44 src/Equipamento.cpp File Reference

```

#include "Equipamento.hpp"
#include "Excecoes.hpp"
#include <iostream>

```

12.45 Equipamento.cpp

[Go to the documentation of this file.](#)

```

00001 #include "Equipamento.hpp"
00002 #include "Excecoes.hpp"
00003
00004 #include <iostream>
00005
00006 Equipamento::Equipamento(int id, std::string desc, std::string nome) {
00007     if (id < 0) {
00008         throw IdInvalido("Equipamento: id nao pode ser negativo");
00009     }
00010     if (desc.empty()) {
00011         throw EntradaInvalida("Equipamento: descricao vazia");
00012     }
00013     this->_id = id;
00014     this->_desc = desc;
00015     this->_nome = nome;
00016 }
00017
00018 std::string Equipamento::Tipo() const { return "Equipamento"; }
00019
00020 void Equipamento::AdicionarParametro(double ValorAtual, std::string Desc, int id) {
00021     if (this->_Parametros.count(id) > 0) {
00022         throw IdDuplicado("Equipamento: parametro com id ja existente");
00023     }
00024     Parametro p(ValorAtual, Desc, id);
00025
00026     this->_Parametros.insert({id, p});
00027     if (id >= this->_ProximoIdParametro) {
00028         this->_ProximoIdParametro = id + 1;
00029     }
00030 }
00031
00032 int Equipamento::AdicionarParametro(double ValorAtual, std::string Desc) {
00033     int id = this->_ProximoIdParametro;
00034     this->AdicionarParametro(ValorAtual, Desc, id);
00035     return id;
00036 }
00037

```

```

00038 void Equipamento::AdicionarParametro(double ValorAtual, double Max, double Min, std::string Desc, int
    id) {
00039     if (this->_Parametros.count(id) > 0) {
00040         throw IdDuplicado("Equipamento: parametro com id ja existente");
00041     }
00042     Parametro p(ValorAtual, Max, Min, Desc, id);
00043
00044     this->_Parametros.insert({id, p});
00045     if (id >= this->_ProximoIdParametro) {
00046         this->_ProximoIdParametro = id + 1;
00047     }
00048 }
00049
00050 int Equipamento::AdicionarParametro(double ValorAtual, double Max, double Min, std::string Desc) {
00051     int id = this->_ProximoIdParametro;
00052     this->AdicionarParametro(ValorAtual, Max, Min, Desc, id);
00053     return id;
00054 }
00055
00056 int Equipamento::ProximoIdParametro() const {
00057     return this->_ProximoIdParametro;
00058 }
00059
00060 void Equipamento::RemoverParametro(int id) {
00061     if (this->_Parametros.count(id) == 0) {
00062         throw IdInexistente("Equipamento: parametro inexistente");
00063     }
00064     this->_Parametros.erase(id);
00065 }
00066
00067 std::vector<int> Equipamento::DiagnosticarEquipamento() const {
00068     std::vector<int> falhas;
00069     for (const auto& p : this->_Parametros) {
00070         if (!p.second.ValorValido(p.second.GetValorAtual())) {
00071             falhas.push_back(p.first);
00072         }
00073     }
00074     return falhas;
00075 }
00076
00077 void Equipamento::AtualizarAlarmes() {
00078     this->_Alarmes.clear();
00079     for (auto& p : this->_Parametros) {
00080         Alarme a(p.second.GetDesc(), p.first, p.second);
00081         a.AtualizarAlarme();
00082         this->_Alarmes.push_back(a);
00083     }
00084 }
00085
00086 void Equipamento::ExibirParametros() const {
00087     std::cout << "Equipamento " << this->_nome << std::endl;
00088     for (const auto& p : this->_Parametros) {
00089         std::cout << p.first << " - " << p.second.GetDesc() << " " << p.second.GetValorAtual() << std::endl;
00090     }
00091 }
00092
00093 void Equipamento::ExibirAlarmes() const {
00094     std::cout << "Alarmes do equipamento " << this->_nome << std::endl;
00095     for (const auto& a : this->_Alarmes) {
00096         std::cout << "Alarme " << (a.GetEstado() ? "ATIVO" : "inativo") << std::endl;
00097     }
00098 }
00099
00100 int Equipamento::QuantidadeParametros() const { return static_cast<int>(this->_Parametros.size()); }
00101
00102 int Equipamento::QuantidadeAlarmes() const { return static_cast<int>(this->_Alarmes.size()); }
00103
00104 int Equipamento::GetId() const {
00105     return this->_id;
00106 }
00107
00108 std::string Equipamento::GetDesc() const { return this->_desc; }
00109
00110 std::string Equipamento::GetNome() const { return this->_nome; }
00111
00112 const std::map<int, Parametro>& Equipamento::GetParametros() const {
00113     return this->_Parametros;
00114 }
00115
00116 const std::vector<Alarme>& Equipamento::GetAlarmes() const {
00117     return this->_Alarmes;
00118 }
00119
00120 void Equipamento::SetDesc(std::string desc) {
00121     this->_desc = desc;
00122 }

```

12.46 src/LinhaDeProducao.cpp File Reference

```
#include "LinhaDeProducao.hpp"
#include "Excecoes.hpp"
#include <iostream>
#include <utility>
```

12.47 LinhaDeProducao.cpp

[Go to the documentation of this file.](#)

```
00001 #include "LinhaDeProducao.hpp"
00002 #include "Excecoes.hpp"
00003
00004 #include <iostream>
00005 #include <utility>
00006
00007 LinhaDeProducao::LinhaDeProducao(int id, std::string desc, std::string local, std::string nome) {
00008     if (id < 0) {
00009         throw IdInvalido("LinhaDeProducao: id nao pode ser negativo");
00010     }
00011     if (desc.empty()) {
00012         throw EntradaInvalida("LinhaDeProducao: descricao vazia");
00013     }
00014     this->_id = id;
00015     this->_desc = desc;
00016     this->_local = local;
00017     this->_nome = nome;
00018 }
00019
00020 void LinhaDeProducao::AdicionarConjunto(int id, std::string nome, std::string desc) {
00021     if (this->_Conjuntos.count(id) > 0) {
00022         throw IdDuplicado("LinhaDeProducao: conjunto com id ja existente");
00023     }
00024     Conjunto c(id, nome, desc);
00025     this->_Conjuntos.emplace(id, std::move(c));
00026     if (id >= this->_ProximoIdConjunto) {
00027         this->_ProximoIdConjunto = id + 1;
00028     }
00029 }
00030
00031 int LinhaDeProducao::AdicionarConjunto(std::string nome, std::string desc) {
00032     int id = this->_ProximoIdConjunto;
00033     this->AdicionarConjunto(id, nome, desc);
00034     return id;
00035 }
00036
00037 int LinhaDeProducao::ProximoIdConjunto() const {
00038     return this->_ProximoIdConjunto;
00039 }
00040
00041 void LinhaDeProducao::RemoverConjunto(int id) {
00042     if (this->_Conjuntos.count(id) == 0) {
00043         throw IdInexistente("LinhaDeProducao: conjunto inexistente");
00044     }
00045     this->_Conjuntos.erase(id);
00046 }
00047
00048 void LinhaDeProducao::ExibirConjuntos() const {
00049     for (const auto& conjunto : this->_Conjuntos) {
00050         std::cout << "Conjunto ID: " << conjunto.first << " - " << conjunto.second.GetNome() << std::endl;
00051         std::cout << "Desc: " << conjunto.second.GetDesc() << std::endl;
00052     }
00053 }
00054
00055 void LinhaDeProducao::ExibirAlarmes() const {
00056     for (const auto& conjunto : this->_Conjuntos) {
00057         conjunto.second.ExibirAlarmes();
00058     }
00059 }
00060
00061 void LinhaDeProducao::ExibirTudo() const {
00062     for (const auto& conjunto : this->_Conjuntos) {
00063         std::cout << "Conjunto ID: " << conjunto.first << " - " << conjunto.second.GetNome() << std::endl;
00064         std::cout << "Desc: " << conjunto.second.GetDesc() << std::endl;
00065         conjunto.second.ExibirTudo();
00066     }
00067 }
```

```

00066     }
00067 }
00068
00069 int LinhaDeProducao::QuantidadeConjuntos() const {
00070     return static_cast<int>(this->_Conjuntos.size());
00071 }
00072
00073 int LinhaDeProducao::QuantidadeAlarmes() const {
00074     int total = 0;
00075     for (const auto& conjunto : this->_Conjuntos){
00076         total += conjunto.second.QuantidadeAlarmes();
00077     }
00078     return total;
00079 }
00080
00081 std::vector<int> LinhaDeProducao::DiagnosticarLinha() const {
00082     std::vector<int> falhas;
00083     for (const auto& conjunto : this->_Conjuntos){
00084         std::vector<int> f = conjunto.second.DiagnosticarConjunto();
00085         falhas.insert(falhas.end(), f.begin(), f.end());
00086     }
00087     return falhas;
00088 }
00089
00090 void LinhaDeProducao::AtualizarAlarmes() {
00091     for (auto& conjunto : this->_Conjuntos){
00092         conjunto.second.AtualizarAlarmes();
00093     }
00094 }
00095
00096 void LinhaDeProducao::SetDesc(std::string desc) {
00097     this->_desc = desc;
00098 }
00099
00100 void LinhaDeProducao::SetLocal(std::string local) {
00101     this->_local = local;
00102 }
00103
00104 void LinhaDeProducao::SetNome(std::string nome) {
00105     this->_nome = nome;
00106 }
00107
00108 const std::map<int, Conjunto>& LinhaDeProducao::GetConjuntos() const { return _Conjuntos; }
00109
00110 Conjunto& LinhaDeProducao::AcessarConjunto(int id) {
00111     auto it = this->_Conjuntos.find(id);
00112     if (it == this->_Conjuntos.end()) {
00113         throw IdInexistente("LinhaDeProducao: conjunto inexistente");
00114     }
00115     return it->second;
00116 }
00117
00118 int LinhaDeProducao::GetId() const {
00119     return this->_id;
00120 }
00121
00122 std::string LinhaDeProducao::GetDesc() const {
00123     return this->_desc;
00124 }
00125
00126 std::string LinhaDeProducao::GetLocal() const {
00127     return this->_local;
00128 }
00129
00130 std::string LinhaDeProducao::GetNome() const {
00131     return this->_nome;
00132 }

```

12.48 src/Log.cpp File Reference

```

#include "Log.hpp"
#include <fstream>

```

12.49 Log.cpp

[Go to the documentation of this file.](#)


```

00001 #include "Log.hpp"
00002
00003 #include <fstream>
00004
00005 Log::Log(const std::string& arquivo) {
00006     this->_arquivo = arquivo;
00007 }
00008
00009 bool Log::Registrar(const std::string& mensagem) const {
00010     if (mensagem.empty() || mensagem.size() > 500) {
00011         return false;
00012     }
00013
00014     std::ofstream ofs(this->_arquivo, std::ios::app);
00015     if (!ofs.is_open()) {
00016         return false;
00017     }
00018
00019     ofs << mensagem << '\n';
00020     return ofs.good();
00021 }
00022
00023 std::string Log::GetArquivoLog() const {
00024     return this->_arquivo;
00025 }
00026
00027 bool Log::SetArquivoLog(const std::string& arquivo) {
00028     if (arquivo.empty()) {
00029         return false;
00030     }
00031     this->_arquivo = arquivo;
00032     return true;
00033 }

```

12.50 src/main.cpp File Reference

```

#include "Sistema.hpp"
#include "Cargo.hpp"
#include "Excecoes.hpp"
#include "Motor.hpp"
#include "Sensor.hpp"
#include "Valvula.hpp"
#include "Atuador.hpp"
#include <iostream>
#include <string>
#include <memory>
#include <limits>
#include <cstdlib>

```

Functions

- `int main ()`

12.50.1 Function Documentation

12.50.1.1 `main()`

```
int main ()
```

Definition at line [282](#) of file [main.cpp](#).

12.51 main.cpp

[Go to the documentation of this file.](#)

```
00001 #include "Sistema.hpp"
00002 #include "Cargo.hpp"
00003 #include "Excecoes.hpp"
00004 #include "Motor.hpp"
00005 #include "Sensor.hpp"
00006 #include "Valvula.hpp"
00007 #include "Atuador.hpp"
00008
00009 #include <iostream>
00010 #include <string>
00011 #include <memory>
00012 #include <limits>
00013 #include <cstdlib>
00014
00015 namespace {
00016
00017     const std::string ARQUIVO_INICIAL = "dados/inicial.txt";
00018
00019     // ---- Leitura robusta de entrada ----
00020
00021     std::string lerLinha(const std::string& prompt) {
00022         std::cout << prompt;
00023         std::string s;
00024         std::getline(std::cin, s);
00025         return s;
00026     }
00027
00028     int lerInt(const std::string& prompt) {
00029         while (true) {
00030             std::string s = lerLinha(prompt);
00031             try {
00032                 return std::stoi(s);
00033             } catch (const std::exception&) {
00034                 std::cout << "Valor invalido, digite um numero inteiro.\n";
00035             }
00036         }
00037     }
00038
00039     double lerDouble(const std::string& prompt) {
00040         while (true) {
00041             std::string s = lerLinha(prompt);
00042             try {
00043                 return std::stod(s);
00044             } catch (const std::exception&) {
00045                 std::cout << "Valor invalido, digite um numero.\n";
00046             }
00047         }
00048     }
00049
00050     void limparTela() {
00051 #ifdef _WIN32
00052         std::system("cls");
00053 #else
00054         std::system("clear");
00055 #endif
00056     }
00057
00058     void pausar() {
00059         std::cout << "\nPressione ENTER para continuar...";
00060         std::string lixo;
00061         std::getline(std::cin, lixo);
00062     }
00063
00064     // ---- Permissões ----
00065
00066     int rank(Cargo c) {
00067         switch (c) {
00068             case Cargo::TECNICO:    return 1;
00069             case Cargo::ENGENHEIRO: return 2;
00070             case Cargo::ADMIN:      return 3;
00071         }
00072         return 0;
00073     }
00074
00075     std::string cargoTexto(Cargo c) {
00076         switch (c) {
00077             case Cargo::ADMIN:      return "ADMIN";
00078             case Cargo::ENGENHEIRO: return "ENGENHEIRO";
00079             case Cargo::TECNICO:    return "TECNICO";
00080         }
00081         return "?";
00082     }
```

```

00083
00084 void exigir(const Sistema& s, int minimo) {
00085     Usuario* u = s.GetUsuarioLogado();
00086     if (u == nullptr) {
00087         throw SessaoInvalida("Nenhum usuario logado.");
00088     }
00089     if (rank(u->GetCargo()) < minimo) {
00090         throw AcessoNegado("Cargo sem permissao para esta operacao.");
00091     }
00092 }
00093
00094 // ---- Operações ----
00095
00096 void criarLinha(Sistema& s) {
00097     exigir(s, rank(Cargo::ENGENHEIRO));
00098     std::string nome = lerLinha("Nome: ");
00099     std::string local = lerLinha("Local: ");
00100     std::string desc = lerLinha("Descricao: ");
00101     int id = s.AdicionarLinha(desc, local, nome);
00102     std::cout << "Linha criada com ID " << id << ".\n";
00103 }
00104
00105 void criarConjunto(Sistema& s) {
00106     exigir(s, rank(Cargo::ENGENHEIRO));
00107     int lid = lerInt("ID da linha: ");
00108     std::string nome = lerLinha("Nome: ");
00109     std::string desc = lerLinha("Descricao: ");
00110     int next = s.ProximoIdDisponivel();
00111     s.AcessarLinha(lid).AdicionarConjunto(next, nome, desc);
00112     std::cout << "Conjunto criado com ID " << next << ".\n";
00113 }
00114
00115 void criarEquipamento(Sistema& s) {
00116     exigir(s, rank(Cargo::ENGENHEIRO));
00117     int lid = lerInt("ID da linha: ");
00118     int cid = lerInt("ID do conjunto: ");
00119     Conjunto& conj = s.AcessarLinha(lid).AcessarConjunto(cid);
00120     int eid = conj.ProximoIdEquipamento();
00121     std::string desc = lerLinha("Descricao: ");
00122     std::cout << "Tipo: 1-Motor 2-Sensor 3-Valvula 4-Atuador 5-Generico\n";
00123     int tipo = lerInt("Opcao: ");
00124
00125     std::unique_ptr<Equipamento> eq;
00126     switch (tipo) {
00127     case 1: {
00128         double v = lerDouble("Velocidade nominal: ");
00129         double t = lerDouble("Torque nominal: ");
00130         eq = std::make_unique<Motor>(eid, desc, v, t);
00131         break;
00132     }
00133     case 2: {
00134         double sinal = lerDouble("Sinal: ");
00135         eq = std::make_unique<Sensor>(eid, desc, sinal);
00136         break;
00137     }
00138     case 3: {
00139         double vazao = lerDouble("Vazao: ");
00140         eq = std::make_unique<Valvula>(eid, desc, vazao);
00141         break;
00142     }
00143     case 4: {
00144         double retorno = lerDouble("Retorno: ");
00145         eq = std::make_unique<Atuador>(eid, desc, retorno);
00146         break;
00147     }
00148     default: {
00149         std::string nome = lerLinha("Nome: ");
00150         eq = std::make_unique<Equipamento>(eid, desc, nome);
00151         break;
00152     }
00153     }
00154
00155     conj.AdicionarEquipamento(std::move(eq));
00156     std::cout << "Equipamento criado com ID " << eid << ".\n";
00157 }
00158
00159 void criarParametro(Sistema& s) {
00160     exigir(s, rank(Cargo::ENGENHEIRO));
00161     int lid = lerInt("ID da linha: ");
00162     int cid = lerInt("ID do conjunto: ");
00163     int eid = lerInt("ID do equipamento: ");
00164     std::string desc = lerLinha("Descricao: ");
00165     double minimo = lerDouble("Limite minimo: ");
00166     double maximo = lerDouble("Limite maximo: ");
00167     double valor = lerDouble("Leitura atual: ");
00168     int pid = s.AcessarLinha(lid).AcessarConjunto(cid).AcessarEquipamento(eid)
00169         .AdicionarParametro(valor, maximo, minimo, desc);

```

```

00170     std::cout << "Parametro criado com ID " << pid << ".\n";
00171 }
00172
00173 void diagnosticar(Sistema& s) {
00174     int lid = lerInt("ID da linha: ");
00175     LinhaDeProducao& linha = s.AcessarLinha(lid);
00176     linha.AtualizarAlarmes();
00177     std::vector<int> falhas = linha.DiagnosticarLinha();
00178     if (falhas.empty()) {
00179         std::cout << "Nenhuma falha detectada na linha.\n";
00180     } else {
00181         std::cout << "Parametros em falha (IDs): ";
00182         for (int id : falhas) {
00183             std::cout << id << ' ';
00184         }
00185         std::cout << "\nAlarmes ativos: " << linha.QuantidadeAlarmes() << '\n';
00186     }
00187 }
00188
00189 void listar(Sistema& s) {
00190     const auto& linhas = s.GetLinhas();
00191     if (linhas.empty()) {
00192         std::cout << "Nenhuma linha cadastrada.\n";
00193         return;
00194     }
00195     for (const auto& par : linhas) {
00196         std::cout << "== Linha " << par.first << " ==\n";
00197         par.second.ExibirTudo();
00198     }
00199 }
00200
00201 void removerLinha(Sistema& s) {
00202     exigir(s, rank(Cargo::ENGENHEIRO));
00203     int id = lerInt("ID da linha a remover: ");
00204     s.RemoverLinha(id);
00205     std::cout << "Linha removida.\n";
00206 }
00207
00208 void gerenciarUsuarios(Sistema& s) {
00209     exigir(s, rank(Cargo::ADMIN));
00210     std::cout << "1-Criar 2-Remover\n";
00211     int op = lerInt("Opcao: ");
00212     if (op == 1) {
00213         std::string login = lerLinha("Login: ");
00214         std::string senha = lerLinha("Senha: ");
00215         std::cout << "Cargo: 1-Admin 2-Engenheiro 3-Tecnico\n";
00216         int c = lerInt("Opcao: ");
00217         Cargo cargo = (c == 1) ? Cargo::ADMIN
00218             : (c == 2) ? Cargo::ENGENHEIRO
00219             : Cargo::TECNICO;
00220         s.AdicionarUsuario(login, senha, cargo);
00221     } else if (op == 2) {
00222         std::string login = lerLinha("Login a remover: ");
00223         s.RemoverUsuario(login);
00224     }
00225 }
00226
00227 void menu(Sistema& s) {
00228     bool continuar = true;
00229     while (continuar) {
00230         limparTela();
00231         Usuario* u = s.GetUsuarioLogado();
00232         std::cout << "\n== Menu (" << (u ? cargoTexto(u->GetCargo()) : "?")
00233             << ") ==\n"
00234             << "1 - Listar tudo\n"
00235             << "2 - Criar linha\n"
00236             << "3 - Remover linha\n"
00237             << "4 - Criar conjunto\n"
00238             << "5 - Criar equipamento\n"
00239             << "6 - Criar parametro\n"
00240             << "7 - Diagnosticar linha\n"
00241             << "8 - Gerenciar usuarios\n"
00242             << "9 - Salvar\n"
00243             << "10 - Carregar\n"
00244             << "0 - Logout\n";
00245         int op = lerInt("Opcao: ");
00246         try {
00247             switch (op) {
00248                 case 1: listar(s); break;
00249                 case 2: criarLinha(s); break;
00250                 case 3: removerLinha(s); break;
00251                 case 4: criarConjunto(s); break;
00252                 case 5: criarEquipamento(s); break;
00253                 case 6: criarParametro(s); break;
00254                 case 7: diagnosticar(s); break;
00255                 case 8: gerenciarUsuarios(s); break;
00256                 case 9:

```

```

00257         std::cout << (s.SalvarAlteracoes() ? "Salvo.\n"
00258                                     : "Falha ao salvar.\n");
00259         break;
00260     case 10:
00261         std::cout << (s.CarregarUltimoSave() ? "Carregado.\n"
00262                                     : "Falha ao carregar.\n");
00263         break;
00264     case 0:
00265         s.Logout();
00266         continuar = false;
00267         break;
00268     default:
00269         std::cout << "Opcao invalida.\n";
00270     }
00271     } catch (const ExcecaoSistema& e) {
00272         std::cout << "[Erro] " << e.what() << '\n';
00273     }
00274     if (continuar) {
00275         pausar();
00276     }
00277 }
00278 }
00279
00280 } // namespace
00281
00282 int main() {
00283     std::cout << "Sistema de Tolerancia de Falha em Fabricas\n";
00284
00285     Sistema s;
00286
00287     // Carrega usuarios/estrutura inicial fornecidos pelo grupo.
00288     if (!s.CarregarSave(ARQUIVO_INICIAL)) {
00289         std::cout << "(Aviso) Arquivo inicial '" << ARQUIVO_INICIAL
00290                 << "' nao encontrado; criando admin padrao (admin/admin123).\n";
00291         s.AdicionarUsuario("admin", "admin123", Cargo::ADMIN);
00292     }
00293
00294     bool executando = true;
00295     while (executando) {
00296         limparTela();
00297         std::cout << "\n--- Login (login vazio encerra) ---\n";
00298         std::string login = lerLinha("Login: ");
00299         if (login.empty()) {
00300             executando = false;
00301             break;
00302         }
00303         std::string senha = lerLinha("Senha: ");
00304
00305         if (s.Login(login, senha)) {
00306             menu(s);
00307         } else {
00308             std::cout << "Credenciais invalidas.\n";
00309         }
00310     }
00311
00312     std::cout << "Encerrando.\n";
00313     return 0;
00314 }

```

12.52 src/Motor.cpp File Reference

```
#include "Motor.hpp"
```

12.53 Motor.cpp

[Go to the documentation of this file.](#)

```

00001 #include "Motor.hpp"
00002
00003 Motor::Motor(int id, std::string desc, double velocidade, double torque)
00004     : Equipamento(id, desc, "Motor") {
00005     this->AdicionarParametro(velocidade, "Velocidade nominal do motor", id * 100 + 1);
00006     this->AdicionarParametro(torque, "Torque nominal do motor", id * 100 + 2);
00007 }
00008
00009 std::string Motor::Tipo() const { return "Motor"; }

```

12.54 src/Parametro.cpp File Reference

```
#include "Parametro.hpp"
#include "Excecoes.hpp"
#include <limits>
```

12.55 Parametro.cpp

[Go to the documentation of this file.](#)

```
00001 #include "Parametro.hpp"
00002 #include "Excecoes.hpp"
00003
00004 #include <limits>
00005
00006 Parametro::Parametro(double ValorAtual, std::string Desc, int id) {
00007     if (Desc.empty()) {
00008         throw EntradaInvalida("Parametro: descricao vazia");
00009     }
00010     if (id < 0) {
00011         throw IdInvalido("Parametro: id nao pode ser negativo");
00012     }
00013
00014     this->_Estado = false;
00015     this->_Min = std::numeric_limits<double>::lowest();
00016     this->_Max = std::numeric_limits<double>::max();
00017
00018     // Uma leitura pode estar fora dos limites (caso de falha): sempre armazenada.
00019     this->_ValorAtual = ValorAtual;
00020     this->_Hist.push_back(this->_ValorAtual);
00021
00022     this->_Desc = Desc;
00023     this->_id = id;
00024 }
00025
00026 Parametro::Parametro(double ValorAtual, double Max, double Min, std::string Desc, int id) {
00027     if (Desc.empty()) {
00028         throw EntradaInvalida("Parametro: descricao vazia");
00029     }
00030     if (id < 0) {
00031         throw IdInvalido("Parametro: id nao pode ser negativo");
00032     }
00033     if (Max < Min) {
00034         throw LimitesInvalidos("Parametro: Max nao pode ser menor que Min");
00035     }
00036
00037     this->_Estado = false;
00038     this->_Min = Min;
00039     this->_Max = Max;
00040
00041     // Uma leitura pode estar fora dos limites (caso de falha): sempre armazenada.
00042     this->_ValorAtual = ValorAtual;
00043     this->_Hist.push_back(this->_ValorAtual);
00044
00045     this->_Desc = Desc;
00046     this->_id = id;
00047 }
00048
00049 bool Parametro::ValorValido(double Valor) const {
00050     return (Valor >= this->_Min) && (Valor <= this->_Max);
00051 }
00052
00053 void Parametro::SetValorAtual(double Valor) {
00054     this->_ValorAtual = Valor;
00055     this->_Hist.push_back(this->_ValorAtual);
00056 }
00057
00058 void Parametro::SetMax(double Valor) {
00059     if (Valor < this->_Min) {
00060         throw LimitesInvalidos("Parametro: Max nao pode ser menor que Min");
00061     }
00062     this->_Max = Valor;
00063 }
00064
00065 void Parametro::SetMin(double Valor) {
00066     if (Valor > this->_Max) {
00067         throw LimitesInvalidos("Parametro: Min nao pode ser maior que Max");
```

```

00068     }
00069     this->_Min = Valor;
00070 }
00071
00072 void Parametro::SetDesc(std::string Desc) {
00073     if (Desc.empty()) {
00074         throw EntradaInvalida("Parametro: descricao vazia");
00075     }
00076     this->_Desc = Desc;
00077 }
00078
00079 bool Parametro::DiagnosticarParametro() {
00080     this->_Estado = !this->ValorValido(this->_ValorAtual);
00081     return this->_Estado;
00082 }
00083
00084 double Parametro::GetValorAtual() const {
00085     return this->_ValorAtual;
00086 }
00087
00088 double Parametro::GetMax() const {
00089     return this->_Max;
00090 }
00091
00092 double Parametro::GetMin() const {
00093     return this->_Min;
00094 }
00095
00096 std::string Parametro::GetDesc() const {
00097     return this->_Desc;
00098 }
00099
00100 bool Parametro::GetEstado() const {
00101     return this->_Estado;
00102 }
00103
00104 const std::vector<double>& Parametro::GetHist() const {
00105     return this->_Hist;
00106 }
00107
00108 int Parametro::GetId() {
00109     return this->_id;
00110 }
00111
00112 void Parametro::ResetarLimites() {
00113     this->_Min = std::numeric_limits<double>::lowest();
00114     this->_Max = std::numeric_limits<double>::max();
00115 }
00116
00117 void Parametro::ResetarHistorico() {
00118     this->_Hist.clear();
00119 }

```

12.56 src/Sensor.cpp File Reference

```
#include "Sensor.hpp"
```

12.57 Sensor.cpp

[Go to the documentation of this file.](#)

```

00001 #include "Sensor.hpp"
00002
00003 Sensor::Sensor(int id, std::string desc, double sinal)
00004     : Equipamento(id, desc, "Sensor") {
00005     this->AdicionarParametro(sinal, "Sinal de saida do sensor", id * 100 + 1);
00006 }
00007
00008 std::string Sensor::Tipo() const { return "Sensor"; }

```

12.58 src/Sistema.cpp File Reference

```
#include "Sistema.hpp"
#include "Excecoes.hpp"
#include <iostream>
#include <algorithm>
#include <utility>
#include <fstream>
#include <sstream>
```

12.59 Sistema.cpp

[Go to the documentation of this file.](#)

```
00001 #include "Sistema.hpp"
00002 #include "Excecoes.hpp"
00003
00004 #include <iostream>
00005 #include <algorithm>
00006 #include <utility>
00007 #include <fstream>
00008 #include <sstream>
00009
00010 Sistema::Sistema() : _UsuarioLogadoIdx(-1), _arquivoSave("save.txt") {
00011     this->_ProximoId = 1;
00012 }
00013
00014 void Sistema::AdicionarLinha(int id, std::string desc, std::string local, std::string nome) {
00015     if (this->IdAtivo(id)) {
00016         throw IdDuplicado("Sistema: linha com id ja existente");
00017     }
00018     LinhaDeProducao Linha(id, desc, local, nome);
00019
00020     this->_Linhas.emplace(id, std::move(Linha));
00021     if (id >= this->_ProximoId) {
00022         this->_ProximoId = id + 1;
00023     }
00024 }
00025
00026 int Sistema::AdicionarLinha(std::string desc, std::string local, std::string nome) {
00027     int id = this->_ProximoId;
00028     Sistema::AdicionarLinha(id, desc, local, nome);
00029     return id;
00030 }
00031
00032 void Sistema::RemoverLinha(int id) {
00033     if (this->_Linhas.count(id) == 0) {
00034         throw IdInexistente("Sistema: linha inexistente");
00035     }
00036     this->_Linhas.erase(id);
00037 }
00038
00039 const std::map<int, LinhaDeProducao>& Sistema::GetLinhas() const {
00040     return _Linhas;
00041 }
00042
00043 LinhaDeProducao& Sistema::AcessarLinha(int id) {
00044     auto it = this->_Linhas.find(id);
00045     if (it == this->_Linhas.end()) {
00046         throw IdInexistente("Sistema: linha inexistente");
00047     }
00048     return it->second;
00049 }
00050
00051 void Sistema::ExibirLinhas() {
00052     std::cout << "Linhas de producao:" << std::endl;
00053     for (const auto& linha : this->_Linhas) {
00054         std::cout << "Linha ID: " << linha.first << " - " << linha.second.GetNome() << std::endl
00055             << "Local: " << linha.second.GetLocal() << std::endl
00056             << "Descricao: " << linha.second.GetDesc() << std::endl;
00057     }
00058 }
00059
00060 void Sistema::AdicionarUsuario(std::string login, std::string senha, Cargo cargo) {
00061     Usuario user(login, senha, cargo);
```



```

00062
00063     this->_Usuarios.push_back(user);
00064
00065     std::cout << "Usuario " << login << " adicionado." << std::endl;
00066 }
00067
00068 void Sistema::RemoverUsuario(std::string login) {
00069
00070     if (this->_Usuarios.size() < 1){
00071         return;
00072     }
00073
00074     for (size_t i = 0; i < this->_Usuarios.size(); i++){
00075         if (this->_Usuarios[i].GetLogin() == login){
00076             this->_Usuarios.erase(this->_Usuarios.begin() + i);
00077             std::cout << "Usuario " << login << " removido." << std::endl;
00078
00079             return;
00080         }
00081     }
00082     std::cout << "Nenhum usuario removido." << std::endl;
00083 }
00084
00085 int Sistema::ProximoIdDisponivel() const {
00086     return this->_ProximoId;
00087 }
00088
00089 bool Sistema::IdAtivo(int Id) const {
00090     return this->_Linhas.count(Id) > 0;
00091 }
00092
00093 bool Sistema::Login(std::string login, std::string senha) {
00094
00095     for (size_t i = 0; i < this->_Usuarios.size(); i++){
00096         if (this->_Usuarios[i].GetLogin() == login && this->_Usuarios[i].ValidarSenha(senha) == true){
00097             this->_UsuarioLogadoIdx = static_cast<int>(i);
00098
00099             std::cout << "Usuario " << login << " logado." << std::endl;
00100
00101             return true;
00102         }
00103     }
00104     std::cout << "Usuario " << login << " nao encontrado." << std::endl;
00105     return false;
00106 }
00107 }
00108
00109 bool Sistema::Logout() {
00110     if (this->_UsuarioLogadoIdx != -1){
00111         this->_UsuarioLogadoIdx = -1;
00112         return true;
00113     }
00114
00115     return false;
00116 }
00117
00118 Usuario* Sistema::GetUsuarioLogado() const {
00119     if (this->_UsuarioLogadoIdx == -1){
00120         return nullptr;
00121     }
00122
00123     return const_cast<Usuario*>(&this->_Usuarios[this->_UsuarioLogadoIdx]);
00124 }
00125
00126 namespace {
00127
00128 std::string cargoParaTexto(Cargo c) {
00129     switch (c) {
00130         case Cargo::ADMIN:      return "ADMIN";
00131         case Cargo::ENGENHEIRO: return "ENGENHEIRO";
00132         case Cargo::TECNICO:    return "TECNICO";
00133     }
00134     return "TECNICO";
00135 }
00136
00137 Cargo textoParaCargo(const std::string& s) {
00138     if (s == "ADMIN")      return Cargo::ADMIN;
00139     if (s == "ENGENHEIRO") return Cargo::ENGENHEIRO;
00140     return Cargo::TECNICO;
00141 }
00142
00143 // Divide uma linha por separador, preservando campos vazios (inclusive finais).
00144 std::vector<std::string> dividir(const std::string& linha, char sep) {
00145     std::vector<std::string> campos;
00146     std::string atual;
00147     std::istringstream iss(linha);
00148     while (std::getline(iss, atual, sep)) {

```

```

00149         campos.push_back(atual);
00150     }
00151     return campos;
00152 }
00153
00154 } // namespace
00155
00156 bool Sistema::SalvarAlteracoes() {
00157     std::ofstream ofs(this->_arquivoSave);
00158     if (!ofs.is_open()) {
00159         return false;
00160     }
00161
00162     for (const auto& u : this->_Usuarios) {
00163         ofs << "USUARIO;" << u.GetLogin() << ';' << u.GetSenha() << ';'
00164             << cargoParaTexto(u.GetCargo()) << '\n';
00165     }
00166
00167     for (const auto& linhaPar : this->_Linhas) {
00168         int lid = linhaPar.first;
00169         const LinhaDeProducao& linha = linhaPar.second;
00170         ofs << "LINHA;" << lid << ';' << linha.GetNome() << ';'
00171             << linha.GetLocal() << ';' << linha.GetDesc() << '\n';
00172
00173         for (const auto& conjPar : linha.GetConjuntos()) {
00174             int cid = conjPar.first;
00175             const Conjunto& conj = conjPar.second;
00176             ofs << "CONJUNTO;" << lid << ';' << cid << ';'
00177                 << conj.GetNome() << ';' << conj.GetDesc() << '\n';
00178
00179             for (const auto& equipPar : conj.GetEquipamentos()) {
00180                 int eid = equipPar.first;
00181                 const Equipamento& eq = *equipPar.second;
00182                 ofs << "EQUIP;" << lid << ';' << cid << ';' << eid << ';'
00183                     << eq.GetDesc() << ';' << eq.GetNome() << '\n';
00184
00185                 for (const auto& paramPar : eq.GetParametros()) {
00186                     const Parametro& p = paramPar.second;
00187                     ofs << "PARAM;" << lid << ';' << cid << ';' << eid << ';'
00188                         << paramPar.first << ';' << p.GetValorAtual() << ';'
00189                         << p.GetMin() << ';' << p.GetMax() << ';'
00190                         << p.GetDesc() << '\n';
00191                 }
00192             }
00193         }
00194     }
00195
00196     return ofs.good();
00197 }
00198
00199 bool Sistema::CarregarUltimoSave() {
00200     return this->CarregarSave(this->_arquivoSave);
00201 }
00202
00203 bool Sistema::CarregarSave(std::string arquivo) {
00204     std::ifstream ifs(arquivo);
00205     if (!ifs.is_open()) {
00206         return false;
00207     }
00208
00209     // Substitui o estado atual pelo conteúdo do arquivo.
00210     this->_Linhas.clear();
00211     this->_Usuarios.clear();
00212     this->_ProximoId = 1;
00213     this->_UsuarioLogadoIdx = -1;
00214
00215     try {
00216         std::string linha;
00217         while (std::getline(ifs, linha)) {
00218             if (linha.empty()) {
00219                 continue;
00220             }
00221             std::vector<std::string> t = dividir(linha, ';');
00222
00223             if (t[0] == "USUARIO" && t.size() >= 4) {
00224                 this->AdicionarUsuario(t[1], t[2], textoParaCargo(t[3]));
00225             } else if (t[0] == "LINHA" && t.size() >= 5) {
00226                 this->AdicionarLinha(std::stoi(t[1]), t[4], t[3], t[2]);
00227             } else if (t[0] == "CONJUNTO" && t.size() >= 5) {
00228                 this->AcessarLinha(std::stoi(t[1]))
00229                     .AdicionarConjunto(std::stoi(t[2]), t[3], t[4]);
00230             } else if (t[0] == "EQUIP" && t.size() >= 6) {
00231                 int lid = std::stoi(t[1]);
00232                 int cid = std::stoi(t[2]);
00233                 int eid = std::stoi(t[3]);
00234                 auto eq = std::make_unique<Equipamento>(eid, t[4], t[5]);
00235                 this->AcessarLinha(lid).AcessarConjunto(cid)

```

```

00236         .AdicionarEquipamento(std::move(eq));
00237     } else if (t[0] == "PARAM" && t.size() >= 9) {
00238         int lid = std::stoi(t[1]);
00239         int cid = std::stoi(t[2]);
00240         int eid = std::stoi(t[3]);
00241         int pid = std::stoi(t[4]);
00242         double valor = std::stod(t[5]);
00243         double minimo = std::stod(t[6]);
00244         double maximo = std::stod(t[7]);
00245         this->AcessarLinha(lid).AcessarConjunto(cid)
00246             .AcessarEquipamento(eid)
00247             .AdicionarParametro(valor, maximo, minimo, t[8], pid);
00248     }
00249 }
00250 } catch (const std::exception&) {
00251     return false;
00252 }
00253
00254 return true;
00255 }

```

12.60 src/Usuario.cpp File Reference

```

#include "Usuario.hpp"
#include "Excecoes.hpp"

```

12.61 Usuario.cpp

[Go to the documentation of this file.](#)

```

00001 #include "Usuario.hpp"
00002 #include "Excecoes.hpp"
00003
00004 Usuario::Usuario(std::string login, std::string senha, Cargo cargo) {
00005     if (login.empty()) {
00006         throw EntradaInvalida("Usuario: login vazio");
00007     }
00008     if (senha.empty()) {
00009         throw EntradaInvalida("Usuario: senha vazia");
00010     }
00011     this->_login = login;
00012     this->_senha = senha;
00013     this->_cargo = cargo;
00014 }
00015
00016 bool Usuario::ValidarSenha(std::string senha) const {
00017     if (senha == this->_senha){
00018         return true;
00019     }
00020     return false;
00021 }
00022
00023
00024 std::string Usuario::GetLogin() const {
00025     return this->_login;
00026 }
00027
00028 Cargo Usuario::GetCargo(std::string login) {
00029     (void)login;
00030     return this->_cargo;
00031 }
00032
00033 Cargo Usuario::GetCargo() const {
00034     return this->_cargo;
00035 }
00036
00037 std::string Usuario::GetSenha() const {
00038     return this->_senha;
00039 }

```

12.62 src/Valvula.cpp File Reference

```
#include "Valvula.hpp"
```

12.63 Valvula.cpp

[Go to the documentation of this file.](#)

```
00001 #include "Valvula.hpp"
00002
00003 Valvula::Valvula(int id, std::string desc, double vazao)
00004     : Equipamento(id, desc, "Valvula") {
00005     this->AdicionarParametro(vazao, "Vazao nominal da valvula", id * 100 + 1);
00006 }
00007
00008 std::string Valvula::Tipo() const { return "Valvula"; }
```

12.64 tests/doctest.h File Reference

```
#include <signal.h>
```

Classes

- struct [doctest::detail::types::enable_if< COND, T >](#)
- struct [doctest::detail::types::enable_if< true, T >](#)
- struct [doctest::detail::types::true_type](#)
- struct [doctest::detail::types::false_type](#)
- struct [doctest::detail::types::remove_reference< T >](#)
- struct [doctest::detail::types::remove_reference< T & >](#)
- struct [doctest::detail::types::remove_reference< T && >](#)
- struct [doctest::detail::types::is_same< T, U >](#)
- struct [doctest::detail::types::is_same< T, T >](#)
- struct [doctest::detail::types::is_rvalue_reference< T >](#)
- struct [doctest::detail::types::is_rvalue_reference< T && >](#)
- struct [doctest::detail::types::remove_const< T >](#)
- struct [doctest::detail::types::remove_const< const T >](#)
- struct [doctest::detail::types::is_enum< T >](#)
- struct [doctest::detail::types::underlying_type< T >](#)
- struct [doctest::detail::types::is_pointer< T >](#)
- struct [doctest::detail::types::is_pointer< T * >](#)
- struct [doctest::detail::types::is_array< T >](#)
- struct [doctest::detail::types::is_array< T\[SIZE\]>](#)
- struct [doctest::detail::deferred_false< T >](#)
- struct [doctest::detail::is_std_string< T, Enable >](#)
- struct [doctest::detail::is_std_string< T, typename types::enable_if< types::is_same< decltype\(declval< const T & >\(\).c_str\(\)\), const char * >::value &&types::is_same< decltype\(declval< const T & >\(\).size\(\)\), size_t >::value >::type >](#)
- class [doctest::String](#)
- struct [doctest::detail::has_insertion_operator< T, typename >](#)
- struct [doctest::detail::has_insertion_operator< T, decltype\(operator<<\(declval< std::ostream & >\(\)\), declval< const T & >\(\)\), void\(\)\)>](#)

- struct `doctest::detail::should_stringify_as_underlying_type< T >`
- struct `doctest::detail::is_pair< T, typename >`
- struct `doctest::detail::is_pair< T, types::void_t< typename T::first_type, typename T::second_type, decltype(declval< T >().first), decltype(declval< T >().second)> >`
- struct `doctest::detail::is_container< T, typename >`
- struct `doctest::detail::is_container< T, types::void_t< typename T::value_type, typename T::iterator, decltype(declval< T >().begin()), decltype(declval< T >().end())> >`
- struct `doctest::detail::StringMakerBase< C >`
- struct `doctest::detail::StringMakerBase< true >`
- struct `doctest::StringMaker< T >`
- struct `doctest::detail::filldata< T, Enable >`
- struct `doctest::detail::filldata< T[N]>`
- struct `doctest::detail::filldata< const char[N]>`
- struct `doctest::detail::filldata< const void * >`
- struct `doctest::detail::filldata< const volatile void * >`
- struct `doctest::detail::filldata< T * >`
- struct `doctest::detail::filldata< T, typename detail::types::enable_if< !detail::has_insertion_operator< T >::↵ value &&detail::is_pair< T >::value >::type >`
- struct `doctest::detail::filldata< T, typename detail::types::enable_if< !detail::has_insertion_operator< T >::↵ value &&detail::is_container< T >::value >::type >`
- class `doctest::Contains`
- struct `doctest::Approx`
- struct `doctest::IsNaN< F >`
- struct `doctest::ContextOptions`
- struct `doctest::AssertData`
- class `doctest::AssertData::StringContains`
- struct `doctest::detail::RelationalComparator< int, L, R >`
- struct `doctest::detail::Result`
- struct `doctest::detail::ResultBuilder`
- struct `doctest::detail::Expression_lhs< L >`
- struct `doctest::detail::ExpressionDecomposer`
- struct `doctest::SubcaseSignature`
- struct `doctest::detail::Subcase`
- struct `doctest::detail::TestSuite`
- struct `doctest::TestCaseData`
- struct `doctest::detail::TestCase`
- struct `doctest::detail::IExceptionTranslator`
- class `doctest::detail::ExceptionTranslator< T >`
- struct `doctest::IContextScope`
- struct `doctest::detail::ContextScopeBase`
- class `doctest::detail::ContextScope< L >`
- struct `doctest::MessageData`
- struct `doctest::detail::MessageBuilder`
- struct `doctest::detail::TestFailureException`
- class `doctest::Context`
- struct `doctest::CurrentTestCaseStats`
- struct `doctest::TestCaseException`
- struct `doctest::TestRunStats`
- struct `doctest::QueryData`
- struct `doctest::IReporter`

Namespaces

- namespace [doctest](#)
- namespace [doctest::detail](#)
- namespace [std](#)
- namespace [doctest::detail::types](#)
- namespace [doctest::assertType](#)
- namespace [doctest::detail::binaryAssertComparison](#)
- namespace [doctest::Color](#)
- namespace [doctest_detail_test_suite_ns](#)
- namespace [doctest::TestCaseFailureReason](#)

Macros

- `#define DOCTEST_PARTS_PUBLIC_VERSION`
- `#define DOCTEST_VERSION_MAJOR 2`
- `#define DOCTEST_VERSION_MINOR 5`
- `#define DOCTEST_VERSION_PATCH 0`
- `#define DOCTEST_TOSTR_IMPL(x)`
- `#define DOCTEST_TOSTR(x)`
- `#define DOCTEST_VERSION_STR`
- `#define DOCTEST_VERSION (DOCTEST_VERSION_MAJOR * 10000 + DOCTEST_VERSION_MINOR * 100 + DOCTEST_VERSION_PATCH)`
- `#define DOCTEST_PARTS_PUBLIC_COMPILER`
- `#define DOCTEST_CPLUSPLUS __cplusplus`
- `#define DOCTEST_COMPILER(MAJOR, MINOR, PATCH)`
- `#define DOCTEST_MSVC 0`
- `#define DOCTEST_CLANG 0`
- `#define DOCTEST_GCC 0`
- `#define DOCTEST_ICC 0`
- `#define DOCTEST_PARTS_PUBLIC_WARNINGS`
- `#define DOCTEST_CLANG_SUPPRESS_WARNING_PUSH`
- `#define DOCTEST_CLANG_SUPPRESS_WARNING(w)`
- `#define DOCTEST_CLANG_SUPPRESS_WARNING_POP`
- `#define DOCTEST_CLANG_SUPPRESS_WARNING_WITH_PUSH(w)`
- `#define DOCTEST_GCC_SUPPRESS_WARNING_PUSH`
- `#define DOCTEST_GCC_SUPPRESS_WARNING(w)`
- `#define DOCTEST_GCC_SUPPRESS_WARNING_POP`
- `#define DOCTEST_GCC_SUPPRESS_WARNING_WITH_PUSH(w)`
- `#define DOCTEST_MSVC_SUPPRESS_WARNING_PUSH`
- `#define DOCTEST_MSVC_SUPPRESS_WARNING(w)`
- `#define DOCTEST_MSVC_SUPPRESS_WARNING_POP`
- `#define DOCTEST_MSVC_SUPPRESS_WARNING_WITH_PUSH(w)`
- `#define DOCTEST_SUPPRESS_COMMON_WARNINGS_PUSH`
- `#define DOCTEST_SUPPRESS_COMMON_WARNINGS_POP`
- `#define DOCTEST_SUPPRESS_PUBLIC_WARNINGS_PUSH`
- `#define DOCTEST_SUPPRESS_PUBLIC_WARNINGS_POP DOCTEST_SUPPRESS_COMMON_WARNINGS_POP`
- `#define DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH`
- `#define DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP DOCTEST_SUPPRESS_COMMON_WARNINGS_POP`
- `#define DOCTEST_MAKE_STD_HEADERS_CLEAN_FROM_WARNINGS_ON_WALL_BEGIN`
- `#define DOCTEST_MAKE_STD_HEADERS_CLEAN_FROM_WARNINGS_ON_WALL_END DOCTEST_MSVC_SUPPRESS_WARNING_POP`
- `#define DOCTEST_PARTS_PUBLIC_CONFIG`
- `#define DOCTEST_PARTS_PUBLIC_PLATFORM`
- `#define DOCTEST_PLATFORM_LINUX`

- #define [DOCTEST_CONFIG_POSIX_SIGNALS](#)
- #define [DOCTEST_CONFIG_NO_EXCEPTIONS](#)
- #define [DOCTEST_CONFIG_NO_TRY_CATCH_IN_ASSERTS](#)
- #define [DOCTEST_SYMBOL_EXPORT](#) __attribute__((visibility("default")))
- #define [DOCTEST_SYMBOL_IMPORT](#)
- #define [DOCTEST_INTERFACE](#)
- #define [DOCTEST_INTERFACE_DECL](#) DOCTEST_INTERFACE
- #define [DOCTEST_INTERFACE_DEF](#)
- #define [DOCTEST_EMPTY](#)
- #define [DOCTEST_NOINLINE](#) __attribute__((noinline))
- #define [DOCTEST_UNUSED](#) __attribute__((unused))
- #define [DOCTEST_ALIGNMENT](#)(x)
- #define [DOCTEST_INLINE_NOINLINE](#) inline [DOCTEST_NOINLINE](#)
- #define [DOCTEST_NORETURN](#) [[noreturn]]
- #define [DOCTEST_NOEXCEPT](#) noexcept
- #define [DOCTEST_CONSTEXPR](#) constexpr
- #define [DOCTEST_CONSTEXPR_FUNC](#) constexpr
- #define [DOCTEST_NO_SANITIZE_INTEGER](#)
- #define [DOCTEST_HAS_BUILTIN](#)(x)
- #define [DOCTEST_PARTS_PUBLIC_UTILITY](#)
- #define [DOCTEST_DECLARE_INTERFACE](#)(name)
- #define [DOCTEST_DEFINE_INTERFACE](#)(name)
- #define [DOCTEST_COUNTER](#) __LINE__
- #define [DOCTEST_CAT_IMPL](#)(s1, s2)
- #define [DOCTEST_CAT](#)(s1, s2)
- #define [DOCTEST_ANONYMOUS](#)(x)
- #define [DOCTEST_REF_WRAP](#)(x)
- #define [DOCTEST_GLOBAL_NO_WARNINGS](#)(var, ...)
- #define [DOCTEST_PARTS_PUBLIC_DEBUGGER](#)
- #define [DOCTEST_BREAK_INTO_DEBUGGER](#)()
- #define [DOCTEST_PARTS_PUBLIC_STD_FWD](#)
- #define [DOCTEST_PARTS_PUBLIC_STD_TYPE_TRAITS](#)
- #define [DOCTEST_PARTS_PUBLIC_STD_UTILITY](#)
- #define [DOCTEST_PARTS_PUBLIC_STRING](#)
- #define [DOCTEST_CONFIG_STRING_SIZE_TYPE](#) unsigned
- #define [DOCTEST_STRINGIFY](#)(...)
- #define [DOCTEST_PARTS_PUBLIC_MATCHERS_CONTAINS](#)
- #define [DOCTEST_PARTS_PUBLIC_MATCHERS_APPROX](#)
- #define [DOCTEST_PARTS_PUBLIC_MATCHERS_IS_NAN](#)
- #define [DOCTEST_PARTS_PUBLIC_CONTEXT_OPTIONS](#)
- #define [DOCTEST_PARTS_PUBLIC_ASSERT_TYPE](#)
- #define [DOCTEST_PARTS_PUBLIC_ASSERT_DATA](#)
- #define [DOCTEST_PARTS_PUBLIC_ASSERT_COMPARATOR](#)
- #define [DOCTEST_COMPARISON_RETURN_TYPE](#) bool
- #define [DOCTEST_RELATIONAL_OP](#)(name, op)
- #define [DOCTEST_CMP_EQ](#)(l, r)
- #define [DOCTEST_CMP_NE](#)(l, r)
- #define [DOCTEST_CMP_GT](#)(l, r)
- #define [DOCTEST_CMP_LT](#)(l, r)
- #define [DOCTEST_CMP_GE](#)(l, r)
- #define [DOCTEST_CMP_LE](#)(l, r)
- #define [DOCTEST_BINARY_RELATIONAL_OP](#)(n, op)
- #define [DOCTEST_PARTS_PUBLIC_ASSERT_RESULT](#)
- #define [DOCTEST_FORBIT_EXPRESSION](#)(rt, op)
- #define [DOCTEST_PARTS_PUBLIC_ASSERT_EXPRESSION](#)

- `#define SFINAE_OP(ret, op)`
- `#define DOCTEST_DO_BINARY_EXPRESSION_COMPARISON(op, op_str, op_macro)`
- `#define DOCTEST_PARTS_PUBLIC_COLOR`
- `#define DOCTEST_PARTS_PUBLIC_SUBCASE`
- `#define DOCTEST_PARTS_PUBLIC_TEST_SUITE`
- `#define DOCTEST_PARTS_PUBLIC_TEST_CASE`
- `#define DOCTEST_PARTS_PUBLIC_DECORATORS`
- `#define DOCTEST_DEFINE_DECORATOR(name, type, def)`
- `#define DOCTEST_PARTS_PUBLIC_EXCEPTION_TRANSLATOR`
- `#define DOCTEST_PARTS_PUBLIC_CONTEXT_SCOPE`
- `#define DOCTEST_PARTS_PUBLIC_ASSERT_MESSAGE`
- `#define DOCTEST_PARTS_PUBLIC_PATH`
- `#define DOCTEST_PARTS_PUBLIC_EXCEPTIONS`
- `#define DOCTEST_PARTS_PUBLIC_CONTEXT`
- `#define DOCTEST_PARTS_PUBLIC_ASSERT_HANDLER`
- `#define DOCTEST_ASSERT_OUT_OF_TESTS(decomp)`
- `#define DOCTEST_ASSERT_IN_TESTS(decomp)`
- `#define DOCTEST_PARTS_PUBLIC_REPORTER`
- `#define DOCTEST_PARTS_PUBLIC_GENERATOR`
- `#define DOCTEST_PARTS_PUBLIC_MACROS`
- `#define DOCTEST_FUNC_EMPTY (void)0`
- `#define DOCTEST_FUNC_SCOPE_BEGIN do /* NOLINT(cppcoreguidelines-avoid-do-while)*/`
- `#define DOCTEST_FUNC_SCOPE_END while (false)`
- `#define DOCTEST_FUNC_SCOPE_RET(v)`
- `#define DOCTEST_ASSERT_LOG_REACT_RETURN(b)`
- `#define DOCTEST_WRAP_IN_TRY(x)`
- `#define DOCTEST_CAST_TO_VOID(...)`
- `#define DOCTEST_REGISTER_FUNCTION(global_prefix, f, decorators)`
- `#define DOCTEST_IMPLEMENT_FIXTURE(der, base, func, decorators)`
- `#define DOCTEST_CREATE_AND_REGISTER_FUNCTION(f, decorators)`
- `#define DOCTEST_CREATE_AND_REGISTER_FUNCTION_IN_CLASS(f, proxy, decorators)`
- `#define DOCTEST_TEST_CASE(decorators)`
- `#define DOCTEST_TEST_CASE_CLASS(...)`
- `#define DOCTEST_TEST_CASE_FIXTURE(c, decorators)`
- `#define DOCTEST_TYPE_TO_STRING_AS(str, ...)`
- `#define DOCTEST_TYPE_TO_STRING(...)`
- `#define DOCTEST_TEST_CASE_TEMPLATE_DEFINE_IMPL(dec, T, iter, func)`
- `#define DOCTEST_TEST_CASE_TEMPLATE_DEFINE(dec, T, id)`
- `#define DOCTEST_TEST_CASE_TEMPLATE_INSTANTIATE_IMPL(id, anon, ...)`
- `#define DOCTEST_TEST_CASE_TEMPLATE_INVOKE(id, ...)`
- `#define DOCTEST_TEST_CASE_TEMPLATE_APPLY(id, ...)`
- `#define DOCTEST_TEST_CASE_TEMPLATE_IMPL(dec, T, anon, ...)`
- `#define DOCTEST_TEST_CASE_TEMPLATE(dec, T, ...)`
- `#define DOCTEST_SUBCASE(name)`
- `#define DOCTEST_GENERATE(...)`
- `#define DOCTEST_TEST_SUITE_IMPL(decorators, ns_name)`
- `#define DOCTEST_TEST_SUITE(decorators)`
- `#define DOCTEST_TEST_SUITE_BEGIN(decorators)`
- `#define DOCTEST_TEST_SUITE_END`
- `#define DOCTEST_REGISTER_EXCEPTION_TRANSLATOR_IMPL(translatorName, signature)`
- `#define DOCTEST_REGISTER_EXCEPTION_TRANSLATOR(signature)`
- `#define DOCTEST_REGISTER_REPORTER(name, priority, reporter)`
- `#define DOCTEST_REGISTER_LISTENER(name, priority, reporter)`
- `#define DOCTEST_INFO(...)`
- `#define DOCTEST_INFO_IMPL(mb_name, s_name, ...)`

- `#define DOCTEST_CAPTURE(x)`
- `#define DOCTEST_ADD_AT_IMPL(type, file, line, mb, ...)`
- `#define DOCTEST_ADD_MESSAGE_AT(file, line, ...)`
- `#define DOCTEST_ADD_FAIL_CHECK_AT(file, line, ...)`
- `#define DOCTEST_ADD_FAIL_AT(file, line, ...)`
- `#define DOCTEST_MESSAGE(...)`
- `#define DOCTEST_FAIL_CHECK(...)`
- `#define DOCTEST_FAIL(...)`
- `#define DOCTEST_TO_LVALUE(...)`
- `#define DOCTEST_ASSERT_IMPLEMENT_2(assert_type, ...)`
- `#define DOCTEST_ASSERT_IMPLEMENT_1(assert_type, ...)`
- `#define DOCTEST_BINARY_ASSERT(assert_type, comp, ...)`
- `#define DOCTEST_UNARY_ASSERT(assert_type, ...)`
- `#define DOCTEST_WARN(...)`
- `#define DOCTEST_CHECK(...)`
- `#define DOCTEST_REQUIRE(...)`
- `#define DOCTEST_WARN_FALSE(...)`
- `#define DOCTEST_CHECK_FALSE(...)`
- `#define DOCTEST_REQUIRE_FALSE(...)`
- `#define DOCTEST_WARN_MESSAGE(cond, ...)`
- `#define DOCTEST_CHECK_MESSAGE(cond, ...)`
- `#define DOCTEST_REQUIRE_MESSAGE(cond, ...)`
- `#define DOCTEST_WARN_FALSE_MESSAGE(cond, ...)`
- `#define DOCTEST_CHECK_FALSE_MESSAGE(cond, ...)`
- `#define DOCTEST_REQUIRE_FALSE_MESSAGE(cond, ...)`
- `#define DOCTEST_WARN_EQ(...)`
- `#define DOCTEST_CHECK_EQ(...)`
- `#define DOCTEST_REQUIRE_EQ(...)`
- `#define DOCTEST_WARN_NE(...)`
- `#define DOCTEST_CHECK_NE(...)`
- `#define DOCTEST_REQUIRE_NE(...)`
- `#define DOCTEST_WARN_GT(...)`
- `#define DOCTEST_CHECK_GT(...)`
- `#define DOCTEST_REQUIRE_GT(...)`
- `#define DOCTEST_WARN_LT(...)`
- `#define DOCTEST_CHECK_LT(...)`
- `#define DOCTEST_REQUIRE_LT(...)`
- `#define DOCTEST_WARN_GE(...)`
- `#define DOCTEST_CHECK_GE(...)`
- `#define DOCTEST_REQUIRE_GE(...)`
- `#define DOCTEST_WARN_LE(...)`
- `#define DOCTEST_CHECK_LE(...)`
- `#define DOCTEST_REQUIRE_LE(...)`
- `#define DOCTEST_WARN_UNARY(...)`
- `#define DOCTEST_CHECK_UNARY(...)`
- `#define DOCTEST_REQUIRE_UNARY(...)`
- `#define DOCTEST_WARN_UNARY_FALSE(...)`
- `#define DOCTEST_CHECK_UNARY_FALSE(...)`
- `#define DOCTEST_REQUIRE_UNARY_FALSE(...)`
- `#define DOCTEST_EXCEPTION_EMPTY_FUNC`
- `#define DOCTEST_REQUIRE DOCTEST_EXCEPTION_EMPTY_FUNC`
- `#define DOCTEST_REQUIRE_FALSE DOCTEST_EXCEPTION_EMPTY_FUNC`
- `#define DOCTEST_REQUIRE_MESSAGE DOCTEST_EXCEPTION_EMPTY_FUNC`
- `#define DOCTEST_REQUIRE_FALSE_MESSAGE DOCTEST_EXCEPTION_EMPTY_FUNC`
- `#define DOCTEST_REQUIRE_EQ DOCTEST_EXCEPTION_EMPTY_FUNC`

- `#define DOCTEST_REQUIRE_NE DOCTEST_EXCEPTION_EMPTY_FUNC`
- `#define DOCTEST_REQUIRE_GT DOCTEST_EXCEPTION_EMPTY_FUNC`
- `#define DOCTEST_REQUIRE_LT DOCTEST_EXCEPTION_EMPTY_FUNC`
- `#define DOCTEST_REQUIRE_GE DOCTEST_EXCEPTION_EMPTY_FUNC`
- `#define DOCTEST_REQUIRE_LE DOCTEST_EXCEPTION_EMPTY_FUNC`
- `#define DOCTEST_REQUIRE_UNARY DOCTEST_EXCEPTION_EMPTY_FUNC`
- `#define DOCTEST_REQUIRE_UNARY_FALSE DOCTEST_EXCEPTION_EMPTY_FUNC`
- `#define DOCTEST_WARN_THROWS(...)`
- `#define DOCTEST_CHECK_THROWS(...)`
- `#define DOCTEST_REQUIRE_THROWS(...)`
- `#define DOCTEST_WARN_THROWS_AS(expr, ...)`
- `#define DOCTEST_CHECK_THROWS_AS(expr, ...)`
- `#define DOCTEST_REQUIRE_THROWS_AS(expr, ...)`
- `#define DOCTEST_WARN_THROWS_WITH(expr, ...)`
- `#define DOCTEST_CHECK_THROWS_WITH(expr, ...)`
- `#define DOCTEST_REQUIRE_THROWS_WITH(expr, ...)`
- `#define DOCTEST_WARN_THROWS_WITH_AS(expr, with, ...)`
- `#define DOCTEST_CHECK_THROWS_WITH_AS(expr, with, ...)`
- `#define DOCTEST_REQUIRE_THROWS_WITH_AS(expr, with, ...)`
- `#define DOCTEST_WARN_NOTHROW(...)`
- `#define DOCTEST_CHECK_NOTHROW(...)`
- `#define DOCTEST_REQUIRE_NOTHROW(...)`
- `#define DOCTEST_WARN_THROWS_MESSAGE(expr, ...)`
- `#define DOCTEST_CHECK_THROWS_MESSAGE(expr, ...)`
- `#define DOCTEST_REQUIRE_THROWS_MESSAGE(expr, ...)`
- `#define DOCTEST_WARN_THROWS_AS_MESSAGE(expr, ex, ...)`
- `#define DOCTEST_CHECK_THROWS_AS_MESSAGE(expr, ex, ...)`
- `#define DOCTEST_REQUIRE_THROWS_AS_MESSAGE(expr, ex, ...)`
- `#define DOCTEST_WARN_THROWS_WITH_MESSAGE(expr, with, ...)`
- `#define DOCTEST_CHECK_THROWS_WITH_MESSAGE(expr, with, ...)`
- `#define DOCTEST_REQUIRE_THROWS_WITH_MESSAGE(expr, with, ...)`
- `#define DOCTEST_WARN_THROWS_WITH_AS_MESSAGE(expr, with, ex, ...)`
- `#define DOCTEST_CHECK_THROWS_WITH_AS_MESSAGE(expr, with, ex, ...)`
- `#define DOCTEST_REQUIRE_THROWS_WITH_AS_MESSAGE(expr, with, ex, ...)`
- `#define DOCTEST_WARN_NOTHROW_MESSAGE(expr, ...)`
- `#define DOCTEST_CHECK_NOTHROW_MESSAGE(expr, ...)`
- `#define DOCTEST_REQUIRE_NOTHROW_MESSAGE(expr, ...)`
- `#define DOCTEST_FAST_WARN_EQ DOCTEST_WARN_EQ`
- `#define DOCTEST_FAST_CHECK_EQ DOCTEST_CHECK_EQ`
- `#define DOCTEST_FAST_REQUIRE_EQ DOCTEST_REQUIRE_EQ`
- `#define DOCTEST_FAST_WARN_NE DOCTEST_WARN_NE`
- `#define DOCTEST_FAST_CHECK_NE DOCTEST_CHECK_NE`
- `#define DOCTEST_FAST_REQUIRE_NE DOCTEST_REQUIRE_NE`
- `#define DOCTEST_FAST_WARN_GT DOCTEST_WARN_GT`
- `#define DOCTEST_FAST_CHECK_GT DOCTEST_CHECK_GT`
- `#define DOCTEST_FAST_REQUIRE_GT DOCTEST_REQUIRE_GT`
- `#define DOCTEST_FAST_WARN_LT DOCTEST_WARN_LT`
- `#define DOCTEST_FAST_CHECK_LT DOCTEST_CHECK_LT`
- `#define DOCTEST_FAST_REQUIRE_LT DOCTEST_REQUIRE_LT`
- `#define DOCTEST_FAST_WARN_GE DOCTEST_WARN_GE`
- `#define DOCTEST_FAST_CHECK_GE DOCTEST_CHECK_GE`
- `#define DOCTEST_FAST_REQUIRE_GE DOCTEST_REQUIRE_GE`
- `#define DOCTEST_FAST_WARN_LE DOCTEST_WARN_LE`
- `#define DOCTEST_FAST_CHECK_LE DOCTEST_CHECK_LE`
- `#define DOCTEST_FAST_REQUIRE_LE DOCTEST_REQUIRE_LE`

- `#define DOCTEST_FAST_WARN_UNARY DOCTEST_WARN_UNARY`
- `#define DOCTEST_FAST_CHECK_UNARY DOCTEST_CHECK_UNARY`
- `#define DOCTEST_FAST_REQUIRE_UNARY DOCTEST_REQUIRE_UNARY`
- `#define DOCTEST_FAST_WARN_UNARY_FALSE DOCTEST_WARN_UNARY_FALSE`
- `#define DOCTEST_FAST_CHECK_UNARY_FALSE DOCTEST_CHECK_UNARY_FALSE`
- `#define DOCTEST_FAST_REQUIRE_UNARY_FALSE DOCTEST_REQUIRE_UNARY_FALSE`
- `#define DOCTEST_TEST_CASE_TEMPLATE_INSTANTIATE(id, ...)`
- `#define DOCTEST_SCENARIO(name)`
- `#define DOCTEST_SCENARIO_CLASS(name)`
- `#define DOCTEST_SCENARIO_METHOD(x, name)`
- `#define DOCTEST_SCENARIO_TEMPLATE(name, T, ...)`
- `#define DOCTEST_SCENARIO_TEMPLATE_DEFINE(name, T, id)`
- `#define DOCTEST_GIVEN(name)`
- `#define DOCTEST_AND_GIVEN(name)`
- `#define DOCTEST_WHEN(name)`
- `#define DOCTEST_AND_WHEN(name)`
- `#define DOCTEST_THEN(name)`
- `#define DOCTEST_AND_THEN(name)`
- `#define TEST_CASE(name)`
- `#define TEST_CASE_CLASS(name)`
- `#define TEST_CASE_FIXTURE(x, name)`
- `#define TYPE_TO_STRING_AS(str, ...)`
- `#define TYPE_TO_STRING(...)`
- `#define TEST_CASE_TEMPLATE(name, T, ...)`
- `#define TEST_CASE_TEMPLATE_DEFINE(name, T, id)`
- `#define TEST_CASE_TEMPLATE_INVOKE(id, ...)`
- `#define TEST_CASE_TEMPLATE_APPLY(id, ...)`
- `#define SUBCASE(name)`
- `#define GENERATE(...)`
- `#define TEST_SUITE(decorators)`
- `#define TEST_SUITE_BEGIN(name)`
- `#define TEST_SUITE_END DOCTEST_TEST_SUITE_END`
- `#define REGISTER_EXCEPTION_TRANSLATOR(signature)`
- `#define REGISTER_REPORTER(name, priority, reporter)`
- `#define REGISTER_LISTENER(name, priority, reporter)`
- `#define INFO(...)`
- `#define CAPTURE(x)`
- `#define ADD_MESSAGE_AT(file, line, ...)`
- `#define ADD_FAIL_CHECK_AT(file, line, ...)`
- `#define ADD_FAIL_AT(file, line, ...)`
- `#define MESSAGE(...)`
- `#define FAIL_CHECK(...)`
- `#define FAIL(...)`
- `#define TO_LVALUE(...)`
- `#define WARN(...)`
- `#define WARN_FALSE(...)`
- `#define WARN_THROWS(...)`
- `#define WARN_THROWS_AS(expr, ...)`
- `#define WARN_THROWS_WITH(expr, ...)`
- `#define WARN_THROWS_WITH_AS(expr, with, ...)`
- `#define WARN_NOTHROW(...)`
- `#define CHECK(...)`
- `#define CHECK_FALSE(...)`
- `#define CHECK_THROWS(...)`
- `#define CHECK_THROWS_AS(expr, ...)`

- `#define CHECK_THROWS_WITH(expr, ...)`
- `#define CHECK_THROWS_WITH_AS(expr, with, ...)`
- `#define CHECK_NOTHROW(...)`
- `#define REQUIRE(...)`
- `#define REQUIRE_FALSE(...)`
- `#define REQUIRE_THROWS(...)`
- `#define REQUIRE_THROWS_AS(expr, ...)`
- `#define REQUIRE_THROWS_WITH(expr, ...)`
- `#define REQUIRE_THROWS_WITH_AS(expr, with, ...)`
- `#define REQUIRE_NOTHROW(...)`
- `#define WARN_MESSAGE(cond, ...)`
- `#define WARN_FALSE_MESSAGE(cond, ...)`
- `#define WARN_THROWS_MESSAGE(expr, ...)`
- `#define WARN_THROWS_AS_MESSAGE(expr, ex, ...)`
- `#define WARN_THROWS_WITH_MESSAGE(expr, with, ...)`
- `#define WARN_THROWS_WITH_AS_MESSAGE(expr, with, ex, ...)`
- `#define WARN_NOTHROW_MESSAGE(expr, ...)`
- `#define CHECK_MESSAGE(cond, ...)`
- `#define CHECK_FALSE_MESSAGE(cond, ...)`
- `#define CHECK_THROWS_MESSAGE(expr, ...)`
- `#define CHECK_THROWS_AS_MESSAGE(expr, ex, ...)`
- `#define CHECK_THROWS_WITH_MESSAGE(expr, with, ...)`
- `#define CHECK_THROWS_WITH_AS_MESSAGE(expr, with, ex, ...)`
- `#define CHECK_NOTHROW_MESSAGE(expr, ...)`
- `#define REQUIRE_MESSAGE(cond, ...)`
- `#define REQUIRE_FALSE_MESSAGE(cond, ...)`
- `#define REQUIRE_THROWS_MESSAGE(expr, ...)`
- `#define REQUIRE_THROWS_AS_MESSAGE(expr, ex, ...)`
- `#define REQUIRE_THROWS_WITH_MESSAGE(expr, with, ...)`
- `#define REQUIRE_THROWS_WITH_AS_MESSAGE(expr, with, ex, ...)`
- `#define REQUIRE_NOTHROW_MESSAGE(expr, ...)`
- `#define SCENARIO(name)`
- `#define SCENARIO_METHOD(x, name)`
- `#define SCENARIO_CLASS(name)`
- `#define SCENARIO_TEMPLATE(name, T, ...)`
- `#define SCENARIO_TEMPLATE_DEFINE(name, T, id)`
- `#define GIVEN(name)`
- `#define AND_GIVEN(name)`
- `#define WHEN(name)`
- `#define AND_WHEN(name)`
- `#define THEN(name)`
- `#define AND_THEN(name)`
- `#define WARN_EQ(...)`
- `#define CHECK_EQ(...)`
- `#define REQUIRE_EQ(...)`
- `#define WARN_NE(...)`
- `#define CHECK_NE(...)`
- `#define REQUIRE_NE(...)`
- `#define WARN_GT(...)`
- `#define CHECK_GT(...)`
- `#define REQUIRE_GT(...)`
- `#define WARN_LT(...)`
- `#define CHECK_LT(...)`
- `#define REQUIRE_LT(...)`
- `#define WARN_GE(...)`

- `#define CHECK_GE(...)`
- `#define REQUIRE_GE(...)`
- `#define WARN_LE(...)`
- `#define CHECK_LE(...)`
- `#define REQUIRE_LE(...)`
- `#define WARN_UNARY(...)`
- `#define CHECK_UNARY(...)`
- `#define REQUIRE_UNARY(...)`
- `#define WARN_UNARY_FALSE(...)`
- `#define CHECK_UNARY_FALSE(...)`
- `#define REQUIRE_UNARY_FALSE(...)`
- `#define FAST_WARN_EQ(...)`
- `#define FAST_CHECK_EQ(...)`
- `#define FAST_REQUIRE_EQ(...)`
- `#define FAST_WARN_NE(...)`
- `#define FAST_CHECK_NE(...)`
- `#define FAST_REQUIRE_NE(...)`
- `#define FAST_WARN_GT(...)`
- `#define FAST_CHECK_GT(...)`
- `#define FAST_REQUIRE_GT(...)`
- `#define FAST_WARN_LT(...)`
- `#define FAST_CHECK_LT(...)`
- `#define FAST_REQUIRE_LT(...)`
- `#define FAST_WARN_GE(...)`
- `#define FAST_CHECK_GE(...)`
- `#define FAST_REQUIRE_GE(...)`
- `#define FAST_WARN_LE(...)`
- `#define FAST_CHECK_LE(...)`
- `#define FAST_REQUIRE_LE(...)`
- `#define FAST_WARN_UNARY(...)`
- `#define FAST_CHECK_UNARY(...)`
- `#define FAST_REQUIRE_UNARY(...)`
- `#define FAST_WARN_UNARY_FALSE(...)`
- `#define FAST_CHECK_UNARY_FALSE(...)`
- `#define FAST_REQUIRE_UNARY_FALSE(...)`
- `#define TEST_CASE_TEMPLATE_INSTANTIATE(id, ...)`

Typedefs

- `typedef decltype(nullptr) std::nullptr_t`
- `typedef decltype(sizeof(void*)) std::size_t`
- `typedef basic_ostream< char, char_traits< char > > std::ostream`
- `typedef basic_istream< char, char_traits< char > > std::istream`
- `template<typename... Unused>`
`using doctest::detail::types::void_t = void`
- `using doctest::detail::funcType = void (*)(void)`
- `using doctest::detail::assert_handler = void (*)(const AssertData &)`
- `using doctest::detail::reporterCreatorFunc = IReporter (*)(const ContextOptions &)`
- `typedef decltype(sizeof(void*)) doctest::size_t`

Enumerations

- enum `doctest::assertType::Enum` {
`doctest::assertType::is_warn` = 1, `doctest::assertType::is_check` = 2 * `is_warn`, `doctest::assertType::is_require`
= 2 * `is_check`, `doctest::assertType::is_normal` = 2 * `is_require`,
`doctest::assertType::is_throws` = 2 * `is_normal`, `doctest::assertType::is_throws_as` = 2 * `is_throws`,
`doctest::assertType::is_throws_with` = 2 * `is_throws_as`, `doctest::assertType::is_nothrow` = 2 * `is_throws_↵`
with ,
`doctest::assertType::is_false` = 2 * `is_nothrow`, `doctest::assertType::is_unary` = 2 * `is_false`, `doctest::assertType::is_eq`
= 2 * `is_unary`, `doctest::assertType::is_ne` = 2 * `is_eq`,
`doctest::assertType::is_lt` = 2 * `is_ne`, `doctest::assertType::is_gt` = 2 * `is_lt`, `doctest::assertType::is_ge` = 2 *
`is_gt`, `doctest::assertType::is_le` = 2 * `is_ge`,
`doctest::assertType::DT_WARN` = `is_normal` | `is_warn`, `doctest::assertType::DT_CHECK` = `is_normal` | `is_↵`
`check`, `doctest::assertType::DT_REQUIRE` = `is_normal` | `is_require`, `doctest::assertType::DT_WARN_FALSE`
= `is_normal` | `is_false` | `is_warn`,
`doctest::assertType::DT_CHECK_FALSE` = `is_normal` | `is_false` | `is_check`, `doctest::assertType::DT_REQUIRE_FALSE`
= `is_normal` | `is_false` | `is_require`, `doctest::assertType::DT_WARN_THROWS` = `is_throws` | `is_warn`,
`doctest::assertType::DT_CHECK_THROWS` = `is_throws` | `is_check`,
`doctest::assertType::DT_REQUIRE_THROWS` = `is_throws` | `is_require`, `doctest::assertType::DT_WARN_THROWS_AS`
= `is_throws_as` | `is_warn`, `doctest::assertType::DT_CHECK_THROWS_AS` = `is_throws_as` | `is_check`,
`doctest::assertType::DT_REQUIRE_THROWS_AS` = `is_throws_as` | `is_require`,
`doctest::assertType::DT_WARN_THROWS_WITH` = `is_throws_with` | `is_warn`, `doctest::assertType::DT_CHECK_THROWS_W↵`
= `is_throws_with` | `is_check`, `doctest::assertType::DT_REQUIRE_THROWS_WITH` = `is_throws_with` | `is_↵`
`require`, `doctest::assertType::DT_WARN_THROWS_WITH_AS` = `is_throws_with` | `is_throws_as` | `is_warn`,
`doctest::assertType::DT_CHECK_THROWS_WITH_AS` = `is_throws_with` | `is_throws_as` | `is_check`,
`doctest::assertType::DT_REQUIRE_THROWS_WITH_AS` = `is_throws_with` | `is_throws_as` | `is_require`,
`doctest::assertType::DT_WARN_NOTHROW` = `is_nothrow` | `is_warn`, `doctest::assertType::DT_CHECK_NOTHROW`
= `is_nothrow` | `is_check`,
`doctest::assertType::DT_REQUIRE_NOTHROW` = `is_nothrow` | `is_require`, `doctest::assertType::DT_WARN_EQ`
= `is_normal` | `is_eq` | `is_warn`, `doctest::assertType::DT_CHECK_EQ` = `is_normal` | `is_eq` | `is_check`,
`doctest::assertType::DT_REQUIRE_EQ` = `is_normal` | `is_eq` | `is_require`,
`doctest::assertType::DT_WARN_NE` = `is_normal` | `is_ne` | `is_warn`, `doctest::assertType::DT_CHECK_NE`
= `is_normal` | `is_ne` | `is_check`, `doctest::assertType::DT_REQUIRE_NE` = `is_normal` | `is_ne` | `is_require`,
`doctest::assertType::DT_WARN_GT` = `is_normal` | `is_gt` | `is_warn`,
`doctest::assertType::DT_CHECK_GT` = `is_normal` | `is_gt` | `is_check`, `doctest::assertType::DT_REQUIRE_GT`
= `is_normal` | `is_gt` | `is_require`, `doctest::assertType::DT_WARN_LT` = `is_normal` | `is_lt` | `is_warn`,
`doctest::assertType::DT_CHECK_LT` = `is_normal` | `is_lt` | `is_check`,
`doctest::assertType::DT_REQUIRE_LT` = `is_normal` | `is_lt` | `is_require`, `doctest::assertType::DT_WARN_GE`
= `is_normal` | `is_ge` | `is_warn`, `doctest::assertType::DT_CHECK_GE` = `is_normal` | `is_ge` | `is_check`,
`doctest::assertType::DT_REQUIRE_GE` = `is_normal` | `is_ge` | `is_require`,
`doctest::assertType::DT_WARN_LE` = `is_normal` | `is_le` | `is_warn`, `doctest::assertType::DT_CHECK_LE`
= `is_normal` | `is_le` | `is_check`, `doctest::assertType::DT_REQUIRE_LE` = `is_normal` | `is_le` | `is_require`,
`doctest::assertType::DT_WARN_UNARY` = `is_normal` | `is_unary` | `is_warn`,
`doctest::assertType::DT_CHECK_UNARY` = `is_normal` | `is_unary` | `is_check`, `doctest::assertType::DT_REQUIRE_UNARY`
= `is_normal` | `is_unary` | `is_require`, `doctest::assertType::DT_WARN_UNARY_FALSE` = `is_normal` | `is_false`
| `is_unary` | `is_warn`, `doctest::assertType::DT_CHECK_UNARY_FALSE` = `is_normal` | `is_false` | `is_unary` |
`is_check`,
`doctest::assertType::DT_REQUIRE_UNARY_FALSE` = `is_normal` | `is_false` | `is_unary` | `is_require` }
- enum `doctest::detail::binaryAssertComparison::Enum` {
`doctest::detail::binaryAssertComparison::eq` = 0, `doctest::detail::binaryAssertComparison::ne`, `doctest::detail::binaryAssertCom↵`
, `doctest::detail::binaryAssertComparison::lt`,
`doctest::detail::binaryAssertComparison::ge`, `doctest::detail::binaryAssertComparison::le` }
- enum `doctest::Color::Enum` {
`doctest::Color::None` = 0, `doctest::Color::White`, `doctest::Color::Red`, `doctest::Color::Green`,
`doctest::Color::Blue`, `doctest::Color::Cyan`, `doctest::Color::Yellow`, `doctest::Color::Grey`,
`doctest::Color::Bright` = 0x10, `doctest::Color::BrightRed` = `Bright` | `Red`, `doctest::Color::BrightGreen` = `Bright`
| `Green`, `doctest::Color::LightGrey` = `Bright` | `Grey`,
`doctest::Color::BrightWhite` = `Bright` | `White` }

- enum `doctest::TestCaseFailureReason::Enum` {
`doctest::TestCaseFailureReason::None` = 0 , `doctest::TestCaseFailureReason::AssertFailure` = 1 ,
`doctest::TestCaseFailureReason::Exception` = 2 , `doctest::TestCaseFailureReason::Crash` = 4 ,
`doctest::TestCaseFailureReason::TooManyFailedAsserts` = 8 , `doctest::TestCaseFailureReason::Timeout` =
16 , `doctest::TestCaseFailureReason::ShouldHaveFailedButDidnt` = 32 , `doctest::TestCaseFailureReason::ShouldHaveFailedAn`
= 64 ,
`doctest::TestCaseFailureReason::DidntFailExactlyNumTimes` = 128 , `doctest::TestCaseFailureReason::FailedExactlyNumTimes`
= 256 , `doctest::TestCaseFailureReason::CouldHaveFailedAndDid` = 512 }

Functions

- `DOctest_INTERFACE` bool `doctest::detail::isDebuggerActive` ()
- template<class traits>
`basic_ostream`< char, traits > & `std::operator<<` (`basic_ostream`< char, traits > &, const char *)
- template<typename T>
T && `doctest::detail::declval` ()
- template<class T>
`DOctest_INTERFACE` CONSTEXPR_FUNC T && `doctest::detail::forward` (typename `types::remove_reference`< T
>::type &t) `DOctest_INTERFACE` NOEXCEPT
- template<class T>
`DOctest_INTERFACE` CONSTEXPR_FUNC T && `doctest::detail::forward` (typename `types::remove_reference`< T
>::type &&t) `DOctest_INTERFACE` NOEXCEPT
- `DOctest_INTERFACE` String `doctest::operator+` (const String &lhs, const String &rhs)
- `DOctest_INTERFACE` bool `doctest::operator==` (const String &lhs, const String &rhs)
- `DOctest_INTERFACE` bool `doctest::operator!=` (const String &lhs, const String &rhs)
- `DOctest_INTERFACE` bool `doctest::operator<` (const String &lhs, const String &rhs)
- `DOctest_INTERFACE` bool `doctest::operator>` (const String &lhs, const String &rhs)
- `DOctest_INTERFACE` bool `doctest::operator<=` (const String &lhs, const String &rhs)
- `DOctest_INTERFACE` bool `doctest::operator>=` (const String &lhs, const String &rhs)
- `DOctest_INTERFACE` std::ostream * `doctest::detail::tlssPush` ()
- `DOctest_INTERFACE` String `doctest::detail::tlssPop` ()
- template<typename T>
void `doctest::detail::fillloss` (std::ostream *stream, const T &in)
- template<typename T, size_t N>
void `doctest::detail::fillloss` (std::ostream *stream, const T(&in)[N])
- template<typename T>
String `doctest::detail::toString` (const T &in)
- template<typename T>
String `doctest::toString` ()
- template<typename T, typename `detail::types::enable_if`<!`detail::should_stringify_as_underlying_type`< T >::value, bool >::type =
true>
String `doctest::toString` (const `DOctest_REF_WRAP`(T) value)
- String && `doctest::toString` (String &&in)
- `DOctest_INTERFACE` String `doctest::toString` (const String &in)
- `DOctest_INTERFACE` String `doctest::toString` (std::nullptr_t)
- `DOctest_INTERFACE` String `doctest::toString` (bool in)
- `DOctest_INTERFACE` String `doctest::toString` (float in)
- `DOctest_INTERFACE` String `doctest::toString` (double in)
- `DOctest_INTERFACE` String `doctest::toString` (double long in)
- `DOctest_INTERFACE` String `doctest::toString` (char in)
- `DOctest_INTERFACE` String `doctest::toString` (char signed in)
- `DOctest_INTERFACE` String `doctest::toString` (char unsigned in)
- `DOctest_INTERFACE` String `doctest::toString` (short in)
- `DOctest_INTERFACE` String `doctest::toString` (short unsigned in)
- `DOctest_INTERFACE` String `doctest::toString` (signed in)

- [DOCTEST_INTERFACE String doctest::toString](#) (unsigned in)
- [DOCTEST_INTERFACE String doctest::toString](#) (long in)
- [DOCTEST_INTERFACE String doctest::toString](#) (long unsigned in)
- [DOCTEST_INTERFACE String doctest::toString](#) (long long in)
- [DOCTEST_INTERFACE String doctest::toString](#) (long long unsigned in)
- [template<typename L, typename R>](#)
[String doctest::detail::stringifyBinaryExpr](#) (const [DOCTEST_REF_WRAP](#)(L) lhs, const char *op, const [DOCTEST_REF_WRAP](#)(R) rhs)
- [DOCTEST_INTERFACE String doctest::toString](#) (const [Contains](#) &in)
- [DOCTEST_INTERFACE bool doctest::operator==](#) (const [String](#) &lhs, const [Contains](#) &rhs)
- [DOCTEST_INTERFACE bool doctest::operator==](#) (const [Contains](#) &lhs, const [String](#) &rhs)
- [DOCTEST_INTERFACE bool doctest::operator!=](#) (const [String](#) &lhs, const [Contains](#) &rhs)
- [DOCTEST_INTERFACE bool doctest::operator!=](#) (const [Contains](#) &lhs, const [String](#) &rhs)
- [DOCTEST_INTERFACE String doctest::toString](#) (const [Approx](#) &in)
- [DOCTEST_INTERFACE String doctest::toString](#) ([IsNaN](#)< float > in)
- [DOCTEST_INTERFACE String doctest::toString](#) ([IsNaN](#)< double > in)
- [DOCTEST_INTERFACE String doctest::toString](#) ([IsNaN](#)< double long > in)
- [DOCTEST_INTERFACE const ContextOptions * doctest::getContextOptions](#) ()
- [DOCTEST_INTERFACE const char * doctest::assertString](#) ([assertType::Enum](#) at)
- [DOCTEST_INTERFACE const char * doctest::failureString](#) ([assertType::Enum](#) at)
- [DOCTEST_INTERFACE std::ostream & doctest::Color::operator<<](#) (std::ostream &s, [Color::Enum](#) code)
- [DOCTEST_INTERFACE int doctest::detail::setTestSuite](#) (const [TestSuite](#) &ts) noexcept
- [DOCTEST_INTERFACE doctest::detail::TestSuite & doctest_detail_test_suite_ns::getCurrentTestSuite](#) ()
noexcept
- [DOCTEST_INTERFACE int doctest::detail::regTest](#) (const [TestCase](#) &tc) noexcept
- [doctest::DOCTEST_DEFINE_DECORATOR](#) (test_suite, const char *, "")
- [doctest::DOCTEST_DEFINE_DECORATOR](#) (description, const char *, "")
- [doctest::DOCTEST_DEFINE_DECORATOR](#) (skip, bool, true)
- [doctest::DOCTEST_DEFINE_DECORATOR](#) (no_breaks, bool, true)
- [doctest::DOCTEST_DEFINE_DECORATOR](#) (no_output, bool, true)
- [doctest::DOCTEST_DEFINE_DECORATOR](#) (timeout, double, 0)
- [doctest::DOCTEST_DEFINE_DECORATOR](#) (may_fail, bool, true)
- [doctest::DOCTEST_DEFINE_DECORATOR](#) (should_fail, bool, true)
- [doctest::DOCTEST_DEFINE_DECORATOR](#) (expected_failures, int, 0)
- [DOCTEST_INTERFACE void doctest::detail::registerExceptionTranslatorImpl](#) (const [IExceptionTranslator](#) *et) noexcept
- [template<typename T>](#)
[int doctest::registerExceptionTranslator](#) ([String](#)(*translateFunction)(T)) noexcept
- [template<typename L>](#)
[ContextScope< L > doctest::detail::MakeContextScope](#) (const L &lambda)
- [DOCTEST_INTERFACE const char * doctest::skipPathFromFilename](#) (const char *file)
- [DOCTEST_INTERFACE bool doctest::detail::checkIfShouldThrow](#) ([assertType::Enum](#) at)
- [DOCTEST_INTERFACE void doctest::detail::throwException](#) ()
- [DOCTEST_INTERFACE void doctest::detail::failed_out_of_a_testing_context](#) (const [AssertData](#) &ad)
- [DOCTEST_INTERFACE bool doctest::detail::decomp_assert](#) ([assertType::Enum](#) at, const char *file, int line, const char *expr, const [Result](#) &result)
- [template<int comparison, typename L, typename R>](#)
[DOCTEST_NOINLINE bool doctest::detail::binary_assert](#) ([assertType::Enum](#) at, const char *file, int line, const char *expr, const [DOCTEST_REF_WRAP](#)(L) lhs, const [DOCTEST_REF_WRAP](#)(R) rhs)
- [template<typename L>](#)
[DOCTEST_NOINLINE bool doctest::detail::unary_assert](#) ([assertType::Enum](#) at, const char *file, int line, const char *expr, const [DOCTEST_REF_WRAP](#)(L) val)
- [DOCTEST_INTERFACE void doctest::detail::registerReporterImpl](#) (const char *name, int prio, [reporterCreatorFunc](#) c, bool isReporter) noexcept
- [template<typename Reporter>](#)
[IReporter * doctest::detail::reporterCreator](#) (const [ContextOptions](#) &o)

- `template<typename Reporter>`
`int doctest::registerReporter (const char *name, int priority, bool isReporter) noexcept`
- `DOctest_INTERFACE size_t doctest::detail::acquireGeneratorDecisionIndex (size_t count)`
- `template<typename T, typename... Rest>`
`T doctest::detail::acquireGeneratorValue (T first, Rest... rest)`
- `template<typename T>`
`int doctest::detail::instantiationHelper (const T &) noexcept`

Variables

- `template struct DOctest_INTERFACE_DECL doctest::IsNaN< float >`
- `template struct DOctest_INTERFACE_DECL doctest::IsNaN< double >`
- `template struct DOctest_INTERFACE_DECL doctest::IsNaN< long double >`
- `struct DOctest_INTERFACE doctest::detail::TestCase`
- `struct DOctest_INTERFACE doctest::TestCaseData`
- `DOctest_INTERFACE bool doctest::is_running_in_test`

12.64.1 Macro Definition Documentation

12.64.1.1 ADD_FAIL_AT

```
#define ADD_FAIL_AT(  
    file,  
    line,  
    ...)
```

Value:

`DOctest_ADD_FAIL_AT(file, line, __VA_ARGS__)`

Definition at line 3768 of file `doctest.h`.

12.64.1.2 ADD_FAIL_CHECK_AT

```
#define ADD_FAIL_CHECK_AT(  
    file,  
    line,  
    ...)
```

Value:

`DOctest_ADD_FAIL_CHECK_AT(file, line, __VA_ARGS__)`

Definition at line 3767 of file `doctest.h`.

12.64.1.3 ADD_MESSAGE_AT

```
#define ADD_MESSAGE_AT(  
    file,  
    line,  
    ...)
```

Value:

`DOctest_ADD_MESSAGE_AT(file, line, __VA_ARGS__)`

Definition at line 3766 of file `doctest.h`.

12.64.1.4 AND_GIVEN

```
#define AND_GIVEN(  
    name)
```

Value:

[DOCTEST_AND_GIVEN](#)(*name*)

Definition at line [3826](#) of file [doctest.h](#).

12.64.1.5 AND_THEN

```
#define AND_THEN(  
    name)
```

Value:

[DOCTEST_AND_THEN](#)(*name*)

Definition at line [3830](#) of file [doctest.h](#).

12.64.1.6 AND_WHEN

```
#define AND_WHEN(  
    name)
```

Value:

[DOCTEST_AND_WHEN](#)(*name*)

Definition at line [3828](#) of file [doctest.h](#).

12.64.1.7 CAPTURE

```
#define CAPTURE(  
    x)
```

Value:

[DOCTEST_CAPTURE](#)(*x*)

Definition at line [3765](#) of file [doctest.h](#).

12.64.1.8 CHECK

```
#define CHECK(  
    ...)
```

Value:

[DOCTEST_CHECK](#)(__VA_ARGS__)

Definition at line [3781](#) of file [doctest.h](#).

12.64.1.9 CHECK_EQ

```
#define CHECK_EQ(  
    ...)
```

Value:

[DOCTEST_CHECK_EQ](#)(__VA_ARGS__)

Definition at line [3833](#) of file [doctest.h](#).

12.64.1.10 CHECK_FALSE

```
#define CHECK_FALSE(  
    ...)
```

Value:

[DOCTEST_CHECK_FALSE](#)(__VA_ARGS__)

Definition at line [3782](#) of file [doctest.h](#).

12.64.1.11 CHECK_FALSE_MESSAGE

```
#define CHECK_FALSE_MESSAGE(  
    cond,  
    ...)
```

Value:

[DOCTEST_CHECK_FALSE_MESSAGE](#)(cond, __VA_ARGS__)

Definition at line [3805](#) of file [doctest.h](#).

12.64.1.12 CHECK_GE

```
#define CHECK_GE(  
    ...)
```

Value:

[DOCTEST_CHECK_GE](#)(__VA_ARGS__)

Definition at line [3845](#) of file [doctest.h](#).

12.64.1.13 CHECK_GT

```
#define CHECK_GT(  
    ...)
```

Value:

[DOCTEST_CHECK_GT](#)(__VA_ARGS__)

Definition at line [3839](#) of file [doctest.h](#).

12.64.1.14 CHECK_LE

```
#define CHECK_LE(  
    ...)
```

Value:

[DOCTEST_CHECK_LE](#)(__VA_ARGS__)

Definition at line [3848](#) of file [doctest.h](#).

12.64.1.15 CHECK_LT

```
#define CHECK_LT(  
    ...)
```

Value:

[DOCTEST_CHECK_LT](#)(__VA_ARGS__)

Definition at line [3842](#) of file [doctest.h](#).

12.64.1.16 CHECK_MESSAGE

```
#define CHECK_MESSAGE(  
    cond,  
    ...)
```

Value:

[DOCTEST_CHECK_MESSAGE](#)(cond, __VA_ARGS__)

Definition at line [3804](#) of file [doctest.h](#).

12.64.1.17 CHECK_NE

```
#define CHECK_NE(  
    ...)
```

Value:

[DOCTEST_CHECK_NE](#)(__VA_ARGS__)

Definition at line [3836](#) of file [doctest.h](#).

12.64.1.18 CHECK_NOTHROW

```
#define CHECK_NOTHROW(  
    ...)
```

Value:

[DOCTEST_CHECK_NOTHROW](#)(__VA_ARGS__)

Definition at line [3787](#) of file [doctest.h](#).

12.64.1.19 CHECK_NOTHROW_MESSAGE

```
#define CHECK_NOTHROW_MESSAGE(  
    expr,  
    ...)
```

Value:

[DOCTEST_CHECK_NOTHROW_MESSAGE](#)(*expr*, __VA_ARGS__)

Definition at line 3810 of file [doctest.h](#).

12.64.1.20 CHECK_THROWS

```
#define CHECK_THROWS(  
    ...)
```

Value:

[DOCTEST_CHECK_THROWS](#)(__VA_ARGS__)

Definition at line 3783 of file [doctest.h](#).

12.64.1.21 CHECK_THROWS_AS

```
#define CHECK_THROWS_AS(  
    expr,  
    ...)
```

Value:

[DOCTEST_CHECK_THROWS_AS](#)(*expr*, __VA_ARGS__)

Definition at line 3784 of file [doctest.h](#).

12.64.1.22 CHECK_THROWS_AS_MESSAGE

```
#define CHECK_THROWS_AS_MESSAGE(  
    expr,  
    ex,  
    ...)
```

Value:

[DOCTEST_CHECK_THROWS_AS_MESSAGE](#)(*expr*, *ex*, __VA_ARGS__)

Definition at line 3807 of file [doctest.h](#).

12.64.1.23 CHECK_THROWS_MESSAGE

```
#define CHECK_THROWS_MESSAGE(  
    expr,  
    ...)
```

Value:

[DOCTEST_CHECK_THROWS_MESSAGE](#)(*expr*, __VA_ARGS__)

Definition at line 3806 of file [doctest.h](#).

12.64.1.24 CHECK_THROWS_WITH

```
#define CHECK_THROWS_WITH(  
    expr,  
    ...)
```

Value:

`DOCTEST_CHECK_THROWS_WITH(expr, __VA_ARGS__)`

Definition at line [3785](#) of file [doctest.h](#).

12.64.1.25 CHECK_THROWS_WITH_AS

```
#define CHECK_THROWS_WITH_AS(  
    expr,  
    with,  
    ...)
```

Value:

`DOCTEST_CHECK_THROWS_WITH_AS(expr, with, __VA_ARGS__)`

Definition at line [3786](#) of file [doctest.h](#).

12.64.1.26 CHECK_THROWS_WITH_AS_MESSAGE

```
#define CHECK_THROWS_WITH_AS_MESSAGE(  
    expr,  
    with,  
    ex,  
    ...)
```

Value:

`DOCTEST_CHECK_THROWS_WITH_AS_MESSAGE(expr, with, ex, __VA_ARGS__)`

Definition at line [3809](#) of file [doctest.h](#).

12.64.1.27 CHECK_THROWS_WITH_MESSAGE

```
#define CHECK_THROWS_WITH_MESSAGE(  
    expr,  
    with,  
    ...)
```

Value:

`DOCTEST_CHECK_THROWS_WITH_MESSAGE(expr, with, __VA_ARGS__)`

Definition at line [3808](#) of file [doctest.h](#).

12.64.1.28 CHECK_UNARY

```
#define CHECK_UNARY(  
    ...)
```

Value:

`DOCTEST_CHECK_UNARY` (`__VA_ARGS__`)

Definition at line 3851 of file [doctest.h](#).

12.64.1.29 CHECK_UNARY_FALSE

```
#define CHECK_UNARY_FALSE(  
    ...)
```

Value:

`DOCTEST_CHECK_UNARY_FALSE` (`__VA_ARGS__`)

Definition at line 3854 of file [doctest.h](#).

12.64.1.30 DOCTEST_ADD_AT_IMPL

```
#define DOCTEST_ADD_AT_IMPL(  
    type,  
    file,  
    line,  
    mb,  
    ...)
```

Value:

```
DOCTEST_FUNC_SCOPE_BEGIN {  
    \   
    doctest::detail::MessageBuilder mb(file, line, doctest::assertType::type);  
    \   
    mb *__VA_ARGS__;  
    \   
    if (mb.log())  
    \   
        DOCTEST_BREAK_INTO_DEBUGGER();  
    \   
    mb.react();  
    \   
}  
\   
DOCTEST_FUNC_SCOPE_END
```

Definition at line 3147 of file [doctest.h](#).

12.64.1.31 DOCTEST_ADD_FAIL_AT

```
#define DOCTEST_ADD_FAIL_AT(  
    file,  
    line,  
    ...)
```

Value:

`DOCTEST_ADD_AT_IMPL`(`is_require`, `file`, `line`, `DOCTEST_ANONYMOUS(DOCTEST_MESSAGE_)`, `__VA_ARGS__`)

Definition at line 3161 of file [doctest.h](#).

12.64.1.32 DOCTEST_ADD_FAIL_CHECK_AT

```
#define DOCTEST_ADD_FAIL_CHECK_AT(  
    file,  
    line,  
    ...)
```

Value:

`DOCTEST_ADD_AT_IMPL(is_check, file, line, DOCTEST_ANONYMOUS(DOCTEST_MESSAGE_), __VA_ARGS__)`

Definition at line 3159 of file [doctest.h](#).

12.64.1.33 DOCTEST_ADD_MESSAGE_AT

```
#define DOCTEST_ADD_MESSAGE_AT(  
    file,  
    line,  
    ...)
```

Value:

`DOCTEST_ADD_AT_IMPL(is_warn, file, line, DOCTEST_ANONYMOUS(DOCTEST_MESSAGE_), __VA_ARGS__)`

Definition at line 3157 of file [doctest.h](#).

12.64.1.34 DOCTEST_ALIGNMENT

```
#define DOCTEST_ALIGNMENT(  
    x)
```

Value:

`__attribute__((aligned(x)))`

Definition at line 452 of file [doctest.h](#).

12.64.1.35 DOCTEST_AND_GIVEN

```
#define DOCTEST_AND_GIVEN(  
    name)
```

Value:

`DOCTEST_SUBCASE(" And: " name)`

Definition at line 3737 of file [doctest.h](#).

12.64.1.36 DOCTEST_AND_THEN

```
#define DOCTEST_AND_THEN(  
    name)
```

Value:

`DOCTEST_SUBCASE(" And: " name)`

Definition at line 3741 of file [doctest.h](#).

12.64.1.37 DOCTEST_AND_WHEN

```
#define DOCTEST_AND_WHEN(  
    name)
```

Value:

```
DOCTEST_SUBCASE("    And: " name)
```

Definition at line 3739 of file [doctest.h](#).

12.64.1.38 DOCTEST_ANONYMOUS

```
#define DOCTEST_ANONYMOUS(  
    x)
```

Value:

```
DOCTEST_CAT(x, DOCTEST_COUNTER)
```

Definition at line 569 of file [doctest.h](#).

12.64.1.39 DOCTEST_ASSERT_IMPLEMENT_1

```
#define DOCTEST_ASSERT_IMPLEMENT_1(  
    assert_type,  
    ...)
```

Value:

```
DOCTEST_FUNC_SCOPE_BEGIN {  
    \ DOCTEST_ASSERT_IMPLEMENT_2(assert_type, __VA_ARGS__);  
    \  
}  
    \ DOCTEST_FUNC_SCOPE_END
```

Definition at line 3182 of file [doctest.h](#).

12.64.1.40 DOCTEST_ASSERT_IMPLEMENT_2

```
#define DOCTEST_ASSERT_IMPLEMENT_2(  
    assert_type,  
    ...)
```

Value:

```
DOCTEST_CLANG_SUPPRESS_WARNING_WITH_PUSH("-Woverloaded-shift-op-parentheses")  
    \  
/* NOLINTNEXTLINE(clang-analyzer-cplusplus.NewDeleteLeaks) */  
    \  
doctest::detail::ResultBuilder DOCTEST_RB(doctest::assertType::assert_type, __FILE__, __LINE__,  
    #__VA_ARGS__);    \  
DOCTEST_WRAP_IN_TRY(  
    \  
    DOCTEST_RB.setResult(doctest::detail::ExpressionDecomposer(doctest::assertType::assert_type) «  
        __VA_ARGS__)    \  
    ) /* NOLINTNEXTLINE(clang-analyzer-cplusplus.NewDeleteLeaks) */  
    \  
DOCTEST_ASSERT_LOG_REACT_RETURN(DOCTEST_RB)  
    \  
DOCTEST_CLANG_SUPPRESS_WARNING_POP
```

Definition at line 3172 of file [doctest.h](#).

12.64.1.41 DOCTEST_ASSERT_IN_TESTS

```
#define DOCTEST_ASSERT_IN_TESTS(  
    decomp)
```

Value:

```
ResultBuilder rb(at, file, line, expr);  
\  
rb.m_failed = failed;  
\  
if (rb.m_failed || getContextOptions()->success)  
\  
    rb.m_decomp = decomp;  
\  
if (rb.log())  
\  
    DOCTEST_BREAK_INTO_DEBUGGER();  
\  
if (rb.m_failed && checkIfShouldThrow(at))  
\  
    throwException();
```

Definition at line [2667](#) of file [doctest.h](#).

12.64.1.42 DOCTEST_ASSERT_LOG_REACT_RETURN

```
#define DOCTEST_ASSERT_LOG_REACT_RETURN(  
    b)
```

Value:

```
if (b.log())  
\  
    DOCTEST_BREAK_INTO_DEBUGGER();  
\  
b.react();  
\  
DOCTEST_FUNC_SCOPE_RET(!b.m_failed)
```

Definition at line [2910](#) of file [doctest.h](#).

12.64.1.43 DOCTEST_ASSERT_OUT_OF_TESTS

```
#define DOCTEST_ASSERT_OUT_OF_TESTS(  
    decomp)
```

Value:

```
do { /* NOLINT(cppcoreguidelines-avoid-do-while) */  
    \  
    if (!is_running_in_test) {  
        \  
        if (failed) {  
            \  
            ResultBuilder rb(at, file, line, expr);  
            \  
            rb.m_failed = failed;  
            \  
            rb.m_decomp = decomp;  
            \  
            failed_out_of_a_testing_context(rb);  
            \  
            if (isDebuggerActive() && !getContextOptions()->no_breaks)  
            \  
                DOCTEST_BREAK_INTO_DEBUGGER();  
            \  
            if (checkIfShouldThrow(at))  
            \  
                throwException();  
            \  
        }  
        \  
        return !failed;  
    }  
    \  
} while (false)
```

Definition at line [2650](#) of file [doctest.h](#).

12.64.1.44 DOCTEST_BINARY_ASSERT

```
#define DOCTEST_BINARY_ASSERT(
    assert_type,
    comp,
    ...)
```

Value:

```
DOCTEST_FUNC_SCOPE_BEGIN {
    \
    doctest::detail::ResultBuilder DOCTEST_RB(doctest::assertType::assert_type, __FILE__, __LINE__,
    #__VA_ARGS__); \
    DOCTEST_WRAP_IN_TRY(DOCTEST_RB.binary_assert<doctest::detail::binaryAssertComparison::comp>(__VA_ARGS__))
    \
    DOCTEST_ASSERT_LOG_REACT_RETURN(DOCTEST_RB);
    \
}
DOCTEST_FUNC_SCOPE_END
```

Definition at line 3188 of file [doctest.h](#).

12.64.1.45 DOCTEST_BINARY_RELATIONAL_OP

```
#define DOCTEST_BINARY_RELATIONAL_OP(
    n,
    op)
```

Value:

```
template <class L, class R> struct RelationalComparator<n, L, R> { bool operator()(const DOCTEST_REF_WRAP(L)
    lhs, const DOCTEST_REF_WRAP(R) rhs) const { return op(lhs, rhs); } };
```

Definition at line 1774 of file [doctest.h](#).

12.64.1.46 DOCTEST_BREAK_INTO_DEBUGGER

```
#define DOCTEST_BREAK_INTO_DEBUGGER()
```

Value:

```
raise(SIGTRAP)
```

Definition at line 613 of file [doctest.h](#).

12.64.1.47 DOCTEST_CAPTURE

```
#define DOCTEST_CAPTURE(
    x)
```

Value:

```
DOCTEST_INFO(#x " := ", x)
```

Definition at line 3145 of file [doctest.h](#).

12.64.1.48 DOCTEST_CAST_TO_VOID

```
#define DOCTEST_CAST_TO_VOID(  
    ...)
```

Value:

`__VA_ARGS__`;

Definition at line 2931 of file [doctest.h](#).

12.64.1.49 DOCTEST_CAT

```
#define DOCTEST_CAT(  
    s1,  
    s2)
```

Value:

`DOCTEST_CAT_IMPL(s1, s2)`

Definition at line 568 of file [doctest.h](#).

12.64.1.50 DOCTEST_CAT_IMPL

```
#define DOCTEST_CAT_IMPL(  
    s1,  
    s2)
```

Value:

`s1##s2`

Definition at line 567 of file [doctest.h](#).

12.64.1.51 DOCTEST_CHECK

```
#define DOCTEST_CHECK(  
    ...)
```

Value:

`DOCTEST_ASSERT_IMPLEMENT_1(DT_CHECK, __VA_ARGS__)`

Definition at line 3230 of file [doctest.h](#).

12.64.1.52 DOCTEST_CHECK_EQ

```
#define DOCTEST_CHECK_EQ(  
    ...)
```

Value:

`DOCTEST_BINARY_ASSERT(DT_CHECK_EQ, eq, __VA_ARGS__)`

Definition at line 3246 of file [doctest.h](#).

12.64.1.53 DOCTEST_CHECK_FALSE

```
#define DOCTEST_CHECK_FALSE(  
    ...)
```

Value:

[DOCTEST_ASSERT_IMPLEMENT_1](#)(DT_CHECK_FALSE, __VA_ARGS__)

Definition at line [3233](#) of file [doctest.h](#).

12.64.1.54 DOCTEST_CHECK_FALSE_MESSAGE

```
#define DOCTEST_CHECK_FALSE_MESSAGE(  
    cond,  
    ...)
```

Value:

[DOCTEST_FUNC_SCOPE_BEGIN](#) { [DOCTEST_INFO](#)(__VA_ARGS__); [DOCTEST_ASSERT_IMPLEMENT_2](#)(DT_CHECK_FALSE, cond); }
[DOCTEST_FUNC_SCOPE_END](#)

Definition at line [3241](#) of file [doctest.h](#).

12.64.1.55 DOCTEST_CHECK_GE

```
#define DOCTEST_CHECK_GE(  
    ...)
```

Value:

[DOCTEST_BINARY_ASSERT](#)(DT_CHECK_GE, ge, __VA_ARGS__)

Definition at line [3258](#) of file [doctest.h](#).

12.64.1.56 DOCTEST_CHECK_GT

```
#define DOCTEST_CHECK_GT(  
    ...)
```

Value:

[DOCTEST_BINARY_ASSERT](#)(DT_CHECK_GT, gt, __VA_ARGS__)

Definition at line [3252](#) of file [doctest.h](#).

12.64.1.57 DOCTEST_CHECK_LE

```
#define DOCTEST_CHECK_LE(  
    ...)
```

Value:

[DOCTEST_BINARY_ASSERT](#)(DT_CHECK_LE, le, __VA_ARGS__)

Definition at line [3261](#) of file [doctest.h](#).

12.64.1.58 DOCTEST_CHECK_LT

```
#define DOCTEST_CHECK_LT(  
    ...)
```

Value:

[DOCTEST_BINARY_ASSERT](#) (DT_CHECK_LT, lt, __VA_ARGS__)

Definition at line 3255 of file [doctest.h](#).

12.64.1.59 DOCTEST_CHECK_MESSAGE

```
#define DOCTEST_CHECK_MESSAGE(  
    cond,  
    ...)
```

Value:

[DOCTEST_FUNC_SCOPE_BEGIN](#) { [DOCTEST_INFO](#) (__VA_ARGS__); [DOCTEST_ASSERT_IMPLEMENT_2](#) (DT_CHECK, cond); }
[DOCTEST_FUNC_SCOPE_END](#)

Definition at line 3238 of file [doctest.h](#).

12.64.1.60 DOCTEST_CHECK_NE

```
#define DOCTEST_CHECK_NE(  
    ...)
```

Value:

[DOCTEST_BINARY_ASSERT](#) (DT_CHECK_NE, ne, __VA_ARGS__)

Definition at line 3249 of file [doctest.h](#).

12.64.1.61 DOCTEST_CHECK_NOTHROW

```
#define DOCTEST_CHECK_NOTHROW(  
    ...)
```

Value:

[DOCTEST_EXCEPTION_EMPTY_FUNC](#)

Definition at line 3678 of file [doctest.h](#).

12.64.1.62 DOCTEST_CHECK_NOTHROW_MESSAGE

```
#define DOCTEST_CHECK_NOTHROW_MESSAGE(  
    expr,  
    ...)
```

Value:

[DOCTEST_EXCEPTION_EMPTY_FUNC](#)

Definition at line 3694 of file [doctest.h](#).

12.64.1.63 DOCTEST_CHECK_THROWS

```
#define DOCTEST_CHECK_THROWS(  
    ...)
```

Value:

[DOCTEST_EXCEPTION_EMPTY_FUNC](#)

Definition at line 3666 of file [doctest.h](#).

12.64.1.64 DOCTEST_CHECK_THROWS_AS

```
#define DOCTEST_CHECK_THROWS_AS(  
    expr,  
    ...)
```

Value:

[DOCTEST_EXCEPTION_EMPTY_FUNC](#)

Definition at line 3669 of file [doctest.h](#).

12.64.1.65 DOCTEST_CHECK_THROWS_AS_MESSAGE

```
#define DOCTEST_CHECK_THROWS_AS_MESSAGE(  
    expr,  
    ex,  
    ...)
```

Value:

[DOCTEST_EXCEPTION_EMPTY_FUNC](#)

Definition at line 3685 of file [doctest.h](#).

12.64.1.66 DOCTEST_CHECK_THROWS_MESSAGE

```
#define DOCTEST_CHECK_THROWS_MESSAGE(  
    expr,  
    ...)
```

Value:

[DOCTEST_EXCEPTION_EMPTY_FUNC](#)

Definition at line 3682 of file [doctest.h](#).

12.64.1.67 DOCTEST_CHECK_THROWS_WITH

```
#define DOCTEST_CHECK_THROWS_WITH(  
    expr,  
    ...)
```

Value:

[DOCTEST_EXCEPTION_EMPTY_FUNC](#)

Definition at line 3672 of file [doctest.h](#).

12.64.1.68 DOCTEST_CHECK_THROWS_WITH_AS

```
#define DOCTEST_CHECK_THROWS_WITH_AS(  
    expr,  
    with,  
    ...)
```

Value:

[DOCTEST_EXCEPTION_EMPTY_FUNC](#)

Definition at line [3675](#) of file [doctest.h](#).

12.64.1.69 DOCTEST_CHECK_THROWS_WITH_AS_MESSAGE

```
#define DOCTEST_CHECK_THROWS_WITH_AS_MESSAGE(  
    expr,  
    with,  
    ex,  
    ...)
```

Value:

[DOCTEST_EXCEPTION_EMPTY_FUNC](#)

Definition at line [3691](#) of file [doctest.h](#).

12.64.1.70 DOCTEST_CHECK_THROWS_WITH_MESSAGE

```
#define DOCTEST_CHECK_THROWS_WITH_MESSAGE(  
    expr,  
    with,  
    ...)
```

Value:

[DOCTEST_EXCEPTION_EMPTY_FUNC](#)

Definition at line [3688](#) of file [doctest.h](#).

12.64.1.71 DOCTEST_CHECK_UNARY

```
#define DOCTEST_CHECK_UNARY(  
    ...)
```

Value:

[DOCTEST_UNARY_ASSERT](#)(DT_CHECK_UNARY, __VA_ARGS__)

Definition at line [3265](#) of file [doctest.h](#).

12.64.1.72 DOCTEST_CHECK_UNARY_FALSE

```
#define DOCTEST_CHECK_UNARY_FALSE(  
    ...)
```

Value:

[DOCTEST_UNARY_ASSERT](#)([DT_CHECK_UNARY_FALSE](#), __VA_ARGS__)

Definition at line [3268](#) of file [doctest.h](#).

12.64.1.73 DOCTEST_CLANG

```
#define DOCTEST_CLANG 0
```

Definition at line [110](#) of file [doctest.h](#).

12.64.1.74 DOCTEST_CLANG_SUPPRESS_WARNING

```
#define DOCTEST_CLANG_SUPPRESS_WARNING(  
    w)
```

Definition at line [137](#) of file [doctest.h](#).

12.64.1.75 DOCTEST_CLANG_SUPPRESS_WARNING_POP

```
#define DOCTEST_CLANG_SUPPRESS_WARNING_POP
```

Definition at line [138](#) of file [doctest.h](#).

12.64.1.76 DOCTEST_CLANG_SUPPRESS_WARNING_PUSH

```
#define DOCTEST_CLANG_SUPPRESS_WARNING_PUSH
```

Definition at line [136](#) of file [doctest.h](#).

12.64.1.77 DOCTEST_CLANG_SUPPRESS_WARNING_WITH_PUSH

```
#define DOCTEST_CLANG_SUPPRESS_WARNING_WITH_PUSH(  
    w)
```

Definition at line [139](#) of file [doctest.h](#).

12.64.1.78 DOCTEST_CMP_EQ

```
#define DOCTEST_CMP_EQ(  
    l,  
    r)
```

Value:

`l == r`

Definition at line [1752](#) of file [doctest.h](#).

12.64.1.79 DOCTEST_CMP_GE

```
#define DOCTEST_CMP_GE(  
    l,  
    r)
```

Value:

$l \geq r$

Definition at line 1756 of file [doctest.h](#).

12.64.1.80 DOCTEST_CMP_GT

```
#define DOCTEST_CMP_GT(  
    l,  
    r)
```

Value:

$l > r$

Definition at line 1754 of file [doctest.h](#).

12.64.1.81 DOCTEST_CMP_LE

```
#define DOCTEST_CMP_LE(  
    l,  
    r)
```

Value:

$l \leq r$

Definition at line 1757 of file [doctest.h](#).

12.64.1.82 DOCTEST_CMP_LT

```
#define DOCTEST_CMP_LT(  
    l,  
    r)
```

Value:

$l < r$

Definition at line 1755 of file [doctest.h](#).

12.64.1.83 DOCTEST_CMP_NE

```
#define DOCTEST_CMP_NE(  
    l,  
    r)
```

Value:

$l \neq r$

Definition at line 1753 of file [doctest.h](#).

12.64.1.84 DOCTEST_COMPARISON_RETURN_TYPE

```
#define DOCTEST_COMPARISON_RETURN_TYPE bool
```

Definition at line 1725 of file [doctest.h](#).

12.64.1.85 DOCTEST_COMPILER

```
#define DOCTEST_COMPILER(  
    MAJOR,  
    MINOR,  
    PATCH)
```

Value:

```
((MAJOR) * 10000000 + (MINOR) * 100000 + (PATCH))
```

Definition at line 87 of file [doctest.h](#).

12.64.1.86 DOCTEST_CONFIG_NO_EXCEPTIONS

```
#define DOCTEST_CONFIG_NO_EXCEPTIONS
```

Definition at line 384 of file [doctest.h](#).

12.64.1.87 DOCTEST_CONFIG_NO_TRY_CATCH_IN_ASSERTS

```
#define DOCTEST_CONFIG_NO_TRY_CATCH_IN_ASSERTS
```

Definition at line 395 of file [doctest.h](#).

12.64.1.88 DOCTEST_CONFIG_POSIX_SIGNALS

```
#define DOCTEST_CONFIG_POSIX_SIGNALS
```

Definition at line 376 of file [doctest.h](#).

12.64.1.89 DOCTEST_CONFIG_STRING_SIZE_TYPE

```
#define DOCTEST_CONFIG_STRING_SIZE_TYPE unsigned
```

Definition at line 858 of file [doctest.h](#).

12.64.1.90 DOCTEST_CONSTEXPR

```
#define DOCTEST_CONSTEXPR constexpr
```

Definition at line 482 of file [doctest.h](#).

12.64.1.91 DOCTEST_CONSTEXPR_FUNC

```
#define DOCTEST_CONSTEXPR_FUNC constexpr
```

Definition at line [483](#) of file [doctest.h](#).

12.64.1.92 DOCTEST_COUNTER

```
#define DOCTEST_COUNTER __LINE__
```

Definition at line [562](#) of file [doctest.h](#).

12.64.1.93 DOCTEST_CPLUSPLUS

```
#define DOCTEST_CPLUSPLUS __cplusplus
```

Definition at line [84](#) of file [doctest.h](#).

12.64.1.94 DOCTEST_CREATE_AND_REGISTER_FUNCTION

```
#define DOCTEST_CREATE_AND_REGISTER_FUNCTION(  
    f,  
    decorators)
```

Value:

```
static void f();  
DOCTEST_REGISTER_FUNCTION(DOCTEST_EMPTY, f, decorators)  
static void f()
```

Definition at line [2957](#) of file [doctest.h](#).

12.64.1.95 DOCTEST_CREATE_AND_REGISTER_FUNCTION_IN_CLASS

```
#define DOCTEST_CREATE_AND_REGISTER_FUNCTION_IN_CLASS(  
    f,  
    proxy,  
    decorators)
```

Value:

```
static doctest::detail::funcType proxy() {  
    return f;  
}  
DOCTEST_REGISTER_FUNCTION(inline, proxy(), decorators)  
static void f()
```

Definition at line [2962](#) of file [doctest.h](#).

12.64.1.96 DOCTEST_DECLARE_INTERFACE

```
#define DOCTEST_DECLARE_INTERFACE(  
    name)
```

Value:

```
virtual ~name();  
name() = default;  
name(const name &) = delete;  
name(name &&) = delete;  
name &operator=(const name &) = delete;  
name &operator=(name &&) = delete;
```

Definition at line 546 of file [doctest.h](#).

12.64.1.97 DOCTEST_DEFINE_DECORATOR

```
#define DOCTEST_DEFINE_DECORATOR(  
    name,  
    type,  
    def)
```

Value:

```
struct name {  
    type data;  
    name(type in = def) noexcept  
        : data(in) {}  
    void fill(detail::TestCase &state) const noexcept {  
        state.DOSTEST_CAT(m_, name) = data;  
    }  
    void fill(detail::TestSuite &state) const noexcept {  
        state.DOSTEST_CAT(m_, name) = data;  
    }  
}
```

Definition at line 2295 of file [doctest.h](#).

12.64.1.98 DOCTEST_DEFINE_INTERFACE

```
#define DOCTEST_DEFINE_INTERFACE(  
    name)
```

Value:

```
name::~~name() = default;
```

Definition at line 554 of file [doctest.h](#).

12.64.1.99 DOCTEST_DO_BINARY_EXPRESSION_COMPARISON

```
#define DOCTEST_DO_BINARY_EXPRESSION_COMPARISON(
    op,
    op_str,
    op_macro)
```

Value:

```
/* NOLINTBEGIN(cppcoreguidelines-missing-std-forward) */
template <typename R>
DOCTEST_NOINLINE SFINAE_OP(Result, op) operator op(R &&rhs) {
    bool res = op_macro(doctest::detail::forward<const L>(lhs), doctest::detail::forward<R>(rhs));
    if (m_at & assertType::is_false)
        res = !res;
    if (!res || doctest::getContextOptions()->success)
        return Result(res, stringifyBinaryExpr(lhs, op_str, rhs));
    return Result(res);
}
/* NOLINTEND(cppcoreguidelines-missing-std-forward) */
```

Definition at line 1931 of file [doctest.h](#).

12.64.1.100 DOCTEST_EMPTY

```
#define DOCTEST_EMPTY
```

Definition at line 439 of file [doctest.h](#).

12.64.1.101 DOCTEST_EXCEPTION_EMPTY_FUNC

```
#define DOCTEST_EXCEPTION_EMPTY_FUNC
```

Value:

```
[] {
    static_assert(
        false,
        "Exceptions are disabled! "
        "Use DOCTEST_CONFIG_NO_EXCEPTIONS_BUT_WITH_ALL_ASSERTS if you want "
        "to compile with exceptions disabled."
    );
    return false;
}()
```

Definition at line 3626 of file [doctest.h](#).

12.64.1.102 DOCTEST_FAIL

```
#define DOCTEST_FAIL(  
    ...)
```

Value:

[DOCTEST_ADD_FAIL_AT](#)(__FILE__, __LINE__, __VA_ARGS__)

Definition at line 3166 of file [doctest.h](#).

12.64.1.103 DOCTEST_FAIL_CHECK

```
#define DOCTEST_FAIL_CHECK(  
    ...)
```

Value:

[DOCTEST_ADD_FAIL_CHECK_AT](#)(__FILE__, __LINE__, __VA_ARGS__)

Definition at line 3165 of file [doctest.h](#).

12.64.1.104 DOCTEST_FAST_CHECK_EQ

```
#define DOCTEST_FAST_CHECK_EQ DOCTEST_CHECK_EQ
```

Definition at line 3701 of file [doctest.h](#).

12.64.1.105 DOCTEST_FAST_CHECK_GE

```
#define DOCTEST_FAST_CHECK_GE DOCTEST_CHECK_GE
```

Definition at line 3713 of file [doctest.h](#).

12.64.1.106 DOCTEST_FAST_CHECK_GT

```
#define DOCTEST_FAST_CHECK_GT DOCTEST_CHECK_GT
```

Definition at line 3707 of file [doctest.h](#).

12.64.1.107 DOCTEST_FAST_CHECK_LE

```
#define DOCTEST_FAST_CHECK_LE DOCTEST_CHECK_LE
```

Definition at line 3716 of file [doctest.h](#).

12.64.1.108 DOCTEST_FAST_CHECK_LT

```
#define DOCTEST_FAST_CHECK_LT DOCTEST_CHECK_LT
```

Definition at line 3710 of file [doctest.h](#).

12.64.1.109 DOCTEST_FAST_CHECK_NE

```
#define DOCTEST_FAST_CHECK_NE DOCTEST_CHECK_NE
```

Definition at line 3704 of file [doctest.h](#).

12.64.1.110 DOCTEST_FAST_CHECK_UNARY

```
#define DOCTEST_FAST_CHECK_UNARY DOCTEST_CHECK_UNARY
```

Definition at line 3720 of file [doctest.h](#).

12.64.1.111 DOCTEST_FAST_CHECK_UNARY_FALSE

```
#define DOCTEST_FAST_CHECK_UNARY_FALSE DOCTEST_CHECK_UNARY_FALSE
```

Definition at line 3723 of file [doctest.h](#).

12.64.1.112 DOCTEST_FAST_REQUIRE_EQ

```
#define DOCTEST_FAST_REQUIRE_EQ DOCTEST_REQUIRE_EQ
```

Definition at line 3702 of file [doctest.h](#).

12.64.1.113 DOCTEST_FAST_REQUIRE_GE

```
#define DOCTEST_FAST_REQUIRE_GE DOCTEST_REQUIRE_GE
```

Definition at line 3714 of file [doctest.h](#).

12.64.1.114 DOCTEST_FAST_REQUIRE_GT

```
#define DOCTEST_FAST_REQUIRE_GT DOCTEST_REQUIRE_GT
```

Definition at line 3708 of file [doctest.h](#).

12.64.1.115 DOCTEST_FAST_REQUIRE_LE

```
#define DOCTEST_FAST_REQUIRE_LE DOCTEST_REQUIRE_LE
```

Definition at line 3717 of file [doctest.h](#).

12.64.1.116 DOCTEST_FAST_REQUIRE_LT

```
#define DOCTEST_FAST_REQUIRE_LT DOCTEST_REQUIRE_LT
```

Definition at line 3711 of file [doctest.h](#).

12.64.1.117 DOCTEST_FAST_REQUIRE_NE

```
#define DOCTEST_FAST_REQUIRE_NE DOCTEST_REQUIRE_NE
```

Definition at line 3705 of file [doctest.h](#).

12.64.1.118 DOCTEST_FAST_REQUIRE_UNARY

```
#define DOCTEST_FAST_REQUIRE_UNARY DOCTEST_REQUIRE_UNARY
```

Definition at line 3721 of file [doctest.h](#).

12.64.1.119 DOCTEST_FAST_REQUIRE_UNARY_FALSE

```
#define DOCTEST_FAST_REQUIRE_UNARY_FALSE DOCTEST_REQUIRE_UNARY_FALSE
```

Definition at line 3724 of file [doctest.h](#).

12.64.1.120 DOCTEST_FAST_WARN_EQ

```
#define DOCTEST_FAST_WARN_EQ DOCTEST_WARN_EQ
```

Definition at line 3700 of file [doctest.h](#).

12.64.1.121 DOCTEST_FAST_WARN_GE

```
#define DOCTEST_FAST_WARN_GE DOCTEST_WARN_GE
```

Definition at line 3712 of file [doctest.h](#).

12.64.1.122 DOCTEST_FAST_WARN_GT

```
#define DOCTEST_FAST_WARN_GT DOCTEST_WARN_GT
```

Definition at line 3706 of file [doctest.h](#).

12.64.1.123 DOCTEST_FAST_WARN_LE

```
#define DOCTEST_FAST_WARN_LE DOCTEST_WARN_LE
```

Definition at line 3715 of file [doctest.h](#).

12.64.1.124 DOCTEST_FAST_WARN_LT

```
#define DOCTEST_FAST_WARN_LT DOCTEST_WARN_LT
```

Definition at line 3709 of file [doctest.h](#).

12.64.1.125 DOCTEST_FAST_WARN_NE

```
#define DOCTEST_FAST_WARN_NE DOCTEST_WARN_NE
```

Definition at line 3703 of file [doctest.h](#).

12.64.1.126 DOCTEST_FAST_WARN_UNARY

```
#define DOCTEST_FAST_WARN_UNARY DOCTEST_WARN_UNARY
```

Definition at line 3719 of file [doctest.h](#).

12.64.1.127 DOCTEST_FAST_WARN_UNARY_FALSE

```
#define DOCTEST_FAST_WARN_UNARY_FALSE DOCTEST_WARN_UNARY_FALSE
```

Definition at line 3722 of file [doctest.h](#).

12.64.1.128 DOCTEST_FORBIT_EXPRESSION

```
#define DOCTEST_FORBIT_EXPRESSION(  
    rt,  
    op)
```

Value:

```
template <typename R>  
    \  
    rt &operator op(const R &) {  
        \  
        static_assert(deferred_false<R>::value, "Expression Too Complex Please Rewrite As Binary Comparison!");  
        \  
        return *this;  
    }  
    \
```

Definition at line 1807 of file [doctest.h](#).

12.64.1.129 DOCTEST_FUNC_EMPTY

```
#define DOCTEST_FUNC_EMPTY (void)0
```

Definition at line 2893 of file [doctest.h](#).

12.64.1.130 DOCTEST_FUNC_SCOPE_BEGIN

```
#define DOCTEST_FUNC_SCOPE_BEGIN do /* NOLINT(cppcoreguidelines-avoid-do-while)*/
```

Definition at line 2904 of file [doctest.h](#).

12.64.1.131 DOCTEST_FUNC_SCOPE_END

```
#define DOCTEST_FUNC_SCOPE_END while (false)
```

Definition at line 2905 of file [doctest.h](#).

12.64.1.132 DOCTEST_FUNC_SCOPE_RET

```
#define DOCTEST_FUNC_SCOPE_RET(  
    v)
```

Value:

```
(void)0
```

Definition at line 2906 of file [doctest.h](#).

12.64.1.133 DOCTEST_GCC

```
#define DOCTEST_GCC 0
```

Definition at line 113 of file [doctest.h](#).

12.64.1.134 DOCTEST_GCC_SUPPRESS_WARNING

```
#define DOCTEST_GCC_SUPPRESS_WARNING(  
    w)
```

Definition at line 150 of file [doctest.h](#).

12.64.1.135 DOCTEST_GCC_SUPPRESS_WARNING_POP

```
#define DOCTEST_GCC_SUPPRESS_WARNING_POP
```

Definition at line 151 of file [doctest.h](#).

12.64.1.136 DOCTEST_GCC_SUPPRESS_WARNING_PUSH

```
#define DOCTEST_GCC_SUPPRESS_WARNING_PUSH
```

Definition at line 149 of file [doctest.h](#).

12.64.1.137 DOCTEST_GCC_SUPPRESS_WARNING_WITH_PUSH

```
#define DOCTEST_GCC_SUPPRESS_WARNING_WITH_PUSH(  
    w)
```

Definition at line 152 of file [doctest.h](#).

12.64.1.138 DOCTEST_GENERATE

```
#define DOCTEST_GENERATE(  
    ...)
```

Value:

```
doctest::detail::acquireGeneratorValue(__VA_ARGS__)
```

Definition at line 3064 of file [doctest.h](#).

12.64.1.139 DOCTEST_GIVEN

```
#define DOCTEST_GIVEN(  
    name)
```

Value:

```
DOCTEST_SUBCASE("    Given: " name)
```

Definition at line 3736 of file [doctest.h](#).

12.64.1.140 DOCTEST_GLOBAL_NO_WARNINGS

```
#define DOCTEST_GLOBAL_NO_WARNINGS(  
    var,  
    ...)
```

Value:

```
DOCTEST_CLANG_SUPPRESS_WARNING_WITH_PUSH("-Wglobal-constructors")  
    \  
static const int var = doctest::detail::consume(&var, __VA_ARGS__);  
    \  
DOCTEST_CLANG_SUPPRESS_WARNING_POP
```

Definition at line 587 of file [doctest.h](#).

12.64.1.141 DOCTEST_HAS_BUILTIN

```
#define DOCTEST_HAS_BUILTIN(  
    x)
```

Value:

```
0
```

Definition at line 532 of file [doctest.h](#).

12.64.1.142 DOCTEST_ICC

```
#define DOCTEST_ICC 0
```

Definition at line 116 of file [doctest.h](#).

12.64.1.143 DOCTEST_IMPLEMENT_FIXTURE

```
#define DOCTEST_IMPLEMENT_FIXTURE(
    der,
    base,
    func,
    decorators)
```

Value:

```
namespace { /* NOLINT */
    struct der : public base {
        void f();
    };
    static DOCTEST_INLINE_NOINLINE void func() {
        der v;
        v.f();
    }
    DOCTEST_REGISTER_FUNCTION(DOCTEST_EMPTY, func, decorators)
}
DOCTEST_INLINE_NOINLINE void der::f()
```

Definition at line 2944 of file [doctest.h](#).

12.64.1.144 DOCTEST_INFO

```
#define DOCTEST_INFO(
    ...)
```

Value:

```
DOCTEST_INFO_IMPL(DOCTEST_ANONYMOUS(DOCTEST_CAPTURE_MB_), DOCTEST_ANONYMOUS(DOCTEST_CAPTURE_OTHER_),
    __VA_ARGS__)
```

Definition at line 3135 of file [doctest.h](#).

12.64.1.145 DOCTEST_INFO_IMPL

```
#define DOCTEST_INFO_IMPL(
    mb_name,
    s_name,
    ...)
```

Value:

```
auto DOCTEST_ANONYMOUS(DOCTEST_CAPTURE_) = doctest::detail::MakeContextScope([&](std::ostream *s_name) {
    doctest::detail::MessageBuilder mb_name(__FILE__, __LINE__, doctest::assertType::is_warn);
    mb_name.m_stream = s_name;
    mb_name *__VA_ARGS__;
})
```

Definition at line 3138 of file [doctest.h](#).

12.64.1.146 DOCTEST_INLINE_NOINLINE

```
#define DOCTEST_INLINE_NOINLINE inline DOCTEST_NOINLINE
```

Definition at line 458 of file [doctest.h](#).

12.64.1.147 DOCTEST_INTERFACE

```
#define DOCTEST_INTERFACE
```

Definition at line 426 of file [doctest.h](#).

12.64.1.148 DOCTEST_INTERFACE_DECL

```
#define DOCTEST_INTERFACE_DECL DOCTEST_INTERFACE
```

Definition at line 435 of file [doctest.h](#).

12.64.1.149 DOCTEST_INTERFACE_DEF

```
#define DOCTEST_INTERFACE_DEF
```

Definition at line 436 of file [doctest.h](#).

12.64.1.150 DOCTEST_MAKE_STD_HEADERS_CLEAN_FROM_WARNINGS_ON_WALL_BEGIN

```
#define DOCTEST_MAKE_STD_HEADERS_CLEAN_FROM_WARNINGS_ON_WALL_BEGIN
```

Value:

```
DOCTEST_MSVC_SUPPRESS_WARNING_PUSH
\
DOCTEST_MSVC_SUPPRESS_WARNING(4548) /* before comma no effect; expected side - effect */
\
DOCTEST_MSVC_SUPPRESS_WARNING(4265) /* virtual functions, but destructor is not virtual */
\
DOCTEST_MSVC_SUPPRESS_WARNING(4986) /* exception specification does not match previous */
\
DOCTEST_MSVC_SUPPRESS_WARNING(4350) /* 'member1' called instead of 'member2' */
\
DOCTEST_MSVC_SUPPRESS_WARNING(4668) /* not defined as a preprocessor macro */
\
DOCTEST_MSVC_SUPPRESS_WARNING(4365) /* signed/unsigned mismatch */
\
DOCTEST_MSVC_SUPPRESS_WARNING(4774) /* format string not a string literal */
\
DOCTEST_MSVC_SUPPRESS_WARNING(4820) /* padding */
\
DOCTEST_MSVC_SUPPRESS_WARNING(4625) /* copy constructor was implicitly deleted */
\
DOCTEST_MSVC_SUPPRESS_WARNING(4626) /* assignment operator was implicitly deleted */
\
DOCTEST_MSVC_SUPPRESS_WARNING(5027) /* move assignment operator implicitly deleted */
\
DOCTEST_MSVC_SUPPRESS_WARNING(5026) /* move constructor was implicitly deleted */
\
DOCTEST_MSVC_SUPPRESS_WARNING(4623) /* default constructor was implicitly deleted */
\
DOCTEST_MSVC_SUPPRESS_WARNING(5039) /* pointer to pot. throwing function passed to extern C */
\
DOCTEST_MSVC_SUPPRESS_WARNING(5045) /* Spectre mitigation for memory load */
\
DOCTEST_MSVC_SUPPRESS_WARNING(5105) /* macro producing 'defined' has undefined behavior */
\
DOCTEST_MSVC_SUPPRESS_WARNING(4738) /* storing float result in memory, loss of performance */
\
DOCTEST_MSVC_SUPPRESS_WARNING(5262) /* implicit fall-through */
```

Definition at line 287 of file [doctest.h](#).

12.64.1.151 DOCTEST_MAKE_STD_HEADERS_CLEAN_FROM_WARNINGS_ON_WALL_END

```
#define DOCTEST_MAKE_STD_HEADERS_CLEAN_FROM_WARNINGS_ON_WALL_END DOCTEST_MSVC_SUPPRESS_WARNING_POP
```

Definition at line 308 of file [doctest.h](#).

12.64.1.152 DOCTEST_MESSAGE

```
#define DOCTEST_MESSAGE(  
    ...)
```

Value:

```
DOCTEST_ADD_MESSAGE_AT(__FILE__, __LINE__, __VA_ARGS__)
```

Definition at line 3164 of file [doctest.h](#).

12.64.1.153 DOCTEST_MSVC

```
#define DOCTEST_MSVC 0
```

Definition at line 107 of file [doctest.h](#).

12.64.1.154 DOCTEST_MSVC_SUPPRESS_WARNING

```
#define DOCTEST_MSVC_SUPPRESS_WARNING(  
    w)
```

Definition at line 162 of file [doctest.h](#).

12.64.1.155 DOCTEST_MSVC_SUPPRESS_WARNING_POP

```
#define DOCTEST_MSVC_SUPPRESS_WARNING_POP
```

Definition at line 163 of file [doctest.h](#).

12.64.1.156 DOCTEST_MSVC_SUPPRESS_WARNING_PUSH

```
#define DOCTEST_MSVC_SUPPRESS_WARNING_PUSH
```

Definition at line 161 of file [doctest.h](#).

12.64.1.157 DOCTEST_MSVC_SUPPRESS_WARNING_WITH_PUSH

```
#define DOCTEST_MSVC_SUPPRESS_WARNING_WITH_PUSH(  
    w)
```

Definition at line 164 of file [doctest.h](#).

12.64.1.158 DOCTEST_NO_SANITIZE_INTEGER

```
#define DOCTEST_NO_SANITIZE_INTEGER
```

Definition at line 491 of file [doctest.h](#).

12.64.1.159 DOCTEST_NOEXCEPT

```
#define DOCTEST_NOEXCEPT noexcept
```

Definition at line 473 of file [doctest.h](#).

12.64.1.160 DOCTEST_NOINLINE

```
#define DOCTEST_NOINLINE __attribute__((noinline))
```

Definition at line 450 of file [doctest.h](#).

12.64.1.161 DOCTEST_NORETURN

```
#define DOCTEST_NORETURN [[noreturn]]
```

Definition at line 465 of file [doctest.h](#).

12.64.1.162 DOCTEST_PARTS_PUBLIC_ASSERT_COMPARATOR

```
#define DOCTEST_PARTS_PUBLIC_ASSERT_COMPARATOR
```

Definition at line 1700 of file [doctest.h](#).

12.64.1.163 DOCTEST_PARTS_PUBLIC_ASSERT_DATA

```
#define DOCTEST_PARTS_PUBLIC_ASSERT_DATA
```

Definition at line 1630 of file [doctest.h](#).

12.64.1.164 DOCTEST_PARTS_PUBLIC_ASSERT_EXPRESSION

```
#define DOCTEST_PARTS_PUBLIC_ASSERT_EXPRESSION
```

Definition at line 1907 of file [doctest.h](#).

12.64.1.165 DOCTEST_PARTS_PUBLIC_ASSERT_HANDLER

```
#define DOCTEST_PARTS_PUBLIC_ASSERT_HANDLER
```

Definition at line 2635 of file [doctest.h](#).

12.64.1.166 DOCTEST_PARTS_PUBLIC_ASSERT_MESSAGE

```
#define DOCTEST_PARTS_PUBLIC_ASSERT_MESSAGE
```

Definition at line 2470 of file [doctest.h](#).

12.64.1.167 DOCTEST_PARTS_PUBLIC_ASSERT_RESULT

```
#define DOCTEST_PARTS_PUBLIC_ASSERT_RESULT
```

Definition at line 1794 of file [doctest.h](#).

12.64.1.168 DOCTEST_PARTS_PUBLIC_ASSERT_TYPE

```
#define DOCTEST_PARTS_PUBLIC_ASSERT_TYPE
```

Definition at line 1525 of file [doctest.h](#).

12.64.1.169 DOCTEST_PARTS_PUBLIC_COLOR

```
#define DOCTEST_PARTS_PUBLIC_COLOR
```

Definition at line 2078 of file [doctest.h](#).

12.64.1.170 DOCTEST_PARTS_PUBLIC_COMPILER

```
#define DOCTEST_PARTS_PUBLIC_COMPILER
```

Definition at line 77 of file [doctest.h](#).

12.64.1.171 DOCTEST_PARTS_PUBLIC_CONFIG

```
#define DOCTEST_PARTS_PUBLIC_CONFIG
```

Definition at line 319 of file [doctest.h](#).

12.64.1.172 DOCTEST_PARTS_PUBLIC_CONTEXT

```
#define DOCTEST_PARTS_PUBLIC_CONTEXT
```

Definition at line 2581 of file [doctest.h](#).

12.64.1.173 DOCTEST_PARTS_PUBLIC_CONTEXT_OPTIONS

```
#define DOCTEST_PARTS_PUBLIC_CONTEXT_OPTIONS
```

Definition at line 1459 of file [doctest.h](#).

12.64.1.174 DOCTEST_PARTS_PUBLIC_CONTEXT_SCOPE

```
#define DOCTEST_PARTS_PUBLIC_CONTEXT_SCOPE
```

Definition at line 2394 of file [doctest.h](#).

12.64.1.175 DOCTEST_PARTS_PUBLIC_DEBUGGER

```
#define DOCTEST_PARTS_PUBLIC_DEBUGGER
```

Definition at line 596 of file [doctest.h](#).

12.64.1.176 DOCTEST_PARTS_PUBLIC_DECORATORS

```
#define DOCTEST_PARTS_PUBLIC_DECORATORS
```

Definition at line 2288 of file [doctest.h](#).

12.64.1.177 DOCTEST_PARTS_PUBLIC_EXCEPTION_TRANSLATOR

```
#define DOCTEST_PARTS_PUBLIC_EXCEPTION_TRANSLATOR
```

Definition at line 2324 of file [doctest.h](#).

12.64.1.178 DOCTEST_PARTS_PUBLIC_EXCEPTIONS

```
#define DOCTEST_PARTS_PUBLIC_EXCEPTIONS
```

Definition at line 2553 of file [doctest.h](#).

12.64.1.179 DOCTEST_PARTS_PUBLIC_GENERATOR

```
#define DOCTEST_PARTS_PUBLIC_GENERATOR
```

Definition at line 2847 of file [doctest.h](#).

12.64.1.180 DOCTEST_PARTS_PUBLIC_MACROS

```
#define DOCTEST_PARTS_PUBLIC_MACROS
```

Definition at line 2873 of file [doctest.h](#).

12.64.1.181 DOCTEST_PARTS_PUBLIC_MATCHERS_APPROX

```
#define DOCTEST_PARTS_PUBLIC_MATCHERS_APPROX
```

Definition at line 1330 of file [doctest.h](#).

12.64.1.182 DOCTEST_PARTS_PUBLIC_MATCHERS_CONTAINS

```
#define DOCTEST_PARTS_PUBLIC_MATCHERS_CONTAINS
```

Definition at line 1301 of file [doctest.h](#).

12.64.1.183 DOCTEST_PARTS_PUBLIC_MATCHERS_IS_NAN

```
#define DOCTEST_PARTS_PUBLIC_MATCHERS_IS_NAN
```

Definition at line 1422 of file [doctest.h](#).

12.64.1.184 DOCTEST_PARTS_PUBLIC_PATH

```
#define DOCTEST_PARTS_PUBLIC_PATH
```

Definition at line 2538 of file [doctest.h](#).

12.64.1.185 DOCTEST_PARTS_PUBLIC_PLATFORM

```
#define DOCTEST_PARTS_PUBLIC_PLATFORM
```

Definition at line 322 of file [doctest.h](#).

12.64.1.186 DOCTEST_PARTS_PUBLIC_REPORTER

```
#define DOCTEST_PARTS_PUBLIC_REPORTER
```

Definition at line 2723 of file [doctest.h](#).

12.64.1.187 DOCTEST_PARTS_PUBLIC_STD_FWD

```
#define DOCTEST_PARTS_PUBLIC_STD_FWD
```

Definition at line 652 of file [doctest.h](#).

12.64.1.188 DOCTEST_PARTS_PUBLIC_STD_TYPE_TRAITS

```
#define DOCTEST_PARTS_PUBLIC_STD_TYPE_TRAITS
```

Definition at line 710 of file [doctest.h](#).

12.64.1.189 DOCTEST_PARTS_PUBLIC_STD_UTILITY

```
#define DOCTEST_PARTS_PUBLIC_STD_UTILITY
```

Definition at line 818 of file [doctest.h](#).

12.64.1.190 DOCTEST_PARTS_PUBLIC_STRING

```
#define DOCTEST_PARTS_PUBLIC_STRING
```

Definition at line 851 of file [doctest.h](#).

12.64.1.191 DOCTEST_PARTS_PUBLIC_SUBCASE

```
#define DOCTEST_PARTS_PUBLIC_SUBCASE
```

Definition at line 2111 of file [doctest.h](#).

12.64.1.192 DOCTEST_PARTS_PUBLIC_TEST_CASE

```
#define DOCTEST_PARTS_PUBLIC_TEST_CASE
```

Definition at line 2210 of file [doctest.h](#).

12.64.1.193 DOCTEST_PARTS_PUBLIC_TEST_SUITE

```
#define DOCTEST_PARTS_PUBLIC_TEST_SUITE
```

Definition at line 2154 of file [doctest.h](#).

12.64.1.194 DOCTEST_PARTS_PUBLIC_UTILITY

```
#define DOCTEST_PARTS_PUBLIC_UTILITY
```

Definition at line 541 of file [doctest.h](#).

12.64.1.195 DOCTEST_PARTS_PUBLIC_VERSION

```
#define DOCTEST_PARTS_PUBLIC_VERSION
```

Definition at line 50 of file [doctest.h](#).

12.64.1.196 DOCTEST_PARTS_PUBLIC_WARNINGS

```
#define DOCTEST_PARTS_PUBLIC_WARNINGS
```

Definition at line 125 of file [doctest.h](#).

12.64.1.197 DOCTEST_PLATFORM_LINUX

```
#define DOCTEST_PLATFORM_LINUX
```

Definition at line 343 of file [doctest.h](#).

12.64.1.198 DOCTEST_REF_WRAP

```
#define DOCTEST_REF_WRAP(  
    x)
```

Value:

```
x &
```

Definition at line 572 of file [doctest.h](#).

12.64.1.199 DOCTEST_REGISTER_EXCEPTION_TRANSLATOR

```
#define DOCTEST_REGISTER_EXCEPTION_TRANSLATOR(  
    signature)
```

Value:

```
DOCTEST_REGISTER_EXCEPTION_TRANSLATOR_IMPL(DOCTEST_ANONYMOUS(DOCTEST_ANON_TRANSLATOR_), signature)
```

Definition at line 3116 of file [doctest.h](#).

12.64.1.200 DOCTEST_REGISTER_EXCEPTION_TRANSLATOR_IMPL

```
#define DOCTEST_REGISTER_EXCEPTION_TRANSLATOR_IMPL(  
    translatorName,  
    signature)
```

Value:

```
inline doctest::String translatorName(signature);  
DOCTEST_GLOBAL_NO_WARNINGS(  
    DOCTEST_ANONYMOUS(DOCTEST_ANON_TRANSLATOR_VAR_), /* NOLINT(cert-err58-cpp) */  
    doctest::registerExceptionTranslator(translatorName)  
)  
doctest::String translatorName(signature)
```

Definition at line 3108 of file [doctest.h](#).

12.64.1.201 DOCTEST_REGISTER_FUNCTION

```
#define DOCTEST_REGISTER_FUNCTION(  
    global_prefix,  
    f,  
    decorators)
```

Value:

```
global_prefix DOCTEST_GLOBAL_NO_WARNINGS(  
    DOCTEST_ANONYMOUS(DOCTEST_ANON_VAR_), /* NOLINT */  
    doctest::detail::regTest(  
        doctest::detail::TestCase(f, __FILE__, __LINE__,  
        doctest_detail_test_suite_ns::getCurrentTestSuite()) *  
        decorators  
    )  
)
```

Definition at line 2935 of file [doctest.h](#).

12.64.1.202 DOCTEST_REGISTER_LISTENER

```
#define DOCTEST_REGISTER_LISTENER(
    name,
    priority,
    reporter)
```

Value:

```
DOCTEST_GLOBAL_NO_WARNINGS(
    \
    DOCTEST_ANONYMOUS(DOCTEST_ANON_REPORTER_VAR_), /* NOLINT(cert-err58-cpp) */
    \
    doctest::registerReporter<reporter>(name, priority, false)
)
\
static_assert(true, "")
```

Definition at line 3128 of file [doctest.h](#).

12.64.1.203 DOCTEST_REGISTER_REPORTER

```
#define DOCTEST_REGISTER_REPORTER(
    name,
    priority,
    reporter)
```

Value:

```
DOCTEST_GLOBAL_NO_WARNINGS(
    \
    DOCTEST_ANONYMOUS(DOCTEST_ANON_REPORTER_VAR_), /* NOLINT(cert-err58-cpp) */
    \
    doctest::registerReporter<reporter>(name, priority, true)
)
\
static_assert(true, "")
```

Definition at line 3120 of file [doctest.h](#).

12.64.1.204 DOCTEST_RELATIONAL_OP

```
#define DOCTEST_RELATIONAL_OP(
    name,
    op)
```

Value:

```
template <typename L, typename R>
DOCTEST_COMPARISON_RETURN_TYPE name(const DOCTEST_REF_WRAP(L) lhs, const DOCTEST_REF_WRAP(R) rhs) {
    return lhs op rhs;
}
```

Definition at line 1738 of file [doctest.h](#).

12.64.1.205 DOCTEST_REQUIRE [1/2]

```
#define DOCTEST_REQUIRE DOCTEST_EXCEPTION_EMPTY_FUNC
```

Definition at line 3231 of file [doctest.h](#).

12.64.1.206 DOCTEST_REQUIRE [2/2]

```
#define DOCTEST_REQUIRE(  
    ...)
```

Value:

[DOCTEST_ASSERT_IMPLEMENT_1](#)(DT_REQUIRE, __VA_ARGS__)

Definition at line [3231](#) of file [doctest.h](#).

12.64.1.207 DOCTEST_REQUIRE_EQ [1/2]

```
#define DOCTEST_REQUIRE_EQ DOCTEST_EXCEPTION_EMPTY_FUNC
```

Definition at line [3247](#) of file [doctest.h](#).

12.64.1.208 DOCTEST_REQUIRE_EQ [2/2]

```
#define DOCTEST_REQUIRE_EQ(  
    ...)
```

Value:

[DOCTEST_BINARY_ASSERT](#)(DT_REQUIRE_EQ, eq, __VA_ARGS__)

Definition at line [3247](#) of file [doctest.h](#).

12.64.1.209 DOCTEST_REQUIRE_FALSE [1/2]

```
#define DOCTEST_REQUIRE_FALSE DOCTEST_EXCEPTION_EMPTY_FUNC
```

Definition at line [3234](#) of file [doctest.h](#).

12.64.1.210 DOCTEST_REQUIRE_FALSE [2/2]

```
#define DOCTEST_REQUIRE_FALSE(  
    ...)
```

Value:

[DOCTEST_ASSERT_IMPLEMENT_1](#)(DT_REQUIRE_FALSE, __VA_ARGS__)

Definition at line [3234](#) of file [doctest.h](#).

12.64.1.211 DOCTEST_REQUIRE_FALSE_MESSAGE [1/2]

```
#define DOCTEST_REQUIRE_FALSE_MESSAGE DOCTEST_EXCEPTION_EMPTY_FUNC
```

Definition at line [3242](#) of file [doctest.h](#).

12.64.1.212 DOCTEST_REQUIRE_FALSE_MESSAGE [2/2]

```
#define DOCTEST_REQUIRE_FALSE_MESSAGE(  
    cond,  
    ...)
```

Value:

```
DOCTEST_FUNC_SCOPE_BEGIN { DOCTEST_INFO(__VA_ARGS__); DOCTEST_ASSERT_IMPLEMENT_2(DT_REQUIRE_FALSE, cond); }  
DOCTEST_FUNC_SCOPE_END
```

Definition at line 3242 of file [doctest.h](#).

12.64.1.213 DOCTEST_REQUIRE_GE [1/2]

```
#define DOCTEST_REQUIRE_GE DOCTEST_EXCEPTION_EMPTY_FUNC
```

Definition at line 3259 of file [doctest.h](#).

12.64.1.214 DOCTEST_REQUIRE_GE [2/2]

```
#define DOCTEST_REQUIRE_GE(  
    ...)
```

Value:

```
DOCTEST_BINARY_ASSERT(DT_REQUIRE_GE, ge, __VA_ARGS__)
```

Definition at line 3259 of file [doctest.h](#).

12.64.1.215 DOCTEST_REQUIRE_GT [1/2]

```
#define DOCTEST_REQUIRE_GT DOCTEST_EXCEPTION_EMPTY_FUNC
```

Definition at line 3253 of file [doctest.h](#).

12.64.1.216 DOCTEST_REQUIRE_GT [2/2]

```
#define DOCTEST_REQUIRE_GT(  
    ...)
```

Value:

```
DOCTEST_BINARY_ASSERT(DT_REQUIRE_GT, gt, __VA_ARGS__)
```

Definition at line 3253 of file [doctest.h](#).

12.64.1.217 DOCTEST_REQUIRE_LE [1/2]

```
#define DOCTEST_REQUIRE_LE DOCTEST_EXCEPTION_EMPTY_FUNC
```

Definition at line 3262 of file [doctest.h](#).

12.64.1.218 DOCTEST_REQUIRE_LE [2/2]

```
#define DOCTEST_REQUIRE_LE(  
    ...)
```

Value:

```
DOCTEST_BINARY_ASSERT(DT_REQUIRE_LE, le, __VA_ARGS__)
```

Definition at line 3262 of file [doctest.h](#).

12.64.1.219 DOCTEST_REQUIRE_LT [1/2]

```
#define DOCTEST_REQUIRE_LT DOCTEST_EXCEPTION_EMPTY_FUNC
```

Definition at line 3256 of file [doctest.h](#).

12.64.1.220 DOCTEST_REQUIRE_LT [2/2]

```
#define DOCTEST_REQUIRE_LT(  
    ...)
```

Value:

```
DOCTEST_BINARY_ASSERT(DT_REQUIRE_LT, lt, __VA_ARGS__)
```

Definition at line 3256 of file [doctest.h](#).

12.64.1.221 DOCTEST_REQUIRE_MESSAGE [1/2]

```
#define DOCTEST_REQUIRE_MESSAGE DOCTEST_EXCEPTION_EMPTY_FUNC
```

Definition at line 3239 of file [doctest.h](#).

12.64.1.222 DOCTEST_REQUIRE_MESSAGE [2/2]

```
#define DOCTEST_REQUIRE_MESSAGE(  
    cond,  
    ...)
```

Value:

```
DOCTEST_FUNC_SCOPE_BEGIN { DOCTEST_INFO(__VA_ARGS__); DOCTEST_ASSERT_IMPLEMENT_2(DT_REQUIRE, cond); }  
DOCTEST_FUNC_SCOPE_END
```

Definition at line 3239 of file [doctest.h](#).

12.64.1.223 DOCTEST_REQUIRE_NE [1/2]

```
#define DOCTEST_REQUIRE_NE DOCTEST_EXCEPTION_EMPTY_FUNC
```

Definition at line 3250 of file [doctest.h](#).

12.64.1.224 DOCTEST_REQUIRE_NE [2/2]

```
#define DOCTEST_REQUIRE_NE(  
    ...)
```

Value:

[DOCTEST_BINARY_ASSERT](#) (DT_REQUIRE_NE, ne, __VA_ARGS__)

Definition at line [3250](#) of file [doctest.h](#).

12.64.1.225 DOCTEST_REQUIRE_NOTHROW

```
#define DOCTEST_REQUIRE_NOTHROW(  
    ...)
```

Value:

[DOCTEST_EXCEPTION_EMPTY_FUNC](#)

Definition at line [3679](#) of file [doctest.h](#).

12.64.1.226 DOCTEST_REQUIRE_NOTHROW_MESSAGE

```
#define DOCTEST_REQUIRE_NOTHROW_MESSAGE(  
    expr,  
    ...)
```

Value:

[DOCTEST_EXCEPTION_EMPTY_FUNC](#)

Definition at line [3695](#) of file [doctest.h](#).

12.64.1.227 DOCTEST_REQUIRE_THROWS

```
#define DOCTEST_REQUIRE_THROWS(  
    ...)
```

Value:

[DOCTEST_EXCEPTION_EMPTY_FUNC](#)

Definition at line [3667](#) of file [doctest.h](#).

12.64.1.228 DOCTEST_REQUIRE_THROWS_AS

```
#define DOCTEST_REQUIRE_THROWS_AS(  
    expr,  
    ...)
```

Value:

[DOCTEST_EXCEPTION_EMPTY_FUNC](#)

Definition at line [3670](#) of file [doctest.h](#).

12.64.1.229 DOCTEST_REQUIRE_THROWS_AS_MESSAGE

```
#define DOCTEST_REQUIRE_THROWS_AS_MESSAGE(  
    expr,  
    ex,  
    ...)
```

Value:

[DOCTEST_EXCEPTION_EMPTY_FUNC](#)

Definition at line [3686](#) of file [doctest.h](#).

12.64.1.230 DOCTEST_REQUIRE_THROWS_MESSAGE

```
#define DOCTEST_REQUIRE_THROWS_MESSAGE(  
    expr,  
    ...)
```

Value:

[DOCTEST_EXCEPTION_EMPTY_FUNC](#)

Definition at line [3683](#) of file [doctest.h](#).

12.64.1.231 DOCTEST_REQUIRE_THROWS_WITH

```
#define DOCTEST_REQUIRE_THROWS_WITH(  
    expr,  
    ...)
```

Value:

[DOCTEST_EXCEPTION_EMPTY_FUNC](#)

Definition at line [3673](#) of file [doctest.h](#).

12.64.1.232 DOCTEST_REQUIRE_THROWS_WITH_AS

```
#define DOCTEST_REQUIRE_THROWS_WITH_AS(  
    expr,  
    with,  
    ...)
```

Value:

[DOCTEST_EXCEPTION_EMPTY_FUNC](#)

Definition at line [3676](#) of file [doctest.h](#).

12.64.1.233 DOCTEST_REQUIRE_THROWS_WITH_AS_MESSAGE

```
#define DOCTEST_REQUIRE_THROWS_WITH_AS_MESSAGE(  
    expr,  
    with,  
    ex,  
    ...)
```

Value:

[DOCTEST_EXCEPTION_EMPTY_FUNC](#)

Definition at line [3692](#) of file [doctest.h](#).

12.64.1.234 DOCTEST_REQUIRE_THROWS_WITH_MESSAGE

```
#define DOCTEST_REQUIRE_THROWS_WITH_MESSAGE(  
    expr,  
    with,  
    ...)
```

Value:

[DOCTEST_EXCEPTION_EMPTY_FUNC](#)

Definition at line [3689](#) of file [doctest.h](#).

12.64.1.235 DOCTEST_REQUIRE_UNARY [1/2]

```
#define DOCTEST_REQUIRE_UNARY DOCTEST\_EXCEPTION\_EMPTY\_FUNC
```

Definition at line [3266](#) of file [doctest.h](#).

12.64.1.236 DOCTEST_REQUIRE_UNARY [2/2]

```
#define DOCTEST_REQUIRE_UNARY(  
    ...)
```

Value:

[DOCTEST_UNARY_ASSERT](#)(DT_REQUIRE_UNARY, __VA_ARGS__)

Definition at line [3266](#) of file [doctest.h](#).

12.64.1.237 DOCTEST_REQUIRE_UNARY_FALSE [1/2]

```
#define DOCTEST_REQUIRE_UNARY_FALSE DOCTEST\_EXCEPTION\_EMPTY\_FUNC
```

Definition at line [3269](#) of file [doctest.h](#).

12.64.1.238 DOCTEST_REQUIRE_UNARY_FALSE [2/2]

```
#define DOCTEST_REQUIRE_UNARY_FALSE(  
    ...)
```

Value:

[DOCTEST_UNARY_ASSERT](#)(DT_REQUIRE_UNARY_FALSE, __VA_ARGS__)

Definition at line 3269 of file [doctest.h](#).

12.64.1.239 DOCTEST_SCENARIO

```
#define DOCTEST_SCENARIO(  
    name)
```

Value:

[DOCTEST_TEST_CASE](#)(" Scenario: " name)

Definition at line 3730 of file [doctest.h](#).

12.64.1.240 DOCTEST_SCENARIO_CLASS

```
#define DOCTEST_SCENARIO_CLASS(  
    name)
```

Value:

[DOCTEST_TEST_CASE_CLASS](#)(" Scenario: " name)

Definition at line 3731 of file [doctest.h](#).

12.64.1.241 DOCTEST_SCENARIO_METHOD

```
#define DOCTEST_SCENARIO_METHOD(  
    x,  
    name)
```

Value:

[DOCTEST_TEST_CASE_FIXTURE](#)(x, " Scenario: " name)

Definition at line 3732 of file [doctest.h](#).

12.64.1.242 DOCTEST_SCENARIO_TEMPLATE

```
#define DOCTEST_SCENARIO_TEMPLATE(  
    name,  
    T,  
    ...)
```

Value:

[DOCTEST_TEST_CASE_TEMPLATE](#)(" Scenario: " name, T, __VA_ARGS__)

Definition at line 3733 of file [doctest.h](#).

12.64.1.243 DOCTEST_SCENARIO_TEMPLATE_DEFINE

```
#define DOCTEST_SCENARIO_TEMPLATE_DEFINE(  
    name,  
    T,  
    id)
```

Value:

```
DOCTEST_TEST_CASE_TEMPLATE_DEFINE(" Scenario: " name, T, id)
```

Definition at line 3734 of file [doctest.h](#).

12.64.1.244 DOCTEST_STRINGIFY

```
#define DOCTEST_STRINGIFY(  
    ...)
```

Value:

```
toString(__VA_ARGS__)
```

Definition at line 1123 of file [doctest.h](#).

12.64.1.245 DOCTEST_SUBCASE

```
#define DOCTEST_SUBCASE(  
    name)
```

Value:

```
if (const doctest::detail::Subcase &DOCTEST_ANONYMOUS(DOCTEST_ANON_SUBCASE_) DOCTEST_UNUSED =  
    \  
    doctest::detail::Subcase(name, __FILE__, __LINE__))
```

Definition at line 3059 of file [doctest.h](#).

12.64.1.246 DOCTEST_SUPPRESS_COMMON_WARNINGS_POP

```
#define DOCTEST_SUPPRESS_COMMON_WARNINGS_POP
```

Value:

```
DOCTEST_CLANG_SUPPRESS_WARNING_POP  
DOCTEST_GCC_SUPPRESS_WARNING_POP  
DOCTEST_MSVC_SUPPRESS_WARNING_POP
```

Definition at line 223 of file [doctest.h](#).

12.64.1.247 DOCTEST_SUPPRESS_COMMON_WARNINGS_PUSH

```
#define DOCTEST_SUPPRESS_COMMON_WARNINGS_PUSH
```

Definition at line 173 of file [doctest.h](#).

12.64.1.248 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP

```
#define DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP DOCTEST_SUPPRESS_COMMON_WARNINGS_POP
```

Definition at line 285 of file [doctest.h](#).

12.64.1.249 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH

```
#define DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
```

Definition at line 242 of file [doctest.h](#).

12.64.1.250 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_POP

```
#define DOCTEST_SUPPRESS_PUBLIC_WARNINGS_POP DOCTEST_SUPPRESS_COMMON_WARNINGS_POP
```

Definition at line 240 of file [doctest.h](#).

12.64.1.251 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_PUSH

```
#define DOCTEST_SUPPRESS_PUBLIC_WARNINGS_PUSH
```

Value:

```
DOCTEST_SUPPRESS_COMMON_WARNINGS_PUSH
\
DOCTEST_CLANG_SUPPRESS_WARNING("-Wnon-virtual-dtor")
\
DOCTEST_CLANG_SUPPRESS_WARNING("-Wdeprecated")
\
DOCTEST_GCC_SUPPRESS_WARNING("-Wctor-dtor-privacy")
\
DOCTEST_GCC_SUPPRESS_WARNING("-Wnon-virtual-dtor")
\
DOCTEST_GCC_SUPPRESS_WARNING("-Wsign-promo")
\
DOCTEST_MSVC_SUPPRESS_WARNING(4623) /* default constructor was implicitly deleted */
```

Definition at line 228 of file [doctest.h](#).

12.64.1.252 DOCTEST_SYMBOL_EXPORT

```
#define DOCTEST_SYMBOL_EXPORT __attribute__((visibility("default")))
```

Definition at line 415 of file [doctest.h](#).

12.64.1.253 DOCTEST_SYMBOL_IMPORT

```
#define DOCTEST_SYMBOL_IMPORT
```

Definition at line 416 of file [doctest.h](#).

12.64.1.254 DOCTEST_TEST_CASE

```
#define DOCTEST_TEST_CASE(  
    decorators)
```

Value:

```
DOCTEST_CREATE_AND_REGISTER_FUNCTION(DOCTEST_ANONYMOUS(DOCTEST_ANON_FUNC_), decorators)
```

Definition at line 2970 of file [doctest.h](#).

12.64.1.255 DOCTEST_TEST_CASE_CLASS

```
#define DOCTEST_TEST_CASE_CLASS(  
    ...)
```

Value:

```
TEST_CASES_CAN_BE_REGISTERED_IN_CLASSES_ONLY_IN_CPP17_MODE_OR_WITH_VS_2017_OR_NEWER
```

Definition at line 2980 of file [doctest.h](#).

12.64.1.256 DOCTEST_TEST_CASE_FIXTURE

```
#define DOCTEST_TEST_CASE_FIXTURE(  
    c,  
    decorators)
```

Value:

```
DOCTEST_IMPLEMENT_FIXTURE(  
    \ DOCTEST_ANONYMOUS(DOCTEST_ANON_CLASS_), c, DOCTEST_ANONYMOUS(DOCTEST_ANON_FUNC_), decorators  
    )
```

Definition at line 2984 of file [doctest.h](#).

12.64.1.257 DOCTEST_TEST_CASE_TEMPLATE

```
#define DOCTEST_TEST_CASE_TEMPLATE(  
    dec,  
    T,  
    ...)
```

Value:

```
DOCTEST_TEST_CASE_TEMPLATE_IMPL(dec, T, DOCTEST_ANONYMOUS(DOCTEST_ANON_TMP_), __VA_ARGS__)
```

Definition at line 3055 of file [doctest.h](#).

12.64.1.258 DOCTEST_TEST_CASE_TEMPLATE_APPLY

```
#define DOCTEST_TEST_CASE_TEMPLATE_APPLY(  
    id,  
    ...)
```

Value:

```
DOCTEST_TEST_CASE_TEMPLATE_INSTANTIATE_IMPL(id, DOCTEST_ANONYMOUS(DOCTEST_ANON_TMP_), __VA_ARGS__)  
    \ static_assert(true, "")
```

Definition at line 3045 of file [doctest.h](#).

12.64.1.259 DOCTEST_TEST_CASE_TEMPLATE_DEFINE

```
#define DOCTEST_TEST_CASE_TEMPLATE_DEFINE(
    dec,
    T,
    id)
```

Value:

```
DOCTEST_TEST_CASE_TEMPLATE_DEFINE_IMPL(dec, T, DOCTEST_CAT(id, ITERATOR),
    DOCTEST_ANONYMOUS(DOCTEST_ANON_TMP_))
```

Definition at line 3032 of file [doctest.h](#).

12.64.1.260 DOCTEST_TEST_CASE_TEMPLATE_DEFINE_IMPL

```
#define DOCTEST_TEST_CASE_TEMPLATE_DEFINE_IMPL(
    dec,
    T,
    iter,
    func)
```

Value:

```
template <typename T>
\
static void func();
\
namespace { /* NOLINT */
\
template <typename Tuple>
\
struct iter;
\
template <typename Type, typename... Rest>
\
struct iter<std::tuple<Type, Rest...> {
\
    iter(const char *file, unsigned line, int index) noexcept {
\
        doctest::detail::regTest(
\
            doctest::detail::TestCase(
\
                func<Type>,
\
                file,
\
                line,
\
                doctest_detail_test_suite_ns::getCurrentTestSuite(),
\
                doctest::toString<Type>(),
\
                int(line) * 1000 + index
\
            ) *
\
            dec
\
        );
\
        iter<std::tuple<Rest...>(file, line, index + 1);
\
    }
\
};
\
template <>
\
struct iter<std::tuple<> {
\
    iter(const char *, unsigned, int) {}
\
}
```

```
};
\
}
\
template <typename T>
\
static void func()
```

Definition at line 3001 of file [doctest.h](#).

12.64.1.261 DOCTEST_TEST_CASE_TEMPLATE_IMPL

```
#define DOCTEST_TEST_CASE_TEMPLATE_IMPL(
    dec,
    T,
    anon,
    ...)
```

Value:

```
DOCTEST_TEST_CASE_TEMPLATE_DEFINE_IMPL(dec, T, DOCTEST_CAT(anon, ITERATOR), anon);
\
DOCTEST_TEST_CASE_TEMPLATE_INSTANTIATE_IMPL(anon, anon, std::tuple<__VA_ARGS__>)
\
template <typename T>
\
static void anon()
```

Definition at line 3049 of file [doctest.h](#).

12.64.1.262 DOCTEST_TEST_CASE_TEMPLATE_INSTANTIATE

```
#define DOCTEST_TEST_CASE_TEMPLATE_INSTANTIATE(
    id,
    ...)
```

Value:

```
DOCTEST_TEST_CASE_TEMPLATE_INVOKE(id, __VA_ARGS__)
```

Definition at line 3726 of file [doctest.h](#).

12.64.1.263 DOCTEST_TEST_CASE_TEMPLATE_INSTANTIATE_IMPL

```
#define DOCTEST_TEST_CASE_TEMPLATE_INSTANTIATE_IMPL(
    id,
    anon,
    ...)
```

Value:

```
DOCTEST_GLOBAL_NO_WARNINGS(
    \
    DOCTEST_CAT(anon, DUMMY), /* NOLINT(cert-err58-cpp, fuchsia-statically-constructed-objects) */
    \
    doctest::detail::instantiationHelper(DOCTEST_CAT(id, ITERATOR) < __VA_ARGS__ > (__FILE__, __LINE__, 0))
    \
)
```

Definition at line 3035 of file [doctest.h](#).

12.64.1.264 DOCTEST_TEST_CASE_TEMPLATE_INVOKE

```
#define DOCTEST_TEST_CASE_TEMPLATE_INVOKE(
    id,
    ...)
```

Value:

```
DOCTEST_TEST_CASE_TEMPLATE_INSTANTIATE_IMPL(id, DOCTEST_ANONYMOUS(DOCTEST_ANON_TMP_),
    std::tuple<__VA_ARGS__>() \
    static_assert(true, "")
```

Definition at line 3041 of file [doctest.h](#).

12.64.1.265 DOCTEST_TEST_SUITE

```
#define DOCTEST_TEST_SUITE(
    decorators)
```

Value:

```
DOCTEST_TEST_SUITE_IMPL(decorators, DOCTEST_ANONYMOUS(DOCTEST_ANON_SUITE_))
```

Definition at line 3089 of file [doctest.h](#).

12.64.1.266 DOCTEST_TEST_SUITE_BEGIN

```
#define DOCTEST_TEST_SUITE_BEGIN(
    decorators)
```

Value:

```
DOCTEST_GLOBAL_NO_WARNINGS(
    \
    DOCTEST_ANONYMOUS(DOCTEST_ANON_VAR_), /* NOLINT(cert-err58-cpp) */
    \
    doctest::detail::setTestSuite(doctest::detail::TestSuite() * decorators)
    \
)
    \
static_assert(true, "")
```

Definition at line 3092 of file [doctest.h](#).

12.64.1.267 DOCTEST_TEST_SUITE_END

```
#define DOCTEST_TEST_SUITE_END
```

Value:

```
DOCTEST_GLOBAL_NO_WARNINGS(
    \
    DOCTEST_ANONYMOUS(DOCTEST_ANON_VAR_), /* NOLINT(cert-err58-cpp) */
    \
    doctest::detail::setTestSuite(doctest::detail::TestSuite() * "")
    \
)
    \
using DOCTEST_ANONYMOUS(DOCTEST_ANON_FOR_SEMICOLON_) = int
```

Definition at line 3100 of file [doctest.h](#).

12.64.1.268 DOCTEST_TEST_SUITE_IMPL

```
#define DOCTEST_TEST_SUITE_IMPL(
    decorators,
    ns_name)
```

Value:

```
namespace ns_name {
    namespace doctest_detail_test_suite_ns {
        static DOCTEST_NOINLINE doctest::detail::TestSuite &getCurrentTestSuite() noexcept {
            DOCTEST_MSVC_SUPPRESS_WARNING_WITH_PUSH(4640)
            DOCTEST_CLANG_SUPPRESS_WARNING_WITH_PUSH("-Wexit-time-destructors")
            DOCTEST_GCC_SUPPRESS_WARNING_WITH_PUSH("-Wmissing-field-initializers")
            static doctest::detail::TestSuite data{};
            static bool initied = false;
            DOCTEST_MSVC_SUPPRESS_WARNING_POP
            DOCTEST_CLANG_SUPPRESS_WARNING_POP
            DOCTEST_GCC_SUPPRESS_WARNING_POP
            if (!initied) {
                data *decorators;
                initied = true;
            }
            return data;
        }
    }
}
namespace ns_name
```

Definition at line [3067](#) of file [doctest.h](#).

12.64.1.269 DOCTEST_THEN

```
#define DOCTEST_THEN(
    name)
```

Value:

```
DOCTEST_SUBCASE("    Then: " name)
```

Definition at line [3740](#) of file [doctest.h](#).

12.64.1.270 DOCTEST_TO_LVALUE

```
#define DOCTEST_TO_LVALUE(
    ...)
```

Value:

```
__VA_ARGS__
```

Definition at line [3168](#) of file [doctest.h](#).

12.64.1.271 DOCTEST_TOSTR

```
#define DOCTEST_TOSTR(  
    x)
```

Value:

`DOCTEST_TOSTR_IMPL(x)`

Definition at line 60 of file [doctest.h](#).

12.64.1.272 DOCTEST_TOSTR_IMPL

```
#define DOCTEST_TOSTR_IMPL(  
    x)
```

Value:

`#x`

Definition at line 59 of file [doctest.h](#).

12.64.1.273 DOCTEST_TYPE_TO_STRING

```
#define DOCTEST_TYPE_TO_STRING(  
    ...)
```

Value:

`DOCTEST_TYPE_TO_STRING_AS(#__VA_ARGS__, __VA_ARGS__)`

Definition at line 2999 of file [doctest.h](#).

12.64.1.274 DOCTEST_TYPE_TO_STRING_AS

```
#define DOCTEST_TYPE_TO_STRING_AS(  
    str,  
    ...)
```

Value:

```
namespace doctest {  
    \  
    template <>  
    \  
    inline String toString<__VA_ARGS__>() {  
        \  
        return str;  
    }  
    \  
    }  
    \  
    static_assert(true, "")
```

Definition at line 2990 of file [doctest.h](#).

12.64.1.275 DOCTEST_UNARY_ASSERT

```
#define DOCTEST_UNARY_ASSERT(
    assert_type,
    ...)
```

Value:

```
DOCTEST_FUNC_SCOPE_BEGIN {
    \
    doctest::detail::ResultBuilder DOCTEST_RB(doctest::assertType::assert_type, __FILE__, __LINE__,
    #__VA_ARGS__); \
    DOCTEST_WRAP_IN_TRY(DOCTEST_RB.unary_assert(__VA_ARGS__))
    \
    DOCTEST_ASSERT_LOG_REACT_RETURN(DOCTEST_RB);
    \
}
DOCTEST_FUNC_SCOPE_END
```

Definition at line 3196 of file [doctest.h](#).

12.64.1.276 DOCTEST_UNUSED

```
#define DOCTEST_UNUSED __attribute__((unused))
```

Definition at line 451 of file [doctest.h](#).

12.64.1.277 DOCTEST_VERSION

```
#define DOCTEST_VERSION (DOCTEST_VERSION_MAJOR * 10000 + DOCTEST_VERSION_MINOR * 100 + DOCTEST_VERSION_PATCH)
```

Definition at line 69 of file [doctest.h](#).

12.64.1.278 DOCTEST_VERSION_MAJOR

```
#define DOCTEST_VERSION_MAJOR 2
```

Definition at line 53 of file [doctest.h](#).

12.64.1.279 DOCTEST_VERSION_MINOR

```
#define DOCTEST_VERSION_MINOR 5
```

Definition at line 54 of file [doctest.h](#).

12.64.1.280 DOCTEST_VERSION_PATCH

```
#define DOCTEST_VERSION_PATCH 0
```

Definition at line 55 of file [doctest.h](#).

12.64.1.281 DOCTEST_VERSION_STR

```
#define DOCTEST_VERSION_STR
```

Value:

```
DOCTEST_TOSTR(DOCTEST_VERSION_MAJOR) "."  
  \  
DOCTEST_TOSTR(DOCTEST_VERSION_MINOR) "."  
  \  
DOCTEST_TOSTR(DOCTEST_VERSION_PATCH)
```

Definition at line 63 of file [doctest.h](#).

12.64.1.282 DOCTEST_WARN

```
#define DOCTEST_WARN(  
    ...)
```

Value:

```
DOCTEST_ASSERT_IMPLEMENT_1(DT_WARN, __VA_ARGS__)
```

Definition at line 3229 of file [doctest.h](#).

12.64.1.283 DOCTEST_WARN_EQ

```
#define DOCTEST_WARN_EQ(  
    ...)
```

Value:

```
DOCTEST_BINARY_ASSERT(DT_WARN_EQ, eq, __VA_ARGS__)
```

Definition at line 3245 of file [doctest.h](#).

12.64.1.284 DOCTEST_WARN_FALSE

```
#define DOCTEST_WARN_FALSE(  
    ...)
```

Value:

```
DOCTEST_ASSERT_IMPLEMENT_1(DT_WARN_FALSE, __VA_ARGS__)
```

Definition at line 3232 of file [doctest.h](#).

12.64.1.285 DOCTEST_WARN_FALSE_MESSAGE

```
#define DOCTEST_WARN_FALSE_MESSAGE(  
    cond,  
    ...)
```

Value:

```
DOCTEST_FUNC_SCOPE_BEGIN { DOCTEST_INFO(__VA_ARGS__); DOCTEST_ASSERT_IMPLEMENT_2(DT_WARN_FALSE, cond); }  
DOCTEST_FUNC_SCOPE_END
```

Definition at line 3240 of file [doctest.h](#).

12.64.1.286 DOCTEST_WARN_GE

```
#define DOCTEST_WARN_GE(  
    ...)
```

Value:

[DOCTEST_BINARY_ASSERT](#) (DT_WARN_GE, ge, __VA_ARGS__)

Definition at line [3257](#) of file [doctest.h](#).

12.64.1.287 DOCTEST_WARN_GT

```
#define DOCTEST_WARN_GT(  
    ...)
```

Value:

[DOCTEST_BINARY_ASSERT](#) (DT_WARN_GT, gt, __VA_ARGS__)

Definition at line [3251](#) of file [doctest.h](#).

12.64.1.288 DOCTEST_WARN_LE

```
#define DOCTEST_WARN_LE(  
    ...)
```

Value:

[DOCTEST_BINARY_ASSERT](#) (DT_WARN_LE, le, __VA_ARGS__)

Definition at line [3260](#) of file [doctest.h](#).

12.64.1.289 DOCTEST_WARN_LT

```
#define DOCTEST_WARN_LT(  
    ...)
```

Value:

[DOCTEST_BINARY_ASSERT](#) (DT_WARN_LT, lt, __VA_ARGS__)

Definition at line [3254](#) of file [doctest.h](#).

12.64.1.290 DOCTEST_WARN_MESSAGE

```
#define DOCTEST_WARN_MESSAGE(  
    cond,  
    ...)
```

Value:

[DOCTEST_FUNC_SCOPE_BEGIN](#) { [DOCTEST_INFO](#) (__VA_ARGS__); [DOCTEST_ASSERT_IMPLEMENT_2](#) (DT_WARN, cond); }
[DOCTEST_FUNC_SCOPE_END](#)

Definition at line [3237](#) of file [doctest.h](#).

12.64.1.291 DOCTEST_WARN_NE

```
#define DOCTEST_WARN_NE(  
    ...)
```

Value:

[DOCTEST_BINARY_ASSERT](#) (DT_WARN_NE, ne, __VA_ARGS__)

Definition at line 3248 of file [doctest.h](#).

12.64.1.292 DOCTEST_WARN_NOTHROW

```
#define DOCTEST_WARN_NOTHROW(  
    ...)
```

Value:

[DOCTEST_EXCEPTION_EMPTY_FUNC](#)

Definition at line 3677 of file [doctest.h](#).

12.64.1.293 DOCTEST_WARN_NOTHROW_MESSAGE

```
#define DOCTEST_WARN_NOTHROW_MESSAGE(  
    expr,  
    ...)
```

Value:

[DOCTEST_EXCEPTION_EMPTY_FUNC](#)

Definition at line 3693 of file [doctest.h](#).

12.64.1.294 DOCTEST_WARN_THROWS

```
#define DOCTEST_WARN_THROWS(  
    ...)
```

Value:

[DOCTEST_EXCEPTION_EMPTY_FUNC](#)

Definition at line 3665 of file [doctest.h](#).

12.64.1.295 DOCTEST_WARN_THROWS_AS

```
#define DOCTEST_WARN_THROWS_AS(  
    expr,  
    ...)
```

Value:

[DOCTEST_EXCEPTION_EMPTY_FUNC](#)

Definition at line 3668 of file [doctest.h](#).

12.64.1.296 DOCTEST_WARN_THROWS_AS_MESSAGE

```
#define DOCTEST_WARN_THROWS_AS_MESSAGE(  
    expr,  
    ex,  
    ...)
```

Value:

[DOCTEST_EXCEPTION_EMPTY_FUNC](#)

Definition at line [3684](#) of file [doctest.h](#).

12.64.1.297 DOCTEST_WARN_THROWS_MESSAGE

```
#define DOCTEST_WARN_THROWS_MESSAGE(  
    expr,  
    ...)
```

Value:

[DOCTEST_EXCEPTION_EMPTY_FUNC](#)

Definition at line [3681](#) of file [doctest.h](#).

12.64.1.298 DOCTEST_WARN_THROWS_WITH

```
#define DOCTEST_WARN_THROWS_WITH(  
    expr,  
    ...)
```

Value:

[DOCTEST_EXCEPTION_EMPTY_FUNC](#)

Definition at line [3671](#) of file [doctest.h](#).

12.64.1.299 DOCTEST_WARN_THROWS_WITH_AS

```
#define DOCTEST_WARN_THROWS_WITH_AS(  
    expr,  
    with,  
    ...)
```

Value:

[DOCTEST_EXCEPTION_EMPTY_FUNC](#)

Definition at line [3674](#) of file [doctest.h](#).

12.64.1.300 DOCTEST_WARN_THROWS_WITH_AS_MESSAGE

```
#define DOCTEST_WARN_THROWS_WITH_AS_MESSAGE(  
    expr,  
    with,  
    ex,  
    ...)
```

Value:

[DOCTEST_EXCEPTION_EMPTY_FUNC](#)

Definition at line 3690 of file [doctest.h](#).

12.64.1.301 DOCTEST_WARN_THROWS_WITH_MESSAGE

```
#define DOCTEST_WARN_THROWS_WITH_MESSAGE(  
    expr,  
    with,  
    ...)
```

Value:

[DOCTEST_EXCEPTION_EMPTY_FUNC](#)

Definition at line 3687 of file [doctest.h](#).

12.64.1.302 DOCTEST_WARN_UNARY

```
#define DOCTEST_WARN_UNARY(  
    ...)
```

Value:

[DOCTEST_UNARY_ASSERT](#)(DT_WARN_UNARY, __VA_ARGS__)

Definition at line 3264 of file [doctest.h](#).

12.64.1.303 DOCTEST_WARN_UNARY_FALSE

```
#define DOCTEST_WARN_UNARY_FALSE(  
    ...)
```

Value:

[DOCTEST_UNARY_ASSERT](#)(DT_WARN_UNARY_FALSE, __VA_ARGS__)

Definition at line 3267 of file [doctest.h](#).

12.64.1.304 DOCTEST_WHEN

```
#define DOCTEST_WHEN(  
    name)
```

Value:

[DOCTEST_SUBCASE](#)(" When: " *name*)

Definition at line 3738 of file [doctest.h](#).

12.64.1.305 DOCTEST_WRAP_IN_TRY

```
#define DOCTEST_WRAP_IN_TRY(  
    x)
```

Value:

```
x;
```

Definition at line 2917 of file [doctest.h](#).

12.64.1.306 FAIL

```
#define FAIL(  
    ...)
```

Value:

```
DOCTEST_FAIL(__VA_ARGS__)
```

Definition at line 3771 of file [doctest.h](#).

12.64.1.307 FAIL_CHECK

```
#define FAIL_CHECK(  
    ...)
```

Value:

```
DOCTEST_FAIL_CHECK(__VA_ARGS__)
```

Definition at line 3770 of file [doctest.h](#).

12.64.1.308 FAST_CHECK_EQ

```
#define FAST_CHECK_EQ(  
    ...)
```

Value:

```
DOCTEST_FAST_CHECK_EQ(__VA_ARGS__)
```

Definition at line 3859 of file [doctest.h](#).

12.64.1.309 FAST_CHECK_GE

```
#define FAST_CHECK_GE(  
    ...)
```

Value:

```
DOCTEST_FAST_CHECK_GE(__VA_ARGS__)
```

Definition at line 3871 of file [doctest.h](#).

12.64.1.310 FAST_CHECK_GT

```
#define FAST_CHECK_GT(  
    ...)
```

Value:

[DOCTEST_FAST_CHECK_GT](#) (__VA_ARGS__)

Definition at line [3865](#) of file [doctest.h](#).

12.64.1.311 FAST_CHECK_LE

```
#define FAST_CHECK_LE(  
    ...)
```

Value:

[DOCTEST_FAST_CHECK_LE](#) (__VA_ARGS__)

Definition at line [3874](#) of file [doctest.h](#).

12.64.1.312 FAST_CHECK_LT

```
#define FAST_CHECK_LT(  
    ...)
```

Value:

[DOCTEST_FAST_CHECK_LT](#) (__VA_ARGS__)

Definition at line [3868](#) of file [doctest.h](#).

12.64.1.313 FAST_CHECK_NE

```
#define FAST_CHECK_NE(  
    ...)
```

Value:

[DOCTEST_FAST_CHECK_NE](#) (__VA_ARGS__)

Definition at line [3862](#) of file [doctest.h](#).

12.64.1.314 FAST_CHECK_UNARY

```
#define FAST_CHECK_UNARY(  
    ...)
```

Value:

[DOCTEST_FAST_CHECK_UNARY](#) (__VA_ARGS__)

Definition at line [3878](#) of file [doctest.h](#).

12.64.1.315 FAST_CHECK_UNARY_FALSE

```
#define FAST_CHECK_UNARY_FALSE(  
    ...)
```

Value:

[DOCTEST_FAST_CHECK_UNARY_FALSE](#)(__VA_ARGS__)

Definition at line [3881](#) of file [doctest.h](#).

12.64.1.316 FAST_REQUIRE_EQ

```
#define FAST_REQUIRE_EQ(  
    ...)
```

Value:

[DOCTEST_FAST_REQUIRE_EQ](#)(__VA_ARGS__)

Definition at line [3860](#) of file [doctest.h](#).

12.64.1.317 FAST_REQUIRE_GE

```
#define FAST_REQUIRE_GE(  
    ...)
```

Value:

[DOCTEST_FAST_REQUIRE_GE](#)(__VA_ARGS__)

Definition at line [3872](#) of file [doctest.h](#).

12.64.1.318 FAST_REQUIRE_GT

```
#define FAST_REQUIRE_GT(  
    ...)
```

Value:

[DOCTEST_FAST_REQUIRE_GT](#)(__VA_ARGS__)

Definition at line [3866](#) of file [doctest.h](#).

12.64.1.319 FAST_REQUIRE_LE

```
#define FAST_REQUIRE_LE(  
    ...)
```

Value:

[DOCTEST_FAST_REQUIRE_LE](#)(__VA_ARGS__)

Definition at line [3875](#) of file [doctest.h](#).

12.64.1.320 FAST_REQUIRE_LT

```
#define FAST_REQUIRE_LT(  
    ...)
```

Value:

[DOCTEST_FAST_REQUIRE_LT](#)(__VA_ARGS__)

Definition at line [3869](#) of file [doctest.h](#).

12.64.1.321 FAST_REQUIRE_NE

```
#define FAST_REQUIRE_NE(  
    ...)
```

Value:

[DOCTEST_FAST_REQUIRE_NE](#)(__VA_ARGS__)

Definition at line [3863](#) of file [doctest.h](#).

12.64.1.322 FAST_REQUIRE_UNARY

```
#define FAST_REQUIRE_UNARY(  
    ...)
```

Value:

[DOCTEST_FAST_REQUIRE_UNARY](#)(__VA_ARGS__)

Definition at line [3879](#) of file [doctest.h](#).

12.64.1.323 FAST_REQUIRE_UNARY_FALSE

```
#define FAST_REQUIRE_UNARY_FALSE(  
    ...)
```

Value:

[DOCTEST_FAST_REQUIRE_UNARY_FALSE](#)(__VA_ARGS__)

Definition at line [3882](#) of file [doctest.h](#).

12.64.1.324 FAST_WARN_EQ

```
#define FAST_WARN_EQ(  
    ...)
```

Value:

[DOCTEST_FAST_WARN_EQ](#)(__VA_ARGS__)

Definition at line [3858](#) of file [doctest.h](#).

12.64.1.325 FAST_WARN_GE

```
#define FAST_WARN_GE(  
    ...)
```

Value:

[DOCTEST_FAST_WARN_GE](#)(__VA_ARGS__)

Definition at line [3870](#) of file [doctest.h](#).

12.64.1.326 FAST_WARN_GT

```
#define FAST_WARN_GT(  
    ...)
```

Value:

[DOCTEST_FAST_WARN_GT](#)(__VA_ARGS__)

Definition at line [3864](#) of file [doctest.h](#).

12.64.1.327 FAST_WARN_LE

```
#define FAST_WARN_LE(  
    ...)
```

Value:

[DOCTEST_FAST_WARN_LE](#)(__VA_ARGS__)

Definition at line [3873](#) of file [doctest.h](#).

12.64.1.328 FAST_WARN_LT

```
#define FAST_WARN_LT(  
    ...)
```

Value:

[DOCTEST_FAST_WARN_LT](#)(__VA_ARGS__)

Definition at line [3867](#) of file [doctest.h](#).

12.64.1.329 FAST_WARN_NE

```
#define FAST_WARN_NE(  
    ...)
```

Value:

[DOCTEST_FAST_WARN_NE](#)(__VA_ARGS__)

Definition at line [3861](#) of file [doctest.h](#).

12.64.1.330 FAST_WARN_UNARY

```
#define FAST_WARN_UNARY(  
    ...)
```

Value:

[DOCTEST_FAST_WARN_UNARY](#)(__VA_ARGS__)

Definition at line [3877](#) of file [doctest.h](#).

12.64.1.331 FAST_WARN_UNARY_FALSE

```
#define FAST_WARN_UNARY_FALSE(  
    ...)
```

Value:

[DOCTEST_FAST_WARN_UNARY_FALSE](#)(__VA_ARGS__)

Definition at line [3880](#) of file [doctest.h](#).

12.64.1.332 GENERATE

```
#define GENERATE(  
    ...)
```

Value:

[DOCTEST_GENERATE](#)(__VA_ARGS__)

Definition at line [3757](#) of file [doctest.h](#).

12.64.1.333 GIVEN

```
#define GIVEN(  
    name)
```

Value:

[DOCTEST_GIVEN](#)(*name*)

Definition at line [3825](#) of file [doctest.h](#).

12.64.1.334 INFO

```
#define INFO(  
    ...)
```

Value:

[DOCTEST_INFO](#)(__VA_ARGS__)

Definition at line [3764](#) of file [doctest.h](#).

12.64.1.335 MESSAGE

```
#define MESSAGE(  
    ...)
```

Value:

`DOCTEST_MESSAGE` (__VA_ARGS__)

Definition at line 3769 of file [doctest.h](#).

12.64.1.336 REGISTER_EXCEPTION_TRANSLATOR

```
#define REGISTER_EXCEPTION_TRANSLATOR(  
    signature)
```

Value:

`DOCTEST_REGISTER_EXCEPTION_TRANSLATOR`(signature)

Definition at line 3761 of file [doctest.h](#).

12.64.1.337 REGISTER_LISTENER

```
#define REGISTER_LISTENER(  
    name,  
    priority,  
    reporter)
```

Value:

`DOCTEST_REGISTER_LISTENER`(name, priority, reporter)

Definition at line 3763 of file [doctest.h](#).

12.64.1.338 REGISTER_REPORTER

```
#define REGISTER_REPORTER(  
    name,  
    priority,  
    reporter)
```

Value:

`DOCTEST_REGISTER_REPORTER`(name, priority, reporter)

Definition at line 3762 of file [doctest.h](#).

12.64.1.339 REQUIRE

```
#define REQUIRE(  
    ...)
```

Value:

`DOCTEST_REQUIRE` (__VA_ARGS__)

Definition at line 3788 of file [doctest.h](#).

12.64.1.340 REQUIRE_EQ

```
#define REQUIRE_EQ(  
    ...)
```

Value:

[DOCTEST_REQUIRE_EQ](#)(__VA_ARGS__)

Definition at line [3834](#) of file [doctest.h](#).

12.64.1.341 REQUIRE_FALSE

```
#define REQUIRE_FALSE(  
    ...)
```

Value:

[DOCTEST_REQUIRE_FALSE](#)(__VA_ARGS__)

Definition at line [3789](#) of file [doctest.h](#).

12.64.1.342 REQUIRE_FALSE_MESSAGE

```
#define REQUIRE_FALSE_MESSAGE(  
    cond,  
    ...)
```

Value:

[DOCTEST_REQUIRE_FALSE_MESSAGE](#)(cond, __VA_ARGS__)

Definition at line [3812](#) of file [doctest.h](#).

12.64.1.343 REQUIRE_GE

```
#define REQUIRE_GE(  
    ...)
```

Value:

[DOCTEST_REQUIRE_GE](#)(__VA_ARGS__)

Definition at line [3846](#) of file [doctest.h](#).

12.64.1.344 REQUIRE_GT

```
#define REQUIRE_GT(  
    ...)
```

Value:

[DOCTEST_REQUIRE_GT](#)(__VA_ARGS__)

Definition at line [3840](#) of file [doctest.h](#).

12.64.1.345 REQUIRE_LE

```
#define REQUIRE_LE(  
    ...)
```

Value:

[DOCTEST_REQUIRE_LE](#)(__VA_ARGS__)

Definition at line [3849](#) of file [doctest.h](#).

12.64.1.346 REQUIRE_LT

```
#define REQUIRE_LT(  
    ...)
```

Value:

[DOCTEST_REQUIRE_LT](#)(__VA_ARGS__)

Definition at line [3843](#) of file [doctest.h](#).

12.64.1.347 REQUIRE_MESSAGE

```
#define REQUIRE_MESSAGE(  
    cond,  
    ...)
```

Value:

[DOCTEST_REQUIRE_MESSAGE](#)(cond, __VA_ARGS__)

Definition at line [3811](#) of file [doctest.h](#).

12.64.1.348 REQUIRE_NE

```
#define REQUIRE_NE(  
    ...)
```

Value:

[DOCTEST_REQUIRE_NE](#)(__VA_ARGS__)

Definition at line [3837](#) of file [doctest.h](#).

12.64.1.349 REQUIRE_NOTHROW

```
#define REQUIRE_NOTHROW(  
    ...)
```

Value:

[DOCTEST_REQUIRE_NOTHROW](#)(__VA_ARGS__)

Definition at line [3794](#) of file [doctest.h](#).

12.64.1.350 REQUIRE_NOTHROW_MESSAGE

```
#define REQUIRE_NOTHROW_MESSAGE(  
    expr,  
    ...)
```

Value:

`DOCTEST_REQUIRE_NOTHROW_MESSAGE(expr, __VA_ARGS__)`

Definition at line 3817 of file [doctest.h](#).

12.64.1.351 REQUIRE_THROWS

```
#define REQUIRE_THROWS(  
    ...)
```

Value:

`DOCTEST_REQUIRE_THROWS(__VA_ARGS__)`

Definition at line 3790 of file [doctest.h](#).

12.64.1.352 REQUIRE_THROWS_AS

```
#define REQUIRE_THROWS_AS(  
    expr,  
    ...)
```

Value:

`DOCTEST_REQUIRE_THROWS_AS(expr, __VA_ARGS__)`

Definition at line 3791 of file [doctest.h](#).

12.64.1.353 REQUIRE_THROWS_AS_MESSAGE

```
#define REQUIRE_THROWS_AS_MESSAGE(  
    expr,  
    ex,  
    ...)
```

Value:

`DOCTEST_REQUIRE_THROWS_AS_MESSAGE(expr, ex, __VA_ARGS__)`

Definition at line 3814 of file [doctest.h](#).

12.64.1.354 REQUIRE_THROWS_MESSAGE

```
#define REQUIRE_THROWS_MESSAGE(  
    expr,  
    ...)
```

Value:

`DOCTEST_REQUIRE_THROWS_MESSAGE(expr, __VA_ARGS__)`

Definition at line 3813 of file [doctest.h](#).

12.64.1.355 REQUIRE_THROWS_WITH

```
#define REQUIRE_THROWS_WITH(  
    expr,  
    ...)
```

Value:

`DOCTEST_REQUIRE_THROWS_WITH(expr, __VA_ARGS__)`

Definition at line [3792](#) of file [doctest.h](#).

12.64.1.356 REQUIRE_THROWS_WITH_AS

```
#define REQUIRE_THROWS_WITH_AS(  
    expr,  
    with,  
    ...)
```

Value:

`DOCTEST_REQUIRE_THROWS_WITH_AS(expr, with, __VA_ARGS__)`

Definition at line [3793](#) of file [doctest.h](#).

12.64.1.357 REQUIRE_THROWS_WITH_AS_MESSAGE

```
#define REQUIRE_THROWS_WITH_AS_MESSAGE(  
    expr,  
    with,  
    ex,  
    ...)
```

Value:

`DOCTEST_REQUIRE_THROWS_WITH_AS_MESSAGE(expr, with, ex, __VA_ARGS__)`

Definition at line [3816](#) of file [doctest.h](#).

12.64.1.358 REQUIRE_THROWS_WITH_MESSAGE

```
#define REQUIRE_THROWS_WITH_MESSAGE(  
    expr,  
    with,  
    ...)
```

Value:

`DOCTEST_REQUIRE_THROWS_WITH_MESSAGE(expr, with, __VA_ARGS__)`

Definition at line [3815](#) of file [doctest.h](#).

12.64.1.359 REQUIRE_UNARY

```
#define REQUIRE_UNARY(  
    ...)
```

Value:

`DOCTEST_REQUIRE_UNARY(__VA_ARGS__)`

Definition at line 3852 of file [doctest.h](#).

12.64.1.360 REQUIRE_UNARY_FALSE

```
#define REQUIRE_UNARY_FALSE(  
    ...)
```

Value:

`DOCTEST_REQUIRE_UNARY_FALSE(__VA_ARGS__)`

Definition at line 3855 of file [doctest.h](#).

12.64.1.361 SCENARIO

```
#define SCENARIO(  
    name)
```

Value:

`DOCTEST_SCENARIO(name)`

Definition at line 3820 of file [doctest.h](#).

12.64.1.362 SCENARIO_CLASS

```
#define SCENARIO_CLASS(  
    name)
```

Value:

`DOCTEST_SCENARIO_CLASS(name)`

Definition at line 3822 of file [doctest.h](#).

12.64.1.363 SCENARIO_METHOD

```
#define SCENARIO_METHOD(  
    x,  
    name)
```

Value:

`DOCTEST_SCENARIO_METHOD(x, name)`

Definition at line 3821 of file [doctest.h](#).

12.64.1.364 SCENARIO_TEMPLATE

```
#define SCENARIO_TEMPLATE(  
    name,  
    T,  
    ...)
```

Value:

`DOCTEST_SCENARIO_TEMPLATE`(*name*, *T*, __VA_ARGS__)

Definition at line 3823 of file [doctest.h](#).

12.64.1.365 SCENARIO_TEMPLATE_DEFINE

```
#define SCENARIO_TEMPLATE_DEFINE(  
    name,  
    T,  
    id)
```

Value:

`DOCTEST_SCENARIO_TEMPLATE_DEFINE`(*name*, *T*, *id*)

Definition at line 3824 of file [doctest.h](#).

12.64.1.366 SFINAE_OP

```
#define SFINAE_OP(  
    ret,  
    op)
```

Value:

`decltype`((void) (`doctest::detail::declval`<*L*>()) *op* `doctest::detail::declval`<*R*>()), `ret`{})

Definition at line 1928 of file [doctest.h](#).

12.64.1.367 SUBCASE

```
#define SUBCASE(  
    name)
```

Value:

`DOCTEST_SUBCASE`(*name*)

Definition at line 3756 of file [doctest.h](#).

12.64.1.368 TEST_CASE

```
#define TEST_CASE(  
    name)
```

Value:

`DOCTEST_TEST_CASE`(*name*)

Definition at line 3747 of file [doctest.h](#).

12.64.1.369 TEST_CASE_CLASS

```
#define TEST_CASE_CLASS(  
    name)
```

Value:

[DOCTEST_TEST_CASE_CLASS](#)(*name*)

Definition at line 3748 of file [doctest.h](#).

12.64.1.370 TEST_CASE_FIXTURE

```
#define TEST_CASE_FIXTURE(  
    x,  
    name)
```

Value:

[DOCTEST_TEST_CASE_FIXTURE](#)(*x*, *name*)

Definition at line 3749 of file [doctest.h](#).

12.64.1.371 TEST_CASE_TEMPLATE

```
#define TEST_CASE_TEMPLATE(  
    name,  
    T,  
    ...)
```

Value:

[DOCTEST_TEST_CASE_TEMPLATE](#)(*name*, *T*, __VA_ARGS__)

Definition at line 3752 of file [doctest.h](#).

12.64.1.372 TEST_CASE_TEMPLATE_APPLY

```
#define TEST_CASE_TEMPLATE_APPLY(  
    id,  
    ...)
```

Value:

[DOCTEST_TEST_CASE_TEMPLATE_APPLY](#)(*id*, __VA_ARGS__)

Definition at line 3755 of file [doctest.h](#).

12.64.1.373 TEST_CASE_TEMPLATE_DEFINE

```
#define TEST_CASE_TEMPLATE_DEFINE(  
    name,  
    T,  
    id)
```

Value:

[DOCTEST_TEST_CASE_TEMPLATE_DEFINE](#)(*name*, *T*, *id*)

Definition at line 3753 of file [doctest.h](#).

12.64.1.374 TEST_CASE_TEMPLATE_INSTANTIATE

```
#define TEST_CASE_TEMPLATE_INSTANTIATE(  
    id,  
    ...)
```

Value:

[DOCTEST_TEST_CASE_TEMPLATE_INSTANTIATE](#)(*id*, __VA_ARGS__)

Definition at line [3884](#) of file [doctest.h](#).

12.64.1.375 TEST_CASE_TEMPLATE_INVOKE

```
#define TEST_CASE_TEMPLATE_INVOKE(  
    id,  
    ...)
```

Value:

[DOCTEST_TEST_CASE_TEMPLATE_INVOKE](#)(*id*, __VA_ARGS__)

Definition at line [3754](#) of file [doctest.h](#).

12.64.1.376 TEST_SUITE

```
#define TEST_SUITE(  
    decorators)
```

Value:

[DOCTEST_TEST_SUITE](#)(*decorators*)

Definition at line [3758](#) of file [doctest.h](#).

12.64.1.377 TEST_SUITE_BEGIN

```
#define TEST_SUITE_BEGIN(  
    name)
```

Value:

[DOCTEST_TEST_SUITE_BEGIN](#)(*name*)

Definition at line [3759](#) of file [doctest.h](#).

12.64.1.378 TEST_SUITE_END

```
#define TEST_SUITE_END DOCTEST\_TEST\_SUITE\_END
```

Definition at line [3760](#) of file [doctest.h](#).

12.64.1.379 THEN

```
#define THEN(  
    name)
```

Value:

[DOCTEST_THEN](#)(*name*)

Definition at line [3829](#) of file [doctest.h](#).

12.64.1.380 TO_LVALUE

```
#define TO_LVALUE(  
    ...)
```

Value:

[DOCTEST_TO_LVALUE](#)(__VA_ARGS__)

Definition at line [3772](#) of file [doctest.h](#).

12.64.1.381 TYPE_TO_STRING

```
#define TYPE_TO_STRING(  
    ...)
```

Value:

[DOCTEST_TYPE_TO_STRING](#)(__VA_ARGS__)

Definition at line [3751](#) of file [doctest.h](#).

12.64.1.382 TYPE_TO_STRING_AS

```
#define TYPE_TO_STRING_AS(  
    str,  
    ...)
```

Value:

[DOCTEST_TYPE_TO_STRING_AS](#)(*str*, __VA_ARGS__)

Definition at line [3750](#) of file [doctest.h](#).

12.64.1.383 WARN

```
#define WARN(  
    ...)
```

Value:

[DOCTEST_WARN](#)(__VA_ARGS__)

Definition at line [3774](#) of file [doctest.h](#).

12.64.1.384 WARN_EQ

```
#define WARN_EQ(  
    ...)
```

Value:

[DOCTEST_WARN_EQ](#)(__VA_ARGS__)

Definition at line [3832](#) of file [doctest.h](#).

12.64.1.385 WARN_FALSE

```
#define WARN_FALSE(  
    ...)
```

Value:

[DOCTEST_WARN_FALSE](#)(__VA_ARGS__)

Definition at line [3775](#) of file [doctest.h](#).

12.64.1.386 WARN_FALSE_MESSAGE

```
#define WARN_FALSE_MESSAGE(  
    cond,  
    ...)
```

Value:

[DOCTEST_WARN_FALSE_MESSAGE](#)(cond, __VA_ARGS__)

Definition at line [3798](#) of file [doctest.h](#).

12.64.1.387 WARN_GE

```
#define WARN_GE(  
    ...)
```

Value:

[DOCTEST_WARN_GE](#)(__VA_ARGS__)

Definition at line [3844](#) of file [doctest.h](#).

12.64.1.388 WARN_GT

```
#define WARN_GT(  
    ...)
```

Value:

[DOCTEST_WARN_GT](#)(__VA_ARGS__)

Definition at line [3838](#) of file [doctest.h](#).

12.64.1.389 WARN_LE

```
#define WARN_LE(  
    ...)
```

Value:

[DOCTEST_WARN_LE](#)(__VA_ARGS__)

Definition at line [3847](#) of file [doctest.h](#).

12.64.1.390 WARN_LT

```
#define WARN_LT(  
    ...)
```

Value:

[DOCTEST_WARN_LT](#)(__VA_ARGS__)

Definition at line [3841](#) of file [doctest.h](#).

12.64.1.391 WARN_MESSAGE

```
#define WARN_MESSAGE(  
    cond,  
    ...)
```

Value:

[DOCTEST_WARN_MESSAGE](#)(cond, __VA_ARGS__)

Definition at line [3797](#) of file [doctest.h](#).

12.64.1.392 WARN_NE

```
#define WARN_NE(  
    ...)
```

Value:

[DOCTEST_WARN_NE](#)(__VA_ARGS__)

Definition at line [3835](#) of file [doctest.h](#).

12.64.1.393 WARN_NOTHROW

```
#define WARN_NOTHROW(  
    ...)
```

Value:

[DOCTEST_WARN_NOTHROW](#)(__VA_ARGS__)

Definition at line [3780](#) of file [doctest.h](#).

12.64.1.394 WARN_NOTHROW_MESSAGE

```
#define WARN_NOTHROW_MESSAGE(  
    expr,  
    ...)
```

Value:

[DOCTEST_WARN_NOTHROW_MESSAGE](#)(*expr*, __VA_ARGS__)

Definition at line 3803 of file [doctest.h](#).

12.64.1.395 WARN_THROWS

```
#define WARN_THROWS(  
    ...)
```

Value:

[DOCTEST_WARN_THROWS](#)(__VA_ARGS__)

Definition at line 3776 of file [doctest.h](#).

12.64.1.396 WARN_THROWS_AS

```
#define WARN_THROWS_AS(  
    expr,  
    ...)
```

Value:

[DOCTEST_WARN_THROWS_AS](#)(*expr*, __VA_ARGS__)

Definition at line 3777 of file [doctest.h](#).

12.64.1.397 WARN_THROWS_AS_MESSAGE

```
#define WARN_THROWS_AS_MESSAGE(  
    expr,  
    ex,  
    ...)
```

Value:

[DOCTEST_WARN_THROWS_AS_MESSAGE](#)(*expr*, *ex*, __VA_ARGS__)

Definition at line 3800 of file [doctest.h](#).

12.64.1.398 WARN_THROWS_MESSAGE

```
#define WARN_THROWS_MESSAGE(  
    expr,  
    ...)
```

Value:

[DOCTEST_WARN_THROWS_MESSAGE](#)(*expr*, __VA_ARGS__)

Definition at line 3799 of file [doctest.h](#).

12.64.1.399 WARN_THROWS_WITH

```
#define WARN_THROWS_WITH(  
    expr,  
    ...)
```

Value:

[DOCTEST_WARN_THROWS_WITH](#)(*expr*, __VA_ARGS__)

Definition at line [3778](#) of file [doctest.h](#).

12.64.1.400 WARN_THROWS_WITH_AS

```
#define WARN_THROWS_WITH_AS(  
    expr,  
    with,  
    ...)
```

Value:

[DOCTEST_WARN_THROWS_WITH_AS](#)(*expr*, *with*, __VA_ARGS__)

Definition at line [3779](#) of file [doctest.h](#).

12.64.1.401 WARN_THROWS_WITH_AS_MESSAGE

```
#define WARN_THROWS_WITH_AS_MESSAGE(  
    expr,  
    with,  
    ex,  
    ...)
```

Value:

[DOCTEST_WARN_THROWS_WITH_AS_MESSAGE](#)(*expr*, *with*, *ex*, __VA_ARGS__)

Definition at line [3802](#) of file [doctest.h](#).

12.64.1.402 WARN_THROWS_WITH_MESSAGE

```
#define WARN_THROWS_WITH_MESSAGE(  
    expr,  
    with,  
    ...)
```

Value:

[DOCTEST_WARN_THROWS_WITH_MESSAGE](#)(*expr*, *with*, __VA_ARGS__)

Definition at line [3801](#) of file [doctest.h](#).

12.64.1.403 WARN_UNARY

```
#define WARN_UNARY(
    ...)
```

Value:

```
DOCTEST_WARN_UNARY(__VA_ARGS__)
```

Definition at line [3850](#) of file [doctest.h](#).

12.64.1.404 WARN_UNARY_FALSE

```
#define WARN_UNARY_FALSE(
    ...)
```

Value:

```
DOCTEST_WARN_UNARY_FALSE(__VA_ARGS__)
```

Definition at line [3853](#) of file [doctest.h](#).

12.64.1.405 WHEN

```
#define WHEN(
    name)
```

Value:

```
DOCTEST_WHEN(name)
```

Definition at line [3827](#) of file [doctest.h](#).

12.65 doctest.h

[Go to the documentation of this file.](#)

```
00001 // =====
00002 // == DO NOT MODIFY THIS FILE BY HAND - IT IS AUTO GENERATED! ==
00003 // =====
00004 //
00005 // doctest.h - the lightest feature-rich C++ single-header testing framework for unit tests and TDD
00006 //
00007 // Copyright (c) 2016-2023 Viktor Kirilov
00008 //
00009 // Distributed under the MIT Software License
00010 // See accompanying file LICENSE.txt or copy at
00011 // https://opensource.org/licenses/MIT
00012 //
00013 // The documentation can be found at the library's page:
00014 // https://github.com/doctest/doctest/blob/master/doc/markdown/readme.md
00015 //
00016 // =====
00017 // =====
00018 // =====
00019 //
00020 // The library is heavily influenced by Catch - https://github.com/catchorg/Catch2
00021 // which uses the Boost Software License - Version 1.0
00022 // see here - https://github.com/catchorg/Catch2/blob/master/LICENSE.txt
00023 //
00024 // The concept of subcases (sections in Catch) and expression decomposition are from there.
00025 // Some parts of the code are taken directly:
00026 // - stringification - the detection of "ostream& operator<<(ostream&, const T&)" and StringMaker<>
00027 // - the Approx() helper class for floating point comparison
00028 // - colors in the console
```



```

00029 // - breaking into a debugger
00030 // - signal / SEH handling
00031 // - timer
00032 // - XmlWriter class - thanks to Phil Nash for allowing the direct reuse (AKA copy/paste)
00033 //
00034 // The expression decomposing templates are taken from lest - https://github.com/martinmoene/lest
00035 // which uses the Boost Software License - Version 1.0
00036 // see here - https://github.com/martinmoene/lest/blob/master/LICENSE.txt
00037 //
00038 // =====
00039 // =====
00040 // =====
00041
00042 #ifndef DOCTEST_LIBRARY_INCLUDED
00043 #define DOCTEST_LIBRARY_INCLUDED
00044
00045 // =====
00046 // == VERSION =====
00047 // =====
00048
00049 #ifndef DOCTEST_PARTS_PUBLIC_VERSION
00050 #define DOCTEST_PARTS_PUBLIC_VERSION
00051
00052 // NOLINTBEGIN(cppcoreguidelines-macro-to-enum, modernize-macro-to-enum)
00053 #define DOCTEST_VERSION_MAJOR 2
00054 #define DOCTEST_VERSION_MINOR 5
00055 #define DOCTEST_VERSION_PATCH 0
00056 // NOLINTEND(cppcoreguidelines-macro-to-enum, modernize-macro-to-enum)
00057
00058 // util we need here
00059 #define DOCTEST_TOSTR_IMPL(x) #x
00060 #define DOCTEST_TOSTR(x) DOCTEST_TOSTR_IMPL(x)
00061
00062 // clang-format off
00063 #define DOCTEST_VERSION_STR
00064 \
00065 \
00066 \
00067 // clang-format on
00068
00069 #define DOCTEST_VERSION (DOCTEST_VERSION_MAJOR * 10000 + DOCTEST_VERSION_MINOR * 100 +
DOCTEST_VERSION_PATCH)
00070
00071 #endif // DOCTEST_PARTS_PUBLIC_VERSION
00072 // =====
00073 // == COMPILER VERSION =====
00074 // =====
00075
00076 #ifndef DOCTEST_PARTS_PUBLIC_COMPILER
00077 #define DOCTEST_PARTS_PUBLIC_COMPILER
00078
00079 // ideas for the version stuff are taken from here: https://github.com/cxxstuff/cxx_detect
00080
00081 #ifdef _MSC_VER
00082 #define DOCTEST_CPLUSPLUS _MSVC_LANG
00083 #else
00084 #define DOCTEST_CPLUSPLUS __cplusplus
00085 #endif
00086
00087 #define DOCTEST_COMPILER(MAJOR, MINOR, PATCH) ((MAJOR) * 10000000 + (MINOR) * 100000 + (PATCH))
00088
00089 // GCC/Clang and GCC/MSVC are mutually exclusive, but Clang/MSVC are not because of clang-cl...
00090 #if defined(_MSC_VER) && defined(_MSC_FULL_VER)
00091 #if _MSC_VER == _MSC_FULL_VER / 10000
00092 #define DOCTEST_MSVC DOCTEST_COMPILER(_MSC_VER / 100, _MSC_VER % 100, _MSC_FULL_VER % 10000)
00093 #else // MSVC
00094 #define DOCTEST_MSVC DOCTEST_COMPILER(_MSC_VER / 100, (_MSC_FULL_VER / 100000) % 100, _MSC_FULL_VER %
100000)
00095 #endif // MSVC
00096 #endif // MSVC
00097 #if defined(__clang__) && defined(__clang_minor__) && defined(__clang_patchlevel__)
00098 #define DOCTEST_CLANG DOCTEST_COMPILER(__clang_major__, __clang_minor__, __clang_patchlevel__)
00099 #elif defined(__GNUC__) && defined(__GNUC_MINOR__) && defined(__GNUC_PATCHLEVEL__) &&
!defined(__INTEL_COMPILER)
00100 #define DOCTEST_GCC DOCTEST_COMPILER(__GNUC__, __GNUC_MINOR__, __GNUC_PATCHLEVEL__)
00101 #endif // GCC
00102 #if defined(__INTEL_COMPILER)
00103 #define DOCTEST_ICC DOCTEST_COMPILER(__INTEL_COMPILER / 100, __INTEL_COMPILER % 100, 0)
00104 #endif // ICC
00105
00106 #ifndef DOCTEST_MSVC
00107 #define DOCTEST_MSVC 0
00108 #endif // DOCTEST_MSVC
00109 #ifndef DOCTEST_CLANG

```

```

00110 #define DOCTEST_CLANG 0
00111 #endif // DOCTEST_CLANG
00112 #ifndef DOCTEST_GCC
00113 #define DOCTEST_GCC 0
00114 #endif // DOCTEST_GCC
00115 #ifndef DOCTEST_ICC
00116 #define DOCTEST_ICC 0
00117 #endif // DOCTEST_ICC
00118
00119 #endif // DOCTEST_PARTS_PUBLIC_COMPILER
00120 // =====
00121 // == COMPILER WARNINGS HELPERS ==
00122 // =====
00123
00124 #ifndef DOCTEST_PARTS_PUBLIC_WARNINGS
00125 #define DOCTEST_PARTS_PUBLIC_WARNINGS
00126
00127
00128 #if DOCTEST_CLANG && !DOCTEST_ICC
00129 #define DOCTEST_PRAGMA_TO_STR(x) _Pragma(#x)
00130 #define DOCTEST_CLANG_SUPPRESS_WARNING_PUSH _Pragma("clang diagnostic push")
00131 #define DOCTEST_CLANG_SUPPRESS_WARNING(w) DOCTEST_PRAGMA_TO_STR(clang diagnostic ignored w)
00132 #define DOCTEST_CLANG_SUPPRESS_WARNING_POP _Pragma("clang diagnostic pop")
00133 #define DOCTEST_CLANG_SUPPRESS_WARNING_WITH_PUSH(w)
    \
00134     DOCTEST_CLANG_SUPPRESS_WARNING_PUSH DOCTEST_CLANG_SUPPRESS_WARNING(w)
00135 #else // DOCTEST_CLANG
00136 #define DOCTEST_CLANG_SUPPRESS_WARNING_PUSH
00137 #define DOCTEST_CLANG_SUPPRESS_WARNING(w)
00138 #define DOCTEST_CLANG_SUPPRESS_WARNING_POP
00139 #define DOCTEST_CLANG_SUPPRESS_WARNING_WITH_PUSH(w)
00140 #endif // DOCTEST_CLANG
00141
00142 #if DOCTEST_GCC
00143 #define DOCTEST_PRAGMA_TO_STR(x) _Pragma(#x)
00144 #define DOCTEST_GCC_SUPPRESS_WARNING_PUSH _Pragma("GCC diagnostic push")
00145 #define DOCTEST_GCC_SUPPRESS_WARNING(w) DOCTEST_PRAGMA_TO_STR(GCC diagnostic ignored w)
00146 #define DOCTEST_GCC_SUPPRESS_WARNING_POP _Pragma("GCC diagnostic pop")
00147 #define DOCTEST_GCC_SUPPRESS_WARNING_WITH_PUSH(w) DOCTEST_GCC_SUPPRESS_WARNING_PUSH
    DOCTEST_GCC_SUPPRESS_WARNING(w)
00148 #else // DOCTEST_GCC
00149 #define DOCTEST_GCC_SUPPRESS_WARNING_PUSH
00150 #define DOCTEST_GCC_SUPPRESS_WARNING(w)
00151 #define DOCTEST_GCC_SUPPRESS_WARNING_POP
00152 #define DOCTEST_GCC_SUPPRESS_WARNING_WITH_PUSH(w)
00153 #endif // DOCTEST_GCC
00154
00155 #if DOCTEST_MSVC
00156 #define DOCTEST_MSVC_SUPPRESS_WARNING_PUSH __pragma(warning(push))
00157 #define DOCTEST_MSVC_SUPPRESS_WARNING(w) __pragma(warning(disable : w))
00158 #define DOCTEST_MSVC_SUPPRESS_WARNING_POP __pragma(warning(pop))
00159 #define DOCTEST_MSVC_SUPPRESS_WARNING_WITH_PUSH(w) DOCTEST_MSVC_SUPPRESS_WARNING_PUSH
    DOCTEST_MSVC_SUPPRESS_WARNING(w)
00160 #else // DOCTEST_MSVC
00161 #define DOCTEST_MSVC_SUPPRESS_WARNING_PUSH
00162 #define DOCTEST_MSVC_SUPPRESS_WARNING(w)
00163 #define DOCTEST_MSVC_SUPPRESS_WARNING_POP
00164 #define DOCTEST_MSVC_SUPPRESS_WARNING_WITH_PUSH(w)
00165 #endif // DOCTEST_MSVC
00166
00167 // =====
00168 // == COMPILER WARNINGS ==
00169 // =====
00170
00171 // both the header and the implementation suppress all of these,
00172 // so it only makes sense to aggregate them like so
00173 #define DOCTEST_SUPPRESS_COMMON_WARNINGS_PUSH
    \
00174     DOCTEST_CLANG_SUPPRESS_WARNING_PUSH
    \
00175     DOCTEST_CLANG_SUPPRESS_WARNING("-Wunknown-pragmas")
    \
00176     DOCTEST_CLANG_SUPPRESS_WARNING("-Wunknown-warning-option")
    \
00177     DOCTEST_CLANG_SUPPRESS_WARNING("-Wweak-vtables")
    \
00178     DOCTEST_CLANG_SUPPRESS_WARNING("-Wpadded")
    \
00179     DOCTEST_CLANG_SUPPRESS_WARNING("-Wmissing-prototypes")
    \
00180     DOCTEST_CLANG_SUPPRESS_WARNING("-Wc++98-compat")
    \
00181     DOCTEST_CLANG_SUPPRESS_WARNING("-Wc++98-compat-pedantic")
    \
00182     DOCTEST_CLANG_SUPPRESS_WARNING("-Wunsafe-buffer-usage")
    \
00183     DOCTEST_CLANG_SUPPRESS_WARNING("-Wunused-macros")

```

```

00184 \
00185 \ DOCTEST_GCC_SUPPRESS_WARNING_PUSH
00186 \ DOCTEST_GCC_SUPPRESS_WARNING("-Wunknown-pragmas")
00187 \ DOCTEST_GCC_SUPPRESS_WARNING("-Wpragmas")
00188 \ DOCTEST_GCC_SUPPRESS_WARNING("-Wefc++")
00189 \ DOCTEST_GCC_SUPPRESS_WARNING("-Wstrict-overflow")
00190 \ DOCTEST_GCC_SUPPRESS_WARNING("-Wstrict-aliasing")
00191 \ DOCTEST_GCC_SUPPRESS_WARNING("-Wmissing-declarations")
00192 \ DOCTEST_GCC_SUPPRESS_WARNING("-Wuseless-cast")
00193 \ DOCTEST_GCC_SUPPRESS_WARNING("-Wnoexcept")
00194 \
00195 \ DOCTEST_MSVC_SUPPRESS_WARNING_PUSH
00196 \ /* these 4 also disabled globally via cmake: */
00197 \ DOCTEST_MSVC_SUPPRESS_WARNING(4514) /* unreferenced inline function has been removed */
00198 \ DOCTEST_MSVC_SUPPRESS_WARNING(4571) /* SEH related */
00199 \ DOCTEST_MSVC_SUPPRESS_WARNING(4710) /* function not inlined */
00200 \ DOCTEST_MSVC_SUPPRESS_WARNING(4711) /* function selected for inline expansion*/
00201 \ /* common ones */
00202 \ DOCTEST_MSVC_SUPPRESS_WARNING(4616) /* invalid compiler warning */
00203 \ DOCTEST_MSVC_SUPPRESS_WARNING(4619) /* invalid compiler warning */
00204 \ DOCTEST_MSVC_SUPPRESS_WARNING(4996) /* The compiler encountered a deprecated declaration */
00205 \ DOCTEST_MSVC_SUPPRESS_WARNING(4706) /* assignment within conditional expression */
00206 \ DOCTEST_MSVC_SUPPRESS_WARNING(4512) /* 'class' : assignment operator could not be generated */
00207 \ DOCTEST_MSVC_SUPPRESS_WARNING(4127) /* conditional expression is constant */
00208 \ DOCTEST_MSVC_SUPPRESS_WARNING(4820) /* padding */
00209 \ DOCTEST_MSVC_SUPPRESS_WARNING(4625) /* copy constructor was implicitly deleted */
00210 \ DOCTEST_MSVC_SUPPRESS_WARNING(4626) /* assignment operator was implicitly deleted */
00211 \ DOCTEST_MSVC_SUPPRESS_WARNING(5027) /* move assignment operator implicitly deleted */
00212 \ DOCTEST_MSVC_SUPPRESS_WARNING(5026) /* move constructor was implicitly deleted */
00213 \ DOCTEST_MSVC_SUPPRESS_WARNING(4640) /* construction of local static object not thread-safe */
00214 \ DOCTEST_MSVC_SUPPRESS_WARNING(5045) /* Spectre mitigation for memory load */
00215 \ DOCTEST_MSVC_SUPPRESS_WARNING(5264) /* 'variable-name': 'const' variable is not used */
00216 \ /* static analysis */
00217 \ DOCTEST_MSVC_SUPPRESS_WARNING(26439) /* Function may not throw. Declare it 'noexcept' */
00218 \ DOCTEST_MSVC_SUPPRESS_WARNING(26495) /* Always initialize a member variable */
00219 \ DOCTEST_MSVC_SUPPRESS_WARNING(26451) /* Arithmetic overflow ... */
00220 \ DOCTEST_MSVC_SUPPRESS_WARNING(26444) /* Avoid unnamed objects with custom ctor and dtor... */
00221 \ DOCTEST_MSVC_SUPPRESS_WARNING(26812) /* Prefer 'enum class' over 'enum' */
00222 \
00223 #define DOCTEST_SUPPRESS_COMMON_WARNINGS_POP
00224 \ DOCTEST_CLANG_SUPPRESS_WARNING_POP
00225 \ DOCTEST_GCC_SUPPRESS_WARNING_POP
00226 \ DOCTEST_MSVC_SUPPRESS_WARNING_POP
00227 \
00228 #define DOCTEST_SUPPRESS_PUBLIC_WARNINGS_PUSH

```

```
00229     DOCTEST_SUPPRESS_COMMON_WARNINGS_PUSH
00230 \
00231     DOCTEST_CLANG_SUPPRESS_WARNING("-Wnon-virtual-dtor")
00232 \
00233     DOCTEST_CLANG_SUPPRESS_WARNING("-Wdeprecated")
00234 \
00235     DOCTEST_GCC_SUPPRESS_WARNING("-Wctor-dtor-privacy")
00236 \
00237     DOCTEST_GCC_SUPPRESS_WARNING("-Wnon-virtual-dtor")
00238 \
00239     DOCTEST_GCC_SUPPRESS_WARNING("-Wsign-promo")
00240 \
00241     DOCTEST_MSVC_SUPPRESS_WARNING(4623) /* default constructor was implicitly deleted */
00242 \
00243 #define DOCTEST_SUPPRESS_PUBLIC_WARNINGS_POP DOCTEST_SUPPRESS_COMMON_WARNINGS_POP
00244 \
00245 #define DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
00246 \
00247     DOCTEST_SUPPRESS_COMMON_WARNINGS_PUSH
00248 \
00249     DOCTEST_CLANG_SUPPRESS_WARNING("-Wglobal-constructors")
00250 \
00251     DOCTEST_CLANG_SUPPRESS_WARNING("-Wexit-time- destructors")
00252 \
00253     DOCTEST_CLANG_SUPPRESS_WARNING("-Wsign-conversion")
00254 \
00255     DOCTEST_CLANG_SUPPRESS_WARNING("-Wshorten-64-to-32")
00256 \
00257     DOCTEST_CLANG_SUPPRESS_WARNING("-Wmissing-variable-declarations")
00258 \
00259     DOCTEST_CLANG_SUPPRESS_WARNING("-Wswitch")
00260 \
00261     DOCTEST_CLANG_SUPPRESS_WARNING("-Wswitch-enum")
00262 \
00263     DOCTEST_CLANG_SUPPRESS_WARNING("-Wcovered-switch-default")
00264 \
00265     DOCTEST_CLANG_SUPPRESS_WARNING("-Wmissing-noreturn")
00266 \
00267     DOCTEST_CLANG_SUPPRESS_WARNING("-Wdisabled-macro-expansion")
00268 \
00269     DOCTEST_CLANG_SUPPRESS_WARNING("-Wmissing-braces")
00270 \
00271     DOCTEST_CLANG_SUPPRESS_WARNING("-Wmissing-field-initializers")
00272 \
00273     DOCTEST_CLANG_SUPPRESS_WARNING("-Wunused-member-function")
00274 \
00275     DOCTEST_CLANG_SUPPRESS_WARNING("-Wunused-function")
00276 \
00277     DOCTEST_CLANG_SUPPRESS_WARNING("-Wnonportable-system-include-path")
00278 \
00279     DOCTEST_CLANG_SUPPRESS_WARNING("-Wnrvo")
00280 \
00281     DOCTEST_GCC_SUPPRESS_WARNING("-Wconversion")
00282 \
00283     DOCTEST_GCC_SUPPRESS_WARNING("-Wsign-conversion")
00284 \
00285     DOCTEST_GCC_SUPPRESS_WARNING("-Wmissing-field-initializers")
00286 \
00287     DOCTEST_GCC_SUPPRESS_WARNING("-Wmissing-braces")
00288 \
00289     DOCTEST_GCC_SUPPRESS_WARNING("-Wswitch")
00290 \
00291     DOCTEST_GCC_SUPPRESS_WARNING("-Wswitch-enum")
00292 \
00293     DOCTEST_GCC_SUPPRESS_WARNING("-Wswitch-default")
00294 \
00295     DOCTEST_GCC_SUPPRESS_WARNING("-Wunsafe-loop-optimizations")
00296 \
00297     DOCTEST_GCC_SUPPRESS_WARNING("-Wold-style-cast")
00298 \
00299     DOCTEST_GCC_SUPPRESS_WARNING("-Wunused-function")
00300 \
00301     DOCTEST_GCC_SUPPRESS_WARNING("-Wmultiple-inheritance")
00302 \
00303     DOCTEST_GCC_SUPPRESS_WARNING("-Wsuggest-attribute")
00304 \
00305     DOCTEST_GCC_SUPPRESS_WARNING("-Wnrvo")
```

```

00275 \
00276 \ DOCTEST_MSVC_SUPPRESS_WARNING(4267) /* conversion from 'x' to 'y', possible loss of data */
00277 \ DOCTEST_MSVC_SUPPRESS_WARNING(4530) /* exception handler, but unwind semantics not enabled */
00278 \ DOCTEST_MSVC_SUPPRESS_WARNING(4577) /* 'noexcept' with no exception handling mode specified */
00279 \ DOCTEST_MSVC_SUPPRESS_WARNING(4774) /* format string in argument is not a string literal */
00280 \ DOCTEST_MSVC_SUPPRESS_WARNING(4365) /* signed/unsigned mismatch */
00281 \ DOCTEST_MSVC_SUPPRESS_WARNING(5039) /* pointer to pot. throwing function passed to extern C */
00282 \ DOCTEST_MSVC_SUPPRESS_WARNING(4800) /* forcing value to bool (performance warning) */
00283 \ DOCTEST_MSVC_SUPPRESS_WARNING(5245) /* unreferenced function with internal linkage removed */
00284
00285 #define DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP DOCTEST_SUPPRESS_COMMON_WARNINGS_POP
00286
00287 #define DOCTEST_MAKE_STD_HEADERS_CLEAN_FROM_WARNINGS_ON_WALL_BEGIN
00288 \ DOCTEST_MSVC_SUPPRESS_WARNING_PUSH
00289 \ DOCTEST_MSVC_SUPPRESS_WARNING(4548) /* before comma no effect; expected side - effect */
00290 \ DOCTEST_MSVC_SUPPRESS_WARNING(4265) /* virtual functions, but destructor is not virtual */
00291 \ DOCTEST_MSVC_SUPPRESS_WARNING(4986) /* exception specification does not match previous */
00292 \ DOCTEST_MSVC_SUPPRESS_WARNING(4350) /* 'member1' called instead of 'member2' */
00293 \ DOCTEST_MSVC_SUPPRESS_WARNING(4668) /* not defined as a preprocessor macro */
00294 \ DOCTEST_MSVC_SUPPRESS_WARNING(4365) /* signed/unsigned mismatch */
00295 \ DOCTEST_MSVC_SUPPRESS_WARNING(4774) /* format string not a string literal */
00296 \ DOCTEST_MSVC_SUPPRESS_WARNING(4820) /* padding */
00297 \ DOCTEST_MSVC_SUPPRESS_WARNING(4625) /* copy constructor was implicitly deleted */
00298 \ DOCTEST_MSVC_SUPPRESS_WARNING(4626) /* assignment operator was implicitly deleted */
00299 \ DOCTEST_MSVC_SUPPRESS_WARNING(5027) /* move assignment operator implicitly deleted */
00300 \ DOCTEST_MSVC_SUPPRESS_WARNING(5026) /* move constructor was implicitly deleted */
00301 \ DOCTEST_MSVC_SUPPRESS_WARNING(4623) /* default constructor was implicitly deleted */
00302 \ DOCTEST_MSVC_SUPPRESS_WARNING(5039) /* pointer to pot. throwing function passed to extern C */
00303 \ DOCTEST_MSVC_SUPPRESS_WARNING(5045) /* Spectre mitigation for memory load */
00304 \ DOCTEST_MSVC_SUPPRESS_WARNING(5105) /* macro producing 'defined' has undefined behavior */
00305 \ DOCTEST_MSVC_SUPPRESS_WARNING(4738) /* storing float result in memory, loss of performance */
00306 \ DOCTEST_MSVC_SUPPRESS_WARNING(5262) /* implicit fall-through */
00307
00308 #define DOCTEST_MAKE_STD_HEADERS_CLEAN_FROM_WARNINGS_ON_WALL_END DOCTEST_MSVC_SUPPRESS_WARNING_POP
00309
00310 #endif // DOCTEST_PARTS_PUBLIC_WARNINGS
00311
00312 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_PUSH
00313
00314 // =====
00315 // == FEATURE DETECTION =====
00316 // =====
00317
00318 #ifndef DOCTEST_PARTS_PUBLIC_CONFIG
00319 #define DOCTEST_PARTS_PUBLIC_CONFIG
00320
00321 #ifndef DOCTEST_PARTS_PUBLIC_PLATFORM
00322 #define DOCTEST_PARTS_PUBLIC_PLATFORM
00323
00324 #if defined(__APPLE__)
00325 // Apple detection taken from Catch2 codebase
00326 // For <TargetConditionals.h> information:
00327 //
00328 // https://github.com/swiftlang/swift-corelibs-foundation/blob/release/5.10/CoreFoundation/Base.subproj/SwiftRuntime/Target
00328 #include <TargetConditionals.h>
00329 #if (defined(TARGET_OS_MAC) && TARGET_OS_MAC == 1) || (defined(TARGET_OS_OSX) && TARGET_OS_OSX == 1)
00330 #define DOCTEST_PLATFORM_MAC
00331
00332 #elif defined(TARGET_OS_IPHONE) && TARGET_OS_IPHONE == 1

```

```

00333 #define DOCTEST_PLATFORM_IPHONE
00334 #endif
00335
00336 #elif defined(WIN32) || defined(_WIN32)
00337 #define DOCTEST_PLATFORM_WINDOWS
00338
00339 #elif defined(__wasi__)
00340 #define DOCTEST_PLATFORM_WASI
00341
00342 #else // defined(linux) || defined(__linux) // defined(__linux__)
00343 #define DOCTEST_PLATFORM_LINUX
00344 #endif // DOCTEST_PLATFORM
00345
00346 #endif // DOCTEST_PARTS_PUBLIC_PLATFORM
00347
00348 // general compiler feature support table: https://en.cppreference.com/w/cpp/compiler_support
00349 // MSVC C++11 feature support table: https://msdn.microsoft.com/en-us/library/hh567368.aspx
00350 // GCC C++11 feature support table: https://gcc.gnu.org/projects/cxx-status.html
00351 // MSVC version table:
00352 // https://en.wikipedia.org/wiki/Microsoft_Visual_C%2B%2B#Internal_version_numbering
00353 // MSVC++ 14.3 (17) _MSC_VER == 1930 (Visual Studio 2022)
00354 // MSVC++ 14.2 (16) _MSC_VER == 1920 (Visual Studio 2019)
00355 // MSVC++ 14.1 (15) _MSC_VER == 1910 (Visual Studio 2017)
00356 // MSVC++ 14.0 _MSC_VER == 1900 (Visual Studio 2015)
00357 // MSVC++ 12.0 _MSC_VER == 1800 (Visual Studio 2013)
00358 // MSVC++ 11.0 _MSC_VER == 1700 (Visual Studio 2012)
00359 // MSVC++ 10.0 _MSC_VER == 1600 (Visual Studio 2010)
00360 // MSVC++ 9.0 _MSC_VER == 1500 (Visual Studio 2008)
00361 // MSVC++ 8.0 _MSC_VER == 1400 (Visual Studio 2005)
00362
00363 // Universal Windows Platform support
00364 #if defined(WINAPI_FAMILY) && (WINAPI_FAMILY == WINAPI_FAMILY_APP)
00365 #define DOCTEST_CONFIG_NO_WINDOWS_SEH
00366 #endif // WINAPI_FAMILY
00367 #if DOCTEST_MSVC && !defined(DOCTEST_CONFIG_WINDOWS_SEH)
00368 #define DOCTEST_CONFIG_WINDOWS_SEH
00369 #endif // MSVC
00370 #if defined(DOCTEST_CONFIG_NO_WINDOWS_SEH) && defined(DOCTEST_CONFIG_WINDOWS_SEH)
00371 #undef DOCTEST_CONFIG_WINDOWS_SEH
00372 #endif // DOCTEST_CONFIG_NO_WINDOWS_SEH
00373
00374 #if !defined(_WIN32) && !defined(__QNX__) && !defined(DOCTEST_CONFIG_POSIX_SIGNALS) &&
!defined(__EMSCRIPTEN__) && \
00375 !defined(__wasi__)
00376 #define DOCTEST_CONFIG_POSIX_SIGNALS
00377 #endif // _WIN32
00378 #if defined(DOCTEST_CONFIG_NO_POSIX_SIGNALS) && defined(DOCTEST_CONFIG_POSIX_SIGNALS)
00379 #undef DOCTEST_CONFIG_POSIX_SIGNALS
00380 #endif // DOCTEST_CONFIG_NO_POSIX_SIGNALS
00381
00382 #ifndef DOCTEST_CONFIG_NO_EXCEPTIONS
00383 #if !defined(__cpp_exceptions) && !defined(__EXCEPTIONS) && !defined(CPPUNWIND) || defined(__wasi__)
00384 #define DOCTEST_CONFIG_NO_EXCEPTIONS
00385 #endif // no exceptions
00386 #endif // DOCTEST_CONFIG_NO_EXCEPTIONS
00387
00388 #ifndef DOCTEST_CONFIG_NO_EXCEPTIONS_BUT_WITH_ALL_ASSERTS
00389 #ifndef DOCTEST_CONFIG_NO_EXCEPTIONS
00390 #define DOCTEST_CONFIG_NO_EXCEPTIONS
00391 #endif // DOCTEST_CONFIG_NO_EXCEPTIONS
00392 #endif // DOCTEST_CONFIG_NO_EXCEPTIONS_BUT_WITH_ALL_ASSERTS
00393
00394 #if defined(DOCTEST_CONFIG_NO_EXCEPTIONS) && !defined(DOCTEST_CONFIG_NO_TRY_CATCH_IN_ASSERTS)
00395 #define DOCTEST_CONFIG_NO_TRY_CATCH_IN_ASSERTS
00396 #endif // DOCTEST_CONFIG_NO_EXCEPTIONS && !DOCTEST_CONFIG_NO_TRY_CATCH_IN_ASSERTS
00397
00398 #ifndef __wasi__
00399 #define DOCTEST_CONFIG_NO_MULTITHREADING
00400 #endif
00401
00402 #if defined(DOCTEST_CONFIG_IMPLEMENT_WITH_MAIN) && !defined(DOCTEST_CONFIG_IMPLEMENT)
00403 #define DOCTEST_CONFIG_IMPLEMENT
00404 #endif // DOCTEST_CONFIG_IMPLEMENT_WITH_MAIN
00405
00406 #if defined(_WIN32) || defined(__CYGWIN__)
00407 #if DOCTEST_MSVC
00408 #define DOCTEST_SYMBOL_EXPORT __declspec(dllexport)
00409 #define DOCTEST_SYMBOL_IMPORT __declspec(dllimport)
00410 #else // MSVC
00411 #define DOCTEST_SYMBOL_EXPORT __attribute__((dllexport))
00412 #define DOCTEST_SYMBOL_IMPORT __attribute__((dllimport))
00413 #endif // MSVC
00414 #else // _WIN32
00415 #define DOCTEST_SYMBOL_EXPORT __attribute__((visibility("default")))
00416 #define DOCTEST_SYMBOL_IMPORT
00417 #endif // _WIN32
00418

```

```

00419 #ifdef DOCTEST_CONFIG_IMPLEMENTATION_IN_DLL
00420 #ifdef DOCTEST_CONFIG_IMPLEMENT
00421 #define DOCTEST_INTERFACE DOCTEST_SYMBOL_EXPORT
00422 #else // DOCTEST_CONFIG_IMPLEMENT
00423 #define DOCTEST_INTERFACE DOCTEST_SYMBOL_IMPORT
00424 #endif // DOCTEST_CONFIG_IMPLEMENT
00425 #else // DOCTEST_CONFIG_IMPLEMENTATION_IN_DLL
00426 #define DOCTEST_INTERFACE
00427 #endif // DOCTEST_CONFIG_IMPLEMENTATION_IN_DLL
00428
00429 // needed for extern template instantiations
00430 // see https://github.com/fmtlib/fmt/issues/2228
00431 #if DOCTEST_MSVC
00432 #define DOCTEST_INTERFACE_DECL
00433 #define DOCTEST_INTERFACE_DEF DOCTEST_INTERFACE
00434 #else // DOCTEST_MSVC
00435 #define DOCTEST_INTERFACE_DECL DOCTEST_INTERFACE
00436 #define DOCTEST_INTERFACE_DEF
00437 #endif // DOCTEST_MSVC
00438
00439 #define DOCTEST_EMPTY
00440
00441 #if DOCTEST_MSVC
00442 #define DOCTEST_NOINLINE __declspec(noinline)
00443 #define DOCTEST_UNUSED
00444 #define DOCTEST_ALIGNMENT(x)
00445 #elif DOCTEST_CLANG && DOCTEST_CLANG < DOCTEST_COMPILER(3, 5, 0)
00446 #define DOCTEST_NOINLINE
00447 #define DOCTEST_UNUSED
00448 #define DOCTEST_ALIGNMENT(x)
00449 #else
00450 #define DOCTEST_NOINLINE __attribute__((noinline))
00451 #define DOCTEST_UNUSED __attribute__((unused))
00452 #define DOCTEST_ALIGNMENT(x) __attribute__((aligned(x)))
00453 #endif
00454
00455 #ifdef DOCTEST_CONFIG_NO_CONTRADICTING_INLINE
00456 #define DOCTEST_INLINE_NOINLINE inline
00457 #else
00458 #define DOCTEST_INLINE_NOINLINE inline DOCTEST_NOINLINE
00459 #endif
00460
00461 #ifndef DOCTEST_NORETURN
00462 #if DOCTEST_MSVC && (DOCTEST_MSVC < DOCTEST_COMPILER(19, 0, 0))
00463 #define DOCTEST_NORETURN
00464 #else // DOCTEST_MSVC
00465 #define DOCTEST_NORETURN [[noreturn]]
00466 #endif // DOCTEST_MSVC
00467 #endif // DOCTEST_NORETURN
00468
00469 #ifndef DOCTEST_NOEXCEPT
00470 #if DOCTEST_MSVC && (DOCTEST_MSVC < DOCTEST_COMPILER(19, 0, 0))
00471 #define DOCTEST_NOEXCEPT
00472 #else // DOCTEST_MSVC
00473 #define DOCTEST_NOEXCEPT noexcept
00474 #endif // DOCTEST_MSVC
00475 #endif // DOCTEST_NOEXCEPT
00476
00477 #ifndef DOCTEST_CONSTEXPR
00478 #if DOCTEST_MSVC && (DOCTEST_MSVC < DOCTEST_COMPILER(19, 0, 0))
00479 #define DOCTEST_CONSTEXPR const
00480 #define DOCTEST_CONSTEXPR_FUNC inline
00481 #else // DOCTEST_MSVC
00482 #define DOCTEST_CONSTEXPR constexpr
00483 #define DOCTEST_CONSTEXPR_FUNC constexpr
00484 #endif // DOCTEST_MSVC
00485 #endif // DOCTEST_CONSTEXPR
00486
00487 #ifndef DOCTEST_NO_SANITIZE_INTEGER
00488 #if DOCTEST_CLANG >= DOCTEST_COMPILER(3, 7, 0)
00489 #define DOCTEST_NO_SANITIZE_INTEGER __attribute__((no_sanitize("integer")))
00490 #else
00491 #define DOCTEST_NO_SANITIZE_INTEGER
00492 #endif
00493 #endif // DOCTEST_NO_SANITIZE_INTEGER
00494
00495 // this is kept here for backwards compatibility since the config option was changed
00496 #ifdef DOCTEST_CONFIG_USE_IOFWD
00497 #ifndef DOCTEST_CONFIG_USE_STD_HEADERS
00498 #define DOCTEST_CONFIG_USE_STD_HEADERS
00499 #endif
00500 #endif // DOCTEST_CONFIG_USE_IOFWD
00501
00502 // for clang - always include <version> or <ciso646> (which drags some std stuff)
00503 // because we want to check if we are using libc++ with the _LIBCPP_VERSION macro in
00504 // which case we don't want to forward declare stuff from std - for reference:
00505 // https://github.com/doctest/doctest/issues/126

```

```

00506 // https://github.com/doctest/doctest/issues/356
00507 #if DOctest_CLANG
00508 DOctest_MAKE_STD_HEADERS_CLEAN_FROM_WARNINGS_ON_WALL_BEGIN
00509 #if DOctest_CPLUSPLUS >= 201703L && __has_include(<version>)
00510 #include <version>
00511 #else
00512 #include <ciso646>
00513 #endif
00514 DOctest_MAKE_STD_HEADERS_CLEAN_FROM_WARNINGS_ON_WALL_END
00515 #endif // clang
00516
00517 #ifndef _LIBCPP_VERSION
00518 #ifndef DOctest_CONFIG_USE_STD_HEADERS
00519 #define DOctest_CONFIG_USE_STD_HEADERS
00520 #endif
00521 #endif // _LIBCPP_VERSION
00522
00523 #ifndef DOctest_CONFIG_USE_STD_HEADERS
00524 #ifndef DOctest_CONFIG_INCLUDE_TYPE_TRAITS
00525 #define DOctest_CONFIG_INCLUDE_TYPE_TRAITS
00526 #endif // DOctest_CONFIG_INCLUDE_TYPE_TRAITS
00527 #endif // DOctest_CONFIG_USE_STD_HEADERS
00528
00529 #if defined(__has_builtin)
00530 #define DOctest_HAS_BUILTIN(x) __has_builtin(x)
00531 #else
00532 #define DOctest_HAS_BUILTIN(x) 0
00533 #endif // __has_builtin
00534
00535 #endif // DOctest_PARTS_PUBLIC_CONFIG
00536
00537 // =====
00538 // == FEATURE DETECTION END ==
00539 // =====
00540 #ifndef DOctest_PARTS_PUBLIC_UTILITY
00541 #define DOctest_PARTS_PUBLIC_UTILITY
00542
00543
00544 DOctest_SUPPRESS_PUBLIC_WARNINGS_PUSH
00545
00546 #define DOctest_DECLARE_INTERFACE(name)
00547 \
00548 \
00549 \
00550 \
00551 \
00552 \
00553
00554 #define DOctest_DEFINE_INTERFACE(name) name::~name() = default;
00555
00556 #if !defined(DOctest_COUNTER)
00557 #if DOctest_CLANG >= DOctest_COMPILER(22, 0, 0)
00558 #define DOctest_COUNTER __LINE__
00559 #elif defined(__COUNTER__)
00560 #define DOctest_COUNTER __COUNTER__
00561 #else
00562 #define DOctest_COUNTER __LINE__
00563 #endif
00564 #endif // defined(DOctest_COUNTER)
00565
00566 // internal macros for string concatenation and anonymous variable name generation
00567 #define DOctest_CAT_IMPL(s1, s2) s1##s2
00568 #define DOctest_CAT(s1, s2) DOctest_CAT_IMPL(s1, s2)
00569 #define DOctest_ANONYMOUS(x) DOctest_CAT(x, DOctest_COUNTER)
00570
00571 #ifndef DOctest_CONFIG_ASSERTION_PARAMETERS_BY_VALUE
00572 #define DOctest_REF_WRAP(x) x &
00573 #else // DOctest_CONFIG_ASSERTION_PARAMETERS_BY_VALUE
00574 #define DOctest_REF_WRAP(x) x
00575 #endif // DOctest_CONFIG_ASSERTION_PARAMETERS_BY_VALUE
00576
00577 namespace doctest {
00578 namespace detail {
00579 DOctest_CLANG_SUPPRESS_WARNING_WITH_PUSH("-Wunused-function")
00580 static DOctest_CONSTEXPR int consume(const int *, int) noexcept {
00581     return 0;
00582 }
00583 DOctest_CLANG_SUPPRESS_WARNING_POP
00584 } // namespace detail
00585 } // namespace doctest
00586

```



```

00587 #define DOCTEST_GLOBAL_NO_WARNINGS(var, ...)
00588 \
00589 \
00590 \
00591 \
00592 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_POP
00593
00594 #endif // DOCTEST_PARTS_PUBLIC_UTILITY
00595 #ifndef DOCTEST_PARTS_PUBLIC_DEBUGGER
00596 #define DOCTEST_PARTS_PUBLIC_DEBUGGER
00597
00598
00599 #ifndef DOCTEST_BREAK_INTO_DEBUGGER
00600
00601 // should probably take a look at https://github.com/scottt/debugbreak
00602 #if DOCTEST_CLANG && DOCTEST_HAS_BUILTIN(__builtin_debugtrap)
00603 #define DOCTEST_BREAK_INTO_DEBUGGER() __builtin_debugtrap()
00604
00605 #elif defined(DOCTEST_PLATFORM_LINUX)
00606 #if defined(__GNUC__) && (defined(__i386) || defined(__x86_64))
00607 // Break at the location of the failing check if possible
00608 #define DOCTEST_BREAK_INTO_DEBUGGER() __asm__("int $3\n" ::) // NOLINT(hicpp-no-assembler)
00609 #else
00610 DOCTEST_MAKE_STD_HEADERS_CLEAN_FROM_WARNINGS_ON_WALL_BEGIN
00611 #include <signal.h>
00612 DOCTEST_MAKE_STD_HEADERS_CLEAN_FROM_WARNINGS_ON_WALL_END
00613 #define DOCTEST_BREAK_INTO_DEBUGGER() raise(SIGTRAP)
00614 #endif
00615
00616 #elif defined(DOCTEST_PLATFORM_MAC)
00617 #if defined(__x86_64) || defined(__x86_64__) || defined(__amd64__) || defined(__i386)
00618 #define DOCTEST_BREAK_INTO_DEBUGGER() __asm__("int $3\n" ::) // NOLINT(hicpp-no-assembler)
00619 #elif defined(__ppc__) || defined(__ppc64__)
00620 // https://www.cocoawithlove.com/2008/03/break-into-debugger.html
00621 #define DOCTEST_BREAK_INTO_DEBUGGER() /* NOLINTNEXTLINE(hicpp-no-assembler) */
00622 \
00623 \
00624 \
00625 \
00626 \
00627 \
00628 \
00629 \
00630 \
00631 \
00632 \
00633 \
00634 \
00635 \
00636 \
00637 \
00638 \
00639 \
00640 #ifndef DOCTEST_CONFIG_DISABLE
00641
00642 namespace doctest {
00643 namespace detail {
00644 DOCTEST_INTERFACE bool isDebuggerActive();
00645 } // namespace detail
00646 } // namespace doctest
00647
00648 #endif
00649
00650 #endif // DOCTEST_PARTS_PUBLIC_DEBUGGER
00651 #ifndef DOCTEST_PARTS_PUBLIC_STD_FWD
00652 #define DOCTEST_PARTS_PUBLIC_STD_FWD
00653
00654
00655 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_PUSH
00656
00657 #ifdef DOCTEST_CONFIG_USE_STD_HEADERS
00658 DOCTEST_MAKE_STD_HEADERS_CLEAN_FROM_WARNINGS_ON_WALL_BEGIN
00659 #include <cstdint>
00660 #include <ostream>
00661 #include <istream>
00662 DOCTEST_MAKE_STD_HEADERS_CLEAN_FROM_WARNINGS_ON_WALL_END
00663 #else // DOCTEST_CONFIG_USE_STD_HEADERS
00664
00665 // Forward declaring 'X' in namespace std is not permitted by the C++ Standard.
00666 DOCTEST_MSVC_SUPPRESS_WARNING_WITH_PUSH(4643)
00667
00668 // NOLINTBEGIN(bugprone-std-namespace-modification, cert-dcl58-cpp)
00669 namespace std {

```

```

00670 typedef decltype(nullptr) nullptr_t; // NOLINT(modernize-use-using)
00671 typedef decltype(sizeof(void*)) size_t; // NOLINT(modernize-use-using)
00672 template <class charT>
00673 struct char_traits;
00674 template <>
00675 struct char_traits<char>;
00676 template <class charT, class traits>
00677 class basic_ostream; // NOLINT(fuchsia-virtual-inheritance)
00678 typedef basic_ostream<char, char_traits<char>> ostream; // NOLINT(modernize-use-using)
00679 template <class traits>
00680 // NOLINTNEXTLINE
00681 basic_ostream<char, traits> &operator<<(basic_ostream<char, traits> &, const char *);
00682 template <class charT, class traits>
00683 class basic_istream;
00684 typedef basic_istream<char, char_traits<char>> istream; // NOLINT(modernize-use-using)
00685 template <class... Types>
00686 class tuple;
00687 #if DOCTEST_MSVC >= DOCTEST_COMPILER(19, 20, 0)
00688 // see this issue on why this is needed: https://github.com/doctest/doctest/issues/183
00689 template <class Ty>
00690 class allocator;
00691 template <class Elem, class Traits, class Alloc>
00692 class basic_string;
00693 using string = basic_string<char, char_traits<char>, allocator<char>;
00694 #endif // VS 2019
00695 } // namespace std
00696 // NOLINTEND(bugprone-std-namespace-modification, cert-dcl58-cpp)
00697
00698 DOCTEST_MSVC_SUPPRESS_WARNING_POP
00699
00700 #endif // DOCTEST_CONFIG_USE_STD_HEADERS
00701
00702 namespace doctest {
00703 using std::size_t;
00704 } // namespace doctest
00705
00706 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_POP
00707
00708 #endif // DOCTEST_PARTS_PUBLIC_STD_FWD
00709 #ifndef DOCTEST_PARTS_PUBLIC_STD_TYPE_TRAITS
00710 #define DOCTEST_PARTS_PUBLIC_STD_TYPE_TRAITS
00711
00712
00713 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_PUSH
00714
00715 #ifdef DOCTEST_CONFIG_INCLUDE_TYPE_TRAITS
00716 DOCTEST_MAKE_STD_HEADERS_CLEAN_FROM_WARNINGS_ON_WALL_BEGIN
00717 #include <type_traits>
00718 DOCTEST_MAKE_STD_HEADERS_CLEAN_FROM_WARNINGS_ON_WALL_END
00719 #endif // DOCTEST_CONFIG_INCLUDE_TYPE_TRAITS
00720
00721 namespace doctest {
00722 namespace detail {
00723 namespace types {
00724
00725 #ifdef DOCTEST_CONFIG_INCLUDE_TYPE_TRAITS
00726 using namespace std;
00727 #else
00728 template <bool COND, typename T = void>
00729 struct enable_if {};
00730
00731 template <typename T>
00732 struct enable_if<true, T> {
00733     using type = T;
00734 };
00735
00736 struct true_type {
00737     static DOCTEST_CONSTEXPR bool value = true;
00738 };
00739
00740 struct false_type {
00741     static DOCTEST_CONSTEXPR bool value = false;
00742 };
00743
00744 template <typename T>
00745 struct remove_reference {
00746     using type = T;
00747 };
00748
00749 template <typename T>
00750 struct remove_reference<T &> {
00751     using type = T;
00752 };
00753
00754 template <typename T>
00755 struct remove_reference<T &&> {
00756     using type = T;

```

```

00757 };
00758
00759 template <typename T, typename U>
00760 struct is_same : false_type {};
00761
00762 template <typename T>
00763 struct is_same<T, T> : true_type {};
00764
00765 template <typename T>
00766 struct is_rvalue_reference : false_type {};
00767
00768 template <typename T>
00769 struct is_rvalue_reference<T &&> : true_type {};
00770
00771 template <typename T>
00772 struct remove_const {
00773     using type = T;
00774 };
00775
00776 template <typename T>
00777 struct remove_const<const T> {
00778     using type = T;
00779 };
00780
00781 // Compiler intrinsics
00782 template <typename T>
00783 struct is_enum {
00784     static DOCTEST_CONSTEXPR bool value = __is_enum(T);
00785 };
00786
00787 template <typename T>
00788 struct underlying_type {
00789     using type = __underlying_type(T);
00790 };
00791
00792 template <typename T>
00793 struct is_pointer : false_type {};
00794
00795 template <typename T>
00796 struct is_pointer<T *> : true_type {};
00797
00798 template <typename T>
00799 struct is_array : false_type {};
00800 // NOLINTNEXTLINE(*-avoid-c-arrays)
00801 template <typename T, size_t SIZE>
00802 struct is_array<T[SIZE]> : true_type {};
00803 #endif
00804
00805 #if !(defined(DOCTEST_CONFIG_INCLUDE_TYPE_TRAITS) && (DOCTEST_CPLUSPLUS >= 201703L))
00806 template <typename... Unused>
00807 using void_t = void;
00808 #endif
00809
00810 } // namespace types
00811 } // namespace detail
00812 } // namespace doctest
00813
00814 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_POP
00815
00816 #endif // DOCTEST_PARTS_PUBLIC_STD_TYPE_TRAITS
00817 #ifndef DOCTEST_PARTS_PUBLIC_STD_UTILITY
00818 #define DOCTEST_PARTS_PUBLIC_STD_UTILITY
00819
00820
00821 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_PUSH
00822
00823 namespace doctest {
00824 namespace detail {
00825
00826 // <utility>
00827 template <typename T>
00828 T &&declval();
00829
00830 template <class T>
00831 DOCTEST_CONSTEXPR_FUNC T &&forward(typename types::remove_reference<T>::type &t) DOCTEST_NOEXCEPT {
00832     return static_cast<T &&>(t);
00833 }
00834
00835 template <class T>
00836 // NOLINTNEXTLINE(cppcoreguidelines-rvalue-reference-param-not-moved)
00837 DOCTEST_CONSTEXPR_FUNC T &&forward(typename types::remove_reference<T>::type &&t) DOCTEST_NOEXCEPT {
00838     return static_cast<T &&>(t);
00839 }
00840
00841 template <typename T>
00842 struct deferred_false : types::false_type {};
00843

```

```

00844 } // namespace detail
00845 } // namespace doctest
00846
00847 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_POP
00848
00849 #endif // DOCTEST_PARTS_PUBLIC_STD_UTILITY
00850 #ifndef DOCTEST_PARTS_PUBLIC_STRING
00851 #define DOCTEST_PARTS_PUBLIC_STRING
00852
00853
00854 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_PUSH
00855
00856 namespace doctest {
00857 #ifndef DOCTEST_CONFIG_STRING_SIZE_TYPE
00858 #define DOCTEST_CONFIG_STRING_SIZE_TYPE unsigned
00859 #endif
00860
00861 namespace detail {
00862
00863 template <typename T, typename Enable = void>
00864 struct is_std_string : types::false_type {};
00865
00866 template <typename T>
00867 struct is_std_string<
00868     T,
00869     typename types::enable_if<
00870         types::is_same<decltype(declval<const T &>().c_str()), const char *>::value &&
00871         types::is_same<decltype(declval<const T &>().size()), size_t>::value>::type> :
00872         types::true_type {};
00873 } // namespace detail
00874
00875 // A 24 byte string class (can be as small as 17 for x64 and 13 for x86) that can hold strings
00876 // with length of up to 23 chars on the stack before going on the heap -
00877 // the last byte of the buffer is used for:
00878 // - "is small" bit - the highest bit - if "0" then it is small - otherwise its "1" (128)
00879 // - if small - capacity left before going on the heap - using the lowest 5 bits
00880 // - if small - 2 bits are left unused - the second and third highest ones
00881 // - if small - acts as a null terminator if strlen() is 23 (24 including the null terminator)
00882 // and the "is small" bit remains "0" ("as well as the capacity left") so its OK
00883 // Idea taken from this lecture about the string implementation of facebook/folly - fbstring
00884 // https://www.youtube.com/watch?v=kPR8h4-qZdk
00885 // TODO:
00886 // - optimizations - like not deleting memory unnecessarily in operator= and etc.
00887 // - resize/reserve/clear
00888 // - replace
00889 // - back/front
00890 // - iterator stuff
00891 // - find & friends
00892 // - push_back/pop_back
00893 // - assign/insert/erase
00894 // - relational operators as free functions - taking const char* as one of the params
00895 class DOCTEST_INTERFACE String {
00896 public:
00897     using size_type = DOCTEST_CONFIG_STRING_SIZE_TYPE;
00898
00899 private:
00900     static DOCTEST_CONSTEXPR size_type len = 24;
00901     static DOCTEST_CONSTEXPR size_type last = len - 1;
00902
00903     struct view // len should be more than sizeof(view) - because of the final byte for flags
00904     {
00905         char *ptr;
00906         size_type size;
00907         size_type capacity;
00908     };
00909
00910     union {
00911         char buf[len]; // NOLINT(*-avoid-c-arrays)
00912         view data;
00913     };
00914
00915     char *allocate(size_type sz);
00916
00917     bool isOnStack() const noexcept {
00918         // NOLINTNEXTLINE(cppcoreguidelines-pro-type-union-access)
00919         return (buf[last] & 128) == 0;
00920     }
00921
00922     void setOnHeap() noexcept;
00923     void setLast(size_type in = last) noexcept;
00924     void setSize(size_type sz) noexcept;
00925
00926     void copy(const String &other);
00927
00928 public:
00929     static DOCTEST_CONSTEXPR size_type npos = static_cast<size_type>(-1);

```

```

00930
00931     String() noexcept;
00932     ~String();
00933
00934     String(const char *in);
00935     String(const char *in, size_type in_size);
00936
00937     String(std::istream &in, size_type in_size);
00938
00939     template <typename T, typename detail::types::enable_if<detail::is_std_string<T>::value,
bool>::type = true>
00940     String(const T &in)
00941         : String(in.c_str(), static_cast<size_type>(in.size())) {}
00942
00943     String(const String &other);
00944     String &operator=(const String &other);
00945
00946     String &operator+=(const String &other);
00947
00948     String(String &&other) noexcept;
00949     String &operator=(String &&other) noexcept;
00950
00951     char operator[](size_type i) const;
00952     char &operator[](size_type i);
00953
00954     // the only functions I'm willing to leave in the interface - available for inlining
00955     const char *c_str() const {
00956         return const_cast<String *>(this)->c_str(); // NOLINT
00957     }
00958
00959     // NOLINTBEGIN(cppcoreguidelines-pro-type-union-access)
00960     char *c_str() {
00961         if (isOnStack()) {
00962             // NOLINTNEXTLINE(cppcoreguidelines-pro-type-reinterpret-cast)
00963             return reinterpret_cast<char *>(buf);
00964         }
00965         return data.ptr;
00966     }
00967     // NOLINTEND(cppcoreguidelines-pro-type-union-access)
00968
00969     size_type size() const;
00970     size_type capacity() const;
00971
00972     String substr(size_type pos, size_type cnt = npos) &&;
00973     String substr(size_type pos, size_type cnt = npos) const &;
00974
00975     size_type find(char ch, size_type pos = 0) const;
00976     size_type rfind(char ch, size_type pos = npos) const;
00977
00978     int compare(const char *other, bool no_case = false) const;
00979     int compare(const String &other, bool no_case = false) const;
00980
00981     friend DOCTEST_INTERFACE std::ostream &operator<<(std::ostream &s, const String &in);
00982 };
00983
00984 DOCTEST_INTERFACE String operator+(const String &lhs, const String &rhs);
00985
00986 DOCTEST_INTERFACE bool operator==(const String &lhs, const String &rhs);
00987 DOCTEST_INTERFACE bool operator!=(const String &lhs, const String &rhs);
00988 DOCTEST_INTERFACE bool operator<(const String &lhs, const String &rhs);
00989 DOCTEST_INTERFACE bool operator>(const String &lhs, const String &rhs);
00990 DOCTEST_INTERFACE bool operator<=(const String &lhs, const String &rhs);
00991 DOCTEST_INTERFACE bool operator>=(const String &lhs, const String &rhs);
00992
00993 namespace detail {
00994
00995     // MSVS 2015 :(
00996     #if !DOCTEST_CLANG && defined(_MSC_VER) && _MSC_VER <= 1900
00997     template <typename T, typename = void>
00998     struct has_global_insertion_operator : types::false_type {};
00999
01000     template <typename T>
01001     struct has_global_insertion_operator<T, decltype(::operator<<(declval<std::ostream &>(), declval<const
T &>()), void()))>
01002         : types::true_type {};
01003
01004     template <typename T, typename = void>
01005     struct has_insertion_operator {
01006         static DOCTEST_CONSTEXPR bool value = has_global_insertion_operator<T>::value;
01007     };
01008
01009     template <typename T, bool global>
01010     struct insert_hack;
01011
01012     template <typename T>
01013     struct insert_hack<T, true> {
01014         static void insert(std::ostream &os, const T &t) {

```

```

01015         ::operator«(os, t);
01016     }
01017 };
01018
01019 template <typename T>
01020 struct insert_hack<T, false> {
01021     static void insert(std::ostream &os, const T &t) {
01022         operator«(os, t);
01023     }
01024 };
01025
01026 template <typename T>
01027 using insert_hack_t = insert_hack<T, has_global_insertion_operator<T>::value>;
01028 #else
01029 template <typename T, typename = void>
01030 struct has_insertion_operator : types::false_type {};
01031 #endif
01032
01033 template <typename T>
01034 struct has_insertion_operator<T, decltype(operator«(declval<std::ostream &>()), declval<const T &>()),
    void())>
01035     : types::true_type {};
01036
01037 template <typename T>
01038 struct should_stringify_as_underlying_type {
01039     static DOCTEST_CONSTEXPR bool value =
01040         detail::types::is_enum<T>::value && !doctest::detail::has_insertion_operator<T>::value;
01041 };
01042
01043 template <typename T, typename = void>
01044 struct is_pair : types::false_type {};
01045
01046 template <typename T>
01047 struct is_pair<
01048     T,
01049     types::void_t<
01050         typename T::first_type,
01051         typename T::second_type,
01052         decltype(declval<T>().first),
01053         decltype(declval<T>().second)> : types::true_type {};
01054
01055 template <typename T, typename = void>
01056 struct is_container : types::false_type {};
01057
01058 template <typename T>
01059 struct is_container<
01060     T,
01061     types::void_t<
01062         typename T::value_type,
01063         typename T::iterator,
01064         decltype(declval<T>().begin()),
01065         decltype(declval<T>().end())> : types::true_type {};
01066
01067 DOCTEST_INTERFACE std::ostream *tlssPush();
01068 DOCTEST_INTERFACE String tlssPop();
01069
01070 template <bool C>
01071 struct StringMakerBase {
01072     template <typename T>
01073     static String convert(const DOCTEST_REF_WRAP(T)) {
01074         #ifdef DOCTEST_CONFIG_REQUIRE_STRINGIFICATION_FOR_ALL_USED_TYPES
01075             static_assert(deferred_false<T>::value, "No stringification detected for type T. See string
conversion manual");
01076         #endif
01077         return "{?}";
01078     }
01079 };
01080
01081 template <typename T, typename Enable = void>
01082 struct filldata;
01083
01084 template <typename T>
01085 void filloss(std::ostream *stream, const T &in) {
01086     filldata<T>::fill(stream, in);
01087 }
01088
01089 template <typename T, size_t N>
01090 void filloss(std::ostream *stream, const T (&in)[N]) { // NOLINT(*-avoid-c-arrays)
01091     // T[N], T(&)[N], T(&&)[N] have same behaviour.
01092     // Hence remove reference.
01093     filloss<typename types::remove_reference<decltype(in)>::type>(stream, in);
01094 }
01095
01096 template <typename T>
01097 String toStream(const T &in) {
01098     std::ostream *stream = tlssPush();
01099     filloss(stream, in);

```

```

01100     return tlssPop();
01101 }
01102
01103 template <>
01104 struct StringMakerBase<true> {
01105     template <typename T>
01106     static String convert(const DOCTEST_REF_WRAP(T) in) {
01107         return toString(in);
01108     }
01109 };
01110
01111 } // namespace detail
01112
01113 template <typename T>
01114 struct StringMaker
01115     : public detail::StringMakerBase<
01116         detail::has_insertion_operator<T>::value || detail::types::is_pointer<T>::value ||
01117         detail::types::is_array<T>::value || detail::is_pair<T>::value ||
01118         detail::is_container<T>::value> {} {
01119 #ifndef DOCTEST_STRINGIFY
01120 #ifdef DOCTEST_CONFIG_DOUBLE_STRINGIFY
01121 #define DOCTEST_STRINGIFY(...) toString(toString(__VA_ARGS__))
01122 #else
01123 #define DOCTEST_STRINGIFY(...) toString(__VA_ARGS__)
01124 #endif
01125 #endif
01126
01127 template <typename T>
01128 String toString() {
01129     #if DOCTEST_CLANG == 0 && DOCTEST_GCC == 0 && DOCTEST_ICC == 0
01130         String ret = __FUNCSIG__; // class doctest::String __cdecl doctest::toString<TYPE>(void)
01131         String::size_type beginPos = ret.find('<');
01132         return ret.substr(beginPos + 1, ret.size() - beginPos -
01133             static_cast<String::size_type>(sizeof(">(void)")));
01134     #else
01135         const String ret = __PRETTY_FUNCTION__; // doctest::String toString() [with T = TYPE]
01136         const String::size_type begin = ret.find('=') + 2;
01137         return ret.substr(begin, ret.size() - begin - 1);
01138     #endif // Compiler
01139 }
01140
01141 template <
01142     typename T,
01143     typename detail::types::enable_if<!detail::should_stringify_as_underlying_type<T>::value,
01144     bool>::type = true>
01145 String toString(const DOCTEST_REF_WRAP(T) value) {
01146     return StringMaker<T>::convert(value);
01147 }
01148
01149 // NOLINTNEXTLINE(cppcoreguidelines-rvalue-reference-param-not-moved)
01150 inline String &&toString(String &&in) {
01151     return static_cast<String &&>(in);
01152 }
01153
01154 #ifdef DOCTEST_CONFIG_TREAT_CHAR_STAR_AS_STRING
01155 DOCTEST_INTERFACE String toString(const char *in);
01156 #endif // DOCTEST_CONFIG_TREAT_CHAR_STAR_AS_STRING
01157
01158 #if DOCTEST_MSVC >= DOCTEST_COMPILER(19, 20, 0)
01159 // see this issue on why this is needed: https://github.com/doctest/doctest/issues/183
01160 DOCTEST_INTERFACE String toString(const std::string &in);
01161 #endif // VS 2019
01162
01163 DOCTEST_INTERFACE String toString(const String &in);
01164
01165 DOCTEST_INTERFACE String toString(std::nullptr_t);
01166
01167 DOCTEST_INTERFACE String toString(bool in);
01168
01169 DOCTEST_INTERFACE String toString(float in);
01170 DOCTEST_INTERFACE String toString(double in);
01171 DOCTEST_INTERFACE String toString(double long in);
01172
01173 DOCTEST_INTERFACE String toString(char in);
01174 DOCTEST_INTERFACE String toString(char signed in);
01175 DOCTEST_INTERFACE String toString(char unsigned in);
01176 DOCTEST_INTERFACE String toString(short in);
01177 DOCTEST_INTERFACE String toString(short unsigned in);
01178 DOCTEST_INTERFACE String toString(signed in);
01179 DOCTEST_INTERFACE String toString(unsigned in);
01180 DOCTEST_INTERFACE String toString(long in);
01181 DOCTEST_INTERFACE String toString(long unsigned in);
01182 DOCTEST_INTERFACE String toString(long long in);
01183 DOCTEST_INTERFACE String toString(long long unsigned in);
01184
01185 template <

```

```

01184     typename T,
01185     typename detail::types::enable_if<detail::should_stringify_as_underlying_type<T>::value,
01186     bool>::type = true>
01187 String toString(const DOCTEST_REF_WRAP(T) value) {
01188     using UT = typename detail::types::underlying_type<T>::type;
01189     return (DOCTEST_STRINGIFY(static_cast<UT>(value)));
01190 }
01191 namespace detail {
01192 template <typename T, typename Enable>
01193 struct filldata {
01194     static void fill(std::ostream *stream, const T &in) {
01195 #if defined(_MSC_VER) && _MSC_VER <= 1900
01196         insert_hack_t<T>::insert(*stream, in);
01197 #else
01198         operator<<(*stream, in);
01199 #endif // _MSV_VER
01200     }
01201 };
01202
01203 DOCTEST_MSVC_SUPPRESS_WARNING_WITH_PUSH(4866)
01204 // NOLINTBEGIN(*-avoid-c-arrays)
01205 template <typename T, size_t N>
01206 struct filldata<T[N]> {
01207     static void fill(std::ostream *stream, const T (&in)[N]) {
01208         *stream << "[";
01209         for (size_t i = 0; i < N; i++) {
01210             if (i != 0) {
01211                 *stream << ", ";
01212             }
01213             *stream << (DOCTEST_STRINGIFY(in[i]));
01214         }
01215         *stream << "]";
01216     }
01217 };
01218 // NOLINTEND(*-avoid-c-arrays)
01219 DOCTEST_MSVC_SUPPRESS_WARNING_POP
01220
01221 // Specialized since we don't want the terminating null byte!
01222 // NOLINTBEGIN(*-avoid-c-arrays)
01223 template <size_t N>
01224 struct filldata<const char[N]> {
01225     static void fill(std::ostream *stream, const char (&in)[N]) {
01226         *stream << String(in, in[N - 1] ? N : N - 1);
01227     } // NOLINT(clang-analyzer-cplusplus.NewDeleteLeaks)
01228 };
01229 // NOLINTEND(*-avoid-c-arrays)
01230
01231 template <>
01232 struct filldata<const void *> {
01233     DOCTEST_INTERFACE static void fill(std::ostream *stream, const void *in);
01234 };
01235
01236 template <>
01237 struct filldata<const volatile void *> {
01238     DOCTEST_INTERFACE static void fill(std::ostream *stream, const volatile void *in);
01239 };
01240
01241 template <typename T>
01242 struct filldata<T *> {
01243     DOCTEST_MSVC_SUPPRESS_WARNING_WITH_PUSH(4180)
01244     static void fill(std::ostream *stream, const T *in) {
01245         DOCTEST_MSVC_SUPPRESS_WARNING_POP
01246         DOCTEST_CLANG_SUPPRESS_WARNING_WITH_PUSH("-Wmicrosoft-cast")
01247         filldata<const volatile void *>::fill(
01248             stream,
01249 #if DOCTEST_GCC == 0 || DOCTEST_GCC >= DOCTEST_COMPILER(4, 9, 0)
01250             // NOLINTNEXTLINE(cppcoreguidelines-pro-type-reinterpret-cast)
01251             reinterpret_cast<const volatile void *>(in)
01252 #else
01253             // NOLINTNEXTLINE(cppcoreguidelines-pro-type-reinterpret-cast)
01254             *reinterpret_cast<const volatile void *const *>(&in)
01255 #endif // DOCTEST_GCC
01256         );
01257         DOCTEST_CLANG_SUPPRESS_WARNING_POP
01258     }
01259 };
01260
01261 template <typename T>
01262 struct filldata<
01263     T,
01264     typename detail::types::enable_if<!detail::has_insertion_operator<T>::value &&
01265     detail::is_pair<T>::value>::type> {
01266     static void fill(std::ostream *stream, const T &in) {
01267         *stream << "{" << DOCTEST_STRINGIFY(in.first) << ", " << DOCTEST_STRINGIFY(in.second) << "}";
01268     }
01269 };

```



```

01269
01270 template <typename T>
01271 struct filldata<
01272     T,
01273     typename detail::types::enable_if<
01274         !detail::has_insertion_operator<T>::value && detail::is_container<T>::value>::type> {
01275     static void fill(std::ostream *stream, const DOCTEST_REF_WRAP(T) in) {
01276         *stream << "{";
01277         for (auto it = in.begin(); it != in.end(); ++it) {
01278             if (it != in.begin()) {
01279                 *stream << ", ";
01280             }
01281             *stream << DOCTEST_STRINGIFY(*it);
01282         }
01283         *stream << "}";
01284     }
01285 };
01286
01287 #ifndef DOCTEST_CONFIG_DISABLE
01288 template <typename L, typename R>
01289 String stringifyBinaryExpr(const DOCTEST_REF_WRAP(L) lhs, const char *op, const DOCTEST_REF_WRAP(R)
    rhs) {
01290     return (DOCTEST_STRINGIFY(lhs)) + op + (DOCTEST_STRINGIFY(rhs));
01291 }
01292 #endif // DOCTEST_CONFIG_DISABLE
01293 } // namespace detail
01294
01295 } // namespace doctest
01296
01297 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_POP
01298
01299 #endif // DOCTEST_PARTS_PUBLIC_STRING
01300 #ifndef DOCTEST_PARTS_PUBLIC_MATCHERS_CONTAINS
01301 #define DOCTEST_PARTS_PUBLIC_MATCHERS_CONTAINS
01302
01303
01304 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_PUSH
01305
01306 namespace doctest {
01307
01308 class DOCTEST_INTERFACE Contains {
01309 public:
01310     explicit Contains(const String &string);
01311
01312     bool checkWith(const String &other) const;
01313
01314     String string;
01315 };
01316
01317 DOCTEST_INTERFACE String toString(const Contains &in);
01318
01319 DOCTEST_INTERFACE bool operator==(const String &lhs, const Contains &rhs);
01320 DOCTEST_INTERFACE bool operator==(const Contains &lhs, const String &rhs);
01321 DOCTEST_INTERFACE bool operator!=(const String &lhs, const Contains &rhs);
01322 DOCTEST_INTERFACE bool operator!=(const Contains &lhs, const String &rhs);
01323
01324 } // namespace doctest
01325
01326 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_POP
01327
01328 #endif // DOCTEST_PARTS_PUBLIC_MATCHERS_CONTAINS
01329 #ifndef DOCTEST_PARTS_PUBLIC_MATCHERS_APPROX
01330 #define DOCTEST_PARTS_PUBLIC_MATCHERS_APPROX
01331
01332
01333 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_PUSH
01334
01335 namespace doctest {
01336
01337 struct DOCTEST_INTERFACE Approx {
01338     Approx(double value);
01339
01340     Approx operator()(double value) const;
01341
01342 #ifdef DOCTEST_CONFIG_INCLUDE_TYPE_TRAITS
01343     template <typename T>
01344     explicit Approx(
01345         const T &value,
01346         typename detail::types::enable_if<std::is_constructible<double, T>::value>::type * =
01347             static_cast<T *>(nullptr)
01348     ) {
01349         *this = static_cast<double>(value);
01350     }
01351 #endif // DOCTEST_CONFIG_INCLUDE_TYPE_TRAITS
01352
01353     Approx &epsilon(double newEpsilon);
01354
01355

```

```

01354 #ifdef DOCTEST_CONFIG_INCLUDE_TYPE_TRAITS
01355     template <typename T>
01356     typename std::enable_if<std::is_constructible<double, T::value, Approx &>::type epsilon(const T
&newEpsilon) {
01357         m_epsilon = static_cast<double>(newEpsilon);
01358         return *this;
01359     }
01360 #endif // DOCTEST_CONFIG_INCLUDE_TYPE_TRAITS
01361
01362     Approx &scale(double newScale);
01363
01364 #ifdef DOCTEST_CONFIG_INCLUDE_TYPE_TRAITS
01365     template <typename T>
01366     typename std::enable_if<std::is_constructible<double, T::value, Approx &>::type scale(const T
&newScale) {
01367         m_scale = static_cast<double>(newScale);
01368         return *this;
01369     }
01370 #endif // DOCTEST_CONFIG_INCLUDE_TYPE_TRAITS
01371
01372     // clang-format off
01373     DOCTEST_INTERFACE friend bool operator==(double lhs, const Approx &rhs);
01374     DOCTEST_INTERFACE friend bool operator==(const Approx &lhs, double rhs);
01375     DOCTEST_INTERFACE friend bool operator!=(double lhs, const Approx &rhs);
01376     DOCTEST_INTERFACE friend bool operator!=(const Approx &lhs, double rhs);
01377     DOCTEST_INTERFACE friend bool operator<=(double lhs, const Approx &rhs);
01378     DOCTEST_INTERFACE friend bool operator<=(const Approx &lhs, double rhs);
01379     DOCTEST_INTERFACE friend bool operator>=(double lhs, const Approx &rhs);
01380     DOCTEST_INTERFACE friend bool operator>=(const Approx &lhs, double rhs);
01381     DOCTEST_INTERFACE friend bool operator< (double lhs, const Approx &rhs);
01382     DOCTEST_INTERFACE friend bool operator< (const Approx &lhs, double rhs);
01383     DOCTEST_INTERFACE friend bool operator> (double lhs, const Approx &rhs);
01384     DOCTEST_INTERFACE friend bool operator> (const Approx &lhs, double rhs);
01385     // clang-format on
01386
01387 #ifdef DOCTEST_CONFIG_INCLUDE_TYPE_TRAITS
01388 #define DOCTEST_APPROX_PREFIX
01389 \
    template <typename T>
01390 \
    friend typename std::enable_if<std::is_constructible<double, T::value, bool>::type
01391
01392     // clang-format off
01393     DOCTEST_APPROX_PREFIX operator==(const T &lhs, const Approx &rhs) { return
operator==(static_cast<double>(lhs), rhs); }
01394     DOCTEST_APPROX_PREFIX operator==(const Approx &lhs, const T &rhs) { return operator==(rhs, lhs); }
01395     DOCTEST_APPROX_PREFIX operator!=(const T &lhs, const Approx &rhs) { return !operator==(lhs, rhs); }
01396     DOCTEST_APPROX_PREFIX operator!=(const Approx &lhs, const T &rhs) { return !operator==(rhs, lhs); }
01397     DOCTEST_APPROX_PREFIX operator<=(const T &lhs, const Approx &rhs) { return
static_cast<double>(lhs) < rhs.m_value || lhs == rhs; }
01398     DOCTEST_APPROX_PREFIX operator<=(const Approx &lhs, const T &rhs) { return lhs.m_value <
static_cast<double>(rhs) || lhs == rhs; }
01399     DOCTEST_APPROX_PREFIX operator>=(const T &lhs, const Approx &rhs) { return
static_cast<double>(lhs) > rhs.m_value || lhs == rhs; }
01400     DOCTEST_APPROX_PREFIX operator>=(const Approx &lhs, const T &rhs) { return lhs.m_value >
static_cast<double>(rhs) || lhs == rhs; }
01401     DOCTEST_APPROX_PREFIX operator< (const T &lhs, const Approx &rhs) { return
static_cast<double>(lhs) < rhs.m_value && lhs != rhs; }
01402     DOCTEST_APPROX_PREFIX operator< (const Approx &lhs, const T &rhs) { return lhs.m_value <
static_cast<double>(rhs) && lhs != rhs; }
01403     DOCTEST_APPROX_PREFIX operator> (const T &lhs, const Approx &rhs) { return
static_cast<double>(lhs) > rhs.m_value && lhs != rhs; }
01404     DOCTEST_APPROX_PREFIX operator> (const Approx &lhs, const T &rhs) { return lhs.m_value >
static_cast<double>(rhs) && lhs != rhs; }
01405     // clang-format on
01406 #undef DOCTEST_APPROX_PREFIX
01407 #endif // DOCTEST_CONFIG_INCLUDE_TYPE_TRAITS
01408
01409     double m_epsilon;
01410     double m_scale;
01411     double m_value;
01412 };
01413
01414 DOCTEST_INTERFACE String toString(const Approx &in);
01415
01416 } // namespace doctest
01417
01418 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_POP
01419
01420 #endif // DOCTEST_PARTS_PUBLIC_MATCHERS_APPROX
01421 #ifndef DOCTEST_PARTS_PUBLIC_MATCHERS_IS_NAN
01422 #define DOCTEST_PARTS_PUBLIC_MATCHERS_IS_NAN
01423
01424
01425 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_PUSH

```

```

01426
01427 namespace doctest {
01428
01429 template <typename F>
01430 struct DOCTEST_INTERFACE_DECL IsNaN {
01431     F value;
01432     bool flipped;
01433     IsNaN(F f, bool flip = false)
01434         : value(f), flipped(flip) {}
01435
01436     IsNaN<F> operator!() const {
01437         return {value, !flipped};
01438     }
01439
01440     operator bool() const;
01441 };
01442
01443 #ifndef __MINGW32__
01444 extern template struct DOCTEST_INTERFACE_DECL IsNaN<float>;
01445 extern template struct DOCTEST_INTERFACE_DECL IsNaN<double>;
01446 extern template struct DOCTEST_INTERFACE_DECL IsNaN<long double>;
01447 #endif
01448
01449 DOCTEST_INTERFACE String toString(IsNaN<float> in);
01450 DOCTEST_INTERFACE String toString(IsNaN<double> in);
01451 DOCTEST_INTERFACE String toString(IsNaN<double long> in);
01452
01453 } // namespace doctest
01454
01455 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_POP
01456
01457 #endif // DOCTEST_PARTS_PUBLIC_MATCHERS_IS_NAN
01458 #ifndef DOCTEST_PARTS_PUBLIC_CONTEXT_OPTIONS
01459 #define DOCTEST_PARTS_PUBLIC_CONTEXT_OPTIONS
01460
01461 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_PUSH
01462
01463 namespace doctest {
01464 namespace detail {
01465 struct DOCTEST_INTERFACE TestCase;
01466 } // namespace detail
01467
01468 struct ContextOptions {
01469     std::ostream *cout = nullptr; // stdout stream
01470     String binary_name;           // the test binary name
01471
01472     const detail::TestCase *currentTest = nullptr;
01473
01474     // == parameters from the command line
01475     String out;           // output filename
01476     String order_by;      // how tests should be ordered
01477     unsigned rand_seed;   // the seed for rand ordering
01478
01479     unsigned first; // the first (matching) test to be executed
01480     unsigned last;  // the last (matching) test to be executed
01481
01482     int abort_after; // stop tests after this many failed assertions
01483     int subcase_filter_levels; // apply the subcase filters for the first N levels
01484
01485     bool success; // include successful assertions in output
01486     bool case_sensitive; // if filtering should be case sensitive
01487     bool exit; // if the program should be exited after the tests are ran/whatever
01488     bool duration; // print the time duration of each test case
01489     bool minimal; // minimal console output (only test failures)
01490     bool quiet; // no console output
01491     bool no_throw; // to skip exceptions-related assertion macros
01492     bool no_exitcode; // if the framework should return 0 as the exitcode
01493     bool no_run; // to not run the tests at all (can be done with an "*" exclude)
01494     bool no_intro; // to not print the intro of the framework
01495     bool no_version; // to not print the version of the framework
01496     bool no_colors; // if output to the console should be colorized
01497     bool force_colors; // forces the use of colors even when a tty cannot be detected
01498     bool no_breaks; // to not break into the debugger
01499     bool no_skip; // don't skip test cases which are marked to be skipped
01500     bool gnu_file_line; // if line numbers should be surrounded with :x: and not (x):
01501     bool no_path_in_filenames; // if the path to files should be removed from the output
01502     String strip_file_prefixes; // remove the longest matching one of these prefixes from any file
01503     paths in the output
01504     bool no_line_numbers; // if source code line numbers should be omitted from the output
01505     bool no_debug_output; // no output in the debug console when a debugger is attached
01506     bool no_skipped_summary; // don't print "skipped" in the summary !!! UNDOCUMENTED !!!
01507     bool no_time_in_output; // omit any time/timestamps from output !!! UNDOCUMENTED !!!
01508
01509     bool help; // to print the help
01510     bool version; // to print the version
01511     bool count; // if only the count of matching tests is to be retrieved

```

```

01512     bool list_test_cases; // to list all tests matching the filters
01513     bool list_test_suites; // to list all suites matching the filters
01514     bool list_reporters; // lists all registered reporters
01515 };
01516
01517 DOCTEST_INTERFACE const ContextOptions *getContextOptions();
01518
01519 } // namespace doctest
01520
01521 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_POP
01522
01523 #endif // DOCTEST_PARTS_PUBLIC_CONTEXT_OPTIONS
01524 #ifndef DOCTEST_PARTS_PUBLIC_ASSERT_TYPE
01525 #define DOCTEST_PARTS_PUBLIC_ASSERT_TYPE
01526
01527 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_PUSH
01528
01529 namespace doctest {
01530     namespace assertType {
01531         enum Enum {
01532             // macro traits
01533
01534             is_warn = 1,
01535             is_check = 2 * is_warn,
01536             is_require = 2 * is_check,
01537
01538             is_normal = 2 * is_require,
01539             is_throws = 2 * is_normal,
01540             is_throws_as = 2 * is_throws,
01541             is_throws_with = 2 * is_throws_as,
01542             is_nothrow = 2 * is_throws_with,
01543
01544             is_false = 2 * is_nothrow,
01545             is_unary = 2 * is_false, // not checked anywhere - used just to distinguish the types
01546
01547             is_eq = 2 * is_unary,
01548             is_ne = 2 * is_eq,
01549
01550             is_lt = 2 * is_ne,
01551             is_gt = 2 * is_lt,
01552
01553             is_ge = 2 * is_gt,
01554             is_le = 2 * is_ge,
01555
01556             // macro types
01557
01558             DT_WARN = is_normal | is_warn,
01559             DT_CHECK = is_normal | is_check,
01560             DT_REQUIRE = is_normal | is_require,
01561
01562             DT_WARN_FALSE = is_normal | is_false | is_warn,
01563             DT_CHECK_FALSE = is_normal | is_false | is_check,
01564             DT_REQUIRE_FALSE = is_normal | is_false | is_require,
01565
01566             DT_WARN_THROWS = is_throws | is_warn,
01567             DT_CHECK_THROWS = is_throws | is_check,
01568             DT_REQUIRE_THROWS = is_throws | is_require,
01569
01570             DT_WARN_THROWS_AS = is_throws_as | is_warn,
01571             DT_CHECK_THROWS_AS = is_throws_as | is_check,
01572             DT_REQUIRE_THROWS_AS = is_throws_as | is_require,
01573
01574             DT_WARN_THROWS_WITH = is_throws_with | is_warn,
01575             DT_CHECK_THROWS_WITH = is_throws_with | is_check,
01576             DT_REQUIRE_THROWS_WITH = is_throws_with | is_require,
01577
01578             DT_WARN_THROWS_WITH_AS = is_throws_with | is_throws_as | is_warn,
01579             DT_CHECK_THROWS_WITH_AS = is_throws_with | is_throws_as | is_check,
01580             DT_REQUIRE_THROWS_WITH_AS = is_throws_with | is_throws_as | is_require,
01581
01582             DT_WARN_NOTHROW = is_nothrow | is_warn,
01583             DT_CHECK_NOTHROW = is_nothrow | is_check,
01584             DT_REQUIRE_NOTHROW = is_nothrow | is_require,
01585
01586             DT_WARN_EQ = is_normal | is_eq | is_warn,
01587             DT_CHECK_EQ = is_normal | is_eq | is_check,
01588             DT_REQUIRE_EQ = is_normal | is_eq | is_require,
01589
01590             DT_WARN_NE = is_normal | is_ne | is_warn,
01591             DT_CHECK_NE = is_normal | is_ne | is_check,
01592             DT_REQUIRE_NE = is_normal | is_ne | is_require,
01593
01594             DT_WARN_GT = is_normal | is_gt | is_warn,
01595             DT_CHECK_GT = is_normal | is_gt | is_check,
01596             DT_REQUIRE_GT = is_normal | is_gt | is_require,
01597
01598

```

```

01599     DT_WARN_LT = is_normal | is_lt | is_warn,
01600     DT_CHECK_LT = is_normal | is_lt | is_check,
01601     DT_REQUIRE_LT = is_normal | is_lt | is_require,
01602
01603     DT_WARN_GE = is_normal | is_ge | is_warn,
01604     DT_CHECK_GE = is_normal | is_ge | is_check,
01605     DT_REQUIRE_GE = is_normal | is_ge | is_require,
01606
01607     DT_WARN_LE = is_normal | is_le | is_warn,
01608     DT_CHECK_LE = is_normal | is_le | is_check,
01609     DT_REQUIRE_LE = is_normal | is_le | is_require,
01610
01611     DT_WARN_UNARY = is_normal | is_unary | is_warn,
01612     DT_CHECK_UNARY = is_normal | is_unary | is_check,
01613     DT_REQUIRE_UNARY = is_normal | is_unary | is_require,
01614
01615     DT_WARN_UNARY_FALSE = is_normal | is_false | is_unary | is_warn,
01616     DT_CHECK_UNARY_FALSE = is_normal | is_false | is_unary | is_check,
01617     DT_REQUIRE_UNARY_FALSE = is_normal | is_false | is_unary | is_require,
01618 };
01619 } // namespace assertType
01620
01621 DOCTEST_INTERFACE const char *assertString(assertType::Enum at);
01622 DOCTEST_INTERFACE const char *failureString(assertType::Enum at);
01623
01624 } // namespace doctest
01625
01626 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_POP
01627
01628 #endif // DOCTEST_PARTS_PUBLIC_ASSERT_TYPE
01629 #ifndef DOCTEST_PARTS_PUBLIC_ASSERT_DATA
01630 #define DOCTEST_PARTS_PUBLIC_ASSERT_DATA
01631
01632
01633 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_PUSH
01634
01635 namespace doctest {
01636
01637 struct DOCTEST_INTERFACE TestCaseData;
01638
01639 struct DOCTEST_INTERFACE AssertData {
01640     // common - for all asserts
01641     const TestCaseData *m_test_case;
01642     assertType::Enum m_at;
01643     const char *m_file;
01644     int m_line;
01645     const char *m_expr;
01646     bool m_failed;
01647
01648     // exception-related - for all asserts
01649     bool m_threw;
01650     String m_exception;
01651
01652     // for normal asserts
01653     String m_decomp;
01654
01655     // for specific exception-related asserts
01656     bool m_threw_as;
01657     const char *m_exception_type;
01658
01659     class DOCTEST_INTERFACE StringContains {
01660     private:
01661         Contains content;
01662         bool isContains;
01663
01664     public:
01665         StringContains(const String &str)
01666             : content(str), isContains(false) {}
01667
01668         StringContains(Contains cntn)
01669             : content(static_cast<Contains>(&>(cntn)), isContains(true)) {}
01670
01671         bool check(const String &str) {
01672             return isContains ? (content == str) : (content.string == str);
01673         }
01674
01675         operator const String &() const {
01676             return content.string;
01677         }
01678
01679         const char *c_str() const {
01680             return content.string.c_str();
01681         }
01682     } m_exception_string;
01683
01684     AssertData(
01685         assertType::Enum at,

```

```

01686         const char *file,
01687         int line,
01688         const char *expr,
01689         const char *exception_type,
01690         const StringContains &exception_string
01691     );
01692 };
01693
01694 } // namespace doctest
01695
01696 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_POP
01697
01698 #endif // DOCTEST_PARTS_PUBLIC_ASSERT_DATA
01699 #ifndef DOCTEST_PARTS_PUBLIC_ASSERT_COMPARATOR
01700 #define DOCTEST_PARTS_PUBLIC_ASSERT_COMPARATOR
01701
01702 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_PUSH
01703
01704 #ifndef DOCTEST_CONFIG_DISABLE
01705 namespace doctest {
01706 namespace detail {
01707
01708 #ifdef DOCTEST_CONFIG_TREAT_CHAR_STAR_AS_STRING
01709 // clang-format off
01710 template<class T> struct decay_array { using type = T; };
01711 template<class T, unsigned N> struct decay_array<T[N]> { using type = T *; };
01712 template<class T> struct decay_array<T[]> { using type = T *; };
01713
01714 template<class T> struct not_char_pointer { static DOCTEST_CONSTEXPR int value = 1; };
01715 template<> struct not_char_pointer<char *> { static DOCTEST_CONSTEXPR int value = 0; };
01716 template<> struct not_char_pointer<const char *> { static DOCTEST_CONSTEXPR int value = 0; };
01717
01718 template<class T> struct can_use_op : public not_char_pointer<typename decay_array<T>::type> {};
01719 // clang-format on
01720 #endif // DOCTEST_CONFIG_TREAT_CHAR_STAR_AS_STRING
01721
01722 #ifndef DOCTEST_CONFIG_TREAT_CHAR_STAR_AS_STRING
01723 #define DOCTEST_COMPARISON_RETURN_TYPE bool
01724 #else // DOCTEST_CONFIG_TREAT_CHAR_STAR_AS_STRING
01725 // clang-format off
01726 #define DOCTEST_COMPARISON_RETURN_TYPE typename types::enable_if<can_use_op<L>::value ||
01727 can_use_op<R>::value, bool>::type
01728
01729 inline bool eq(const char *lhs, const char *rhs) { return String(lhs) == String(rhs); }
01730 inline bool ne(const char *lhs, const char *rhs) { return String(lhs) != String(rhs); }
01731 inline bool lt(const char *lhs, const char *rhs) { return String(lhs) < String(rhs); }
01732 inline bool gt(const char *lhs, const char *rhs) { return String(lhs) > String(rhs); }
01733 inline bool le(const char *lhs, const char *rhs) { return String(lhs) <= String(rhs); }
01734 inline bool ge(const char *lhs, const char *rhs) { return String(lhs) >= String(rhs); }
01735 // clang-format on
01736 #endif // DOCTEST_CONFIG_TREAT_CHAR_STAR_AS_STRING
01737
01738 #define DOCTEST_RELATIONAL_OP(name, op)
01739 \
01740 \
01741 \
01742 \
01743 \
01744 \
01745 \
01746 \
01747 \
01748 \
01749 \
01750 \
01751 \
01752 \
01753 \
01754 \
01755 \
01756 \
01757 \
01758 \
01759 \
01760 \
01761 \
01762 \
01763 \
01764 \
01765 \
01766 \
01767 \
01768 \
01769 \
01770 \
01771 \
01772 \
01773 \
01774 \
01775 \
01776 \
01777 \
01778 \
01779 \
01780 \
01781 \
01782 \
01783 \
01784 \
01785 \
01786 \
01787 \
01788 \
01789 \
01790 \
01791 \
01792 \
01793 \
01794 \
01795 \
01796 \
01797 \
01798 \
01799 \
01800 \
01801 \
01802 \
01803 \
01804 \
01805 \
01806 \
01807 \
01808 \
01809 \
01810 \
01811 \
01812 \
01813 \
01814 \
01815 \
01816 \
01817 \
01818 \
01819 \
01820 \
01821 \
01822 \
01823 \
01824 \
01825 \
01826 \
01827 \
01828 \
01829 \
01830 \
01831 \
01832 \
01833 \
01834 \
01835 \
01836 \
01837 \
01838 \
01839 \
01840 \
01841 \
01842 \
01843 \
01844 \
01845 \
01846 \
01847 \
01848 \
01849 \
01850 \
01851 \
01852 \
01853 \
01854 \
01855 \
01856 \
01857 \
01858 \
01859 \
01860 \
01861 \
01862 \
01863 \
01864 \
01865 \
01866 \
01867 \
01868 \
01869 \
01870 \
01871 \
01872 \
01873 \
01874 \
01875 \
01876 \
01877 \
01878 \
01879 \
01880 \
01881 \
01882 \
01883 \
01884 \
01885 \
01886 \
01887 \
01888 \
01889 \
01890 \
01891 \
01892 \
01893 \
01894 \
01895 \
01896 \
01897 \
01898 \
01899 \
01900 \
01901 \
01902 \
01903 \
01904 \
01905 \
01906 \
01907 \
01908 \
01909 \
01910 \
01911 \
01912 \
01913 \
01914 \
01915 \
01916 \
01917 \
01918 \
01919 \
01920 \
01921 \
01922 \
01923 \
01924 \
01925 \
01926 \
01927 \
01928 \
01929 \
01930 \
01931 \
01932 \
01933 \
01934 \
01935 \
01936 \
01937 \
01938 \
01939 \
01940 \
01941 \
01942 \
01943 \
01944 \
01945 \
01946 \
01947 \
01948 \
01949 \
01950 \
01951 \
01952 \
01953 \
01954 \
01955 \
01956 \
01957 \
01958 \
01959 \
01960 \
01961 \
01962 \
01963 \
01964 \
01965 \
01966 \
01967 \
01968 \
01969 \
01970 \
01971 \
01972 \
01973 \
01974 \
01975 \
01976 \
01977 \
01978 \
01979 \
01980 \
01981 \
01982 \
01983 \
01984 \
01985 \
01986 \
01987 \
01988 \
01989 \
01990 \
01991 \
01992 \
01993 \
01994 \
01995 \
01996 \
01997 \
01998 \
01999 \

```

```

01768 enum Enum { eq = 0, ne, gt, lt, ge, le };
01769 } // namespace binaryAssertComparison
01770
01771 // clang-format off
01772 template <int, class L, class R> struct RelationalComparator { bool operator()(const
DOCTEST_REF_WRAP(L), const DOCTEST_REF_WRAP(R) ) const { return false; } };
01773
01774 #define DOCTEST_BINARY_RELATIONAL_OP(n, op) \
01775     template <class L, class R> struct RelationalComparator<n, L, R> { bool operator()(const
DOCTEST_REF_WRAP(L) lhs, const DOCTEST_REF_WRAP(R) rhs) const { return op(lhs, rhs); } };
01776 // clang-format on
01777
01778 DOCTEST_BINARY_RELATIONAL_OP(0, doctest::detail::eq)
01779 DOCTEST_BINARY_RELATIONAL_OP(1, doctest::detail::ne)
01780 DOCTEST_BINARY_RELATIONAL_OP(2, doctest::detail::gt)
01781 DOCTEST_BINARY_RELATIONAL_OP(3, doctest::detail::lt)
01782 DOCTEST_BINARY_RELATIONAL_OP(4, doctest::detail::ge)
01783 DOCTEST_BINARY_RELATIONAL_OP(5, doctest::detail::le)
01784
01785 } // namespace detail
01786 } // namespace doctest
01787
01788 #endif // DOCTEST_CONFIG_DISABLE
01789
01790 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_POP
01791
01792 #endif // DOCTEST_PARTS_PUBLIC_ASSERT_COMPARATOR
01793 #ifndef DOCTEST_PARTS_PUBLIC_ASSERT_RESULT
01794 #define DOCTEST_PARTS_PUBLIC_ASSERT_RESULT
01795
01796
01797 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_PUSH
01798
01799 #ifndef DOCTEST_CONFIG_DISABLE
01800
01801 namespace doctest {
01802 namespace detail {
01803
01804 // more checks could be added - like in Catch:
01805 // https://github.com/catchorg/Catch2/pull/1480/files
01806 // https://github.com/catchorg/Catch2/pull/1481/files
01807 #define DOCTEST_FORBIT_EXPRESSION(rt, op)
01808 \
    template <typename R>
01809 \
    rt &operator op(const R &) {
01810 \
        static_assert(deferred_false<R>::value, "Expression Too Complex Please Rewrite As Binary
Comparison!");
01811 \
        return *this;
01812 \
    }
01813
01814 struct DOCTEST_INTERFACE Result { // NOLINT(*-member-init)
01815     bool m_passed;
01816     String m_decomp;
01817
01818     Result() = default; // TODO: Why do we need this? (To remove NOLINT)
01819     Result(bool passed, const String &decomposition = String());
01820
01821     // forbidding some expressions based on this table:
01822     // https://en.cppreference.com/w/cpp/language/operator_precedence
01823     DOCTEST_FORBIT_EXPRESSION(Result, &)
01824     DOCTEST_FORBIT_EXPRESSION(Result, ^)
01825     DOCTEST_FORBIT_EXPRESSION(Result, |)
01826     DOCTEST_FORBIT_EXPRESSION(Result, &&)
01827     DOCTEST_FORBIT_EXPRESSION(Result, ||)
01828     DOCTEST_FORBIT_EXPRESSION(Result, ==)
01829     DOCTEST_FORBIT_EXPRESSION(Result, !=)
01830     DOCTEST_FORBIT_EXPRESSION(Result, <)
01831     DOCTEST_FORBIT_EXPRESSION(Result, >)
01832     DOCTEST_FORBIT_EXPRESSION(Result, <=)
01833     DOCTEST_FORBIT_EXPRESSION(Result, >=)
01834     DOCTEST_FORBIT_EXPRESSION(Result, =)
01835     DOCTEST_FORBIT_EXPRESSION(Result, +=)
01836     DOCTEST_FORBIT_EXPRESSION(Result, -=)
01837     DOCTEST_FORBIT_EXPRESSION(Result, *=)
01838     DOCTEST_FORBIT_EXPRESSION(Result, /=)
01839     DOCTEST_FORBIT_EXPRESSION(Result, %=)
01840     DOCTEST_FORBIT_EXPRESSION(Result, <=)
01841     DOCTEST_FORBIT_EXPRESSION(Result, >=)
01842     DOCTEST_FORBIT_EXPRESSION(Result, &=)
01843     DOCTEST_FORBIT_EXPRESSION(Result, ^=)
01844     DOCTEST_FORBIT_EXPRESSION(Result, |=)
01845 };
01846
01847 struct DOCTEST_INTERFACE ResultBuilder : public AssertData {

```

```

01848     ResultBuilder(
01849         assertType::Enum at,
01850         const char *file,
01851         int line,
01852         const char *expr,
01853         const char *exception_type = "",
01854         const String &exception_string = ""
01855     );
01856
01857     ResultBuilder(
01858         assertType::Enum at,
01859         const char *file,
01860         int line,
01861         const char *expr,
01862         const char *exception_type,
01863         const Contains &exception_string
01864     );
01865
01866     void setResult(const Result &res);
01867
01868     template <int comparison, typename L, typename R>
01869     DOCTEST_NOINLINE bool binary_assert(const DOCTEST_REF_WRAP(L) lhs, const DOCTEST_REF_WRAP(R) rhs)
01870     {
01871         m_failed = !RelationalComparator<comparison, L, R>()(lhs, rhs);
01872         if (m_failed || getContextOptions()->success) {
01873             m_decomp = stringifyBinaryExpr(lhs, " ", rhs);
01874         }
01875         return !m_failed;
01876     }
01877
01878     template <typename L>
01879     DOCTEST_NOINLINE bool unary_assert(const DOCTEST_REF_WRAP(L) val) {
01880         m_failed = !val;
01881
01882         if (m_at & assertType::is_false) {
01883             m_failed = !m_failed;
01884         }
01885
01886         if (m_failed || getContextOptions()->success) {
01887             m_decomp = (DOCTEST_STRINGIFY(val));
01888         }
01889
01890         return !m_failed;
01891     }
01892
01893     void translateException();
01894
01895     bool log();
01896     void react() const;
01897 };
01898 } // namespace detail
01899 } // namespace doctest
01900
01901 #endif // DOCTEST_CONFIG_DISABLE
01902
01903 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_POP
01904
01905 #endif // DOCTEST_PARTS_PUBLIC_ASSERT_RESULT
01906 #ifndef DOCTEST_PARTS_PUBLIC_ASSERT_EXPRESSION
01907 #define DOCTEST_PARTS_PUBLIC_ASSERT_EXPRESSION
01908
01909 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_PUSH
01910
01911 #ifndef DOCTEST_CONFIG_DISABLE
01912
01913 namespace doctest {
01914 namespace detail {
01915
01916 #if DOCTEST_CLANG && DOCTEST_CLANG < DOCTEST_COMPILER(3, 6, 0)
01917 DOCTEST_CLANG_SUPPRESS_WARNING_WITH_PUSH("-Wunused-comparison")
01918 #endif
01919
01920 // This will check if there is any way it could find a operator like member or friend and uses it.
01921 // If not it doesn't find the operator or if the operator at global scope is defined after
01922 // this template, the template won't be instantiated due to SFINAE. Once the template is not
01923 // instantiated it can look for global operator using normal conversions.
01924 #ifndef __NVCC__
01925 #define SFINAE_OP(ret, op) ret
01926 #else
01927 #define SFINAE_OP(ret, op) decltype((void)(doctest::detail::declval<L>()) op
01928 doctest::detail::declval<R>()), ret{})
01929 #endif
01930
01931 #define DOCTEST_DO_BINARY_EXPRESSION_COMPARISON(op, op_str, op_macro)

```



```

01932     /* NOLINTBEGIN(cppcoreguidelines-missing-std-forward) */
01933     \
01934     template <typename R>
01935     \
01936     DOCTEST_NOINLINE SFINAE_OP(Result, op) operator op(R &&rhs) {
01937     \
01938         bool res = op_macro(doctest::detail::forward<const L>(lhs), doctest::detail::forward<R>(rhs));
01939     \
01940         if (m_at & assertType::is_false)
01941     \
01942             res = !res;
01943     \
01944         if (!res || doctest::getContextOptions()->success)
01945     \
01946             return Result(res, stringifyBinaryExpr(lhs, op_str, rhs));
01947     \
01948         return Result(res);
01949     \
01950     }
01951     \
01952     /* NOLINTEND(cppcoreguidelines-missing-std-forward) */
01953
01954     #ifndef DOCTEST_CONFIG_NO_COMPARISON_WARNING_SUPPRESSION
01955
01956     DOCTEST_CLANG_SUPPRESS_WARNING_PUSH
01957     DOCTEST_CLANG_SUPPRESS_WARNING("-Wsign-conversion")
01958     DOCTEST_CLANG_SUPPRESS_WARNING("-Wsign-compare")
01959     // DOCTEST_CLANG_SUPPRESS_WARNING("-Wdouble-promotion")
01960     // DOCTEST_CLANG_SUPPRESS_WARNING("-Wconversion")
01961     // DOCTEST_CLANG_SUPPRESS_WARNING("-Wfloat-equal")
01962
01963     DOCTEST_GCC_SUPPRESS_WARNING_PUSH
01964     DOCTEST_GCC_SUPPRESS_WARNING("-Wsign-conversion")
01965     DOCTEST_GCC_SUPPRESS_WARNING("-Wsign-compare")
01966     // DOCTEST_GCC_SUPPRESS_WARNING("-Wdouble-promotion")
01967     // DOCTEST_GCC_SUPPRESS_WARNING("-Wconversion")
01968     // DOCTEST_GCC_SUPPRESS_WARNING("-Wfloat-equal")
01969
01970     DOCTEST_MSVC_SUPPRESS_WARNING_PUSH
01971     // https://stackoverflow.com/questions/39479163 what's the difference between 4018 and 4389
01972     DOCTEST_MSVC_SUPPRESS_WARNING(4388) // signed/unsigned mismatch
01973     DOCTEST_MSVC_SUPPRESS_WARNING(4389) // 'operator' : signed/unsigned mismatch
01974     DOCTEST_MSVC_SUPPRESS_WARNING(4018) // 'expression' : signed/unsigned mismatch
01975     // DOCTEST_MSVC_SUPPRESS_WARNING(4805) // 'operation' : unsafe mix of type 'type' and type 'type' in
01976     // operation
01977
01978     #endif // DOCTEST_CONFIG_NO_COMPARISON_WARNING_SUPPRESSION
01979
01980     template <typename L>
01981     \
01982     struct Expression_lhs {
01983     \
01984         L lhs;
01985     \
01986         assertType::Enum m_at;
01987     \
01988         // NOLINTNEXTLINE(cppcoreguidelines-rvalue-reference-param-not-moved)
01989         explicit Expression_lhs(L &&in, assertType::Enum at)
01990     \
01991             : lhs(static_cast<L &&>(in)), m_at(at) {}
01992     \
01993         DOCTEST_NOINLINE operator Result() {
01994     \
01995             // this is needed only for MSVC 2015
01996             DOCTEST_MSVC_SUPPRESS_WARNING_WITH_PUSH(4800) // 'int': forcing value to bool
01997             bool res = static_cast<bool>(lhs); //
01998         \
01999         NOLINT(bugprone-non-zero-enum-to-bool-conversion)
02000         DOCTEST_MSVC_SUPPRESS_WARNING_POP
02001         if (m_at & assertType::is_false) {
02002     \
02003             res = !res;
02004     \
02005             if (!res || getContextOptions()->success) {
02006     \
02007                 return {res, (DOCTEST_STRINGIFY(lhs))};
02008             }
02009         \
02010             return {res};
02011         }
02012     \
02013     }
02014
02015     /* This is required for user-defined conversions from Expression_lhs to L */
02016     operator L() const {
02017     \
02018         return lhs;
02019     \
02020     }
02021
02022     // clang-format off
02023     DOCTEST_DO_BINARY_EXPRESSION_COMPARISON(==, " == ", DOCTEST_CMP_EQ)
02024     DOCTEST_DO_BINARY_EXPRESSION_COMPARISON(!=, " != ", DOCTEST_CMP_NE)
02025     DOCTEST_DO_BINARY_EXPRESSION_COMPARISON(>, " > ", DOCTEST_CMP_GT)
02026     DOCTEST_DO_BINARY_EXPRESSION_COMPARISON(<, " < ", DOCTEST_CMP_LT)
02027     DOCTEST_DO_BINARY_EXPRESSION_COMPARISON(>=, " >= ", DOCTEST_CMP_GE)
02028     DOCTEST_DO_BINARY_EXPRESSION_COMPARISON(<=, " <= ", DOCTEST_CMP_LE)
02029     // clang-format on
02030
02031

```

```

02008 // forbidding some expressions based on this table:
02009 // https://en.cppreference.com/w/cpp/language/operator_precedence
02010 DOCTEST_FORBIT_EXPRESSION(Expression_lhs, &)
02011 DOCTEST_FORBIT_EXPRESSION(Expression_lhs, ^)
02012 DOCTEST_FORBIT_EXPRESSION(Expression_lhs, |)
02013 DOCTEST_FORBIT_EXPRESSION(Expression_lhs, &&)
02014 DOCTEST_FORBIT_EXPRESSION(Expression_lhs, ||)
02015 DOCTEST_FORBIT_EXPRESSION(Expression_lhs, =)
02016 DOCTEST_FORBIT_EXPRESSION(Expression_lhs, +=)
02017 DOCTEST_FORBIT_EXPRESSION(Expression_lhs, -=)
02018 DOCTEST_FORBIT_EXPRESSION(Expression_lhs, *=)
02019 DOCTEST_FORBIT_EXPRESSION(Expression_lhs, /=)
02020 DOCTEST_FORBIT_EXPRESSION(Expression_lhs, %=)
02021 DOCTEST_FORBIT_EXPRESSION(Expression_lhs, <=)
02022 DOCTEST_FORBIT_EXPRESSION(Expression_lhs, >=)
02023 DOCTEST_FORBIT_EXPRESSION(Expression_lhs, &=)
02024 DOCTEST_FORBIT_EXPRESSION(Expression_lhs, ^=)
02025 DOCTEST_FORBIT_EXPRESSION(Expression_lhs, |=)
02026 // these 2 are unfortunate because they should be allowed - they have higher precedence over the
02027 // comparisons, but the ExpressionDecomposer class uses the left shift operator to capture the
02028 // left operand of the binary expression...
02029 DOCTEST_FORBIT_EXPRESSION(Expression_lhs, <<)
02030 DOCTEST_FORBIT_EXPRESSION(Expression_lhs, >>)
02031 };
02032
02033 #ifndef DOCTEST_CONFIG_NO_COMPARISON_WARNING_SUPPRESSION
02034
02035 DOCTEST_CLANG_SUPPRESS_WARNING_POP
02036 DOCTEST_MSVC_SUPPRESS_WARNING_POP
02037 DOCTEST_GCC_SUPPRESS_WARNING_POP
02038
02039 #endif // DOCTEST_CONFIG_NO_COMPARISON_WARNING_SUPPRESSION
02040
02041 #if DOCTEST_CLANG && DOCTEST_CLANG < DOCTEST_COMPILER(3, 6, 0)
02042 DOCTEST_CLANG_SUPPRESS_WARNING_POP
02043 #endif
02044
02045 struct DOCTEST_INTERFACE ExpressionDecomposer {
02046     assertType::Enum m_at;
02047
02048     ExpressionDecomposer(assertType::Enum at);
02049
02050     // The right operator for capturing expressions is "<=" instead of "<" (based on the operator
02051     // precedence table) but then there will be warnings from GCC about "-Wparentheses" and since
02052     // "_Pragma()" is problematic this will stay for now...
02053     // https://github.com/catchorg/Catch2/issues/870
02054     // https://github.com/catchorg/Catch2/issues/565
02055     template <typename L>
02056     Expression_lhs<const L &&>
02057     operator<<(const L &&operand) { // bitfields bind to universal ref but not const rvalue ref
02058         return Expression_lhs<const L &&>(static_cast<const L &&>(operand), m_at);
02059     }
02060
02061     template <
02062         typename L,
02063         typename types::enable_if<!doctest::detail::types::is_rvalue_reference<L>::value, void>::type
02064     * = nullptr>
02065     Expression_lhs<const L &> operator<<(const L &operand) {
02066         return Expression_lhs<const L &>(operand, m_at);
02067     };
02068
02069 } // namespace detail
02070 } // namespace doctest
02071
02072 #endif // DOCTEST_CONFIG_DISABLE
02073
02074 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_POP
02075
02076 #endif // DOCTEST_PARTS_PUBLIC_ASSERT_EXPRESSION
02077 #ifndef DOCTEST_PARTS_PUBLIC_COLOR
02078 #define DOCTEST_PARTS_PUBLIC_COLOR
02079
02080
02081 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_PUSH
02082
02083 namespace doctest {
02084 namespace Color {
02085 enum Enum { // NOLINT(cert-int09-c, readability-enum-initial-value)
02086     None = 0,
02087     White,
02088     Red,
02089     Green,
02090     Blue,
02091     Cyan,
02092     Yellow,
02093     Grey,

```

```

02094
02095     Bright = 0x10,
02096
02097     BrightRed = Bright | Red,
02098     BrightGreen = Bright | Green,
02099     LightGrey = Bright | Grey,
02100     BrightWhite = Bright | White
02101 };
02102
02103 DOCTEST_INTERFACE std::ostream &operator<<(std::ostream &s, Color::Enum code);
02104 } // namespace Color
02105 } // namespace doctest
02106
02107 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_POP
02108
02109 #endif // DOCTEST_PARTS_PUBLIC_COLOR
02110 #ifndef DOCTEST_PARTS_PUBLIC_SUBCASE
02111 #define DOCTEST_PARTS_PUBLIC_SUBCASE
02112
02113 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_PUSH
02114
02115 namespace doctest {
02116
02117 struct DOCTEST_INTERFACE SubcaseSignature {
02118     String m_name;
02119     const char *m_file;
02120     int m_line;
02121
02122     bool operator==(const SubcaseSignature &other) const;
02123     bool operator<(const SubcaseSignature &other) const;
02124 };
02125
02126 #ifndef DOCTEST_CONFIG_DISABLE
02127 namespace detail {
02128 struct DOCTEST_INTERFACE Subcase {
02129     SubcaseSignature m_signature;
02130     bool m_entered = false;
02131
02132     Subcase(const String &name, const char *file, int line);
02133     Subcase(const Subcase &) = delete;
02134     Subcase(Subcase &&) = delete;
02135     Subcase &operator=(const Subcase &) = delete;
02136     Subcase &operator=(Subcase &&) = delete;
02137     ~Subcase();
02138
02139     operator bool() const;
02140
02141 private:
02142     bool checkFilters();
02143 };
02144 } // namespace detail
02145 #endif // DOCTEST_CONFIG_DISABLE
02146 } // namespace doctest
02147
02148 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_POP
02149
02150 #endif // DOCTEST_PARTS_PUBLIC_SUBCASE
02151 #ifndef DOCTEST_PARTS_PUBLIC_TEST_SUITE
02152 #define DOCTEST_PARTS_PUBLIC_TEST_SUITE
02153
02154 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_PUSH
02155
02156 #ifndef DOCTEST_CONFIG_DISABLE
02157 namespace doctest {
02158 namespace detail {
02159
02160 struct DOCTEST_INTERFACE TestSuite {
02161     const char *m_test_suite = nullptr;
02162     const char *m_description = nullptr;
02163     bool m_skip = false;
02164     bool m_no_breaks = false;
02165     bool m_no_output = false;
02166     bool m_may_fail = false;
02167     bool m_should_fail = false;
02168     int m_expected_failures = 0;
02169     double m_timeout = 0;
02170
02171     TestSuite &operator*(const char *in) noexcept;
02172
02173     template <typename T>
02174     TestSuite &operator*(const T &in) noexcept {
02175         in.fill(*this);
02176         return *this;
02177     }
02178 }
02179
02180 }

```

```

02181     }
02182 };
02183
02184 // forward declarations of functions used by the macros
02185 DOCTEST_INTERFACE int setTestSuite(const TestSuite &ts) noexcept;
02186
02187 } // namespace detail
02188
02189 } // namespace doctest
02190
02191 // in a separate namespace outside of doctest because the DOCTEST_TEST_SUITE macro
02192 // introduces an anonymous namespace in which getCurrentTestSuite gets overridden
02193 namespace doctest_detail_test_suite_ns {
02194 DOCTEST_INTERFACE doctest::detail::TestSuite &getCurrentTestSuite() noexcept;
02195
02196 // this is here to clear the 'current test suite' for the current translation unit - at the top
02197 DOCTEST_GLOBAL_NO_WARNINGS( /* NOLINT(cert-err58-cpp) */
02198                             DOCTEST_ANONYMOUS(DOCTEST_ANON_VAR_),
02199                             doctest::detail::setTestSuite(doctest::detail::TestSuite() * ""))
02200 )
02201
02202 } // namespace doctest_detail_test_suite_ns
02203
02204 #endif // DOCTEST_CONFIG_DISABLE
02205
02206 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_POP
02207
02208 #endif // DOCTEST_PARTS_PUBLIC_TEST_SUITE
02209 #ifndef DOCTEST_PARTS_PUBLIC_TEST_CASE
02210 #define DOCTEST_PARTS_PUBLIC_TEST_CASE
02211
02212 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_PUSH
02213
02214 namespace doctest {
02215
02216 struct DOCTEST_INTERFACE TestCaseData {
02217     String m_file; // the file in which the test was registered (using String - see #350)
02218     unsigned m_line; // the line where the test was registered
02219     const char *m_name; // name of the test case
02220     const char *m_test_suite; // the test suite in which the test was added
02221     const char *m_description;
02222     bool m_skip;
02223     bool m_no_breaks;
02224     bool m_no_output;
02225     bool m_may_fail;
02226     bool m_should_fail;
02227     int m_expected_failures;
02228     double m_timeout;
02229 };
02230
02231 #ifndef DOCTEST_CONFIG_DISABLE
02232 namespace detail {
02233
02234 using funcType = void (*)();
02235
02236 struct DOCTEST_INTERFACE TestCase : public TestCaseData {
02237     funcType m_test; // a function pointer to the test case
02238
02239     String m_type; // for templated test cases - gets appended to the real name
02240     int m_template_id; // an ID used to distinguish between the different versions of a templated
02241                       // test case
02242     String m_full_name; // contains the name (only for templated test cases!) + the template type
02243
02244     TestCase(
02245         funcType test,
02246         const char *file,
02247         unsigned line,
02248         const TestSuite &test_suite,
02249         const String &type = String(),
02250         int template_id = -1
02251     ) noexcept;
02252
02253     TestCase(const TestCase &other) noexcept;
02254     TestCase(TestCase &&) = delete;
02255
02256     DOCTEST_MSVC_SUPPRESS_WARNING_WITH_PUSH(26434) // hides a non-virtual function
02257     TestCase &operator=(const TestCase &other) noexcept;
02258     DOCTEST_MSVC_SUPPRESS_WARNING_POP
02259
02260     TestCase &operator=(TestCase &&) = delete;
02261
02262     TestCase &operator*(const char *in) noexcept;
02263
02264     template <typename T>
02265     TestCase &operator*(const T &in) noexcept {
02266         in.fill(*this);
02267     }

```

```

02268         return *this;
02269     }
02270
02271     bool operator<(const TestCase &other) const noexcept;
02272
02273     ~TestCase() = default;
02274 };
02275
02276 // forward declarations of functions used by the macros
02277 DOCTEST_INTERFACE int regTest(const TestCase &tc) noexcept;
02278
02279 } // namespace detail
02280 #endif // DOCTEST_CONFIG_DISABLE
02281
02282 } // namespace doctest
02283
02284 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_POP
02285
02286 #endif // DOCTEST_PARTS_PUBLIC_TEST_CASE
02287 #ifndef DOCTEST_PARTS_PUBLIC_DECORATORS
02288 #define DOCTEST_PARTS_PUBLIC_DECORATORS
02289
02290 #ifndef DOCTEST_CONFIG_DISABLE
02291 namespace doctest {
02292
02293 #define DOCTEST_DEFINE_DECORATOR(name, type, def)
02294
02295     struct name {
02296         type data;
02297
02298         name(type in = def) noexcept
02299             : data(in) {}
02300
02301         void fill(detail::TestCase &state) const noexcept {
02302             state.DOSTEST_CAT(m_, name) = data;
02303         }
02304
02305         void fill(detail::TestSuite &state) const noexcept {
02306             state.DOSTEST_CAT(m_, name) = data;
02307         }
02308     }
02309
02310 DOCTEST_DEFINE_DECORATOR(test_suite, const char *, "");
02311 DOCTEST_DEFINE_DECORATOR(description, const char *, "");
02312 DOCTEST_DEFINE_DECORATOR(skip, bool, true);
02313 DOCTEST_DEFINE_DECORATOR(no_breaks, bool, true);
02314 DOCTEST_DEFINE_DECORATOR(no_output, bool, true);
02315 DOCTEST_DEFINE_DECORATOR(timeout, double, 0);
02316 DOCTEST_DEFINE_DECORATOR(may_fail, bool, true);
02317 DOCTEST_DEFINE_DECORATOR(should_fail, bool, true);
02318 DOCTEST_DEFINE_DECORATOR(expected_failures, int, 0);
02319
02320 } // namespace doctest
02321
02322 #endif // DOCTEST_CONFIG_DISABLE
02323
02324 #endif // DOCTEST_PARTS_PUBLIC_DECORATORS
02325 #ifndef DOCTEST_PARTS_PUBLIC_EXCEPTION_TRANSLATOR
02326 #define DOCTEST_PARTS_PUBLIC_EXCEPTION_TRANSLATOR
02327
02328 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_PUSH
02329
02330 namespace doctest {
02331 namespace detail {
02332
02333 #ifndef DOCTEST_CONFIG_DISABLE
02334 struct DOCTEST_INTERFACE IExceptionTranslator {
02335     DOCTEST_DECLARE_INTERFACE(IExceptionTranslator)
02336     virtual bool translate(String &) const = 0;
02337 };
02338
02339 template <typename T>
02340 class ExceptionTranslator : public IExceptionTranslator {
02341 public:
02342     explicit ExceptionTranslator(String (*translateFunction)(T)) noexcept
02343         : m_translateFunction(translateFunction) {}

```

```

02344
02345     bool translate(String &res) const override {
02346 #ifndef DOCTEST_CONFIG_NO_EXCEPTIONS
02347         try {
02348             throw;
02349         } catch (const T &ex) {
02350             res = m_translateFunction(ex);
02351             return true;
02352         } catch (...) {} // NOLINT(bugprone-empty-catch)
02353 #endif // DOCTEST_CONFIG_NO_EXCEPTIONS
02354         static_cast<void>(res); // to silence -Wunused-parameter
02355         return false;
02356     }
02357
02358 private:
02359     String (*m_translateFunction)(T);
02360 };
02361
02362 DOCTEST_INTERFACE void registerExceptionTranslatorImpl(const IExceptionTranslator *et) noexcept;
02363
02364 #endif // DOCTEST_CONFIG_DISABLE
02365 } // namespace detail
02366
02367 #ifndef DOCTEST_CONFIG_DISABLE
02368
02369 template <typename T>
02370 int registerExceptionTranslator(String (*translateFunction)(T)) noexcept {
02371     DOCTEST_CLANG_SUPPRESS_WARNING_WITH_PUSH("-Wexit-time-destructors")
02372     static detail::ExceptionTranslator<T> exceptionTranslator(translateFunction);
02373     DOCTEST_CLANG_SUPPRESS_WARNING_POP
02374     detail::registerExceptionTranslatorImpl(&exceptionTranslator);
02375     return 0;
02376 }
02377
02378 #else // DOCTEST_CONFIG_DISABLE
02379
02380 template <typename T>
02381 int registerExceptionTranslator(String (*) (T)) noexcept {
02382     return 0;
02383 }
02384
02385 #endif // DOCTEST_CONFIG_DISABLE
02386
02387 } // namespace doctest
02388
02389 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_POP
02390
02391 #endif // DOCTEST_PARTS_PUBLIC_EXCEPTION_TRANSLATOR
02392 #ifndef DOCTEST_PARTS_PUBLIC_CONTEXT_SCOPE
02393 #define DOCTEST_PARTS_PUBLIC_CONTEXT_SCOPE
02394
02395 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_PUSH
02396
02397 namespace doctest {
02398
02399 struct DOCTEST_INTERFACE IContextScope {
02400     DOCTEST_DECLARE_INTERFACE(IContextScope)
02401     virtual void stringify(std::ostream *) const = 0;
02402 };
02403
02404 #ifndef DOCTEST_CONFIG_DISABLE
02405 namespace detail {
02406
02407 // ContextScope base class used to allow implementing methods of ContextScope
02408 // that don't depend on the template parameter in doctest.cpp.
02409 struct DOCTEST_INTERFACE ContextScopeBase : public IContextScope {
02410     ContextScopeBase(const ContextScopeBase &) = delete;
02411     ContextScopeBase &operator=(const ContextScopeBase &) = delete;
02412     ContextScopeBase &operator=(ContextScopeBase &&) = delete;
02413     ~ContextScopeBase() override = default;
02414
02415 protected:
02416     ContextScopeBase();
02417     ContextScopeBase(ContextScopeBase &&other) noexcept;
02418     void destroy();
02419     bool need_to_destroy{true};
02420 };
02421
02422 template <typename L>
02423 class ContextScope : public ContextScopeBase {
02424     L lambda_;
02425 };
02426
02427
02428
02429
02430

```

```

02431 public:
02432     explicit ContextScope(const L &lambda)
02433         : lambda_(lambda) {}
02434
02435     // NOLINTNEXTLINE (cppcoreguidelines-rvalue-reference-param-not-moved)
02436     explicit ContextScope(L &&lambda)
02437         : lambda_(static_cast<L &&>(lambda)) {}
02438
02439     ContextScope(const ContextScope &) = delete;
02440     ContextScope(ContextScope &&) noexcept = default;
02441
02442     ContextScope &operator=(const ContextScope &) = delete;
02443     ContextScope &operator=(ContextScope &&) = delete;
02444
02445     void stringify(std::ostream *s) const override {
02446         lambda_(s);
02447     }
02448
02449     ~ContextScope() override {
02450         if (need_to_destroy) {
02451             destroy();
02452         }
02453     }
02454 };
02455
02456 template <typename L>
02457 ContextScope<L> MakeContextScope(const L &lambda) {
02458     return ContextScope<L>(lambda);
02459 }
02460
02461 } // namespace detail
02462 #endif // DOCTEST_CONFIG_DISABLE
02463
02464 } // namespace doctest
02465
02466 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_POP
02467
02468 #endif // DOCTEST_PARTS_PUBLIC_CONTEXT_SCOPE
02469 #ifndef DOCTEST_PARTS_PUBLIC_ASSERT_MESSAGE
02470 #define DOCTEST_PARTS_PUBLIC_ASSERT_MESSAGE
02471
02472
02473 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_PUSH
02474
02475 namespace doctest {
02476
02477     struct DOCTEST_INTERFACE MessageData {
02478         String m_string;
02479         const char *m_file;
02480         int m_line;
02481         assertType::Enum m_severity;
02482     };
02483
02484     #ifndef DOCTEST_CONFIG_DISABLE
02485     namespace detail {
02486
02487         struct DOCTEST_INTERFACE MessageBuilder : public MessageData {
02488             std::ostream *m_stream;
02489             bool logged = false;
02490
02491             MessageBuilder(const char *file, int line, assertType::Enum severity);
02492
02493             MessageBuilder(const MessageBuilder &) = delete;
02494             MessageBuilder(MessageBuilder &&) = delete;
02495
02496             MessageBuilder &operator=(const MessageBuilder &) = delete;
02497             MessageBuilder &operator=(MessageBuilder &&) = delete;
02498
02499             ~MessageBuilder() noexcept(false);
02500
02501             // the preferred way of chaining parameters for stringification
02502             DOCTEST_MSVC_SUPPRESS_WARNING_WITH_PUSH(4866)
02503             template <typename T>
02504             MessageBuilder &operator,(const T &in) {
02505                 *m_stream << (DOCTEST_STRINGIFY(in));
02506                 return *this;
02507             }
02508             DOCTEST_MSVC_SUPPRESS_WARNING_POP
02509
02510             // kept here just for backwards-compatibility - the comma operator should be preferred now
02511             template <typename T>
02512             MessageBuilder &operator<<(const T &in) {
02513                 return this->operator,(in);
02514             }
02515
02516             // the `,' operator has the lowest operator precedence - if `<<' is used by the user then
02517             // the `,' operator will be called last which is not what we want and thus the `*` operator

```

```

02518     // is used first (has higher operator precedence compared to `«`) so that we guarantee that
02519     // an operator of the MessageBuilder class is called first before the rest of the parameters
02520     template <typename T>
02521     MessageBuilder &operator*(const T &in) {
02522         return this->operator,(in);
02523     }
02524
02525     bool log();
02526     void react();
02527 };
02528
02529 } // namespace detail
02530 #endif // DOCTEST_CONFIG_DISABLE
02531
02532 } // namespace doctest
02533
02534 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_POP
02535
02536 #endif // DOCTEST_PARTS_PUBLIC_ASSERT_MESSAGE
02537 #ifndef DOCTEST_PARTS_PUBLIC_PATH
02538 #define DOCTEST_PARTS_PUBLIC_PATH
02539
02540 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_PUSH
02541
02542 namespace doctest {
02543
02544 DOCTEST_INTERFACE const char *skipPathFromFilename(const char *file);
02545
02546 } // namespace doctest
02547
02548 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_POP
02549
02550 #endif // DOCTEST_PARTS_PUBLIC_PATH
02551 #ifndef DOCTEST_PARTS_PUBLIC_EXCEPTIONS
02552 #define DOCTEST_PARTS_PUBLIC_EXCEPTIONS
02553
02554 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_PUSH
02555
02556 #ifndef DOCTEST_CONFIG_DISABLE
02557
02558 namespace doctest {
02559 namespace detail {
02560
02561 struct DOCTEST_INTERFACE TestFailureException {};
02562
02563 DOCTEST_INTERFACE bool checkIfShouldThrow(assertType::Enum at);
02564
02565 #ifndef DOCTEST_CONFIG_NO_EXCEPTIONS
02566 DOCTEST_NORETURN
02567 #endif // DOCTEST_CONFIG_NO_EXCEPTIONS
02568 DOCTEST_INTERFACE void throwException();
02569
02570 } // namespace detail
02571
02572 } // namespace doctest
02573
02574 #endif // DOCTEST_CONFIG_DISABLE
02575
02576 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_POP
02577
02578 #endif // DOCTEST_PARTS_PUBLIC_EXCEPTIONS
02579 #ifndef DOCTEST_PARTS_PUBLIC_CONTEXT
02580 #define DOCTEST_PARTS_PUBLIC_CONTEXT
02581
02582 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_PUSH
02583
02584 namespace doctest {
02585
02586 DOCTEST_INTERFACE extern bool is_running_in_test;
02587
02588 namespace detail {
02589 using assert_handler = void (*)(const AssertData &);
02590 struct ContextState;
02591 } // namespace detail
02592
02593 class DOCTEST_INTERFACE Context {
02594     detail::ContextState *p;
02595
02596     void parseArgs(int argc, const char *const *argv, bool withDefaults = false);
02597
02598 public:
02599     explicit Context(int argc = 0, const char *const *argv = nullptr);
02600
02601     Context(const Context &) = delete;
02602     Context(Context &&) = delete;

```



```

02605
02606     Context &operator=(const Context &) = delete;
02607     Context &operator=(Context &&) = delete;
02608
02609     ~Context(); // NOLINT(performance-trivially-destructible)
02610
02611     void applyCommandLine(int argc, const char *const *argv);
02612
02613     void addFilter(const char *filter, const char *value);
02614     void clearFilters();
02615     void setOption(const char *option, bool value);
02616     void setOption(const char *option, int value);
02617     void setOption(const char *option, const char *value);
02618
02619     bool shouldExit();
02620
02621     void setAsDefaultForAssertsOutOfTestCases();
02622
02623     void setAssertHandler(detail::assert_handler ah);
02624
02625     void setCout(std::ostream *out);
02626
02627     int run();
02628 };
02629 } // namespace doctest
02630
02631 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_POP
02632
02633 #endif // DOCTEST_PARTS_PUBLIC_CONTEXT
02634 #ifndef DOCTEST_PARTS_PUBLIC_ASSERT_HANDLER
02635 #define DOCTEST_PARTS_PUBLIC_ASSERT_HANDLER
02636
02637 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_PUSH
02638
02639 #ifndef DOCTEST_CONFIG_DISABLE
02640
02641 namespace doctest {
02642 namespace detail {
02643
02644 DOCTEST_INTERFACE void failed_out_of_a_testing_context(const AssertData &ad);
02645
02646 DOCTEST_INTERFACE bool
02647 decomp_assert(assertType::Enum at, const char *file, int line, const char *expr, const Result
02648 &result);
02649
02650 #define DOCTEST_ASSERT_OUT_OF_TESTS(decomp)
02651 \
02652 \
02653 \
02654 \
02655 \
02656 \
02657 \
02658 \
02659 \
02660 \
02661 \
02662 \
02663 \
02664 \
02665 \
02666 \
02667 #define DOCTEST_ASSERT_IN_TESTS(decomp)
02668 \
02669 \
02670 \
02671 \

```

```

02672     \
02673     \
02674     \
02675     \
02676
02677 template <int comparison, typename L, typename R>
02678 DOCTEST_NOINLINE bool binary_assert(
02679     assertType::Enum at,
02680     const char *file,
02681     int line,
02682     const char *expr,
02683     const DOCTEST_REF_WRAP(L) lhs,
02684     const DOCTEST_REF_WRAP(R) rhs
02685 ) {
02686     const bool failed = !RelationalComparator<comparison, L, R>()(lhs, rhs);
02687
02688     // #####
02689     // IF THE DEBUGGER BREAKS HERE - GO 1 LEVEL UP IN THE CALLSTACK FOR THE FAILING ASSERT
02690     // THIS IS THE EFFECT OF HAVING 'DOCTEST_CONFIG_SUPER_FAST_ASSERTS' DEFINED
02691     // #####
02692     DOCTEST_ASSERT_OUT_OF_TESTS(stringifyBinaryExpr(lhs, " ", rhs));
02693     DOCTEST_ASSERT_IN_TESTS(stringifyBinaryExpr(lhs, " ", rhs));
02694     return !failed;
02695 }
02696
02697 template <typename L>
02698 DOCTEST_NOINLINE bool
02699 unary_assert(assertType::Enum at, const char *file, int line, const char *expr, const
02700     DOCTEST_REF_WRAP(L) val) {
02701     bool failed = !val;
02702
02703     if (at & assertType::is_false)
02704         failed = !failed;
02705
02706     // #####
02707     // IF THE DEBUGGER BREAKS HERE - GO 1 LEVEL UP IN THE CALLSTACK FOR THE FAILING ASSERT
02708     // THIS IS THE EFFECT OF HAVING 'DOCTEST_CONFIG_SUPER_FAST_ASSERTS' DEFINED
02709     // #####
02710     DOCTEST_ASSERT_OUT_OF_TESTS((DOCTEST_STRINGIFY(val)));
02711     DOCTEST_ASSERT_IN_TESTS((DOCTEST_STRINGIFY(val)));
02712     return !failed;
02713 }
02714 } // namespace detail
02715 } // namespace doctest
02716
02717 #endif // DOCTEST_CONFIG_DISABLE
02718
02719 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_POP
02720
02721 #endif // DOCTEST_PARTS_PUBLIC_ASSERT_HANDLER
02722 #ifndef DOCTEST_PARTS_PUBLIC_REPORTER
02723 #define DOCTEST_PARTS_PUBLIC_REPORTER
02724
02725 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_PUSH
02726
02727 namespace doctest {
02728
02729 namespace TestCaseFailureReason {
02730     enum Enum {
02731         None = 0,
02732         AssertFailure = 1, // an assertion has failed in the test case
02733         Exception = 2, // test case threw an exception
02734         Crash = 4, // a crash...
02735         TooManyFailedAsserts = 8, // the abort-after option
02736         Timeout = 16, // see the timeout decorator
02737         ShouldHaveFailedButDidnt = 32, // see the should_fail decorator
02738         ShouldHaveFailedAndDid = 64, // see the should_fail decorator
02739         DidntFailExactlyNumTimes = 128, // see the expected_failures decorator
02740         FailedExactlyNumTimes = 256, // see the expected_failures decorator
02741         CouldHaveFailedAndDid = 512 // see the may_fail decorator
02742     };
02743 } // namespace TestCaseFailureReason
02744 } // namespace doctest
02745
02746 struct DOCTEST_INTERFACE CurrentTestCaseStats {
02747     int numAssertsCurrentTest;
02748     int numAssertsFailedCurrentTest;
02749     double seconds;
02750     int failure_flags; // use TestCaseFailureReason::Enum
02751     bool testCaseSuccess;
02752 };
02753

```

```

02754 struct DOCTEST_INTERFACE TestCaseException {
02755     String error_string;
02756     bool is_crash;
02757 };
02758
02759 struct DOCTEST_INTERFACE TestRunStats {
02760     unsigned numTestCases;
02761     unsigned numTestCasesPassingFilters;
02762     unsigned numTestSuitesPassingFilters;
02763     unsigned numTestCasesFailed;
02764     int numAsserts;
02765     int numAssertsFailed;
02766 };
02767
02768 struct QueryData {
02769     const TestRunStats *run_stats = nullptr;
02770     const TestCaseData **data = nullptr;
02771     unsigned num_data = 0;
02772 };
02773
02774 struct DOCTEST_INTERFACE IReporter {
02775     // The constructor has to accept "const ContextOptions&" as a single argument
02776     // which has most of the options for the run + a pointer to the stdout stream
02777     // Reporter(const ContextOptions& in)
02778
02779     // called when a query should be reported (listing test cases, printing the version, etc.)
02780     virtual void report_query(const QueryData &) = 0;
02781
02782     // called when the whole test run starts
02783     virtual void test_run_start() = 0;
02784     // called when the whole test run ends (caching a pointer to the input doesn't make sense here)
02785     virtual void test_run_end(const TestRunStats &) = 0;
02786
02787     // called when a test case is started (safe to cache a pointer to the input)
02788     virtual void test_case_start(const TestCaseData &) = 0;
02789     // called when a test case is reentered because of unfinished subcases (safe to cache a pointer to
the input)
02790     virtual void test_case_reenter(const TestCaseData &) = 0;
02791     // called when a test case has ended
02792     virtual void test_case_end(const CurrentTestCaseStats &) = 0;
02793
02794     // called when an exception is thrown from the test case (or it crashes)
02795     virtual void test_case_exception(const TestCaseException &) = 0;
02796
02797     // called whenever a subcase is entered (don't cache pointers to the input)
02798     virtual void subcase_start(const SubcaseSignature &) = 0;
02799     // called whenever a subcase is exited (don't cache pointers to the input)
02800     virtual void subcase_end() = 0;
02801
02802     // called for each assert (don't cache pointers to the input)
02803     virtual void log_assert(const AssertData &) = 0;
02804     // called for each message (don't cache pointers to the input)
02805     virtual void log_message(const MessageData &) = 0;
02806
02807     // called when a test case is skipped either because it doesn't pass the filters,
02808     // has a skip decorator or isn't in the execution range (between first and last)
02809     // (safe to cache a pointer to the input)
02810     virtual void test_case_skipped(const TestCaseData &) = 0;
02811
02812     DOCTEST_DECLARE_INTERFACE(IReporter)
02813
02814     // can obtain all currently active contexts and stringify them if one wishes to do so
02815     static int get_num_active_contexts();
02816     static const IContextScope *const *get_active_contexts();
02817
02818     // can iterate through contexts which have been stringified automatically
02819     // in their destructors when an exception has been thrown
02820     static int get_num_stringified_contexts();
02821     static const String *get_stringified_contexts();
02822 };
02823
02824 namespace detail {
02825     using reporterCreatorFunc = IReporter *(*)(const ContextOptions &);
02826
02827     DOCTEST_INTERFACE void
02828     registerReporterImpl(const char *name, int prio, reporterCreatorFunc c, bool isReporter) noexcept;
02829
02830     template <typename Reporter>
02831     IReporter *reporterCreator(const ContextOptions &o) {
02832         return new Reporter(o);
02833     }
02834 } // namespace detail
02835
02836 template <typename Reporter>
02837 int registerReporter(const char *name, int priority, bool isReporter) noexcept {
02838     detail::registerReporterImpl(name, priority, detail::reporterCreator<Reporter>, isReporter);
02839     return 0;

```

```

02840 }
02841 } // namespace doctest
02842
02843 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_POP
02844
02845 #endif // DOCTEST_PARTS_PUBLIC_REPORTER
02846 #ifndef DOCTEST_PARTS_PUBLIC_GENERATOR
02847 #define DOCTEST_PARTS_PUBLIC_GENERATOR
02848
02849
02850 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_PUSH
02851
02852 #ifndef DOCTEST_CONFIG_DISABLE
02853 namespace doctest {
02854 namespace detail {
02855
02856 DOCTEST_INTERFACE size_t acquireGeneratorDecisionIndex(size_t count);
02857
02858 template <typename T, typename... Rest>
02859 T acquireGeneratorValue(T first, Rest... rest) {
02860     const T values[] = {first, static_cast<T>(rest)...};
02861     const size_t idx = acquireGeneratorDecisionIndex(1 + sizeof...(Rest));
02862     return values[idx];
02863 }
02864
02865 } // namespace detail
02866 } // namespace doctest
02867 #endif // DOCTEST_CONFIG_DISABLE
02868
02869 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_POP
02870
02871 #endif // DOCTEST_PARTS_PUBLIC_GENERATOR
02872 #ifndef DOCTEST_PARTS_PUBLIC_MACROS
02873 #define DOCTEST_PARTS_PUBLIC_MACROS
02874
02875
02876 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_PUSH
02877
02878 #ifndef DOCTEST_CONFIG_DISABLE
02879 namespace doctest {
02880 namespace detail {
02881 template <typename T>
02882 int instantiationHelper(const T &) noexcept {
02883     return 0;
02884 }
02885
02886 } // namespace detail
02887 } // namespace doctest
02888 #endif // DOCTEST_CONFIG_DISABLE
02889
02890 #ifdef DOCTEST_CONFIG_ASSERTS_RETURN_VALUES
02891 #define DOCTEST_FUNC_EMPTY [] { return false; }()
02892 #else
02893 #define DOCTEST_FUNC_EMPTY (void)0
02894 #endif
02895
02896 // if registering is not disabled
02897 #ifndef DOCTEST_CONFIG_DISABLE
02898
02899 #ifdef DOCTEST_CONFIG_ASSERTS_RETURN_VALUES
02900 #define DOCTEST_FUNC_SCOPE_BEGIN [&]
02901 #define DOCTEST_FUNC_SCOPE_END ()
02902 #define DOCTEST_FUNC_SCOPE_RET(v) return v
02903 #else
02904 #define DOCTEST_FUNC_SCOPE_BEGIN do /* NOLINT(cppcoreguidelines-avoid-do-while)*/
02905 #define DOCTEST_FUNC_SCOPE_END while (false)
02906 #define DOCTEST_FUNC_SCOPE_RET(v) (void)0
02907 #endif
02908
02909 // common code in asserts - for convenience
02910 #define DOCTEST_ASSERT_LOG_REACT_RETURN(b)
02911 \
02912     if (b.log())
02912 \
02913     DOCTEST_BREAK_INTO_DEBUGGER();
02912 \
02913     b.react();
02912 \
02914     DOCTEST_FUNC_SCOPE_RET(!b.m_failed)
02915
02916 #ifdef DOCTEST_CONFIG_NO_TRY_CATCH_IN_ASSERTS
02917 #define DOCTEST_WRAP_IN_TRY(x) x;
02918 #else // DOCTEST_CONFIG_NO_TRY_CATCH_IN_ASSERTS
02919 #define DOCTEST_WRAP_IN_TRY(x)
02920 \
02921     try {
02922 \

```

```

02921         x;
02922     } catch (...) { DOCTEST_RB.translateException(); }
02923 #endif // DOCTEST_CONFIG_NO_TRY_CATCH_IN_ASSERTS
02924
02925 #ifndef DOCTEST_CONFIG_VOID_CAST_EXPRESSIONS
02926 #define DOCTEST_CAST_TO_VOID(...)
02927     DOCTEST_GCC_SUPPRESS_WARNING_WITH_PUSH("-Wuseless-cast")
02928     static_cast<void>(__VA_ARGS__);
02929     DOCTEST_GCC_SUPPRESS_WARNING_POP
02930 #else // DOCTEST_CONFIG_VOID_CAST_EXPRESSIONS
02931 #define DOCTEST_CAST_TO_VOID(...) __VA_ARGS__;
02932 #endif // DOCTEST_CONFIG_VOID_CAST_EXPRESSIONS
02933
02934 // registers the test by initializing a dummy var with a function
02935 #define DOCTEST_REGISTER_FUNCTION(global_prefix, f, decorators)
02936     global_prefix DOCTEST_GLOBAL_NO_WARNINGS (
02937         DOCTEST_ANONYMOUS(DOCTEST_ANON_VAR_), /* NOLINT */
02938         doctest::detail::regTest (
02939             doctest::detail::TestCase(f, __FILE__, __LINE__,
02940 doctest_detail_test_suite_ns::getCurrentTestSuite()) * \
02941             decorators
02942         )
02943
02944 #define DOCTEST_IMPLEMENT_FIXTURE(der, base, func, decorators)
02945     namespace { /* NOLINT */
02946     struct der : public base {
02947         void f();
02948     };
02949     static DOCTEST_INLINE_NOINLINE void func() {
02950         der v;
02951         v.f();
02952     }
02953     DOCTEST_REGISTER_FUNCTION(DOCTEST_EMPTY, func, decorators)
02954 }
02955 DOCTEST_INLINE_NOINLINE void der::f() // NOLINT(misc-definitions-in-headers)
02956
02957 #define DOCTEST_CREATE_AND_REGISTER_FUNCTION(f, decorators)
02958     static void f();
02959     DOCTEST_REGISTER_FUNCTION(DOCTEST_EMPTY, f, decorators)
02960     static void f()
02961
02962 #define DOCTEST_CREATE_AND_REGISTER_FUNCTION_IN_CLASS(f, proxy, decorators)
02963     static doctest::detail::funcType proxy() {
02964         return f;
02965     }
02966     DOCTEST_REGISTER_FUNCTION(inline, proxy(), decorators)
02967     static void f()
02968
02969 // for registering tests
02970 #define DOCTEST_TEST_CASE(decorators)
02971     DOCTEST_CREATE_AND_REGISTER_FUNCTION(DOCTEST_ANONYMOUS(DOCTEST_ANON_FUNC_), decorators)
02972
02973 // for registering tests in classes - requires C++17 for inline variables!
02974 #if DOCTEST_CPLUSPLUS >= 201703L
02975 #define DOCTEST_TEST_CASE_CLASS(decorators)

```

```

02976     DOCTEST_CREATE_AND_REGISTER_FUNCTION_IN_CLASS(
02977         DOCTEST_ANONYMOUS(DOCTEST_ANON_FUNC_), DOCTEST_ANONYMOUS(DOCTEST_ANON_PROXY_), decorators
02978     )
02979 #else // DOCTEST_TEST_CASE_CLASS
02980 #define DOCTEST_TEST_CASE_CLASS(...)
02981     TEST_CASES_CAN_BE_REGISTERED_IN_CLASSES_ONLY_IN_CPP17_MODE_OR_WITH_VS_2017_OR_NEWER
02982 #endif // DOCTEST_TEST_CASE_CLASS
02983 // for registering tests with a fixture
02984 #define DOCTEST_TEST_CASE_FIXTURE(c, decorators)
02985     DOCTEST_IMPLEMENT_FIXTURE(
02986         DOCTEST_ANONYMOUS(DOCTEST_ANON_CLASS_), c, DOCTEST_ANONYMOUS(DOCTEST_ANON_FUNC_), decorators
02987     )
02988 // for converting types to strings without the <typeinfo> header and demangling
02989 #define DOCTEST_TYPE_TO_STRING_AS(str, ...)
02990 namespace doctest {
02991     template <>
02992     inline String toString<__VA_ARGS__>() {
02993         return str;
02994     }
02995 }
02996 static_assert(true, "")
02997 #define DOCTEST_TYPE_TO_STRING(...) DOCTEST_TYPE_TO_STRING_AS(#__VA_ARGS__, __VA_ARGS__)
02998 #define DOCTEST_TEST_CASE_TEMPLATE_DEFINE_IMPL(dec, T, iter, func)
02999 template <typename T>
03000 static void func();
03001 namespace { /* NOLINT */
03002     template <typename Tuple>
03003     struct iter;
03004     template <typename Type, typename... Rest>
03005     struct iter<std::tuple<Type, Rest...> > {
03006         iter(const char *file, unsigned line, int index) noexcept {
03007             doctest::detail::regTest(
03008                 doctest::detail::TestCase(
03009                     func<Type>,
03010                     file,
03011                     line,
03012                     doctest_detail_test_suite_ns::getCurrentTestSuite(),
03013                     doctest::toString<Type>(),
03014                     int(line) * 1000 + index
03015                 ) *
03016                 dec
03017             );
03018             iter<std::tuple<Rest...>>(file, line, index + 1);
03019         }
03020     };
03021     template <>
03022     struct iter<std::tuple<>> {

```

```

03026     iter(const char *, unsigned, int) {}
03027 };
03028 }
03029 template <typename T>
03030 static void func()
03031
03032 #define DOCTEST_TEST_CASE_TEMPLATE_DEFINE(dec, T, id)
03033     DOCTEST_TEST_CASE_TEMPLATE_DEFINE_IMPL(dec, T, DOCTEST_CAT(id, ITERATOR),
DOCTEST_ANONYMOUS(DOCTEST_ANON_TMP_))
03034
03035 #define DOCTEST_TEST_CASE_TEMPLATE_INSTANTIATE_IMPL(id, anon, ...)
03036     DOCTEST_GLOBAL_NO_WARNINGS(
03037         DOCTEST_CAT(anon, DUMMY), /* NOLINT(cert-err58-cpp, fuchsia-statically-constructed-objects) */
03038         doctest::detail::instantiationHelper(DOCTEST_CAT(id, ITERATOR) < __VA_ARGS__ > (__FILE__,
__LINE__, 0))
03039     )
03040
03041 #define DOCTEST_TEST_CASE_TEMPLATE_INVOKE(id, ...)
03042     DOCTEST_TEST_CASE_TEMPLATE_INSTANTIATE_IMPL(id, DOCTEST_ANONYMOUS(DOCTEST_ANON_TMP_),
std::tuple<__VA_ARGS__>)
03043     static_assert(true, "")
03044
03045 #define DOCTEST_TEST_CASE_TEMPLATE_APPLY(id, ...)
03046     DOCTEST_TEST_CASE_TEMPLATE_INSTANTIATE_IMPL(id, DOCTEST_ANONYMOUS(DOCTEST_ANON_TMP_), __VA_ARGS__)
03047     static_assert(true, "")
03048
03049 #define DOCTEST_TEST_CASE_TEMPLATE_IMPL(dec, T, anon, ...)
03050     DOCTEST_TEST_CASE_TEMPLATE_DEFINE_IMPL(dec, T, DOCTEST_CAT(anon, ITERATOR), anon);
03051     DOCTEST_TEST_CASE_TEMPLATE_INSTANTIATE_IMPL(anon, anon, std::tuple<__VA_ARGS__>)
03052     template <typename T>
03053     static void anon()
03054
03055 #define DOCTEST_TEST_CASE_TEMPLATE(dec, T, ...)
03056     DOCTEST_TEST_CASE_TEMPLATE_IMPL(dec, T, DOCTEST_ANONYMOUS(DOCTEST_ANON_TMP_), __VA_ARGS__)
03057
03058 // for subcases
03059 #define DOCTEST_SUBCASE(name)
03060     if (const doctest::detail::Subcase &DOCTEST_ANONYMOUS(DOCTEST_ANON_SUBCASE_) DOCTEST_UNUSED =
03061         doctest::detail::Subcase(name, __FILE__, __LINE__))
03062
03063 // for generating value-parameterized test inputs
03064 #define DOCTEST_GENERATE(...) doctest::detail::acquireGeneratorValue(__VA_ARGS__)
03065
03066 // for grouping tests in test suites by using code blocks
03067 #define DOCTEST_TEST_SUITE_IMPL(decorators, ns_name)
03068     namespace ns_name {
03069     namespace doctest_detail_test_suite_ns {
03070     static DOCTEST_NOINLINE doctest::detail::TestSuite &getCurrentTestSuite() noexcept {
03071         DOCTEST_MSVC_SUPPRESS_WARNING_WITH_PUSH(4640)
03072         DOCTEST_CLANG_SUPPRESS_WARNING_WITH_PUSH("-Wexit-time-destructors")
03073         DOCTEST_GCC_SUPPRESS_WARNING_WITH_PUSH("-Wmissing-field-initializers")
03074         static doctest::detail::TestSuite data{};
03075         static bool initd = false;
03076         DOCTEST_MSVC_SUPPRESS_WARNING_POP
03077         DOCTEST_CLANG_SUPPRESS_WARNING_POP
03078         DOCTEST_GCC_SUPPRESS_WARNING_POP

```

```

03079         if (!inited) {
03080             data *decorators;
03081             inited = true;
03082         }
03083         return data;
03084     }
03085 }
03086 }
03087 namespace ns_name
03088 {
03089 #define DOCTEST_TEST_SUITE(decorators) DOCTEST_TEST_SUITE_IMPL(decorators,
03090 DOCTEST_ANONYMOUS(DOCTEST_ANON_SUITE_))
03091 // for starting a testsuite block
03092 #define DOCTEST_TEST_SUITE_BEGIN(decorators)
03093     DOCTEST_GLOBAL_NO_WARNINGS(
03094         DOCTEST_ANONYMOUS(DOCTEST_ANON_VAR_), /* NOLINT(cert-err58-cpp) */
03095         doctest::detail::setTestSuite(doctest::detail::TestSuite() * decorators)
03096     )
03097     static_assert(true, "")
03098 // for ending a testsuite block
03099 #define DOCTEST_TEST_SUITE_END
03100     DOCTEST_GLOBAL_NO_WARNINGS(
03101         DOCTEST_ANONYMOUS(DOCTEST_ANON_VAR_), /* NOLINT(cert-err58-cpp) */
03102         doctest::detail::setTestSuite(doctest::detail::TestSuite() * "")
03103     )
03104     using DOCTEST_ANONYMOUS(DOCTEST_ANON_FOR_SEMICOLON_) = int
03105 // for registering exception translators
03106 #define DOCTEST_REGISTER_EXCEPTION_TRANSLATOR_IMPL(translatorName, signature)
03107     inline doctest::String translatorName(signature);
03108     DOCTEST_GLOBAL_NO_WARNINGS(
03109         DOCTEST_ANONYMOUS(DOCTEST_ANON_TRANSLATOR_VAR_), /* NOLINT(cert-err58-cpp) */
03110         doctest::registerExceptionTranslator(translatorName)
03111     )
03112     doctest::String translatorName(signature)
03113 #define DOCTEST_REGISTER_EXCEPTION_TRANSLATOR(signature)
03114     DOCTEST_REGISTER_EXCEPTION_TRANSLATOR_IMPL(DOCTEST_ANONYMOUS(DOCTEST_ANON_TRANSLATOR_), signature)
03115 // for registering reporters
03116 #define DOCTEST_REGISTER_REPORTER(name, priority, reporter)
03117     DOCTEST_GLOBAL_NO_WARNINGS(
03118         DOCTEST_ANONYMOUS(DOCTEST_ANON_REPORTER_VAR_), /* NOLINT(cert-err58-cpp) */
03119         doctest::registerReporter<reporter>(name, priority, true)
03120     )
03121     static_assert(true, "")
03122 // for registering listeners
03123 #define DOCTEST_REGISTER_LISTENER(name, priority, reporter)
03124     DOCTEST_GLOBAL_NO_WARNINGS(
03125         DOCTEST_ANONYMOUS(DOCTEST_ANON_REPORTER_VAR_), /* NOLINT(cert-err58-cpp) */
03126         doctest::registerReporter<reporter>(name, priority, false)

```



```

03132     \
03133     \
03134     static_assert(true, "")
03135 #define DOCTEST_INFO(...)
03136     \
03137     DOCTEST_INFO_IMPL(DOCTEST_ANONYMOUS(DOCTEST_CAPTURE_MB_),
03138     DOCTEST_ANONYMOUS(DOCTEST_CAPTURE_OTHER_), __VA_ARGS__)
03139 #define DOCTEST_INFO_IMPL(mb_name, s_name, ...)
03140     \
03141     auto DOCTEST_ANONYMOUS(DOCTEST_CAPTURE_) = doctest::detail::MakeContextScope([&](std::ostream
03142     *s_name) {
03143         \
03144         doctest::detail::MessageBuilder mb_name(__FILE__, __LINE__, doctest::assertType::is_warn);
03145         \
03146         mb_name.m_stream = s_name;
03147         \
03148         mb_name *__VA_ARGS__;
03149     \
03150     })
03151 #define DOCTEST_CAPTURE(x) DOCTEST_INFO(#x " := ", x)
03152 #define DOCTEST_ADD_AT_IMPL(type, file, line, mb, ...)
03153     \
03154     DOCTEST_FUNC_SCOPE_BEGIN {
03155     \
03156     doctest::detail::MessageBuilder mb(file, line, doctest::assertType::type);
03157     \
03158     mb *__VA_ARGS__;
03159     \
03160     if (mb.log())
03161     \
03162         DOCTEST_BREAK_INTO_DEBUGGER();
03163     \
03164     mb.react();
03165     \
03166     }
03167     \
03168     DOCTEST_FUNC_SCOPE_END
03169 #define DOCTEST_ADD_MESSAGE_AT(file, line, ...)
03170     \
03171     DOCTEST_ADD_AT_IMPL(is_warn, file, line, DOCTEST_ANONYMOUS(DOCTEST_MESSAGE_), __VA_ARGS__)
03172 #define DOCTEST_ADD_FAIL_CHECK_AT(file, line, ...)
03173     \
03174     DOCTEST_ADD_AT_IMPL(is_check, file, line, DOCTEST_ANONYMOUS(DOCTEST_MESSAGE_), __VA_ARGS__)
03175 #define DOCTEST_ADD_FAIL_AT(file, line, ...)
03176     \
03177     DOCTEST_ADD_AT_IMPL(is_require, file, line, DOCTEST_ANONYMOUS(DOCTEST_MESSAGE_), __VA_ARGS__)
03178 #define DOCTEST_MESSAGE(...) DOCTEST_ADD_MESSAGE_AT(__FILE__, __LINE__, __VA_ARGS__)
03179 #define DOCTEST_FAIL_CHECK(...) DOCTEST_ADD_FAIL_CHECK_AT(__FILE__, __LINE__, __VA_ARGS__)
03180 #define DOCTEST_FAIL(...) DOCTEST_ADD_FAIL_AT(__FILE__, __LINE__, __VA_ARGS__)
03181 #define DOCTEST_TO_LVALUE(...) __VA_ARGS__ // Not removed to keep backwards compatibility.
03182 #ifndef DOCTEST_CONFIG_SUPER_FAST_ASSERTS
03183 #define DOCTEST_ASSERT_IMPLEMENT_2(assert_type, ...)
03184     \
03185     DOCTEST_CLANG_SUPPRESS_WARNING_WITH_PUSH("-Woverloaded-shift-op-parentheses")
03186     \
03187     /* NOLINTNEXTLINE(clang-analyzer-cplusplus.NewDeleteLeaks) */
03188     \
03189     doctest::detail::ResultBuilder DOCTEST_RB(doctest::assertType::assert_type, __FILE__, __LINE__,
03190     #__VA_ARGS__);
03191     \
03192     DOCTEST_WRAP_IN_TRY(
03193     \
03194         DOCTEST_RB.setResult(doctest::detail::ExpressionDecomposer(doctest::assertType::assert_type) «
03195     __VA_ARGS__)
03196     \
03197     ) /* NOLINTNEXTLINE(clang-analyzer-cplusplus.NewDeleteLeaks) */
03198     \
03199     DOCTEST_ASSERT_LOG_REACT_RETURN(DOCTEST_RB)
03200     \
03201     DOCTEST_CLANG_SUPPRESS_WARNING_POP
03202 #define DOCTEST_ASSERT_IMPLEMENT_1(assert_type, ...)
03203     \
03204     DOCTEST_FUNC_SCOPE_BEGIN {
03205     \
03206         DOCTEST_ASSERT_IMPLEMENT_2(assert_type, __VA_ARGS__);
03207     \
03208     }
03209     \
03210     DOCTEST_FUNC_SCOPE_END // NOLINT(clang-analyzer-cplusplus.NewDeleteLeaks)

```

```

03187
03188 #define DOCTEST_BINARY_ASSERT(assert_type, comp, ...)
03189 \
03190     DOCTEST_FUNC_SCOPE_BEGIN {
03191 \
03192         doctest::detail::ResultBuilder DOCTEST_RB(doctest::assertType::assert_type, __FILE__,
03193         __LINE__, #__VA_ARGS__); \
03194     DOCTEST_WRAP_IN_TRY(DOCTEST_RB.binary_assert<doctest::detail::binaryAssertComparison::comp>(__VA_ARGS__))
03195 \
03196     DOCTEST_ASSERT_LOG_REACT_RETURN(DOCTEST_RB);
03197 \
03198 }
03199 \
03200     DOCTEST_FUNC_SCOPE_END
03201
03202 #define DOCTEST_UNARY_ASSERT(assert_type, ...)
03203 \
03204     DOCTEST_FUNC_SCOPE_BEGIN {
03205 \
03206         doctest::detail::ResultBuilder DOCTEST_RB(doctest::assertType::assert_type, __FILE__,
03207         __LINE__, #__VA_ARGS__); \
03208     DOCTEST_WRAP_IN_TRY(DOCTEST_RB.unary_assert(__VA_ARGS__))
03209 \
03210     DOCTEST_ASSERT_LOG_REACT_RETURN(DOCTEST_RB);
03211 \
03212 }
03213 \
03214     DOCTEST_FUNC_SCOPE_END
03215
03216 #else // DOCTEST_CONFIG_SUPER_FAST_ASSERTS
03217 // necessary for <ASSERT>.MESSAGE
03218 #define DOCTEST_ASSERT_IMPLEMENT_2 DOCTEST_ASSERT_IMPLEMENT_1
03219 #define DOCTEST_ASSERT_IMPLEMENT_1(assert_type, ...)
03220 \
03221     DOCTEST_CLANG_SUPPRESS_WARNING_WITH_PUSH("-Woverloaded-shift-op-parentheses")
03222 \
03223     doctest::detail::decomp_assert(
03224 \
03225         doctest::assertType::assert_type,
03226 \
03227         __FILE__,
03228 \
03229         __LINE__,
03230 \
03231         #__VA_ARGS__,
03232 \
03233         doctest::detail::ExpressionDecomposer(doctest::assertType::assert_type) « __VA_ARGS__
03234     ) DOCTEST_CLANG_SUPPRESS_WARNING_POP
03235 #define DOCTEST_BINARY_ASSERT(assert_type, comparison, ...)
03236 \
03237     doctest::detail::binary_assert<doctest::detail::binaryAssertComparison::comparison>(
03238 \
03239         doctest::assertType::assert_type, __FILE__, __LINE__, #__VA_ARGS__, __VA_ARGS__
03240 \
03241     )
03242 #define DOCTEST_UNARY_ASSERT(assert_type, ...)
03243 \
03244     doctest::detail::unary_assert(doctest::assertType::assert_type, __FILE__, __LINE__, #__VA_ARGS__,
03245     __VA_ARGS__)
03246
03247 #endif // DOCTEST_CONFIG_SUPER_FAST_ASSERTS
03248
03249 #define DOCTEST_WARN(...) DOCTEST_ASSERT_IMPLEMENT_1(DT_WARN, __VA_ARGS__)
03250 #define DOCTEST_CHECK(...) DOCTEST_ASSERT_IMPLEMENT_1(DT_CHECK, __VA_ARGS__)
03251 #define DOCTEST_REQUIRE(...) DOCTEST_ASSERT_IMPLEMENT_1(DT_REQUIRE, __VA_ARGS__)
03252 #define DOCTEST_WARN_FALSE(...) DOCTEST_ASSERT_IMPLEMENT_1(DT_WARN_FALSE, __VA_ARGS__)
03253 #define DOCTEST_CHECK_FALSE(...) DOCTEST_ASSERT_IMPLEMENT_1(DT_CHECK_FALSE, __VA_ARGS__)
03254 #define DOCTEST_REQUIRE_FALSE(...) DOCTEST_ASSERT_IMPLEMENT_1(DT_REQUIRE_FALSE, __VA_ARGS__)
03255
03256 // clang-format off
03257 #define DOCTEST_WARN_MESSAGE(cond, ...) DOCTEST_FUNC_SCOPE_BEGIN { DOCTEST_INFO(__VA_ARGS__);
03258     DOCTEST_ASSERT_IMPLEMENT_2(DT_WARN, cond); } DOCTEST_FUNC_SCOPE_END
03259 #define DOCTEST_CHECK_MESSAGE(cond, ...) DOCTEST_FUNC_SCOPE_BEGIN { DOCTEST_INFO(__VA_ARGS__);
03260     DOCTEST_ASSERT_IMPLEMENT_2(DT_CHECK, cond); } DOCTEST_FUNC_SCOPE_END
03261 #define DOCTEST_REQUIRE_MESSAGE(cond, ...) DOCTEST_FUNC_SCOPE_BEGIN { DOCTEST_INFO(__VA_ARGS__);
03262     DOCTEST_ASSERT_IMPLEMENT_2(DT_REQUIRE, cond); } DOCTEST_FUNC_SCOPE_END
03263 #define DOCTEST_WARN_FALSE_MESSAGE(cond, ...) DOCTEST_FUNC_SCOPE_BEGIN { DOCTEST_INFO(__VA_ARGS__);
03264     DOCTEST_ASSERT_IMPLEMENT_2(DT_WARN_FALSE, cond); } DOCTEST_FUNC_SCOPE_END
03265 #define DOCTEST_CHECK_FALSE_MESSAGE(cond, ...) DOCTEST_FUNC_SCOPE_BEGIN { DOCTEST_INFO(__VA_ARGS__);
03266     DOCTEST_ASSERT_IMPLEMENT_2(DT_CHECK_FALSE, cond); } DOCTEST_FUNC_SCOPE_END
03267 #define DOCTEST_REQUIRE_FALSE_MESSAGE(cond, ...) DOCTEST_FUNC_SCOPE_BEGIN { DOCTEST_INFO(__VA_ARGS__);

```

```

DOCTEST_ASSERT_IMPLEMENT_2(DT_REQUIRE_FALSE, cond); } DOCTEST_FUNC_SCOPE_END
03243 // clang-format on
03244
03245 #define DOCTEST_WARN_EQ(...) DOCTEST_BINARY_ASSERT(DT_WARN_EQ, eq, __VA_ARGS__)
03246 #define DOCTEST_CHECK_EQ(...) DOCTEST_BINARY_ASSERT(DT_CHECK_EQ, eq, __VA_ARGS__)
03247 #define DOCTEST_REQUIRE_EQ(...) DOCTEST_BINARY_ASSERT(DT_REQUIRE_EQ, eq, __VA_ARGS__)
03248 #define DOCTEST_WARN_NE(...) DOCTEST_BINARY_ASSERT(DT_WARN_NE, ne, __VA_ARGS__)
03249 #define DOCTEST_CHECK_NE(...) DOCTEST_BINARY_ASSERT(DT_CHECK_NE, ne, __VA_ARGS__)
03250 #define DOCTEST_REQUIRE_NE(...) DOCTEST_BINARY_ASSERT(DT_REQUIRE_NE, ne, __VA_ARGS__)
03251 #define DOCTEST_WARN_GT(...) DOCTEST_BINARY_ASSERT(DT_WARN_GT, gt, __VA_ARGS__)
03252 #define DOCTEST_CHECK_GT(...) DOCTEST_BINARY_ASSERT(DT_CHECK_GT, gt, __VA_ARGS__)
03253 #define DOCTEST_REQUIRE_GT(...) DOCTEST_BINARY_ASSERT(DT_REQUIRE_GT, gt, __VA_ARGS__)
03254 #define DOCTEST_WARN_LT(...) DOCTEST_BINARY_ASSERT(DT_WARN_LT, lt, __VA_ARGS__)
03255 #define DOCTEST_CHECK_LT(...) DOCTEST_BINARY_ASSERT(DT_CHECK_LT, lt, __VA_ARGS__)
03256 #define DOCTEST_REQUIRE_LT(...) DOCTEST_BINARY_ASSERT(DT_REQUIRE_LT, lt, __VA_ARGS__)
03257 #define DOCTEST_WARN_GE(...) DOCTEST_BINARY_ASSERT(DT_WARN_GE, ge, __VA_ARGS__)
03258 #define DOCTEST_CHECK_GE(...) DOCTEST_BINARY_ASSERT(DT_CHECK_GE, ge, __VA_ARGS__)
03259 #define DOCTEST_REQUIRE_GE(...) DOCTEST_BINARY_ASSERT(DT_REQUIRE_GE, ge, __VA_ARGS__)
03260 #define DOCTEST_WARN_LE(...) DOCTEST_BINARY_ASSERT(DT_WARN_LE, le, __VA_ARGS__)
03261 #define DOCTEST_CHECK_LE(...) DOCTEST_BINARY_ASSERT(DT_CHECK_LE, le, __VA_ARGS__)
03262 #define DOCTEST_REQUIRE_LE(...) DOCTEST_BINARY_ASSERT(DT_REQUIRE_LE, le, __VA_ARGS__)
03263
03264 #define DOCTEST_WARN_UNARY(...) DOCTEST_UNARY_ASSERT(DT_WARN_UNARY, __VA_ARGS__)
03265 #define DOCTEST_CHECK_UNARY(...) DOCTEST_UNARY_ASSERT(DT_CHECK_UNARY, __VA_ARGS__)
03266 #define DOCTEST_REQUIRE_UNARY(...) DOCTEST_UNARY_ASSERT(DT_REQUIRE_UNARY, __VA_ARGS__)
03267 #define DOCTEST_WARN_UNARY_FALSE(...) DOCTEST_UNARY_ASSERT(DT_WARN_UNARY_FALSE, __VA_ARGS__)
03268 #define DOCTEST_CHECK_UNARY_FALSE(...) DOCTEST_UNARY_ASSERT(DT_CHECK_UNARY_FALSE, __VA_ARGS__)
03269 #define DOCTEST_REQUIRE_UNARY_FALSE(...) DOCTEST_UNARY_ASSERT(DT_REQUIRE_UNARY_FALSE, __VA_ARGS__)
03270
03271 #ifndef DOCTEST_CONFIG_NO_EXCEPTIONS
03272
03273 #define DOCTEST_ASSERT_THROWS_AS(expr, assert_type, message, ...)
03274 \
DOCTEST_FUNC_SCOPE_BEGIN {
03275 \
    if (!doctest::getContextOptions()->no_throw) {
03276 \
        doctest::detail::ResultBuilder DOCTEST_RB(
03277 \
            doctest::assertType::assert_type, __FILE__, __LINE__, #expr, #__VA_ARGS__, message
03278 \
        );
03279 \
        try {
03280 \
            DOCTEST_CAST_TO_VOID(expr)
03281 \
        } catch (const typename doctest::detail::types::remove_const<
03282 \
            typename doctest::detail::types::remove_reference<__VA_ARGS__>::type>::type &) {
03283 \
            DOCTEST_RB.translateException();
03284 \
            DOCTEST_RB.m_threw_as = true;
03285 \
        } catch (...) { DOCTEST_RB.translateException(); }
03286 \
        DOCTEST_ASSERT_LOG_REACT_RETURN(DOCTEST_RB);
03287 \
    } else { /* NOLINT(*-else-after-return) */
03288 \
        DOCTEST_FUNC_SCOPE_RET(false);
03289 \
    }
03290 \
}
03291 \
DOCTEST_FUNC_SCOPE_END
03292
03293 #define DOCTEST_ASSERT_THROWS_WITH(expr, expr_str, assert_type, ...)
03294 \
DOCTEST_FUNC_SCOPE_BEGIN {
03295 \
    if (!doctest::getContextOptions()->no_throw) {
03296 \
        doctest::detail::ResultBuilder DOCTEST_RB(
03297 \
            doctest::assertType::assert_type, __FILE__, __LINE__, expr_str, "", __VA_ARGS__
03298 \
        );
03299 \
        try {
03300 \
            DOCTEST_CAST_TO_VOID(expr)
03301 \
        } catch (...) { DOCTEST_RB.translateException(); }

```

```

03302         DOCTEST_ASSERT_LOG_REACT_RETURN(DOCTEST_RB);
03303     } else { /* NOLINT(*-else-after-return) */
03304         DOCTEST_FUNC_SCOPE_RET(false);
03305     }
03306 }
03307 DOCTEST_FUNC_SCOPE_END
03308
03309 #define DOCTEST_ASSERT_NOTHROW(assert_type, ...)
03310     DOCTEST_FUNC_SCOPE_BEGIN {
03311         doctest::detail::ResultBuilder DOCTEST_RB(doctest::assertType::assert_type, __FILE__,
03312         __LINE__, #__VA_ARGS__); \
03313         try {
03314             DOCTEST_CAST_TO_VOID(__VA_ARGS__)
03315         } catch (...) { DOCTEST_RB.translateException(); }
03316         DOCTEST_ASSERT_LOG_REACT_RETURN(DOCTEST_RB);
03317     }
03318 DOCTEST_FUNC_SCOPE_END
03319 // clang-format off
03320 #define DOCTEST_WARN_THROWS(...) DOCTEST_ASSERT_THROWS_WITH((__VA_ARGS__), #__VA_ARGS__,
03321     DT_WARN_THROWS, "")
03322 #define DOCTEST_CHECK_THROWS(...) DOCTEST_ASSERT_THROWS_WITH((__VA_ARGS__), #__VA_ARGS__,
03323     DT_CHECK_THROWS, "")
03324 #define DOCTEST_REQUIRE_THROWS(...) DOCTEST_ASSERT_THROWS_WITH((__VA_ARGS__), #__VA_ARGS__,
03325     DT_REQUIRE_THROWS, "")
03326 #define DOCTEST_WARN_THROWS_AS(expr, ...) DOCTEST_ASSERT_THROWS_AS(expr, DT_WARN_THROWS_AS, "",
03327     __VA_ARGS__)
03328 #define DOCTEST_CHECK_THROWS_AS(expr, ...) DOCTEST_ASSERT_THROWS_AS(expr, DT_CHECK_THROWS_AS, "",
03329     __VA_ARGS__)
03330 #define DOCTEST_REQUIRE_THROWS_AS(expr, ...) DOCTEST_ASSERT_THROWS_AS(expr, DT_REQUIRE_THROWS_AS, "",
03331     __VA_ARGS__)
03332 #define DOCTEST_WARN_THROWS_WITH(expr, ...) DOCTEST_ASSERT_THROWS_WITH(expr, #expr,
03333     DT_WARN_THROWS_WITH, __VA_ARGS__)
03334 #define DOCTEST_CHECK_THROWS_WITH(expr, ...) DOCTEST_ASSERT_THROWS_WITH(expr, #expr,
03335     DT_CHECK_THROWS_WITH, __VA_ARGS__)
03336 #define DOCTEST_REQUIRE_THROWS_WITH(expr, ...) DOCTEST_ASSERT_THROWS_WITH(expr, #expr,
03337     DT_REQUIRE_THROWS_WITH, __VA_ARGS__)
03338 #define DOCTEST_WARN_THROWS_WITH_AS(expr, message, ...) DOCTEST_ASSERT_THROWS_AS(expr,
03339     DT_WARN_THROWS_WITH_AS, message, __VA_ARGS__)
03340 #define DOCTEST_CHECK_THROWS_WITH_AS(expr, message, ...) DOCTEST_ASSERT_THROWS_AS(expr,
03341     DT_CHECK_THROWS_WITH_AS, message, __VA_ARGS__)
03342 #define DOCTEST_REQUIRE_THROWS_WITH_AS(expr, message, ...) DOCTEST_ASSERT_THROWS_AS(expr,
03343     DT_REQUIRE_THROWS_WITH_AS, message, __VA_ARGS__)
03344 #define DOCTEST_WARN_NOTHROW(...) DOCTEST_ASSERT_NOTHROW(DT_WARN_NOTHROW, __VA_ARGS__)
03345 #define DOCTEST_CHECK_NOTHROW(...) DOCTEST_ASSERT_NOTHROW(DT_CHECK_NOTHROW, __VA_ARGS__)
03346 #define DOCTEST_REQUIRE_NOTHROW(...) DOCTEST_ASSERT_NOTHROW(DT_REQUIRE_NOTHROW, __VA_ARGS__)
03347
03348 #define DOCTEST_WARN_THROWS_MESSAGE(expr, ...) DOCTEST_FUNC_SCOPE_BEGIN { DOCTEST_INFO(__VA_ARGS__);
03349     DOCTEST_WARN_THROWS(expr); } DOCTEST_FUNC_SCOPE_END
03350 #define DOCTEST_CHECK_THROWS_MESSAGE(expr, ...) DOCTEST_FUNC_SCOPE_BEGIN { DOCTEST_INFO(__VA_ARGS__);
03351     DOCTEST_CHECK_THROWS(expr); } DOCTEST_FUNC_SCOPE_END
03352 #define DOCTEST_REQUIRE_THROWS_MESSAGE(expr, ...) DOCTEST_FUNC_SCOPE_BEGIN {
03353     DOCTEST_INFO(__VA_ARGS__); DOCTEST_REQUIRE_THROWS(expr, ex); } DOCTEST_FUNC_SCOPE_END
03354 #define DOCTEST_WARN_THROWS_AS_MESSAGE(expr, ex, ...) DOCTEST_FUNC_SCOPE_BEGIN {
03355     DOCTEST_INFO(__VA_ARGS__); DOCTEST_WARN_THROWS_AS(expr, ex); } DOCTEST_FUNC_SCOPE_END
03356 #define DOCTEST_CHECK_THROWS_AS_MESSAGE(expr, ex, ...) DOCTEST_FUNC_SCOPE_BEGIN {
03357     DOCTEST_INFO(__VA_ARGS__); DOCTEST_CHECK_THROWS_AS(expr, ex); } DOCTEST_FUNC_SCOPE_END
03358 #define DOCTEST_REQUIRE_THROWS_AS_MESSAGE(expr, ex, ...) DOCTEST_FUNC_SCOPE_BEGIN {
03359     DOCTEST_INFO(__VA_ARGS__); DOCTEST_REQUIRE_THROWS_AS(expr, ex); } DOCTEST_FUNC_SCOPE_END
03360 #define DOCTEST_WARN_THROWS_WITH_MESSAGE(expr, with, ...) DOCTEST_FUNC_SCOPE_BEGIN {
03361     DOCTEST_INFO(__VA_ARGS__); DOCTEST_WARN_THROWS_WITH(expr, with); } DOCTEST_FUNC_SCOPE_END
03362 #define DOCTEST_CHECK_THROWS_WITH_MESSAGE(expr, with, ...) DOCTEST_FUNC_SCOPE_BEGIN {
03363     DOCTEST_INFO(__VA_ARGS__); DOCTEST_CHECK_THROWS_WITH(expr, with); } DOCTEST_FUNC_SCOPE_END
03364 #define DOCTEST_REQUIRE_THROWS_WITH_MESSAGE(expr, with, ...) DOCTEST_FUNC_SCOPE_BEGIN {
03365     DOCTEST_INFO(__VA_ARGS__); DOCTEST_REQUIRE_THROWS_WITH(expr, with); } DOCTEST_FUNC_SCOPE_END
03366 #define DOCTEST_WARN_THROWS_WITH_AS_MESSAGE(expr, with, ex, ...) DOCTEST_FUNC_SCOPE_BEGIN {
03367     DOCTEST_INFO(__VA_ARGS__); DOCTEST_WARN_THROWS_WITH_AS(expr, with, ex); } DOCTEST_FUNC_SCOPE_END
03368 #define DOCTEST_CHECK_THROWS_WITH_AS_MESSAGE(expr, with, ex, ...) DOCTEST_FUNC_SCOPE_BEGIN {
03369     DOCTEST_INFO(__VA_ARGS__); DOCTEST_CHECK_THROWS_WITH_AS(expr, with, ex); } DOCTEST_FUNC_SCOPE_END
03370 #define DOCTEST_REQUIRE_THROWS_WITH_AS_MESSAGE(expr, with, ex, ...) DOCTEST_FUNC_SCOPE_BEGIN {
03371     DOCTEST_INFO(__VA_ARGS__); DOCTEST_REQUIRE_THROWS_WITH_AS(expr, with, ex); } DOCTEST_FUNC_SCOPE_END

```

```

03352 #define DOCTEST_WARN_NOTHROW_MESSAGE(expr, ...) DOCTEST_FUNC_SCOPE_BEGIN { DOCTEST_INFO(__VA_ARGS__);
DOCTEST_WARN_NOTHROW(expr); } DOCTEST_FUNC_SCOPE_END
03353 #define DOCTEST_CHECK_NOTHROW_MESSAGE(expr, ...) DOCTEST_FUNC_SCOPE_BEGIN { DOCTEST_INFO(__VA_ARGS__);
DOCTEST_CHECK_NOTHROW(expr); } DOCTEST_FUNC_SCOPE_END
03354 #define DOCTEST_REQUIRE_NOTHROW_MESSAGE(expr, ...) DOCTEST_FUNC_SCOPE_BEGIN {
DOCTEST_INFO(__VA_ARGS__); DOCTEST_REQUIRE_NOTHROW(expr); } DOCTEST_FUNC_SCOPE_END
03355 // clang-format on
03356
03357 #endif // DOCTEST_CONFIG_NO_EXCEPTIONS
03358
03359 // =====
03360 // == WHAT FOLLOWS IS VERSIONS OF THE MACROS THAT DO NOT DO ANY REGISTERING! ==
03361 // == THIS CAN BE ENABLED BY DEFINING DOCTEST_CONFIG_DISABLE GLOBALLY! ==
03362 // =====
03363 #else // DOCTEST_CONFIG_DISABLE
03364
03365 #define DOCTEST_IMPLEMENT_FIXTURE(der, base, func, name)
03366 \
03367 \
03368 \
03369 \
03370 \
03371 \
03372 \
03373 \
03374 \
03375 #define DOCTEST_CREATE_AND_REGISTER_FUNCTION(f, name)
03376 \
03377 \
03378 \
03379 // for registering tests
03380 #define DOCTEST_TEST_CASE(name)
DOCTEST_CREATE_AND_REGISTER_FUNCTION(DOCTEST_ANONYMOUS(DOCTEST_ANON_FUNC_), name)
03381
03382 // for registering tests in classes
03383 #define DOCTEST_TEST_CASE_CLASS(name)
DOCTEST_CREATE_AND_REGISTER_FUNCTION(DOCTEST_ANONYMOUS(DOCTEST_ANON_FUNC_), name)
03384
03385 // for registering tests with a fixture
03386 #define DOCTEST_TEST_CASE_FIXTURE(x, name)
03387 \
DOCTEST_IMPLEMENT_FIXTURE(DOCTEST_ANONYMOUS(DOCTEST_ANON_CLASS_), x,
DOCTEST_ANONYMOUS(DOCTEST_ANON_FUNC_), name)
03388
03389 // for converting types to strings without the <typeinfo> header and demangling
03390 #define DOCTEST_TYPE_TO_STRING_AS(str, ...) static_assert(true, "")
03391 #define DOCTEST_TYPE_TO_STRING(...) static_assert(true, "")
03392
03393 // for typed tests
03394 #define DOCTEST_TEST_CASE_TEMPLATE(name, type, ...)
03395 \
03396 \
03397 \
03398 #define DOCTEST_TEST_CASE_TEMPLATE_DEFINE(name, type, id)
03399 \
03400 \
03401 \
03402 #define DOCTEST_TEST_CASE_TEMPLATE_INVOKE(id, ...) static_assert(true, "")
03403 #define DOCTEST_TEST_CASE_TEMPLATE_APPLY(id, ...) static_assert(true, "")
03404
03405 // for subcases
03406 #define DOCTEST_SUBCASE(name)
03407
03408 // for generating value-parameterized test inputs
03409 #define DOCTEST_GENERATE_IMPL(first, ...) (first)
03410 #define DOCTEST_GENERATE(...) DOCTEST_GENERATE_IMPL(__VA_ARGS__, DOCTEST_EMPTY)
03411
03412 // for a testsuite block
03413 #define DOCTEST_TEST_SUITE(name) namespace // NOLINT
03414
03415 // for starting a testsuite block
03416 #define DOCTEST_TEST_SUITE_BEGIN(name) static_assert(true, "")
03417

```

```

03418 // for ending a testsuite block
03419 #define DOCTEST_TEST_SUITE_END using DOCTEST_ANONYMOUS(DOCTEST_ANON_FOR_SEMICOLON_) = int
03420
03421 #define DOCTEST_REGISTER_EXCEPTION_TRANSLATOR(signature)
03422 \
03423 \
03424 \
03425 \
03426 \
03427 \
03428 \
03429 \
03430 \
03431 \
03432 \
03433 \
03434 \
03435 \
03436 \
03437 #if defined(DOCTEST_CONFIG_EVALUATE_ASSERTS_EVEN_WHEN_DISABLED) &&
03438 \
03439 \
03440 \
03441 \
03442 \
03443 \
03444 \
03445 \
03446 \
03447 \
03448 \
03449 \
03450 \
03451 \
03452 \
03453 namespace doctest {
03454 namespace detail {
03455 #define DOCTEST_RELATIONAL_OP(name, op)
03456 \
03457 \
03458 \
03459 \
03460 \
03461 DOCTEST_RELATIONAL_OP(eq, ==)
03462 DOCTEST_RELATIONAL_OP(ne, !=)
03463 DOCTEST_RELATIONAL_OP(lt, <)
03464 DOCTEST_RELATIONAL_OP(gt, >)
03465 DOCTEST_RELATIONAL_OP(le, <=)
03466 DOCTEST_RELATIONAL_OP(ge, >=)
03467 } // namespace detail
03468 } // namespace doctest
03469
03470 #define DOCTEST_WARN_EQ(...) [&] { return doctest::detail::eq(__VA_ARGS__); }()
03471 #define DOCTEST_CHECK_EQ(...) [&] { return doctest::detail::eq(__VA_ARGS__); }()
03472 #define DOCTEST_REQUIRE_EQ(...) [&] { return doctest::detail::eq(__VA_ARGS__); }()
03473 #define DOCTEST_WARN_NE(...) [&] { return doctest::detail::ne(__VA_ARGS__); }()
03474 #define DOCTEST_CHECK_NE(...) [&] { return doctest::detail::ne(__VA_ARGS__); }()
03475 #define DOCTEST_REQUIRE_NE(...) [&] { return doctest::detail::ne(__VA_ARGS__); }()
03476 #define DOCTEST_WARN_LT(...) [&] { return doctest::detail::lt(__VA_ARGS__); }()
03477 #define DOCTEST_CHECK_LT(...) [&] { return doctest::detail::lt(__VA_ARGS__); }()
03478 #define DOCTEST_REQUIRE_LT(...) [&] { return doctest::detail::lt(__VA_ARGS__); }()
03479 #define DOCTEST_WARN_GT(...) [&] { return doctest::detail::gt(__VA_ARGS__); }()
03480 #define DOCTEST_CHECK_GT(...) [&] { return doctest::detail::gt(__VA_ARGS__); }()
03481 #define DOCTEST_REQUIRE_GT(...) [&] { return doctest::detail::gt(__VA_ARGS__); }()
03482 #define DOCTEST_WARN_LE(...) [&] { return doctest::detail::le(__VA_ARGS__); }()
03483 #define DOCTEST_CHECK_LE(...) [&] { return doctest::detail::le(__VA_ARGS__); }()
03484 #define DOCTEST_REQUIRE_LE(...) [&] { return doctest::detail::le(__VA_ARGS__); }()
03485 #define DOCTEST_WARN_GE(...) [&] { return doctest::detail::ge(__VA_ARGS__); }()
03486 #define DOCTEST_CHECK_GE(...) [&] { return doctest::detail::ge(__VA_ARGS__); }()
03487 #define DOCTEST_REQUIRE_GE(...) [&] { return doctest::detail::ge(__VA_ARGS__); }()
03488 #define DOCTEST_WARN_UNARY(...) [&] { return __VA_ARGS__; }()
03489 #define DOCTEST_CHECK_UNARY(...) [&] { return __VA_ARGS__; }()
03490 #define DOCTEST_REQUIRE_UNARY(...) [&] { return __VA_ARGS__; }()
03491 #define DOCTEST_WARN_UNARY_FALSE(...) [&] { return !(__VA_ARGS__); }()
03492 #define DOCTEST_CHECK_UNARY_FALSE(...) [&] { return !(__VA_ARGS__); }()
03493 #define DOCTEST_REQUIRE_UNARY_FALSE(...) [&] { return !(__VA_ARGS__); }()
03494
03495 #ifndef DOCTEST_CONFIG_NO_EXCEPTIONS
03496
03497 #define DOCTEST_WARN_THROWS_WITH(expr, with, ...)

```

```

\
03498     [] {
\
03499         static_assert(false, "Exception translation is not available when doctest is disabled.");
\
03500         return false;
\
03501     }()
03502 #define DOCTEST_CHECK_THROWS_WITH(expr, with, ...) DOCTEST_WARN_THROWS_WITH(, , )
03503 #define DOCTEST_REQUIRE_THROWS_WITH(expr, with, ...) DOCTEST_WARN_THROWS_WITH(, , )
03504 #define DOCTEST_WARN_THROWS_WITH_AS(expr, with, ex, ...) DOCTEST_WARN_THROWS_WITH(, , )
03505 #define DOCTEST_CHECK_THROWS_WITH_AS(expr, with, ex, ...) DOCTEST_WARN_THROWS_WITH(, , )
03506 #define DOCTEST_REQUIRE_THROWS_WITH_AS(expr, with, ex, ...) DOCTEST_WARN_THROWS_WITH(, , )
03507
03508 #define DOCTEST_WARN_THROWS_WITH_MESSAGE(expr, with, ...) DOCTEST_WARN_THROWS_WITH(, , )
03509 #define DOCTEST_CHECK_THROWS_WITH_MESSAGE(expr, with, ...) DOCTEST_WARN_THROWS_WITH(, , )
03510 #define DOCTEST_REQUIRE_THROWS_WITH_MESSAGE(expr, with, ...) DOCTEST_WARN_THROWS_WITH(, , )
03511 #define DOCTEST_WARN_THROWS_WITH_AS_MESSAGE(expr, with, ex, ...) DOCTEST_WARN_THROWS_WITH(, , )
03512 #define DOCTEST_CHECK_THROWS_WITH_AS_MESSAGE(expr, with, ex, ...) DOCTEST_WARN_THROWS_WITH(, , )
03513 #define DOCTEST_REQUIRE_THROWS_WITH_AS_MESSAGE(expr, with, ex, ...) DOCTEST_WARN_THROWS_WITH(, , )
03514
03515 // clang-format off
03516 #define DOCTEST_WARN_THROWS(...) [&] { try { __VA_ARGS__; return false; } catch (...) { return true; } }()
03517 #define DOCTEST_CHECK_THROWS(...) [&] { try { __VA_ARGS__; return false; } catch (...) { return true; } }()
03518 #define DOCTEST_REQUIRE_THROWS(...) [&] { try { __VA_ARGS__; return false; } catch (...) { return true; } }()
03519 #define DOCTEST_WARN_THROWS_AS(expr, ...) [&] { try { expr; } catch (__VA_ARGS__) { return true; } catch (...) { } return false; }()
03520 #define DOCTEST_CHECK_THROWS_AS(expr, ...) [&] { try { expr; } catch (__VA_ARGS__) { return true; } catch (...) { } return false; }()
03521 #define DOCTEST_REQUIRE_THROWS_AS(expr, ...) [&] { try { expr; } catch (__VA_ARGS__) { return true; } catch (...) { } return false; }()
03522 #define DOCTEST_WARN_NOTHROW(...) [&] { try { __VA_ARGS__; return true; } catch (...) { return false; } }()
03523 #define DOCTEST_CHECK_NOTHROW(...) [&] { try { __VA_ARGS__; return true; } catch (...) { return false; } }()
03524 #define DOCTEST_REQUIRE_NOTHROW(...) [&] { try { __VA_ARGS__; return true; } catch (...) { return false; } }()
03525
03526 #define DOCTEST_WARN_THROWS_MESSAGE(expr, ...) [&] { try { __VA_ARGS__; return false; } catch (...) { return true; } }()
03527 #define DOCTEST_CHECK_THROWS_MESSAGE(expr, ...) [&] { try { __VA_ARGS__; return false; } catch (...) { return true; } }()
03528 #define DOCTEST_REQUIRE_THROWS_MESSAGE(expr, ...) [&] { try { __VA_ARGS__; return false; } catch (...) { return true; } }()
03529 #define DOCTEST_WARN_THROWS_AS_MESSAGE(expr, ex, ...) [&] { try { expr; } catch (__VA_ARGS__) { return true; } catch (...) { } return false; }()
03530 #define DOCTEST_CHECK_THROWS_AS_MESSAGE(expr, ex, ...) [&] { try { expr; } catch (__VA_ARGS__) { return true; } catch (...) { } return false; }()
03531 #define DOCTEST_REQUIRE_THROWS_AS_MESSAGE(expr, ex, ...) [&] { try { expr; } catch (__VA_ARGS__) { return true; } catch (...) { } return false; }()
03532 #define DOCTEST_WARN_NOTHROW_MESSAGE(expr, ...) [&] { try { __VA_ARGS__; return true; } catch (...) { return false; } }()
03533 #define DOCTEST_CHECK_NOTHROW_MESSAGE(expr, ...) [&] { try { __VA_ARGS__; return true; } catch (...) { return false; } }()
03534 #define DOCTEST_REQUIRE_NOTHROW_MESSAGE(expr, ...) [&] { try { __VA_ARGS__; return true; } catch (...) { return false; } }()
03535 // clang-format on
03536
03537 #endif // DOCTEST_CONFIG_NO_EXCEPTIONS
03538
03539 #else // DOCTEST_CONFIG_EVALUATE_ASSERTS_EVEN_WHEN_DISABLED
03540
03541 #define DOCTEST_WARN(...) DOCTEST_FUNC_EMPTY
03542 #define DOCTEST_CHECK(...) DOCTEST_FUNC_EMPTY
03543 #define DOCTEST_REQUIRE(...) DOCTEST_FUNC_EMPTY
03544 #define DOCTEST_WARN_FALSE(...) DOCTEST_FUNC_EMPTY
03545 #define DOCTEST_CHECK_FALSE(...) DOCTEST_FUNC_EMPTY
03546 #define DOCTEST_REQUIRE_FALSE(...) DOCTEST_FUNC_EMPTY
03547
03548 #define DOCTEST_WARN_MESSAGE(cond, ...) DOCTEST_FUNC_EMPTY
03549 #define DOCTEST_CHECK_MESSAGE(cond, ...) DOCTEST_FUNC_EMPTY
03550 #define DOCTEST_REQUIRE_MESSAGE(cond, ...) DOCTEST_FUNC_EMPTY
03551 #define DOCTEST_WARN_FALSE_MESSAGE(cond, ...) DOCTEST_FUNC_EMPTY
03552 #define DOCTEST_CHECK_FALSE_MESSAGE(cond, ...) DOCTEST_FUNC_EMPTY
03553 #define DOCTEST_REQUIRE_FALSE_MESSAGE(cond, ...) DOCTEST_FUNC_EMPTY
03554
03555 #define DOCTEST_WARN_EQ(...) DOCTEST_FUNC_EMPTY
03556 #define DOCTEST_CHECK_EQ(...) DOCTEST_FUNC_EMPTY
03557 #define DOCTEST_REQUIRE_EQ(...) DOCTEST_FUNC_EMPTY
03558 #define DOCTEST_WARN_NE(...) DOCTEST_FUNC_EMPTY
03559 #define DOCTEST_CHECK_NE(...) DOCTEST_FUNC_EMPTY
03560 #define DOCTEST_REQUIRE_NE(...) DOCTEST_FUNC_EMPTY
03561 #define DOCTEST_WARN_GT(...) DOCTEST_FUNC_EMPTY
03562 #define DOCTEST_CHECK_GT(...) DOCTEST_FUNC_EMPTY

```



```

03563 #define DOCTEST_REQUIRE_GT(...) DOCTEST_FUNC_EMPTY
03564 #define DOCTEST_WARN_LT(...) DOCTEST_FUNC_EMPTY
03565 #define DOCTEST_CHECK_LT(...) DOCTEST_FUNC_EMPTY
03566 #define DOCTEST_REQUIRE_LT(...) DOCTEST_FUNC_EMPTY
03567 #define DOCTEST_WARN_GE(...) DOCTEST_FUNC_EMPTY
03568 #define DOCTEST_CHECK_GE(...) DOCTEST_FUNC_EMPTY
03569 #define DOCTEST_REQUIRE_GE(...) DOCTEST_FUNC_EMPTY
03570 #define DOCTEST_WARN_LE(...) DOCTEST_FUNC_EMPTY
03571 #define DOCTEST_CHECK_LE(...) DOCTEST_FUNC_EMPTY
03572 #define DOCTEST_REQUIRE_LE(...) DOCTEST_FUNC_EMPTY
03573
03574 #define DOCTEST_WARN_UNARY(...) DOCTEST_FUNC_EMPTY
03575 #define DOCTEST_CHECK_UNARY(...) DOCTEST_FUNC_EMPTY
03576 #define DOCTEST_REQUIRE_UNARY(...) DOCTEST_FUNC_EMPTY
03577 #define DOCTEST_WARN_UNARY_FALSE(...) DOCTEST_FUNC_EMPTY
03578 #define DOCTEST_CHECK_UNARY_FALSE(...) DOCTEST_FUNC_EMPTY
03579 #define DOCTEST_REQUIRE_UNARY_FALSE(...) DOCTEST_FUNC_EMPTY
03580
03581 #ifndef DOCTEST_CONFIG_NO_EXCEPTIONS
03582
03583 #define DOCTEST_WARN_THROWS(...) DOCTEST_FUNC_EMPTY
03584 #define DOCTEST_CHECK_THROWS(...) DOCTEST_FUNC_EMPTY
03585 #define DOCTEST_REQUIRE_THROWS(...) DOCTEST_FUNC_EMPTY
03586 #define DOCTEST_WARN_THROWS_AS(expr, ...) DOCTEST_FUNC_EMPTY
03587 #define DOCTEST_CHECK_THROWS_AS(expr, ...) DOCTEST_FUNC_EMPTY
03588 #define DOCTEST_REQUIRE_THROWS_AS(expr, ...) DOCTEST_FUNC_EMPTY
03589 #define DOCTEST_WARN_THROWS_WITH(expr, ...) DOCTEST_FUNC_EMPTY
03590 #define DOCTEST_CHECK_THROWS_WITH(expr, ...) DOCTEST_FUNC_EMPTY
03591 #define DOCTEST_REQUIRE_THROWS_WITH(expr, ...) DOCTEST_FUNC_EMPTY
03592 #define DOCTEST_WARN_THROWS_WITH_AS(expr, with, ...) DOCTEST_FUNC_EMPTY
03593 #define DOCTEST_CHECK_THROWS_WITH_AS(expr, with, ...) DOCTEST_FUNC_EMPTY
03594 #define DOCTEST_REQUIRE_THROWS_WITH_AS(expr, with, ...) DOCTEST_FUNC_EMPTY
03595 #define DOCTEST_WARN_NOTHROW(...) DOCTEST_FUNC_EMPTY
03596 #define DOCTEST_CHECK_NOTHROW(...) DOCTEST_FUNC_EMPTY
03597 #define DOCTEST_REQUIRE_NOTHROW(...) DOCTEST_FUNC_EMPTY
03598
03599 #define DOCTEST_WARN_THROWS_MESSAGE(expr, ...) DOCTEST_FUNC_EMPTY
03600 #define DOCTEST_CHECK_THROWS_MESSAGE(expr, ...) DOCTEST_FUNC_EMPTY
03601 #define DOCTEST_REQUIRE_THROWS_MESSAGE(expr, ...) DOCTEST_FUNC_EMPTY
03602 #define DOCTEST_WARN_THROWS_AS_MESSAGE(expr, ex, ...) DOCTEST_FUNC_EMPTY
03603 #define DOCTEST_CHECK_THROWS_AS_MESSAGE(expr, ex, ...) DOCTEST_FUNC_EMPTY
03604 #define DOCTEST_REQUIRE_THROWS_AS_MESSAGE(expr, ex, ...) DOCTEST_FUNC_EMPTY
03605 #define DOCTEST_WARN_THROWS_WITH_MESSAGE(expr, with, ...) DOCTEST_FUNC_EMPTY
03606 #define DOCTEST_CHECK_THROWS_WITH_MESSAGE(expr, with, ...) DOCTEST_FUNC_EMPTY
03607 #define DOCTEST_REQUIRE_THROWS_WITH_MESSAGE(expr, with, ...) DOCTEST_FUNC_EMPTY
03608 #define DOCTEST_WARN_THROWS_WITH_AS_MESSAGE(expr, with, ex, ...) DOCTEST_FUNC_EMPTY
03609 #define DOCTEST_CHECK_THROWS_WITH_AS_MESSAGE(expr, with, ex, ...) DOCTEST_FUNC_EMPTY
03610 #define DOCTEST_REQUIRE_THROWS_WITH_AS_MESSAGE(expr, with, ex, ...) DOCTEST_FUNC_EMPTY
03611 #define DOCTEST_WARN_NOTHROW_MESSAGE(expr, ...) DOCTEST_FUNC_EMPTY
03612 #define DOCTEST_CHECK_NOTHROW_MESSAGE(expr, ...) DOCTEST_FUNC_EMPTY
03613 #define DOCTEST_REQUIRE_NOTHROW_MESSAGE(expr, ...) DOCTEST_FUNC_EMPTY
03614
03615 #endif // DOCTEST_CONFIG_NO_EXCEPTIONS
03616
03617 #endif // DOCTEST_CONFIG_EVALUATE_ASSERTS_EVEN_WHEN_DISABLED
03618
03619 #endif // DOCTEST_CONFIG_DISABLE
03620
03621 #ifndef DOCTEST_CONFIG_NO_EXCEPTIONS
03622
03623 #ifdef DOCTEST_CONFIG_NO_EXCEPTIONS_BUT_WITH_ALL_ASSERTS
03624 #define DOCTEST_EXCEPTION_EMPTY_FUNC DOCTEST_FUNC_EMPTY
03625 #else // DOCTEST_CONFIG_NO_EXCEPTIONS_BUT_WITH_ALL_ASSERTS
03626 #define DOCTEST_EXCEPTION_EMPTY_FUNC
03627 \
03628 \
03629 \
03630 \
03631 \
03632 \
03633 \
03634 \
03635 \
03636 \
03637 #undef DOCTEST_REQUIRE
03638 #undef DOCTEST_REQUIRE_FALSE
03639 #undef DOCTEST_REQUIRE_MESSAGE
03640 #undef DOCTEST_REQUIRE_FALSE_MESSAGE

```



```

03641 #undef DOCTEST_REQUIRE_EQ
03642 #undef DOCTEST_REQUIRE_NE
03643 #undef DOCTEST_REQUIRE_GT
03644 #undef DOCTEST_REQUIRE_LT
03645 #undef DOCTEST_REQUIRE_GE
03646 #undef DOCTEST_REQUIRE_LE
03647 #undef DOCTEST_REQUIRE_UNARY
03648 #undef DOCTEST_REQUIRE_UNARY_FALSE
03649
03650 #define DOCTEST_REQUIRE DOCTEST_EXCEPTION_EMPTY_FUNC
03651 #define DOCTEST_REQUIRE_FALSE DOCTEST_EXCEPTION_EMPTY_FUNC
03652 #define DOCTEST_REQUIRE_MESSAGE DOCTEST_EXCEPTION_EMPTY_FUNC
03653 #define DOCTEST_REQUIRE_FALSE_MESSAGE DOCTEST_EXCEPTION_EMPTY_FUNC
03654 #define DOCTEST_REQUIRE_EQ DOCTEST_EXCEPTION_EMPTY_FUNC
03655 #define DOCTEST_REQUIRE_NE DOCTEST_EXCEPTION_EMPTY_FUNC
03656 #define DOCTEST_REQUIRE_GT DOCTEST_EXCEPTION_EMPTY_FUNC
03657 #define DOCTEST_REQUIRE_LT DOCTEST_EXCEPTION_EMPTY_FUNC
03658 #define DOCTEST_REQUIRE_GE DOCTEST_EXCEPTION_EMPTY_FUNC
03659 #define DOCTEST_REQUIRE_LE DOCTEST_EXCEPTION_EMPTY_FUNC
03660 #define DOCTEST_REQUIRE_UNARY DOCTEST_EXCEPTION_EMPTY_FUNC
03661 #define DOCTEST_REQUIRE_UNARY_FALSE DOCTEST_EXCEPTION_EMPTY_FUNC
03662
03663 #endif // DOCTEST_CONFIG_NO_EXCEPTIONS_BUT_WITH_ALL_ASSERTS
03664
03665 #define DOCTEST_WARN_THROWS(...) DOCTEST_EXCEPTION_EMPTY_FUNC
03666 #define DOCTEST_CHECK_THROWS(...) DOCTEST_EXCEPTION_EMPTY_FUNC
03667 #define DOCTEST_REQUIRE_THROWS(...) DOCTEST_EXCEPTION_EMPTY_FUNC
03668 #define DOCTEST_WARN_THROWS_AS(expr, ...) DOCTEST_EXCEPTION_EMPTY_FUNC
03669 #define DOCTEST_CHECK_THROWS_AS(expr, ...) DOCTEST_EXCEPTION_EMPTY_FUNC
03670 #define DOCTEST_REQUIRE_THROWS_AS(expr, ...) DOCTEST_EXCEPTION_EMPTY_FUNC
03671 #define DOCTEST_WARN_THROWS_WITH(expr, ...) DOCTEST_EXCEPTION_EMPTY_FUNC
03672 #define DOCTEST_CHECK_THROWS_WITH(expr, ...) DOCTEST_EXCEPTION_EMPTY_FUNC
03673 #define DOCTEST_REQUIRE_THROWS_WITH(expr, ...) DOCTEST_EXCEPTION_EMPTY_FUNC
03674 #define DOCTEST_WARN_THROWS_WITH_AS(expr, with, ...) DOCTEST_EXCEPTION_EMPTY_FUNC
03675 #define DOCTEST_CHECK_THROWS_WITH_AS(expr, with, ...) DOCTEST_EXCEPTION_EMPTY_FUNC
03676 #define DOCTEST_REQUIRE_THROWS_WITH_AS(expr, with, ...) DOCTEST_EXCEPTION_EMPTY_FUNC
03677 #define DOCTEST_WARN_NOTHROW(...) DOCTEST_EXCEPTION_EMPTY_FUNC
03678 #define DOCTEST_CHECK_NOTHROW(...) DOCTEST_EXCEPTION_EMPTY_FUNC
03679 #define DOCTEST_REQUIRE_NOTHROW(...) DOCTEST_EXCEPTION_EMPTY_FUNC
03680
03681 #define DOCTEST_WARN_THROWS_MESSAGE(expr, ...) DOCTEST_EXCEPTION_EMPTY_FUNC
03682 #define DOCTEST_CHECK_THROWS_MESSAGE(expr, ...) DOCTEST_EXCEPTION_EMPTY_FUNC
03683 #define DOCTEST_REQUIRE_THROWS_MESSAGE(expr, ...) DOCTEST_EXCEPTION_EMPTY_FUNC
03684 #define DOCTEST_WARN_THROWS_AS_MESSAGE(expr, ex, ...) DOCTEST_EXCEPTION_EMPTY_FUNC
03685 #define DOCTEST_CHECK_THROWS_AS_MESSAGE(expr, ex, ...) DOCTEST_EXCEPTION_EMPTY_FUNC
03686 #define DOCTEST_REQUIRE_THROWS_AS_MESSAGE(expr, ex, ...) DOCTEST_EXCEPTION_EMPTY_FUNC
03687 #define DOCTEST_WARN_THROWS_WITH_MESSAGE(expr, with, ...) DOCTEST_EXCEPTION_EMPTY_FUNC
03688 #define DOCTEST_CHECK_THROWS_WITH_AS_MESSAGE(expr, with, ...) DOCTEST_EXCEPTION_EMPTY_FUNC
03689 #define DOCTEST_REQUIRE_THROWS_WITH_MESSAGE(expr, with, ...) DOCTEST_EXCEPTION_EMPTY_FUNC
03690 #define DOCTEST_WARN_THROWS_WITH_AS_MESSAGE(expr, with, ex, ...) DOCTEST_EXCEPTION_EMPTY_FUNC
03691 #define DOCTEST_CHECK_THROWS_WITH_AS_MESSAGE(expr, with, ex, ...) DOCTEST_EXCEPTION_EMPTY_FUNC
03692 #define DOCTEST_REQUIRE_THROWS_WITH_AS_MESSAGE(expr, with, ex, ...) DOCTEST_EXCEPTION_EMPTY_FUNC
03693 #define DOCTEST_WARN_NOTHROW_MESSAGE(expr, ...) DOCTEST_EXCEPTION_EMPTY_FUNC
03694 #define DOCTEST_CHECK_NOTHROW_MESSAGE(expr, ...) DOCTEST_EXCEPTION_EMPTY_FUNC
03695 #define DOCTEST_REQUIRE_NOTHROW_MESSAGE(expr, ...) DOCTEST_EXCEPTION_EMPTY_FUNC
03696
03697 #endif // DOCTEST_CONFIG_NO_EXCEPTIONS
03698
03699 // KEPT FOR BACKWARDS COMPATIBILITY - FORWARDING TO THE RIGHT MACROS
03700 #define DOCTEST_FAST_WARN_EQ DOCTEST_WARN_EQ
03701 #define DOCTEST_FAST_CHECK_EQ DOCTEST_CHECK_EQ
03702 #define DOCTEST_FAST_REQUIRE_EQ DOCTEST_REQUIRE_EQ
03703 #define DOCTEST_FAST_WARN_NE DOCTEST_WARN_NE
03704 #define DOCTEST_FAST_CHECK_NE DOCTEST_CHECK_NE
03705 #define DOCTEST_FAST_REQUIRE_NE DOCTEST_REQUIRE_NE
03706 #define DOCTEST_FAST_WARN_GT DOCTEST_WARN_GT
03707 #define DOCTEST_FAST_CHECK_GT DOCTEST_CHECK_GT
03708 #define DOCTEST_FAST_REQUIRE_GT DOCTEST_REQUIRE_GT
03709 #define DOCTEST_FAST_WARN_LT DOCTEST_WARN_LT
03710 #define DOCTEST_FAST_CHECK_LT DOCTEST_CHECK_LT
03711 #define DOCTEST_FAST_REQUIRE_LT DOCTEST_REQUIRE_LT
03712 #define DOCTEST_FAST_WARN_GE DOCTEST_WARN_GE
03713 #define DOCTEST_FAST_CHECK_GE DOCTEST_CHECK_GE
03714 #define DOCTEST_FAST_REQUIRE_GE DOCTEST_REQUIRE_GE
03715 #define DOCTEST_FAST_WARN_LE DOCTEST_WARN_LE
03716 #define DOCTEST_FAST_CHECK_LE DOCTEST_CHECK_LE
03717 #define DOCTEST_FAST_REQUIRE_LE DOCTEST_REQUIRE_LE
03718
03719 #define DOCTEST_FAST_WARN_UNARY DOCTEST_WARN_UNARY
03720 #define DOCTEST_FAST_CHECK_UNARY DOCTEST_CHECK_UNARY
03721 #define DOCTEST_FAST_REQUIRE_UNARY DOCTEST_REQUIRE_UNARY
03722 #define DOCTEST_FAST_WARN_UNARY_FALSE DOCTEST_WARN_UNARY_FALSE
03723 #define DOCTEST_FAST_CHECK_UNARY_FALSE DOCTEST_CHECK_UNARY_FALSE
03724 #define DOCTEST_FAST_REQUIRE_UNARY_FALSE DOCTEST_REQUIRE_UNARY_FALSE
03725
03726 #define DOCTEST_TEST_CASE_TEMPLATE_INSTANTIATE(id, ...) DOCTEST_TEST_CASE_TEMPLATE_INVOKE(id,
    __VA_ARGS__)

```

```

03727
03728 // BDD style macros
03729 // clang-format off
03730 #define DOCTEST_SCENARIO(name) DOCTEST_TEST_CASE(" Scenario: "
name)
03731 #define DOCTEST_SCENARIO_CLASS(name) DOCTEST_TEST_CASE_CLASS(" Scenario: "
name)
03732 #define DOCTEST_SCENARIO_METHOD(x, name) DOCTEST_TEST_CASE_FIXTURE(x, " Scenario: "
name)
03733 #define DOCTEST_SCENARIO_TEMPLATE(name, T, ...) DOCTEST_TEST_CASE_TEMPLATE(" Scenario: "
name, T, __VA_ARGS__)
03734 #define DOCTEST_SCENARIO_TEMPLATE_DEFINE(name, T, id) DOCTEST_TEST_CASE_TEMPLATE_DEFINE(" Scenario: "
name, T, id)
03735
03736 #define DOCTEST_GIVEN(name) DOCTEST_SUBCASE(" Given: " name)
03737 #define DOCTEST_AND_GIVEN(name) DOCTEST_SUBCASE(" And: " name)
03738 #define DOCTEST_WHEN(name) DOCTEST_SUBCASE(" When: " name)
03739 #define DOCTEST_AND_WHEN(name) DOCTEST_SUBCASE(" And: " name)
03740 #define DOCTEST_THEN(name) DOCTEST_SUBCASE(" Then: " name)
03741 #define DOCTEST_AND_THEN(name) DOCTEST_SUBCASE(" And: " name)
03742 // clang-format on
03743
03744 // == SHORT VERSIONS OF THE MACROS
03745 #ifndef DOCTEST_CONFIG_NO_SHORT_MACRO_NAMES
03746
03747 #define TEST_CASE(name) DOCTEST_TEST_CASE(name)
03748 #define TEST_CASE_CLASS(name) DOCTEST_TEST_CASE_CLASS(name)
03749 #define TEST_CASE_FIXTURE(x, name) DOCTEST_TEST_CASE_FIXTURE(x, name)
03750 #define TYPE_TO_STRING_AS(str, ...) DOCTEST_TYPE_TO_STRING_AS(str, __VA_ARGS__)
03751 #define TYPE_TO_STRING(...) DOCTEST_TYPE_TO_STRING(__VA_ARGS__)
03752 #define TEST_CASE_TEMPLATE(name, T, ...) DOCTEST_TEST_CASE_TEMPLATE(name, T, __VA_ARGS__)
03753 #define TEST_CASE_TEMPLATE_DEFINE(name, T, id) DOCTEST_TEST_CASE_TEMPLATE_DEFINE(name, T, id)
03754 #define TEST_CASE_TEMPLATE_INVOKE(id, ...) DOCTEST_TEST_CASE_TEMPLATE_INVOKE(id, __VA_ARGS__)
03755 #define TEST_CASE_TEMPLATE_APPLY(id, ...) DOCTEST_TEST_CASE_TEMPLATE_APPLY(id, __VA_ARGS__)
03756 #define SUBCASE(name) DOCTEST_SUBCASE(name)
03757 #define GENERATE(...) DOCTEST_GENERATE(__VA_ARGS__)
03758 #define TEST_SUITE(decorators) DOCTEST_TEST_SUITE(decorators)
03759 #define TEST_SUITE_BEGIN(name) DOCTEST_TEST_SUITE_BEGIN(name)
03760 #define TEST_SUITE_END DOCTEST_TEST_SUITE_END
03761 #define REGISTER_EXCEPTION_TRANSLATOR(signature) DOCTEST_REGISTER_EXCEPTION_TRANSLATOR(signature)
03762 #define REGISTER_REPORTER(name, priority, reporter) DOCTEST_REGISTER_REPORTER(name, priority,
reporter)
03763 #define REGISTER_LISTENER(name, priority, reporter) DOCTEST_REGISTER_LISTENER(name, priority,
reporter)
03764 #define INFO(...) DOCTEST_INFO(__VA_ARGS__)
03765 #define CAPTURE(x) DOCTEST_CAPTURE(x)
03766 #define ADD_MESSAGE_AT(file, line, ...) DOCTEST_ADD_MESSAGE_AT(file, line, __VA_ARGS__)
03767 #define ADD_FAIL_CHECK_AT(file, line, ...) DOCTEST_ADD_FAIL_CHECK_AT(file, line, __VA_ARGS__)
03768 #define ADD_FAIL_AT(file, line, ...) DOCTEST_ADD_FAIL_AT(file, line, __VA_ARGS__)
03769 #define MESSAGE(...) DOCTEST_MESSAGE(__VA_ARGS__)
03770 #define FAIL_CHECK(...) DOCTEST_FAIL_CHECK(__VA_ARGS__)
03771 #define FAIL(...) DOCTEST_FAIL(__VA_ARGS__)
03772 #define TO_LVALUE(...) DOCTEST_TO_LVALUE(__VA_ARGS__)
03773
03774 #define WARN(...) DOCTEST_WARN(__VA_ARGS__)
03775 #define WARN_FALSE(...) DOCTEST_WARN_FALSE(__VA_ARGS__)
03776 #define WARN_THROWS(...) DOCTEST_WARN_THROWS(__VA_ARGS__)
03777 #define WARN_THROWS_AS(expr, ...) DOCTEST_WARN_THROWS_AS(expr, __VA_ARGS__)
03778 #define WARN_THROWS_WITH(expr, ...) DOCTEST_WARN_THROWS_WITH(expr, __VA_ARGS__)
03779 #define WARN_THROWS_WITH_AS(expr, with, ...) DOCTEST_WARN_THROWS_WITH_AS(expr, with, __VA_ARGS__)
03780 #define WARN_NOTHROW(...) DOCTEST_WARN_NOTHROW(__VA_ARGS__)
03781 #define CHECK(...) DOCTEST_CHECK(__VA_ARGS__)
03782 #define CHECK_FALSE(...) DOCTEST_CHECK_FALSE(__VA_ARGS__)
03783 #define CHECK_THROWS(...) DOCTEST_CHECK_THROWS(__VA_ARGS__)
03784 #define CHECK_THROWS_AS(expr, ...) DOCTEST_CHECK_THROWS_AS(expr, __VA_ARGS__)
03785 #define CHECK_THROWS_WITH(expr, ...) DOCTEST_CHECK_THROWS_WITH(expr, __VA_ARGS__)
03786 #define CHECK_THROWS_WITH_AS(expr, with, ...) DOCTEST_CHECK_THROWS_WITH_AS(expr, with, __VA_ARGS__)
03787 #define CHECK_NOTHROW(...) DOCTEST_CHECK_NOTHROW(__VA_ARGS__)
03788 #define REQUIRE(...) DOCTEST_REQUIRE(__VA_ARGS__)
03789 #define REQUIRE_FALSE(...) DOCTEST_REQUIRE_FALSE(__VA_ARGS__)
03790 #define REQUIRE_THROWS(...) DOCTEST_REQUIRE_THROWS(__VA_ARGS__)
03791 #define REQUIRE_THROWS_AS(expr, ...) DOCTEST_REQUIRE_THROWS_AS(expr, __VA_ARGS__)
03792 #define REQUIRE_THROWS_WITH(expr, ...) DOCTEST_REQUIRE_THROWS_WITH(expr, __VA_ARGS__)
03793 #define REQUIRE_THROWS_WITH_AS(expr, with, ...) DOCTEST_REQUIRE_THROWS_WITH_AS(expr, with,
__VA_ARGS__)
03794 #define REQUIRE_NOTHROW(...) DOCTEST_REQUIRE_NOTHROW(__VA_ARGS__)
03795
03796 // clang-format off
03797 #define WARN_MESSAGE(cond, ...) DOCTEST_WARN_MESSAGE(cond, __VA_ARGS__)
03798 #define WARN_FALSE_MESSAGE(cond, ...) DOCTEST_WARN_FALSE_MESSAGE(cond, __VA_ARGS__)
03799 #define WARN_THROWS_MESSAGE(expr, ...) DOCTEST_WARN_THROWS_MESSAGE(expr, __VA_ARGS__)
03800 #define WARN_THROWS_AS_MESSAGE(expr, ex, ...) DOCTEST_WARN_THROWS_AS_MESSAGE(expr, ex, __VA_ARGS__)
03801 #define WARN_THROWS_WITH_MESSAGE(expr, with, ...) DOCTEST_WARN_THROWS_WITH_MESSAGE(expr, with,
__VA_ARGS__)
03802 #define WARN_THROWS_WITH_AS_MESSAGE(expr, with, ex, ...) DOCTEST_WARN_THROWS_WITH_AS_MESSAGE(expr,
with, ex, __VA_ARGS__)
03803 #define WARN_NOTHROW_MESSAGE(expr, ...) DOCTEST_WARN_NOTHROW_MESSAGE(expr, __VA_ARGS__)

```

```
03804 #define CHECK_MESSAGE(cond, ...) DOCTEST_CHECK_MESSAGE(cond, __VA_ARGS__)
03805 #define CHECK_FALSE_MESSAGE(cond, ...) DOCTEST_CHECK_FALSE_MESSAGE(cond, __VA_ARGS__)
03806 #define CHECK_THROWS_MESSAGE(expr, ...) DOCTEST_CHECK_THROWS_MESSAGE(expr, __VA_ARGS__)
03807 #define CHECK_THROWS_AS_MESSAGE(expr, ex, ...) DOCTEST_CHECK_THROWS_AS_MESSAGE(expr, ex, __VA_ARGS__)
03808 #define CHECK_THROWS_WITH_MESSAGE(expr, with, ...) DOCTEST_CHECK_THROWS_WITH_MESSAGE(expr, with,
__VA_ARGS__)
03809 #define CHECK_THROWS_WITH_AS_MESSAGE(expr, with, ex, ...) DOCTEST_CHECK_THROWS_WITH_AS_MESSAGE(expr,
with, ex, __VA_ARGS__)
03810 #define CHECK_NOTHROW_MESSAGE(expr, ...) DOCTEST_CHECK_NOTHROW_MESSAGE(expr, __VA_ARGS__)
03811 #define REQUIRE_MESSAGE(cond, ...) DOCTEST_REQUIRE_MESSAGE(cond, __VA_ARGS__)
03812 #define REQUIRE_FALSE_MESSAGE(cond, ...) DOCTEST_REQUIRE_FALSE_MESSAGE(cond, __VA_ARGS__)
03813 #define REQUIRE_THROWS_MESSAGE(expr, ...) DOCTEST_REQUIRE_THROWS_MESSAGE(expr, __VA_ARGS__)
03814 #define REQUIRE_THROWS_AS_MESSAGE(expr, ex, ...) DOCTEST_REQUIRE_THROWS_AS_MESSAGE(expr, ex,
__VA_ARGS__)
03815 #define REQUIRE_THROWS_WITH_MESSAGE(expr, with, ...) DOCTEST_REQUIRE_THROWS_WITH_MESSAGE(expr, with,
__VA_ARGS__)
03816 #define REQUIRE_THROWS_WITH_AS_MESSAGE(expr, with, ex, ...)
DOCTEST_REQUIRE_THROWS_WITH_AS_MESSAGE(expr, with, ex, __VA_ARGS__)
03817 #define REQUIRE_NOTHROW_MESSAGE(expr, ...) DOCTEST_REQUIRE_NOTHROW_MESSAGE(expr, __VA_ARGS__)
03818 // clang-format on
03819
03820 #define SCENARIO(name) DOCTEST_SCENARIO(name)
03821 #define SCENARIO_METHOD(x, name) DOCTEST_SCENARIO_METHOD(x, name)
03822 #define SCENARIO_CLASS(name) DOCTEST_SCENARIO_CLASS(name)
03823 #define SCENARIO_TEMPLATE(name, T, ...) DOCTEST_SCENARIO_TEMPLATE(name, T, __VA_ARGS__)
03824 #define SCENARIO_TEMPLATE_DEFINE(name, T, id) DOCTEST_SCENARIO_TEMPLATE_DEFINE(name, T, id)
03825 #define GIVEN(name) DOCTEST_GIVEN(name)
03826 #define AND_GIVEN(name) DOCTEST_AND_GIVEN(name)
03827 #define WHEN(name) DOCTEST_WHEN(name)
03828 #define AND_WHEN(name) DOCTEST_AND_WHEN(name)
03829 #define THEN(name) DOCTEST_THEN(name)
03830 #define AND_THEN(name) DOCTEST_AND_THEN(name)
03831
03832 #define WARN_EQ(...) DOCTEST_WARN_EQ(__VA_ARGS__)
03833 #define CHECK_EQ(...) DOCTEST_CHECK_EQ(__VA_ARGS__)
03834 #define REQUIRE_EQ(...) DOCTEST_REQUIRE_EQ(__VA_ARGS__)
03835 #define WARN_NE(...) DOCTEST_WARN_NE(__VA_ARGS__)
03836 #define CHECK_NE(...) DOCTEST_CHECK_NE(__VA_ARGS__)
03837 #define REQUIRE_NE(...) DOCTEST_REQUIRE_NE(__VA_ARGS__)
03838 #define WARN_GT(...) DOCTEST_WARN_GT(__VA_ARGS__)
03839 #define CHECK_GT(...) DOCTEST_CHECK_GT(__VA_ARGS__)
03840 #define REQUIRE_GT(...) DOCTEST_REQUIRE_GT(__VA_ARGS__)
03841 #define WARN_LT(...) DOCTEST_WARN_LT(__VA_ARGS__)
03842 #define CHECK_LT(...) DOCTEST_CHECK_LT(__VA_ARGS__)
03843 #define REQUIRE_LT(...) DOCTEST_REQUIRE_LT(__VA_ARGS__)
03844 #define WARN_GE(...) DOCTEST_WARN_GE(__VA_ARGS__)
03845 #define CHECK_GE(...) DOCTEST_CHECK_GE(__VA_ARGS__)
03846 #define REQUIRE_GE(...) DOCTEST_REQUIRE_GE(__VA_ARGS__)
03847 #define WARN_LE(...) DOCTEST_WARN_LE(__VA_ARGS__)
03848 #define CHECK_LE(...) DOCTEST_CHECK_LE(__VA_ARGS__)
03849 #define REQUIRE_LE(...) DOCTEST_REQUIRE_LE(__VA_ARGS__)
03850 #define WARN_UNARY(...) DOCTEST_WARN_UNARY(__VA_ARGS__)
03851 #define CHECK_UNARY(...) DOCTEST_CHECK_UNARY(__VA_ARGS__)
03852 #define REQUIRE_UNARY(...) DOCTEST_REQUIRE_UNARY(__VA_ARGS__)
03853 #define WARN_UNARY_FALSE(...) DOCTEST_WARN_UNARY_FALSE(__VA_ARGS__)
03854 #define CHECK_UNARY_FALSE(...) DOCTEST_CHECK_UNARY_FALSE(__VA_ARGS__)
03855 #define REQUIRE_UNARY_FALSE(...) DOCTEST_REQUIRE_UNARY_FALSE(__VA_ARGS__)
03856
03857 // KEPT FOR BACKWARDS COMPATIBILITY
03858 #define FAST_WARN_EQ(...) DOCTEST_FAST_WARN_EQ(__VA_ARGS__)
03859 #define FAST_CHECK_EQ(...) DOCTEST_FAST_CHECK_EQ(__VA_ARGS__)
03860 #define FAST_REQUIRE_EQ(...) DOCTEST_FAST_REQUIRE_EQ(__VA_ARGS__)
03861 #define FAST_WARN_NE(...) DOCTEST_FAST_WARN_NE(__VA_ARGS__)
03862 #define FAST_CHECK_NE(...) DOCTEST_FAST_CHECK_NE(__VA_ARGS__)
03863 #define FAST_REQUIRE_NE(...) DOCTEST_FAST_REQUIRE_NE(__VA_ARGS__)
03864 #define FAST_WARN_GT(...) DOCTEST_FAST_WARN_GT(__VA_ARGS__)
03865 #define FAST_CHECK_GT(...) DOCTEST_FAST_CHECK_GT(__VA_ARGS__)
03866 #define FAST_REQUIRE_GT(...) DOCTEST_FAST_REQUIRE_GT(__VA_ARGS__)
03867 #define FAST_WARN_LT(...) DOCTEST_FAST_WARN_LT(__VA_ARGS__)
03868 #define FAST_CHECK_LT(...) DOCTEST_FAST_CHECK_LT(__VA_ARGS__)
03869 #define FAST_REQUIRE_LT(...) DOCTEST_FAST_REQUIRE_LT(__VA_ARGS__)
03870 #define FAST_WARN_GE(...) DOCTEST_FAST_WARN_GE(__VA_ARGS__)
03871 #define FAST_CHECK_GE(...) DOCTEST_FAST_CHECK_GE(__VA_ARGS__)
03872 #define FAST_REQUIRE_GE(...) DOCTEST_FAST_REQUIRE_GE(__VA_ARGS__)
03873 #define FAST_WARN_LE(...) DOCTEST_FAST_WARN_LE(__VA_ARGS__)
03874 #define FAST_CHECK_LE(...) DOCTEST_FAST_CHECK_LE(__VA_ARGS__)
03875 #define FAST_REQUIRE_LE(...) DOCTEST_FAST_REQUIRE_LE(__VA_ARGS__)
03876
03877 #define FAST_WARN_UNARY(...) DOCTEST_FAST_WARN_UNARY(__VA_ARGS__)
03878 #define FAST_CHECK_UNARY(...) DOCTEST_FAST_CHECK_UNARY(__VA_ARGS__)
03879 #define FAST_REQUIRE_UNARY(...) DOCTEST_FAST_REQUIRE_UNARY(__VA_ARGS__)
03880 #define FAST_WARN_UNARY_FALSE(...) DOCTEST_FAST_WARN_UNARY_FALSE(__VA_ARGS__)
03881 #define FAST_CHECK_UNARY_FALSE(...) DOCTEST_FAST_CHECK_UNARY_FALSE(__VA_ARGS__)
03882 #define FAST_REQUIRE_UNARY_FALSE(...) DOCTEST_FAST_REQUIRE_UNARY_FALSE(__VA_ARGS__)
03883
03884 #define TEST_CASE_TEMPLATE_INSTANTIATE(id, ...) DOCTEST_TEST_CASE_TEMPLATE_INSTANTIATE(id,
__VA_ARGS__)
```

```

03885
03886 #endif // DOCTEST_CONFIG_NO_SHORT_MACRO_NAMES
03887
03888 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_POP
03889
03890 #endif // DOCTEST_PARTS_PUBLIC_MACROS
03891
03892 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_POP
03893
03894 #endif // DOCTEST_LIBRARY_INCLUDED
03895
03896 #if defined(DOCTEST_CONFIG_IMPLEMENT) && !defined(DOCTEST_LIBRARY_IMPLEMENTATION)
03897
03898 DOCTEST_CLANG_SUPPRESS_WARNING_WITH_PUSH("-Wunused-macros")
03899 #define DOCTEST_LIBRARY_IMPLEMENTATION
03900 DOCTEST_CLANG_SUPPRESS_WARNING_POP
03901
03902 #ifndef DOCTEST_PARTS_PRIVATE_PRELUDE
03903 #define DOCTEST_PARTS_PRIVATE_PRELUDE
03904
03905
03906 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
03907
03908 DOCTEST_MAKE_STD_HEADERS_CLEAN_FROM_WARNINGS_ON_WALL_BEGIN
03909
03910 // required includes - will go only in one translation unit!
03911 #include <ctime>
03912 #include <cmath>
03913 #include <climits>
03914 // borland (Embarcadero) compiler requires math.h and not cmath -
03915 // https://github.com/doctest/doctest/pull/37
03916 #ifdef __BORLANDC__
03917 #include <math.h>
03918 #endif // __BORLANDC__
03919 #include <new>
03920 #include <cstdio>
03921 #include <cstdlib>
03922 #include <cstring>
03923 #include <limits>
03924 #include <utility>
03925 #include <fstream>
03926 #include <sstream>
03927 #ifndef DOCTEST_CONFIG_NO_INCLUDE_Iostream
03928 #include <iostream>
03929 #endif // DOCTEST_CONFIG_NO_INCLUDE_Iostream
03930 #include <algorithm>
03931 #include <iomanip>
03932 #include <vector>
03933 #ifndef DOCTEST_CONFIG_NO_MULTITHREADING
03934 #include <atomic>
03935 #include <mutex>
03936 #define DOCTEST_DECLARE_MUTEX(name) std::mutex name;
03937 #define DOCTEST_DECLARE_STATIC_MUTEX(name) static DOCTEST_DECLARE_MUTEX(name)
03938 #define DOCTEST_LOCK_MUTEX(name) const std::lock_guard<std::mutex>
03939 DOCTEST_ANONYMOUS(DOCTEST_ANON_LOCK_)(name);
03940 #else // DOCTEST_CONFIG_NO_MULTITHREADING
03941 #define DOCTEST_DECLARE_MUTEX(name)
03942 #define DOCTEST_DECLARE_STATIC_MUTEX(name)
03943 #define DOCTEST_LOCK_MUTEX(name)
03944 #endif // DOCTEST_CONFIG_NO_MULTITHREADING
03945 #include <set>
03946 #include <map>
03947 #include <unordered_set>
03948 #include <exception>
03949 #include <stdexcept>
03950 #if defined(DOCTEST_CONFIG_POSIX_SIGNALS) || defined(DOCTEST_CONFIG_WINDOWS_SEH)
03951 #include <csignal>
03952 #endif // DOCTEST_CONFIG_POSIX_SIGNALS
03953 #include <cfloating>
03954 #include <cctype>
03955 #include <cstdint>
03956 #include <string>
03957
03958 #ifdef DOCTEST_PLATFORM_MAC
03959 #include <sys/types.h>
03960 #include <unistd.h>
03961 #include <sys/sysctl.h>
03962 #endif // DOCTEST_PLATFORM_MAC
03963
03964 #ifdef DOCTEST_PLATFORM_WINDOWS
03965 // defines for a leaner windows.h
03966 #ifndef WIN32_LEAN_AND_MEAN
03967 #define WIN32_LEAN_AND_MEAN
03968 #define DOCTEST_UNDEF_WIN32_LEAN_AND_MEAN
03969 #endif // WIN32_LEAN_AND_MEAN
03970 #ifndef NOMINMAX

```

```

03971 #define NOMINMAX
03972 #define DOCTEST_UNDEF_NOMINMAX
03973 #endif // NOMINMAX
03974
03975 // not sure what AfxWin.h is for - here I do what Catch does
03976 #ifndef __AFXDLL
03977 #include <AfxWin.h>
03978 #else
03979 #include <windows.h>
03980 #endif
03981 #include <io.h>
03982
03983 #ifdef DOCTEST_UNDEF_WIN32_LEAN_AND_MEAN
03984 #undef WIN32_LEAN_AND_MEAN
03985 #undef DOCTEST_UNDEF_WIN32_LEAN_AND_MEAN
03986 #endif // DOCTEST_UNDEF_WIN32_LEAN_AND_MEAN
03987 #ifdef DOCTEST_UNDEF_NOMINMAX
03988 #undef NOMINMAX
03989 #undef DOCTEST_UNDEF_NOMINMAX
03990 #endif // DOCTEST_UNDEF_NOMINMAX
03991
03992 #else // DOCTEST_PLATFORM_WINDOWS
03993
03994 #include <sys/time.h>
03995 #include <unistd.h>
03996
03997 #endif // DOCTEST_PLATFORM_WINDOWS
03998
03999 // this is a fix for https://github.com/doctest/doctest/issues/348
04000 // https://mail.gnome.org/archives/xml/2012-January/msg00000.html
04001 #if !defined(HAVE_UNISTD_H) && !defined(STDOUT_FILENO)
04002 #define STDOUT_FILENO fileno(stdout)
04003 #endif // HAVE_UNISTD_H
04004
04005 DOCTEST_MAKE_STD_HEADERS_CLEAN_FROM_WARNINGS_ON_WALL_END
04006
04007 // counts the number of elements in a C array
04008 #define DOCTEST_COUNTOF(x) (sizeof(x) / sizeof(x[0]))
04009
04010 #ifdef DOCTEST_CONFIG_DISABLE
04011 #define DOCTEST_BRANCH_ON_DISABLED(if_disabled, if_not_disabled) if_disabled
04012 #else // DOCTEST_CONFIG_DISABLE
04013 #define DOCTEST_BRANCH_ON_DISABLED(if_disabled, if_not_disabled) if_not_disabled
04014 #endif // DOCTEST_CONFIG_DISABLE
04015
04016 #ifndef DOCTEST_THREAD_LOCAL
04017 #if defined(DOCTEST_CONFIG_NO_MULTITHREADING) || DOCTEST_MSVC && (DOCTEST_MSVC < DOCTEST_COMPILER(19,
0, 0))
04018 #define DOCTEST_THREAD_LOCAL
04019 #else // DOCTEST_MSVC
04020 #define DOCTEST_THREAD_LOCAL thread_local
04021 #endif // DOCTEST_MSVC
04022 #endif // DOCTEST_THREAD_LOCAL
04023
04024 #ifndef DOCTEST_MULTI_LANE_ATOMICS_THREAD_LANES
04025 #define DOCTEST_MULTI_LANE_ATOMICS_THREAD_LANES 32
04026 #endif
04027
04028 #ifndef DOCTEST_MULTI_LANE_ATOMICS_CACHE_LINE_SIZE
04029 #define DOCTEST_MULTI_LANE_ATOMICS_CACHE_LINE_SIZE 64
04030 #endif
04031
04032 #if defined(WINAPI_FAMILY) && (WINAPI_FAMILY == WINAPI_FAMILY_APP)
04033 #define DOCTEST_CONFIG_NO_MULTI_LANE_ATOMICS
04034 #endif
04035
04036 #ifndef DOCTEST_CDECL
04037 #define DOCTEST_CDECL __cdecl
04038 #endif
04039
04040 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
04041
04042 #endif // DOCTEST_PARTS_PRIVATE_PRELUDE
04043 #ifndef DOCTEST_PARTS_PRIVATE_CONTEXT_STATE
04044 #define DOCTEST_PARTS_PRIVATE_CONTEXT_STATE
04045
04046 #ifndef DOCTEST_PARTS_PRIVATE_TIMER
04047 #define DOCTEST_PARTS_PRIVATE_TIMER
04048
04049
04050 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
04051
04052 #ifndef DOCTEST_CONFIG_DISABLE
04053
04054 namespace doctest {
04055 namespace detail {
04056

```



```

04144     }
04145
04146     T fetch_sub(T arg, std::memory_order order = std::memory_order_seq_cst) DOCTEST_NOEXCEPT {
04147         return myAtomic().fetch_sub(arg, order);
04148     }
04149
04150     operator T() const DOCTEST_NOEXCEPT {
04151         return load();
04152     }
04153
04154     T load(std::memory_order order = std::memory_order_seq_cst) const DOCTEST_NOEXCEPT {
04155         auto result = T();
04156         for (const auto &c: m_atomics) {
04157             result += c.atomic.load(order);
04158         }
04159         return result;
04160     }
04161
04162     // NOLINTNEXTLINE(cppcoreguidelines-c-copy-assignment-signature,
04163     // misc-unconventional-assign-operator)
04164     T operator=(T desired) DOCTEST_NOEXCEPT {
04165         store(desired);
04166         return desired;
04167     }
04168
04169     void store(T desired, std::memory_order order = std::memory_order_seq_cst) DOCTEST_NOEXCEPT {
04170         // first value becomes desired", all others become 0.
04171         for (auto &c: m_atomics) {
04172             c.atomic.store(desired, order);
04173             desired = {};
04174         }
04175     }
04176 private:
04177     // Each thread has a different atomic that it operates on. If more than NumLanes threads
04178     // use this, some will use the same atomic. So performance will degrade a bit, but still
04179     // everything will work.
04180     //
04181     // The logic here is a bit tricky. The call should be as fast as possible, so that there
04182     // is minimal to no overhead in determining the correct atomic for the current thread.
04183     //
04184     // 1. A global static counter laneCounter counts continuously up.
04185     // 2. Each successive thread will use modulo operation of that counter so it gets an atomic
04186     //    assigned in a round-robin fashion.
04187     // 3. This tlsLaneIdx is stored in the thread local data, so it is directly available with
04188     //    little overhead.
04189     Atomic<T> &myAtomic() DOCTEST_NOEXCEPT {
04190         static Atomic<size_t> laneCounter;
04191         DOCTEST_THREAD_LOCAL size_t tlsLaneIdx = laneCounter++ %
04192         DOCTEST_MULTI_LANE_ATOMICS_THREAD_LANES;
04193         return m_atomics[tlsLaneIdx].atomic;
04194     }
04195 };
04196 #endif // DOCTEST_CONFIG_NO_MULTI_LANE_ATOMICS
04197
04198 } // namespace detail
04199 } // namespace doctest
04200
04201 #endif // DOCTEST_CONFIG_DISABLE
04202
04203 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
04204
04205 #endif // DOCTEST_PARTS_PRIVATE_ATOMIC
04206 #ifndef DOCTEST_PARTS_PRIVATE_TRAVERSAL
04207 #define DOCTEST_PARTS_PRIVATE_TRAVERSAL
04208
04209 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
04210
04211 #ifndef DOCTEST_CONFIG_DISABLE
04212
04213 namespace doctest {
04214     namespace detail {
04215
04216         struct DecisionPoint {
04217             // Number of branches available at this depth for the current traversal path.
04218             size_t branch_count = 0;
04219             // Encountered sibling subcases in source order for subcase decision points.
04220             std::vector<SubcaseSignature> subcases;
04221         };
04222     };
04223
04224     class TraversalState {
04225     public:
04226         size_t activeSubcaseDepth() const {
04227             return m_activeSubcaseDepth;
04228         }
04229     };

```



```

04229
04230     void resetForTestCase();
04231     void resetForRun();
04232     bool advance();
04233     bool tryEnterSubcase(const SubcaseSignature &signature);
04234     void leaveSubcase();
04235     size_t unwindActiveSubcases();
04236     size_t acquireGeneratorIndex(size_t count);
04237
04238 private:
04239     // decisionPath is the selected traversal prefix; discoveredDecisionPath is rebuilt
04240     // on each rerun to describe the branches encountered at each depth.
04241     std::vector<DecisionPoint> m_discoveredDecisionPath;
04242     std::vector<size_t> m_decisionPath;
04243     size_t m_decisionDepth = 0;
04244     std::vector<size_t> m_enteredSubcaseDepths;
04245     size_t m_activeSubcaseDepth = 0;
04246
04247     DecisionPoint &ensureDecisionPointAtCurrentDepth();
04248 };
04249
04250 } // namespace detail
04251 } // namespace doctest
04252
04253 #endif // DOCTEST_CONFIG_DISABLE
04254
04255 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
04256
04257 #endif // DOCTEST_PARTS_PRIVATE_TRAVERSAL
04258
04259 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
04260
04261 #ifndef DOCTEST_CONFIG_DISABLE
04262
04263 namespace doctest {
04264 namespace detail {
04265
04266     // this holds both parameters from the command line and runtime data for tests
04267     struct ContextState : ContextOptions, TestRunStats, CurrentTestCaseStats {
04268         MultiLaneAtomic<int> numAssertsCurrentTest_atomic;
04269         MultiLaneAtomic<int> numAssertsFailedCurrentTest_atomic;
04270
04271         std::vector<std::vector<String>> filters = decltype(filters)(9); // 9 different filters
04272
04273         std::vector<IReporter *> reporters_currently_used;
04274
04275         assert_handler ah = nullptr;
04276
04277         Timer timer;
04278
04279         std::vector<String> stringifiedContexts; // logging from INFO() due to an exception
04280
04281         // Backtrack traversal state shared by SUBCASE and GENERATE.
04282         TraversalState traversal;
04283         Atomic<bool> shouldLogCurrentException;
04284
04285         void resetRunData();
04286
04287         void finalizeTestCaseData();
04288     };
04289
04290     extern ContextState *g_cs;
04291
04292     // used to avoid locks for the debug output
04293     // TODO: figure out if this is indeed necessary/correct - seems like either there still
04294     // could be a race or that there wouldn't be a race even if using the context directly
04295     extern DOCTEST_THREAD_LOCAL bool g_no_colors;
04296
04297 } // namespace detail
04298 } // namespace doctest
04299
04300 #endif // DOCTEST_CONFIG_DISABLE
04301
04302 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
04303
04304 #endif // DOCTEST_PARTS_PRIVATE_CONTEXT_STATE
04305
04306 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
04307
04308 #ifndef DOCTEST_CONFIG_DISABLE
04309
04310 namespace doctest {
04311 using detail::g_cs;
04312
04313     AssertData::AssertData(
04314         assertType::Enum at,

```



```

04316     const char *file,
04317     int line,
04318     const char *expr,
04319     const char *exception_type,
04320     const StringContains &exception_string
04321 )
04322 : m_test_case(g_cs->currentTest),
04323   m_at(at),
04324   m_file(file),
04325   m_line(line),
04326   m_expr(expr),
04327   m_failed(true),
04328   m_threw(false),
04329   m_threw_as(false),
04330   m_exception_type(exception_type),
04331   m_exception_string(exception_string) {
04332 #if DOCTEST_MSVC
04333     if (m_expr[0] == ' ') // this happens when variadic macros are disabled under MSVC
04334         ++m_expr;
04335 #endif // MSVC
04336 }
04337
04338 } // namespace doctest
04339
04340 #endif // DOCTEST_CONFIG_DISABLE
04341
04342 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
04343
04344 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
04345
04346 #ifndef DOCTEST_CONFIG_DISABLE
04347 namespace doctest {
04348 namespace detail {
04349
04350     ExpressionComposer::ExpressionComposer(assertType::Enum at)
04351     : m_at(at) {}
04352
04353 } // namespace detail
04354 } // namespace doctest
04355
04356 #endif // DOCTEST_CONFIG_DISABLE
04357
04358 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
04359 #ifndef DOCTEST_PARTS_PRIVATE_REPORTER
04360 #define DOCTEST_PARTS_PRIVATE_REPORTER
04361
04362
04363 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
04364
04365 #ifndef DOCTEST_CONFIG_DISABLE
04366 namespace doctest {
04367 namespace detail {
04368 // the int (priority) is part of the key for automatic sorting - sadly one can register a
04369 // reporter with a duplicate name and a different priority but hopefully that won't happen often :|
04370 using reporterMap = std::map<std::pair<int, String>, detail::reporterCreatorFunc>;
04371
04372 reporterMap &getReporters() noexcept;
04373 reporterMap &getListeners() noexcept;
04374 } // namespace detail
04375
04376 #define DOCTEST_ITERATE_THROUGH_REPORTERS(function, ...)
04377 \
04378     for (auto &curr_rep: g_cs->reporters_currently_used)
04379 \
04380     curr_rep->function(__VA_ARGS__)
04381
04382 } // namespace doctest
04383
04384 #endif // DOCTEST_CONFIG_DISABLE
04385
04386 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
04387
04388 #endif // DOCTEST_PARTS_PRIVATE_REPORTER
04389 #ifndef DOCTEST_PARTS_PRIVATE_ASSERT_HANDLER
04390 #define DOCTEST_PARTS_PRIVATE_ASSERT_HANDLER
04391
04392 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
04393
04394 #ifndef DOCTEST_CONFIG_DISABLE
04395 namespace doctest {
04396 namespace detail {
04397
04398 void addAssert(assertType::Enum at);

```

```

04401
04402 void addFailedAssert(assertType::Enum at);
04403
04404 #if defined(DOCTEST_CONFIG_POSIX_SIGNALS) || defined(DOCTEST_CONFIG_WINDOWS_SEH)
04405 void reportFatal(const std::string &message);
04406 #endif // DOCTEST_CONFIG_POSIX_SIGNALS || DOCTEST_CONFIG_WINDOWS_SEH
04407
04408 } // namespace detail
04409 } // namespace doctest
04410
04411 #endif // DOCTEST_CONFIG_DISABLE
04412
04413 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
04414
04415 #endif // DOCTEST_PARTS_PRIVATE_ASSERT_HANDLER
04416
04417 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
04418
04419 #ifndef DOCTEST_CONFIG_DISABLE
04420
04421 namespace doctest {
04422 namespace detail {
04423
04424 void addAssert(assertType::Enum at) {
04425     if ((at & assertType::is_warn) == 0)
04426         g_cs->numAssertsCurrentTest_atomic++;
04427 }
04428
04429 void addFailedAssert(assertType::Enum at) {
04430     if ((at & assertType::is_warn) == 0)
04431         g_cs->numAssertsFailedCurrentTest_atomic++;
04432 }
04433
04434 #if defined(DOCTEST_CONFIG_POSIX_SIGNALS) || defined(DOCTEST_CONFIG_WINDOWS_SEH)
04435 void reportFatal(const std::string &message) {
04436     g_cs->failure_flags |= TestCaseFailureReason::Crash;
04437
04438     DOCTEST_ITERATE_THROUGH_REPORTERS(test_case_exception, {message.c_str(), true});
04439
04440     for (size_t i = g_cs->traversal.unwindActiveSubcases(); i > 0; --i)
04441         DOCTEST_ITERATE_THROUGH_REPORTERS(subcase_end, DOCTEST_EMPTY);
04442     g_cs->finalizeTestCaseData();
04443
04444     DOCTEST_ITERATE_THROUGH_REPORTERS(test_case_end, *g_cs);
04445
04446     DOCTEST_ITERATE_THROUGH_REPORTERS(test_run_end, *g_cs);
04447 }
04448 #endif // DOCTEST_CONFIG_POSIX_SIGNALS || DOCTEST_CONFIG_WINDOWS_SEH
04449
04450 void failed_out_of_a_testing_context(const AssertData &ad) {
04451     if (g_cs->ah)
04452         g_cs->ah(ad);
04453     else
04454         std::abort();
04455 }
04456
04457 bool decomp_assert(assertType::Enum at, const char *file, int line, const char *expr, const Result
&result) {
04458     const bool failed = !result.m_passed;
04459
04460     // #####
04461     // IF THE DEBUGGER BREAKS HERE - GO 1 LEVEL UP IN THE CALLSTACK FOR THE FAILING ASSERT
04462     // THIS IS THE EFFECT OF HAVING 'DOCTEST_CONFIG_SUPER_FAST_ASSERTS' DEFINED
04463     // #####
04464     DOCTEST_ASSERT_OUT_OF_TESTS(result.m_decomp);
04465     DOCTEST_ASSERT_IN_TESTS(result.m_decomp);
04466     return !failed;
04467 }
04468
04469 } // namespace detail
04470 } // namespace doctest
04471
04472 #endif // DOCTEST_CONFIG_DISABLE
04473
04474 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
04475
04476 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
04477
04478 #ifndef DOCTEST_CONFIG_DISABLE
04479
04480 namespace doctest {
04481 namespace detail {
04482
04483 MessageBuilder::MessageBuilder(const char *file, int line, assertType::Enum severity) {
04484     m_stream = tlssPush();
04485     m_file = file;
04486     m_line = line;

```

```

04487     m_severity = severity;
04488 }
04489
04490 MessageBuilder::~MessageBuilder() noexcept(false) {
04491     if (!logged)
04492         tlssPop();
04493 }
04494
04495 bool MessageBuilder::log() {
04496     if (!logged) {
04497         m_string = tlssPop();
04498         logged = true;
04499     }
04500
04501     DOCTEST_ITERATE_THROUGH_REPORTERS(log_message, *this);
04502
04503     const bool isWarn = m_severity & assertType::is_warn;
04504
04505     // warn is just a message in this context so we don't treat it as an assert
04506     if (!isWarn) {
04507         addAssert(m_severity);
04508         addFailedAssert(m_severity);
04509     }
04510
04511     return isDebuggerActive() && !getContextOptions()->no_breaks && !isWarn &&
04512         (g_cs->currentTest == nullptr || !g_cs->currentTest->m_no_breaks); // break into debugger
04513 }
04514
04515 void MessageBuilder::react() {
04516     if (m_severity & assertType::is_require)
04517         throwException();
04518 }
04519
04520 } // namespace detail
04521 } // namespace doctest
04522
04523 #endif // DOCTEST_CONFIG_DISABLE
04524
04525 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
04526 #ifndef DOCTEST_PARTS_PRIVATE_EXCEPTION_TRANSLATOR
04527 #define DOCTEST_PARTS_PRIVATE_EXCEPTION_TRANSLATOR
04528
04529 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
04530
04531 #ifndef DOCTEST_CONFIG_DISABLE
04532
04533 namespace doctest {
04534 namespace detail {
04535
04536 std::vector<const IExceptionTranslator*> &getExceptionTranslators() noexcept;
04537 String translateActiveException() noexcept;
04538
04539 } // namespace detail
04540 } // namespace doctest
04541
04542 #endif // DOCTEST_CONFIG_DISABLE
04543
04544 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
04545
04546 #ifndef DOCTEST_PARTS_PRIVATE_EXCEPTION_TRANSLATOR
04547 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
04548
04549 #ifndef DOCTEST_CONFIG_DISABLE
04550
04551 namespace doctest {
04552 namespace detail {
04553
04554 Result::Result(bool passed, const String &decomposition)
04555     : m_passed(passed), m_decomp(decomposition) {}
04556
04557 ResultBuilder::ResultBuilder(
04558     assertType::Enum at,
04559     const char *file,
04560     int line,
04561     const char *expr,
04562     const char *exception_type,
04563     const String &exception_string
04564 )
04565     : AssertData(at, file, line, expr, exception_type, exception_string) {}
04566 } // NOLINT(clang-analyzer-cplusplus.NewDeleteLeaks)
04567
04568 ResultBuilder::ResultBuilder(
04569     assertType::Enum at,
04570     const char *file,
04571     int line,

```

```

04574     const char *expr,
04575     const char *exception_type,
04576     const Contains &exception_string
04577 )
04578     : AssertData(at, file, line, expr, exception_type, exception_string) {}
04579
04580 void ResultBuilder::setResult(const Result &res) {
04581     m_decomp = res.m_decomp;
04582     m_failed = !res.m_passed;
04583 }
04584
04585 void ResultBuilder::translateException() {
04586     m_threw = true;
04587     m_exception = translateActiveException();
04588 }
04589
04590 bool ResultBuilder::log() {
04591     if (m_at & assertType::is_throws) {
04592         m_failed = !m_threw;
04593     } else if ((m_at & assertType::is_throws_as) && (m_at & assertType::is_throws_with)) {
04594         m_failed = !m_threw_as || !m_exception_string.check(m_exception);
04595     } else if (m_at & assertType::is_throws_as) {
04596         m_failed = !m_threw_as;
04597     } else if (m_at & assertType::is_throws_with) {
04598         m_failed = !m_exception_string.check(m_exception);
04599     } else if (m_at & assertType::is_nothrow) {
04600         m_failed = m_threw;
04601     }
04602
04603     if (m_exception.size())
04604         m_exception = "\"" + m_exception + "\"";
04605
04606     if (is_running_in_test) {
04607         addAssert(m_at);
04608         DOCTEST_ITERATE_THROUGH_REPORTERS(log_assert, *this);
04609
04610         if (m_failed)
04611             addFailedAssert(m_at);
04612     } else if (m_failed) {
04613         failed_out_of_a_testing_context(*this);
04614     }
04615
04616     return m_failed && isDebuggerActive() && !getContextOptions()->no_breaks &&
04617         (g_cs->currentTest == nullptr || !g_cs->currentTest->m_no_breaks); // break into debugger
04618 }
04619
04620 void ResultBuilder::react() const {
04621     if (m_failed && checkIfShouldThrow(m_at))
04622         throwException();
04623 }
04624
04625 } // namespace detail
04626 } // namespace doctest
04627
04628 #endif // DOCTEST_CONFIG_DISABLE
04629
04630 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
04631 #ifndef DOCTEST_PARTS_PRIVATE_EXCEPTIONS
04632 #define DOCTEST_PARTS_PRIVATE_EXCEPTIONS
04633
04634 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
04635
04636 namespace doctest {
04637 namespace detail {
04638
04639 template <typename Ex>
04640 DOCTEST_NORETURN void throw_exception(const Ex &e) {
04641     #ifndef DOCTEST_CONFIG_NO_EXCEPTIONS
04642         throw e;
04643     #else // DOCTEST_CONFIG_NO_EXCEPTIONS
04644         #ifdef DOCTEST_CONFIG_HANDLE_EXCEPTION
04645             DOCTEST_CONFIG_HANDLE_EXCEPTION(e);
04646         #else // DOCTEST_CONFIG_HANDLE_EXCEPTION
04647             #ifndef DOCTEST_CONFIG_NO_INCLUDE_Iostream
04648                 std::cerr << "doctest will terminate because it needed to throw an exception.\n"
04649                     << "The message was: " << e.what() << '\n';
04650             #endif // DOCTEST_CONFIG_NO_INCLUDE_Iostream
04651             #ifndef DOCTEST_CONFIG_HANDLE_EXCEPTION
04652                 std::terminate();
04653             #endif // DOCTEST_CONFIG_NO_EXCEPTIONS
04654         #endif
04655     }
04656
04657 #ifndef DOCTEST_INTERNAL_ERROR
04658 #define DOCTEST_INTERNAL_ERROR(msg)
04659     \
04660     detail::throw_exception(std::logic_error(__FILE__ ":" DOCTEST_TOSTR(__LINE__) ": Internal doctest

```

```

        error: " msg))
04660 #endif // DOCTEST_INTERNAL_ERROR
04661 } // namespace detail
04662
04663 } // namespace doctest
04664
04665 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
04666
04667 #endif // DOCTEST_PARTS_PRIVATE_EXCEPTIONS
04668
04669 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
04670
04671 namespace doctest {
04672
04673     const char *assertString(assertType::Enum at) {
04674         DOCTEST_MSVC_SUPPRESS_WARNING_WITH_PUSH(4061) // enum 'x' in switch of enum 'y' is not explicitly
            handled
04675         #define DOCTEST_GENERATE_ASSERT_TYPE_CASE(assert_type)
            \
04676             case assertType::DT_##assert_type: return #assert_type
04677         #define DOCTEST_GENERATE_ASSERT_TYPE_CASES(assert_type)
            \
04678             DOCTEST_GENERATE_ASSERT_TYPE_CASE(WARN_##assert_type);
            \
04679             DOCTEST_GENERATE_ASSERT_TYPE_CASE(CHECK_##assert_type);
            \
04680             DOCTEST_GENERATE_ASSERT_TYPE_CASE(REQUIRE_##assert_type)
04681         DOCTEST_CLANG_SUPPRESS_WARNING_PUSH
04682         DOCTEST_CLANG_SUPPRESS_WARNING("-Wswitch-enum")
04683         DOCTEST_GCC_SUPPRESS_WARNING_PUSH
04684         DOCTEST_GCC_SUPPRESS_WARNING("-Wswitch-enum")
04685         switch (at) {
04686             DOCTEST_GENERATE_ASSERT_TYPE_CASE(WARN);
04687             DOCTEST_GENERATE_ASSERT_TYPE_CASE(CHECK);
04688             DOCTEST_GENERATE_ASSERT_TYPE_CASE(REQUIRE);
04689
04690             DOCTEST_GENERATE_ASSERT_TYPE_CASES(FALSE);
04691
04692             DOCTEST_GENERATE_ASSERT_TYPE_CASES(THROWS);
04693
04694             DOCTEST_GENERATE_ASSERT_TYPE_CASES(THROWS_AS);
04695
04696             DOCTEST_GENERATE_ASSERT_TYPE_CASES(THROWS_WITH);
04697
04698             DOCTEST_GENERATE_ASSERT_TYPE_CASES(THROWS_WITH_AS);
04699
04700             DOCTEST_GENERATE_ASSERT_TYPE_CASES(NOTHROW);
04701
04702             DOCTEST_GENERATE_ASSERT_TYPE_CASES(EQ);
04703             DOCTEST_GENERATE_ASSERT_TYPE_CASES(NE);
04704             DOCTEST_GENERATE_ASSERT_TYPE_CASES(GT);
04705             DOCTEST_GENERATE_ASSERT_TYPE_CASES(LT);
04706             DOCTEST_GENERATE_ASSERT_TYPE_CASES(GE);
04707             DOCTEST_GENERATE_ASSERT_TYPE_CASES(LE);
04708
04709             DOCTEST_GENERATE_ASSERT_TYPE_CASES(UNARY);
04710             DOCTEST_GENERATE_ASSERT_TYPE_CASES(UNARY_FALSE);
04711
04712             default: DOCTEST_INTERNAL_ERROR("Tried stringifying invalid assert type!");
04713         }
04714         DOCTEST_CLANG_SUPPRESS_WARNING_POP
04715         DOCTEST_GCC_SUPPRESS_WARNING_POP
04716         DOCTEST_MSVC_SUPPRESS_WARNING_POP
04717     }
04718
04719     const char *failureString(assertType::Enum at) {
04720         if (at & assertType::is_warn)
04721             return "WARNING";
04722         if (at & assertType::is_check)
04723             return "ERROR";
04724         if (at & assertType::is_require)
04725             return "FATAL ERROR";
04726         return "";
04727     }
04728
04729 } // namespace doctest
04730
04731 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
04732
04733 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
04734
04735 #if !defined(DOCTEST_CONFIG_COLORS_NONE)
04736 #if !defined(DOCTEST_CONFIG_COLORS_WINDOWS) && !defined(DOCTEST_CONFIG_COLORS_ANSI)
04737 #ifdef DOCTEST_PLATFORM_WINDOWS
04738     #define DOCTEST_CONFIG_COLORS_WINDOWS
04739 #else // linux
04740     #define DOCTEST_CONFIG_COLORS_ANSI

```

```

04741 #endif // platform
04742 #endif // DOCTEST_CONFIG_COLORS_WINDOWS && DOCTEST_CONFIG_COLORS_ANSI
04743 #endif // DOCTEST_CONFIG_COLORS_NONE
04744
04745 namespace doctest {
04746
04747 namespace detail {
04748 void color_to_stream(std::ostream &, Color::Enum) DOCTEST_BRANCH_ON_DISABLED({}, ;);
04749 } // namespace detail
04750
04751 namespace Color {
04752 std::ostream &operator<<(std::ostream &s, Color::Enum code) {
04753     detail::color_to_stream(s, code);
04754     return s;
04755 }
04756 } // namespace Color
04757
04758 #ifndef DOCTEST_CONFIG_DISABLE
04759 namespace detail {
04760 DOCTEST_CLANG_SUPPRESS_WARNING_WITH_PUSH("-Wdeprecated-declarations")
04761 void color_to_stream(std::ostream &s, Color::Enum code) {
04762     static_cast<void>(s); // for DOCTEST_CONFIG_COLORS_NONE or DOCTEST_CONFIG_COLORS_WINDOWS
04763     static_cast<void>(code); // for DOCTEST_CONFIG_COLORS_NONE
04764 #ifndef DOCTEST_CONFIG_COLORS_ANSI
04765     if (g_no_colors || (isatty(STDOUT_FILENO) == false && getContextOptions()->force_colors == false))
04766         return;
04767
04768     auto col = ""; // NOLINT(clang-analyzer-deadcode.DeadStores)
04769     DOCTEST_CLANG_SUPPRESS_WARNING_PUSH
04770     DOCTEST_CLANG_SUPPRESS_WARNING("-Wcovered-switch-default")
04771     switch (code) {
04772     case Color::Red: col = "[0;31m"; break;
04773     case Color::Green: col = "[0;32m"; break;
04774     case Color::Blue: col = "[0;34m"; break;
04775     case Color::Cyan: col = "[0;36m"; break;
04776     case Color::Yellow: col = "[0;33m"; break;
04777     case Color::Grey: col = "[1;30m"; break;
04778     case Color::LightGrey: col = "[0;37m"; break;
04779     case Color::BrightRed: col = "[1;31m"; break;
04780     case Color::BrightGreen: col = "[1;32m"; break;
04781     case Color::BrightWhite: col = "[1;37m"; break;
04782     case Color::Bright: // invalid
04783     case Color::None:
04784     case Color::White:
04785     default: col = "[0m";
04786     }
04787     DOCTEST_CLANG_SUPPRESS_WARNING_POP
04788     s << "\033" << col;
04789 #endif // DOCTEST_CONFIG_COLORS_ANSI
04790
04791 #ifndef DOCTEST_CONFIG_COLORS_WINDOWS
04792     if (g_no_colors || (isatty(_fileno(stdout)) == false && getContextOptions()->force_colors ==
04793 false))
04794         return;
04795
04796     static struct ConsoleHelper {
04797         HANDLE stdoutHandle;
04798         WORD origFgAttrs;
04799         WORD origBgAttrs;
04800
04801         ConsoleHelper() {
04802             stdoutHandle = GetStdHandle(STD_OUTPUT_HANDLE);
04803             CONSOLE_SCREEN_BUFFER_INFO csbiInfo;
04804             GetConsoleScreenBufferInfo(stdoutHandle, &csbiInfo);
04805             origFgAttrs =
04806                 csbiInfo.wAttributes & ~(BACKGROUND_GREEN | BACKGROUND_RED | BACKGROUND_BLUE |
04807 BACKGROUND_INTENSITY);
04808             origBgAttrs =
04809                 csbiInfo.wAttributes & ~(FOREGROUND_GREEN | FOREGROUND_RED | FOREGROUND_BLUE |
04810 FOREGROUND_INTENSITY);
04811         }
04812     } ch;
04813 #define DOCTEST_SET_ATTR(x) SetConsoleTextAttribute(ch.stdoutHandle, x | ch.origBgAttrs)
04814
04815     // clang-format off
04816     switch (code) {
04817     case Color::White: DOCTEST_SET_ATTR(FOREGROUND_GREEN | FOREGROUND_RED |
04818 FOREGROUND_BLUE); break;
04819     case Color::Red: DOCTEST_SET_ATTR(FOREGROUND_RED);
04820 break;
04821     case Color::Green: DOCTEST_SET_ATTR(FOREGROUND_GREEN);
04822 break;
04823     case Color::Blue: DOCTEST_SET_ATTR(FOREGROUND_BLUE);
04824 break;
04825     case Color::Cyan: DOCTEST_SET_ATTR(FOREGROUND_BLUE | FOREGROUND_GREEN);
04826 break;

```

```

04820         case Color::Yellow:      DOCTEST_SET_ATTR(FOREGROUND_RED | FOREGROUND_GREEN);
04821     break; case Color::Grey:      DOCTEST_SET_ATTR(0);
04822     break; case Color::LightGrey: DOCTEST_SET_ATTR(FOREGROUND_INTENSITY);
04823     break; case Color::BrightRed: DOCTEST_SET_ATTR(FOREGROUND_INTENSITY | FOREGROUND_RED);
04824     break; case Color::BrightGreen: DOCTEST_SET_ATTR(FOREGROUND_INTENSITY | FOREGROUND_GREEN);
04825     break; case Color::BrightWhite: DOCTEST_SET_ATTR(FOREGROUND_INTENSITY | FOREGROUND_GREEN |
FOREGROUND_RED | FOREGROUND_BLUE); break;
04826         case Color::None:
04827         case Color::Bright:      // invalid
04828         default:                 DOCTEST_SET_ATTR(ch.origFgAttrs);
04829     }
04830     // clang-format on
04831 #endif // DOCTEST_CONFIG_COLORS_WINDOWS
04832 }
04833 DOCTEST_CLANG_SUPPRESS_WARNING_POP
04834 } // namespace detail
04835 #endif // DOCTEST_CONFIG_DISABLED
04836 } // namespace doctest
04837
04838 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
04839
04840 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
04841
04842 namespace doctest {
04843
04844     const ContextOptions *getContextOptions() {
04845         return DOCTEST_BRANCH_ON_DISABLED(nullptr, detail::g_cs);
04846     }
04847
04848 } // namespace doctest
04849
04850 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
04851 #ifndef DOCTEST_PARTS_PRIVATE_REPORTERS_COMMON
04852 #define DOCTEST_PARTS_PRIVATE_REPORTERS_COMMON
04853
04854 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
04855
04856 #ifndef DOCTEST_CONFIG_OPTIONS_PREFIX
04857 #define DOCTEST_CONFIG_OPTIONS_PREFIX "dt-"
04858 #endif
04859
04860 #ifndef DOCTEST_CONFIG_DISABLE
04861
04862 namespace doctest {
04863
04864     void fulltext_log_assert_to_stream(std::ostream &s, const AssertData &rb);
04865
04866 } // namespace doctest
04867
04868 #endif // DOCTEST_CONFIG_DISABLE
04869
04870 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
04871
04872 #endif // DOCTEST_PARTS_PRIVATE_REPORTERS_COMMON
04873 #ifndef DOCTEST_PARTS_PRIVATE_REPORTERS_DEBUG_OUTPUT_WINDOW
04874 #define DOCTEST_PARTS_PRIVATE_REPORTERS_DEBUG_OUTPUT_WINDOW
04875
04876 #ifndef DOCTEST_PARTS_PRIVATE_REPORTERS_CONSOLE
04877 #define DOCTEST_PARTS_PRIVATE_REPORTERS_CONSOLE
04878
04879 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
04880
04881 #ifndef DOCTEST_CONFIG_NO_UNPREFIXED_OPTIONS
04882 #define DOCTEST_OPTIONS_PREFIX_DISPLAY DOCTEST_CONFIG_OPTIONS_PREFIX
04883 #else
04884 #define DOCTEST_OPTIONS_PREFIX_DISPLAY ""
04885 #endif
04886
04887 #ifndef DOCTEST_CONFIG_DISABLE
04888
04889 namespace doctest {
04890
04891     struct Whitespace {
04892         int nrSpaces;
04893         explicit Whitespace(int nr);
04894     };
04895
04896     std::ostream &operator<<(std::ostream &out, const Whitespace &ws);
04897
04898     struct ConsoleReporter : public IReporter {
04899
04900

```

```

04901     std::ostream &s;
04902     bool hasLoggedCurrentTestStart;
04903     std::vector<SubcaseSignature> subcasesStack;
04904     size_t currentSubcaseLevel;
04905     DOCTEST_DECLARE_MUTEX(mutex)
04906
04907     // caching pointers/references to objects of these types - safe to do
04908     const ContextOptions &opt;
04909     const TestCaseData *tc;
04910
04911     ConsoleReporter(const ContextOptions &co);
04912
04913     ConsoleReporter(const ContextOptions &co, std::ostream &ostr);
04914
04915     // =====
04916     // WHAT FOLLOWS ARE HELPERS USED BY THE OVERRIDES OF THE VIRTUAL METHODS OF THE INTERFACE
04917     // =====
04918
04919     void separator_to_stream();
04920
04921     static const char *getSuccessOrFailString(bool success, assertType::Enum at, const char
*success_str);
04922
04923     static Color::Enum getSuccessOrFailColor(bool success, assertType::Enum at);
04924
04925     void successOrFailColoredStringToStream(bool success, assertType::Enum at, const char *success_str
= "SUCCESS");
04926
04927     void log_contexts();
04928
04929     // this was requested to be made virtual so users could override it
04930     virtual void file_line_to_stream(const char *file, int line, const char *tail = "");
04931
04932     void logTestStart();
04933
04934     void printVersion();
04935
04936     void printIntro();
04937
04938     void printHelp();
04939
04940     void printRegisteredReporters();
04941
04942     // =====
04943     // WHAT FOLLOWS ARE OVERRIDES OF THE VIRTUAL METHODS OF THE REPORTER INTERFACE
04944     // =====
04945
04946     void report_query(const QueryData &in) override;
04947
04948     void test_run_start() override;
04949
04950     void test_run_end(const TestRunStats &p) override;
04951
04952     void test_case_start(const TestCaseData &in) override;
04953
04954     void test_case_reenter(const TestCaseData &) override;
04955
04956     void test_case_end(const CurrentTestCaseStats &st) override;
04957
04958     void test_case_exception(const TestCaseException &e) override;
04959
04960     void subcase_start(const SubcaseSignature &subc) override;
04961
04962     void subcase_end() override;
04963
04964     void log_assert(const AssertData &rb) override;
04965
04966     void log_message(const MessageData &mb) override;
04967
04968     void test_case_skipped(const TestCaseData &) override;
04969 };
04970
04971 DOCTEST_REGISTER_REPORTER("console", 0, ConsoleReporter);
04972
04973 } // namespace doctest
04974
04975 #endif // DOCTEST_CONFIG_DISABLE
04976
04977 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
04978
04979 #endif // DOCTEST_PARTS_PRIVATE_REPORTERS_CONSOLE
04980
04981 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
04982
04983 #ifndef DOCTEST_CONFIG_DISABLE
04984
04985 namespace doctest {

```



```

04986 namespace detail {
04987
04988 #ifndef DOCTEST_PLATFORM_WINDOWS
04989 struct DebugOutputWindowReporter : public ConsoleReporter {
04990     DOCTEST_THREAD_LOCAL static std::ostringstream oss;
04991
04992     DebugOutputWindowReporter(const ContextOptions &co);
04993
04994     void test_run_start() override;
04995     void test_run_end(const TestRunStats &in) override;
04996     void test_case_start(const TestCaseData &in) override;
04997     void test_case_reenter(const TestCaseData &in) override;
04998     void test_case_end(const CurrentTestCaseStats &in) override;
04999     void test_case_exception(const TestCaseException &in) override;
05000     void subcase_start(const SubcaseSignature &in) override;
05001     void subcase_end(DOCTEST_EMPTY DOCTEST_EMPTY) override;
05002     void log_assert(const AssertData &in) override;
05003     void log_message(const MessageData &in) override;
05004     void test_case_skipped(const TestCaseData &in) override;
05005 };
05006 #endif // DOCTEST_PLATFORM_WINDOWS
05007
05008 } // namespace detail
05009 } // namespace doctest
05010
05011 #endif // DOCTEST_CONFIG_DISABLE
05012
05013 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
05014
05015 #endif // DOCTEST_PARTS_PRIVATE_REPORTERS_DEBUG_OUTPUT_WINDOW
05016 #ifndef DOCTEST_PARTS_PRIVATE_TEST_CASE
05017 #define DOCTEST_PARTS_PRIVATE_TEST_CASE
05018
05019
05020 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
05021
05022 #ifndef DOCTEST_CONFIG_DISABLE
05023
05024 namespace doctest {
05025 namespace detail {
05026
05027 // all the registered tests
05028 std::set<TestCase> &getRegisteredTests();
05029
05030 } // namespace detail
05031 } // namespace doctest
05032
05033 #endif // DOCTEST_CONFIG_DISABLE
05034
05035 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
05036
05037 #endif // DOCTEST_PARTS_PRIVATE_TEST_CASE
05038 #ifndef DOCTEST_PARTS_PRIVATE_FILTERS
05039 #define DOCTEST_PARTS_PRIVATE_FILTERS
05040
05041
05042 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
05043
05044 #ifndef DOCTEST_CONFIG_DISABLE
05045
05046 namespace doctest {
05047 namespace detail {
05048
05049 // matching of a string against a wildcard mask (case sensitivity configurable) taken from
05050 // https://www.codeproject.com/Articles/1088/Wildcard-string-compare-globbing
05051 int wilddcmp(const char *str, const char *wild, bool caseSensitive);
05052
05053 // checks if the name matches any of the filters (and can be configured what to do when empty)
05054 bool matchesAny(const char *name, const std::vector<String> &filters, bool matchEmpty, bool
    caseSensitive);
05055
05056 } // namespace detail
05057 } // namespace doctest
05058
05059 #endif // DOCTEST_CONFIG_DISABLE
05060
05061 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
05062
05063 #endif // DOCTEST_PARTS_PRIVATE_FILTERS
05064 #ifndef DOCTEST_PARTS_PRIVATE_SIGNALS
05065 #define DOCTEST_PARTS_PRIVATE_SIGNALS
05066
05067
05068 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
05069
05070 #ifndef DOCTEST_CONFIG_DISABLE
05071

```

```

05072 namespace doctest {
05073 namespace detail {
05074
05075 #if !defined(DOCTEST_CONFIG_POSIX_SIGNALS) && !defined(DOCTEST_CONFIG_WINDOWS_SEH)
05076 struct FatalConditionHandler {
05077     static void reset();
05078     static void allocateAltStackMem();
05079     static void freeAltStackMem();
05080 };
05081 #else // DOCTEST_CONFIG_POSIX_SIGNALS || DOCTEST_CONFIG_WINDOWS_SEH
05082
05083 #ifdef DOCTEST_PLATFORM_WINDOWS
05084
05085 struct SignalDefs {
05086     DWORD id;
05087     const char *name;
05088 };
05089
05090 struct FatalConditionHandler {
05091     static LONG CALLBACK handleException(PEXCEPTION_POINTERS ExceptionInfo);
05092     static void allocateAltStackMem();
05093     static void freeAltStackMem();
05094
05095     FatalConditionHandler();
05096
05097     static void reset();
05098
05099     ~FatalConditionHandler();
05100
05101 private:
05102     static UINT prev_error_mode_1;
05103     static int prev_error_mode_2;
05104     static unsigned int prev_abort_behavior;
05105     static int prev_report_mode;
05106     static _HFILE prev_report_file;
05107     static void(DOCTEST_CDECL *prev_sigabrt_handler)(int);
05108     static std::terminate_handler original_terminate_handler;
05109     static bool isSet;
05110     static ULONG guaranteeSize;
05111     static LPTOP_LEVEL_EXCEPTION_FILTER previousTop;
05112 };
05113
05114 #else // DOCTEST_PLATFORM_WINDOWS
05115
05116 struct SignalDefs {
05117     int id;
05118     const char *name;
05119 };
05120
05121 struct FatalConditionHandler {
05122     static bool isSet;
05123     static struct sigaction oldSigActions[6];
05124     static stack_t oldSigStack;
05125     static size_t altStackSize;
05126     static char *altStackMem;
05127
05128     static void handleSignal(int sig);
05129
05130     static void allocateAltStackMem();
05131
05132     static void freeAltStackMem();
05133
05134     FatalConditionHandler();
05135
05136     ~FatalConditionHandler();
05137     static void reset();
05138 };
05139
05140 #endif // DOCTEST_PLATFORM_WINDOWS
05141 #endif // DOCTEST_CONFIG_POSIX_SIGNALS || DOCTEST_CONFIG_WINDOWS_SEH
05142
05143 } // namespace detail
05144 } // namespace doctest
05145
05146 #endif // DOCTEST_CONFIG_DISABLE
05147
05148 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
05149
05150 #endif // DOCTEST_PARTS_PRIVATE_SIGNALS
05151
05152 // Fix for #1035
05153 #ifndef DOCTEST_PARTS_PRIVATE_REPORTERS_JUNIT
05154 #define DOCTEST_PARTS_PRIVATE_REPORTERS_JUNIT
05155
05156 #ifndef DOCTEST_PARTS_PRIVATE_XML
05157 #define DOCTEST_PARTS_PRIVATE_XML
05158

```

```

05159
05160 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
05161
05162 #ifndef DOCTEST_CONFIG_DISABLE
05163
05164 namespace doctest {
05165 namespace detail {
05166
05167 // =====
05168 // The following code has been taken verbatim from Catch2/include/internal/catch_xmlwriter.h
05169 // This is done so cherry-picking bug fixes is trivial - even the style/formatting is untouched.
05170 // =====
05171 /* clang-format off */ /* NOLINTBEGIN */
05172
05173     class XmlEncode {
05174     public:
05175         enum ForWhat { ForTextNodes, ForAttributes };
05176
05177         XmlEncode( std::string const& str, ForWhat forWhat = ForTextNodes );
05178
05179         void encodeTo( std::ostream& os ) const;
05180
05181         friend std::ostream& operator < ( std::ostream& os, XmlEncode const& xmlEncode );
05182
05183     private:
05184         std::string m_str;
05185         ForWhat m_forWhat;
05186     };
05187
05188     class XmlWriter {
05189     public:
05190
05191         class ScopedElement {
05192         public:
05193             ScopedElement( XmlWriter* writer );
05194
05195             ScopedElement( ScopedElement&& other ) DOCTEST_NOEXCEPT;
05196             ScopedElement& operator=( ScopedElement&& other ) DOCTEST_NOEXCEPT;
05197
05198             ~ScopedElement();
05199
05200             ScopedElement& writeText( std::string const& text, bool indent = true );
05201
05202             template<typename T>
05203             ScopedElement& writeAttribute( std::string const& name, T const& attribute ) {
05204                 m_writer->writeAttribute( name, attribute );
05205                 return *this;
05206             }
05207
05208         private:
05209             mutable XmlWriter* m_writer = nullptr;
05210         };
05211
05212 #ifndef DOCTEST_CONFIG_NO_INCLUDE_Iostream
05213         XmlWriter( std::ostream& os = std::cout );
05214 #else // DOCTEST_CONFIG_NO_INCLUDE_Iostream
05215         XmlWriter( std::ostream& os );
05216 #endif // DOCTEST_CONFIG_NO_INCLUDE_Iostream
05217         ~XmlWriter();
05218
05219         XmlWriter( XmlWriter const& ) = delete;
05220         XmlWriter& operator=( XmlWriter const& ) = delete;
05221
05222         XmlWriter& startElement( std::string const& name );
05223
05224         ScopedElement scopedElement( std::string const& name );
05225
05226         XmlWriter& endElement();
05227
05228         XmlWriter& writeAttribute( std::string const& name, std::string const& attribute );
05229
05230         XmlWriter& writeAttribute( std::string const& name, const char* attribute );
05231
05232         XmlWriter& writeAttribute( std::string const& name, bool attribute );
05233
05234         template<typename T>
05235         XmlWriter& writeAttribute( std::string const& name, T const& attribute ) {
05236             std::stringstream rss;
05237             rss << attribute;
05238             return writeAttribute( name, rss.str() );
05239         }
05240
05241         XmlWriter& writeText( std::string const& text, bool indent = true );
05242
05243         //XmlWriter& writeComment( std::string const& text );
05244
05245         //void writeStylesheetRef( std::string const& url );

```

```

05246
05247     //XmlWriter& writeBlankLine();
05248
05249     void ensureTagClosed();
05250
05251     void writeDeclaration();
05252
05253     private:
05254
05255     void newlineIfNecessary();
05256
05257     bool m_tagIsOpen = false;
05258     bool m_needsNewline = false;
05259     std::vector<std::string> m_tags;
05260     std::string m_indent;
05261     std::ostream& m_os;
05262 };
05263
05264 /* clang-format on */ /* NOLINTEND */
05265 // =====
05266 // End of copy-pasted code from Catch
05267 // =====
05268
05269 } // namespace detail
05270 } // namespace doctest
05271
05272 #endif // DOCTEST_CONFIG_DISABLE
05273
05274 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
05275
05276 #endif // DOCTEST_PARTS_PRIVATE_XML
05277
05278 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
05279
05280 #ifndef DOCTEST_CONFIG_DISABLE
05281
05282 namespace doctest {
05283
05284     // TODO:
05285     // - log_message()
05286     // - respond to queries
05287     // - honor remaining options
05288     // - more attributes in tags
05289     struct JUnitReporter : public IReporter {
05290         detail::XmlWriter xml;
05291         DOCTEST_DECLARE_MUTEX(mutex)
05292         detail::Timer timer;
05293         std::vector<String> deepestSubcaseStackNames;
05294
05295         struct JUnitTestCaseData {
05296             static std::string getCurrentTimestamp();
05297
05298             struct JUnitTestMessage {
05299                 JUnitTestMessage(const std::string &_message, const std::string &_type, const std::string
&_details);
05300
05301                 JUnitTestMessage(const std::string &_message, const std::string &_details);
05302
05303                 std::string message, type, details;
05304             };
05305
05306             struct JUnitTestCase {
05307                 JUnitTestCase(const std::string &_classname, const std::string &_name);
05308
05309                 std::string classname, name;
05310                 double time;
05311                 std::vector<JUnitTestMessage> failures, errors;
05312             };
05313
05314             void add(const std::string &classname, const std::string &name);
05315
05316             void appendSubcaseNamesToLastTestcase(std::vector<String> nameStack);
05317
05318             void addTime(double time);
05319
05320             void addFailure(const std::string &message, const std::string &type, const std::string
&details);
05321
05322             void addError(const std::string &message, const std::string &details);
05323
05324             std::vector<JUnitTestCase> testcases;
05325             double totalSeconds = 0;
05326             int totalErrors = 0, totalFailures = 0;
05327         };
05328
05329         JUnitTestCaseData testCaseData;
05330

```

```

05331     // caching pointers/references to objects of these types - safe to do
05332     const ContextOptions &opt;
05333     const TestCaseData *tc = nullptr;
05334
05335     JUnitReporter(const ContextOptions &co);
05336
05337     unsigned line(unsigned l) const;
05338
05339     // =====
05340     // WHAT FOLLOWS ARE OVERRIDES OF THE VIRTUAL METHODS OF THE REPORTER INTERFACE
05341     // =====
05342
05343     void report_query(const QueryData &) override;
05344
05345     void test_run_start() override;
05346
05347     void test_run_end(const TestRunStats &p) override;
05348
05349     void test_case_start(const TestCaseData &in) override;
05350
05351     void test_case_reenter(const TestCaseData &in) override;
05352
05353     void test_case_end(const CurrentTestCaseStats &) override;
05354
05355     void test_case_exception(const TestCaseException &e) override;
05356
05357     void subcase_start(const SubcaseSignature &in) override;
05358
05359     void subcase_end() override;
05360
05361     void log_assert(const AssertData &rb) override;
05362
05363     void log_message(const MessageData &mb) override;
05364
05365     void test_case_skipped(const TestCaseData &) override;
05366
05367     static void log_contexts(std::ostream &s);
05368 };
05369
05370 DOCTEST_REGISTER_REPORTER("junit", 0, JUnitReporter);
05371
05372 } // namespace doctest
05373
05374 #endif // DOCTEST_CONFIG_DISABLE
05375
05376 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
05377
05378 #endif // DOCTEST_PARTS_PRIVATE_REPORTERS_JUNIT
05379 #ifndef DOCTEST_PARTS_PRIVATE_REPORTERS_XML
05380 #define DOCTEST_PARTS_PRIVATE_REPORTERS_XML
05381
05382 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
05383
05384 #ifndef DOCTEST_CONFIG_DISABLE
05385 namespace doctest {
05386
05387 struct XmlReporter : public IReporter {
05388     detail::XmlWriter xml;
05389     DOCTEST_DECLARE_MUTEX(mutex)
05390
05391     // caching pointers/references to objects of these types - safe to do
05392     const ContextOptions &opt;
05393     const TestCaseData *tc = nullptr;
05394
05395     XmlReporter(const ContextOptions &co);
05396
05397     void log_contexts();
05398
05399     unsigned line(unsigned l) const;
05400
05401     void test_case_start_impl(const TestCaseData &in);
05402
05403     // =====
05404     // WHAT FOLLOWS ARE OVERRIDES OF THE VIRTUAL METHODS OF THE REPORTER INTERFACE
05405     // =====
05406
05407     void report_query(const QueryData &in) override;
05408
05409     void test_run_start() override;
05410
05411     void test_run_end(const TestRunStats &p) override;
05412
05413     void test_case_start(const TestCaseData &in) override;
05414
05415     void test_case_reenter(const TestCaseData &) override;
05416
05417

```

```

05418
05419     void test_case_end(const CurrentTestCaseStats &st) override;
05420
05421     void test_case_exception(const TestCaseException &e) override;
05422
05423     void subcase_start(const SubcaseSignature &in) override;
05424
05425     void subcase_end() override;
05426
05427     void log_assert(const AssertData &rb) override;
05428
05429     void log_message(const MessageData &mb) override;
05430
05431     void test_case_skipped(const TestCaseData &in) override;
05432 };
05433
05434 DOCTEST_REGISTER_REPORTER("xml", 0, XmlReporter);
05435
05436 } // namespace doctest
05437
05438 #endif // DOCTEST_CONFIG_DISABLE
05439
05440 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
05441
05442 #endif // DOCTEST_PARTS_PRIVATE_REPORTERS_XML
05443
05444 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
05445
05446 namespace doctest {
05447
05448     bool is_running_in_test = false;
05449
05450     #ifdef DOCTEST_CONFIG_DISABLE
05451
05452     // NOLINTBEGIN(readability-convert-member-functions-to-static)
05453     Context::Context(int, const char *const *) {}
05454     Context::~Context() = default;
05455     void Context::applyCommandLine(int, const char *const *) {}
05456     void Context::addFilter(const char *, const char *) {}
05457     void Context::clearFilters() {}
05458     void Context::setOption(const char *, bool) {}
05459     void Context::setOption(const char *, int) {}
05460     void Context::setOption(const char *, const char *) {}
05461     bool Context::shouldExit() {
05462         return false;
05463     }
05464     void Context::setAsDefaultForAssertsOutOfTestCases() {}
05465     void Context::setAssertHandler(detail::assert_handler) {}
05466     void Context::setCout(std::ostream *) {}
05467     int Context::run() {
05468         return 0;
05469     }
05470     // NOLINTEND(readability-convert-member-functions-to-static)
05471
05472     #else
05473
05474     namespace detail {
05475         // for sorting tests by file/line
05476         bool fileOrderComparator(const TestCase *lhs, const TestCase *rhs) {
05477             // this is needed because MSVC gives different case for drive letters
05478             // for __FILE__ when evaluated in a header and a source file
05479             const int res = lhs->m_file.compare(rhs->m_file, static_cast<bool>(DOCTEST_MSVC));
05480             if (res != 0)
05481                 return res < 0;
05482             if (lhs->m_line != rhs->m_line)
05483                 return lhs->m_line < rhs->m_line;
05484             return lhs->m_template_id < rhs->m_template_id;
05485         }
05486
05487         // for sorting tests by suite/file/line
05488         bool suiteOrderComparator(const TestCase *lhs, const TestCase *rhs) {
05489             const int res = std::strcmp(lhs->m_test_suite, rhs->m_test_suite);
05490             if (res != 0)
05491                 return res < 0;
05492             return fileOrderComparator(lhs, rhs);
05493         }
05494
05495         // for sorting tests by name/suite/file/line
05496         bool nameOrderComparator(const TestCase *lhs, const TestCase *rhs) {
05497             const int res = std::strcmp(lhs->m_name, rhs->m_name);
05498             if (res != 0)
05499                 return res < 0;
05500             return suiteOrderComparator(lhs, rhs);
05501         }
05502
05503         // the implementation of parseOption()
05504         bool parseOptionImpl(int argc, const char *const *argv, const char *pattern, String *value) {

```

```

05505 // going from the end to the beginning and stopping on the first occurrence from the end
05506 for (int i = argc; i > 0; --i) {
05507     auto index = i - 1;
05508     auto temp = std::strstr(argv[index], pattern);
05509     if (temp && (value || strlen(temp) == strlen(pattern))) {
05510         // eliminate matches in which the chars before the option are not '-'
05511         bool noBadCharsFound = true;
05512         auto curr = argv[index];
05513         while (curr != temp) {
05514             if (*curr++ != '-') {
05515                 noBadCharsFound = false;
05516                 break;
05517             }
05518         }
05519         if (noBadCharsFound && argv[index][0] == '-') {
05520             if (value) {
05521                 // parsing the value of an option
05522                 temp += strlen(pattern);
05523                 const unsigned len = strlen(temp);
05524                 if (len) {
05525                     *value = temp;
05526                     return true;
05527                 }
05528             } else {
05529                 // just a flag - no value
05530                 return true;
05531             }
05532         }
05533     }
05534 }
05535 return false;
05536 }
05537
05538 // parses an option and returns the string after the '=' character
05539 bool parseOption(
05540     int argc, const char *const *argv, const char *pattern, String *value = nullptr, const String
    &defaultVal = String()
05541 ) {
05542     if (value)
05543         *value = defaultVal;
05544 #ifndef DOCTEST_CONFIG_NO_UNPREFIXED_OPTIONS
05545     // offset (normally 3 for "dt-") to skip prefix
05546     if (parseOptionImpl(argc, argv, pattern + strlen(DOCTEST_CONFIG_OPTIONS_PREFIX), value))
05547         return true;
05548 #endif // DOCTEST_CONFIG_NO_UNPREFIXED_OPTIONS
05549     return parseOptionImpl(argc, argv, pattern, value);
05550 }
05551
05552 // locates a flag on the command line
05553 bool parseFlag(int argc, const char *const *argv, const char *pattern) {
05554     return parseOption(argc, argv, pattern);
05555 }
05556
05557 // parses a comma separated list of words after a pattern in one of the arguments in argv
05558 bool parseCommaSepArgs(int argc, const char *const *argv, const char *pattern, std::vector<String>
    &res) {
05559     String filtersString;
05560     if (parseOption(argc, argv, pattern, &filtersString)) {
05561         // tokenize with "," as a separator, unless escaped with backslash
05562         std::ostringstream s;
05563         auto flush = [&s, &res]() {
05564             auto string = s.str();
05565             if (!string.empty()) {
05566                 res.emplace_back(string.c_str());
05567             }
05568             s.str("");
05569         };
05570
05571         bool seenBackslash = false;
05572         const char *current = filtersString.c_str();
05573         const char *end = current + strlen(current);
05574         while (current != end) {
05575             const char character = *current++;
05576             if (seenBackslash) {
05577                 seenBackslash = false;
05578                 if (character == ',' || character == '\\') {
05579                     s.put(character);
05580                     continue;
05581                 }
05582                 s.put('\\');
05583             }
05584             if (character == '\\') {
05585                 seenBackslash = true;
05586             } else if (character == ',') {
05587                 flush();
05588             } else {
05589                 s.put(character);
05590             }
05591         }
05592     }
05593 }

```

```

05590     }
05591 }
05592
05593     if (seenBackslash) {
05594         s.put('\\');
05595     }
05596     flush();
05597     return true;
05598 }
05599 return false;
05600 }
05601
05602 enum optionType { option_bool, option_int };
05603
05604 // parses an int/bool option from the command line
05605 bool parseIntOption(int argc, const char *const *argv, const char *pattern, optionType type, int &res)
05606 {
05607     String parsedValue;
05608     if (!parseOption(argc, argv, pattern, &parsedValue))
05609         return false;
05610
05611     if (type) {
05612         // integer
05613         // TODO: change this to use std::stoi or something else! currently it uses undefined
05614         // behavior - assumes '0' on failed parse...
05615         // NOLINTNEXTLINE (bugprone-unchecked-string-to-number-conversion, cert-err34-c)
05616         const int theInt = std::atoi(parsedValue.c_str());
05617         if (theInt != 0) {
05618             res = theInt;
05619             return true;
05620         }
05621     } else {
05622         // boolean
05623         const char positive[][5] = {"1", "true", "on", "yes"}; // 5 - strlen("true") + 1
05624         const char negative[][6] = {"0", "false", "off", "no"}; // 6 - strlen("false") + 1
05625
05626         // if the value matches any of the positive/negative possibilities
05627         for (unsigned i = 0; i < 4; i++) {
05628             if (parsedValue.compare(positive[i], true) == 0) {
05629                 res = 1;
05630                 return true;
05631             }
05632             if (parsedValue.compare(negative[i], true) == 0) {
05633                 res = 0;
05634                 return true;
05635             }
05636         }
05637         return false;
05638     }
05639 }
05640 // namespace detail
05641
05642 Context::Context(int argc, const char *const *argv)
05643 : p(new detail::ContextState) {
05644     parseArgs(argc, argv, true);
05645     if (argc)
05646         p->binary_name = argv[0];
05647 }
05648
05649 Context::~Context() {
05650     if (detail::g_cs == p)
05651         detail::g_cs = nullptr;
05652     delete p;
05653 }
05654
05655 void Context::applyCommandLine(int argc, const char *const *argv) {
05656     parseArgs(argc, argv);
05657     if (argc)
05658         p->binary_name = argv[0];
05659 }
05660
05661 // parses args
05662 void Context::parseArgs(int argc, const char *const *argv, bool withDefaults) {
05663     using namespace detail;
05664
05665     // clang-format off
05666     parseCommaSepArgs(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "source-file=", p->filters[0]);
05667     parseCommaSepArgs(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "sf=", p->filters[0]);
05668     parseCommaSepArgs(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "source-file-exclude=", p->filters[1]);
05669     parseCommaSepArgs(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "sfe=", p->filters[1]);
05670     parseCommaSepArgs(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "test-suite=", p->filters[2]);
05671     parseCommaSepArgs(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "ts=", p->filters[2]);
05672     parseCommaSepArgs(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "test-suite-exclude=", p->filters[3]);
05673     parseCommaSepArgs(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "tse=", p->filters[3]);
05674     parseCommaSepArgs(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "test-case=", p->filters[4]);
05675     parseCommaSepArgs(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "tc=", p->filters[4]);

```



```

05676     parseCommaSepArgs(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "test-case-exclude=", p->filters[5]);
05677     parseCommaSepArgs(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "tce=", p->filters[5]);
05678     parseCommaSepArgs(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "subcase=", p->filters[6]);
05679     parseCommaSepArgs(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "sc=", p->filters[6]);
05680     parseCommaSepArgs(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "subcase-exclude=", p->filters[7]);
05681     parseCommaSepArgs(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "sce=", p->filters[7]);
05682     parseCommaSepArgs(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "reporters=", p->filters[8]);
05683     parseCommaSepArgs(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "r=", p->filters[8]);
05684     // clang-format on
05685
05686     int intRes = 0;
05687     String strRes;
05688
05689 #define DOCTEST_PARSE_AS_BOOL_OR_FLAG(name, sname, var, default)
05690 \
05691 \
05692 \
05693 \
05694 \
05695 \
05696 \
05697 \
05698 \
05699 \
05700
05701 #define DOCTEST_PARSE_INT_OPTION(name, sname, var, default)
05702 \
05703 \
05704 \
05705 \
05706 \
05707
05708 #define DOCTEST_PARSE_STR_OPTION(name, sname, var, default)
05709 \
05710 \
05711 \
05712 \
05713 \
05714 \
05715 \
05716 \
05717 \
05718 \
05719 \
05720 \
05721 \
05722 \
05723 \
05724 \
05725 \
05726 \
05727 \
05728 \
05729 \
05730 \
05731 \
05732 \
05733 \
05734 \
05735 \
05736 \
05737 \
05738 \
05739 \
05740 \
05741 \
05742 \
05743 \
05744 \

```

```

05745
05746     if (withDefaults) {
05747         p->help = false;
05748         p->version = false;
05749         p->count = false;
05750         p->list_test_cases = false;
05751         p->list_test_suites = false;
05752         p->list_reporters = false;
05753     }
05754     if (parseFlag(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "help") ||
05755         parseFlag(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "h") ||
05756         parseFlag(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "?")) {
05757         p->help = true;
05758         p->exit = true;
05759     }
05760     if (parseFlag(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "version") ||
05761         parseFlag(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "v")) {
05762         p->version = true;
05763         p->exit = true;
05764     }
05765     if (parseFlag(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "count") ||
05766         parseFlag(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "c")) {
05767         p->count = true;
05768         p->exit = true;
05769     }
05770     if (parseFlag(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "list-test-cases") ||
05771         parseFlag(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "ltc")) {
05772         p->list_test_cases = true;
05773         p->exit = true;
05774     }
05775     if (parseFlag(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "list-test-suites") ||
05776         parseFlag(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "lts")) {
05777         p->list_test_suites = true;
05778         p->exit = true;
05779     }
05780     if (parseFlag(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "list-reporters") ||
05781         parseFlag(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "lr")) {
05782         p->list_reporters = true;
05783         p->exit = true;
05784     }
05785 }
05786
05787 // allows the user to add procedurally to the filters from the command line
05788 void Context::addFilter(const char *filter, const char *value) {
05789     setOption(filter, value);
05790 }
05791
05792 // allows the user to clear all filters from the command line
05793 void Context::clearFilters() {
05794     for (auto &curr: p->filters)
05795         curr.clear();
05796 }
05797
05798 // allows the user to override procedurally the bool options from the command line
05799 void Context::setOption(const char *option, bool value) {
05800     setOption(option, value ? "true" : "false");
05801 }
05802
05803 // allows the user to override procedurally the int options from the command line
05804 void Context::setOption(const char *option, int value) {
05805     setOption(option, toString(value).c_str());
05806 }
05807
05808 // allows the user to override procedurally the string options from the command line
05809 void Context::setOption(const char *option, const char *value) {
05810     auto argv = String("-") + option + "=" + value;
05811     auto lvalue = argv.c_str();
05812     parseArgs(1, &lvalue);
05813 }
05814
05815 // users should query this in their main() and exit the program if true
05816 bool Context::shouldExit() {
05817     return p->exit;
05818 }
05819
05820 void Context::setAsDefaultForAssertsOutOfTestCases() {
05821     detail::g_cs = p;
05822 }
05823
05824 void Context::setAssertHandler(detail::assert_handler ah) {
05825     p->ah = ah;
05826 }
05827
05828 void Context::setCout(std::ostream *out) {
05829     p->cout = out;
05830 }
05831

```

```

05832 static class DiscardOStream : public std::ostream {
05833 private:
05834     class : public std::streambuf {
05835     private:
05836         // allowing some buffering decreases the amount of calls to overflow
05837         char buf[1024];
05838
05839     protected:
05840         std::streamsize xspn(const char_type *, std::streamsize count) override {
05841             return count;
05842         }
05843
05844         int_type overflow(int_type ch) override {
05845             setp(std::begin(buf), std::end(buf));
05846             return traits_type::not_eof(ch);
05847         }
05848     } discardBuf;
05849
05850 public:
05851     DiscardOStream() noexcept
05852         : std::ostream(&discardBuf) {}
05853 } discardOut;
05854
05855 // the main function that does all the filtering and test running
05856 int Context::run() {
05857     using namespace detail;
05858
05859     // save the old context state in case such was setup - for using asserts out of a testing context
05860     auto old_cs = g_cs;
05861     // this is the current contest
05862     g_cs = p;
05863     is_running_in_test = true;
05864
05865     g_no_colors = p->no_colors;
05866     p->resetRunData();
05867
05868     std::fstream fstr;
05869     if (p->cout == nullptr) {
05870         if (p->quiet) {
05871             p->cout = &discardOut;
05872         } else if (p->out.size()) {
05873             // to a file if specified
05874             fstr.open(p->out.c_str(), std::fstream::out);
05875             p->cout = &fstr;
05876             if (!fstr.is_open()) {
05877                 // clang-format off
05878                 std::cerr << Color::Cyan << "[doctest] " << Color::None << "Could not open " << p->out << "
05879                 for writing!" << std::endl;
05880                 std::cerr << Color::Cyan << "[doctest] " << Color::None << "Defaulting to std::cout
05881                 instead" << std::endl;
05882                 p->cout = &std::cout;
05883                 // clang-format on
05884             }
05885         } else {
05886             #ifndef DOCTEST_CONFIG_NO_INCLUDE_Iostream
05887             // stdout by default
05888             p->cout = &std::cout;
05889             #else // DOCTEST_CONFIG_NO_INCLUDE_Iostream
05890             return EXIT_FAILURE;
05891             #endif // DOCTEST_CONFIG_NO_INCLUDE_Iostream
05892         }
05893     }
05894
05895     FatalConditionHandler::allocateAltStackMem();
05896
05897     auto cleanup_and_return = [&]() {
05898         FatalConditionHandler::freeAltStackMem();
05899
05900         if (fstr.is_open())
05901             fstr.close();
05902
05903         // restore context
05904         g_cs = old_cs;
05905         is_running_in_test = false;
05906
05907         // we have to free the reporters which were allocated when the run started
05908         for (auto &curr: p->reporters_currently_used)
05909             delete curr;
05910         p->reporters_currently_used.clear();
05911
05912         if (p->numTestCasesFailed && !p->no_exitcode)
05913             return EXIT_FAILURE;
05914         return EXIT_SUCCESS;
05915     };
05916
05917     // setup default reporter if none is given through the command line

```

```

05917     if (p->filters[8].empty())
05918         p->filters[8].emplace_back("console");
05919
05920     // check to see if any of the registered reporters has been selected
05921     for (auto &curr: getReporters()) {
05922         if (matchesAny(curr.first.second.c_str(), p->filters[8], false, p->case_sensitive))
05923             p->reporters_currently_used.push_back(curr.second(*g_cs));
05924     }
05925
05926     // TODO: check if there is nothing in reporters_currently_used
05927
05928     // prepend all listeners
05929     for (auto &curr: getListeners())
05930         p->reporters_currently_used.insert(p->reporters_currently_used.begin(), curr.second(*g_cs));
05931
05932 #ifdef DOCTEST_PLATFORM_WINDOWS
05933     if (isDebuggerActive() && p->no_debug_output == false)
05934         p->reporters_currently_used.push_back(new DebugOutputWindowReporter(*g_cs));
05935 #endif // DOCTEST_PLATFORM_WINDOWS
05936
05937     // handle version, help and no_run
05938     if (p->no_run || p->version || p->help || p->list_reporters) {
05939         DOCTEST_ITERATE_THROUGH_REPORTERS(report_query, QueryData());
05940
05941         return cleanup_and_return();
05942     }
05943
05944     std::vector<const TestCase *> testArray;
05945     for (auto &curr: getRegisteredTests())
05946         testArray.push_back(&curr);
05947     p->numTestCases = testArray.size();
05948
05949     // sort the collected records
05950     if (!testArray.empty()) {
05951         if (p->order_by.compare("file", true) == 0) {
05952             std::sort(testArray.begin(), testArray.end(), fileOrderComparator);
05953         } else if (p->order_by.compare("suite", true) == 0) {
05954             std::sort(testArray.begin(), testArray.end(), suiteOrderComparator);
05955         } else if (p->order_by.compare("name", true) == 0) {
05956             std::sort(testArray.begin(), testArray.end(), nameOrderComparator);
05957         } else if (p->order_by.compare("rand", true) == 0) {
05958             std::srand(p->rand_seed);
05959
05960             // random_shuffle implementation
05961             const auto first = testArray.data();
05962             for (size_t i = testArray.size() - 1; i > 0; --i) {
05963                 // NOLINTNEXTLINE(cert-msc30-c, cert-msc50-cpp, concurrency-mt-unsafe,
misc-predictable-rand)
05964                 const int idxToSwap = static_cast<int>(std::rand() % (i + 1));
05965
05966                 const auto temp = first[i];
05967
05968                 first[i] = first[idxToSwap];
05969                 first[idxToSwap] = temp;
05970             }
05971         } else if (p->order_by.compare("none", true) == 0) {
05972             // means no sorting - beneficial for death tests which call into the executable
05973             // with a specific test case in mind - we don't want to slow down the startup times
05974         }
05975     }
05976
05977     std::set<String> testSuitesPassingFilter;
05978
05979     const bool query_mode = p->count || p->list_test_cases || p->list_test_suites;
05980     std::vector<const TestCaseData *> queryResults;
05981
05982     if (!query_mode)
05983         DOCTEST_ITERATE_THROUGH_REPORTERS(test_run_start, DOCTEST_EMPTY);
05984
05985     // invoke the registered functions if they match the filter criteria (or just count them)
05986     for (auto &curr: testArray) {
05987         const auto &tc = *curr;
05988
05989         bool skip_me = false;
05990         if (tc.m_skip && !p->no_skip)
05991             skip_me = true;
05992
05993         if (!matchesAny(tc.m_file.c_str(), p->filters[0], true, p->case_sensitive))
05994             skip_me = true;
05995         if (matchesAny(tc.m_file.c_str(), p->filters[1], false, p->case_sensitive))
05996             skip_me = true;
05997         if (!matchesAny(tc.m_test_suite, p->filters[2], true, p->case_sensitive))
05998             skip_me = true;
05999         if (matchesAny(tc.m_test_suite, p->filters[3], false, p->case_sensitive))
06000             skip_me = true;
06001         if (!matchesAny(tc.m_name, p->filters[4], true, p->case_sensitive))
06002             skip_me = true;

```

```

06003         if (matchesAny(tc.m_name, p->filters[5], false, p->case_sensitive))
06004             skip_me = true;
06005
06006         if (!skip_me)
06007             p->numTestCasesPassingFilters++;
06008
06009         // skip the test if it is not in the execution range
06010         if ((p->last < p->numTestCasesPassingFilters && p->first <= p->last) ||
06011             (p->first > p->numTestCasesPassingFilters))
06012             skip_me = true;
06013
06014         if (skip_me) {
06015             if (!query_mode)
06016                 DOCTEST_ITERATE_THROUGH_REPORTERS(test_case_skipped, tc);
06017             continue;
06018         }
06019
06020         // do not execute the test if we are to only count the number of filter passing tests
06021         if (p->count)
06022             continue;
06023
06024         // print the name of the test and don't execute it
06025         if (p->list_test_cases) {
06026             queryResults.push_back(&tc);
06027             continue;
06028         }
06029
06030         // print the name of the test suite if not done already and don't execute it
06031         if (p->list_test_suites) {
06032             if ((testSuitesPassingFilt.count(tc.m_test_suite) == 0) && tc.m_test_suite[0] != '\0') {
06033                 queryResults.push_back(&tc);
06034                 testSuitesPassingFilt.insert(tc.m_test_suite);
06035                 p->numTestSuitesPassingFilters++;
06036             }
06037             continue;
06038         }
06039
06040         // execute the test if it passes all the filtering
06041         {
06042             p->currentTest = &tc;
06043
06044             p->failure_flags = TestCaseFailureReason::None;
06045             p->seconds = 0;
06046
06047             // reset atomic counters
06048             p->numAssertsFailedCurrentTest_atomic = 0;
06049             p->numAssertsCurrentTest_atomic = 0;
06050
06051             p->traversal.resetForTestCase();
06052
06053             DOCTEST_ITERATE_THROUGH_REPORTERS(test_case_start, tc);
06054
06055             p->timer.start();
06056
06057             bool run_test = true;
06058
06059             do { // NOLINT(cppcoreguidelines-avoid-do-while)
06060                 // Reset per-run traversal data while keeping the current decision path prefix.
06061                 p->traversal.resetForRun();
06062
06063                 p->shouldLogCurrentException = true;
06064
06065                 // reset stuff for logging with INFO()
06066                 p->stringifiedContexts.clear();
06067
06068                 #ifndef DOCTEST_CONFIG_NO_EXCEPTIONS
06069                     try {
06070                 #endif // DOCTEST_CONFIG_NO_EXCEPTIONS
06071                 // MSVC 2015 diagnoses fatalConditionHandler as unused (because reset() is a
06072                 // static method)
06073                     DOCTEST_MSVC_SUPPRESS_WARNING_WITH_PUSH(4101) // unreferenced local variable
06074                     const FatalConditionHandler fatalConditionHandler; // Handle signals
06075                     static_cast<void>(fatalConditionHandler);
06076                     // execute the test
06077                     tc.m_test();
06078                     FatalConditionHandler::reset();
06079                     DOCTEST_MSVC_SUPPRESS_WARNING_POP
06080                 #ifndef DOCTEST_CONFIG_NO_EXCEPTIONS
06081                     } catch (const TestFailureException &) {
06082                         p->failure_flags |= TestCaseFailureReason::AssertFailure;
06083                     } catch (...) {
06084                         DOCTEST_ITERATE_THROUGH_REPORTERS(test_case_exception,
06085 {translateActiveException(), false});
06086                         p->failure_flags |= TestCaseFailureReason::Exception;
06087                     }
06088                 #endif // DOCTEST_CONFIG_NO_EXCEPTIONS

```

```

06089         // exit this loop if enough assertions have failed - even if there are more subcases
06090         if (p->abort_after > 0 &&
06091             p->numAssertsFailed + p->numAssertsFailedCurrentTest_atomic >= p->abort_after) {
06092             run_test = false;
06093             p->failure_flags |= TestCaseFailureReason::TooManyFailedAsserts;
06094         }
06095
06096         const bool has_next_path = run_test ? p->traversal.advance() : false;
06097
06098         if (has_next_path && run_test)
06099             DOCTEST_ITERATE_THROUGH_REPORTERS(test_case_reenter, tc);
06100         if (!has_next_path)
06101             run_test = false;
06102     } while (run_test);
06103
06104     p->finalizeTestCaseData();
06105
06106     DOCTEST_ITERATE_THROUGH_REPORTERS(test_case_end, *g_cs);
06107
06108     p->currentTest = nullptr;
06109
06110     // stop executing tests if enough assertions have failed
06111     if (p->abort_after > 0 && p->numAssertsFailed >= p->abort_after)
06112         break;
06113     }
06114 }
06115
06116 if (!query_mode) {
06117     DOCTEST_ITERATE_THROUGH_REPORTERS(test_run_end, *g_cs);
06118 } else {
06119     QueryData qdata;
06120     qdata.run_stats = g_cs;
06121     qdata.data = queryResults.data();
06122     qdata.num_data = static_cast<unsigned>(queryResults.size());
06123     DOCTEST_ITERATE_THROUGH_REPORTERS(report_query, qdata);
06124 }
06125
06126 return cleanup_and_return();
06127 }
06128
06129 #endif // DOCTEST_CONFIG_DISABLE
06130
06131 } // namespace doctest
06132
06133 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
06134 #ifndef DOCTEST_PARTS_PRIVATE_CONTEXT_SCOPE
06135 #define DOCTEST_PARTS_PRIVATE_CONTEXT_SCOPE
06136
06137 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
06138
06139 #ifndef DOCTEST_CONFIG_DISABLE
06140 namespace doctest {
06141 namespace detail {
06142 extern DOCTEST_THREAD_LOCAL std::vector<IContextScope*> g_infoContexts; // for logging with INFO()
06143 } // namespace detail
06144 } // namespace doctest
06145
06146 #endif // DOCTEST_CONFIG_DISABLE
06147
06148 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
06149
06150 #endif // DOCTEST_PARTS_PRIVATE_CONTEXT_SCOPE
06151
06152 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
06153 namespace doctest {
06154 DOCTEST_DEFINE_INTERFACE(IContextScope)
06155
06156 #ifndef DOCTEST_CONFIG_DISABLE
06157 namespace detail {
06158 DOCTEST_THREAD_LOCAL std::vector<IContextScope*> g_infoContexts; // for logging with INFO()
06159
06160 ContextScopeBase::ContextScopeBase() {
06161     g_infoContexts.push_back(this);
06162 }
06163
06164 ContextScopeBase::ContextScopeBase(ContextScopeBase &&other) noexcept {
06165     if (other.need_to_destroy) {
06166         other.destroy();
06167     }
06168     other.need_to_destroy = false;
06169     g_infoContexts.push_back(this);
06170 }
06171 }
06172
06173 }
06174
06175

```

```

06176 DOCTEST_MSVC_SUPPRESS_WARNING_WITH_PUSH(4996) // std::uncaught_exception is deprecated in C++17
06177 DOCTEST_GCC_SUPPRESS_WARNING_WITH_PUSH("-Wdeprecated-declarations")
06178 DOCTEST_CLANG_SUPPRESS_WARNING_WITH_PUSH("-Wdeprecated-declarations")
06179 // destroy cannot be inlined into the destructor because that would mean calling stringify after
06180 // ContextScope has been destroyed (base class destructors run after derived class destructors).
06181 // Instead, ContextScope calls this method directly from its destructor.
06182 void ContextScopeBase::destroy() {
06183     #if defined(__cpp_lib_uncaught_exceptions) && __cpp_lib_uncaught_exceptions >= 201411L &&
    \
    (!defined(__MAC_OS_X_VERSION_MIN_REQUIRED) || __MAC_OS_X_VERSION_MIN_REQUIRED >= 101200)
06184         if (std::uncaught_exceptions() > 0) {
06185             #else
06186                 if (std::uncaught_exception()) {
06187                     #endif
06188                         std::ostringstream s;
06189                         this->stringify(&s);
06190                         g_cs->stringifiedContexts.emplace_back(s.str().c_str());
06191                     }
06192                     g_infoContexts.pop_back();
06193                 }
06194             }
06195 DOCTEST_CLANG_SUPPRESS_WARNING_POP
06196 DOCTEST_GCC_SUPPRESS_WARNING_POP
06197 DOCTEST_MSVC_SUPPRESS_WARNING_POP
06198 } // namespace detail
06199 #endif // DOCTEST_CONFIG_DISABLE
06200
06201 } // namespace doctest
06202
06203 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
06204
06205 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
06206
06207 #ifndef DOCTEST_CONFIG_DISABLE
06208
06209 namespace doctest {
06210 namespace detail {
06211
06212 ContextState *g_cs = nullptr;
06213 DOCTEST_THREAD_LOCAL bool g_no_colors;
06214
06215 void ContextState::resetRunData() {
06216     numTestCases = 0;
06217     numTestCasesPassingFilters = 0;
06218     numTestSuitesPassingFilters = 0;
06219     numTestCasesFailed = 0;
06220     numAsserts = 0;
06221     numAssertsFailed = 0;
06222     numAssertsCurrentTest = 0;
06223     numAssertsFailedCurrentTest = 0;
06224 }
06225
06226 void ContextState::finalizeTestCaseData() {
06227     seconds = timer.getElapsedSeconds();
06228
06229     // update the non-atomic counters
06230     numAsserts += numAssertsCurrentTest_atomic;
06231     numAssertsFailed += numAssertsFailedCurrentTest_atomic;
06232     numAssertsCurrentTest = numAssertsCurrentTest_atomic;
06233     numAssertsFailedCurrentTest = numAssertsFailedCurrentTest_atomic;
06234
06235     if (numAssertsFailedCurrentTest)
06236         failure_flags |= TestCaseFailureReason::AssertFailure;
06237
06238     if (Approx(currentTest->m_timeout).epsilon(DBL_EPSILON) != 0 &&
    Approx(seconds).epsilon(DBL_EPSILON) > currentTest->m_timeout)
06239         failure_flags |= TestCaseFailureReason::Timeout;
06240
06241     if (currentTest->m_should_fail) {
06242         if (failure_flags) {
06243             failure_flags |= TestCaseFailureReason::ShouldHaveFailedAndDid;
06244         } else {
06245             failure_flags |= TestCaseFailureReason::ShouldHaveFailedButDidnt;
06246         }
06247     }
06248     } else if (failure_flags && currentTest->m_may_fail) {
06249         failure_flags |= TestCaseFailureReason::CouldHaveFailedAndDid;
06250     } else if (currentTest->m_expected_failures > 0) {
06251         if (numAssertsFailedCurrentTest == currentTest->m_expected_failures) {
06252             failure_flags |= TestCaseFailureReason::FailedExactlyNumTimes;
06253         } else {
06254             failure_flags |= TestCaseFailureReason::DidntFailExactlyNumTimes;
06255         }
06256     }
06257
06258     const bool ok_to_fail = (TestCaseFailureReason::ShouldHaveFailedAndDid & failure_flags) ||
    (TestCaseFailureReason::CouldHaveFailedAndDid & failure_flags) ||
    (TestCaseFailureReason::FailedExactlyNumTimes & failure_flags);
06259
06260
06261

```

```

06262 // if any subcase has failed - the whole test case has failed
06263 testCaseSuccess = !(failure_flags && !ok_to_fail);
06264 if (!testCaseSuccess)
06265     numTestCasesFailed++;
06266 }
06267
06268 } // namespace detail
06269 } // namespace doctest
06270
06271 #endif // DOCTEST_CONFIG_DISABLE
06272
06273 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
06274
06275 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
06276
06277 #ifndef DOCTEST_CONFIG_DISABLE
06278
06279 namespace doctest {
06280 namespace detail {
06281 #ifdef DOCTEST_IS_DEBUGGER_ACTIVE
06282 bool isDebuggerActive() {
06283     return DOCTEST_IS_DEBUGGER_ACTIVE();
06284 }
06285 #else // DOCTEST_IS_DEBUGGER_ACTIVE
06286 #ifdef DOCTEST_PLATFORM_LINUX
06287 class ErrnoGuard {
06288 public:
06289     ErrnoGuard()
06290         : m_oldErrno(errno) {}
06291     ~ErrnoGuard() {
06292         errno = m_oldErrno;
06293     }
06294
06295 private:
06296     int m_oldErrno;
06297 };
06298 // See the comments in Catch2 for the reasoning behind this implementation:
06299 // https://github.com/catchorg/Catch2/blob/v2.13.1/include/internal/catch_debugger.cpp#L79-L102
06300 bool isDebuggerActive() {
06301     const ErrnoGuard guard;
06302     std::ifstream in("/proc/self/status");
06303     for (std::string line; std::getline(in, line);) {
06304         static const int PREFIX_LEN = 11;
06305         if (line.compare(0, PREFIX_LEN, "TracerPid:") == 0) {
06306             return line.length() > PREFIX_LEN && line[PREFIX_LEN] != '0';
06307         }
06308     }
06309     return false;
06310 }
06311 #elif defined(DOCTEST_PLATFORM_MAC)
06312 // The following function is taken directly from the following technical note:
06313 // https://developer.apple.com/library/archive/qa/qa1361/_index.html
06314 // Returns true if the current process is being debugged (either
06315 // running under the debugger or has a debugger attached post facto).
06316 bool isDebuggerActive() {
06317     int mib[4];
06318     kinfo_proc info;
06319     size_t size;
06320     // Initialize the flags so that, if sysctl fails for some bizarre
06321     // reason, we get a predictable result.
06322     info.kp_proc.p_flag = 0;
06323     // Initialize mib, which tells sysctl the info we want, in this case
06324     // we're looking for information about a specific process ID.
06325     mib[0] = CTL_KERN;
06326     mib[1] = KERN_PROC;
06327     mib[2] = KERN_PROC_PID;
06328     mib[3] = getpid();
06329     // Call sysctl.
06330     size = sizeof(info);
06331     if (sysctl(mib, DOCTEST_COUNTOF(mib), &info, &size, nullptr, 0) != 0) {
06332         std::cerr << "\nCall to sysctl failed - unable to determine if debugger is active **\n";
06333         return false;
06334     }
06335     // We're being debugged if the P_TRACED flag is set.
06336     return ((info.kp_proc.p_flag & P_TRACED) != 0);
06337 }
06338 #elif DOCTEST_MSVC || defined(__MINGW32__) || defined(__MINGW64__)
06339 bool isDebuggerActive() {
06340     return ::IsDebuggerPresent() != 0;
06341 }
06342 #else
06343 bool isDebuggerActive() {
06344     return false;
06345 }
06346 #endif // Platform
06347 #endif // DOCTEST_IS_DEBUGGER_ACTIVE
06348 } // namespace detail

```



```

06349 } // namespace doctest
06350
06351 #endif // DOCTEST_CONFIG_DISABLE
06352
06353 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
06354
06355 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
06356
06357 #ifndef DOCTEST_CONFIG_DISABLE
06358
06359 namespace doctest {
06360 namespace detail {
06361
06362 DOCTEST_DEFINE_INTERFACE(IExceptionTranslator)
06363
06364 void registerExceptionTranslatorImpl(const IExceptionTranslator *et) noexcept {
06365     if (std::find(getExceptionTranslators().begin(), getExceptionTranslators().end(), et) ==
06366         getExceptionTranslators().end())
06367         getExceptionTranslators().push_back(et);
06368 }
06369
06370 std::vector<const IExceptionTranslator *> &getExceptionTranslators() noexcept {
06371     static std::vector<const IExceptionTranslator *> data;
06372     return data;
06373 }
06374
06375 String translateActiveException() noexcept {
06376 #ifndef DOCTEST_CONFIG_NO_EXCEPTIONS
06377     String res;
06378     auto &translators = getExceptionTranslators();
06379     for (auto &curr: translators)
06380         if (curr->translate(res))
06381             return res;
06382     // clang-format off
06383     DOCTEST_GCC_SUPPRESS_WARNING_WITH_PUSH("-Wcatch-value")
06384     try {
06385         throw;
06386     } catch (std::exception &ex) {
06387         return ex.what();
06388     } catch (std::string &msg) {
06389         return msg.c_str();
06390     } catch (const char *msg) {
06391         return msg;
06392     } catch (...) {
06393         return "unknown exception";
06394     }
06395     DOCTEST_GCC_SUPPRESS_WARNING_POP
06396     // clang-format on
06397 #else // DOCTEST_CONFIG_NO_EXCEPTIONS
06398     return "";
06399 #endif // DOCTEST_CONFIG_NO_EXCEPTIONS
06400 }
06401
06402 } // namespace detail
06403 } // namespace doctest
06404
06405 #endif // DOCTEST_CONFIG_DISABLE
06406
06407 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
06408
06409 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
06410
06411 #ifndef DOCTEST_CONFIG_DISABLE
06412
06413 namespace doctest {
06414 namespace detail {
06415
06416 bool checkIfShouldThrow(assertType::Enum at) {
06417     if (at & assertType::is_require)
06418         return true;
06419
06420     if ((at & assertType::is_check) && getContextOptions()->abort_after > 0 &&
06421         (g_cs->numAssertsFailed + g_cs->numAssertsFailedCurrentTest_atomic) >=
06422         getContextOptions()->abort_after)
06423         return true;
06424     return false;
06425 }
06426
06427 #ifndef DOCTEST_CONFIG_NO_EXCEPTIONS
06428 DOCTEST_NORETURN void throwException() {
06429     g_cs->shouldLogCurrentException = false;
06430     throw TestFailureException(); // NOLINT(hicpp-exception-baseclass)
06431 }
06432 #else // DOCTEST_CONFIG_NO_EXCEPTIONS
06433 void throwException() {}
06434 #endif // DOCTEST_CONFIG_NO_EXCEPTIONS

```

```

06435
06436 } // namespace detail
06437 } // namespace doctest
06438
06439 #endif // DOCTEST_CONFIG_DISABLE
06440
06441 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
06442
06443 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
06444
06445 namespace doctest {
06446 namespace detail {
06447
06448 int wilddcmp(const char *str, const char *wild, bool caseSensitive) {
06449     const char *cp = str;
06450     const char *mp = wild;
06451
06452     while ((*str) && (*wild != '*')) {
06453         if ((caseSensitive ? (*wild != *str) : (tolower(*wild) != tolower(*str))) && (*wild != '?')) {
06454             return 0;
06455         }
06456         wild++;
06457         str++;
06458     }
06459
06460     while (*str) {
06461         if (*wild == '*') {
06462             if (!++mp) {
06463                 return 1;
06464             }
06465             mp = wild;
06466             cp = str + 1;
06467         } else if ((caseSensitive ? (*wild == *str) : (tolower(*wild) == tolower(*str))) || (*wild ==
06468 '?')) {
06469             wild++;
06470             str++;
06471         } else {
06472             wild = mp;
06473             str = cp++;
06474         }
06475     }
06476
06477     while (*wild == '*') {
06478         wild++;
06479     }
06480     return !*wild;
06481 }
06482
06483 bool matchesAny(const char *name, const std::vector<String> &filters, bool matchEmpty, bool
06484 caseSensitive) {
06485     if (filters.empty() && matchEmpty)
06486         return true;
06487     for (auto &curr: filters)
06488         if (wilddcmp(name, curr.c_str(), caseSensitive))
06489             return true;
06490     return false;
06491 }
06492 } // namespace detail
06493 } // namespace doctest
06494
06495 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
06496
06497 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
06498
06499 #ifdef DOCTEST_CONFIG_IMPLEMENT_WITH_MAIN
06500 DOCTEST_MSVC_SUPPRESS_WARNING_WITH_PUSH(4007) // 'function' : must be 'attribute' - see issue #182
06501 int main(int argc, char **argv) {
06502     return doctest::Context(argc, argv).run();
06503 }
06504 #endif // DOCTEST_CONFIG_IMPLEMENT_WITH_MAIN
06505
06506 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
06507
06508 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
06509
06510 namespace doctest {
06511
06512 Approx::Approx(double value)
06513     : m_epsilon(static_cast<double>(std::numeric_limits<float>::epsilon()) * 100), m_scale(1.0),
06514       m_value(value) {}
06515
06516 Approx Approx::operator()(double value) const {
06517     Approx approx(value);
06518     approx.epsilon(m_epsilon);
06519     approx.scale(m_scale);

```

```

06519     return approx;
06520 }
06521
06522 Approx &Approx::epsilon(double newEpsilon) {
06523     m_epsilon = newEpsilon;
06524     return *this;
06525 }
06526 Approx &Approx::scale(double newScale) {
06527     m_scale = newScale;
06528     return *this;
06529 }
06530
06531 bool operator==(double lhs, const Approx &rhs) {
06532     // Thanks to Richard Harris for his help refining this formula
06533     return std::fabs(lhs - rhs.m_value) <
06534         rhs.m_epsilon * (rhs.m_scale + std::max<double>(std::fabs(lhs), std::fabs(rhs.m_value)));
06535 }
06536
06537 bool operator==(const Approx &lhs, double rhs) {
06538     return operator==(rhs, lhs);
06539 }
06540
06541 bool operator!=(double lhs, const Approx &rhs) {
06542     return !operator==(lhs, rhs);
06543 }
06544
06545 bool operator!=(const Approx &lhs, double rhs) {
06546     return !operator==(rhs, lhs);
06547 }
06548
06549 bool operator<=(double lhs, const Approx &rhs) {
06550     return lhs < rhs.m_value || lhs == rhs;
06551 }
06552
06553 bool operator<=(const Approx &lhs, double rhs) {
06554     return lhs.m_value < rhs || lhs == rhs;
06555 }
06556
06557 bool operator>=(double lhs, const Approx &rhs) {
06558     return lhs > rhs.m_value || lhs == rhs;
06559 }
06560
06561 bool operator>=(const Approx &lhs, double rhs) {
06562     return lhs.m_value > rhs || lhs == rhs;
06563 }
06564
06565 bool operator<(double lhs, const Approx &rhs) {
06566     return lhs < rhs.m_value && lhs != rhs;
06567 }
06568
06569 bool operator<(const Approx &lhs, double rhs) {
06570     return lhs.m_value < rhs && lhs != rhs;
06571 }
06572
06573 bool operator>(double lhs, const Approx &rhs) {
06574     return lhs > rhs.m_value && lhs != rhs;
06575 }
06576
06577 bool operator>(const Approx &lhs, double rhs) {
06578     return lhs.m_value > rhs && lhs != rhs;
06579 }
06580
06581 String toString(const Approx &in) {
06582     return "Approx( " + doctest::toString(in.m_value) + " )";
06583 }
06584
06585 } // namespace doctest
06586
06587 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
06588
06589 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
06590
06591 namespace doctest {
06592
06593 Contains::Contains(const String &str)
06594     : string(str) {}
06595
06596 bool Contains::checkWith(const String &other) const {
06597     return strstr(other.c_str(), string.c_str()) != nullptr;
06598 }
06599
06600 String toString(const Contains &in) {
06601     return "Contains( " + in.string + " )";
06602 }
06603
06604 bool operator==(const String &lhs, const Contains &rhs) {
06605     return rhs.checkWith(lhs);

```

```

06606 }
06607
06608 bool operator==(const Contains &lhs, const String &rhs) {
06609     return lhs.checkWith(rhs);
06610 }
06611
06612 bool operator!=(const String &lhs, const Contains &rhs) {
06613     return !rhs.checkWith(lhs);
06614 }
06615
06616 bool operator!=(const Contains &lhs, const String &rhs) {
06617     return !lhs.checkWith(rhs);
06618 }
06619
06620 } // namespace doctest
06621
06622 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
06623
06624 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
06625
06626 namespace doctest {
06627
06628 DOCTEST_MSVC_SUPPRESS_WARNING_WITH_PUSH(4738)
06629 template <typename F>
06630 IsNaN<F>::operator bool() const {
06631     return std::isnan(value) ^ flipped;
06632 }
06633 DOCTEST_MSVC_SUPPRESS_WARNING_POP
06634 template struct DOCTEST_INTERFACE_DEF IsNaN<float>;
06635 template struct DOCTEST_INTERFACE_DEF IsNaN<double>;
06636 template struct DOCTEST_INTERFACE_DEF IsNaN<long double>;
06637
06638 template <typename F>
06639 String toString(IsNaN<F> in) {
06640     return String(in.flipped ? "! " : "") + "IsNaN( " + doctest::toString(in.value) + " )";
06641 }
06642
06643 String toString(IsNaN<float> in) {
06644     return toString<float>(in);
06645 }
06646
06647 String toString(IsNaN<double> in) {
06648     return toString<double>(in);
06649 }
06650
06651 String toString(IsNaN<double long> in) {
06652     return toString<double long>(in);
06653 }
06654
06655 } // namespace doctest
06656
06657 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
06658
06659 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
06660
06661 #ifndef DOCTEST_CONFIG_OPTIONS_FILE_PREFIX_SEPARATOR
06662 #define DOCTEST_CONFIG_OPTIONS_FILE_PREFIX_SEPARATOR ':'
06663 #endif
06664
06665 namespace doctest {
06666
06667 DOCTEST_CLANG_SUPPRESS_WARNING_WITH_PUSH("-Wnull-dereference")
06668 DOCTEST_GCC_SUPPRESS_WARNING_WITH_PUSH("-Wnull-dereference")
06669 // depending on the current options this will remove the path of filenames
06670 const char *skipPathFromFilename(const char *file) {
06671     #ifndef DOCTEST_CONFIG_DISABLE
06672         if (getContextOptions()->no_path_in_filenames) {
06673             auto back = std::strrchr(file, '\\');
06674             auto forward = std::strrchr(file, '/');
06675             if (back || forward) {
06676                 if (back > forward)
06677                     forward = back;
06678                 return forward + 1;
06679             }
06680         } else {
06681             const auto prefixes = getContextOptions()->strip_file_prefixes;
06682             const char separator = DOCTEST_CONFIG_OPTIONS_FILE_PREFIX_SEPARATOR;
06683             String::size_type longest_match = 0U;
06684             for (String::size_type pos = 0U; pos < prefixes.size(); ++pos) {
06685                 const auto prefix_start = pos;
06686                 pos = std::min(prefixes.find(separator, prefix_start), prefixes.size());
06687
06688                 const auto prefix_size = pos - prefix_start;
06689                 if (prefix_size > longest_match) {
06690                     // TODO under DOCTEST_MSVC: does the comparison need strnicmp() to work with drive
06691                     // letter capitalization?
06692                     if (0 == std::strncmp(prefixes.c_str() + prefix_start, file, prefix_size)) {

```

```

06693         longest_match = prefix_size;
06694     }
06695     }
06696 }
06697     return &file[longest_match];
06698 }
06699 #endif // DOCTEST_CONFIG_DISABLE
06700     return file;
06701 }
06702 DOCTEST_CLANG_SUPPRESS_WARNING_POP
06703 DOCTEST_GCC_SUPPRESS_WARNING_POP
06704
06705 } // namespace doctest
06706
06707 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
06708
06709 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
06710
06711 namespace doctest {
06712 #ifdef DOCTEST_CONFIG_DISABLE
06713
06714 DOCTEST_DEFINE_INTERFACE(IReporter)
06715
06716 int IReporter::get_num_active_contexts() {
06717     return 0;
06718 }
06719
06720 const IContextScope *const *IReporter::get_active_contexts() {
06721     return nullptr;
06722 }
06723
06724 int IReporter::get_num_stringified_contexts() {
06725     return 0;
06726 }
06727
06728 const String *IReporter::get_stringified_contexts() {
06729     return nullptr;
06730 }
06731
06732 int registerReporter(const char *, int, IReporter *) {
06733     return 0;
06734 }
06735
06736 #else
06737
06738 namespace detail {
06739 reporterMap &getReporters() noexcept {
06740     static reporterMap data;
06741     return data;
06742 }
06743
06744 reporterMap &getListeners() noexcept {
06745     static reporterMap data;
06746     return data;
06747 }
06748 } // namespace detail
06749
06750 DOCTEST_DEFINE_INTERFACE(IReporter)
06751
06752 int IReporter::get_num_active_contexts() {
06753     return static_cast<int>(detail::g_infoContexts.size());
06754 }
06755
06756 const IContextScope *const *IReporter::get_active_contexts() {
06757     return get_num_active_contexts() ? detail::g_infoContexts.data() : nullptr;
06758 }
06759
06760 int IReporter::get_num_stringified_contexts() {
06761     return static_cast<int>(detail::g_cs->stringifiedContexts.size());
06762 }
06763
06764 const String *IReporter::get_stringified_contexts() {
06765     return get_num_stringified_contexts() ? detail::g_cs->stringifiedContexts.data() : nullptr;
06766 }
06767
06768 namespace detail {
06769 void registerReporterImpl(const char *name, int priority, reporterCreatorFunc c, bool isReporter)
06770     noexcept {
06771     if (isReporter)
06772         getReporters().insert(reporterMap::value_type(reporterMap::key_type(priority, name), c));
06773     else
06774         getListeners().insert(reporterMap::value_type(reporterMap::key_type(priority, name), c));
06775 } // namespace detail
06776
06777 #endif // DOCTEST_CONFIG_DISABLE
06778 } // namespace doctest

```

```

06779
06780 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
06781
06782 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
06783
06784 #ifndef DOCTEST_CONFIG_DISABLE
06785
06786 namespace doctest {
06787
06788 void fulltext_log_assert_to_stream(std::ostream &s, const AssertData &rb) {
06789     // clang-format off
06790     if ((rb.m_at & (assertType::is_throws_as | assertType::is_throws_with)) == 0)
06791         s << Color::Cyan << assertString(rb.m_at) << "( " << rb.m_expr << " ) " << Color::None;
06792
06793     if (rb.m_at & assertType::is_throws) {
06794         s << (rb.m_threw ? "threw as expected!" : "did NOT throw at all!") << "\n";
06795     } else if ((rb.m_at & assertType::is_throws_as) && (rb.m_at & assertType::is_throws_with)) {
06796         s << Color::Cyan << assertString(rb.m_at) << "( "
06797         << rb.m_expr << ", \" " << rb.m_exception_string.c_str() << "\", " << rb.m_exception_type
06798         << " ) " << Color::None;
06799
06800         if (rb.m_threw) {
06801             if (!rb.m_failed) {
06802                 s << "threw as expected!\n";
06803             } else {
06804                 s << "threw a DIFFERENT exception! (contents: " << rb.m_exception << ")\n";
06805             }
06806         } else {
06807             s << "did NOT throw at all!\n";
06808         }
06809     } else if (rb.m_at & assertType::is_throws_as) {
06810         s << Color::Cyan << assertString(rb.m_at) << "( "
06811         << rb.m_expr << ", " << rb.m_exception_type
06812         << " ) " << Color::None
06813         << (rb.m_threw ? (rb.m_threw_as ? "threw as expected!" : "threw a DIFFERENT exception: ") :
06814         "did NOT throw at all!")
06815         << Color::Cyan << rb.m_exception << "\n";
06816     } else if (rb.m_at & assertType::is_throws_with) {
06817         s << Color::Cyan << assertString(rb.m_at) << "( "
06818         << rb.m_expr << ", \" " << rb.m_exception_string.c_str()
06819         << "\" ) " << Color::None
06820         << (rb.m_threw ? (!rb.m_failed ? "threw as expected!" : "threw a DIFFERENT exception: ") :
06821         "did NOT throw at all!")
06822         << Color::Cyan << rb.m_exception << "\n";
06823     } else if (rb.m_at & assertType::is_nothrow) {
06824         s << (rb.m_threw ? "THREW exception: " : "didn't throw!") << Color::Cyan << rb.m_exception <<
06825         "\n";
06826     } else {
06827         s << (rb.m_threw ? "THREW exception: " : (!rb.m_failed ? "is correct!\n" : "is NOT
06828         correct!\n"));
06829     }
06830     if (rb.m_threw)
06831         s << rb.m_exception << "\n";
06832     else
06833         s << " values: " << assertString(rb.m_at) << "( " << rb.m_decomp << " )\n";
06834 }
06835 // clang-format on
06836 }
06837 } // namespace doctest
06838
06839 #endif // DOCTEST_CONFIG_DISABLE
06840
06841 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
06842
06843 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
06844
06845 #ifndef DOCTEST_CONFIG_DISABLE
06846
06847 namespace doctest {
06848
06849 using detail::g_cs;
06850
06851 Whitespace::Whitespace(int nr)
06852     : nrSpaces(nr) {}
06853
06854 std::ostream &operator<<(std::ostream &out, const Whitespace &ws) {
06855     if (ws.nrSpaces != 0)
06856         out << std::setw(ws.nrSpaces) << ' ';
06857     return out;
06858 }
06859
06860 ConsoleReporter::ConsoleReporter(const ContextOptions &co)
06861     : s(&co.cout), opt(co) {}
06862
06863 ConsoleReporter::ConsoleReporter(const ContextOptions &co, std::ostream &ostr)
06864     : s(ostr), opt(co) {}
06865

```

```

06862 void ConsoleReporter::separator_to_stream() {
06863     s « Color::Yellow
06864     « "=====
06865     "\n";
06866 }
06867
06868 const char *ConsoleReporter::getSuccessOrFailString(bool success, assertType::Enum at, const char
    *success_str) {
06869     if (success)
06870         return success_str;
06871     return failureString(at);
06872 }
06873
06874 Color::Enum ConsoleReporter::getSuccessOrFailColor(bool success, assertType::Enum at) {
06875     return success ? Color::BrightGreen : (at & assertType::is_warn) ? Color::Yellow : Color::Red;
06876 }
06877
06878 void ConsoleReporter::successOrFailColoredStringToStream(bool success, assertType::Enum at, const char
    *success_str) {
06879     s « getSuccessOrFailColor(success, at) « getSuccessOrFailString(success, at, success_str) « ": ";
06880 }
06881
06882 void ConsoleReporter::log_contexts() {
06883     const int num_contexts = get_num_active_contexts();
06884     if (num_contexts) {
06885         auto contexts = get_active_contexts();
06886
06887         s « Color::None « " logged: ";
06888         for (int i = 0; i < num_contexts; ++i) {
06889             s « (i == 0 ? "" : " ");
06890             contexts[i]->stringify(&s);
06891             s « "\n";
06892         }
06893     }
06894
06895     s « "\n";
06896 }
06897
06898 // this was requested to be made virtual so users could override it
06899 void ConsoleReporter::file_line_to_stream(const char *file, int line, const char *tail) {
06900     s « Color::LightGrey « skipPathFromFilename(file) « (opt.gnu_file_line ? ":" : "(")
06901     « (opt.no_line_numbers ? 0 : line) // 0 or the real num depending on the option
06902     « (opt.gnu_file_line ? ":" : ")") « tail;
06903 }
06904
06905 void ConsoleReporter::logTestStart() {
06906     if (hasLoggedCurrentTestStart)
06907         return;
06908
06909     separator_to_stream();
06910     file_line_to_stream(tc->m_file.c_str(), static_cast<int>(tc->m_line), "\n");
06911     if (tc->m_description)
06912         s « Color::Yellow « "DESCRIPTION: " « Color::None « tc->m_description « "\n";
06913     if (tc->m_test_suite && tc->m_test_suite[0] != '\0')
06914         s « Color::Yellow « "TEST SUITE: " « Color::None « tc->m_test_suite « "\n";
06915     if (strcmp(tc->m_name, " Scenario: ", 11) != 0)
06916         s « Color::Yellow « "TEST CASE: ";
06917     s « Color::None « tc->m_name « "\n";
06918
06919     for (size_t i = 0; i < currentSubcaseLevel; ++i) {
06920         if (subcasesStack[i].m_name[0] != '\0')
06921             s « " " « subcasesStack[i].m_name « "\n";
06922     }
06923
06924     if (currentSubcaseLevel != subcasesStack.size()) {
06925         s « Color::Yellow « "\nDEEPEST SUBCASE STACK REACHED (DIFFERENT FROM THE CURRENT ONE):\n" «
Color::None;
06926         for (size_t i = 0; i < subcasesStack.size(); ++i) {
06927             if (subcasesStack[i].m_name[0] != '\0')
06928                 s « " " « subcasesStack[i].m_name « "\n";
06929         }
06930     }
06931
06932     s « "\n";
06933     hasLoggedCurrentTestStart = true;
06934 }
06935
06936 void ConsoleReporter::printVersion() {
06937     if (opt.no_version == false)
06938         s « Color::Cyan « "[doctest] " « Color::None « "doctest version is \" « DOCTEST_VERSION_STR «
        "\"\n";
06939 }
06940
06941 void ConsoleReporter::printIntro() {
06942     if (opt.no_intro == false) {
06943         printVersion();
06944     }

```

```

06945         s << Color::Cyan << "[doctest] " << Color::None
06946         << "run with \"--\" DOCTEST_OPTIONS_PREFIX_DISPLAY \"help\" for options\n";
06947     }
06948 }
06949
06950 void ConsoleReporter::printHelp() {
06951     const int sizePrefixDisplay = static_cast<int>(strlen(DOCTEST_OPTIONS_PREFIX_DISPLAY));
06952     printVersion();
06953     // clang-format off
06954     s << Color::Cyan << "[doctest]\n" << Color::None;
06955     s << Color::Cyan << "[doctest] " << Color::None;
06956     s << "boolean values: \"1/on/yes/true\" or \"0/off/no/false\"\n";
06957     s << Color::Cyan << "[doctest] " << Color::None;
06958     s << "filter values: \"str1,str2,str3\" (comma separated strings)\n";
06959     s << Color::Cyan << "[doctest]\n" << Color::None;
06960     s << Color::Cyan << "[doctest] " << Color::None;
06961     s << "filters use wildcards for matching strings\n";
06962     s << Color::Cyan << "[doctest] " << Color::None;
06963     s << "something passes a filter if any of the strings in a filter matches\n";
06964 #ifndef DOCTEST_CONFIG_NO_UNPREFIXED_OPTIONS
06965     s << Color::Cyan << "[doctest]\n" << Color::None;
06966     s << Color::Cyan << "[doctest] " << Color::None;
06967     s << "ALL FLAGS, OPTIONS AND FILTERS ALSO AVAILABLE WITH A \" DOCTEST_CONFIG_OPTIONS_PREFIX \"\n"
PREFIX!!!\n";
06968 #endif
06969     s << Color::Cyan << "[doctest]\n" << Color::None;
06970     s << Color::Cyan << "[doctest] " << Color::None;
06971     s << "Query flags - the program quits after them. Available:\n\n";
06972     s << " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "?", --" DOCTEST_OPTIONS_PREFIX_DISPLAY "help, -"
DOCTEST_OPTIONS_PREFIX_DISPLAY "h
"
06973     << Whitespace(sizePrefixDisplay * 0) << "prints this message\n";
06974     s << " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "v, --" DOCTEST_OPTIONS_PREFIX_DISPLAY "version
"
06975     << Whitespace(sizePrefixDisplay * 1) << "prints the version\n";
06976     s << " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "c, --" DOCTEST_OPTIONS_PREFIX_DISPLAY "count
"
06977     << Whitespace(sizePrefixDisplay * 1) << "prints the number of matching tests\n";
06978     s << " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "ltc, --" DOCTEST_OPTIONS_PREFIX_DISPLAY "list-test-cases
"
06979     << Whitespace(sizePrefixDisplay * 1) << "lists all matching tests by name\n";
06980     s << " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "lts, --" DOCTEST_OPTIONS_PREFIX_DISPLAY "list-test-suites
"
06981     << Whitespace(sizePrefixDisplay * 1) << "lists all matching test suites\n";
06982     s << " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "lr, --" DOCTEST_OPTIONS_PREFIX_DISPLAY "list-reporters
"
06983     << Whitespace(sizePrefixDisplay * 1) << "lists all registered reporters\n\n";
06984     // ===== << 79
06985     s << Color::Cyan << "[doctest] " << Color::None;
06986     s << "The available <int>/<string> options/filters are:\n\n";
06987     s << " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "tc, --" DOCTEST_OPTIONS_PREFIX_DISPLAY
"test-case=<filters>
"
06988     << Whitespace(sizePrefixDisplay * 1) << "filters tests by their name\n";
06989     s << " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "tce, --" DOCTEST_OPTIONS_PREFIX_DISPLAY
"test-case-exclude=<filters>
"
06990     << Whitespace(sizePrefixDisplay * 1) << "filters OUT tests by their name\n";
06991     s << " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "sf, --" DOCTEST_OPTIONS_PREFIX_DISPLAY
"source-file=<filters>
"
06992     << Whitespace(sizePrefixDisplay * 1) << "filters tests by their file\n";
06993     s << " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "sfe, --" DOCTEST_OPTIONS_PREFIX_DISPLAY
"source-file-exclude=<filters>
"
06994     << Whitespace(sizePrefixDisplay * 1) << "filters OUT tests by their file\n";
06995     s << " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "ts, --" DOCTEST_OPTIONS_PREFIX_DISPLAY
"test-suite=<filters>
"
06996     << Whitespace(sizePrefixDisplay * 1) << "filters tests by their test suite\n";
06997     s << " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "tse, --" DOCTEST_OPTIONS_PREFIX_DISPLAY
"test-suite-exclude=<filters>
"
06998     << Whitespace(sizePrefixDisplay * 1) << "filters OUT tests by their test suite\n";
06999     s << " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "sc, --" DOCTEST_OPTIONS_PREFIX_DISPLAY
"subcase=<filters>
"
07000     << Whitespace(sizePrefixDisplay * 1) << "filters subcases by their name\n";
07001     s << " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "sce, --" DOCTEST_OPTIONS_PREFIX_DISPLAY
"subcase-exclude=<filters>
"
07002     << Whitespace(sizePrefixDisplay * 1) << "filters OUT subcases by their name\n";
07003     s << " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "r, --" DOCTEST_OPTIONS_PREFIX_DISPLAY
"reporters=<filters>
"
07004     << Whitespace(sizePrefixDisplay * 1) << "reporters to use (console is default)\n";
07005     s << " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "o, --" DOCTEST_OPTIONS_PREFIX_DISPLAY "out=<string>
"
07006     << Whitespace(sizePrefixDisplay * 1) << "output filename\n";
07007     s << " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "ob, --" DOCTEST_OPTIONS_PREFIX_DISPLAY
"order-by=<string>
"
07008     << Whitespace(sizePrefixDisplay * 1) << "how the tests should be ordered\n";
07009     s << Whitespace(sizePrefixDisplay * 3) << " <string> -
[file/suite/name/rand/none]\n";
07010     s << " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "rs, --" DOCTEST_OPTIONS_PREFIX_DISPLAY "rand-seed=<int>
"
07011     << Whitespace(sizePrefixDisplay * 1) << "seed for random ordering\n";

```



```

07012     s < " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "f, --" DOCTEST_OPTIONS_PREFIX_DISPLAY "first=<int>"
07013     « Whitespace(sizePrefixDisplay * 1) « "the first test passing the filters to\n";
07014     s < Whitespace(sizePrefixDisplay * 3) « "                                execute - for
range-based execution\n";
07015     s < " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "l, --" DOCTEST_OPTIONS_PREFIX_DISPLAY "last=<int>"
07016     « Whitespace(sizePrefixDisplay * 1) « "the last test passing the filters to\n";
07017     s < Whitespace(sizePrefixDisplay * 3) « "                                execute - for
range-based execution\n";
07018     s < " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "aa, --" DOCTEST_OPTIONS_PREFIX_DISPLAY
"abort-after=<int>"
07019     « Whitespace(sizePrefixDisplay * 1) « "stop after <int> failed assertions\n";
07020     s < " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "scfl,--" DOCTEST_OPTIONS_PREFIX_DISPLAY
"subcase-filter-levels=<int>"
07021     « Whitespace(sizePrefixDisplay * 1) « "apply filters for the first <int> levels\n";
07022     s < Color::Cyan « "\n[doctest] " « Color::None;
07023     s < "Bool options - can be used like flags and true is assumed. Available:\n\n";
07024     s < " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "s, --" DOCTEST_OPTIONS_PREFIX_DISPLAY "success=<bool>"
07025     « Whitespace(sizePrefixDisplay * 1) « "include successful assertions in output\n";
07026     s < " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "cs, --" DOCTEST_OPTIONS_PREFIX_DISPLAY
"case-sensitive=<bool>"
07027     « Whitespace(sizePrefixDisplay * 1) « "filters being treated as case sensitive\n";
07028     s < " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "e, --" DOCTEST_OPTIONS_PREFIX_DISPLAY "exit=<bool>"
07029     « Whitespace(sizePrefixDisplay * 1) « "exits after the tests finish\n";
07030     s < " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "d, --" DOCTEST_OPTIONS_PREFIX_DISPLAY "duration=<bool>"
07031     « Whitespace(sizePrefixDisplay * 1) « "prints the time duration of each test\n";
07032     s < " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "m, --" DOCTEST_OPTIONS_PREFIX_DISPLAY "minimal=<bool>"
07033     « Whitespace(sizePrefixDisplay * 1) « "minimal console output (only failures)\n";
07034     s < " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "q, --" DOCTEST_OPTIONS_PREFIX_DISPLAY "quiet=<bool>"
07035     « Whitespace(sizePrefixDisplay * 1) « "no console output\n";
07036     s < " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "nt, --" DOCTEST_OPTIONS_PREFIX_DISPLAY "no-throw=<bool>"
07037     « Whitespace(sizePrefixDisplay * 1) « "skips exceptions-related assert checks\n";
07038     s < " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "ne, --" DOCTEST_OPTIONS_PREFIX_DISPLAY
"no-exitcode=<bool>"
07039     « Whitespace(sizePrefixDisplay * 1) « "returns (or exits) always with success\n";
07040     s < " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "nr, --" DOCTEST_OPTIONS_PREFIX_DISPLAY "no-run=<bool>"
07041     « Whitespace(sizePrefixDisplay * 1) « "skips all runtime doctest operations\n";
07042     s < " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "ni, --" DOCTEST_OPTIONS_PREFIX_DISPLAY "no-intro=<bool>"
07043     « Whitespace(sizePrefixDisplay * 1) « "omit the framework intro in the output\n";
07044     s < " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "nv, --" DOCTEST_OPTIONS_PREFIX_DISPLAY
"no-version=<bool>"
07045     « Whitespace(sizePrefixDisplay * 1) « "omit the framework version in the output\n";
07046     s < " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "nc, --" DOCTEST_OPTIONS_PREFIX_DISPLAY "no-colors=<bool>"
07047     « Whitespace(sizePrefixDisplay * 1) « "disables colors in output\n";
07048     s < " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "fc, --" DOCTEST_OPTIONS_PREFIX_DISPLAY
"force-colors=<bool>"
07049     « Whitespace(sizePrefixDisplay * 1) « "use colors even when not in a tty\n";
07050     s < " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "nb, --" DOCTEST_OPTIONS_PREFIX_DISPLAY "no-breaks=<bool>"
07051     « Whitespace(sizePrefixDisplay * 1) « "disables breakpoints in debuggers\n";
07052     s < " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "ns, --" DOCTEST_OPTIONS_PREFIX_DISPLAY "no-skip=<bool>"
07053     « Whitespace(sizePrefixDisplay * 1) « "don't skip test cases marked as skip\n";
07054     s < " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "gfl, --" DOCTEST_OPTIONS_PREFIX_DISPLAY
"gnu-file-line=<bool>"
07055     « Whitespace(sizePrefixDisplay * 1) « ":n: vs (n): for line numbers in output\n";
07056     s < " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "npf, --" DOCTEST_OPTIONS_PREFIX_DISPLAY
"no-path-filenames=<bool>"
07057     « Whitespace(sizePrefixDisplay * 1) « "only filenames and no paths in output\n";
07058     s < " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "sfp, --" DOCTEST_OPTIONS_PREFIX_DISPLAY
"strip-file-prefixes=<p1:p2>"
07059     « Whitespace(sizePrefixDisplay * 1) « "whenever file paths start with this prefix, remove it
from the output\n";
07060     s < " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "nln, --" DOCTEST_OPTIONS_PREFIX_DISPLAY
"no-line-numbers=<bool>"
07061     « Whitespace(sizePrefixDisplay * 1) « "0 instead of real line numbers in output\n";
07062     // ===== « 79
07063     // clang-format on
07064
07065     s < Color::Cyan « "\n[doctest] " « Color::None;
07066     s < "for more information visit the project documentation\n\n";
07067 }
07068
07069 void ConsoleReporter::printRegisteredReporters() {
07070     printVersion();
07071     auto printReporters = [this](const detail::reporterMap &reporters, const char *type) {
07072         if (!reporters.empty()) {

```

```

07073         s « Color::Cyan « "[doctest] " « Color::None « "listing all registered " « type « "\n";
07074         for (auto &curr: reporters)
07075             s « "priority: " « std::setw(5) « curr.first.first « " name: " « curr.first.second «
"\n";
07076     }
07077 };
07078 printReporters(detail::getListeners(), "listeners");
07079 printReporters(detail::getReporters(), "reporters");
07080 }
07081
07082 void ConsoleReporter::report_query(const QueryData &in) {
07083     if (opt.version) {
07084         printVersion();
07085     } else if (opt.help) {
07086         printHelp();
07087     } else if (opt.list_reporters) {
07088         printRegisteredReporters();
07089     } else if (opt.count || opt.list_test_cases) {
07090         if (opt.list_test_cases) {
07091             s « Color::Cyan « "[doctest] " « Color::None « "listing all test case names\n";
07092             separator_to_stream();
07093         }
07094
07095         for (unsigned i = 0; i < in.num_data; ++i)
07096             s « Color::None « in.data[i]->m_name « "\n";
07097
07098         separator_to_stream();
07099
07100         s « Color::Cyan « "[doctest] " « Color::None
07101             « "unskipped test cases passing the current filters: " « g_cs->numTestCasesPassingFilters «
"\n";
07102     } else if (opt.list_test_suites) {
07103         s « Color::Cyan « "[doctest] " « Color::None « "listing all test suites\n";
07104         separator_to_stream();
07105
07106         for (unsigned i = 0; i < in.num_data; ++i)
07107             s « Color::None « in.data[i]->m_test_suite « "\n";
07108
07109         separator_to_stream();
07110
07111         s « Color::Cyan « "[doctest] " « Color::None
07112             « "unskipped test cases passing the current filters: " « g_cs->numTestCasesPassingFilters «
"\n";
07113     }
07114     s « Color::Cyan « "[doctest] " « Color::None
07115         « "test suites with unskipped test cases passing the current filters: " «
g_cs->numTestSuitesPassingFilters
07116         « "\n";
07117 }
07118 }
07119
07120 void ConsoleReporter::test_run_start() {
07121     if (!opt.minimal)
07122         printIntro();
07123 }
07124
07125 void ConsoleReporter::test_run_end(const TestRunStats &p) {
07126     if (opt.minimal && p.numTestCasesFailed == 0)
07127         return;
07128
07129     separator_to_stream();
07130     s « std::dec;
07131
07132     auto totwidth = static_cast<int>(std::ceil(
07133         log10(static_cast<double>(std::max(p.numTestCasesPassingFilters,
static_cast<unsigned>(p.numAsserts))) + 1)
07134     ));
07135     auto passwidth = static_cast<int>(std::ceil(log10(
07136         static_cast<double>(std::max(
07137             p.numTestCasesPassingFilters - p.numTestCasesFailed,
07138             static_cast<unsigned>(p.numAsserts - p.numAssertsFailed)
07139         )) +
07140         1
07141     )));
07142     auto failwidth = static_cast<int>(std::ceil(
07143         log10(static_cast<double>(std::max(p.numTestCasesFailed,
static_cast<unsigned>(p.numAssertsFailed))) + 1)
07144     ));
07145     const bool anythingFailed = p.numTestCasesFailed > 0 || p.numAssertsFailed > 0;
07146     s « Color::Cyan « "[doctest] " « Color::None « "test cases: " « std::setw(totwidth)
07147         « p.numTestCasesPassingFilters « " | "
07148         « ((p.numTestCasesPassingFilters == 0 || anythingFailed) ? Color::None : Color::Green) «
std::setw(passwidth)
07149         « p.numTestCasesPassingFilters - p.numTestCasesFailed « " passed" « Color::None « " | "
07150         « (p.numTestCasesFailed > 0 ? Color::Red : Color::None) « std::setw(failwidth) «
p.numTestCasesFailed
07151         « " failed" « Color::None « " |";

```

```

07152     if (opt.no_skipped_summary == false) {
07153         const unsigned int numSkipped = p.numTestCases - p.numTestCasesPassingFilters;
07154         s << " " << (numSkipped == 0 ? Color::None : Color::Yellow) << numSkipped << " skipped" <<
Color::None;
07155     }
07156     s << "\n";
07157     s << Color::Cyan << "[doctest] " << Color::None << "assertions: " << std::setw(totwidth) << p.numAsserts
<< " | "
07158     << ((p.numAsserts == 0 || anythingFailed) ? Color::None : Color::Green) << std::setw(passwidth)
07159     << (p.numAsserts - p.numAssertsFailed) << " passed" << Color::None << " | "
07160     << (p.numAssertsFailed > 0 ? Color::Red : Color::None) << std::setw(failwidth) <<
p.numAssertsFailed << " failed"
07161     << Color::None << " |\n";
07162     s << Color::Cyan << "[doctest] " << Color::None
07163     << "Status: " << (p.numTestCasesFailed > 0 ? Color::Red : Color::Green)
07164     << ((p.numTestCasesFailed > 0) ? "FAILURE!" : "SUCCESS!") << Color::None << std::endl;
07165 }
07166
07167 void ConsoleReporter::test_case_start(const TestCaseData &in) {
07168     DOCTEST_LOCK_MUTEX(mutex)
07169     hasLoggedCurrentTestStart = false;
07170     tc = &in;
07171     subcasesStack.clear();
07172     currentSubcaseLevel = 0;
07173 }
07174
07175 void ConsoleReporter::test_case_reenter(const TestCaseData &) {
07176     DOCTEST_LOCK_MUTEX(mutex)
07177     subcasesStack.clear();
07178 }
07179
07180 void ConsoleReporter::test_case_end(const CurrentTestCaseStats &st) {
07181     DOCTEST_LOCK_MUTEX(mutex)
07182     if (tc->m_no_output)
07183         return;
07184
07185     // log the preamble of the test case only if there is something
07186     // else to print - something other than that an assert has failed
07187     if (opt.duration ||
07188         (st.failure_flags && st.failure_flags !=
static_cast<int>(TestCaseFailureReason::AssertFailure)))
07189         logTestStart();
07190
07191     if (opt.duration)
07192         s << Color::None << std::setprecision(6) << std::fixed << st.seconds << " s: " << tc->m_name << "\n";
07193
07194     if (st.failure_flags & TestCaseFailureReason::Timeout)
07195         s << Color::Red << "Test case exceeded time limit of " << std::setprecision(6) << std::fixed <<
tc->m_timeout
07196         << "!\n";
07197
07198     if (st.failure_flags & TestCaseFailureReason::ShouldHaveFailedButDidnt) {
07199         s << Color::Red << "Should have failed but didn't! Marking it as failed!\n";
07200     } else if (st.failure_flags & TestCaseFailureReason::ShouldHaveFailedAndDid) {
07201         s << Color::Yellow << "Failed as expected so marking it as not failed\n";
07202     } else if (st.failure_flags & TestCaseFailureReason::CouldHaveFailedAndDid) {
07203         s << Color::Yellow << "Allowed to fail so marking it as not failed\n";
07204     } else if (st.failure_flags & TestCaseFailureReason::DidntFailExactlyNumTimes) {
07205         s << Color::Red << "Didn't fail exactly " << tc->m_expected_failures << " times so marking it as
failed!\n";
07206     } else if (st.failure_flags & TestCaseFailureReason::FailedExactlyNumTimes) {
07207         s << Color::Yellow << "Failed exactly " << tc->m_expected_failures
07208         << " times as expected so marking it as not failed!\n";
07209     }
07210     if (st.failure_flags & TestCaseFailureReason::TooManyFailedAsserts) {
07211         s << Color::Red << "Aborting - too many failed asserts!\n";
07212     }
07213     s << Color::None;
07214 }
07215
07216 void ConsoleReporter::test_case_exception(const TestCaseException &e) {
07217     DOCTEST_LOCK_MUTEX(mutex)
07218     if (tc->m_no_output)
07219         return;
07220
07221     logTestStart();
07222
07223     file_line_to_stream(tc->m_file.c_str(), static_cast<int>(tc->m_line), " ");
07224     successOrFailColoredStringToStream(false, e.is_crash ? assertType::is_require :
assertType::is_check);
07225     s << Color::Red << (e.is_crash ? "test case CRASHED: " : "test case THREW exception: ");
07226     s << Color::Cyan << e.error_string << "\n";
07227
07228     const int num_stringified_contexts = get_num_stringified_contexts();
07229     if (num_stringified_contexts) {
07230         auto stringified_contexts = get_stringified_contexts();
07231         s << Color::None << " logged: ";

```

```

07232         for (int i = num_stringified_contexts; i > 0; --i) {
07233             s << (i == num_stringified_contexts ? "" : " ") << stringified_contexts[i - 1] <<
07234                 "\n";
07235         }
07236         s << "\n" << Color::None;
07237     }
07238
07239     void ConsoleReporter::subcase_start(const SubcaseSignature &subc) {
07240         DOCTEST_LOCK_MUTEX(mutex)
07241         subcasesStack.push_back(subc);
07242         ++currentSubcaseLevel;
07243         hasLoggedCurrentTestStart = false;
07244     }
07245
07246     void ConsoleReporter::subcase_end() {
07247         DOCTEST_LOCK_MUTEX(mutex)
07248         --currentSubcaseLevel;
07249         hasLoggedCurrentTestStart = false;
07250     }
07251
07252     void ConsoleReporter::log_assert(const AssertData &rb) {
07253         if (!rb.m_failed && !opt.success) || tc->m_no_output)
07254             return;
07255
07256         DOCTEST_LOCK_MUTEX(mutex)
07257
07258         logTestStart();
07259
07260         file_line_to_stream(rb.m_file, rb.m_line, " ");
07261         successOrFailColoredStringToStream(!rb.m_failed, rb.m_at);
07262
07263         fulltext_log_assert_to_stream(s, rb);
07264
07265         log_contexts();
07266     }
07267
07268     void ConsoleReporter::log_message(const MessageData &mb) {
07269         if (tc->m_no_output)
07270             return;
07271
07272         DOCTEST_LOCK_MUTEX(mutex)
07273
07274         logTestStart();
07275
07276         file_line_to_stream(mb.m_file, mb.m_line, " ");
07277         s << getSuccessOrFailColor(false, mb.m_severity)
07278           << getSuccessOrFailString(mb.m_severity & assertType::is_warn, mb.m_severity, "MESSAGE") << ": ";
07279         s << Color::None << mb.m_string << "\n";
07280         log_contexts();
07281     }
07282
07283     void ConsoleReporter::test_case_skipped(const TestCaseData &) {}
07284
07285 } // namespace doctest
07286
07287 #endif // DOCTEST_CONFIG_DISABLE
07288
07289 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
07290
07291 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
07292
07293 #ifndef DOCTEST_CONFIG_DISABLE
07294
07295 namespace doctest {
07296 namespace detail {
07297
07298 #ifdef DOCTEST_PLATFORM_WINDOWS
07299 #define DOCTEST_OUTPUT_DEBUG_STRING(text) ::OutputDebugStringA(text)
07300 #endif // Platform
07301
07302 #ifdef DOCTEST_PLATFORM_WINDOWS
07303
07304 DOCTEST_THREAD_LOCAL std::ostringstream DebugOutputWindowReporter::oss;
07305
07306 DebugOutputWindowReporter::DebugOutputWindowReporter(const ContextOptions &co)
07307     : ConsoleReporter(co, oss) {}
07308
07309 #define DOCTEST_DEBUG_OUTPUT_REPORTER_OVERRIDE(func, type, arg)
07310 \
07311 \
07312 \
07313 \
07314     void DebugOutputWindowReporter::func(type arg) {

```

```

07314 \
07315 \
07316 \
07317 \
07318 \
07319 \
07320 \
07321 \
07322 DOCTEST_DEBUG_OUTPUT_REPORTER_OVERRIDE(test_run_start, DOCTEST_EMPTY, DOCTEST_EMPTY)
07323 DOCTEST_DEBUG_OUTPUT_REPORTER_OVERRIDE(test_run_end, const TestRunStats &, in)
07324 DOCTEST_DEBUG_OUTPUT_REPORTER_OVERRIDE(test_case_start, const TestCaseData &, in)
07325 DOCTEST_DEBUG_OUTPUT_REPORTER_OVERRIDE(test_case_reenter, const TestCaseData &, in)
07326 DOCTEST_DEBUG_OUTPUT_REPORTER_OVERRIDE(test_case_end, const CurrentTestCaseStats &, in)
07327 DOCTEST_DEBUG_OUTPUT_REPORTER_OVERRIDE(test_case_exception, const TestCaseException &, in)
07328 DOCTEST_DEBUG_OUTPUT_REPORTER_OVERRIDE(subcase_start, const SubcaseSignature &, in)
07329 DOCTEST_DEBUG_OUTPUT_REPORTER_OVERRIDE(subcase_end, DOCTEST_EMPTY, DOCTEST_EMPTY)
07330 DOCTEST_DEBUG_OUTPUT_REPORTER_OVERRIDE(log_assert, const AssertData &, in)
07331 DOCTEST_DEBUG_OUTPUT_REPORTER_OVERRIDE(log_message, const MessageData &, in)
07332 DOCTEST_DEBUG_OUTPUT_REPORTER_OVERRIDE(test_case_skipped, const TestCaseData &, in)
07333
07334 #endif // DOCTEST_PLATFORM_WINDOWS
07335
07336 } // namespace detail
07337 } // namespace doctest
07338
07339 #endif // DOCTEST_CONFIG_DISABLE
07340
07341 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
07342
07343 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
07344
07345 #ifndef DOCTEST_CONFIG_DISABLE
07346
07347 namespace doctest {
07348
07349 std::string JUnitReporter::JUnitTestCaseData::GetCurrentTimestamp() {
07350     // Beware, this is not reentrant because of backward compatibility issues
07351     // Also, UTC only, again because of backward compatibility (%z is C++11)
07352     time_t rawtime{};
07353     static_cast<void>(std::time(&rawtime));
07354     const auto timeStampSize = sizeof("2017-01-16T17:06:45Z");
07355
07356     std::tm timeInfo;
07357     #if defined(DOCTEST_PLATFORM_WINDOWS)
07358     gmtime_s(&timeInfo, &rawtime);
07359     #elif defined(__STDC_LIB_EXT1__)
07360     gmtime_s(&rawtime, &timeInfo);
07361     #else // DOCTEST_PLATFORM_WINDOWS
07362     gmtime_r(&rawtime, &timeInfo);
07363     #endif // DOCTEST_PLATFORM_WINDOWS
07364
07365     char timeStamp[timeStampSize];
07366     const char *const fmt = "%Y-%m-%dT%H:%M:%SZ";
07367
07368     static_cast<void>(std::strftime(timeStamp, timeStampSize, fmt, &timeInfo));
07369     return std::string(timeStamp);
07370 }
07371
07372 JUnitReporter::JUnitTestCaseData::JUnitTestMessage::JUnitTestMessage(
07373     const std::string &_message, const std::string &_type, const std::string &_details
07374 )
07375     : message(_message), type(_type), details(_details) {}
07376
07377 JUnitReporter::JUnitTestCaseData::JUnitTestMessage::JUnitTestMessage(
07378     const std::string &_message, const std::string &_details
07379 )
07380     : message(_message), type(), details(_details) {}
07381
07382 JUnitReporter::JUnitTestCaseData::JUnitTestCase::JUnitTestCase(const std::string &_classname, const
std::string &_name)
07383     : classname(_classname), name(_name), time(0), failures(), errors() {}
07384
07385 void JUnitReporter::JUnitTestCaseData::add(const std::string &classname, const std::string &name) {
07386     testcases.emplace_back(classname, name);
07387 }
07388
07389 void JUnitReporter::JUnitTestCaseData::appendSubcaseNamesToLastTestcase(std::vector<String> nameStack)
{
07390     for (auto &curr: nameStack)
07391         if (curr.size())

```

```

07392         testcases.back().name += std::string("/") + curr.c_str();
07393     }
07394
07395     void JUnitReporter::JUnitTestCaseData::addTime(double time) {
07396         if (time < 1e-4)
07397             time = 0;
07398         testcases.back().time = time;
07399         totalSeconds += time;
07400     }
07401
07402     void JUnitReporter::JUnitTestCaseData::addFailure(
07403         const std::string &message, const std::string &type, const std::string &details
07404     ) {
07405         testcases.back().failures.emplace_back(message, type, details);
07406         ++totalFailures;
07407     }
07408
07409     void JUnitReporter::JUnitTestCaseData::addError(const std::string &message, const std::string
07410         &details) {
07411         testcases.back().errors.emplace_back(message, details);
07412         ++totalErrors;
07413     }
07414
07415     JUnitReporter::JUnitReporter(const ContextOptions &co)
07416         : xml(*co.cout), opt(co) {}
07417
07418     unsigned JUnitReporter::line(unsigned l) const {
07419         return opt.no_line_numbers ? 0 : l;
07420     }
07421
07422     void JUnitReporter::report_query(const QueryData &) {
07423         xml.writeDeclaration();
07424     }
07425
07426     void JUnitReporter::test_run_start() {
07427         xml.writeDeclaration();
07428     }
07429
07430     void JUnitReporter::test_run_end(const TestRunStats &p) {
07431         // remove .exe extension - mainly to have the same output on UNIX and Windows
07432         // NOLINTNEXTLINE(misc-const-correctness)
07433         std::string binary_name = skipPathFromFilename(opt.binary_name.c_str());
07434         #ifdef DOCTEST_PLATFORM_WINDOWS
07435         if (binary_name.rfind(".exe") != std::string::npos)
07436             binary_name = binary_name.substr(0, binary_name.length() - 4);
07437         #endif // DOCTEST_PLATFORM_WINDOWS
07438
07439         xml.startElement("testsuites");
07440         xml.startElement("testsuite")
07441             .writeAttribute("name", binary_name)
07442             .writeAttribute("errors", testCaseData.totalErrors)
07443             .writeAttribute("failures", testCaseData.totalFailures)
07444             .writeAttribute("tests", p.numAsserts);
07445         if (opt.no_time_in_output == false) {
07446             xml.writeAttribute("time", testCaseData.totalSeconds);
07447             xml.writeAttribute("timestamp", JUnitTestCaseData::getCurrentTimestamp());
07448         }
07449         if (opt.no_version == false)
07450             xml.writeAttribute("doctest_version", DOCTEST_VERSION_STR);
07451
07452         for (const auto &testCase: testCaseData.testcases) {
07453             xml.startElement("testcase")
07454                 .writeAttribute("classname", testCase.classname)
07455                 .writeAttribute("name", testCase.name);
07456             if (opt.no_time_in_output == false)
07457                 xml.writeAttribute("time", testCase.time);
07458             // This is not ideal, but it should be enough to mimic gtest's junit output.
07459             xml.writeAttribute("status", "run");
07460
07461             for (const auto &failure: testCase.failures) {
07462                 xml.scopedElement("failure")
07463                     .writeAttribute("message", failure.message)
07464                     .writeAttribute("type", failure.type)
07465                     .writeText(failure.details, false);
07466             }
07467
07468             for (const auto &error: testCase.errors) {
07469                 xml.scopedElement("error").writeAttribute("message",
07470                     error.message).writeText(error.details);
07471             }
07472             xml.endElement();
07473         }
07474         xml.endElement();
07475     }
07476

```

```

07477 void JUnitReporter::test_case_start(const TestCaseData &in) {
07478     DOCTEST_LOCK_MUTEX(mutex)
07479     testCaseData.add(skipPathFromFilename(in.m_file.c_str(), in.m_name);
07480     timer.start();
07481     tc = &in;
07482 }
07483
07484 void JUnitReporter::test_case_reenter(const TestCaseData &in) {
07485     DOCTEST_LOCK_MUTEX(mutex)
07486     testCaseData.addTime(timer.getElapsedSeconds());
07487     testCaseData.appendSubcaseNamesToLastTestcase(deepestSubcaseStackNames);
07488     deepestSubcaseStackNames.clear();
07489
07490     timer.start();
07491     testCaseData.add(skipPathFromFilename(in.m_file.c_str(), in.m_name);
07492     tc = &in;
07493 }
07494
07495 void JUnitReporter::test_case_end(const CurrentTestCaseStats &st) {
07496     DOCTEST_LOCK_MUTEX(mutex)
07497     testCaseData.addTime(timer.getElapsedSeconds());
07498     testCaseData.appendSubcaseNamesToLastTestcase(deepestSubcaseStackNames);
07499     deepestSubcaseStackNames.clear();
07500
07501     if (st.failure_flags & TestCaseFailureReason::Timeout) {
07502         auto *stream = detail::tlssPush();
07503         tc->m_timeout;
07504         testCaseData.addError("timeout", detail::tlssPop().c_str());
07505     }
07506
07507     if (st.failure_flags & TestCaseFailureReason::ShouldHaveFailedButDidnt) {
07508         testCaseData.addError("should_fail", "Should have failed, but didn't");
07509     } else if (st.failure_flags & TestCaseFailureReason::DidntFailExactlyNumTimes) {
07510         auto *stream = detail::tlssPush();
07511         *stream << "Should have failed exactly " << tc->m_expected_failures << " times, but didn't";
07512         testCaseData.addError("should_fail", detail::tlssPop().c_str());
07513     }
07514
07515     if (st.failure_flags & TestCaseFailureReason::TooManyFailedAsserts) {
07516         testCaseData.addError("abort_after", "Too many failed asserts");
07517     }
07518
07519     tc = nullptr;
07520 }
07521
07522 void JUnitReporter::test_case_exception(const TestCaseException &e) {
07523     DOCTEST_LOCK_MUTEX(mutex)
07524     testCaseData.addError("exception", e.error_string.c_str());
07525 }
07526
07527 void JUnitReporter::subcase_start(const SubcaseSignature &in) {
07528     DOCTEST_LOCK_MUTEX(mutex)
07529     deepestSubcaseStackNames.push_back(in.m_name);
07530 }
07531
07532 void JUnitReporter::subcase_end() {}
07533
07534 void JUnitReporter::log_assert(const AssertData &rb) {
07535     if (!rb.m_failed) // report only failures & ignore the `success` option
07536         return;
07537
07538     DOCTEST_LOCK_MUTEX(mutex)
07539
07540     std::ostringstream os;
07541     os << skipPathFromFilename(rb.m_file) << (opt.gnu_file_line ? ":" : "(") << line(rb.m_line)
07542     << (opt.gnu_file_line ? ":" : ":") << std::endl;
07543
07544     fulltext_log_assert_to_stream(os, rb);
07545     log_contexts(os);
07546     testCaseData.addFailure(rb.m_decomp.c_str(), assertString(rb.m_at), os.str());
07547 }
07548
07549 void JUnitReporter::log_message(const MessageData &mb) {
07550     if (mb.m_severity & assertType::is_warn) // report only failures
07551         return;
07552
07553     DOCTEST_LOCK_MUTEX(mutex)
07554
07555     std::ostringstream os;
07556     os << skipPathFromFilename(mb.m_file) << (opt.gnu_file_line ? ":" : "(") << line(mb.m_line)
07557     << (opt.gnu_file_line ? ":" : ":") << std::endl;
07558
07559     os << mb.m_string.c_str() << "\n";
07560     log_contexts(os);
07561
07562     testCaseData.addFailure(

```

```

07563         mb.m_string.c_str(), mb.m_severity & assertType::is_check ? "FAIL_CHECK" : "FAIL", os.str()
07564     );
07565 }
07566
07567 void JUnitReporter::test_case_skipped(const TestCaseData &) {}
07568
07569 void JUnitReporter::log_contexts(std::ostream &s) {
07570     const int num_contexts = get_num_active_contexts();
07571     if (num_contexts) {
07572         auto contexts = get_active_contexts();
07573
07574         s << "  logged: ";
07575         for (int i = 0; i < num_contexts; ++i) {
07576             s << (i == 0 ? "" : "    ") << "    ";
07577             contexts[i]->stringify(&s);
07578             s << std::endl;
07579         }
07580     }
07581 }
07582
07583 } // namespace doctest
07584
07585 #endif // DOCTEST_CONFIG_DISABLE
07586
07587 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
07588
07589 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
07590
07591 #ifndef DOCTEST_CONFIG_DISABLE
07592
07593 namespace doctest {
07594
07595     XmlReporter::XmlReporter(const ContextOptions &co)
07596         : xml(*co.cout), opt(co) {}
07597
07598     void XmlReporter::log_contexts() {
07599         const int num_contexts = get_num_active_contexts();
07600         if (num_contexts) {
07601             auto contexts = get_active_contexts();
07602             std::stringstream ss;
07603             for (int i = 0; i < num_contexts; ++i) {
07604                 contexts[i]->stringify(&ss);
07605                 xml.scopedElement("Info").writeText(ss.str());
07606                 ss.str("");
07607             }
07608         }
07609     }
07610
07611     unsigned XmlReporter::line(unsigned l) const {
07612         return opt.no_line_numbers ? 0 : l;
07613     }
07614
07615     void XmlReporter::test_case_start_impl(const TestCaseData &in) {
07616         bool open_ts_tag = false;
07617         if (tc != nullptr) { // we have already opened a test suite
07618             if (std::strcmp(tc->m_test_suite, in.m_test_suite) != 0) {
07619                 xml.endElement();
07620                 open_ts_tag = true;
07621             }
07622         } else {
07623             open_ts_tag = true; // first test case ==> first test suite
07624         }
07625
07626         if (open_ts_tag) {
07627             xml.startElement("TestSuite");
07628             xml.writeAttribute("name", in.m_test_suite);
07629         }
07630
07631         tc = &in;
07632         xml.startElement("TestCase")
07633             .writeAttribute("name", in.m_name)
07634             .writeAttribute("filename", skipPathFromFilename(in.m_file.c_str()))
07635             .writeAttribute("line", line(in.m_line))
07636             .writeAttribute("description", in.m_description);
07637
07638         if (Approx(in.m_timeout) != 0)
07639             xml.writeAttribute("timeout", in.m_timeout);
07640         if (in.m_may_fail)
07641             xml.writeAttribute("may_fail", true);
07642         if (in.m_should_fail)
07643             xml.writeAttribute("should_fail", true);
07644     }
07645
07646     void XmlReporter::report_query(const QueryData &in) {
07647         test_run_start();
07648         if (opt.list_reporters) {
07649             for (auto &curr: detail::getListeners())

```



```

07650         xml.scopedElement("Listener")
07651             .writeAttribute("priority", curr.first.first)
07652             .writeAttribute("name", curr.first.second);
07653     for (auto &curr: detail::getReporters())
07654         xml.scopedElement("Reporter")
07655             .writeAttribute("priority", curr.first.first)
07656             .writeAttribute("name", curr.first.second);
07657 } else if (opt.count || opt.list_test_cases) {
07658     for (unsigned i = 0; i < in.num_data; ++i) {
07659         xml.scopedElement("TestCase")
07660             .writeAttribute("name", in.data[i]->m_name)
07661             .writeAttribute("testsuite", in.data[i]->m_test_suite)
07662             .writeAttribute("filename", skipPathFromFilename(in.data[i]->m_file.c_str()))
07663             .writeAttribute("line", line(in.data[i]->m_line))
07664             .writeAttribute("skipped", in.data[i]->m_skip);
07665     }
07666     xml.scopedElement("OverallResultsTestCases")
07667         .writeAttribute("unskipped", in.run_stats->numTestCasesPassingFilters);
07668 } else if (opt.list_test_suites) {
07669     for (unsigned i = 0; i < in.num_data; ++i)
07670         xml.scopedElement("TestSuite").writeAttribute("name", in.data[i]->m_test_suite);
07671     xml.scopedElement("OverallResultsTestCases")
07672         .writeAttribute("unskipped", in.run_stats->numTestCasesPassingFilters);
07673     xml.scopedElement("OverallResultsTestSuites")
07674         .writeAttribute("unskipped", in.run_stats->numTestSuitesPassingFilters);
07675 }
07676 xml.endElement();
07677 }
07678
07679 void XmlReporter::test_run_start() {
07680     xml.writeDeclaration();
07681
07682     // remove .exe extension - mainly to have the same output on UNIX and Windows
07683     // NOLINTNEXTLINE(misc-const-correctness)
07684     std::string binary_name = skipPathFromFilename(opt.binary_name.c_str());
07685     #ifdef DOCTEST_PLATFORM_WINDOWS
07686     if (binary_name.rfind(".exe") != std::string::npos)
07687         binary_name = binary_name.substr(0, binary_name.length() - 4);
07688     #endif // DOCTEST_PLATFORM_WINDOWS
07689
07690     xml.startElement("doctest").writeAttribute("binary", binary_name);
07691     if (opt.no_version == false)
07692         xml.writeAttribute("version", DOCTEST_VERSION_STR);
07693
07694     // only the consequential ones (TODO: filters)
07695     xml.scopedElement("Options")
07696         .writeAttribute("order_by", opt.order_by.c_str())
07697         .writeAttribute("rand_seed", opt.rand_seed)
07698         .writeAttribute("first", opt.first)
07699         .writeAttribute("last", opt.last)
07700         .writeAttribute("abort_after", opt.abort_after)
07701         .writeAttribute("subcase_filter_levels", opt.subcase_filter_levels)
07702         .writeAttribute("case_sensitive", opt.case_sensitive)
07703         .writeAttribute("no_throw", opt.no_throw)
07704         .writeAttribute("no_skip", opt.no_skip);
07705 }
07706
07707 void XmlReporter::test_run_end(const TestRunStats &p) {
07708     if (tc) // the TestSuite tag - only if there has been at least 1 test case
07709         xml.endElement();
07710
07711     xml.scopedElement("OverallResultsAsserts")
07712         .writeAttribute("successes", p.numAsserts - p.numAssertsFailed)
07713         .writeAttribute("failures", p.numAssertsFailed);
07714
07715     xml.startElement("OverallResultsTestCases")
07716         .writeAttribute("successes", p.numTestCasesPassingFilters - p.numTestCasesFailed)
07717         .writeAttribute("failures", p.numTestCasesFailed);
07718     if (opt.no_skipped_summary == false)
07719         xml.writeAttribute("skipped", p.numTestCases - p.numTestCasesPassingFilters);
07720     xml.endElement();
07721
07722     xml.endElement();
07723 }
07724
07725 void XmlReporter::test_case_start(const TestCaseData &in) {
07726     DOCTEST_LOCK_MUTEX(mutex)
07727     test_case_start_impl(in);
07728     xml.ensureTagClosed();
07729 }
07730
07731 void XmlReporter::test_case_reenter(const TestCaseData &) {}
07732
07733 void XmlReporter::test_case_end(const CurrentTestCaseStats &st) {
07734     DOCTEST_LOCK_MUTEX(mutex)
07735     xml.startElement("OverallResultsAsserts")
07736         .writeAttribute("successes", st.numAssertsCurrentTest - st.numAssertsFailedCurrentTest)

```

```

07737         .writeAttribute("failures", st.numAssertsFailedCurrentTest)
07738         .writeAttribute("test_case_success", st.testCaseSuccess);
07739     if (opt.duration)
07740         xml.writeAttribute("duration", st.seconds);
07741     if (tc->m_expected_failures)
07742         xml.writeAttribute("expected_failures", tc->m_expected_failures);
07743     xml.endElement();
07744
07745     xml.endElement();
07746 }
07747
07748 void XmlReporter::test_case_exception(const TestCaseException &e) {
07749     DOCTEST_LOCK_MUTEX(mutex)
07750
07751     xml.scopedElement("Exception").writeAttribute("crash",
07752 e.is_crash).writeText(e.error_string.c_str());
07753 }
07754
07755 void XmlReporter::subcase_start(const SubcaseSignature &in) {
07756     DOCTEST_LOCK_MUTEX(mutex)
07757     xml.startElement("SubCase")
07758         .writeAttribute("name", in.m_name)
07759         .writeAttribute("filename", skipPathFromFilename(in.m_file))
07760         .writeAttribute("line", line(in.m_line));
07761     xml.ensureTagClosed();
07762 }
07763
07764 void XmlReporter::subcase_end() {
07765     DOCTEST_LOCK_MUTEX(mutex)
07766     xml.endElement();
07767 }
07768
07769 void XmlReporter::log_assert(const AssertData &rb) {
07770     if (!rb.m_failed && !opt.success)
07771         return;
07772     DOCTEST_LOCK_MUTEX(mutex)
07773
07774     xml.startElement("Expression")
07775         .writeAttribute("success", !rb.m_failed)
07776         .writeAttribute("type", assertString(rb.m_at))
07777         .writeAttribute("filename", skipPathFromFilename(rb.m_file))
07778         .writeAttribute("line", line(rb.m_line));
07779
07780     xml.scopedElement("Original").writeText(rb.m_expr);
07781
07782     if (rb.m_threw)
07783         xml.scopedElement("Exception").writeText(rb.m_exception.c_str());
07784
07785     if (rb.m_at & assertType::is_throws_as)
07786         xml.scopedElement("ExpectedException").writeText(rb.m_exception_type);
07787     if (rb.m_at & assertType::is_throws_with)
07788         xml.scopedElement("ExpectedExceptionString").writeText(rb.m_exception_string.c_str());
07789     if ((rb.m_at & assertType::is_normal) && !rb.m_threw)
07790         xml.scopedElement("Expanded").writeText(rb.m_decomp.c_str());
07791
07792     log_contexts();
07793
07794     xml.endElement();
07795 }
07796
07797 void XmlReporter::log_message(const MessageData &mb) {
07798     DOCTEST_LOCK_MUTEX(mutex)
07799
07800     xml.startElement("Message")
07801         .writeAttribute("type", failureString(mb.m_severity))
07802         .writeAttribute("filename", skipPathFromFilename(mb.m_file))
07803         .writeAttribute("line", line(mb.m_line));
07804
07805     xml.scopedElement("Text").writeText(mb.m_string.c_str());
07806
07807     log_contexts();
07808
07809     xml.endElement();
07810 }
07811
07812 void XmlReporter::test_case_skipped(const TestCaseData &in) {
07813     if (opt.no_skipped_summary == false) {
07814         DOCTEST_LOCK_MUTEX(mutex)
07815         test_case_start_impl(in);
07816         xml.writeAttribute("skipped", "true");
07817         xml.endElement();
07818     }
07819 }
07820
07821 } // namespace doctest
07822

```

```

07823 #endif // DOCTEST_CONFIG_DISABLE
07824
07825 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
07826
07827 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
07828
07829 #ifndef DOCTEST_CONFIG_DISABLE
07830
07831 namespace doctest {
07832 namespace detail {
07833
07834 #if !defined(DOCTEST_CONFIG_POSIX_SIGNALS) && !defined(DOCTEST_CONFIG_WINDOWS_SEH)
07835 void FatalConditionHandler::reset() {}
07836 void FatalConditionHandler::allocateAltStackMem() {}
07837 void FatalConditionHandler::freeAltStackMem() {}
07838 #else // DOCTEST_CONFIG_POSIX_SIGNALS || DOCTEST_CONFIG_WINDOWS_SEH
07839
07840 #ifdef DOCTEST_PLATFORM_WINDOWS
07841
07842 // There is no 1-1 mapping between signals and windows exceptions.
07843 // Windows can easily distinguish between SO and SigSegV,
07844 // but SigInt, SigTerm, etc are handled differently.
07845 SignalDefs signalDefs[] = {
07846     {static_cast<DWORD>(EXCEPTION_ILLEGAL_INSTRUCTION), "SIGILL - Illegal instruction signal"},
07847     {static_cast<DWORD>(EXCEPTION_STACK_OVERFLOW), "SIGSEGV - Stack overflow"},
07848     {static_cast<DWORD>(EXCEPTION_ACCESS_VIOLATION), "SIGSEGV - Segmentation violation signal"},
07849     {static_cast<DWORD>(EXCEPTION_INT_DIVIDE_BY_ZERO), "Divide by zero error"},
07850 };
07851
07852 LONG CALLBACK FatalConditionHandler::handleException(PEXCEPTION_POINTERS ExceptionInfo) {
07853     // Multiple threads may enter this filter/handler at once. We want the error message to be
07854     // printed on the console just once no matter how many threads have crashed.
07855     DOCTEST_DECLARE_STATIC_MUTEX(mutex)
07856     static bool execute = true;
07857     {
07858         DOCTEST_LOCK_MUTEX(mutex)
07859         if (execute) {
07860             bool reported = false;
07861             for (size_t i = 0; i < DOCTEST_COUNTOF(signalDefs); ++i) {
07862                 if (ExceptionInfo->ExceptionRecord->ExceptionCode == signalDefs[i].id) {
07863                     reportFatal(signalDefs[i].name);
07864                     reported = true;
07865                     break;
07866                 }
07867             }
07868             if (reported == false)
07869                 reportFatal("Unhandled SEH exception caught");
07870             if (isDebuggerActive() && !g_cs->no_breaks)
07871                 DOCTEST_BREAK_INTO_DEBUGGER();
07872         }
07873         execute = false;
07874     }
07875     std::exit(EXIT_FAILURE);
07876 }
07877
07878 void FatalConditionHandler::allocateAltStackMem() {}
07879 void FatalConditionHandler::freeAltStackMem() {}
07880
07881 FatalConditionHandler::FatalConditionHandler() {
07882     isSet = true;
07883     // 32k seems enough for doctest to handle stack overflow,
07884     // but the value was found experimentally, so there is no strong guarantee
07885     guaranteeSize = 32 * 1024;
07886     // Register an unhandled exception filter
07887     previousTop = SetUnhandledExceptionFilter(handleException);
07888     // Pass in guarantee size to be filled
07889     SetThreadStackGuarantee(&guaranteeSize);
07890
07891     // On Windows uncaught exceptions from another thread, exceptions from
07892     // destructors, or calls to std::terminate are not a SEH exception
07893
07894     // The terminal handler gets called when:
07895     // - std::terminate is called FROM THE TEST RUNNER THREAD
07896     // - an exception is thrown from a destructor FROM THE TEST RUNNER THREAD
07897     original_terminate_handler = std::get_terminate();
07898     std::set_terminate([]() DOCTEST_NOEXCEPT {
07899         reportFatal("Terminate handler called");
07900         if (isDebuggerActive() && !g_cs->no_breaks)
07901             DOCTEST_BREAK_INTO_DEBUGGER();
07902         std::exit(EXIT_FAILURE); // explicitly exit - otherwise the SIGABRT handler may be called as
07903         well
07904     });
07905
07906     // SIGABRT is raised when:
07907     // - std::terminate is called FROM A DIFFERENT THREAD
07908     // - an exception is thrown from a destructor FROM A DIFFERENT THREAD
07909     // - an uncaught exception is thrown FROM A DIFFERENT THREAD

```

```

07909     prev_sigabrt_handler = std::signal(SIGABRT, [](int signal) DOCTEST_NOEXCEPT {
07910         if (signal == SIGABRT) {
07911             reportFatal("SIGABRT - Abort (abnormal termination) signal");
07912             if (isDebuggerActive() && !g_cs->no_breaks)
07913                 DOCTEST_BREAK_INTO_DEBUGGER();
07914             std::exit(EXIT_FAILURE);
07915         }
07916     });
07917
07918     // The following settings are taken from google test, and more
07919     // specifically from UnitTest::Run() inside of gtest.cc
07920
07921     // the user does not want to see pop-up dialogs about crashes
07922     prev_error_mode_1 = SetErrorMode(
07923         SEM_FAILCRITICALERRORS | SEM_NOALIGNMENTFAULTEXCEPT | SEM_NOGPFAULTERRORBOX |
07924         SEM_NOOPENFILEERRORBOX
07925     );
07926     // This forces the abort message to go to stderr in all circumstances.
07927     prev_error_mode_2 = _set_error_mode(_OUT_TO_STDERR);
07928     // In the debug version, Visual Studio pops up a separate dialog
07929     // offering a choice to debug the aborted program - we want to disable that.
07930     prev_abort_behavior = _set_abort_behavior(0x0, _WRITE_ABORT_MSG | _CALL_REPORTFAULT);
07931     // In debug mode, the Windows CRT can crash with an assertion over invalid
07932     // input (e.g. passing an invalid file descriptor). The default handling
07933     // for these assertions is to pop up a dialog and wait for user input.
07934     // Instead ask the CRT to dump such assertions to stderr non-interactively.
07935     prev_report_mode = _CrtSetReportMode(_CRT_ASSERT, _CRTDBG_MODE_FILE | _CRTDBG_MODE_DEBUG);
07936     prev_report_file = _CrtSetReportFile(_CRT_ASSERT, _CRTDBG_FILE_STDERR);
07937
07938 void FatalConditionHandler::reset() {
07939     if (isSet) {
07940         // Unregister handler and restore the old guarantee
07941         SetUnhandledExceptionFilter(previousTop);
07942         SetThreadStackGuarantee(&guaranteeSize);
07943         std::set_terminate(original_terminate_handler);
07944         std::signal(SIGABRT, prev_sigabrt_handler);
07945         SetErrorMode(prev_error_mode_1);
07946         _set_error_mode(prev_error_mode_2);
07947         _set_abort_behavior(prev_abort_behavior, _WRITE_ABORT_MSG | _CALL_REPORTFAULT);
07948         static_cast<void>(_CrtSetReportMode(_CRT_ASSERT, prev_report_mode));
07949         static_cast<void>(_CrtSetReportFile(_CRT_ASSERT, prev_report_file));
07950         isSet = false;
07951     }
07952 }
07953
07954 FatalConditionHandler::~FatalConditionHandler() {
07955     reset();
07956 }
07957
07958 UINT FatalConditionHandler::prev_error_mode_1;
07959 int FatalConditionHandler::prev_error_mode_2;
07960 unsigned int FatalConditionHandler::prev_abort_behavior;
07961 int FatalConditionHandler::prev_report_mode;
07962 _HFILE FatalConditionHandler::prev_report_file;
07963 void(DOCTEST_CDECL *FatalConditionHandler::prev_sigabrt_handler)(int);
07964 std::terminate_handler FatalConditionHandler::original_terminate_handler;
07965 bool FatalConditionHandler::isSet = false;
07966 ULONG FatalConditionHandler::guaranteeSize = 0;
07967 LPTOP_LEVEL_EXCEPTION_FILTER FatalConditionHandler::previousTop = nullptr;
07968
07969 #else // DOCTEST_PLATFORM_WINDOWS
07970
07971 SignalDefs signalDefs[] = {
07972     {SIGINT, "SIGINT - Terminal interrupt signal"},
07973     {SIGILL, "SIGILL - Illegal instruction signal"},
07974     {SIGFPE, "SIGFPE - Floating point error signal"},
07975     {SIGSEGV, "SIGSEGV - Segmentation violation signal"},
07976     {SIGTERM, "SIGTERM - Termination request signal"},
07977     {SIGABRT, "SIGABRT - Abort (abnormal termination) signal"}
07978 };
07979 static_assert(
07980     DOCTEST_COUNTOF(signalDefs) == DOCTEST_COUNTOF(FatalConditionHandler::oldSigActions), "arrays
    should match in size"
07981 );
07982
07983 void FatalConditionHandler::handleSignal(int sig) {
07984     const char *name = "<unknown signal>";
07985     for (std::size_t i = 0; i < DOCTEST_COUNTOF(signalDefs); ++i) {
07986         const SignalDefs &def = signalDefs[i];
07987         if (sig == def.id) {
07988             name = def.name;
07989             break;
07990         }
07991     }
07992     reset();
07993     reportFatal(name);

```

```

07994     static_cast<void>(raise(sig));
07995 }
07996
07997 void FatalConditionHandler::allocateAltStackMem() {
07998     altStackMem = new char[altStackSize];
07999 }
08000
08001 void FatalConditionHandler::freeAltStackMem() {
08002     delete[] altStackMem;
08003 }
08004
08005 FatalConditionHandler::FatalConditionHandler() {
08006     isSet = true;
08007     stack_t sigStack;
08008     sigStack.ss_sp = altStackMem;
08009     sigStack.ss_size = altStackSize;
08010     sigStack.ss_flags = 0;
08011     sigaltstack(&sigStack, &oldSigStack);
08012     struct sigaction sa = {};
08013     sa.sa_handler = handleSignal;
08014     sa.sa_flags = SA_ONSTACK;
08015     for (std::size_t i = 0; i < DOCTEST_COUNTOF(signalDefs); ++i) {
08016         sigaction(signalDefs[i].id, &sa, &oldSigActions[i]);
08017     }
08018 }
08019
08020 FatalConditionHandler::~FatalConditionHandler() {
08021     reset();
08022 }
08023
08024 void FatalConditionHandler::reset() {
08025     if (isSet) {
08026         // Set signals back to previous values -- hopefully nobody overwrote them in the meantime
08027         for (std::size_t i = 0; i < DOCTEST_COUNTOF(signalDefs); ++i) {
08028             sigaction(signalDefs[i].id, &oldSigActions[i], nullptr);
08029         }
08030         // Return the old stack
08031         sigaltstack(&oldSigStack, nullptr);
08032         isSet = false;
08033     }
08034 }
08035
08036 bool FatalConditionHandler::isSet = false;
08037 struct sigaction FatalConditionHandler::oldSigActions[DOCTEST_COUNTOF(signalDefs)] = {};
08038 stack_t FatalConditionHandler::oldSigStack = {};
08039 size_t FatalConditionHandler::altStackSize = 4 * SIGSTKSZ;
08040 char *FatalConditionHandler::altStackMem = nullptr;
08041
08042 #endif // DOCTEST_PLATFORM_WINDOWS
08043 #endif // DOCTEST_CONFIG_POSIX_SIGNALS || DOCTEST_CONFIG_WINDOWS_SEH
08044
08045 } // namespace detail
08046 } // namespace doctest
08047
08048 #endif // DOCTEST_CONFIG_DISABLE
08049
08050 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
08051
08052 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
08053
08054 namespace doctest {
08055 namespace detail {
08056
08057 DOCTEST_THREAD_LOCAL class oss {
08058     std::vector<std::streampos> stack;
08059     std::stringstream ss;
08060
08061 public:
08062     std::ostream *push() {
08063         stack.push_back(ss.tellp());
08064         return &ss;
08065     }
08066
08067     String pop() {
08068         if (stack.empty())
08069             DOCTEST_INTERNAL_ERROR("TLSS was empty when trying to pop!");
08070
08071         const std::streampos pos = stack.back();
08072         stack.pop_back();
08073         const unsigned sz = static_cast<unsigned>(ss.tellp() - pos);
08074         ss.rdbuf()->pubseekpos(pos, std::ios::in | std::ios::out);
08075         return String(ss, sz);
08076     }
08077 } g_oss; // NOLINT(bugprone-throwing-static-initialization, cert-err58-cpp)
08078
08079 std::ostream *tlssPush() {
08080     return g_oss.push();

```

```

08081 }
08082
08083 String tlssPop() {
08084     return g_oss.pop();
08085 }
08086
08087 } // namespace detail
08088
08089 // case insensitive strcmp
08090 static int stricmp(const char *a, const char *b) {
08091     for (; a++, b++) {
08092         const int d = tolower(*a) - tolower(*b);
08093         if (d != 0 || !*a)
08094             return d;
08095     }
08096 }
08097
08098 // NOLINTBEGIN(cppcoreguidelines-pro-type-union-access)
08099
08100 char *String::allocate(size_type sz) {
08101     if (sz <= last) {
08102         buf[sz] = '\0';
08103         setLast(last - sz);
08104         return buf;
08105     } else {
08106         setOnHeap();
08107         data.size = sz;
08108         data.capacity = data.size + 1;
08109         data.ptr = new char[data.capacity];
08110         data.ptr[sz] = '\0';
08111         return data.ptr;
08112     }
08113 }
08114
08115 void String::setOnHeap() noexcept {
08116     // NOLINTNEXTLINE(cppcoreguidelines-pro-type-reinterpret-cast)
08117     *reinterpret_cast<unsigned char *>(&buf[last]) = 128;
08118 }
08119
08120 void String::setLast(size_type in) noexcept {
08121     buf[last] = static_cast<char>(in);
08122 }
08123
08124 void String::setSize(size_type sz) noexcept {
08125     if (isOnStack()) {
08126         buf[sz] = '\0';
08127         setLast(last - sz);
08128     } else {
08129         data.ptr[sz] = '\0';
08130         data.size = sz;
08131     }
08132 }
08133
08134 void String::copy(const String &other) {
08135     if (other.isOnStack()) {
08136         memcpy(buf, other.buf, len);
08137     } else {
08138         memcpy(allocate(other.data.size), other.data.ptr, other.data.size);
08139     }
08140 } // NOLINT(clang-analyzer-cplusplus.NewDeleteLeaks)
08141
08142 String::String() noexcept {
08143     buf[0] = '\0';
08144     setLast();
08145 }
08146
08147 String::~String() {
08148     if (!isOnStack())
08149         delete[] data.ptr;
08150 } // NOLINT(clang-analyzer-cplusplus.NewDeleteLeaks)
08151
08152 String::String(const char *in)
08153     : String(in, strlen(in)) {}
08154
08155 String::String(const char *in, size_type in_size) {
08156     memcpy(allocate(in_size), in, in_size);
08157 }
08158
08159 String::String(std::istream &in, size_type in_size) {
08160     in.read(allocate(in_size), in_size);
08161 }
08162
08163 String::String(const String &other) {
08164     copy(other);
08165 }
08166
08167 String &String::operator=(const String &other) {

```

```

08168     if (this != &other) {
08169         if (!isOnStack())
08170             delete[] data.ptr;
08171
08172         copy(other);
08173     }
08174
08175     return *this;
08176 }
08177
08178 String &String::operator+=(const String &other) {
08179     const size_type my_old_size = size();
08180     const size_type other_size = other.size();
08181     const size_type total_size = my_old_size + other_size;
08182     if (isOnStack()) {
08183         if (total_size < len) {
08184             // append to the current stack space
08185             memcpy(buf + my_old_size, other.c_str(), other_size + 1);
08186             // NOLINTNEXTLINE(clang-analyzer-cplusplus.NewDeleteLeaks)
08187             setLast(last - total_size);
08188         } else {
08189             // alloc new chunk
08190             char *temp = new char[total_size + 1];
08191             // copy current data to new location before writing in the union
08192             memcpy(temp, buf, my_old_size); // skip the +1 ('\0') for speed
08193             // update data in union
08194             setOnHeap();
08195             data.size = total_size;
08196             data.capacity = data.size + 1;
08197             data.ptr = temp;
08198             // transfer the rest of the data
08199             memcpy(data.ptr + my_old_size, other.c_str(), other_size + 1);
08200         }
08201     } else {
08202         if (data.capacity > total_size) {
08203             // append to the current heap block
08204             data.size = total_size;
08205             memcpy(data.ptr + my_old_size, other.c_str(), other_size + 1);
08206         } else {
08207             // resize
08208             data.capacity *= 2;
08209             if (data.capacity <= total_size)
08210                 data.capacity = total_size + 1;
08211             // alloc new chunk
08212             char *temp = new char[data.capacity];
08213             // copy current data to new location before releasing it
08214             memcpy(temp, data.ptr, my_old_size); // skip the +1 ('\0') for speed
08215             // release old chunk
08216             delete[] data.ptr;
08217             // update the rest of the union members
08218             data.size = total_size;
08219             data.ptr = temp;
08220             // transfer the rest of the data
08221             memcpy(data.ptr + my_old_size, other.c_str(), other_size + 1);
08222         }
08223     }
08224
08225     return *this;
08226 }
08227
08228 String::String(String &&other) noexcept {
08229     memcpy(buf, other.buf, len);
08230     other.buf[0] = '\0';
08231     other.setLast();
08232 }
08233
08234 String &String::operator=(String &&other) noexcept {
08235     if (this != &other) {
08236         if (!isOnStack())
08237             delete[] data.ptr;
08238         memcpy(buf, other.buf, len);
08239         other.buf[0] = '\0';
08240         other.setLast();
08241     }
08242     return *this;
08243 }
08244
08245 char String::operator[](size_type i) const {
08246     // NOLINTNEXTLINE(cppcoreguidelines-pro-type-const-cast)
08247     return const_cast<String *>(this)->operator[](i);
08248 }
08249
08250 char &String::operator[](size_type i) {
08251     if (isOnStack())
08252         // NOLINTNEXTLINE(cppcoreguidelines-pro-type-reinterpret-cast)
08253         return reinterpret_cast<char *>(buf)[i];
08254     return data.ptr[i];

```

```

08255 }
08256
08257 DOCTEST_GCC_SUPPRESS_WARNING_WITH_PUSH("-Wmaybe-uninitialized")
08258 String::size_type String::size() const {
08259     if (isOnStack())
08260         return last - (static_cast<size_type>(buf[last]) & 31); // using "last" would work only if
    "len" is 32
08261     return data.size;
08262 }
08263 DOCTEST_GCC_SUPPRESS_WARNING_POP
08264
08265 String::size_type String::capacity() const {
08266     if (isOnStack())
08267         return len;
08268     return data.capacity;
08269 }
08270
08271 String String::substr(size_type pos, size_type cnt) && {
08272     cnt = std::min(cnt, size() - pos);
08273     char *cptr = c_str();
08274     memmove(cptr, cptr + pos, cnt);
08275     setSize(cnt);
08276     return std::move(*this);
08277 }
08278
08279 String String::substr(size_type pos, size_type cnt) const & {
08280     cnt = std::min(cnt, size() - pos);
08281     return String(c_str() + pos, cnt);
08282 }
08283
08284 String::size_type String::find(char ch, size_type pos) const {
08285     const char *begin = c_str();
08286     const char *end = begin + size();
08287     const char *it = begin + pos;
08288     for (; it < end && *it != ch; it++) {}
08289     if (it < end) {
08290         return static_cast<size_type>(it - begin);
08291     } else {
08292         return npos;
08293     }
08294 }
08295
08296 String::size_type String::rfind(char ch, size_type pos) const {
08297     if (size() == 0) {
08298         return npos;
08299     }
08300
08301     const char *begin = c_str();
08302     const char *it = begin + std::min(pos, size() - 1);
08303     for (; it >= begin && *it != ch; it--) {}
08304     if (it >= begin) {
08305         return static_cast<size_type>(it - begin);
08306     } else {
08307         return npos;
08308     }
08309 }
08310
08311 int String::compare(const char *other, bool no_case) const {
08312     if (no_case)
08313         return doctest::stricmp(c_str(), other);
08314     return std::strcmp(c_str(), other);
08315 }
08316
08317 int String::compare(const String &other, bool no_case) const {
08318     return compare(other.c_str(), no_case);
08319 }
08320
08321 String operator+(const String &lhs, const String &rhs) {
08322     return String(lhs) += rhs;
08323 }
08324
08325 bool operator==(const String &lhs, const String &rhs) {
08326     return lhs.compare(rhs) == 0;
08327 }
08328
08329 bool operator!=(const String &lhs, const String &rhs) {
08330     return lhs.compare(rhs) != 0;
08331 }
08332
08333 bool operator<(const String &lhs, const String &rhs) {
08334     return lhs.compare(rhs) < 0;
08335 }
08336
08337 bool operator>(const String &lhs, const String &rhs) {
08338     return lhs.compare(rhs) > 0;
08339 }
08340

```



```

08341 bool operator<=(const String &lhs, const String &rhs) {
08342     return (lhs != rhs) ? lhs.compare(rhs) < 0 : true;
08343 }
08344
08345 bool operator>=(const String &lhs, const String &rhs) {
08346     return (lhs != rhs) ? lhs.compare(rhs) > 0 : true;
08347 }
08348
08349 std::ostream &operator<<(std::ostream &s, const String &in) {
08350     return s << in.c_str();
08351 }
08352
08353 namespace detail {
08354
08355 void filldata<const void *>::fill(std::ostream *stream, const void *in) {
08356     filldata<const volatile void *>::fill(stream, in);
08357 }
08358
08359 void filldata<const volatile void *>::fill(std::ostream *stream, const volatile void *in) {
08360     if (in) {
08361         *stream << in;
08362     } else {
08363         *stream << "nullptr";
08364     }
08365 }
08366
08367 template <typename T>
08368 String toStringLit(T t) {
08369     // NOLINTNEXTLINE(misc-const-correctness)
08370     std::ostream *os = tlssPush();
08371     os->operator<<(t);
08372     return tlssPop();
08373 }
08374 } // namespace detail
08375
08376 #ifdef DOCTEST_CONFIG_TREAT_CHAR_STAR_AS_STRING
08377 String toString(const char *in) {
08378     return String("\\") + (in ? in : "{null string}") + "\"";
08379 }
08380 #endif // DOCTEST_CONFIG_TREAT_CHAR_STAR_AS_STRING
08381
08382 #if DOCTEST_MSVC >= DOCTEST_COMPILER(19, 20, 0)
08383 // see this issue on why this is needed: https://github.com/doctest/doctest/issues/183
08384 String toString(const std::string &in) {
08385     return in.c_str();
08386 }
08387 #endif // VS 2019
08388
08389 String toString(const String &in) {
08390     return in;
08391 }
08392
08393 String toString(std::nullptr_t) {
08394     return "nullptr";
08395 }
08396
08397 String toString(bool in) {
08398     return in ? "true" : "false";
08399 }
08400
08401 String toString(float in) {
08402     return detail::toStringLit(in);
08403 }
08404
08405 String toString(double in) {
08406     return detail::toStringLit(in);
08407 }
08408
08409 String toString(double long in) {
08410     return detail::toStringLit(in);
08411 }
08412
08413 String toString(char in) {
08414     return detail::toStringLit(static_cast<signed>(in));
08415 }
08416
08417 String toString(char signed in) {
08418     return detail::toStringLit(static_cast<signed>(in));
08419 }
08420
08421 String toString(char unsigned in) {
08422     return detail::toStringLit(static_cast<unsigned>(in));
08423 }
08424
08425 String toString(short in) {
08426     return detail::toStringLit(in);
08427 }
08428
08429 String toString(short unsigned in) {
08430     return detail::toStringLit(in);
08431 }
08432
08433 String toString(signed in) {
08434     return detail::toStringLit(in);
08435 }
08436
08437 String toString(unsigned in) {
08438     return detail::toStringLit(in);
08439 }

```

```

08428 }
08429 String toString(unsigned in) {
08430     return detail::toStreamLit(in);
08431 }
08432 String toString(long in) {
08433     return detail::toStreamLit(in);
08434 }
08435 String toString(long unsigned in) {
08436     return detail::toStreamLit(in);
08437 }
08438 String toString(long long in) {
08439     return detail::toStreamLit(in);
08440 }
08441 String toString(long long unsigned in) {
08442     return detail::toStreamLit(in);
08443 }
08444
08445 // NOLINTEND(cppcoreguidelines-pro-type-union-access)
08446
08447 } // namespace doctest
08448
08449 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
08450
08451 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
08452
08453 namespace doctest {
08454
08455 bool SubcaseSignature::operator==(const SubcaseSignature &other) const {
08456     return m_line == other.m_line && std::strcmp(m_file, other.m_file) == 0 && m_name == other.m_name;
08457 }
08458
08459 bool SubcaseSignature::operator<(const SubcaseSignature &other) const {
08460     if (m_line != other.m_line)
08461         return m_line < other.m_line;
08462     if (std::strcmp(m_file, other.m_file) != 0)
08463         return std::strcmp(m_file, other.m_file) < 0;
08464     return m_name.compare(other.m_name) < 0;
08465 }
08466
08467 #ifndef DOCTEST_CONFIG_DISABLE
08468 namespace detail {
08469
08470 bool Subcase::checkFilters() {
08471     if (g_cs->traversal.activeSubcaseDepth() < static_cast<size_t>(g_cs->subcase_filter_levels)) {
08472         if (!matchesAny(m_signature.m_name.c_str(), g_cs->filters[6], true, g_cs->case_sensitive))
08473             return true;
08474         if (matchesAny(m_signature.m_name.c_str(), g_cs->filters[7], false, g_cs->case_sensitive))
08475             return true;
08476     }
08477     return false;
08478 }
08479
08480 Subcase::Subcase(const String &name, const char *file, int line)
08481 : m_signature({name, file, line}) {
08482     if (checkFilters())
08483         return;
08484
08485     if (!g_cs->traversal.tryEnterSubcase(m_signature))
08486         return;
08487
08488     m_entered = true;
08489     DOCTEST_ITERATE_THROUGH_REPORTERS(subcase_start, m_signature);
08490 }
08491
08492 DOCTEST_MSVC_SUPPRESS_WARNING_WITH_PUSH(4996) // std::uncaught_exception is deprecated in C++17
08493 DOCTEST_GCC_SUPPRESS_WARNING_WITH_PUSH("-Wdeprecated-declarations")
08494 DOCTEST_CLANG_SUPPRESS_WARNING_WITH_PUSH("-Wdeprecated-declarations")
08495
08496 Subcase::~Subcase() {
08497     if (m_entered) {
08498         g_cs->traversal.leaveSubcase();
08499
08500 #if defined(__cpp_lib_uncaught_exceptions) && __cpp_lib_uncaught_exceptions >= 201411L &&
08501     \
08502     (!defined(__MAC_OS_X_VERSION_MIN_REQUIRED) || __MAC_OS_X_VERSION_MIN_REQUIRED >= 101200)
08503     if (std::uncaught_exceptions() > 0
08504 #else
08505     if (std::uncaught_exception()
08506 #endif
08507         && g_cs->shouldLogCurrentException() {
08508         DOCTEST_ITERATE_THROUGH_REPORTERS(
08509             test_case_exception,
08510             {"exception thrown in subcase - will translate later "
08511              "when the whole test case has been exited (cannot "
08512              "translate while there is an active exception)",
08513              false}
08514         );
08515     }

```

```

08514         g_cs->shouldLogCurrentException = false;
08515     }
08516
08517     DOCTEST_ITERATE_THROUGH_REPORTERS(subcase_end, DOCTEST_EMPTY);
08518 }
08519 }
08520
08521 DOCTEST_CLANG_SUPPRESS_WARNING_POP
08522 DOCTEST_GCC_SUPPRESS_WARNING_POP
08523 DOCTEST_MSVC_SUPPRESS_WARNING_POP
08524
08525 Subcase::operator bool() const {
08526     return m_entered;
08527 }
08528
08529 } // namespace detail
08530 #endif // DOCTEST_CONFIG_DISABLE
08531
08532 } // namespace doctest
08533
08534 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
08535
08536 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
08537
08538 #ifndef DOCTEST_CONFIG_DISABLE
08539 namespace doctest {
08540 namespace detail {
08541
08542     std::set<TestCase> &getRegisteredTests() {
08543         static std::set<TestCase> data;
08544         return data;
08545     }
08546 }
08547
08548 TestCase::TestCase(
08549     funcType test, const char *file, unsigned line, const TestSuite &test_suite, const String &type,
08550     int template_id
08551 ) noexcept {
08552     m_file = file;
08553     m_line = line;
08554     m_name = nullptr; // will be later overridden in operator*
08555     m_test_suite = test_suite.m_test_suite;
08556     m_description = test_suite.m_description;
08557     m_skip = test_suite.m_skip;
08558     m_no_breaks = test_suite.m_no_breaks;
08559     m_no_output = test_suite.m_no_output;
08560     m_may_fail = test_suite.m_may_fail;
08561     m_should_fail = test_suite.m_should_fail;
08562     m_expected_failures = test_suite.m_expected_failures;
08563     m_timeout = test_suite.m_timeout;
08564
08565     m_test = test;
08566     m_type = type;
08567     m_template_id = template_id;
08568 }
08569
08570 TestCase::TestCase(const TestCase &other) noexcept // NOLINT(bugprone-copy-constructor-init)
08571 : TestCaseData() {
08572     *this = other;
08573 }
08574
08575 DOCTEST_MSVC_SUPPRESS_WARNING_WITH_PUSH(26434) // hides a non-virtual function
08576 TestCase &TestCase::operator=(const TestCase &other) noexcept { // NOLINT(cert-oop54-cpp)
08577     TestCaseData::operator=(other);
08578     m_test = other.m_test;
08579     m_type = other.m_type;
08580     m_template_id = other.m_template_id;
08581     m_full_name = other.m_full_name;
08582
08583     if (m_template_id != -1)
08584         m_name = m_full_name.c_str();
08585     return *this;
08586 }
08587
08588 DOCTEST_MSVC_SUPPRESS_WARNING_POP
08589
08590 TestCase &TestCase::operator*(const char *in) noexcept {
08591     m_name = in;
08592     // make a new name with an appended type for templated test case
08593     if (m_template_id != -1) {
08594         m_full_name = String(m_name) + "<" + m_type + ">";
08595         // redirect the name to point to the newly constructed full name
08596         m_name = m_full_name.c_str();
08597     }
08598     return *this;
08599 }
08600
08601 bool TestCase::operator<(const TestCase &other) const noexcept {

```

```

08600 // this will be used only to differentiate between test cases - not relevant for sorting
08601 if (m_line != other.m_line)
08602     return m_line < other.m_line;
08603 const int name_cmp = strcmp(m_name, other.m_name);
08604 if (name_cmp != 0)
08605     return name_cmp < 0;
08606 const int file_cmp = m_file.compare(other.m_file);
08607 if (file_cmp != 0)
08608     return file_cmp < 0;
08609 return m_template_id < other.m_template_id;
08610 }
08611
08612 // used by the macros for registering tests
08613 int regTest(const TestCase &tc) noexcept {
08614     getRegisteredTests().insert(tc);
08615     return 0;
08616 }
08617
08618 } // namespace detail
08619 } // namespace doctest
08620
08621 #endif // DOCTEST_CONFIG_DISABLE
08622
08623 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
08624
08625 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
08626
08627 #ifndef DOCTEST_CONFIG_DISABLE
08628 namespace doctest {
08629 namespace detail {
08630
08631     TestSuite &TestSuite::operator*(const char *in) noexcept {
08632         m_test_suite = in;
08633         return *this;
08634     }
08635 }
08636
08637 // sets the current test suite
08638 int setTestSuite(const TestSuite &ts) noexcept {
08639     doctest_detail_test_suite_ns::getCurrentTestSuite() = ts;
08640     return 0;
08641 }
08642
08643 } // namespace detail
08644 } // namespace doctest
08645
08646 namespace doctest_detail_test_suite_ns {
08647 // holds the current test suite
08648 doctest::detail::TestSuite &getCurrentTestSuite() noexcept {
08649     static doctest::detail::TestSuite data{};
08650     return data;
08651 }
08652 } // namespace doctest_detail_test_suite_ns
08653
08654 #endif // DOCTEST_CONFIG_DISABLE
08655
08656 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
08657
08658 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
08659
08660 #ifndef DOCTEST_CONFIG_DISABLE
08661 namespace doctest {
08662 namespace detail {
08663
08664 #ifdef DOCTEST_CONFIG_GETCURRENTTICKS
08665 ticks_t getCurrentTicks() {
08666     return DOCTEST_CONFIG_GETCURRENTTICKS();
08667 }
08668 #elif defined(DOCTEST_PLATFORM_WINDOWS)
08669 ticks_t getCurrentTicks() {
08670     static LARGE_INTEGER hz = {{0}}, hzo = {{0}};
08671     if (!hz.QuadPart) {
08672         QueryPerformanceFrequency(&hz);
08673         QueryPerformanceCounter(&hzo);
08674     }
08675     LARGE_INTEGER t;
08676     QueryPerformanceCounter(&t);
08677     return ((t.QuadPart - hzo.QuadPart) * LONGLONG(1000000)) / hz.QuadPart;
08678 }
08679 #else // DOCTEST_PLATFORM_WINDOWS
08680 ticks_t getCurrentTicks() {
08681     timeval t;
08682     gettimeofday(&t, nullptr);
08683     return static_cast<ticks_t>(t.tv_sec) * 1000000 + static_cast<ticks_t>(t.tv_usec);
08684 }
08685 #endif // DOCTEST_PLATFORM_WINDOWS
08686

```

```

08687
08688 void Timer::start() {
08689     m_ticks = getCurrentTicks();
08690 }
08691
08692 unsigned int Timer::getElapsedMicroseconds() const {
08693     return static_cast<unsigned int>(getCurrentTicks() - m_ticks);
08694 }
08695
08696 // unsigned int Timer::getElapsedMilliseconds() const {
08697 //     return static_cast<unsigned int>(getElapsedMicroseconds() / 1000);
08698 // }
08699
08700 double Timer::getElapsedSeconds() const {
08701     return static_cast<double>(getCurrentTicks() - m_ticks) / 1000000.0;
08702 }
08703
08704 } // namespace detail
08705 } // namespace doctest
08706
08707 #endif // DOCTEST_CONFIG_DISABLE
08708
08709 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
08710
08711
08712 #include <algorithm>
08713
08714 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
08715
08716 #ifndef DOCTEST_CONFIG_DISABLE
08717
08718 namespace doctest {
08719 namespace detail {
08720
08721 DOCTEST_NOINLINE DecisionPoint &TraversalState::ensureDecisionPointAtCurrentDepth() {
08722     const size_t depth = m_decisionDepth;
08723
08724     if (m_discoveredDecisionPath.size() == depth) {
08725         m_discoveredDecisionPath.emplace_back();
08726
08727         if (m_decisionPath.size() == depth)
08728             m_decisionPath.push_back(0);
08729     }
08730
08731     return m_discoveredDecisionPath[depth];
08732 }
08733
08734 void TraversalState::resetForTestCase() {
08735     m_decisionPath.clear();
08736     m_discoveredDecisionPath.clear();
08737     m_enteredSubcaseDepths.clear();
08738     m_activeSubcaseDepth = 0;
08739     m_decisionDepth = 0;
08740 }
08741
08742 void TraversalState::resetForRun() {
08743     m_activeSubcaseDepth = 0;
08744     m_discoveredDecisionPath.clear();
08745     m_decisionDepth = 0;
08746     m_enteredSubcaseDepths.clear();
08747 }
08748
08749 bool TraversalState::advance() {
08750     const size_t maxDepth = std::min(m_decisionPath.size(), m_discoveredDecisionPath.size());
08751     for (size_t depth = maxDepth; depth > 0; --depth) {
08752         const size_t index = depth - 1;
08753         if (m_decisionPath[index] + 1 < m_discoveredDecisionPath[index].branch_count) {
08754             ++m_decisionPath[index];
08755             m_decisionPath.resize(index + 1);
08756             return true;
08757         }
08758     }
08759
08760     return false;
08761 }
08762
08763 bool TraversalState::tryEnterSubcase(const SubcaseSignature &signature) {
08764     DecisionPoint &point = ensureDecisionPointAtCurrentDepth();
08765     std::vector<SubcaseSignature> &subcases = point.subcases;
08766     size_t siblingIndex = 0;
08767
08768     for (; siblingIndex < subcases.size(); ++siblingIndex) {
08769         if (subcases[siblingIndex] == signature)
08770             break;
08771     }
08772
08773     if (siblingIndex == subcases.size())

```

```

08774         subcases.push_back(signature);
08775
08776         point.branch_count = subcases.size();
08777
08778         if (siblingIndex != m_decisionPath[m_decisionDepth])
08779             return false;
08780
08781         m_enteredSubcaseDepths.push_back(m_decisionDepth);
08782         m_activeSubcaseDepth++;
08783         m_decisionDepth++;
08784         return true;
08785     }
08786
08787     void TraversalState::leaveSubcase() {
08788         m_decisionDepth = m_enteredSubcaseDepths.back();
08789         m_enteredSubcaseDepths.pop_back();
08790         m_activeSubcaseDepth--;
08791     }
08792
08793     size_t TraversalState::unwindActiveSubcases() {
08794         const size_t activeSubcaseCount = m_activeSubcaseDepth;
08795
08796         while (m_activeSubcaseDepth > 0)
08797             leaveSubcase();
08798
08799         return activeSubcaseCount;
08800     }
08801
08802     size_t TraversalState::acquireGeneratorIndex(size_t count) {
08803         DecisionPoint &point = ensureDecisionPointAtCurrentDepth();
08804         point.branch_count = count;
08805
08806         const size_t index = m_decisionPath[m_decisionDepth];
08807         m_decisionDepth++;
08808         return index < count ? index : 0;
08809     }
08810
08811     size_t acquireGeneratorDecisionIndex(size_t count) {
08812         return g_cs->traversal.acquireGeneratorIndex(count);
08813     }
08814 } // namespace detail
08815 } // namespace doctest
08816
08817 #endif // DOCTEST_CONFIG_DISABLE
08818
08819 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
08820
08821 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
08822
08823 #ifndef DOCTEST_CONFIG_DISABLE
08824
08825 namespace doctest {
08826 namespace detail {
08827
08828 // =====
08829 // The following code has been taken verbatim from Catch2/include/internal/catch_xmlwriter.cpp
08830 // This is done so cherry-picking bug fixes is trivial - even the style/formatting is untouched.
08831 // =====
08832 /* clang-format off */ /* NOLINTBEGIN */
08833
08834 using uchar = unsigned char;
08835
08836     size_t trailingBytes(unsigned char c) {
08837         if ((c & 0xE0) == 0xC0) {
08838             return 2;
08839         }
08840         if ((c & 0xF0) == 0xE0) {
08841             return 3;
08842         }
08843         if ((c & 0xF8) == 0xF0) {
08844             return 4;
08845         }
08846         DOCTEST_INTERNAL_ERROR("Invalid multibyte utf-8 start byte encountered");
08847     }
08848
08849     uint32_t headerValue(unsigned char c) {
08850         if ((c & 0xE0) == 0xC0) {
08851             return c & 0x1F;
08852         }
08853         if ((c & 0xF0) == 0xE0) {
08854             return c & 0x0F;
08855         }
08856         if ((c & 0xF8) == 0xF0) {
08857             return c & 0x07;
08858         }
08859         DOCTEST_INTERNAL_ERROR("Invalid multibyte utf-8 start byte encountered");
08860     }

```

```

08861
08862 void hexEscapeChar(std::ostream& os, unsigned char c) {
08863     std::ios_base::fmtflags f(os.flags());
08864     os << "\\x"
08865         << std::uppercase << std::hex << std::setfill('0') << std::setw(2)
08866         << static_cast<int>(c);
08867     os.flags(f);
08868 }
08869
08870 XmlEncode::XmlEncode( std::string const& str, ForWhat forWhat )
08871 :   m_str( str ),
08872   m_forWhat( forWhat )
08873 {}
08874
08875 void XmlEncode::encodeTo( std::ostream& os ) const {
08876     // Apostrophe escaping not necessary if we always use " to write attributes
08877     // (see: https://www.w3.org/TR/xml/#syntax)
08878
08879     for( std::size_t idx = 0; idx < m_str.size(); ++ idx ) {
08880         uchar c = m_str[idx];
08881         switch (c) {
08882             case '<': os << "<"; break;
08883             case '&': os << "&"; break;
08884
08885             case '>':
08886                 // See: https://www.w3.org/TR/xml/#syntax
08887                 if (idx > 2 && m_str[idx - 1] == '[' && m_str[idx - 2] == '[')
08888                     os << ">";
08889                 else
08890                     os << c;
08891                 break;
08892
08893             case '\"':
08894                 if (m_forWhat == ForAttributes)
08895                     os << """;
08896                 else
08897                     os << c;
08898                 break;
08899
08900             default:
08901                 // Check for control characters and invalid utf-8
08902
08903                 // Escape control characters in standard ascii
08904                 // see
08905                 https://stackoverflow.com/questions/404107/why-are-control-characters-illegal-in-xml-1-0
08906                 if (c < 0x09 || (c > 0x0D && c < 0x20) || c == 0x7F) {
08907                     hexEscapeChar(os, c);
08908                     break;
08909
08910                     // Plain ASCII: Write it to stream
08911                     if (c < 0x7F) {
08912                         os << c;
08913                         break;
08914                     }
08915
08916                     // UTF-8 territory
08917                     // Check if the encoding is valid and if it is not, hex escape bytes.
08918                     // Important: We do not check the exact decoded values for validity, only the encoding
08919                     format
08920                     // First check that this bytes is a valid lead byte:
08921                     // This means that it is not encoded as 1111 1XXX
08922                     // Or as 10XX XXXX
08923                     if (c < 0xC0 ||
08924                         c >= 0xF8) {
08925                         hexEscapeChar(os, c);
08926                         break;
08927                     }
08928
08929                     auto encBytes = trailingBytes(c);
08930                     // Are there enough bytes left to avoid accessing out-of-bounds memory?
08931                     if (idx + encBytes - 1 >= m_str.size()) {
08932                         hexEscapeChar(os, c);
08933                         break;
08934                     }
08935                     // The header is valid, check data
08936                     // The next encBytes bytes must together be a valid utf-8
08937                     // This means: bitpattern 10XX XXXX and the extracted value is sane (ish)
08938                     bool valid = true;
08939                     uint32_t value = headerValue(c);
08940                     for (std::size_t n = 1; n < encBytes; ++n) {
08941                         uchar nc = m_str[idx + n];
08942                         valid &= ((nc & 0xC0) == 0x80);
08943                         value = (value << 6) | (nc & 0x3F);
08944                     }
08945                     if (

```

```

08946         // Wrong bit pattern of following bytes
08947         (!valid) ||
08948         // Overlong encodings
08949         (value < 0x80) ||
08950         (
08951             value < 0x800    && encBytes > 2) || // removed "0x80 <= value
08952             &&" because redundant
08953             (0x800 < value && value < 0x10000 && encBytes > 3) ||
08954             // Encoded value out of range
08955             (value >= 0x110000)
08956         ) {
08957             hexEscapeChar(os, c);
08958             break;
08959         }
08960         // If we got here, this is in fact a valid(ish) utf-8 sequence
08961         for (std::size_t n = 0; n < encBytes; ++n) {
08962             os << m_str[idx + n];
08963         }
08964         idx += encBytes - 1;
08965         break;
08966     }
08967 }
08968
08969 std::ostream& operator << ( std::ostream& os, XmlEncode const& xmlEncode ) {
08970     xmlEncode.encodeTo( os );
08971     return os;
08972 }
08973
08974 XmlWriter::ScopedElement::ScopedElement( XmlWriter* writer )
08975 :   m_writer( writer )
08976 {}
08977
08978 XmlWriter::ScopedElement::ScopedElement( ScopedElement&& other ) DOCTEST_NOEXCEPT
08979 :   m_writer( other.m_writer ){
08980     other.m_writer = nullptr;
08981 }
08982
08983 XmlWriter::ScopedElement& XmlWriter::ScopedElement::operator=( ScopedElement&& other )
08984 DOCTEST_NOEXCEPT {
08985     if ( m_writer ) {
08986         m_writer->endElement();
08987     }
08988     m_writer = other.m_writer;
08989     other.m_writer = nullptr;
08990     return *this;
08991 }
08992
08993 XmlWriter::ScopedElement::~~ScopedElement() {
08994     if( m_writer )
08995         m_writer->endElement();
08996 }
08997
08998 XmlWriter::ScopedElement& XmlWriter::ScopedElement::writeText( std::string const& text, bool
08999 indent ) {
09000     m_writer->writeText( text, indent );
09001     return *this;
09002 }
09003
09004 XmlWriter::XmlWriter( std::ostream& os ) : m_os( os )
09005 {
09006     // writeDeclaration(); // called explicitly by the reporters that use the writer class - see
09007     // issue #627
09008 }
09009
09010 XmlWriter::~~XmlWriter() {
09011     while( !m_tags.empty() )
09012         endElement();
09013 }
09014
09015 XmlWriter& XmlWriter::startElement( std::string const& name ) {
09016     ensureTagClosed();
09017     newlineIfNecessary();
09018     m_os << m_indent << '<' << name;
09019     m_tags.push_back( name );
09020     m_indent += "  ";
09021     m_tagIsOpen = true;
09022     return *this;
09023 }
09024
09025 XmlWriter::ScopedElement XmlWriter::scopedElement( std::string const& name ) {
09026     ScopedElement scoped( this );
09027     startElement( name );
09028     return scoped;
09029 }
09030
09031 XmlWriter& XmlWriter::endElement() {

```



```

09029         newlineIfNecessary();
09030         m_indent = m_indent.substr( 0, m_indent.size()-2 );
09031         if( m_tagIsOpen ) {
09032             m_os << ">";
09033             m_tagIsOpen = false;
09034         }
09035         else {
09036             m_os << m_indent << "</" << m_tags.back() << ">";
09037         }
09038         m_os << std::endl;
09039         m_tags.pop_back();
09040         return *this;
09041     }
09042
09043     XmlWriter& XmlWriter::writeAttribute( std::string const& name, std::string const& attribute ) {
09044         if( !name.empty() && !attribute.empty() )
09045             m_os << ' ' << name << "=" << XmlEncode( attribute, XmlEncode::ForAttributes ) << "'";
09046         return *this;
09047     }
09048
09049     XmlWriter& XmlWriter::writeAttribute( std::string const& name, const char* attribute ) {
09050         if( !name.empty() && attribute && attribute[0] != '\0' )
09051             m_os << ' ' << name << "=" << XmlEncode( attribute, XmlEncode::ForAttributes ) << "'";
09052         return *this;
09053     }
09054
09055     XmlWriter& XmlWriter::writeAttribute( std::string const& name, bool attribute ) {
09056         m_os << ' ' << name << "=" << ( attribute ? "true" : "false" ) << "'";
09057         return *this;
09058     }
09059
09060     XmlWriter& XmlWriter::writeText( std::string const& text, bool indent ) {
09061         if( !text.empty() ){
09062             bool tagWasOpen = m_tagIsOpen;
09063             ensureTagClosed();
09064             if( tagWasOpen && indent )
09065                 m_os << m_indent;
09066             m_os << XmlEncode( text );
09067             m_needsNewline = true;
09068         }
09069         return *this;
09070     }
09071
09072     //XmlWriter& XmlWriter::writeComment( std::string const& text ) {
09073     //    ensureTagClosed();
09074     //    m_os << m_indent << "<!--" << text << "-->";
09075     //    m_needsNewline = true;
09076     //    return *this;
09077     //}
09078
09079     //void XmlWriter::writeStylesheetRef( std::string const& url ) {
09080     //    m_os << "<?xml-stylesheet type='text/xsl' href='" << url << "'?>\n";
09081     //}
09082
09083     //XmlWriter& XmlWriter::writeBlankLine() {
09084     //    ensureTagClosed();
09085     //    m_os << '\n';
09086     //    return *this;
09087     //}
09088
09089     void XmlWriter::ensureTagClosed() {
09090         if( m_tagIsOpen ) {
09091             m_os << ">" << std::endl;
09092             m_tagIsOpen = false;
09093         }
09094     }
09095
09096     void XmlWriter::writeDeclaration() {
09097         m_os << "<?xml version='1.0' encoding='UTF-8'?>\n";
09098     }
09099
09100     void XmlWriter::newlineIfNecessary() {
09101         if( m_needsNewline ) {
09102             m_os << std::endl;
09103             m_needsNewline = false;
09104         }
09105     }
09106
09107     /* clang-format on */ /* NOLINTEND */
09108     // =====
09109     // End of copy-pasted code from Catch
09110     // =====
09111
09112 } // namespace detail
09113 } // namespace doctest
09114
09115 #endif // DOCTEST_CONFIG_DISABLE

```

```
09116
09117 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
09118
09119 #endif // defined(DOCTEST_CONFIG_IMPLEMENT) && !defined(DOCTEST_LIBRARY_IMPLEMENTATION)
```

12.66 tests/test_Alarme.cpp File Reference

```
#include "doctest.h"
#include "Alarme.hpp"
#include "Parametro.hpp"
```

Functions

- [TEST_CASE](#) ("Alarme - estado inicial e false")
- [TEST_CASE](#) ("Alarme - SetEstado ativa alarme")
- [TEST_CASE](#) ("Alarme - SetEstado desativa alarme")
- [TEST_CASE](#) ("Alarme - AtualizarAlarme retorna true quando parametro em falha")
- [TEST_CASE](#) ("Alarme - AtualizarAlarme retorna false quando parametro ok")
- [TEST_CASE](#) ("Alarme - AtualizarAlarme sincroniza estado interno")
- [TEST_CASE](#) ("Alarme - SetDesc nao quebra estado")

12.66.1 Function Documentation

12.66.1.1 [TEST_CASE\(\)](#) [1/7]

```
TEST_CASE (
    "Alarme - AtualizarAlarme retorna false quando parametro ok" )
```

Definition at line 33 of file [test_Alarme.cpp](#).

12.66.1.2 [TEST_CASE\(\)](#) [2/7]

```
TEST_CASE (
    "Alarme - AtualizarAlarme retorna true quando parametro em falha" )
```

Definition at line 26 of file [test_Alarme.cpp](#).

12.66.1.3 [TEST_CASE\(\)](#) [3/7]

```
TEST_CASE (
    "Alarme - AtualizarAlarme sincroniza estado interno" )
```

Definition at line 39 of file [test_Alarme.cpp](#).

12.66.1.4 TEST_CASE() [4/7]

```
TEST_CASE (
    "Alarme - estado inicial e false" )
```

Definition at line 5 of file [test_Alarme.cpp](#).

12.66.1.5 TEST_CASE() [5/7]

```
TEST_CASE (
    "Alarme - SetDesc nao quebra estado" )
```

Definition at line 47 of file [test_Alarme.cpp](#).

12.66.1.6 TEST_CASE() [6/7]

```
TEST_CASE (
    "Alarme - SetEstado ativa alarme" )
```

Definition at line 11 of file [test_Alarme.cpp](#).

12.66.1.7 TEST_CASE() [7/7]

```
TEST_CASE (
    "Alarme - SetEstado desativa alarme" )
```

Definition at line 18 of file [test_Alarme.cpp](#).

12.67 test_Alarme.cpp

[Go to the documentation of this file.](#)

```
00001 #include "doctest.h"
00002 #include "Alarme.hpp"
00003 #include "Parametro.hpp"
00004
00005 TEST_CASE("Alarme - estado inicial e false") {
00006     Parametro p(5.0, 10.0, 0.0, "Temp", 1);
00007     Alarme a("Alarme Temperatura", 1, p);
00008     CHECK(a.GetEstado() == false);
00009 }
00010
00011 TEST_CASE("Alarme - SetEstado ativa alarme") {
00012     Parametro p(5.0, 10.0, 0.0, "Temp", 2);
00013     Alarme a("Alarme Temp", 2, p);
00014     a.SetEstado(true);
00015     CHECK(a.GetEstado() == true);
00016 }
00017
00018 TEST_CASE("Alarme - SetEstado desativa alarme") {
00019     Parametro p(5.0, 10.0, 0.0, "Temp", 3);
00020     Alarme a("Alarme Temp", 3, p);
00021     a.SetEstado(true);
00022     a.SetEstado(false);
00023     CHECK(a.GetEstado() == false);
00024 }
00025
00026 TEST_CASE("Alarme - AtualizarAlarme retorna true quando parametro em falha") {
00027     Parametro p(15.0, 10.0, 0.0, "Pressao", 4);
00028     p.DiagnosticarParametro();
```

```

00029     Alarme a("Alarme Pressao", 4, p);
00030     CHECK(a.AtualizarAlarme() == true);
00031 }
00032
00033 TEST_CASE("Alarme - AtualizarAlarme retorna false quando parametro ok") {
00034     Parametro p(5.0, 10.0, 0.0, "Pressao", 5);
00035     Alarme a("Alarme Pressao", 5, p);
00036     CHECK(a.AtualizarAlarme() == false);
00037 }
00038
00039 TEST_CASE("Alarme - AtualizarAlarme sincroniza estado interno") {
00040     Parametro p(15.0, 10.0, 0.0, "Vibr", 6);
00041     p.DiagnosticarParametro();
00042     Alarme a("Alarme Vibr", 6, p);
00043     a.AtualizarAlarme();
00044     CHECK(a.GetEstado() == true);
00045 }
00046
00047 TEST_CASE("Alarme - SetDesc nao quebra estado") {
00048     Parametro p(5.0, 10.0, 0.0, "Temp", 7);
00049     Alarme a("Antiga", 7, p);
00050     a.SetDesc("Nova descricao");
00051     CHECK(a.GetEstado() == false);
00052 }

```

12.68 tests/test_Conjunto.cpp File Reference

```

#include "doctest.h"
#include "Conjunto.hpp"
#include "Equipamento.hpp"
#include "Excecoes.hpp"
#include <iostream>
#include <sstream>
#include <memory>

```

Functions

- [TEST_CASE](#) ("Conjunto - GetId retorna id correto")
- [TEST_CASE](#) ("Conjunto - GetDesc retorna descricao correta")
- [TEST_CASE](#) ("Conjunto - QuantidadeEquipamentos inicia em zero")
- [TEST_CASE](#) ("Conjunto - AdicionarEquipamento incrementa quantidade")
- [TEST_CASE](#) ("Conjunto - AdicionarEquipamento multiplos")
- [TEST_CASE](#) ("Conjunto - RemoverEquipamento decrementa quantidade")
- [TEST_CASE](#) ("Conjunto - SetNome atualiza nome")
- [TEST_CASE](#) ("Conjunto - SetDesc atualiza descricao")
- [TEST_CASE](#) ("Conjunto - DiagnosticarConjunto vazio sem falhas")
- [TEST_CASE](#) ("Conjunto - GetEquipamentos retorna vetor correto")
- [TEST_CASE](#) ("Conjunto - construtor rejeita id negativo")
- [TEST_CASE](#) ("Conjunto - construtor rejeita descricao vazia")
- [TEST_CASE](#) ("Conjunto - AdicionarEquipamento(unique_ptr) rejeita nullptr")
- [TEST_CASE](#) ("Conjunto - AdicionarEquipamento(unique_ptr) rejeita duplicado")
- [TEST_CASE](#) ("Conjunto - AdicionarEquipamento(int,desc) rejeita duplicado")
- [TEST_CASE](#) ("Conjunto - ProximoldEquipamento avanca a cada insercao")
- [TEST_CASE](#) ("Conjunto - RemoverEquipamento inexistente lanca")
- [TEST_CASE](#) ("Conjunto - SetDesc rejeita vazia")
- [TEST_CASE](#) ("Conjunto - AcessarEquipamento OK e erro")
- [TEST_CASE](#) ("Conjunto - ExibirEquipamentos vazio mostra mensagem")
- [TEST_CASE](#) ("Conjunto - ExibirEquipamentos com itens lista nome")
- [TEST_CASE](#) ("Conjunto - ExibirAlarmes vazio e cheio")
- [TEST_CASE](#) ("Conjunto - ExibirTudo executa")
- [TEST_CASE](#) ("Conjunto - QuantidadeAlarmes soma equipamentos")
- [TEST_CASE](#) ("Conjunto - GetEquipamentos const ref")

12.68.1 Function Documentation

12.68.1.1 TEST_CASE() [1/25]

```
TEST_CASE (
    "Conjunto - AcessarEquipamento OK e erro" )
```

Definition at line 126 of file [test_Conjunto.cpp](#).

12.68.1.2 TEST_CASE() [2/25]

```
TEST_CASE (
    "Conjunto - AdicionarEquipamento incrementa quantidade" )
```

Definition at line 36 of file [test_Conjunto.cpp](#).

12.68.1.3 TEST_CASE() [3/25]

```
TEST_CASE (
    "Conjunto - AdicionarEquipamento multiplos" )
```

Definition at line 42 of file [test_Conjunto.cpp](#).

12.68.1.4 TEST_CASE() [4/25]

```
TEST_CASE (
    "Conjunto - AdicionarEquipamento(int,desc) rejeita duplicado" )
```

Definition at line 101 of file [test_Conjunto.cpp](#).

12.68.1.5 TEST_CASE() [5/25]

```
TEST_CASE (
    "Conjunto - AdicionarEquipamento(unique_ptr) rejeita duplicado" )
```

Definition at line 94 of file [test_Conjunto.cpp](#).

12.68.1.6 TEST_CASE() [6/25]

```
TEST_CASE (
    "Conjunto - AdicionarEquipamento(unique_ptr) rejeita nullptr" )
```

Definition at line 88 of file [test_Conjunto.cpp](#).

12.68.1.7 TEST_CASE() [7/25]

```
TEST_CASE (
    "Conjunto - construtor rejeita descricao vazia" )
```

Definition at line 84 of file [test_Conjunto.cpp](#).

12.68.1.8 TEST_CASE() [8/25]

```
TEST_CASE (
    "Conjunto - construtor rejeita id negativo" )
```

Definition at line 80 of file [test_Conjunto.cpp](#).

12.68.1.9 TEST_CASE() [9/25]

```
TEST_CASE (
    "Conjunto - DiagnosticarConjunto vazio sem falhas" )
```

Definition at line 68 of file [test_Conjunto.cpp](#).

12.68.1.10 TEST_CASE() [10/25]

```
TEST_CASE (
    "Conjunto - ExibirAlarmes vazio e cheio" )
```

Definition at line 146 of file [test_Conjunto.cpp](#).

12.68.1.11 TEST_CASE() [11/25]

```
TEST_CASE (
    "Conjunto - ExibirEquipamentos com itens lista nome" )
```

Definition at line 139 of file [test_Conjunto.cpp](#).

12.68.1.12 TEST_CASE() [12/25]

```
TEST_CASE (
    "Conjunto - ExibirEquipamentos vazio mostra mensagem" )
```

Definition at line 133 of file [test_Conjunto.cpp](#).

12.68.1.13 TEST_CASE() [13/25]

```
TEST_CASE (
    "Conjunto - ExibirTudo executa" )
```

Definition at line 156 of file [test_Conjunto.cpp](#).

12.68.1.14 TEST_CASE() [14/25]

```
TEST_CASE (
    "Conjunto - GetDesc retorna descricao correta" )
```

Definition at line 26 of file [test_Conjunto.cpp](#).

12.68.1.15 TEST_CASE() [15/25]

```
TEST_CASE (
    "Conjunto - GetEquipamentos const ref" )
```

Definition at line 170 of file [test_Conjunto.cpp](#).

12.68.1.16 TEST_CASE() [16/25]

```
TEST_CASE (
    "Conjunto - GetEquipamentos retorna vetor correto" )
```

Definition at line 74 of file [test_Conjunto.cpp](#).

12.68.1.17 TEST_CASE() [17/25]

```
TEST_CASE (
    "Conjunto - GetId retorna id correto" )
```

Definition at line 21 of file [test_Conjunto.cpp](#).

12.68.1.18 TEST_CASE() [18/25]

```
TEST_CASE (
    "Conjunto - ProximoIdEquipamento avanca a cada insercao" )
```

Definition at line 107 of file [test_Conjunto.cpp](#).

12.68.1.19 TEST_CASE() [19/25]

```
TEST_CASE (
    "Conjunto - QuantidadeAlarmes soma equipamentos" )
```

Definition at line 162 of file [test_Conjunto.cpp](#).

12.68.1.20 TEST_CASE() [20/25]

```
TEST_CASE (
    "Conjunto - QuantidadeEquipamentos inicia em zero" )
```

Definition at line 31 of file [test_Conjunto.cpp](#).

12.68.1.21 TEST_CASE() [21/25]

```
TEST_CASE (
    "Conjunto - RemoverEquipamento decrementa quantidade" )
```

Definition at line 49 of file [test_Conjunto.cpp](#).

12.68.1.22 TEST_CASE() [22/25]

```
TEST_CASE (
    "Conjunto - RemoverEquipamento inexistente lanca" )
```

Definition at line 116 of file [test_Conjunto.cpp](#).

12.68.1.23 TEST_CASE() [23/25]

```
TEST_CASE (
    "Conjunto - SetDesc atualiza descricao" )
```

Definition at line 62 of file [test_Conjunto.cpp](#).

12.68.1.24 TEST_CASE() [24/25]

```
TEST_CASE (
    "Conjunto - SetDesc rejeita vazia" )
```

Definition at line 121 of file [test_Conjunto.cpp](#).

12.68.1.25 TEST_CASE() [25/25]

```
TEST_CASE (
    "Conjunto - SetNome atualiza nome" )
```

Definition at line 56 of file [test_Conjunto.cpp](#).

12.69 test_Conjunto.cpp

[Go to the documentation of this file.](#)

```

00001 #include "doctest.h"
00002 #include "Conjunto.hpp"
00003 #include "Equipamento.hpp"
00004 #include "Excecoes.hpp"
00005
00006 #include <iostream>
00007 #include <sstream>
00008 #include <memory>
00009
00010 namespace {
00011 template <typename F>
00012 std::string capturaSaida(F&& f) {
00013     std::stringstream buf;
00014     auto* old = std::cout.rdbuf(buf.rdbuf());
00015     f();
00016     std::cout.rdbuf(old);
00017     return buf.str();
00018 }
00019 }
00020
00021 TEST_CASE("Conjunto - GetId retorna id correto") {
00022     Conjunto c(1, "Conjunto A", "Conjunto A");
00023     CHECK(c.GetId() == 1);
00024 }
00025
00026 TEST_CASE("Conjunto - GetDesc retorna descricao correta") {
00027     Conjunto c(2, "Conjunto B", "Linha Principal");
00028     CHECK(c.GetDesc() == "Linha Principal");
00029 }
00030
00031 TEST_CASE("Conjunto - QuantidadeEquipamentos inicia em zero") {
00032     Conjunto c(3, "Setor B", "Setor B");
00033     CHECK(c.QuantidadeEquipamentos() == 0);
00034 }
00035
00036 TEST_CASE("Conjunto - AdicionarEquipamento incrementa quantidade") {
00037     Conjunto c(4, "Setor C", "Setor C");
00038     c.AdicionarEquipamento(1, "Motor");
00039     CHECK(c.QuantidadeEquipamentos() == 1);
00040 }
00041
00042 TEST_CASE("Conjunto - AdicionarEquipamento multiplos") {
00043     Conjunto c(5, "Setor D", "Setor D");
00044     c.AdicionarEquipamento(1, "Motor");
00045     c.AdicionarEquipamento(2, "Bomba");
00046     CHECK(c.QuantidadeEquipamentos() == 2);
00047 }
00048
00049 TEST_CASE("Conjunto - RemoverEquipamento decrementa quantidade") {
00050     Conjunto c(6, "Setor E", "Setor E");
00051     c.AdicionarEquipamento(1, "Motor");
00052     c.RemoverEquipamento(1);
00053     CHECK(c.QuantidadeEquipamentos() == 0);
00054 }
00055
00056 TEST_CASE("Conjunto - SetNome atualiza nome") {
00057     Conjunto c(7, "Nome", "Desc");
00058     c.SetNome("ConjuntoAlfa");
00059     CHECK(c.GetNome() == "ConjuntoAlfa");
00060 }
00061
00062 TEST_CASE("Conjunto - SetDesc atualiza descricao") {
00063     Conjunto c(8, "Nome", "Antiga");
00064     c.SetDesc("Nova");
00065     CHECK(c.GetDesc() == "Nova");
00066 }
00067
00068 TEST_CASE("Conjunto - DiagnosticarConjunto vazio sem falhas") {
00069     Conjunto c(9, "Setor F", "Setor F");
00070     std::vector<int> falhas = c.DiagnosticarConjunto();
00071     CHECK(falhas.empty() == true);
00072 }
00073
00074 TEST_CASE("Conjunto - GetEquipamentos retorna vetor correto") {
00075     Conjunto c(10, "Setor G", "Setor G");
00076     c.AdicionarEquipamento(1, "Reator");
00077     CHECK(c.GetEquipamentos().size() == 1);
00078 }
00079
00080 TEST_CASE("Conjunto - construtor rejeita id negativo") {
00081     CHECK_THROWS_AS(Conjunto(-1, "n", "d"), IdInvalido);
00082 }

```

```

00083
00084 TEST_CASE("Conjunto - construtor rejeita descricao vazia") {
00085     CHECK_THROWS_AS(Conjunto(1, "n", ""), EntradaInvalida);
00086 }
00087
00088 TEST_CASE("Conjunto - AdicionarEquipamento(unique_ptr) rejeita nullptr") {
00089     Conjunto c(11, "n", "d");
00090     CHECK_THROWS_AS(c.AdicionarEquipamento(std::unique_ptr<Equipamento>(nullptr)),
00091                     EntradaInvalida);
00092 }
00093
00094 TEST_CASE("Conjunto - AdicionarEquipamento(unique_ptr) rejeita duplicado") {
00095     Conjunto c(12, "n", "d");
00096     c.AdicionarEquipamento(1, "eq");
00097     auto novo = std::make_unique<Equipamento>(1, "outro", "outro");
00098     CHECK_THROWS_AS(c.AdicionarEquipamento(std::move(novo), IdDuplicado);
00099 }
00100
00101 TEST_CASE("Conjunto - AdicionarEquipamento(int,desc) rejeita duplicado") {
00102     Conjunto c(13, "n", "d");
00103     c.AdicionarEquipamento(1, "eq");
00104     CHECK_THROWS_AS(c.AdicionarEquipamento(1, "eq2"), IdDuplicado);
00105 }
00106
00107 TEST_CASE("Conjunto - ProximoIdEquipamento avanca a cada insercao") {
00108     Conjunto c(14, "n", "d");
00109     CHECK(c.ProximoIdEquipamento() == 1);
00110     c.AdicionarEquipamento(1, "eq");
00111     CHECK(c.ProximoIdEquipamento() == 2);
00112     c.AdicionarEquipamento(std::make_unique<Equipamento>(5, "eq", "eq"));
00113     CHECK(c.ProximoIdEquipamento() == 6);
00114 }
00115
00116 TEST_CASE("Conjunto - RemoverEquipamento inexistente lanca") {
00117     Conjunto c(15, "n", "d");
00118     CHECK_THROWS_AS(c.RemoverEquipamento(99), IdInexistente);
00119 }
00120
00121 TEST_CASE("Conjunto - SetDesc rejeita vazia") {
00122     Conjunto c(16, "n", "d");
00123     CHECK_THROWS_AS(c.SetDesc(""), EntradaInvalida);
00124 }
00125
00126 TEST_CASE("Conjunto - AcessarEquipamento OK e erro") {
00127     Conjunto c(17, "n", "d");
00128     c.AdicionarEquipamento(1, "eq");
00129     CHECK(c.AcessarEquipamento(1).GetId() == 1);
00130     CHECK_THROWS_AS(c.AcessarEquipamento(99), IdInexistente);
00131 }
00132
00133 TEST_CASE("Conjunto - ExibirEquipamentos vazio mostra mensagem") {
00134     Conjunto c(18, "n", "d");
00135     std::string s = capturaSaida([&]{ c.ExibirEquipamentos(); });
00136     CHECK(s.find("Nenhum") != std::string::npos);
00137 }
00138
00139 TEST_CASE("Conjunto - ExibirEquipamentos com itens lista nome") {
00140     Conjunto c(19, "Bloco", "d");
00141     c.AdicionarEquipamento(1, "eq");
00142     std::string s = capturaSaida([&]{ c.ExibirEquipamentos(); });
00143     CHECK(s.find("Bloco") != std::string::npos);
00144 }
00145
00146 TEST_CASE("Conjunto - ExibirAlarmes vazio e cheio") {
00147     Conjunto c1(20, "n", "d");
00148     std::string s1 = capturaSaida([&]{ c1.ExibirAlarmes(); });
00149     CHECK(s1.find("Nenhum") != std::string::npos);
00150
00151     Conjunto c2(21, "n", "d");
00152     c2.AdicionarEquipamento(1, "eq");
00153     capturaSaida([&]{ c2.ExibirAlarmes(); });
00154 }
00155
00156 TEST_CASE("Conjunto - ExibirTudo executa") {
00157     Conjunto c(22, "n", "d");
00158     c.AdicionarEquipamento(1, "eq");
00159     capturaSaida([&]{ c.ExibirTudo(); });
00160 }
00161
00162 TEST_CASE("Conjunto - QuantidadeAlarmes soma equipamentos") {
00163     Conjunto c(23, "n", "d");
00164     c.AdicionarEquipamento(1, "eq");
00165     c.AcessarEquipamento(1).AdicionarParametro(15.0, 10.0, 0.0, "P", 1);
00166     c.AtualizarAlarmes();
00167     CHECK(c.QuantidadeAlarmes() == 1);
00168 }
00169

```

```

00170 TEST_CASE("Conjunto - GetEquipamentos const ref") {
00171     Conjunto c(24, "n", "d");
00172     c.AdicionarEquipamento(1, "eq");
00173     const auto& m = c.GetEquipamentos();
00174     CHECK(m.size() == 1);
00175 }

```

12.70 tests/test_Equipamento.cpp File Reference

```

#include "doctest.h"
#include "Equipamento.hpp"
#include "Excecoes.hpp"
#include <iostream>
#include <sstream>

```

Functions

- [TEST_CASE](#) ("Equipamento - GetId retorna id correto")
- [TEST_CASE](#) ("Equipamento - QuantidadeParametros inicia em zero")
- [TEST_CASE](#) ("Equipamento - AdicionarParametro incrementa quantidade")
- [TEST_CASE](#) ("Equipamento - AdicionarParametro multiplos")
- [TEST_CASE](#) ("Equipamento - RemoverParametro decrementa quantidade")
- [TEST_CASE](#) ("Equipamento - AdicionarParametroComLimites incrementa quantidade")
- [TEST_CASE](#) ("Equipamento - DiagnosticarEquipamento vazio quando sem falhas")
- [TEST_CASE](#) ("Equipamento - GetParametros retorna vetor correto")
- [TEST_CASE](#) ("Equipamento - QuantidadeAlarmes inicia em zero")
- [TEST_CASE](#) ("Equipamento - construtor rejeita id negativo")
- [TEST_CASE](#) ("Equipamento - AdicionarParametro 2-arg auto-id retorna ids monotonicos")
- [TEST_CASE](#) ("Equipamento - AdicionarParametro 4-arg auto-id retorna ids monotonicos")
- [TEST_CASE](#) ("Equipamento - AdicionarParametro 4-arg explicito rejeita duplicado")
- [TEST_CASE](#) ("Equipamento - AdicionarParametro 3-arg rejeita duplicado")
- [TEST_CASE](#) ("Equipamento - RemoverParametro inexistente lanca")
- [TEST_CASE](#) ("Equipamento - AtualizarAlarmes sincroniza vetor de alarmes")
- [TEST_CASE](#) ("Equipamento - ExibirParametros executa")
- [TEST_CASE](#) ("Equipamento - ExibirAlarmes executa")
- [TEST_CASE](#) ("Equipamento - SetDesc atualiza descricao")
- [TEST_CASE](#) ("Equipamento - Tipo base retorna 'Equipamento'")

12.70.1 Function Documentation

12.70.1.1 TEST_CASE() [1/20]

```

TEST_CASE (
    "Equipamento - AdicionarParametro 2-arg auto-id retorna ids monotonicos" )

```

Definition at line 77 of file [test_Equipamento.cpp](#).

12.70.1.2 TEST_CASE() [2/20]

```
TEST_CASE (
    "Equipamento - AdicionarParametro 3-arg rejeita duplicado" )
```

Definition at line 100 of file [test_Equipamento.cpp](#).

12.70.1.3 TEST_CASE() [3/20]

```
TEST_CASE (
    "Equipamento - AdicionarParametro 4-arg auto-id retorna ids monotonicos" )
```

Definition at line 86 of file [test_Equipamento.cpp](#).

12.70.1.4 TEST_CASE() [4/20]

```
TEST_CASE (
    "Equipamento - AdicionarParametro 4-arg explicito rejeita duplicado" )
```

Definition at line 94 of file [test_Equipamento.cpp](#).

12.70.1.5 TEST_CASE() [5/20]

```
TEST_CASE (
    "Equipamento - AdicionarParametro incrementa quantidade" )
```

Definition at line 29 of file [test_Equipamento.cpp](#).

12.70.1.6 TEST_CASE() [6/20]

```
TEST_CASE (
    "Equipamento - AdicionarParametro multiplos" )
```

Definition at line 35 of file [test_Equipamento.cpp](#).

12.70.1.7 TEST_CASE() [7/20]

```
TEST_CASE (
    "Equipamento - AdicionarParametroComLimites incrementa quantidade" )
```

Definition at line 49 of file [test_Equipamento.cpp](#).

12.70.1.8 TEST_CASE() [8/20]

```
TEST_CASE (
    "Equipamento - AtualizarAlarmes sincroniza vetor de alarmes" )
```

Definition at line 111 of file [test_Equipamento.cpp](#).

12.70.1.9 TEST_CASE() [9/20]

```
TEST_CASE (
    "Equipamento - construtor rejeita id negativo" )
```

Definition at line 73 of file [test_Equipamento.cpp](#).

12.70.1.10 TEST_CASE() [10/20]

```
TEST_CASE (
    "Equipamento - DiagnosticarEquipamento vazio quando sem falhas" )
```

Definition at line 55 of file [test_Equipamento.cpp](#).

12.70.1.11 TEST_CASE() [11/20]

```
TEST_CASE (
    "Equipamento - ExibirAlarmes executa" )
```

Definition at line 126 of file [test_Equipamento.cpp](#).

12.70.1.12 TEST_CASE() [12/20]

```
TEST_CASE (
    "Equipamento - ExibirParametros executa" )
```

Definition at line 119 of file [test_Equipamento.cpp](#).

12.70.1.13 TEST_CASE() [13/20]

```
TEST_CASE (
    "Equipamento - GetId retorna id correto" )
```

Definition at line 19 of file [test_Equipamento.cpp](#).

12.70.1.14 TEST_CASE() [14/20]

```
TEST_CASE (
    "Equipamento - GetParametros retorna vetor correto" )
```

Definition at line 62 of file [test_Equipamento.cpp](#).

12.70.1.15 TEST_CASE() [15/20]

```
TEST_CASE (
    "Equipamento - QuantidadeAlarmes inicia em zero" )
```

Definition at line 68 of file [test_Equipamento.cpp](#).

12.70.1.16 TEST_CASE() [16/20]

```
TEST_CASE (
    "Equipamento - QuantidadeParametros inicia em zero" )
```

Definition at line 24 of file [test_Equipamento.cpp](#).

12.70.1.17 TEST_CASE() [17/20]

```
TEST_CASE (
    "Equipamento - RemoverParametro decrementa quantidade" )
```

Definition at line 42 of file [test_Equipamento.cpp](#).

12.70.1.18 TEST_CASE() [18/20]

```
TEST_CASE (
    "Equipamento - RemoverParametro inexistente lanca" )
```

Definition at line 106 of file [test_Equipamento.cpp](#).

12.70.1.19 TEST_CASE() [19/20]

```
TEST_CASE (
    "Equipamento - SetDesc atualiza descricao" )
```

Definition at line 134 of file [test_Equipamento.cpp](#).

12.70.1.20 TEST_CASE() [20/20]

```
TEST_CASE (
    "Equipamento - Tipo base retorna 'Equipamento'" )
```

Definition at line 140 of file [test_Equipamento.cpp](#).

12.71 test_Equipamento.cpp

[Go to the documentation of this file.](#)

```

00001 #include "doctest.h"
00002 #include "Equipamento.hpp"
00003 #include "Excecoes.hpp"
00004
00005 #include <iostream>
00006 #include <sstream>
00007
00008 namespace {
00009 template <typename F>
00010 std::string capturaSaida(F&& f) {
00011     std::stringstream buf;
00012     auto* old = std::cout.rdbuf(buf.rdbuf());
00013     f();
00014     std::cout.rdbuf(old);
00015     return buf.str();
00016 }
00017 }
00018
00019 TEST_CASE("Equipamento - GetId retorna id correto") {
00020     Equipamento e(1, "Motor A", "Motor A");
00021     CHECK(e.GetId() == 1);
00022 }
00023
00024 TEST_CASE("Equipamento - QuantidadeParametros inicia em zero") {
00025     Equipamento e(2, "Bomba", "Bomba");
00026     CHECK(e.QuantidadeParametros() == 0);
00027 }
00028
00029 TEST_CASE("Equipamento - AdicionarParametro incrementa quantidade") {
00030     Equipamento e(3, "Compressor", "Compressor");
00031     e.AdicionarParametro(10.0, "Temperatura", 1);
00032     CHECK(e.QuantidadeParametros() == 1);
00033 }
00034
00035 TEST_CASE("Equipamento - AdicionarParametro multiplos") {
00036     Equipamento e(4, "Turbina", "Turbina");
00037     e.AdicionarParametro(10.0, "Temperatura", 1);
00038     e.AdicionarParametro(5.0, "Pressao", 2);
00039     CHECK(e.QuantidadeParametros() == 2);
00040 }
00041
00042 TEST_CASE("Equipamento - RemoverParametro decrementa quantidade") {
00043     Equipamento e(5, "Valvula", "Valvula");
00044     e.AdicionarParametro(10.0, "Temp", 1);
00045     e.RemoverParametro(1);
00046     CHECK(e.QuantidadeParametros() == 0);
00047 }
00048
00049 TEST_CASE("Equipamento - AdicionarParametroComLimites incrementa quantidade") {
00050     Equipamento e(6, "Sensor", "Sensor");
00051     e.AdicionarParametro(5.0, 10.0, 0.0, "Nivel", 1);
00052     CHECK(e.QuantidadeParametros() == 1);
00053 }
00054
00055 TEST_CASE("Equipamento - DiagnosticarEquipamento vazio quando sem falhas") {
00056     Equipamento e(7, "Reator", "Reator");
00057     e.AdicionarParametro(5.0, 10.0, 0.0, "Temp", 1);
00058     std::vector<int> falhas = e.DiagnosticarEquipamento();
00059     CHECK(falhas.empty() == true);
00060 }
00061
00062 TEST_CASE("Equipamento - GetParametros retorna vetor correto") {
00063     Equipamento e(8, "Eixo", "Eixo");
00064     e.AdicionarParametro(1.0, "Velocidade", 10);
00065     CHECK(e.GetParametros().size() == 1);
00066 }
00067
00068 TEST_CASE("Equipamento - QuantidadeAlarmes inicia em zero") {
00069     Equipamento e(9, "CLP", "CLP");
00070     CHECK(e.QuantidadeAlarmes() == 0);
00071 }
00072
00073 TEST_CASE("Equipamento - construtor rejeita id negativo") {
00074     CHECK_THROWS_AS(Equipamento(-1, "d", "n"), IdInvalido);
00075 }
00076
00077 TEST_CASE("Equipamento - AdicionarParametro 2-arg auto-id retorna ids monotonicos") {
00078     Equipamento e(10, "d", "n");
00079     int a = e.AdicionarParametro(1.0, "P1");
00080     int b = e.AdicionarParametro(2.0, "P2");
00081     CHECK(a == 1);
00082     CHECK(b == 2);

```

```

00083     CHECK(e.ProximoIdParametro() == 3);
00084 }
00085
00086 TEST_CASE("Equipamento - AdicionarParametro 4-arg auto-id retorna ids monotonicos") {
00087     Equipamento e(11, "d", "n");
00088     int a = e.AdicionarParametro(1.0, 10.0, 0.0, "P1");
00089     int b = e.AdicionarParametro(2.0, 10.0, 0.0, "P2");
00090     CHECK(a == 1);
00091     CHECK(b == 2);
00092 }
00093
00094 TEST_CASE("Equipamento - AdicionarParametro 4-arg explicito rejeita duplicado") {
00095     Equipamento e(12, "d", "n");
00096     e.AdicionarParametro(1.0, 10.0, 0.0, "P", 5);
00097     CHECK_THROWS_AS(e.AdicionarParametro(2.0, 10.0, 0.0, "P", 5), IdDuplicado);
00098 }
00099
00100 TEST_CASE("Equipamento - AdicionarParametro 3-arg rejeita duplicado") {
00101     Equipamento e(13, "d", "n");
00102     e.AdicionarParametro(1.0, "P", 1);
00103     CHECK_THROWS_AS(e.AdicionarParametro(2.0, "P", 1), IdDuplicado);
00104 }
00105
00106 TEST_CASE("Equipamento - RemoverParametro inexistente lanca") {
00107     Equipamento e(14, "d", "n");
00108     CHECK_THROWS_AS(e.RemoverParametro(99), IdInexistente);
00109 }
00110
00111 TEST_CASE("Equipamento - AtualizarAlarmes sincroniza vetor de alarmes") {
00112     Equipamento e(15, "d", "n");
00113     e.AdicionarParametro(15.0, 10.0, 0.0, "P", 1);
00114     e.AtualizarAlarmes();
00115     CHECK(e.QuantidadeAlarmes() == 1);
00116     CHECK(e.GetAlarmes().size() == 1);
00117 }
00118
00119 TEST_CASE("Equipamento - ExibirParametros executa") {
00120     Equipamento e(16, "d", "n");
00121     e.AdicionarParametro(1.0, "P", 1);
00122     std::string s = capturaSaida([&]{ e.ExibirParametros(); });
00123     CHECK(!s.empty());
00124 }
00125
00126 TEST_CASE("Equipamento - ExibirAlarmes executa") {
00127     Equipamento e(17, "d", "n");
00128     e.AdicionarParametro(1.0, 10.0, 0.0, "P", 1);
00129     e.AtualizarAlarmes();
00130     std::string s = capturaSaida([&]{ e.ExibirAlarmes(); });
00131     CHECK(!s.empty());
00132 }
00133
00134 TEST_CASE("Equipamento - SetDesc atualiza descricao") {
00135     Equipamento e(18, "d", "n");
00136     e.SetDesc("nova");
00137     CHECK(e.GetDesc() == "nova");
00138 }
00139
00140 TEST_CASE("Equipamento - Tipo base retorna 'Equipamento'") {
00141     Equipamento e(19, "d", "n");
00142     CHECK(e.Tipo() == "Equipamento");
00143 }

```

12.72 tests/test_Excecoes.cpp File Reference

```

#include "doctest.h"
#include "Excecoes.hpp"
#include "Parametro.hpp"
#include "Equipamento.hpp"
#include "Conjunto.hpp"
#include "LinhaDeProducao.hpp"
#include "Sistema.hpp"
#include "Usuario.hpp"
#include "Cargo.hpp"
#include "Motor.hpp"
#include "Sensor.hpp"
#include "Valvula.hpp"

```



```
#include "Atuador.hpp"
#include <memory>
```

Functions

- [TEST_CASE](#) ("Parametro - Max < Min lanca LimitesInvalidos")
- [TEST_CASE](#) ("Parametro - descricao vazia lanca EntradaInvalida")
- [TEST_CASE](#) ("Parametro - id negativo lanca IdInvalido")
- [TEST_CASE](#) ("Parametro - leitura fora dos limites NAO lanca (caso de falha)")
- [TEST_CASE](#) ("Parametro - SetMin maior que Max lanca LimitesInvalidos")
- [TEST_CASE](#) ("Equipamento - id negativo lanca IdInvalido")
- [TEST_CASE](#) ("Equipamento - descricao vazia lanca EntradaInvalida")
- [TEST_CASE](#) ("Equipamento - parametro duplicado lanca IdDuplicado")
- [TEST_CASE](#) ("Equipamento - remover parametro inexistente lanca IdInexistente")
- [TEST_CASE](#) ("Conjunto - equipamento duplicado lanca IdDuplicado")
- [TEST_CASE](#) ("Conjunto - acessar equipamento inexistente lanca IdInexistente")
- [TEST_CASE](#) ("LinhaDeProducao - conjunto duplicado lanca IdDuplicado")
- [TEST_CASE](#) ("Usuario - login vazio lanca EntradaInvalida")
- [TEST_CASE](#) ("Sistema - linha duplicada lanca IdDuplicado")
- [TEST_CASE](#) ("Sistema - remover linha inexistente lanca IdInexistente")
- [TEST_CASE](#) ("Excecoes - derivadas capturaveis como ExcecaoSistema")
- [TEST_CASE](#) ("Polimorfismo - Tipo() despacha para a classe concreta")
- [TEST_CASE](#) ("Polimorfismo - [Motor](#) cria seus parametros")
- [TEST_CASE](#) ("Conjunto - aceita equipamento derivado via unique_ptr")
- [TEST_CASE](#) ("Sistema - salvar e carregar restaura estrutura")

12.72.1 Function Documentation

12.72.1.1 [TEST_CASE\(\)](#) [1/20]

```
TEST_CASE (
    "Conjunto - aceita equipamento derivado via unique_ptr" )
```

Definition at line 115 of file [test_Excecoes.cpp](#).

12.72.1.2 [TEST_CASE\(\)](#) [2/20]

```
TEST_CASE (
    "Conjunto - acessar equipamento inexistente lanca IdInexistente" )
```

Definition at line 66 of file [test_Excecoes.cpp](#).

12.72.1.3 [TEST_CASE\(\)](#) [3/20]

```
TEST_CASE (
    "Conjunto - equipamento duplicado lanca IdDuplicado" )
```

Definition at line 60 of file [test_Excecoes.cpp](#).

12.72.1.4 TEST_CASE() [4/20]

```
TEST_CASE (
    "Equipamento - descricao vazia lanca EntradaInvalida" )
```

Definition at line 45 of file [test_Excecoes.cpp](#).

12.72.1.5 TEST_CASE() [5/20]

```
TEST_CASE (
    "Equipamento - id negativo lanca IdInvalido" )
```

Definition at line 41 of file [test_Excecoes.cpp](#).

12.72.1.6 TEST_CASE() [6/20]

```
TEST_CASE (
    "Equipamento - parametro duplicado lanca IdDuplicado" )
```

Definition at line 49 of file [test_Excecoes.cpp](#).

12.72.1.7 TEST_CASE() [7/20]

```
TEST_CASE (
    "Equipamento - remover parametro inexistente lanca IdInexistente" )
```

Definition at line 55 of file [test_Excecoes.cpp](#).

12.72.1.8 TEST_CASE() [8/20]

```
TEST_CASE (
    "Excecoes - derivadas capturaveis como ExcecaoSistema" )
```

Definition at line 92 of file [test_Excecoes.cpp](#).

12.72.1.9 TEST_CASE() [9/20]

```
TEST_CASE (
    "LinhaDeProducao - conjunto duplicado lanca IdDuplicado" )
```

Definition at line 71 of file [test_Excecoes.cpp](#).

12.72.1.10 TEST_CASE() [10/20]

```
TEST_CASE (
    "Parametro - descricao vazia lanca EntradaInvalida" )
```

Definition at line 24 of file [test_Excecoes.cpp](#).

12.72.1.11 TEST_CASE() [11/20]

```
TEST_CASE (
    "Parametro - id negativo lanca IdInvalido" )
```

Definition at line 28 of file [test_Excecoes.cpp](#).

12.72.1.12 TEST_CASE() [12/20]

```
TEST_CASE (
    "Parametro - leitura fora dos limites NAO lanca (caso de falha)" )
```

Definition at line 32 of file [test_Excecoes.cpp](#).

12.72.1.13 TEST_CASE() [13/20]

```
TEST_CASE ()
```

Definition at line 20 of file [test_Excecoes.cpp](#).

12.72.1.14 TEST_CASE() [14/20]

```
TEST_CASE (
    "Parametro - SetMin maior que Max lanca LimitesInvalidos" )
```

Definition at line 36 of file [test_Excecoes.cpp](#).

12.72.1.15 TEST_CASE() [15/20]

```
TEST_CASE (
    "Polimorfismo - Motor cria seus parametros" )
```

Definition at line 110 of file [test_Excecoes.cpp](#).

12.72.1.16 TEST_CASE() [16/20]

```
TEST_CASE (
    "Polimorfismo - Tipo() despacha para a classe concreta" )
```

Definition at line 98 of file [test_Excecoes.cpp](#).

12.72.1.17 TEST_CASE() [17/20]

```
TEST_CASE (
    "Sistema - linha duplicada lanca IdDuplicado" )
```

Definition at line 81 of file [test_Excecoes.cpp](#).

12.72.1.18 TEST_CASE() [18/20]

```
TEST_CASE (
    "Sistema - remover linha inexistente lanca IdInexistente" )
```

Definition at line 87 of file [test_Excecoes.cpp](#).

12.72.1.19 TEST_CASE() [19/20]

```
TEST_CASE (
    "Sistema - salvar e carregar restaura estrutura" )
```

Definition at line 124 of file [test_Excecoes.cpp](#).

12.72.1.20 TEST_CASE() [20/20]

```
TEST_CASE (
    "Usuario - login vazio lanca EntradaInvalida" )
```

Definition at line 77 of file [test_Excecoes.cpp](#).

12.73 test_Excecoes.cpp

[Go to the documentation of this file.](#)

```
00001 #include "doctest.h"
00002
00003 #include "Excecoes.hpp"
00004 #include "Parametro.hpp"
00005 #include "Equipamento.hpp"
00006 #include "Conjunto.hpp"
00007 #include "LinhaDeProducao.hpp"
00008 #include "Sistema.hpp"
00009 #include "Usuario.hpp"
00010 #include "Cargo.hpp"
00011 #include "Motor.hpp"
00012 #include "Sensor.hpp"
00013 #include "Valvula.hpp"
00014 #include "Atuador.hpp"
00015
00016 #include <memory>
00017
00018 // ----- Validação / exceções -----
00019
00020 TEST_CASE("Parametro - Max < Min lanca LimitesInvalidos") {
00021     CHECK_THROWS_AS(Parametro(5.0, 0.0, 10.0, "Temp", 1), LimitesInvalidos);
00022 }
00023
00024 TEST_CASE("Parametro - descricao vazia lanca EntradaInvalida") {
00025     CHECK_THROWS_AS(Parametro(5.0, "", 1), EntradaInvalida);
00026 }
00027
00028 TEST_CASE("Parametro - id negativo lanca IdInvalido") {
00029     CHECK_THROWS_AS(Parametro(5.0, "Temp", -1), IdInvalido);
00030 }
00031
00032 TEST_CASE("Parametro - leitura fora dos limites NAO lanca (caso de falha)") {
00033     CHECK_NOTHROW(Parametro(15.0, 10.0, 0.0, "Pressao", 1));
00034 }
00035
00036 TEST_CASE("Parametro - SetMin maior que Max lanca LimitesInvalidos") {
00037     Parametro p(5.0, 10.0, 0.0, "Temp", 1);
00038     CHECK_THROWS_AS(p.SetMin(20.0), LimitesInvalidos);
00039 }
00040
00041 TEST_CASE("Equipamento - id negativo lanca IdInvalido") {
```

```

00042     CHECK_THROWS_AS(Equipamento(-1, "Motor", "M"), IdInvalido);
00043 }
00044
00045 TEST_CASE("Equipamento - descricao vazia lanca EntradaInvalida") {
00046     CHECK_THROWS_AS(Equipamento(1, "", "M"), EntradaInvalida);
00047 }
00048
00049 TEST_CASE("Equipamento - parametro duplicado lanca IdDuplicado") {
00050     Equipamento e(1, "Motor", "M");
00051     e.AdicionarParametro(5.0, "Temp", 1);
00052     CHECK_THROWS_AS(e.AdicionarParametro(6.0, "Outra", 1), IdDuplicado);
00053 }
00054
00055 TEST_CASE("Equipamento - remover parametro inexistente lanca IdInexistente") {
00056     Equipamento e(1, "Motor", "M");
00057     CHECK_THROWS_AS(e.RemoverParametro(99), IdInexistente);
00058 }
00059
00060 TEST_CASE("Conjunto - equipamento duplicado lanca IdDuplicado") {
00061     Conjunto c(1, "Nome", "Desc");
00062     c.AdicionarEquipamento(1, "Motor");
00063     CHECK_THROWS_AS(c.AdicionarEquipamento(1, "Bomba"), IdDuplicado);
00064 }
00065
00066 TEST_CASE("Conjunto - acessar equipamento inexistente lanca IdInexistente") {
00067     Conjunto c(1, "Nome", "Desc");
00068     CHECK_THROWS_AS(c.AcessarEquipamento(42), IdInexistente);
00069 }
00070
00071 TEST_CASE("LinhaDeProducao - conjunto duplicado lanca IdDuplicado") {
00072     LinhaDeProducao l(1, "Desc", "Local", "Nome");
00073     l.AdicionarConjunto(1, "C1", "D1");
00074     CHECK_THROWS_AS(l.AdicionarConjunto(1, "C2", "D2"), IdDuplicado);
00075 }
00076
00077 TEST_CASE("Usuario - login vazio lanca EntradaInvalida") {
00078     CHECK_THROWS_AS(Usuario("", "senha", Cargo::ADMIN), EntradaInvalida);
00079 }
00080
00081 TEST_CASE("Sistema - linha duplicada lanca IdDuplicado") {
00082     Sistema s;
00083     s.AdicionarLinha(1, "Desc", "Local", "Nome");
00084     CHECK_THROWS_AS(s.AdicionarLinha(1, "Outra", "Local", "Nome"), IdDuplicado);
00085 }
00086
00087 TEST_CASE("Sistema - remover linha inexistente lanca IdInexistente") {
00088     Sistema s;
00089     CHECK_THROWS_AS(s.RemoverLinha(99), IdInexistente);
00090 }
00091
00092 TEST_CASE("Excecoes - derivadas capturaveis como ExcecaoSistema") {
00093     CHECK_THROWS_AS(Parametro(5.0, 0.0, 10.0, "Temp", 1), ExcecaoSistema);
00094 }
00095
00096 // ----- Polimorfismo via unique_ptr -----
00097
00098 TEST_CASE("Polimorfismo - Tipo() despacha para a classe concreta") {
00099     std::unique_ptr<Equipamento> motor = std::make_unique<Motor>(1, "M", 100.0, 50.0);
00100     std::unique_ptr<Equipamento> sensor = std::make_unique<Sensor>(2, "S", 1.0);
00101     std::unique_ptr<Equipamento> valvula = std::make_unique<Valvula>(3, "V", 2.0);
00102     std::unique_ptr<Equipamento> atuador = std::make_unique<Atuador>(4, "A", 3.0);
00103
00104     CHECK(motor->Tipo() == "Motor");
00105     CHECK(sensor->Tipo() == "Sensor");
00106     CHECK(valvula->Tipo() == "Valvula");
00107     CHECK(atuador->Tipo() == "Atuador");
00108 }
00109
00110 TEST_CASE("Polimorfismo - Motor cria seus parametros") {
00111     Motor m(1, "Motor principal", 100.0, 50.0);
00112     CHECK(m.QuantidadeParametros() == 2);
00113 }
00114
00115 TEST_CASE("Conjunto - aceita equipamento derivado via unique_ptr") {
00116     Conjunto c(1, "Nome", "Desc");
00117     c.AdicionarEquipamento(std::make_unique<Sensor>(5, "Sensor", 1.0));
00118     CHECK(c.QuantidadeEquipamentos() == 1);
00119     CHECK(c.AcessarEquipamento(5).Tipo() == "Sensor");
00120 }
00121
00122 // ----- Persistência (round-trip) -----
00123
00124 TEST_CASE("Sistema - salvar e carregar restaura estrutura") {
00125     Sistema s1;
00126     s1.AdicionarUsuario("admin", "123", Cargo::ADMIN);
00127     s1.AdicionarLinha(1, "Desc", "Local", "Linha");
00128     s1.AcessarLinha(1).AdicionarConjunto(1, "Conj", "DescC");

```

```

00129     s1.AcessarLinha(1).AcessarConjunto(1).AdicionarEquipamento(2, "Equip");
00130     s1.AcessarLinha(1).AcessarConjunto(1).AcessarEquipamento(2)
00131         .AdicionarParametro(15.0, 10.0, 0.0, "Pressao", 3);
00132     CHECK(s1.SalvarAlteracoes() == true);
00133
00134     Sistema s2;
00135     CHECK(s2.CarregarSave("save.txt") == true);
00136     CHECK(s2.GetLinhas().size() == 1);
00137     CHECK(s2.AcessarLinha(1).QuantidadeConjuntos() == 1);
00138     CHECK(s2.AcessarLinha(1).AcessarConjunto(1).QuantidadeEquipamentos() == 1);
00139 }

```

12.74 tests/test_LinhaDeProducao.cpp File Reference

```

#include "doctest.h"
#include "LinhaDeProducao.hpp"
#include "Excecoes.hpp"
#include <iostream>
#include <sstream>

```

Functions

- [TEST_CASE](#) ("LinhaDeProducao - GetId retorna id correto")
- [TEST_CASE](#) ("LinhaDeProducao - GetNome retorna nome correto")
- [TEST_CASE](#) ("LinhaDeProducao - GetLocal retorna local correto")
- [TEST_CASE](#) ("LinhaDeProducao - GetDesc retorna descricao correta")
- [TEST_CASE](#) ("LinhaDeProducao - QuantidadeConjuntos inicia em zero")
- [TEST_CASE](#) ("LinhaDeProducao - AdicionarConjunto incrementa quantidade")
- [TEST_CASE](#) ("LinhaDeProducao - AdicionarConjunto multiplos")
- [TEST_CASE](#) ("LinhaDeProducao - RemoverConjunto decrementa quantidade")
- [TEST_CASE](#) ("LinhaDeProducao - SetNome atualiza nome")
- [TEST_CASE](#) ("LinhaDeProducao - DiagnosticarLinha vazio sem falhas")
- [TEST_CASE](#) ("LinhaDeProducao - GetConjuntos retorna vetor correto")
- [TEST_CASE](#) ("LinhaDeProducao - construtor rejeita id negativo")
- [TEST_CASE](#) ("LinhaDeProducao - construtor rejeita descricao vazia")
- [TEST_CASE](#) ("LinhaDeProducao - AdicionarConjunto auto-id retorna ids monotonicos")
- [TEST_CASE](#) ("LinhaDeProducao - AdicionarConjunto explicito avanca Proximold")
- [TEST_CASE](#) ("LinhaDeProducao - RemoverConjunto inexistente lanca")
- [TEST_CASE](#) ("LinhaDeProducao - AcessarConjunto inexistente lanca")
- [TEST_CASE](#) ("LinhaDeProducao - AcessarConjunto retorna referencia mutavel")
- [TEST_CASE](#) ("LinhaDeProducao - ExibirConjuntos / ExibirAlarmes / ExibirTudo executam")
- [TEST_CASE](#) ("LinhaDeProducao - QuantidadeAlarmes soma conjuntos")
- [TEST_CASE](#) ("LinhaDeProducao - SetDesc atualiza descricao")
- [TEST_CASE](#) ("LinhaDeProducao - SetLocal atualiza local")
- [TEST_CASE](#) ("LinhaDeProducao - DiagnosticarLinha reporta parametro fora de faixa")

12.74.1 Function Documentation

12.74.1.1 TEST_CASE() [1/23]

```

TEST_CASE (
    "LinhaDeProducao - AcessarConjunto inexistente lanca" )

```

Definition at line 110 of file [test_LinhaDeProducao.cpp](#).

12.74.1.2 TEST_CASE() [2/23]

```
TEST_CASE (
    "LinhaDeProducao - AcessarConjunto retorna referencia mutavel" )
```

Definition at line 115 of file [test_LinhaDeProducao.cpp](#).

12.74.1.3 TEST_CASE() [3/23]

```
TEST_CASE (
    "LinhaDeProducao - AdicionarConjunto auto-id retorna ids monotonicos" )
```

Definition at line 90 of file [test_LinhaDeProducao.cpp](#).

12.74.1.4 TEST_CASE() [4/23]

```
TEST_CASE (
    "LinhaDeProducao - AdicionarConjunto explicito avanca ProximoId" )
```

Definition at line 99 of file [test_LinhaDeProducao.cpp](#).

12.74.1.5 TEST_CASE() [5/23]

```
TEST_CASE (
    "LinhaDeProducao - AdicionarConjunto incrementa quantidade" )
```

Definition at line 44 of file [test_LinhaDeProducao.cpp](#).

12.74.1.6 TEST_CASE() [6/23]

```
TEST_CASE (
    "LinhaDeProducao - AdicionarConjunto multiplos" )
```

Definition at line 50 of file [test_LinhaDeProducao.cpp](#).

12.74.1.7 TEST_CASE() [7/23]

```
TEST_CASE (
    "LinhaDeProducao - construtor rejeita descricao vazia" )
```

Definition at line 86 of file [test_LinhaDeProducao.cpp](#).

12.74.1.8 TEST_CASE() [8/23]

```
TEST_CASE (
    "LinhaDeProducao - construtor rejeita id negativo" )
```

Definition at line 82 of file [test_LinhaDeProducao.cpp](#).

12.74.1.9 TEST_CASE() [9/23]

```
TEST_CASE (
    "LinhaDeProducao - DiagnosticarLinha reporta parametro fora de faixa" )
```

Definition at line 156 of file [test_LinhaDeProducao.cpp](#).

12.74.1.10 TEST_CASE() [10/23]

```
TEST_CASE (
    "LinhaDeProducao - DiagnosticarLinha vazio sem falhas" )
```

Definition at line 70 of file [test_LinhaDeProducao.cpp](#).

12.74.1.11 TEST_CASE() [11/23]

```
TEST_CASE (
    "LinhaDeProducao - ExibirConjuntos / ExibirAlarmes / ExibirTudo executam" )
```

Definition at line 122 of file [test_LinhaDeProducao.cpp](#).

12.74.1.12 TEST_CASE() [12/23]

```
TEST_CASE (
    "LinhaDeProducao - GetConjuntos retorna vetor correto" )
```

Definition at line 76 of file [test_LinhaDeProducao.cpp](#).

12.74.1.13 TEST_CASE() [13/23]

```
TEST_CASE (
    "LinhaDeProducao - GetDesc retorna descricao correta" )
```

Definition at line 34 of file [test_LinhaDeProducao.cpp](#).

12.74.1.14 TEST_CASE() [14/23]

```
TEST_CASE (
    "LinhaDeProducao - GetId retorna id correto" )
```

Definition at line 19 of file [test_LinhaDeProducao.cpp](#).

12.74.1.15 TEST_CASE() [15/23]

```
TEST_CASE (
    "LinhaDeProducao - GetLocal retorna local correto" )
```

Definition at line 29 of file [test_LinhaDeProducao.cpp](#).

12.74.1.16 TEST_CASE() [16/23]

```
TEST_CASE (
    "LinhaDeProducao - GetNome retorna nome correto" )
```

Definition at line 24 of file [test_LinhaDeProducao.cpp](#).

12.74.1.17 TEST_CASE() [17/23]

```
TEST_CASE (
    "LinhaDeProducao - QuantidadeAlarmes soma conjuntos" )
```

Definition at line 134 of file [test_LinhaDeProducao.cpp](#).

12.74.1.18 TEST_CASE() [18/23]

```
TEST_CASE (
    "LinhaDeProducao - QuantidadeConjuntos inicia em zero" )
```

Definition at line 39 of file [test_LinhaDeProducao.cpp](#).

12.74.1.19 TEST_CASE() [19/23]

```
TEST_CASE (
    "LinhaDeProducao - RemoverConjunto decrementa quantidade" )
```

Definition at line 57 of file [test_LinhaDeProducao.cpp](#).

12.74.1.20 TEST_CASE() [20/23]

```
TEST_CASE (
    "LinhaDeProducao - RemoverConjunto inexistente lanca" )
```

Definition at line 105 of file [test_LinhaDeProducao.cpp](#).

12.74.1.21 TEST_CASE() [21/23]

```
TEST_CASE (
    "LinhaDeProducao - SetDesc atualiza descricao" )
```

Definition at line 144 of file [test_LinhaDeProducao.cpp](#).

12.74.1.22 TEST_CASE() [22/23]

```
TEST_CASE (
    "LinhaDeProducao - SetLocal atualiza local" )
```

Definition at line 150 of file [test_LinhaDeProducao.cpp](#).

12.74.1.23 TEST_CASE() [23/23]

```
TEST_CASE (
    "LinhaDeProducao - SetNome atualiza nome" )
```

Definition at line 64 of file [test_LinhaDeProducao.cpp](#).

12.75 test_LinhaDeProducao.cpp

[Go to the documentation of this file.](#)

```
00001 #include "doctest.h"
00002 #include "LinhaDeProducao.hpp"
00003 #include "Excecoes.hpp"
00004
00005 #include <iostream>
00006 #include <sstream>
00007
00008 namespace {
00009 template <typename F>
00010 std::string capturaSaida(F&& f) {
00011     std::stringstream buf;
00012     auto* old = std::cout.rdbuf(buf.rdbuf());
00013     f();
00014     std::cout.rdbuf(old);
00015     return buf.str();
00016 }
00017 }
00018
00019 TEST_CASE("LinhaDeProducao - GetId retorna id correto") {
00020     LinhaDeProducao l(1, "Desc", "Local", "Linha1");
00021     CHECK(l.GetId() == 1);
00022 }
00023
00024 TEST_CASE("LinhaDeProducao - GetNome retorna nome correto") {
00025     LinhaDeProducao l(2, "Desc", "Bloco A", "LinhaAlfa");
00026     CHECK(l.GetNome() == "LinhaAlfa");
00027 }
00028
00029 TEST_CASE("LinhaDeProducao - GetLocal retorna local correto") {
00030     LinhaDeProducao l(3, "Desc", "Bloco B", "LinhaBeta");
00031     CHECK(l.GetLocal() == "Bloco B");
00032 }
00033
00034 TEST_CASE("LinhaDeProducao - GetDesc retorna descricao correta") {
00035     LinhaDeProducao l(4, "Producao principal", "Bloco C", "LinhaGama");
00036     CHECK(l.GetDesc() == "Producao principal");
00037 }
00038
00039 TEST_CASE("LinhaDeProducao - QuantidadeConjuntos inicia em zero") {
00040     LinhaDeProducao l(5, "Desc", "Local", "Linha5");
00041     CHECK(l.QuantidadeConjuntos() == 0);
00042 }
00043
00044 TEST_CASE("LinhaDeProducao - AdicionarConjunto incrementa quantidade") {
00045     LinhaDeProducao l(6, "Desc", "Local", "Linha6");
00046     l.AdicionarConjunto(1, "Conjunto 1", "Desc");
00047     CHECK(l.QuantidadeConjuntos() == 1);
00048 }
00049
00050 TEST_CASE("LinhaDeProducao - AdicionarConjunto multiplos") {
00051     LinhaDeProducao l(7, "Desc", "Local", "Linha7");
00052     l.AdicionarConjunto(1, "C1", "Desc");
00053     l.AdicionarConjunto(2, "C2", "Desc");
00054     CHECK(l.QuantidadeConjuntos() == 2);
00055 }
00056
00057 TEST_CASE("LinhaDeProducao - RemoverConjunto decrementa quantidade") {
00058     LinhaDeProducao l(8, "Desc", "Local", "Linha8");
00059     l.AdicionarConjunto(1, "C1", "Desc");
00060     l.RemoverConjunto(1);
00061     CHECK(l.QuantidadeConjuntos() == 0);
00062 }
00063
00064 TEST_CASE("LinhaDeProducao - SetNome atualiza nome") {
00065     LinhaDeProducao l(9, "Desc", "Local", "Antiga");
00066     l.SetNome("Nova");
00067     CHECK(l.GetNome() == "Nova");
```

```

00068 }
00069
00070 TEST_CASE("LinhaDeProducao - DiagnosticarLinha vaziao sem falhas") {
00071     LinhaDeProducao l(10, "Desc", "Local", "Linha10");
00072     std::vector<int> falhas = l.DiagnosticarLinha();
00073     CHECK(falhas.empty() == true);
00074 }
00075
00076 TEST_CASE("LinhaDeProducao - GetConjuntos retorna vetor correto") {
00077     LinhaDeProducao l(11, "Desc", "Local", "Linha11");
00078     l.AdicionarConjunto(1, "C1", "Desc");
00079     CHECK(l.GetConjuntos().size() == 1);
00080 }
00081
00082 TEST_CASE("LinhaDeProducao - construtor rejeita id negativo") {
00083     CHECK_THROWS_AS(LinhaDeProducao(-1, "d", "l", "n"), IdInvalido);
00084 }
00085
00086 TEST_CASE("LinhaDeProducao - construtor rejeita descricao vazia") {
00087     CHECK_THROWS_AS(LinhaDeProducao(1, "", "l", "n"), EntradaInvalida);
00088 }
00089
00090 TEST_CASE("LinhaDeProducao - AdicionarConjunto auto-id retorna ids monotonicos") {
00091     LinhaDeProducao l(20, "d", "l", "n");
00092     int a = l.AdicionarConjunto("C1", "d");
00093     int b = l.AdicionarConjunto("C2", "d");
00094     CHECK(a == 1);
00095     CHECK(b == 2);
00096     CHECK(l.ProximoIdConjunto() == 3);
00097 }
00098
00099 TEST_CASE("LinhaDeProducao - AdicionarConjunto explicito avanca ProximoId") {
00100     LinhaDeProducao l(21, "d", "l", "n");
00101     l.AdicionarConjunto(10, "C", "d");
00102     CHECK(l.ProximoIdConjunto() == 11);
00103 }
00104
00105 TEST_CASE("LinhaDeProducao - RemoverConjunto inexistente lanca") {
00106     LinhaDeProducao l(22, "d", "l", "n");
00107     CHECK_THROWS_AS(l.RemoverConjunto(99), IdInexistente);
00108 }
00109
00110 TEST_CASE("LinhaDeProducao - AcessarConjunto inexistente lanca") {
00111     LinhaDeProducao l(23, "d", "l", "n");
00112     CHECK_THROWS_AS(l.AcessarConjunto(99), IdInexistente);
00113 }
00114
00115 TEST_CASE("LinhaDeProducao - AcessarConjunto retorna referencia mutavel") {
00116     LinhaDeProducao l(24, "d", "l", "n");
00117     int cid = l.AdicionarConjunto("C", "d");
00118     Conjunto& c = l.AcessarConjunto(cid);
00119     CHECK(c.GetNome() == "C");
00120 }
00121
00122 TEST_CASE("LinhaDeProducao - ExibirConjuntos / ExibirAlarmes / ExibirTudo executam") {
00123     LinhaDeProducao l(25, "d", "l", "n");
00124     int cid = l.AdicionarConjunto("C", "d");
00125     l.AcessarConjunto(cid).AdicionarEquipamento(1, "eq");
00126     std::string s1 = capturaSaida([& l.ExibirConjuntos();]);
00127     std::string s2 = capturaSaida([& l.ExibirAlarmes();]);
00128     std::string s3 = capturaSaida([& l.ExibirTudo();]);
00129     CHECK(!s1.empty());
00130     CHECK(!s3.empty());
00131     (void)s2;
00132 }
00133
00134 TEST_CASE("LinhaDeProducao - QuantidadeAlarmes soma conjuntos") {
00135     LinhaDeProducao l(26, "d", "l", "n");
00136     int cid = l.AdicionarConjunto("C", "d");
00137     Conjunto& c = l.AcessarConjunto(cid);
00138     c.AdicionarEquipamento(1, "eq");
00139     c.AcessarEquipamento(1).AdicionarParametro(15.0, 10.0, 0.0, "P", 1);
00140     l.AtualizarAlarmes();
00141     CHECK(l.QuantidadeAlarmes() == 1);
00142 }
00143
00144 TEST_CASE("LinhaDeProducao - SetDesc atualiza descricao") {
00145     LinhaDeProducao l(27, "d", "l", "n");
00146     l.SetDesc("nova");
00147     CHECK(l.GetDesc() == "nova");
00148 }
00149
00150 TEST_CASE("LinhaDeProducao - SetLocal atualiza local") {
00151     LinhaDeProducao l(28, "d", "l", "n");
00152     l.SetLocal("Bloco Z");
00153     CHECK(l.GetLocal() == "Bloco Z");
00154 }

```

```

00155
00156 TEST_CASE("LinhaDeProducao - DiagnosticarLinha reporta parametro fora de faixa") {
00157     LinhaDeProducao l(29, "d", "l", "n");
00158     int cid = l.AdicionarConjunto("C", "d");
00159     Conjunto& c = l.AcessarConjunto(cid);
00160     c.AdicionarEquipamento(1, "eq");
00161     c.AcessarEquipamento(1).AdicionarParametro(99.0, 10.0, 0.0, "P", 1);
00162     std::vector<int> falhas = l.DiagnosticarLinha();
00163     CHECK(falhas.size() == 1);
00164 }

```

12.76 tests/test_Log.cpp File Reference

```

#include "doctest.h"
#include "Log.hpp"

```

Functions

- [TEST_CASE](#) ("Log - GetArquivoLog retorna caminho correto")
- [TEST_CASE](#) ("Log - Registrar retorna true em mensagem valida")
- [TEST_CASE](#) ("Log - Registrar retorna false em mensagem vazia")
- [TEST_CASE](#) ("Log - SetArquivoLog retorna true e atualiza caminho")
- [TEST_CASE](#) ("Log - Registrar retorna false para mensagem acima de 500 chars")
- [TEST_CASE](#) ("Log - SetArquivoLog rejeita caminho vazio")

12.76.1 Function Documentation

12.76.1.1 TEST_CASE() [1/6]

```

TEST_CASE (
    "Log - GetArquivoLog retorna caminho correto" )

```

Definition at line 4 of file [test_Log.cpp](#).

12.76.1.2 TEST_CASE() [2/6]

```

TEST_CASE (
    "Log - Registrar retorna false em mensagem vazia" )

```

Definition at line 14 of file [test_Log.cpp](#).

12.76.1.3 TEST_CASE() [3/6]

```

TEST_CASE (
    "Log - Registrar retorna false para mensagem acima de 500 chars" )

```

Definition at line 26 of file [test_Log.cpp](#).

12.76.1.4 TEST_CASE() [4/6]

```
TEST_CASE (
    "Log - Registrar retorna true em mensagem valida" )
```

Definition at line 9 of file [test_Log.cpp](#).

12.76.1.5 TEST_CASE() [5/6]

```
TEST_CASE (
    "Log - SetArquivoLog rejeita caminho vazio" )
```

Definition at line 32 of file [test_Log.cpp](#).

12.76.1.6 TEST_CASE() [6/6]

```
TEST_CASE (
    "Log - SetArquivoLog retorna true e atualiza caminho" )
```

Definition at line 19 of file [test_Log.cpp](#).

12.77 test_Log.cpp

[Go to the documentation of this file.](#)

```
00001 #include "doctest.h"
00002 #include "Log.hpp"
00003
00004 TEST_CASE("Log - GetArquivoLog retorna caminho correto") {
00005     Log log("sistema.log");
00006     CHECK(log.GetArquivoLog() == "sistema.log");
00007 }
00008
00009 TEST_CASE("Log - Registrar retorna true em mensagem valida") {
00010     Log log("sistema.log");
00011     CHECK(log.Registrar("Evento de teste") == true);
00012 }
00013
00014 TEST_CASE("Log - Registrar retorna false em mensagem vazia") {
00015     Log log("sistema.log");
00016     CHECK(log.Registrar("") == false);
00017 }
00018
00019 TEST_CASE("Log - SetArquivoLog retorna true e atualiza caminho") {
00020     Log log("antigo.log");
00021     bool ok = log.SetArquivoLog("novo.log");
00022     CHECK(ok == true);
00023     CHECK(log.GetArquivoLog() == "novo.log");
00024 }
00025
00026 TEST_CASE("Log - Registrar retorna false para mensagem acima de 500 chars") {
00027     Log log("sistema.log");
00028     std::string msg(501, 'x');
00029     CHECK(log.Registrar(msg) == false);
00030 }
00031
00032 TEST_CASE("Log - SetArquivoLog rejeita caminho vazio") {
00033     Log log("sistema.log");
00034     CHECK(log.SetArquivoLog("") == false);
00035 }
```

12.78 tests/test_main.cpp File Reference

```
#include "doctest.h"
```

Macros

- `#define DOCTEST_CONFIG_IMPLEMENT_WITH_MAIN`

12.78.1 Macro Definition Documentation

12.78.1.1 DOCTEST_CONFIG_IMPLEMENT_WITH_MAIN

```
#define DOCTEST_CONFIG_IMPLEMENT_WITH_MAIN
```

Definition at line 1 of file [test_main.cpp](#).

12.79 test_main.cpp

[Go to the documentation of this file.](#)

```
00001 #define DOCTEST_CONFIG_IMPLEMENT_WITH_MAIN
00002 #include "doctest.h"
```

12.80 tests/test_Parametro.cpp File Reference

```
#include "doctest.h"
#include "Parametro.hpp"
```

Functions

- [TEST_CASE](#) ("Parametro - construtor simples armazena valor inicial")
- [TEST_CASE](#) ("Parametro - construtor com limites armazena max e min")
- [TEST_CASE](#) ("Parametro - GetDesc retorna descricao correta")
- [TEST_CASE](#) ("Parametro - GetId retorna id correto")
- [TEST_CASE](#) ("Parametro - SetValorAtual atualiza valor")
- [TEST_CASE](#) ("Parametro - SetValorAtual adiciona ao historico")
- [TEST_CASE](#) ("Parametro - ValorValido dentro dos limites retorna true")
- [TEST_CASE](#) ("Parametro - ValorValido acima do maximo retorna false")
- [TEST_CASE](#) ("Parametro - DiagnosticarParametro retorna false quando dentro dos limites")
- [TEST_CASE](#) ("Parametro - DiagnosticarParametro retorna true quando fora dos limites")
- [TEST_CASE](#) ("Parametro - GetEstado reflete diagnostico")
- [TEST_CASE](#) ("Parametro - ResetarLimites remove restricoes")
- [TEST_CASE](#) ("Parametro - ResetarHistorico limpa vetor")
- [TEST_CASE](#) ("Parametro - SetDesc atualiza descricao")

12.80.1 Function Documentation

12.80.1.1 TEST_CASE() [1/14]

```
TEST_CASE (
    "Parametro - construtor com limites armazena max e min" )
```

Definition at line 9 of file [test_Parametro.cpp](#).

12.80.1.2 TEST_CASE() [2/14]

```
TEST_CASE (
    "Parametro - construtor simples armazena valor inicial" )
```

Definition at line 4 of file [test_Parametro.cpp](#).

12.80.1.3 TEST_CASE() [3/14]

```
TEST_CASE (
    "Parametro - DiagnosticarParametro retorna false quando dentro dos limites" )
```

Definition at line 48 of file [test_Parametro.cpp](#).

12.80.1.4 TEST_CASE() [4/14]

```
TEST_CASE (
    "Parametro - DiagnosticarParametro retorna true quando fora dos limites" )
```

Definition at line 53 of file [test_Parametro.cpp](#).

12.80.1.5 TEST_CASE() [5/14]

```
TEST_CASE (
    "Parametro - GetDesc retorna descricao correta" )
```

Definition at line 15 of file [test_Parametro.cpp](#).

12.80.1.6 TEST_CASE() [6/14]

```
TEST_CASE (
    "Parametro - GetEstado reflete diagnostico" )
```

Definition at line 58 of file [test_Parametro.cpp](#).

12.80.1.7 TEST_CASE() [7/14]

```
TEST_CASE (
    "Parametro - GetId retorna id correto" )
```

Definition at line 20 of file [test_Parametro.cpp](#).

12.80.1.8 TEST_CASE() [8/14]

```
TEST_CASE (
    "Parametro - ResetarHistorico limpa vetor" )
```

Definition at line 70 of file [test_Parametro.cpp](#).

12.80.1.9 TEST_CASE() [9/14]

```
TEST_CASE (
    "Parametro - ResetarLimites remove restricoes" )
```

Definition at line 64 of file [test_Parametro.cpp](#).

12.80.1.10 TEST_CASE() [10/14]

```
TEST_CASE (
    "Parametro - SetDesc atualiza descricao" )
```

Definition at line 78 of file [test_Parametro.cpp](#).

12.80.1.11 TEST_CASE() [11/14]

```
TEST_CASE (
    "Parametro - SetValorAtual adiciona ao historico" )
```

Definition at line 31 of file [test_Parametro.cpp](#).

12.80.1.12 TEST_CASE() [12/14]

```
TEST_CASE (
    "Parametro - SetValorAtual atualiza valor" )
```

Definition at line 25 of file [test_Parametro.cpp](#).

12.80.1.13 TEST_CASE() [13/14]

```
TEST_CASE (
    "Parametro - ValorValido acima do maximo retorna false" )
```

Definition at line 43 of file [test_Parametro.cpp](#).

12.80.1.14 TEST_CASE() [14/14]

```
TEST_CASE (
    "Parametro - ValorValido dentro dos limites retorna true" )
```

Definition at line 38 of file [test_Parametro.cpp](#).

12.81 test_Parametro.cpp

[Go to the documentation of this file.](#)

```
00001 #include "doctest.h"
00002 #include "Parametro.hpp"
00003
00004 TEST_CASE("Parametro - construtor simples armazena valor inicial") {
00005     Parametro p(10.0, "Temperatura", 1);
00006     CHECK(p.GetValorAtual() == doctest::Approx(10.0));
00007 }
00008
00009 TEST_CASE("Parametro - construtor com limites armazena max e min") {
00010     Parametro p(5.0, 20.0, 0.0, "Pressao", 2);
00011     CHECK(p.GetMax() == doctest::Approx(20.0));
00012     CHECK(p.GetMin() == doctest::Approx(0.0));
00013 }
00014
00015 TEST_CASE("Parametro - GetDesc retorna descricao correta") {
00016     Parametro p(1.0, "Velocidade", 3);
00017     CHECK(p.GetDesc() == "Velocidade");
00018 }
00019
00020 TEST_CASE("Parametro - GetId retorna id correto") {
00021     Parametro p(0.0, "Tensao", 42);
00022     CHECK(p.GetId() == 42);
00023 }
00024
00025 TEST_CASE("Parametro - SetValorAtual atualiza valor") {
00026     Parametro p(0.0, "Corrente", 5);
00027     p.SetValorAtual(15.0);
00028     CHECK(p.GetValorAtual() == doctest::Approx(15.0));
00029 }
00030
00031 TEST_CASE("Parametro - SetValorAtual adiciona ao historico") {
00032     Parametro p(0.0, "Fluxo", 6);
00033     p.SetValorAtual(1.0);
00034     p.SetValorAtual(2.0);
00035     CHECK(p.GetHist().size() >= 2);
00036 }
00037
00038 TEST_CASE("Parametro - ValorValido dentro dos limites retorna true") {
00039     Parametro p(5.0, 10.0, 0.0, "Nivel", 7);
00040     CHECK(p.ValorValido(5.0) == true);
00041 }
00042
00043 TEST_CASE("Parametro - ValorValido acima do maximo retorna false") {
00044     Parametro p(5.0, 10.0, 0.0, "Nivel", 8);
00045     CHECK(p.ValorValido(15.0) == false);
00046 }
00047
00048 TEST_CASE("Parametro - DiagnosticarParametro retorna false quando dentro dos limites") {
00049     Parametro p(5.0, 10.0, 0.0, "Rotacao", 9);
00050     CHECK(p.DiagnosticarParametro() == false);
00051 }
00052
00053 TEST_CASE("Parametro - DiagnosticarParametro retorna true quando fora dos limites") {
00054     Parametro p(15.0, 10.0, 0.0, "Rotacao", 10);
00055     CHECK(p.DiagnosticarParametro() == true);
00056 }
00057
00058 TEST_CASE("Parametro - GetEstado reflete diagnostico") {
00059     Parametro p(15.0, 10.0, 0.0, "Vibr", 11);
00060     p.DiagnosticarParametro();
00061     CHECK(p.GetEstado() == true);
00062 }
00063
00064 TEST_CASE("Parametro - ResetarLimites remove restricoes") {
00065     Parametro p(5.0, 10.0, 0.0, "Temp", 12);
00066     p.ResetarLimites();
00067     CHECK(p.ValorValido(999.0) == true);
```

```

00068 }
00069
00070 TEST_CASE("Parametro - ResetarHistorico limpa vetor") {
00071     Parametro p(0.0, "Pressao", 13);
00072     p.SetValorAtual(1.0);
00073     p.SetValorAtual(2.0);
00074     p.ResetarHistorico();
00075     CHECK(p.GetHist().empty() == true);
00076 }
00077
00078 TEST_CASE("Parametro - SetDesc atualiza descricao") {
00079     Parametro p(0.0, "Antiga", 14);
00080     p.SetDesc("Nova");
00081     CHECK(p.GetDesc() == "Nova");
00082 }

```

12.82 tests/test_Sistema.cpp File Reference

```

#include "doctest.h"
#include "Sistema.hpp"
#include "Cargo.hpp"
#include "Excecoes.hpp"
#include <iostream>
#include <sstream>
#include <cstdio>

```

Functions

- [TEST_CASE](#) ("Sistema - inicia sem linhas")
- [TEST_CASE](#) ("Sistema - inicia sem usuario logado")
- [TEST_CASE](#) ("Sistema - AdicionarLinha incrementa lista")
- [TEST_CASE](#) ("Sistema - RemoverLinha decrementa lista")
- [TEST_CASE](#) ("Sistema - Login com credenciais corretas retorna true")
- [TEST_CASE](#) ("Sistema - Login com senha errada retorna false")
- [TEST_CASE](#) ("Sistema - Login com usuario inexistente retorna false")
- [TEST_CASE](#) ("Sistema - GetUsuarioLogado retorna usuario apos login")
- [TEST_CASE](#) ("Sistema - Logout retorna true quando ha sessao ativa")
- [TEST_CASE](#) ("Sistema - Logout retorna false sem sessao ativa")
- [TEST_CASE](#) ("Sistema - IdAtivo retorna false para id nao usado")
- [TEST_CASE](#) ("Sistema - IdAtivo retorna true para id em uso")
- [TEST_CASE](#) ("Sistema - ProximoldDisponivel retorna 1 no inicio")
- [TEST_CASE](#) ("Sistema - AdicionarUsuario permite login posterior")
- [TEST_CASE](#) ("Sistema - RemoverUsuario impede login posterior")
- [TEST_CASE](#) ("Sistema - AcessarLinha inexistente lanca")
- [TEST_CASE](#) ("Sistema - RemoverLinha inexistente lanca")
- [TEST_CASE](#) ("Sistema - AdicionarLinha duplicada lanca")
- [TEST_CASE](#) ("Sistema - AdicionarLinha auto-id retorna id asignado")
- [TEST_CASE](#) ("Sistema - ExibirLinhas com linha cadastrada executa")
- [TEST_CASE](#) ("Sistema - RemoverUsuario sem usuarios e silencioso")
- [TEST_CASE](#) ("Sistema - RemoverUsuario inexistente quando ha outros")
- [TEST_CASE](#) ("Sistema - persistencia round-trip")
- [TEST_CASE](#) ("Sistema - CarregarSave arquivo inexistente retorna false")

12.82.1 Function Documentation

12.82.1.1 TEST_CASE() [1/24]

```
TEST_CASE (
    "Sistema - AcessarLinha inexistente lanca" )
```

Definition at line 109 of file [test_Sistema.cpp](#).

12.82.1.2 TEST_CASE() [2/24]

```
TEST_CASE (
    "Sistema - AdicionarLinha auto-id retorna id asignado" )
```

Definition at line 125 of file [test_Sistema.cpp](#).

12.82.1.3 TEST_CASE() [3/24]

```
TEST_CASE (
    "Sistema - AdicionarLinha duplicada lanca" )
```

Definition at line 119 of file [test_Sistema.cpp](#).

12.82.1.4 TEST_CASE() [4/24]

```
TEST_CASE (
    "Sistema - AdicionarLinha incrementa lista" )
```

Definition at line 31 of file [test_Sistema.cpp](#).

12.82.1.5 TEST_CASE() [5/24]

```
TEST_CASE (
    "Sistema - AdicionarUsuario permite login posterior" )
```

Definition at line 96 of file [test_Sistema.cpp](#).

12.82.1.6 TEST_CASE() [6/24]

```
TEST_CASE (
    "Sistema - CarregarSave arquivo inexistente retorna false" )
```

Definition at line 172 of file [test_Sistema.cpp](#).

12.82.1.7 TEST_CASE() [7/24]

```
TEST_CASE (
    "Sistema - ExibirLinhas com linha cadastrada executa" )
```

Definition at line 133 of file [test_Sistema.cpp](#).

12.82.1.8 TEST_CASE() [8/24]

```
TEST_CASE (
    "Sistema - GetUsuarioLogado retorna usuario apos login" )
```

Definition at line 61 of file [test_Sistema.cpp](#).

12.82.1.9 TEST_CASE() [9/24]

```
TEST_CASE (
    "Sistema - IdAtivo retorna false para id nao usado" )
```

Definition at line 80 of file [test_Sistema.cpp](#).

12.82.1.10 TEST_CASE() [10/24]

```
TEST_CASE (
    "Sistema - IdAtivo retorna true para id em uso" )
```

Definition at line 85 of file [test_Sistema.cpp](#).

12.82.1.11 TEST_CASE() [11/24]

```
TEST_CASE (
    "Sistema - inicia sem linhas" )
```

Definition at line 21 of file [test_Sistema.cpp](#).

12.82.1.12 TEST_CASE() [12/24]

```
TEST_CASE (
    "Sistema - inicia sem usuario logado" )
```

Definition at line 26 of file [test_Sistema.cpp](#).

12.82.1.13 TEST_CASE() [13/24]

```
TEST_CASE (
    "Sistema - Login com credenciais corretas retorna true" )
```

Definition at line 44 of file [test_Sistema.cpp](#).

12.82.1.14 TEST_CASE() [14/24]

```
TEST_CASE (
    "Sistema - Login com senha errada retorna false" )
```

Definition at line 50 of file [test_Sistema.cpp](#).

12.82.1.15 TEST_CASE() [15/24]

```
TEST_CASE (
    "Sistema - Login com usuario inexistente retorna false" )
```

Definition at line 56 of file [test_Sistema.cpp](#).

12.82.1.16 TEST_CASE() [16/24]

```
TEST_CASE (
    "Sistema - Logout retorna false sem sessao ativa" )
```

Definition at line 75 of file [test_Sistema.cpp](#).

12.82.1.17 TEST_CASE() [17/24]

```
TEST_CASE (
    "Sistema - Logout retorna true quando ha sessao ativa" )
```

Definition at line 68 of file [test_Sistema.cpp](#).

12.82.1.18 TEST_CASE() [18/24]

```
TEST_CASE (
    "Sistema - persistencia round-trip" )
```

Definition at line 153 of file [test_Sistema.cpp](#).

12.82.1.19 TEST_CASE() [19/24]

```
TEST_CASE (
    "Sistema - ProximoIdDisponivel retorna 1 no inicio" )
```

Definition at line 91 of file [test_Sistema.cpp](#).

12.82.1.20 TEST_CASE() [20/24]

```
TEST_CASE (
    "Sistema - RemoverLinha decrementa lista" )
```

Definition at line 37 of file [test_Sistema.cpp](#).

12.82.1.21 TEST_CASE() [21/24]

```
TEST_CASE (
    "Sistema - RemoverLinha inexistente lanca" )
```

Definition at line 114 of file [test_Sistema.cpp](#).

12.82.1.22 TEST_CASE() [22/24]

```
TEST_CASE (
    "Sistema - RemoverUsuario impede login posterior" )
```

Definition at line 102 of file [test_Sistema.cpp](#).

12.82.1.23 TEST_CASE() [23/24]

```
TEST_CASE (
    "Sistema - RemoverUsuario inexistente quando ha outros" )
```

Definition at line 146 of file [test_Sistema.cpp](#).

12.82.1.24 TEST_CASE() [24/24]

```
TEST_CASE (
    "Sistema - RemoverUsuario sem usuarios e silencioso" )
```

Definition at line 140 of file [test_Sistema.cpp](#).

12.83 test_Sistema.cpp

[Go to the documentation of this file.](#)

```
00001 #include "doctest.h"
00002 #include "Sistema.hpp"
00003 #include "Cargo.hpp"
00004 #include "Excecoes.hpp"
00005
00006 #include <iostream>
00007 #include <sstream>
00008 #include <cstdio>
00009
00010 namespace {
00011     template <typename F>
00012     std::string capturaSaida(F&& f) {
00013         std::stringstream buf;
00014         auto* old = std::cout.rdbuf(buf.rdbuf());
00015         f();
00016         std::cout.rdbuf(old);
00017         return buf.str();
00018     }
00019 }
00020
00021 TEST_CASE("Sistema - inicia sem linhas") {
00022     Sistema s;
00023     CHECK(s.GetLinhas().empty() == true);
00024 }
00025
00026 TEST_CASE("Sistema - inicia sem usuario logado") {
00027     Sistema s;
00028     CHECK(s.GetUsuarioLogado() == nullptr);
```

```
00029 }
00030
00031 TEST_CASE("Sistema - AdicionarLinha incrementa lista") {
00032     Sistema s;
00033     s.AdicionarLinha(1, "Desc", "Local", "Linha1");
00034     CHECK(s.GetLinhas().size() == 1);
00035 }
00036
00037 TEST_CASE("Sistema - RemoverLinha decrementa lista") {
00038     Sistema s;
00039     s.AdicionarLinha(1, "Desc", "Local", "Linha1");
00040     s.RemoverLinha(1);
00041     CHECK(s.GetLinhas().empty() == true);
00042 }
00043
00044 TEST_CASE("Sistema - Login com credenciais corretas retorna true") {
00045     Sistema s;
00046     s.AdicionarUsuario("admin", "senha123", Cargo::ADMIN);
00047     CHECK(s.Login("admin", "senha123") == true);
00048 }
00049
00050 TEST_CASE("Sistema - Login com senha errada retorna false") {
00051     Sistema s;
00052     s.AdicionarUsuario("admin", "senha123", Cargo::ADMIN);
00053     CHECK(s.Login("admin", "senhaerrada") == false);
00054 }
00055
00056 TEST_CASE("Sistema - Login com usuario inexistente retorna false") {
00057     Sistema s;
00058     CHECK(s.Login("naoexiste", "qualquer") == false);
00059 }
00060
00061 TEST_CASE("Sistema - GetUsuarioLogado retorna usuario apos login") {
00062     Sistema s;
00063     s.AdicionarUsuario("eng01", "pass", Cargo::ENGENHEIRO);
00064     s.Login("eng01", "pass");
00065     CHECK(s.GetUsuarioLogado() != nullptr);
00066 }
00067
00068 TEST_CASE("Sistema - Logout retorna true quando ha sessao ativa") {
00069     Sistema s;
00070     s.AdicionarUsuario("tec01", "pass", Cargo::TECNICO);
00071     s.Login("tec01", "pass");
00072     CHECK(s.Logout() == true);
00073 }
00074
00075 TEST_CASE("Sistema - Logout retorna false sem sessao ativa") {
00076     Sistema s;
00077     CHECK(s.Logout() == false);
00078 }
00079
00080 TEST_CASE("Sistema - IdAtivo retorna false para id nao usado") {
00081     Sistema s;
00082     CHECK(s.IdAtivo(99) == false);
00083 }
00084
00085 TEST_CASE("Sistema - IdAtivo retorna true para id em uso") {
00086     Sistema s;
00087     s.AdicionarLinha(5, "Desc", "Local", "Linha");
00088     CHECK(s.IdAtivo(5) == true);
00089 }
00090
00091 TEST_CASE("Sistema - ProximoIdDisponivel retorna 1 no inicio") {
00092     Sistema s;
00093     CHECK(s.ProximoIdDisponivel() == 1);
00094 }
00095
00096 TEST_CASE("Sistema - AdicionarUsuario permite login posterior") {
00097     Sistema s;
00098     s.AdicionarUsuario("novouser", "novasenha", Cargo::TECNICO);
00099     CHECK(s.Login("novouser", "novasenha") == true);
00100 }
00101
00102 TEST_CASE("Sistema - RemoverUsuario impede login posterior") {
00103     Sistema s;
00104     s.AdicionarUsuario("tempuser", "pass", Cargo::TECNICO);
00105     s.RemoverUsuario("tempuser");
00106     CHECK(s.Login("tempuser", "pass") == false);
00107 }
00108
00109 TEST_CASE("Sistema - AcessarLinha inexistente lanca") {
00110     Sistema s;
00111     CHECK_THROWS_AS(s.AcessarLinha(99), IdInexistente);
00112 }
00113
00114 TEST_CASE("Sistema - RemoverLinha inexistente lanca") {
00115     Sistema s;
```

```

00116     CHECK_THROWS_AS(s.RemoverLinha(99), IdInexistente);
00117 }
00118
00119 TEST_CASE("Sistema - AdicionarLinha duplicada lanca") {
00120     Sistema s;
00121     s.AdicionarLinha(1, "d", "l", "n");
00122     CHECK_THROWS_AS(s.AdicionarLinha(1, "d", "l", "n"), IdDuplicado);
00123 }
00124
00125 TEST_CASE("Sistema - AdicionarLinha auto-id retorna id assignado") {
00126     Sistema s;
00127     int a = s.AdicionarLinha("d", "l", "n");
00128     int b = s.AdicionarLinha("d", "l", "n");
00129     CHECK(a == 1);
00130     CHECK(b == 2);
00131 }
00132
00133 TEST_CASE("Sistema - ExibirLinhas com linha cadastrada executa") {
00134     Sistema s;
00135     s.AdicionarLinha(1, "Desc", "Local", "L");
00136     std::string out = capturaSaida([&]{ s.ExibirLinhas(); });
00137     CHECK(out.find("Linha") != std::string::npos);
00138 }
00139
00140 TEST_CASE("Sistema - RemoverUsuario sem usuarios e silencioso") {
00141     Sistema s;
00142     s.RemoverUsuario("ninguem");
00143     CHECK(true);
00144 }
00145
00146 TEST_CASE("Sistema - RemoverUsuario inexistente quando ha outros") {
00147     Sistema s;
00148     s.AdicionarUsuario("a", "p", Cargo::ADMIN);
00149     s.RemoverUsuario("inexistente");
00150     CHECK(s.Login("a", "p") == true);
00151 }
00152
00153 TEST_CASE("Sistema - persistencia round-trip") {
00154     std::remove("save.txt");
00155     {
00156         Sistema s;
00157         s.AdicionarUsuario("u", "p", Cargo::ADMIN);
00158         int lid = s.AdicionarLinha("d", "l", "n");
00159         s.AcessarLinha(lid).AdicionarConjunto("c", "cd");
00160         s.AcessarLinha(lid).AcessarConjunto(1).AdicionarEquipamento(1, "eq");
00161         s.AcessarLinha(lid).AcessarConjunto(1).AcessarEquipamento(1)
00162             .AdicionarParametro(1.0, 10.0, 0.0, "P", 1);
00163         REQUIRE(s.SalvarAlteracoes());
00164     }
00165     Sistema s2;
00166     REQUIRE(s2.CarregarUltimoSave());
00167     CHECK(s2.GetLinhas().size() == 1);
00168     CHECK(s2.Login("u", "p") == true);
00169     std::remove("save.txt");
00170 }
00171
00172 TEST_CASE("Sistema - CarregarSave arquivo inexistente retorna false") {
00173     Sistema s;
00174     CHECK(s.CarregarSave("arquivo_que_nao_existe_xyz.txt") == false);
00175 }

```

12.84 tests/test_Usuario.cpp File Reference

```

#include "doctest.h"
#include "Usuario.hpp"
#include "Cargo.hpp"

```

Functions

- [TEST_CASE](#) ("Usuario - GetLogin retorna login correto")
- [TEST_CASE](#) ("Usuario - ValidarSenha retorna true com senha correta")
- [TEST_CASE](#) ("Usuario - ValidarSenha retorna false com senha errada")
- [TEST_CASE](#) ("Usuario - ValidarSenha case-sensitive")

- [TEST_CASE](#) ("Usuario - GetCargo retorna cargo ADMIN")
- [TEST_CASE](#) ("Usuario - GetCargo retorna cargo ENGENHEIRO")
- [TEST_CASE](#) ("Usuario - GetCargo retorna cargo TECNICO")

12.84.1 Function Documentation

12.84.1.1 [TEST_CASE\(\)](#) [1/7]

```
TEST_CASE (
    "Usuario - GetCargo retorna cargo ADMIN" )
```

Definition at line 25 of file [test_Usuario.cpp](#).

12.84.1.2 [TEST_CASE\(\)](#) [2/7]

```
TEST_CASE (
    "Usuario - GetCargo retorna cargo ENGENHEIRO" )
```

Definition at line 30 of file [test_Usuario.cpp](#).

12.84.1.3 [TEST_CASE\(\)](#) [3/7]

```
TEST_CASE (
    "Usuario - GetCargo retorna cargo TECNICO" )
```

Definition at line 35 of file [test_Usuario.cpp](#).

12.84.1.4 [TEST_CASE\(\)](#) [4/7]

```
TEST_CASE (
    "Usuario - GetLogin retorna login correto" )
```

Definition at line 5 of file [test_Usuario.cpp](#).

12.84.1.5 [TEST_CASE\(\)](#) [5/7]

```
TEST_CASE (
    "Usuario - ValidarSenha case-sensitive" )
```

Definition at line 20 of file [test_Usuario.cpp](#).

12.84.1.6 [TEST_CASE\(\)](#) [6/7]

```
TEST_CASE (
    "Usuario - ValidarSenha retorna false com senha errada" )
```

Definition at line 15 of file [test_Usuario.cpp](#).

12.84.1.7 TEST_CASE() [7/7]

```
TEST_CASE (
    "Usuario - ValidarSenha retorna true com senha correta" )
```

Definition at line 10 of file [test_Usuario.cpp](#).

12.85 test_Usuario.cpp

[Go to the documentation of this file.](#)

```
00001 #include "doctest.h"
00002 #include "Usuario.hpp"
00003 #include "Cargo.hpp"
00004
00005 TEST_CASE("Usuario - GetLogin retorna login correto") {
00006     Usuario u("admin", "senha123", Cargo::ADMIN);
00007     CHECK(u.GetLogin() == "admin");
00008 }
00009
00010 TEST_CASE("Usuario - ValidarSenha retorna true com senha correta") {
00011     Usuario u("eng01", "minhasenha", Cargo::ENGENHEIRO);
00012     CHECK(u.ValidarSenha("minhasenha") == true);
00013 }
00014
00015 TEST_CASE("Usuario - ValidarSenha retorna false com senha errada") {
00016     Usuario u("tec01", "senha_certa", Cargo::TECNICO);
00017     CHECK(u.ValidarSenha("senha_errada") == false);
00018 }
00019
00020 TEST_CASE("Usuario - ValidarSenha case-sensitive") {
00021     Usuario u("user1", "SenhaA", Cargo::TECNICO);
00022     CHECK(u.ValidarSenha("senhaa") == false);
00023 }
00024
00025 TEST_CASE("Usuario - GetCargo retorna cargo ADMIN") {
00026     Usuario u("admin", "pass", Cargo::ADMIN);
00027     CHECK(u.GetCargo("admin") == Cargo::ADMIN);
00028 }
00029
00030 TEST_CASE("Usuario - GetCargo retorna cargo ENGENHEIRO") {
00031     Usuario u("eng01", "pass", Cargo::ENGENHEIRO);
00032     CHECK(u.GetCargo("eng01") == Cargo::ENGENHEIRO);
00033 }
00034
00035 TEST_CASE("Usuario - GetCargo retorna cargo TECNICO") {
00036     Usuario u("tec01", "pass", Cargo::TECNICO);
00037     CHECK(u.GetCargo("tec01") == Cargo::TECNICO);
00038 }
```

Index

- ~Atuador
 - Atuador, [59](#)
- ~Context
 - doctest::Context, [69](#)
- ~ContextScope
 - doctest::detail::ContextScope< L >, [78](#)
- ~ContextScopeBase
 - doctest::detail::ContextScopeBase, [80](#)
- ~Equipamento
 - Equipamento, [87](#)
- ~MessageBuilder
 - doctest::detail::MessageBuilder, [132](#)
- ~Motor
 - Motor, [136](#)
- ~Sensor
 - Sensor, [152](#)
- ~String
 - doctest::String, [162](#)
- ~Subcase
 - doctest::detail::Subcase, [170](#)
- ~TestCase
 - doctest::detail::TestCase, [173](#)
- ~Valvula
 - Valvula, [187](#)
- abort_after
 - doctest::ContextOptions, [72](#)
- AcessarConjunto
 - LinhaDeProducao, [123](#)
- AcessarEquipamento
 - Conjunto, [62](#)
- AcessarLinha
 - Sistema, [155](#)
- AcessoNegado, [49](#)
- acquireGeneratorDecisionIndex
 - doctest::detail, [40](#)
- acquireGeneratorValue
 - doctest::detail, [40](#)
- ADD_FAIL_AT
 - doctest.h, [233](#)
- ADD_FAIL_CHECK_AT
 - doctest.h, [233](#)
- ADD_MESSAGE_AT
 - doctest.h, [233](#)
- addFilter
 - doctest::Context, [69](#)
- AdicionarConjunto
 - LinhaDeProducao, [124](#)
- AdicionarEquipamento
 - Conjunto, [62](#), [63](#)
- AdicionarLinha
 - Sistema, [155](#), [156](#)
- AdicionarParametro
 - Equipamento, [87](#), [88](#)
- AdicionarUsuario
 - Sistema, [156](#)
- ADMIN
 - Cargo.hpp, [191](#)
- Alarme, [49](#)
 - Alarme, [50](#)
 - AtualizarAlarme, [51](#)
 - GetEstado, [51](#)
 - SetDesc, [51](#)
 - SetEstado, [51](#)
- AND_GIVEN
 - doctest.h, [233](#)
- AND_THEN
 - doctest.h, [234](#)
- AND_WHEN
 - doctest.h, [234](#)
- applyCommandLine
 - doctest::Context, [69](#)
- Approx
 - doctest::Approx, [53](#)
- assert_handler
 - doctest::detail, [40](#)
- AssertData
 - doctest::AssertData, [56](#)
- AssertFailure
 - doctest::TestCaseFailureReason, [46](#)
- assertString
 - doctest, [27](#)
- Atuador, [58](#)
 - ~Atuador, [59](#)
 - Atuador, [59](#)
 - Tipo, [60](#)
- AtualizarAlarme
 - Alarme, [51](#)
- AtualizarAlarmes
 - Conjunto, [63](#)
 - Equipamento, [89](#)
 - LinhaDeProducao, [124](#)
- binary_assert
 - doctest::detail, [40](#)
 - doctest::detail::ResultBuilder, [150](#)
- binary_name
 - doctest::ContextOptions, [72](#)
- Blue
 - doctest::Color, [37](#)

- Bright
 - doctest::Color, [37](#)
- BrightGreen
 - doctest::Color, [37](#)
- BrightRed
 - doctest::Color, [37](#)
- BrightWhite
 - doctest::Color, [37](#)
- buf
 - doctest::String, [165](#)
- c_str
 - doctest::AssertData::StringContains, [166](#)
 - doctest::String, [163](#)
- capacity
 - doctest::String, [163](#)
- CAPTURE
 - doctest.h, [234](#)
- Cargo
 - Cargo.hpp, [190](#)
- Cargo.hpp
 - ADMIN, [191](#)
 - Cargo, [190](#)
 - ENGENHEIRO, [191](#)
 - TECNICO, [191](#)
- CarregarSave
 - Sistema, [157](#)
- CarregarUltimoSave
 - Sistema, [157](#)
- case_sensitive
 - doctest::ContextOptions, [72](#)
- CHECK
 - doctest.h, [234](#)
- check
 - doctest::AssertData::StringContains, [166](#)
- CHECK_EQ
 - doctest.h, [234](#)
- CHECK_FALSE
 - doctest.h, [235](#)
- CHECK_FALSE_MESSAGE
 - doctest.h, [235](#)
- CHECK_GE
 - doctest.h, [235](#)
- CHECK_GT
 - doctest.h, [235](#)
- CHECK_LE
 - doctest.h, [235](#)
- CHECK_LT
 - doctest.h, [236](#)
- CHECK_MESSAGE
 - doctest.h, [236](#)
- CHECK_NE
 - doctest.h, [236](#)
- CHECK_NOTHROW
 - doctest.h, [236](#)
- CHECK_NOTHROW_MESSAGE
 - doctest.h, [236](#)
- CHECK_THROWS
 - doctest.h, [237](#)
- CHECK_THROWS_AS
 - doctest.h, [237](#)
- CHECK_THROWS_AS_MESSAGE
 - doctest.h, [237](#)
- CHECK_THROWS_MESSAGE
 - doctest.h, [237](#)
- CHECK_THROWS_WITH
 - doctest.h, [237](#)
- CHECK_THROWS_WITH_AS
 - doctest.h, [238](#)
- CHECK_THROWS_WITH_AS_MESSAGE
 - doctest.h, [238](#)
- CHECK_THROWS_WITH_MESSAGE
 - doctest.h, [238](#)
- CHECK_UNARY
 - doctest.h, [238](#)
- CHECK_UNARY_FALSE
 - doctest.h, [239](#)
- checkIfShouldThrow
 - doctest::detail, [41](#)
- checkWith
 - doctest::Contains, [68](#)
- clearFilters
 - doctest::Context, [69](#)
- compare
 - doctest::String, [163](#)
- Conjunto, [61](#)
 - AcessarEquipamento, [62](#)
 - AdicionarEquipamento, [62](#), [63](#)
 - AtualizarAlarmes, [63](#)
 - Conjunto, [62](#)
 - DiagnosticarConjunto, [63](#)
 - ExibirAlarmes, [64](#)
 - ExibirEquipamentos, [64](#)
 - ExibirTudo, [64](#)
 - GetDesc, [64](#)
 - GetEquipamentos, [65](#)
 - GetId, [65](#)
 - GetNome, [65](#)
 - ProximoldEquipamento, [65](#)
 - QuantidadeAlarmes, [65](#)
 - QuantidadeEquipamentos, [66](#)
 - RemoverEquipamento, [66](#)
 - SetDesc, [66](#)
 - SetNome, [67](#)
- Contains
 - doctest::Contains, [68](#)
- Context
 - doctest::Context, [69](#)
- ContextScope
 - doctest::detail::ContextScope< L >, [78](#)
- ContextScopeBase
 - doctest::detail::ContextScopeBase, [80](#)
- convert
 - doctest::detail::StringMakerBase< C >, [168](#)
 - doctest::detail::StringMakerBase< true >, [169](#)
- CouldHaveFailedAndDid
 - doctest::TestCaseFailureReason, [46](#)

- count
 - doctest::ContextOptions, 72
- cout
 - doctest::ContextOptions, 72
- Crash
 - doctest::TestCaseFailureReason, 46
- currentTest
 - doctest::ContextOptions, 72
- Cyan
 - doctest::Color, 37
- data
 - doctest::QueryData, 143
 - doctest::String, 165
- declval
 - doctest::detail, 41
- decomp_assert
 - doctest::detail, 41
- Dependencia, 83
 - Dependencia, 83
 - Get_Condicao, 83
 - Get_Id, 83
 - Get_Resultado, 84
- design Directory Reference, 23
- design/notas-C6.md, 189
- design/notas-C7.md, 189
- destroy
 - doctest::detail::ContextScopeBase, 80
- DiagnosticarConjunto
 - Conjunto, 63
- DiagnosticarEquipamento
 - Equipamento, 89
- DiagnosticarLinha
 - LinhaDeProducao, 124
- DiagnosticarParametro
 - Parametro, 138
- DidntFailExactlyNumTimes
 - doctest::TestCaseFailureReason, 46
- doctest, 25
 - assertString, 27
 - DOCTEST_DEFINE_DECORATOR, 27, 28
 - failureString, 28
 - getContextOptions, 28
 - is_running_in_test, 34
 - IsNaN< double >, 34
 - IsNaN< float >, 34
 - IsNaN< long double >, 34
 - operator!=, 29
 - operator<, 29
 - operator<=, 29
 - operator>, 30
 - operator>=, 30
 - operator+, 29
 - operator==, 29, 30
 - registerExceptionTranslator, 30
 - registerReporter, 30
 - size_t, 27
 - skipPathFromFilename, 30
 - TestCaseData, 34
 - toString, 30–33
- doctest.h
 - ADD_FAIL_AT, 233
 - ADD_FAIL_CHECK_AT, 233
 - ADD_MESSAGE_AT, 233
 - AND_GIVEN, 233
 - AND_THEN, 234
 - AND_WHEN, 234
 - CAPTURE, 234
 - CHECK, 234
 - CHECK_EQ, 234
 - CHECK_FALSE, 235
 - CHECK_FALSE_MESSAGE, 235
 - CHECK_GE, 235
 - CHECK_GT, 235
 - CHECK_LE, 235
 - CHECK_LT, 236
 - CHECK_MESSAGE, 236
 - CHECK_NE, 236
 - CHECK_NOTHROW, 236
 - CHECK_NOTHROW_MESSAGE, 236
 - CHECK_THROWS, 237
 - CHECK_THROWS_AS, 237
 - CHECK_THROWS_AS_MESSAGE, 237
 - CHECK_THROWS_MESSAGE, 237
 - CHECK_THROWS_WITH, 237
 - CHECK_THROWS_WITH_AS, 238
 - CHECK_THROWS_WITH_AS_MESSAGE, 238
 - CHECK_THROWS_WITH_MESSAGE, 238
 - CHECK_UNARY, 238
 - CHECK_UNARY_FALSE, 239
 - DOCTEST_ADD_AT_IMPL, 239
 - DOCTEST_ADD_FAIL_AT, 239
 - DOCTEST_ADD_FAIL_CHECK_AT, 239
 - DOCTEST_ADD_MESSAGE_AT, 240
 - DOCTEST_ALIGNMENT, 240
 - DOCTEST_AND_GIVEN, 240
 - DOCTEST_AND_THEN, 240
 - DOCTEST_AND_WHEN, 240
 - DOCTEST_ANONYMOUS, 241
 - DOCTEST_ASSERT_IMPLEMENT_1, 241
 - DOCTEST_ASSERT_IMPLEMENT_2, 241
 - DOCTEST_ASSERT_IN_TESTS, 241
 - DOCTEST_ASSERT_LOG_REACT_RETURN, 242
 - DOCTEST_ASSERT_OUT_OF_TESTS, 242
 - DOCTEST_BINARY_ASSERT, 242
 - DOCTEST_BINARY_RELATIONAL_OP, 243
 - DOCTEST_BREAK_INTO_DEBUGGER, 243
 - DOCTEST_CAPTURE, 243
 - DOCTEST_CAST_TO_VOID, 243
 - DOCTEST_CAT, 244
 - DOCTEST_CAT_IMPL, 244
 - DOCTEST_CHECK, 244
 - DOCTEST_CHECK_EQ, 244
 - DOCTEST_CHECK_FALSE, 244
 - DOCTEST_CHECK_FALSE_MESSAGE, 245
 - DOCTEST_CHECK_GE, 245

- DOCTEST_CHECK_GT, [245](#)
- DOCTEST_CHECK_LE, [245](#)
- DOCTEST_CHECK_LT, [245](#)
- DOCTEST_CHECK_MESSAGE, [246](#)
- DOCTEST_CHECK_NE, [246](#)
- DOCTEST_CHECK_NOTHROW, [246](#)
- DOCTEST_CHECK_NOTHROW_MESSAGE, [246](#)
- DOCTEST_CHECK_THROWS, [246](#)
- DOCTEST_CHECK_THROWS_AS, [247](#)
- DOCTEST_CHECK_THROWS_AS_MESSAGE, [247](#)
- DOCTEST_CHECK_THROWS_MESSAGE, [247](#)
- DOCTEST_CHECK_THROWS_WITH, [247](#)
- DOCTEST_CHECK_THROWS_WITH_AS, [247](#)
- DOCTEST_CHECK_THROWS_WITH_AS_MESSAGE, [248](#)
- DOCTEST_CHECK_THROWS_WITH_MESSAGE, [248](#)
- DOCTEST_CHECK_UNARY, [248](#)
- DOCTEST_CHECK_UNARY_FALSE, [248](#)
- DOCTEST_CLANG, [249](#)
- DOCTEST_CLANG_SUPPRESS_WARNING, [249](#)
- DOCTEST_CLANG_SUPPRESS_WARNING_POP, [249](#)
- DOCTEST_CLANG_SUPPRESS_WARNING_PUSH, [249](#)
- DOCTEST_CLANG_SUPPRESS_WARNING_WITH_PUSH, [249](#)
- DOCTEST_CMP_EQ, [249](#)
- DOCTEST_CMP_GE, [249](#)
- DOCTEST_CMP_GT, [250](#)
- DOCTEST_CMP_LE, [250](#)
- DOCTEST_CMP_LT, [250](#)
- DOCTEST_CMP_NE, [250](#)
- DOCTEST_COMPARISON_RETURN_TYPE, [250](#)
- DOCTEST_COMPILER, [251](#)
- DOCTEST_CONFIG_NO_EXCEPTIONS, [251](#)
- DOCTEST_CONFIG_NO_TRY_CATCH_IN_ASSERTS, [251](#)
- DOCTEST_CONFIG_POSIX_SIGNALS, [251](#)
- DOCTEST_CONFIG_STRING_SIZE_TYPE, [251](#)
- DOCTEST_CONSTEXPR, [251](#)
- DOCTEST_CONSTEXPR_FUNC, [251](#)
- DOCTEST_COUNTER, [252](#)
- DOCTEST_CPLUSPLUS, [252](#)
- DOCTEST_CREATE_AND_REGISTER_FUNCTION, [252](#)
- DOCTEST_CREATE_AND_REGISTER_FUNCTION_IN_NAMESPACE, [252](#)
- DOCTEST_DECLARE_INTERFACE, [252](#)
- DOCTEST_DEFINE_DECORATOR, [253](#)
- DOCTEST_DEFINE_INTERFACE, [253](#)
- DOCTEST_DO_BINARY_EXPRESSION_COMPARISON, [253](#)
- DOCTEST_EMPTY, [254](#)
- DOCTEST_EXCEPTION_EMPTY_FUNC, [254](#)
- DOCTEST_FAIL, [254](#)
- DOCTEST_FAIL_CHECK, [255](#)
- DOCTEST_FAST_CHECK_EQ, [255](#)
- DOCTEST_FAST_CHECK_GE, [255](#)
- DOCTEST_FAST_CHECK_GT, [255](#)
- DOCTEST_FAST_CHECK_LE, [255](#)
- DOCTEST_FAST_CHECK_LT, [255](#)
- DOCTEST_FAST_CHECK_NE, [255](#)
- DOCTEST_FAST_CHECK_UNARY, [256](#)
- DOCTEST_FAST_CHECK_UNARY_FALSE, [256](#)
- DOCTEST_FAST_REQUIRE_EQ, [256](#)
- DOCTEST_FAST_REQUIRE_GE, [256](#)
- DOCTEST_FAST_REQUIRE_GT, [256](#)
- DOCTEST_FAST_REQUIRE_LE, [256](#)
- DOCTEST_FAST_REQUIRE_LT, [256](#)
- DOCTEST_FAST_REQUIRE_NE, [256](#)
- DOCTEST_FAST_REQUIRE_UNARY, [257](#)
- DOCTEST_FAST_REQUIRE_UNARY_FALSE, [257](#)
- DOCTEST_FAST_WARN_EQ, [257](#)
- DOCTEST_FAST_WARN_GE, [257](#)
- DOCTEST_FAST_WARN_GT, [257](#)
- DOCTEST_FAST_WARN_LE, [257](#)
- DOCTEST_FAST_WARN_LT, [257](#)
- DOCTEST_FAST_WARN_NE, [257](#)
- DOCTEST_FAST_WARN_UNARY, [258](#)
- DOCTEST_FAST_WARN_UNARY_FALSE, [258](#)
- DOCTEST_FORBIT_EXPRESSION, [258](#)
- DOCTEST_FUNC_EMPTY, [258](#)
- DOCTEST_FUNC_SCOPE_BEGIN, [258](#)
- DOCTEST_FUNC_SCOPE_END, [258](#)
- DOCTEST_FUNC_SCOPE_RET, [259](#)
- DOCTEST_GCC, [259](#)
- DOCTEST_GCC_SUPPRESS_WARNING, [259](#)
- DOCTEST_GCC_SUPPRESS_WARNING_POP, [259](#)
- DOCTEST_GCC_SUPPRESS_WARNING_PUSH, [259](#)
- DOCTEST_GCC_SUPPRESS_WARNING_WITH_PUSH, [259](#)
- DOCTEST_GENERATE, [259](#)
- DOCTEST_GIVEN, [260](#)
- DOCTEST_GLOBAL_NO_WARNINGS, [260](#)
- DOCTEST_HAS_BUILTIN, [260](#)
- DOCTEST_ICC, [260](#)
- DOCTEST_IMPLEMENT_FIXTURE, [260](#)
- DOCTEST_INFO, [261](#)
- DOCTEST_INFO_IMPL, [261](#)
- DOCTEST_INLINE_NOINLINE, [261](#)
- DOCTEST_INTERFACE, [262](#)
- DOCTEST_INTERFACE_DECL, [262](#)
- DOCTEST_INTERFACE_DEF, [262](#)
- DOCTEST_MAKE_STD_HEADERS_CLEAN_FROM_WARNINGS_ON_MAC, [262](#)
- DOCTEST_MAKE_STD_HEADERS_CLEAN_FROM_WARNINGS_ON_WINDOWS, [262](#)
- DOCTEST_MESSAGE, [263](#)
- DOCTEST_MSVC, [263](#)
- DOCTEST_MSVC_SUPPRESS_WARNING, [263](#)

- DOCTEST_MSVC_SUPPRESS_WARNING_POP, [263](#)
- DOCTEST_MSVC_SUPPRESS_WARNING_PUSH, [263](#)
- DOCTEST_MSVC_SUPPRESS_WARNING_WITH_PUSH, [263](#)
- DOCTEST_NO_SANITIZE_INTEGER, [263](#)
- DOCTEST_NOEXCEPT, [264](#)
- DOCTEST_NOINLINE, [264](#)
- DOCTEST_NORETURN, [264](#)
- DOCTEST_PARTS_PUBLIC_ASSERT_COMPARATOR, [264](#)
- DOCTEST_PARTS_PUBLIC_ASSERT_DATA, [264](#)
- DOCTEST_PARTS_PUBLIC_ASSERT_EXPRESSION, [264](#)
- DOCTEST_PARTS_PUBLIC_ASSERT_HANDLER, [264](#)
- DOCTEST_PARTS_PUBLIC_ASSERT_MESSAGE, [264](#)
- DOCTEST_PARTS_PUBLIC_ASSERT_RESULT, [265](#)
- DOCTEST_PARTS_PUBLIC_ASSERT_TYPE, [265](#)
- DOCTEST_PARTS_PUBLIC_COLOR, [265](#)
- DOCTEST_PARTS_PUBLIC_COMPILER, [265](#)
- DOCTEST_PARTS_PUBLIC_CONFIG, [265](#)
- DOCTEST_PARTS_PUBLIC_CONTEXT, [265](#)
- DOCTEST_PARTS_PUBLIC_CONTEXT_OPTIONS, [265](#)
- DOCTEST_PARTS_PUBLIC_CONTEXT_SCOPE, [265](#)
- DOCTEST_PARTS_PUBLIC_DEBUGGER, [266](#)
- DOCTEST_PARTS_PUBLIC_DECORATORS, [266](#)
- DOCTEST_PARTS_PUBLIC_EXCEPTION_TRANSLATOR, [266](#)
- DOCTEST_PARTS_PUBLIC_EXCEPTIONS, [266](#)
- DOCTEST_PARTS_PUBLIC_GENERATOR, [266](#)
- DOCTEST_PARTS_PUBLIC_MACROS, [266](#)
- DOCTEST_PARTS_PUBLIC_MATCHERS_APPROX, [266](#)
- DOCTEST_PARTS_PUBLIC_MATCHERS_CONTAINS, [266](#)
- DOCTEST_PARTS_PUBLIC_MATCHERS_IS_NAN, [267](#)
- DOCTEST_PARTS_PUBLIC_PATH, [267](#)
- DOCTEST_PARTS_PUBLIC_PLATFORM, [267](#)
- DOCTEST_PARTS_PUBLIC_REPORTER, [267](#)
- DOCTEST_PARTS_PUBLIC_STD_FWD, [267](#)
- DOCTEST_PARTS_PUBLIC_STD_TYPE_TRAITS, [267](#)
- DOCTEST_PARTS_PUBLIC_STD_UTILITY, [267](#)
- DOCTEST_PARTS_PUBLIC_STRING, [267](#)
- DOCTEST_PARTS_PUBLIC_SUBCASE, [268](#)
- DOCTEST_PARTS_PUBLIC_TEST_CASE, [268](#)
- DOCTEST_PARTS_PUBLIC_TEST_SUITE, [268](#)
- DOCTEST_PARTS_PUBLIC_UTILITY, [268](#)
- DOCTEST_PARTS_PUBLIC_VERSION, [268](#)
- DOCTEST_PARTS_PUBLIC_WARNINGS, [268](#)
- DOCTEST_PLATFORM_LINUX, [268](#)
- DOCTEST_REF_WRAP, [268](#)
- DOCTEST_REGISTER_EXCEPTION_TRANSLATOR, [269](#)
- DOCTEST_REGISTER_EXCEPTION_TRANSLATOR_IMPL, [269](#)
- DOCTEST_REGISTER_FUNCTION, [269](#)
- DOCTEST_REGISTER_LISTENER, [269](#)
- DOCTEST_REGISTER_REPORTER, [270](#)
- DOCTEST_RELATIONAL_OP, [270](#)
- DOCTEST_REQUIRE, [270](#)
- DOCTEST_REQUIRE_EQ, [271](#)
- DOCTEST_REQUIRE_FALSE, [271](#)
- DOCTEST_REQUIRE_FALSE_MESSAGE, [271](#)
- DOCTEST_REQUIRE_GE, [272](#)
- DOCTEST_REQUIRE_GT, [272](#)
- DOCTEST_REQUIRE_LE, [272](#)
- DOCTEST_REQUIRE_LT, [273](#)
- DOCTEST_REQUIRE_MESSAGE, [273](#)
- DOCTEST_REQUIRE_NE, [273](#)
- DOCTEST_REQUIRE_NOTHROW, [274](#)
- DOCTEST_REQUIRE_NOTHROW_MESSAGE, [274](#)
- DOCTEST_REQUIRE_THROWS, [274](#)
- DOCTEST_REQUIRE_THROWS_AS, [274](#)
- DOCTEST_REQUIRE_THROWS_AS_MESSAGE, [274](#)
- DOCTEST_REQUIRE_THROWS_MESSAGE, [275](#)
- DOCTEST_REQUIRE_THROWS_WITH, [275](#)
- DOCTEST_REQUIRE_THROWS_WITH_AS, [275](#)
- DOCTEST_REQUIRE_THROWS_WITH_AS_MESSAGE, [275](#)
- DOCTEST_REQUIRE_THROWS_WITH_MESSAGE, [276](#)
- DOCTEST_REQUIRE_UNARY, [276](#)
- DOCTEST_REQUIRE_UNARY_FALSE, [276](#)
- DOCTEST_SCENARIO, [277](#)
- DOCTEST_SCENARIO_CLASS, [277](#)
- DOCTEST_SCENARIO_METHOD, [277](#)
- DOCTEST_SCENARIO_TEMPLATE, [277](#)
- DOCTEST_SCENARIO_TEMPLATE_DEFINE, [277](#)
- DOCTEST_STRINGIFY, [278](#)
- DOCTEST_SUBCASE, [278](#)
- DOCTEST_SUPPRESS_COMMON_WARNINGS_POP, [278](#)
- DOCTEST_SUPPRESS_COMMON_WARNINGS_PUSH, [278](#)
- DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP, [278](#)
- DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH, [279](#)
- DOCTEST_SUPPRESS_PUBLIC_WARNINGS_POP, [279](#)
- DOCTEST_SUPPRESS_PUBLIC_WARNINGS_PUSH, [279](#)
- DOCTEST_SYMBOL_EXPORT, [279](#)
- DOCTEST_SYMBOL_IMPORT, [279](#)

- DOCTEST_TEST_CASE, [279](#)
- DOCTEST_TEST_CASE_CLASS, [280](#)
- DOCTEST_TEST_CASE_FIXTURE, [280](#)
- DOCTEST_TEST_CASE_TEMPLATE, [280](#)
- DOCTEST_TEST_CASE_TEMPLATE_APPLY, [280](#)
- DOCTEST_TEST_CASE_TEMPLATE_DEFINE, [280](#)
- DOCTEST_TEST_CASE_TEMPLATE_DEFINE_IMPL, [281](#)
- DOCTEST_TEST_CASE_TEMPLATE_IMPL, [282](#)
- DOCTEST_TEST_CASE_TEMPLATE_INSTANTIATE, [282](#)
- DOCTEST_TEST_CASE_TEMPLATE_INSTANTIATE_IMPL, [282](#)
- DOCTEST_TEST_CASE_TEMPLATE_INVOKE, [282](#)
- DOCTEST_TEST_SUITE, [283](#)
- DOCTEST_TEST_SUITE_BEGIN, [283](#)
- DOCTEST_TEST_SUITE_END, [283](#)
- DOCTEST_TEST_SUITE_IMPL, [283](#)
- DOCTEST_THEN, [284](#)
- DOCTEST_TO_LVALUE, [284](#)
- DOCTEST_TOSTR, [284](#)
- DOCTEST_TOSTR_IMPL, [285](#)
- DOCTEST_TYPE_TO_STRING, [285](#)
- DOCTEST_TYPE_TO_STRING_AS, [285](#)
- DOCTEST_UNARY_ASSERT, [285](#)
- DOCTEST_UNUSED, [286](#)
- DOCTEST_VERSION, [286](#)
- DOCTEST_VERSION_MAJOR, [286](#)
- DOCTEST_VERSION_MINOR, [286](#)
- DOCTEST_VERSION_PATCH, [286](#)
- DOCTEST_VERSION_STR, [286](#)
- DOCTEST_WARN, [287](#)
- DOCTEST_WARN_EQ, [287](#)
- DOCTEST_WARN_FALSE, [287](#)
- DOCTEST_WARN_FALSE_MESSAGE, [287](#)
- DOCTEST_WARN_GE, [287](#)
- DOCTEST_WARN_GT, [288](#)
- DOCTEST_WARN_LE, [288](#)
- DOCTEST_WARN_LT, [288](#)
- DOCTEST_WARN_MESSAGE, [288](#)
- DOCTEST_WARN_NE, [288](#)
- DOCTEST_WARN_NOTHROW, [289](#)
- DOCTEST_WARN_NOTHROW_MESSAGE, [289](#)
- DOCTEST_WARN_THROWS, [289](#)
- DOCTEST_WARN_THROWS_AS, [289](#)
- DOCTEST_WARN_THROWS_AS_MESSAGE, [289](#)
- DOCTEST_WARN_THROWS_MESSAGE, [290](#)
- DOCTEST_WARN_THROWS_WITH, [290](#)
- DOCTEST_WARN_THROWS_WITH_AS, [290](#)
- DOCTEST_WARN_THROWS_WITH_AS_MESSAGE, [290](#)
- DOCTEST_WARN_THROWS_WITH_MESSAGE, [291](#)
- DOCTEST_WARN_UNARY, [291](#)
- DOCTEST_WARN_UNARY_FALSE, [291](#)
- DOCTEST_WHEN, [291](#)
- DOCTEST_WRAP_IN_TRY, [291](#)
- FAIL, [292](#)
- FAIL_CHECK, [292](#)
- FAST_CHECK_EQ, [292](#)
- FAST_CHECK_GE, [292](#)
- FAST_CHECK_GT, [292](#)
- FAST_CHECK_LE, [293](#)
- FAST_CHECK_LT, [293](#)
- FAST_CHECK_NE, [293](#)
- FAST_CHECK_UNARY, [293](#)
- FAST_CHECK_UNARY_FALSE, [293](#)
- FAST_REQUIRE_EQ, [294](#)
- FAST_REQUIRE_GE, [294](#)
- FAST_REQUIRE_GT, [294](#)
- FAST_REQUIRE_LE, [294](#)
- FAST_REQUIRE_LT, [294](#)
- FAST_REQUIRE_NE, [295](#)
- FAST_REQUIRE_UNARY, [295](#)
- FAST_REQUIRE_UNARY_FALSE, [295](#)
- FAST_WARN_EQ, [295](#)
- FAST_WARN_GE, [295](#)
- FAST_WARN_GT, [296](#)
- FAST_WARN_LE, [296](#)
- FAST_WARN_LT, [296](#)
- FAST_WARN_NE, [296](#)
- FAST_WARN_UNARY, [296](#)
- FAST_WARN_UNARY_FALSE, [297](#)
- GENERATE, [297](#)
- GIVEN, [297](#)
- INFO, [297](#)
- MESSAGE, [297](#)
- REGISTER_EXCEPTION_TRANSLATOR, [298](#)
- REGISTER_LISTENER, [298](#)
- REGISTER_REPORTER, [298](#)
- REQUIRE, [298](#)
- REQUIRE_EQ, [298](#)
- REQUIRE_FALSE, [299](#)
- REQUIRE_FALSE_MESSAGE, [299](#)
- REQUIRE_GE, [299](#)
- REQUIRE_GT, [299](#)
- REQUIRE_LE, [299](#)
- REQUIRE_LT, [300](#)
- REQUIRE_MESSAGE, [300](#)
- REQUIRE_NE, [300](#)
- REQUIRE_NOTHROW, [300](#)
- REQUIRE_NOTHROW_MESSAGE, [300](#)
- REQUIRE_THROWS, [301](#)
- REQUIRE_THROWS_AS, [301](#)
- REQUIRE_THROWS_AS_MESSAGE, [301](#)
- REQUIRE_THROWS_MESSAGE, [301](#)
- REQUIRE_THROWS_WITH, [301](#)
- REQUIRE_THROWS_WITH_AS, [302](#)
- REQUIRE_THROWS_WITH_AS_MESSAGE, [302](#)
- REQUIRE_THROWS_WITH_MESSAGE, [302](#)
- REQUIRE_UNARY, [302](#)
- REQUIRE_UNARY_FALSE, [303](#)

- SCENARIO, [303](#)
- SCENARIO_CLASS, [303](#)
- SCENARIO_METHOD, [303](#)
- SCENARIO_TEMPLATE, [303](#)
- SCENARIO_TEMPLATE_DEFINE, [304](#)
- SFINAE_OP, [304](#)
- SUBCASE, [304](#)
- TEST_CASE, [304](#)
- TEST_CASE_CLASS, [304](#)
- TEST_CASE_FIXTURE, [305](#)
- TEST_CASE_TEMPLATE, [305](#)
- TEST_CASE_TEMPLATE_APPLY, [305](#)
- TEST_CASE_TEMPLATE_DEFINE, [305](#)
- TEST_CASE_TEMPLATE_INSTANTIATE, [305](#)
- TEST_CASE_TEMPLATE_INVOKE, [306](#)
- TEST_SUITE, [306](#)
- TEST_SUITE_BEGIN, [306](#)
- TEST_SUITE_END, [306](#)
- THEN, [306](#)
- TO_LVALUE, [307](#)
- TYPE_TO_STRING, [307](#)
- TYPE_TO_STRING_AS, [307](#)
- WARN, [307](#)
- WARN_EQ, [307](#)
- WARN_FALSE, [308](#)
- WARN_FALSE_MESSAGE, [308](#)
- WARN_GE, [308](#)
- WARN_GT, [308](#)
- WARN_LE, [308](#)
- WARN_LT, [309](#)
- WARN_MESSAGE, [309](#)
- WARN_NE, [309](#)
- WARN_NOTHROW, [309](#)
- WARN_NOTHROW_MESSAGE, [309](#)
- WARN_THROWS, [310](#)
- WARN_THROWS_AS, [310](#)
- WARN_THROWS_AS_MESSAGE, [310](#)
- WARN_THROWS_MESSAGE, [310](#)
- WARN_THROWS_WITH, [310](#)
- WARN_THROWS_WITH_AS, [311](#)
- WARN_THROWS_WITH_AS_MESSAGE, [311](#)
- WARN_THROWS_WITH_MESSAGE, [311](#)
- WARN_UNARY, [311](#)
- WARN_UNARY_FALSE, [312](#)
- WHEN, [312](#)
- doctest::Approx, [52](#)
 - Approx, [53](#)
 - epsilon, [53](#)
 - m_epsilon, [55](#)
 - m_scale, [55](#)
 - m_value, [55](#)
 - operator!=, [53](#)
 - operator<, [53](#), [54](#)
 - operator<=, [54](#)
 - operator>, [54](#)
 - operator>=, [54](#), [55](#)
 - operator(), [53](#)
 - operator==, [54](#)
 - scale, [53](#)
- doctest::AssertData, [55](#)
 - AssertData, [56](#)
 - m_at, [56](#)
 - m_decomp, [56](#)
 - m_exception, [57](#)
 - m_exception_string, [57](#)
 - m_exception_type, [57](#)
 - m_expr, [57](#)
 - m_failed, [57](#)
 - m_file, [57](#)
 - m_line, [57](#)
 - m_test_case, [57](#)
 - m_threw, [58](#)
 - m_threw_as, [58](#)
- doctest::AssertData::StringContains, [165](#)
 - c_str, [166](#)
 - check, [166](#)
 - operator const String &, [166](#)
 - StringContains, [166](#)
- doctest::assertType, [34](#)
 - DT_CHECK, [35](#)
 - DT_CHECK_EQ, [36](#)
 - DT_CHECK_FALSE, [36](#)
 - DT_CHECK_GE, [36](#)
 - DT_CHECK_GT, [36](#)
 - DT_CHECK_LE, [36](#)
 - DT_CHECK_LT, [36](#)
 - DT_CHECK_NE, [36](#)
 - DT_CHECK_NOTHROW, [36](#)
 - DT_CHECK_THROWS, [36](#)
 - DT_CHECK_THROWS_AS, [36](#)
 - DT_CHECK_THROWS_WITH, [36](#)
 - DT_CHECK_THROWS_WITH_AS, [36](#)
 - DT_CHECK_UNARY, [36](#)
 - DT_CHECK_UNARY_FALSE, [36](#)
 - DT_REQUIRE, [35](#)
 - DT_REQUIRE_EQ, [36](#)
 - DT_REQUIRE_FALSE, [36](#)
 - DT_REQUIRE_GE, [36](#)
 - DT_REQUIRE_GT, [36](#)
 - DT_REQUIRE_LE, [36](#)
 - DT_REQUIRE_LT, [36](#)
 - DT_REQUIRE_NE, [36](#)
 - DT_REQUIRE_NOTHROW, [36](#)
 - DT_REQUIRE_THROWS, [36](#)
 - DT_REQUIRE_THROWS_AS, [36](#)
 - DT_REQUIRE_THROWS_WITH, [36](#)
 - DT_REQUIRE_THROWS_WITH_AS, [36](#)
 - DT_REQUIRE_UNARY, [36](#)
 - DT_REQUIRE_UNARY_FALSE, [36](#)
 - DT_WARN, [35](#)
 - DT_WARN_EQ, [36](#)
 - DT_WARN_FALSE, [36](#)
 - DT_WARN_GE, [36](#)
 - DT_WARN_GT, [36](#)
 - DT_WARN_LE, [36](#)
 - DT_WARN_LT, [36](#)

- DT_WARN_NE, 36
- DT_WARN_NOTHROW, 36
- DT_WARN_THROWS, 36
- DT_WARN_THROWS_AS, 36
- DT_WARN_THROWS_WITH, 36
- DT_WARN_THROWS_WITH_AS, 36
- DT_WARN_UNARY, 36
- DT_WARN_UNARY_FALSE, 36
- Enum, 35
- is_check, 35
- is_eq, 35
- is_false, 35
- is_ge, 35
- is_gt, 35
- is_le, 35
- is_lt, 35
- is_ne, 35
- is_normal, 35
- is_nothrow, 35
- is_require, 35
- is_throws, 35
- is_throws_as, 35
- is_throws_with, 35
- is_unary, 35
- is_warn, 35
- doctest::Color, 37
 - Blue, 37
 - Bright, 37
 - BrightGreen, 37
 - BrightRed, 37
 - BrightWhite, 37
 - Cyan, 37
 - Enum, 37
 - Green, 37
 - Grey, 37
 - LightGrey, 37
 - None, 37
 - operator<<, 37
 - Red, 37
 - White, 37
 - Yellow, 37
- doctest::Contains, 67
 - checkWith, 68
 - Contains, 68
 - string, 68
- doctest::Context, 68
 - ~Context, 69
 - addFilter, 69
 - applyCommandLine, 69
 - clearFilters, 69
 - Context, 69
 - operator=, 69, 70
 - run, 70
 - setAsDefaultForAssertsOutOfTestCases, 70
 - setAssertHandler, 70
 - setCout, 70
 - setOption, 70
 - shouldExit, 70
- doctest::ContextOptions, 71
 - abort_after, 72
 - binary_name, 72
 - case_sensitive, 72
 - count, 72
 - cout, 72
 - currentTest, 72
 - duration, 72
 - exit, 73
 - first, 73
 - force_colors, 73
 - gnu_file_line, 73
 - help, 73
 - last, 73
 - list_reporters, 73
 - list_test_cases, 73
 - list_test_suites, 74
 - minimal, 74
 - no_breaks, 74
 - no_colors, 74
 - no_debug_output, 74
 - no_exitcode, 74
 - no_intro, 74
 - no_line_numbers, 74
 - no_path_in_filenames, 75
 - no_run, 75
 - no_skip, 75
 - no_skipped_summary, 75
 - no_throw, 75
 - no_time_in_output, 75
 - no_version, 75
 - order_by, 75
 - out, 76
 - quiet, 76
 - rand_seed, 76
 - strip_file_prefixes, 76
 - subcase_filter_levels, 76
 - success, 76
 - version, 76
- doctest::CurrentTestCaseStats, 81
 - failure_flags, 81
 - numAssertsCurrentTest, 81
 - numAssertsFailedCurrentTest, 82
 - seconds, 82
 - testCaseSuccess, 82
- doctest::detail, 38
 - acquireGeneratorDecisionIndex, 40
 - acquireGeneratorValue, 40
 - assert_handler, 40
 - binary_assert, 40
 - checkIfShouldThrow, 41
 - declval, 41
 - decomp_assert, 41
 - failed_out_of_a_testing_context, 41
 - filloss, 41
 - forward, 42
 - funcType, 40
 - instantiationHelper, 42

- isDebuggerActive, [42](#)
- MakeContextScope, [42](#)
- registerExceptionTranslatorImpl, [42](#)
- registerReporterImpl, [42](#)
- regTest, [43](#)
- reporterCreator, [43](#)
- reporterCreatorFunc, [40](#)
- setTestSuite, [43](#)
- stringifyBinaryExpr, [43](#)
- TestCase, [44](#)
- throwException, [43](#)
- tlssPop, [43](#)
- tlssPush, [43](#)
- toStream, [44](#)
- unary_assert, [44](#)
- doctest::detail::binaryAssertComparison, [44](#)
 - Enum, [45](#)
 - eq, [45](#)
 - ge, [45](#)
 - gt, [45](#)
 - le, [45](#)
 - lt, [45](#)
 - ne, [45](#)
- doctest::detail::ContextScope< L >, [77](#)
 - ~ContextScope, [78](#)
 - ContextScope, [78](#)
 - operator=, [79](#)
 - stringify, [79](#)
- doctest::detail::ContextScopeBase, [79](#)
 - ~ContextScopeBase, [80](#)
 - ContextScopeBase, [80](#)
 - destroy, [80](#)
 - need_to_destroy, [81](#)
 - operator=, [80](#), [81](#)
- doctest::detail::deferred_false< T >, [82](#)
- doctest::detail::ExceptionTranslator< T >, [93](#)
 - ExceptionTranslator, [94](#)
 - translate, [94](#)
- doctest::detail::Expression_lhs< L >, [94](#)
 - Expression_lhs, [95](#)
 - lhs, [95](#)
 - m_at, [95](#)
 - operator L, [95](#)
 - operator Result, [95](#)
- doctest::detail::ExpressionDecomposer, [96](#)
 - ExpressionDecomposer, [96](#)
 - m_at, [97](#)
 - operator<<, [96](#)
- doctest::detail::filldata< const char[N]>, [98](#)
 - fill, [99](#)
- doctest::detail::filldata< const void * >, [99](#)
 - fill, [99](#)
- doctest::detail::filldata< const volatile void * >, [100](#)
 - fill, [100](#)
- doctest::detail::filldata< T * >, [100](#)
 - fill, [101](#)
- doctest::detail::filldata< T, Enable >, [98](#)
 - fill, [98](#)
- doctest::detail::filldata< T, typename detail::types::enable_if<
 !detail::has_insertion_operator< T >::value
 &&detail::is_container< T >::value >::type
 >, [101](#)
 - fill, [101](#)
- doctest::detail::filldata< T, typename detail::types::enable_if<!detail::has_i
 T >::value &&detail::is_pair< T >::value
 >::type >, [102](#)
 - fill, [102](#)
- doctest::detail::filldata< T[N]>, [102](#)
 - fill, [103](#)
- doctest::detail::has_insertion_operator< T, decltype(operator<<(declval<
 std::ostream & >()), declval< const T & >()),
 void())>, [104](#)
- doctest::detail::has_insertion_operator< T, typename
 >, [103](#)
- doctest::detail::lExceptionTranslator, [107](#)
 - translate, [107](#)
- doctest::detail::is_container< T, typename >, [111](#)
- doctest::detail::is_container< T, types::void_t< type-
 name T::value_type, typename T::iterator,
 decltype(declval< T >().begin()), decltype(declval<
 T >().end())> >, [112](#)
- doctest::detail::is_pair< T, typename >, [113](#)
- doctest::detail::is_pair< T, types::void_t< typename
 T::first_type, typename T::second_type,
 decltype(declval< T >().first), decltype(declval<
 T >().second)> >, [114](#)
- doctest::detail::is_std_string< T, Enable >, [118](#)
- doctest::detail::is_std_string< T, typename types::enable_if<
 types::is_same< decltype(declval< const
 T & >().c_str()), const char * >::value
 &&types::is_same< decltype(declval< const
 T & >().size()), size_t >::value >::type >,
 [119](#)
- doctest::detail::MessageBuilder, [130](#)
 - ~MessageBuilder, [132](#)
 - log, [132](#)
 - logged, [133](#)
 - m_stream, [133](#)
 - MessageBuilder, [131](#)
 - operator<<, [132](#)
 - operator,, [132](#)
 - operator=, [132](#)
 - operator*, [132](#)
 - react, [133](#)
- doctest::detail::RelationalComparator< int, L, R >, [143](#)
 - operator(), [144](#)
- doctest::detail::Result, [147](#)
 - m_decomp, [148](#)
 - m_passed, [148](#)
 - Result, [148](#)
- doctest::detail::ResultBuilder, [148](#)
 - binary_assert, [150](#)
 - log, [150](#)
 - react, [150](#)
 - ResultBuilder, [149](#)
 - setResult, [150](#)

- translateException, 150
- unary_assert, 150
- doctest::detail::should_stringify_as_underlying_type< T >, 153
 - value, 154
- doctest::detail::StringMakerBase< C >, 167
 - convert, 168
- doctest::detail::StringMakerBase< true >, 168
 - convert, 169
- doctest::detail::Subcase, 169
 - ~Subcase, 170
 - m_entered, 170
 - m_signature, 170
 - operator bool, 170
 - operator=, 170
 - Subcase, 169, 170
- doctest::detail::TestCase, 172
 - ~TestCase, 173
 - m_full_name, 174
 - m_template_id, 174
 - m_test, 174
 - m_type, 175
 - operator<, 174
 - operator=, 174
 - operator*, 174
 - TestCase, 173
- doctest::detail::TestFailureException, 178
- doctest::detail::TestSuite, 180
 - m_description, 180
 - m_expected_failures, 180
 - m_may_fail, 181
 - m_no_breaks, 181
 - m_no_output, 181
 - m_should_fail, 181
 - m_skip, 181
 - m_test_suite, 181
 - m_timeout, 181
 - operator*, 180
- doctest::detail::types, 45
 - void_t, 46
- doctest::detail::types::enable_if< COND, T >, 84
- doctest::detail::types::enable_if< true, T >, 84
 - type, 85
- doctest::detail::types::false_type, 97
 - value, 98
- doctest::detail::types::is_array< T >, 110
- doctest::detail::types::is_array< T[SIZE]>, 110
- doctest::detail::types::is_enum< T >, 112
 - value, 113
- doctest::detail::types::is_pointer< T >, 114
- doctest::detail::types::is_pointer< T * >, 115
- doctest::detail::types::is_rvalue_reference< T >, 116
- doctest::detail::types::is_rvalue_reference< T && >, 116
- doctest::detail::types::is_same< T, T >, 118
- doctest::detail::types::is_same< T, U >, 117
- doctest::detail::types::remove_const< const T >, 145
 - type, 145
- doctest::detail::types::remove_const< T >, 144
 - type, 144
- doctest::detail::types::remove_reference< T >, 145
 - type, 146
- doctest::detail::types::remove_reference< T & >, 146
 - type, 146
- doctest::detail::types::remove_reference< T && >, 146
 - type, 147
- doctest::detail::types::true_type, 182
 - value, 182
- doctest::detail::types::underlying_type< T >, 183
 - type, 183
- doctest::IContextScope, 104
 - stringify, 105
- doctest::IReporter, 107
 - get_active_contexts, 108
 - get_num_active_contexts, 108
 - get_num_stringified_contexts, 108
 - get_stringified_contexts, 108
 - log_assert, 108
 - log_message, 108
 - report_query, 108
 - subcase_end, 108
 - subcase_start, 109
 - test_case_end, 109
 - test_case_exception, 109
 - test_case_reenter, 109
 - test_case_skipped, 109
 - test_case_start, 109
 - test_run_end, 109
 - test_run_start, 109
- doctest::IsNaN< F >, 120
 - flipped, 121
 - IsNaN, 120
 - operator bool, 121
 - operator!, 121
 - value, 121
- doctest::MessageData, 133
 - m_file, 134
 - m_line, 134
 - m_severity, 134
 - m_string, 134
- doctest::QueryData, 142
 - data, 143
 - num_data, 143
 - run_stats, 143
- doctest::String, 161
 - ~String, 162
 - buf, 165
 - c_str, 163
 - capacity, 163
 - compare, 163
 - data, 165
 - find, 163
 - npos, 165
 - operator<=, 165
 - operator+=, 163
 - operator=, 164

- operator[], [164](#)
- rfind, [164](#)
- size, [164](#)
- size_type, [162](#)
- String, [162](#), [163](#)
- substr, [164](#)
- doctest::StringMaker< T >, [167](#)
- doctest::SubcaseSignature, [171](#)
 - m_file, [171](#)
 - m_line, [171](#)
 - m_name, [171](#)
 - operator<, [171](#)
 - operator==, [171](#)
- doctest::TestCaseData, [175](#)
 - m_description, [176](#)
 - m_expected_failures, [176](#)
 - m_file, [176](#)
 - m_line, [176](#)
 - m_may_fail, [176](#)
 - m_name, [176](#)
 - m_no_breaks, [176](#)
 - m_no_output, [176](#)
 - m_should_fail, [177](#)
 - m_skip, [177](#)
 - m_test_suite, [177](#)
 - m_timeout, [177](#)
- doctest::TestCaseException, [177](#)
 - error_string, [178](#)
 - is_crash, [178](#)
- doctest::TestCaseFailureReason, [46](#)
 - AssertFailure, [46](#)
 - CouldHaveFailedAndDid, [46](#)
 - Crash, [46](#)
 - DidntFailExactlyNumTimes, [46](#)
 - Enum, [46](#)
 - Exception, [46](#)
 - FailedExactlyNumTimes, [46](#)
 - None, [46](#)
 - ShouldHaveFailedAndDid, [46](#)
 - ShouldHaveFailedButDidnt, [46](#)
 - Timeout, [46](#)
 - TooManyFailedAsserts, [46](#)
- doctest::TestRunStats, [178](#)
 - numAsserts, [179](#)
 - numAssertsFailed, [179](#)
 - numTestCases, [179](#)
 - numTestCasesFailed, [179](#)
 - numTestCasesPassingFilters, [179](#)
 - numTestSuitesPassingFilters, [179](#)
- DOCTEST_ADD_AT_IMPL
 - doctest.h, [239](#)
- DOCTEST_ADD_FAIL_AT
 - doctest.h, [239](#)
- DOCTEST_ADD_FAIL_CHECK_AT
 - doctest.h, [239](#)
- DOCTEST_ADD_MESSAGE_AT
 - doctest.h, [240](#)
- DOCTEST_ALIGNMENT
 - doctest.h, [240](#)
- DOCTEST_AND_GIVEN
 - doctest.h, [240](#)
- DOCTEST_AND_THEN
 - doctest.h, [240](#)
- DOCTEST_AND_WHEN
 - doctest.h, [240](#)
- DOCTEST_ANONYMOUS
 - doctest.h, [241](#)
- DOCTEST_ASSERT_IMPLEMENT_1
 - doctest.h, [241](#)
- DOCTEST_ASSERT_IMPLEMENT_2
 - doctest.h, [241](#)
- DOCTEST_ASSERT_IN_TESTS
 - doctest.h, [241](#)
- DOCTEST_ASSERT_LOG_REACT_RETURN
 - doctest.h, [242](#)
- DOCTEST_ASSERT_OUT_OF_TESTS
 - doctest.h, [242](#)
- DOCTEST_BINARY_ASSERT
 - doctest.h, [242](#)
- DOCTEST_BINARY_RELATIONAL_OP
 - doctest.h, [243](#)
- DOCTEST_BREAK_INTO_DEBUGGER
 - doctest.h, [243](#)
- DOCTEST_CAPTURE
 - doctest.h, [243](#)
- DOCTEST_CAST_TO_VOID
 - doctest.h, [243](#)
- DOCTEST_CAT
 - doctest.h, [244](#)
- DOCTEST_CAT_IMPL
 - doctest.h, [244](#)
- DOCTEST_CHECK
 - doctest.h, [244](#)
- DOCTEST_CHECK_EQ
 - doctest.h, [244](#)
- DOCTEST_CHECK_FALSE
 - doctest.h, [244](#)
- DOCTEST_CHECK_FALSE_MESSAGE
 - doctest.h, [245](#)
- DOCTEST_CHECK_GE
 - doctest.h, [245](#)
- DOCTEST_CHECK_GT
 - doctest.h, [245](#)
- DOCTEST_CHECK_LE
 - doctest.h, [245](#)
- DOCTEST_CHECK_LT
 - doctest.h, [245](#)
- DOCTEST_CHECK_MESSAGE
 - doctest.h, [246](#)
- DOCTEST_CHECK_NE
 - doctest.h, [246](#)
- DOCTEST_CHECK_NOTHROW
 - doctest.h, [246](#)
- DOCTEST_CHECK_NOTHROW_MESSAGE
 - doctest.h, [246](#)
- DOCTEST_CHECK_THROWS

- doctest.h, [246](#)
- DOCTEST_CHECK_THROWS_AS
 - doctest.h, [247](#)
- DOCTEST_CHECK_THROWS_AS_MESSAGE
 - doctest.h, [247](#)
- DOCTEST_CHECK_THROWS_MESSAGE
 - doctest.h, [247](#)
- DOCTEST_CHECK_THROWS_WITH
 - doctest.h, [247](#)
- DOCTEST_CHECK_THROWS_WITH_AS
 - doctest.h, [247](#)
- DOCTEST_CHECK_THROWS_WITH_AS_MESSAGE
 - doctest.h, [248](#)
- DOCTEST_CHECK_THROWS_WITH_MESSAGE
 - doctest.h, [248](#)
- DOCTEST_CHECK_UNARY
 - doctest.h, [248](#)
- DOCTEST_CHECK_UNARY_FALSE
 - doctest.h, [248](#)
- DOCTEST_CLANG
 - doctest.h, [249](#)
- DOCTEST_CLANG_SUPPRESS_WARNING
 - doctest.h, [249](#)
- DOCTEST_CLANG_SUPPRESS_WARNING_POP
 - doctest.h, [249](#)
- DOCTEST_CLANG_SUPPRESS_WARNING_PUSH
 - doctest.h, [249](#)
- DOCTEST_CLANG_SUPPRESS_WARNING_WITH_PUSH
 - doctest.h, [249](#)
- DOCTEST_CMP_EQ
 - doctest.h, [249](#)
- DOCTEST_CMP_GE
 - doctest.h, [249](#)
- DOCTEST_CMP_GT
 - doctest.h, [250](#)
- DOCTEST_CMP_LE
 - doctest.h, [250](#)
- DOCTEST_CMP_LT
 - doctest.h, [250](#)
- DOCTEST_CMP_NE
 - doctest.h, [250](#)
- DOCTEST_COMPARISON_RETURN_TYPE
 - doctest.h, [250](#)
- DOCTEST_COMPILER
 - doctest.h, [251](#)
- DOCTEST_CONFIG_IMPLEMENT_WITH_MAIN
 - test_main.cpp, [454](#)
- DOCTEST_CONFIG_NO_EXCEPTIONS
 - doctest.h, [251](#)
- DOCTEST_CONFIG_NO_TRY_CATCH_IN_ASSERTS
 - doctest.h, [251](#)
- DOCTEST_CONFIG_POSIX_SIGNALS
 - doctest.h, [251](#)
- DOCTEST_CONFIG_STRING_SIZE_TYPE
 - doctest.h, [251](#)
- DOCTEST_CONSTEXPR
 - doctest.h, [251](#)
- DOCTEST_CONSTEXPR_FUNC
 - doctest.h, [251](#)
- DOCTEST_COUNTER
 - doctest.h, [252](#)
- DOCTEST_CPLUSPLUS
 - doctest.h, [252](#)
- DOCTEST_CREATE_AND_REGISTER_FUNCTION
 - doctest.h, [252](#)
- DOCTEST_CREATE_AND_REGISTER_FUNCTION_IN_CLASS
 - doctest.h, [252](#)
- DOCTEST_DECLARE_INTERFACE
 - doctest.h, [252](#)
- DOCTEST_DEFINE_DECORATOR
 - doctest, [27](#), [28](#)
 - doctest.h, [253](#)
- DOCTEST_DEFINE_INTERFACE
 - doctest.h, [253](#)
- doctest_detail_test_suite_ns, [46](#)
 - getCurrentTestSuite, [47](#)
- DOCTEST_DO_BINARY_EXPRESSION_COMPARISON
 - doctest.h, [253](#)
- DOCTEST_EMPTY
 - doctest.h, [254](#)
- DOCTEST_EXCEPTION_EMPTY_FUNC
 - doctest.h, [254](#)
- DOCTEST_FAIL
 - doctest.h, [254](#)
- DOCTEST_FAIL_CHECK
 - doctest.h, [255](#)
- DOCTEST_FAST_CHECK_EQ
 - doctest.h, [255](#)
- DOCTEST_FAST_CHECK_GE
 - doctest.h, [255](#)
- DOCTEST_FAST_CHECK_GT
 - doctest.h, [255](#)
- DOCTEST_FAST_CHECK_LE
 - doctest.h, [255](#)
- DOCTEST_FAST_CHECK_LT
 - doctest.h, [255](#)
- DOCTEST_FAST_CHECK_NE
 - doctest.h, [255](#)
- DOCTEST_FAST_CHECK_UNARY
 - doctest.h, [256](#)
- DOCTEST_FAST_CHECK_UNARY_FALSE
 - doctest.h, [256](#)
- DOCTEST_FAST_REQUIRE_EQ
 - doctest.h, [256](#)
- DOCTEST_FAST_REQUIRE_GE
 - doctest.h, [256](#)
- DOCTEST_FAST_REQUIRE_GT
 - doctest.h, [256](#)
- DOCTEST_FAST_REQUIRE_LE
 - doctest.h, [256](#)
- DOCTEST_FAST_REQUIRE_LT
 - doctest.h, [256](#)
- DOCTEST_FAST_REQUIRE_NE
 - doctest.h, [256](#)
- DOCTEST_FAST_REQUIRE_UNARY
 - doctest.h, [257](#)

- DOCTEST_FAST_REQUIRE_UNARY_FALSE
 - doctest.h, [257](#)
- DOCTEST_FAST_WARN_EQ
 - doctest.h, [257](#)
- DOCTEST_FAST_WARN_GE
 - doctest.h, [257](#)
- DOCTEST_FAST_WARN_GT
 - doctest.h, [257](#)
- DOCTEST_FAST_WARN_LE
 - doctest.h, [257](#)
- DOCTEST_FAST_WARN_LT
 - doctest.h, [257](#)
- DOCTEST_FAST_WARN_NE
 - doctest.h, [257](#)
- DOCTEST_FAST_WARN_UNARY
 - doctest.h, [258](#)
- DOCTEST_FAST_WARN_UNARY_FALSE
 - doctest.h, [258](#)
- DOCTEST_FORBIT_EXPRESSION
 - doctest.h, [258](#)
- DOCTEST_FUNC_EMPTY
 - doctest.h, [258](#)
- DOCTEST_FUNC_SCOPE_BEGIN
 - doctest.h, [258](#)
- DOCTEST_FUNC_SCOPE_END
 - doctest.h, [258](#)
- DOCTEST_FUNC_SCOPE_RET
 - doctest.h, [259](#)
- DOCTEST_GCC
 - doctest.h, [259](#)
- DOCTEST_GCC_SUPPRESS_WARNING
 - doctest.h, [259](#)
- DOCTEST_GCC_SUPPRESS_WARNING_POP
 - doctest.h, [259](#)
- DOCTEST_GCC_SUPPRESS_WARNING_PUSH
 - doctest.h, [259](#)
- DOCTEST_GCC_SUPPRESS_WARNING_WITH_PUSH
 - doctest.h, [259](#)
- DOCTEST_GENERATE
 - doctest.h, [259](#)
- DOCTEST_GIVEN
 - doctest.h, [260](#)
- DOCTEST_GLOBAL_NO_WARNINGS
 - doctest.h, [260](#)
- DOCTEST_HAS_BUILTIN
 - doctest.h, [260](#)
- DOCTEST_ICC
 - doctest.h, [260](#)
- DOCTEST_IMPLEMENT_FIXTURE
 - doctest.h, [260](#)
- DOCTEST_INFO
 - doctest.h, [261](#)
- DOCTEST_INFO_IMPL
 - doctest.h, [261](#)
- DOCTEST_INLINE_NOINLINE
 - doctest.h, [261](#)
- DOCTEST_INTERFACE
 - doctest.h, [262](#)
- DOCTEST_INTERFACE_DECL
 - doctest.h, [262](#)
- DOCTEST_INTERFACE_DEF
 - doctest.h, [262](#)
- DOCTEST_MAKE_STD_HEADERS_CLEAN_FROM_WARNINGS_ON_V
 - doctest.h, [262](#)
- DOCTEST_MAKE_STD_HEADERS_CLEAN_FROM_WARNINGS_ON_V
 - doctest.h, [262](#)
- DOCTEST_MESSAGE
 - doctest.h, [263](#)
- DOCTEST_MSVC
 - doctest.h, [263](#)
- DOCTEST_MSVC_SUPPRESS_WARNING
 - doctest.h, [263](#)
- DOCTEST_MSVC_SUPPRESS_WARNING_POP
 - doctest.h, [263](#)
- DOCTEST_MSVC_SUPPRESS_WARNING_PUSH
 - doctest.h, [263](#)
- DOCTEST_MSVC_SUPPRESS_WARNING_WITH_PUSH
 - doctest.h, [263](#)
- DOCTEST_NO_SANITIZE_INTEGER
 - doctest.h, [263](#)
- DOCTEST_NOEXCEPT
 - doctest.h, [264](#)
- DOCTEST_NOINLINE
 - doctest.h, [264](#)
- DOCTEST_NORETURN
 - doctest.h, [264](#)
- DOCTEST_PARTS_PUBLIC_ASSERT_COMPARATOR
 - doctest.h, [264](#)
- DOCTEST_PARTS_PUBLIC_ASSERT_DATA
 - doctest.h, [264](#)
- DOCTEST_PARTS_PUBLIC_ASSERT_EXPRESSION
 - doctest.h, [264](#)
- DOCTEST_PARTS_PUBLIC_ASSERT_HANDLER
 - doctest.h, [264](#)
- DOCTEST_PARTS_PUBLIC_ASSERT_MESSAGE
 - doctest.h, [264](#)
- DOCTEST_PARTS_PUBLIC_ASSERT_RESULT
 - doctest.h, [265](#)
- DOCTEST_PARTS_PUBLIC_ASSERT_TYPE
 - doctest.h, [265](#)
- DOCTEST_PARTS_PUBLIC_COLOR
 - doctest.h, [265](#)
- DOCTEST_PARTS_PUBLIC_COMPILER
 - doctest.h, [265](#)
- DOCTEST_PARTS_PUBLIC_CONFIG
 - doctest.h, [265](#)
- DOCTEST_PARTS_PUBLIC_CONTEXT
 - doctest.h, [265](#)
- DOCTEST_PARTS_PUBLIC_CONTEXT_OPTIONS
 - doctest.h, [265](#)
- DOCTEST_PARTS_PUBLIC_CONTEXT_SCOPE
 - doctest.h, [265](#)
- DOCTEST_PARTS_PUBLIC_DEBUGGER
 - doctest.h, [266](#)
- DOCTEST_PARTS_PUBLIC_DECORATORS
 - doctest.h, [266](#)

- DOCTEST_PARTS_PUBLIC_EXCEPTION_TRANSLATOR
 - doctest.h, [266](#)
- DOCTEST_PARTS_PUBLIC_EXCEPTIONS
 - doctest.h, [266](#)
- DOCTEST_PARTS_PUBLIC_GENERATOR
 - doctest.h, [266](#)
- DOCTEST_PARTS_PUBLIC_MACROS
 - doctest.h, [266](#)
- DOCTEST_PARTS_PUBLIC_MATCHERS_APPROX
 - doctest.h, [266](#)
- DOCTEST_PARTS_PUBLIC_MATCHERS_CONTAINS
 - doctest.h, [266](#)
- DOCTEST_PARTS_PUBLIC_MATCHERS_IS_NAN
 - doctest.h, [267](#)
- DOCTEST_PARTS_PUBLIC_PATH
 - doctest.h, [267](#)
- DOCTEST_PARTS_PUBLIC_PLATFORM
 - doctest.h, [267](#)
- DOCTEST_PARTS_PUBLIC_REPORTER
 - doctest.h, [267](#)
- DOCTEST_PARTS_PUBLIC_STD_FWD
 - doctest.h, [267](#)
- DOCTEST_PARTS_PUBLIC_STD_TYPE_TRAITS
 - doctest.h, [267](#)
- DOCTEST_PARTS_PUBLIC_STD_UTILITY
 - doctest.h, [267](#)
- DOCTEST_PARTS_PUBLIC_STRING
 - doctest.h, [267](#)
- DOCTEST_PARTS_PUBLIC_SUBCASE
 - doctest.h, [268](#)
- DOCTEST_PARTS_PUBLIC_TEST_CASE
 - doctest.h, [268](#)
- DOCTEST_PARTS_PUBLIC_TEST_SUITE
 - doctest.h, [268](#)
- DOCTEST_PARTS_PUBLIC_UTILITY
 - doctest.h, [268](#)
- DOCTEST_PARTS_PUBLIC_VERSION
 - doctest.h, [268](#)
- DOCTEST_PARTS_PUBLIC_WARNINGS
 - doctest.h, [268](#)
- DOCTEST_PLATFORM_LINUX
 - doctest.h, [268](#)
- DOCTEST_REF_WRAP
 - doctest.h, [268](#)
- DOCTEST_REGISTER_EXCEPTION_TRANSLATOR
 - doctest.h, [269](#)
- DOCTEST_REGISTER_EXCEPTION_TRANSLATOR_IMPL
 - doctest.h, [269](#)
- DOCTEST_REGISTER_FUNCTION
 - doctest.h, [269](#)
- DOCTEST_REGISTER_LISTENER
 - doctest.h, [269](#)
- DOCTEST_REGISTER_REPORTER
 - doctest.h, [270](#)
- DOCTEST_RELATIONAL_OP
 - doctest.h, [270](#)
- DOCTEST_REQUIRE
 - doctest.h, [270](#)
- DOCTEST_REQUIRE_EQ
 - doctest.h, [271](#)
- DOCTEST_REQUIRE_FALSE
 - doctest.h, [271](#)
- DOCTEST_REQUIRE_FALSE_MESSAGE
 - doctest.h, [271](#)
- DOCTEST_REQUIRE_GE
 - doctest.h, [272](#)
- DOCTEST_REQUIRE_GT
 - doctest.h, [272](#)
- DOCTEST_REQUIRE_LE
 - doctest.h, [272](#)
- DOCTEST_REQUIRE_LT
 - doctest.h, [273](#)
- DOCTEST_REQUIRE_MESSAGE
 - doctest.h, [273](#)
- DOCTEST_REQUIRE_NE
 - doctest.h, [273](#)
- DOCTEST_REQUIRE_NOTHROW
 - doctest.h, [274](#)
- DOCTEST_REQUIRE_NOTHROW_MESSAGE
 - doctest.h, [274](#)
- DOCTEST_REQUIRE_THROWS
 - doctest.h, [274](#)
- DOCTEST_REQUIRE_THROWS_AS
 - doctest.h, [274](#)
- DOCTEST_REQUIRE_THROWS_AS_MESSAGE
 - doctest.h, [274](#)
- DOCTEST_REQUIRE_THROWS_MESSAGE
 - doctest.h, [275](#)
- DOCTEST_REQUIRE_THROWS_WITH
 - doctest.h, [275](#)
- DOCTEST_REQUIRE_THROWS_WITH_AS
 - doctest.h, [275](#)
- DOCTEST_REQUIRE_THROWS_WITH_AS_MESSAGE
 - doctest.h, [275](#)
- DOCTEST_REQUIRE_THROWS_WITH_MESSAGE
 - doctest.h, [276](#)
- DOCTEST_REQUIRE_UNARY
 - doctest.h, [276](#)
- DOCTEST_REQUIRE_UNARY_FALSE
 - doctest.h, [276](#)
- DOCTEST_SCENARIO
 - doctest.h, [277](#)
- DOCTEST_SCENARIO_CLASS
 - doctest.h, [277](#)
- DOCTEST_SCENARIO_METHOD
 - doctest.h, [277](#)
- DOCTEST_SCENARIO_TEMPLATE
 - doctest.h, [277](#)
- DOCTEST_SCENARIO_TEMPLATE_DEFINE
 - doctest.h, [277](#)
- DOCTEST_STRINGIFY
 - doctest.h, [278](#)
- DOCTEST_SUBCASE
 - doctest.h, [278](#)
- DOCTEST_SUPPRESS_COMMON_WARNINGS_POP
 - doctest.h, [278](#)

DOCTEST_SUPPRESS_COMMON_WARNINGS_PUSH doctest.h, [278](#)
DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP doctest.h, [278](#)
DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH doctest.h, [279](#)
DOCTEST_SUPPRESS_PUBLIC_WARNINGS_POP doctest.h, [279](#)
DOCTEST_SUPPRESS_PUBLIC_WARNINGS_PUSH doctest.h, [279](#)
DOCTEST_SYMBOL_EXPORT doctest.h, [279](#)
DOCTEST_SYMBOL_IMPORT doctest.h, [279](#)
DOCTEST_TEST_CASE doctest.h, [279](#)
DOCTEST_TEST_CASE_CLASS doctest.h, [280](#)
DOCTEST_TEST_CASE_FIXTURE doctest.h, [280](#)
DOCTEST_TEST_CASE_TEMPLATE doctest.h, [280](#)
DOCTEST_TEST_CASE_TEMPLATE_APPLY doctest.h, [280](#)
DOCTEST_TEST_CASE_TEMPLATE_DEFINE doctest.h, [280](#)
DOCTEST_TEST_CASE_TEMPLATE_DEFINE_IMPL doctest.h, [281](#)
DOCTEST_TEST_CASE_TEMPLATE_IMPL doctest.h, [282](#)
DOCTEST_TEST_CASE_TEMPLATE_INSTANTIATE doctest.h, [282](#)
DOCTEST_TEST_CASE_TEMPLATE_INSTANTIATE_IMPL doctest.h, [282](#)
DOCTEST_TEST_CASE_TEMPLATE_INVOKE doctest.h, [282](#)
DOCTEST_TEST_SUITE doctest.h, [283](#)
DOCTEST_TEST_SUITE_BEGIN doctest.h, [283](#)
DOCTEST_TEST_SUITE_END doctest.h, [283](#)
DOCTEST_TEST_SUITE_IMPL doctest.h, [283](#)
DOCTEST_THEN doctest.h, [284](#)
DOCTEST_TO_LVALUE doctest.h, [284](#)
DOCTEST_TOSTR doctest.h, [284](#)
DOCTEST_TOSTR_IMPL doctest.h, [285](#)
DOCTEST_TYPE_TO_STRING doctest.h, [285](#)
DOCTEST_TYPE_TO_STRING_AS doctest.h, [285](#)
DOCTEST_UNARY_ASSERT doctest.h, [285](#)
DOCTEST_UNUSED doctest.h, [286](#)
DOCTEST_VERSION doctest.h, [286](#)
DOCTEST_VERSION_MAJOR doctest.h, [286](#)
DOCTEST_VERSION_MINOR doctest.h, [286](#)
DOCTEST_VERSION_PATCH doctest.h, [286](#)
DOCTEST_VERSION_STR doctest.h, [286](#)
DOCTEST_WARN doctest.h, [287](#)
DOCTEST_WARN_EQ doctest.h, [287](#)
DOCTEST_WARN_FALSE doctest.h, [287](#)
DOCTEST_WARN_FALSE_MESSAGE doctest.h, [287](#)
DOCTEST_WARN_GE doctest.h, [287](#)
DOCTEST_WARN_GT doctest.h, [288](#)
DOCTEST_WARN_LE doctest.h, [288](#)
DOCTEST_WARN_LT doctest.h, [288](#)
DOCTEST_WARN_MESSAGE doctest.h, [288](#)
DOCTEST_WARN_NE doctest.h, [288](#)
DOCTEST_WARN_NOTHROW doctest.h, [289](#)
DOCTEST_WARN_NOTHROW_MESSAGE doctest.h, [289](#)
DOCTEST_WARN_THROWS doctest.h, [289](#)
DOCTEST_WARN_THROWS_AS doctest.h, [289](#)
DOCTEST_WARN_THROWS_AS_MESSAGE doctest.h, [289](#)
DOCTEST_WARN_THROWS_MESSAGE doctest.h, [290](#)
DOCTEST_WARN_THROWS_WITH doctest.h, [290](#)
DOCTEST_WARN_THROWS_WITH_AS doctest.h, [290](#)
DOCTEST_WARN_THROWS_WITH_AS_MESSAGE doctest.h, [290](#)
DOCTEST_WARN_THROWS_WITH_MESSAGE doctest.h, [291](#)
DOCTEST_WARN_UNARY doctest.h, [291](#)
DOCTEST_WARN_UNARY_FALSE doctest.h, [291](#)
DOCTEST_WHEN doctest.h, [291](#)

- DOCTEST_WRAP_IN_TRY
 - doctest.h, [291](#)
- DT_CHECK
 - doctest::assertType, [35](#)
- DT_CHECK_EQ
 - doctest::assertType, [36](#)
- DT_CHECK_FALSE
 - doctest::assertType, [36](#)
- DT_CHECK_GE
 - doctest::assertType, [36](#)
- DT_CHECK_GT
 - doctest::assertType, [36](#)
- DT_CHECK_LE
 - doctest::assertType, [36](#)
- DT_CHECK_LT
 - doctest::assertType, [36](#)
- DT_CHECK_NE
 - doctest::assertType, [36](#)
- DT_CHECK_NOTHROW
 - doctest::assertType, [36](#)
- DT_CHECK_THROWS
 - doctest::assertType, [36](#)
- DT_CHECK_THROWS_AS
 - doctest::assertType, [36](#)
- DT_CHECK_THROWS_WITH
 - doctest::assertType, [36](#)
- DT_CHECK_THROWS_WITH_AS
 - doctest::assertType, [36](#)
- DT_CHECK_UNARY
 - doctest::assertType, [36](#)
- DT_CHECK_UNARY_FALSE
 - doctest::assertType, [36](#)
- DT_REQUIRE
 - doctest::assertType, [35](#)
- DT_REQUIRE_EQ
 - doctest::assertType, [36](#)
- DT_REQUIRE_FALSE
 - doctest::assertType, [36](#)
- DT_REQUIRE_GE
 - doctest::assertType, [36](#)
- DT_REQUIRE_GT
 - doctest::assertType, [36](#)
- DT_REQUIRE_LE
 - doctest::assertType, [36](#)
- DT_REQUIRE_LT
 - doctest::assertType, [36](#)
- DT_REQUIRE_NE
 - doctest::assertType, [36](#)
- DT_REQUIRE_NOTHROW
 - doctest::assertType, [36](#)
- DT_REQUIRE_THROWS
 - doctest::assertType, [36](#)
- DT_REQUIRE_THROWS_AS
 - doctest::assertType, [36](#)
- DT_REQUIRE_THROWS_WITH
 - doctest::assertType, [36](#)
- DT_REQUIRE_THROWS_WITH_AS
 - doctest::assertType, [36](#)
- DT_REQUIRE_UNARY
 - doctest::assertType, [36](#)
- DT_REQUIRE_UNARY_FALSE
 - doctest::assertType, [36](#)
- DT_WARN
 - doctest::assertType, [35](#)
- DT_WARN_EQ
 - doctest::assertType, [36](#)
- DT_WARN_FALSE
 - doctest::assertType, [35](#)
- DT_WARN_GE
 - doctest::assertType, [36](#)
- DT_WARN_GT
 - doctest::assertType, [36](#)
- DT_WARN_LE
 - doctest::assertType, [36](#)
- DT_WARN_LT
 - doctest::assertType, [36](#)
- DT_WARN_NE
 - doctest::assertType, [36](#)
- DT_WARN_NOTHROW
 - doctest::assertType, [36](#)
- DT_WARN_THROWS
 - doctest::assertType, [36](#)
- DT_WARN_THROWS_AS
 - doctest::assertType, [36](#)
- DT_WARN_THROWS_WITH
 - doctest::assertType, [36](#)
- DT_WARN_THROWS_WITH_AS
 - doctest::assertType, [36](#)
- DT_WARN_UNARY
 - doctest::assertType, [36](#)
- DT_WARN_UNARY_FALSE
 - doctest::assertType, [36](#)
- duration
 - doctest::ContextOptions, [72](#)
- ENGENHEIRO
 - Cargo.hpp, [191](#)
- EntradaInvalida, [85](#)
- Enum
 - doctest::assertType, [35](#)
 - doctest::Color, [37](#)
 - doctest::detail::binaryAssertComparison, [45](#)
 - doctest::TestCaseFailureReason, [46](#)
- epsilon
 - doctest::Approx, [53](#)
- eq
 - doctest::detail::binaryAssertComparison, [45](#)
- Equipamento, [86](#)
 - ~Equipamento, [87](#)
 - AdicionarParametro, [87](#), [88](#)
 - AtualizarAlarmes, [89](#)
 - DiagnosticarEquipamento, [89](#)
 - Equipamento, [87](#)
 - ExibirAlarmes, [89](#)
 - ExibirParametros, [89](#)
 - GetAlarmes, [90](#)
 - GetDesc, [90](#)

- GetId, 90
- GetNome, 90
- GetParametros, 90
- ProximoldParametro, 91
- QuantidadeAlarmes, 91
- QuantidadeParametros, 91
- RemoverParametro, 91
- SetDesc, 92
- Tipo, 92
- error_string
 - doctest::TestCaseException, 178
- EstadoOperacional
 - EstadoOperacional.hpp, 194
- EstadoOperacional.hpp
 - EstadoOperacional, 194
 - FALHA_INTERNA, 194
 - FALHA_NA_ALIMENTACAO, 194
 - FALHA_NA_SAIDA, 194
 - OPERACIONAL, 194
- ExcecaoSistema, 93
- Exception
 - doctest::TestCaseFailureReason, 46
- ExceptionTranslator
 - doctest::detail::ExceptionTranslator< T >, 94
- ExibirAlarmes
 - Conjunto, 64
 - Equipamento, 89
 - LinhaDeProducao, 125
- ExibirConjuntos
 - LinhaDeProducao, 125
- ExibirEquipamentos
 - Conjunto, 64
- ExibirLinhas
 - Sistema, 157
- ExibirParametros
 - Equipamento, 89
- ExibirTudo
 - Conjunto, 64
 - LinhaDeProducao, 125
- exit
 - doctest::ContextOptions, 73
- Expression_lhs
 - doctest::detail::Expression_lhs< L >, 95
- ExpressionDecomposer
 - doctest::detail::ExpressionDecomposer, 96
- FAIL
 - doctest.h, 292
- FAIL_CHECK
 - doctest.h, 292
- failed_out_of_a_testing_context
 - doctest::detail, 41
- FailedExactlyNumTimes
 - doctest::TestCaseFailureReason, 46
- failure_flags
 - doctest::CurrentTestCaseStats, 81
- failureString
 - doctest, 28
- FALHA_INTERNA
 - EstadoOperacional.hpp, 194
- FALHA_NA_ALIMENTACAO
 - EstadoOperacional.hpp, 194
- FALHA_NA_SAIDA
 - EstadoOperacional.hpp, 194
- FAST_CHECK_EQ
 - doctest.h, 292
- FAST_CHECK_GE
 - doctest.h, 292
- FAST_CHECK_GT
 - doctest.h, 292
- FAST_CHECK_LE
 - doctest.h, 293
- FAST_CHECK_LT
 - doctest.h, 293
- FAST_CHECK_NE
 - doctest.h, 293
- FAST_CHECK_UNARY
 - doctest.h, 293
- FAST_CHECK_UNARY_FALSE
 - doctest.h, 293
- FAST_REQUIRE_EQ
 - doctest.h, 294
- FAST_REQUIRE_GE
 - doctest.h, 294
- FAST_REQUIRE_GT
 - doctest.h, 294
- FAST_REQUIRE_LE
 - doctest.h, 294
- FAST_REQUIRE_LT
 - doctest.h, 294
- FAST_REQUIRE_NE
 - doctest.h, 295
- FAST_REQUIRE_UNARY
 - doctest.h, 295
- FAST_REQUIRE_UNARY_FALSE
 - doctest.h, 295
- FAST_WARN_EQ
 - doctest.h, 295
- FAST_WARN_GE
 - doctest.h, 295
- FAST_WARN_GT
 - doctest.h, 296
- FAST_WARN_LE
 - doctest.h, 296
- FAST_WARN_LT
 - doctest.h, 296
- FAST_WARN_NE
 - doctest.h, 296
- FAST_WARN_UNARY
 - doctest.h, 296
- FAST_WARN_UNARY_FALSE
 - doctest.h, 297
- fill
 - doctest::detail::filldata< const char[N]>, 99
 - doctest::detail::filldata< const void * >, 99
 - doctest::detail::filldata< const volatile void * >, 100
 - doctest::detail::filldata< T * >, 101

- doctest::detail::filldata< T, Enable >, 98
- doctest::detail::filldata< T, typename detail::types::enable_if<detail::has_insertion_operator< T >::value &&detail::is_container< T >::value >::type >, 101
- doctest::detail::filldata< T, typename detail::types::enable_if<detail::has_insertion_operator< T >::value &&detail::is_pair< T >::value >::type >, 102
- doctest::detail::filldata< T[N]>, 103
- filloss
 - doctest::detail, 41
- find
 - doctest::String, 163
- first
 - doctest::ContextOptions, 73
- flipped
 - doctest::IsNaN< F >, 121
- force_colors
 - doctest::ContextOptions, 73
- forward
 - doctest::detail, 42
- funcType
 - doctest::detail, 40
- ge
 - doctest::detail::binaryAssertComparison, 45
- GENERATE
 - doctest.h, 297
- get_active_contexts
 - doctest::IReporter, 108
- Get_Condicao
 - Dependencia, 83
- Get_Id
 - Dependencia, 83
- get_num_active_contexts
 - doctest::IReporter, 108
- get_num_stringified_contexts
 - doctest::IReporter, 108
- Get_Resultado
 - Dependencia, 84
- get_stringified_contexts
 - doctest::IReporter, 108
- GetAlarmes
 - Equipamento, 90
- GetArquivoLog
 - Log, 129
- GetCargo
 - Usuario, 185
- GetConjuntos
 - LinhaDeProducao, 125
- getContextOptions
 - doctest, 28
- getCurrentTestSuite
 - doctest_detail_test_suite_ns, 47
- GetDesc
 - Conjunto, 64
 - Equipamento, 90
 - LinhaDeProducao, 126
 - Parametro, 138
- GetEquipamentos
 - Conjunto, 65
- GetEstado
 - Alarme, 51
 - Parametro, 139
- GetHist
 - doctest::detail::has_insertion_operator< T >::value &&detail::is_pair< T >::value >::type >, 102
- GetId
 - Conjunto, 65
 - Equipamento, 90
 - LinhaDeProducao, 126
 - Parametro, 139
- GetLinhas
 - Sistema, 158
- GetLocal
 - LinhaDeProducao, 126
- GetLogin
 - Usuario, 185
- GetMax
 - Parametro, 139
- GetMin
 - Parametro, 140
- GetNome
 - Conjunto, 65
 - Equipamento, 90
 - LinhaDeProducao, 126
- GetParametros
 - Equipamento, 90
- GetSenha
 - Usuario, 185
- GetUsuarioLogado
 - Sistema, 158
- GetValorAtual
 - Parametro, 140
- GIVEN
 - doctest.h, 297
- gnu_file_line
 - doctest::ContextOptions, 73
- Green
 - doctest::Color, 37
- Grey
 - doctest::Color, 37
- gt
 - doctest::detail::binaryAssertComparison, 45
- help
 - doctest::ContextOptions, 73
- IdAtivo
 - Sistema, 158
- IdDuplicado, 105
- IdInexistente, 106
- IdInvalido, 106
- include Directory Reference, 23
- include/Alarme.hpp, 189
- include/Atuador.hpp, 190
- include/Cargo.hpp, 190, 191
- include/Conjunto.hpp, 191
- include/Dependencia.hpp, 192

- include/Equipamento.hpp, [193](#)
- include/EstadoOperacional.hpp, [194](#)
- include/Excecoes.hpp, [194](#), [195](#)
- include/LinhaDeProducao.hpp, [196](#)
- include/Log.hpp, [197](#)
- include/Motor.hpp, [197](#), [198](#)
- include/Parametro.hpp, [198](#)
- include/Sensor.hpp, [199](#)
- include/Sistema.hpp, [199](#), [200](#)
- include/Usuario.hpp, [200](#), [201](#)
- include/Valvula.hpp, [201](#)
- INFO
 - doctest.h, [297](#)
- instantiationHelper
 - doctest::detail, [42](#)
- is_check
 - doctest::assertType, [35](#)
- is_crash
 - doctest::TestCaseException, [178](#)
- is_eq
 - doctest::assertType, [35](#)
- is_false
 - doctest::assertType, [35](#)
- is_ge
 - doctest::assertType, [35](#)
- is_gt
 - doctest::assertType, [35](#)
- is_le
 - doctest::assertType, [35](#)
- is_lt
 - doctest::assertType, [35](#)
- is_ne
 - doctest::assertType, [35](#)
- is_normal
 - doctest::assertType, [35](#)
- is_nothrow
 - doctest::assertType, [35](#)
- is_require
 - doctest::assertType, [35](#)
- is_running_in_test
 - doctest, [34](#)
- is_throws
 - doctest::assertType, [35](#)
- is_throws_as
 - doctest::assertType, [35](#)
- is_throws_with
 - doctest::assertType, [35](#)
- is_unary
 - doctest::assertType, [35](#)
- is_warn
 - doctest::assertType, [35](#)
- isDebuggerActive
 - doctest::detail, [42](#)
- IsNaN
 - doctest::IsNaN< F >, [120](#)
- IsNaN< double >
 - doctest, [34](#)
- IsNaN< float >
 - doctest, [34](#)
- IsNaN< long double >
 - doctest, [34](#)
- istream
 - std, [47](#)
- last
 - doctest::ContextOptions, [73](#)
- le
 - doctest::detail::binaryAssertComparison, [45](#)
- lhs
 - doctest::detail::Expression_lhs< L >, [95](#)
- LightGrey
 - doctest::Color, [37](#)
- LimitesInvalidos, [121](#)
- LinhaDeProducao, [122](#)
 - AcessarConjunto, [123](#)
 - AdicionarConjunto, [124](#)
 - AtualizarAlarmes, [124](#)
 - DiagnosticarLinha, [124](#)
 - ExibirAlarmes, [125](#)
 - ExibirConjuntos, [125](#)
 - ExibirTudo, [125](#)
 - GetConjuntos, [125](#)
 - GetDesc, [126](#)
 - GetId, [126](#)
 - GetLocal, [126](#)
 - GetNome, [126](#)
 - LinhaDeProducao, [123](#)
 - ProximoidConjunto, [126](#)
 - QuantidadeAlarmes, [127](#)
 - QuantidadeConjuntos, [127](#)
 - RemoverConjunto, [127](#)
 - SetDesc, [127](#)
 - SetLocal, [128](#)
 - SetNome, [128](#)
- list_reporters
 - doctest::ContextOptions, [73](#)
- list_test_cases
 - doctest::ContextOptions, [73](#)
- list_test_suites
 - doctest::ContextOptions, [74](#)
- Log, [128](#)
 - GetArquivoLog, [129](#)
 - Log, [129](#)
 - Registrar, [129](#)
 - SetArquivoLog, [130](#)
- log
 - doctest::detail::MessageBuilder, [132](#)
 - doctest::detail::ResultBuilder, [150](#)
- log_assert
 - doctest::IReporter, [108](#)
- log_message
 - doctest::IReporter, [108](#)
- logged
 - doctest::detail::MessageBuilder, [133](#)
- Login
 - Sistema, [158](#)
- Logout

- Sistema, [159](#)
- lt
 - doctest::detail::binaryAssertComparison, [45](#)
- m_at
 - doctest::AssertData, [56](#)
 - doctest::detail::Expression_lhs< L >, [95](#)
 - doctest::detail::ExpressionDecomposer, [97](#)
- m_decomp
 - doctest::AssertData, [56](#)
 - doctest::detail::Result, [148](#)
- m_description
 - doctest::detail::TestSuite, [180](#)
 - doctest::TestCaseData, [176](#)
- m_entered
 - doctest::detail::Subcase, [170](#)
- m_epsilon
 - doctest::Approx, [55](#)
- m_exception
 - doctest::AssertData, [57](#)
- m_exception_string
 - doctest::AssertData, [57](#)
- m_exception_type
 - doctest::AssertData, [57](#)
- m_expected_failures
 - doctest::detail::TestSuite, [180](#)
 - doctest::TestCaseData, [176](#)
- m_expr
 - doctest::AssertData, [57](#)
- m_failed
 - doctest::AssertData, [57](#)
- m_file
 - doctest::AssertData, [57](#)
 - doctest::MessageData, [134](#)
 - doctest::SubcaseSignature, [171](#)
 - doctest::TestCaseData, [176](#)
- m_full_name
 - doctest::detail::TestCase, [174](#)
- m_line
 - doctest::AssertData, [57](#)
 - doctest::MessageData, [134](#)
 - doctest::SubcaseSignature, [171](#)
 - doctest::TestCaseData, [176](#)
- m_may_fail
 - doctest::detail::TestSuite, [181](#)
 - doctest::TestCaseData, [176](#)
- m_name
 - doctest::SubcaseSignature, [171](#)
 - doctest::TestCaseData, [176](#)
- m_no_breaks
 - doctest::detail::TestSuite, [181](#)
 - doctest::TestCaseData, [176](#)
- m_no_output
 - doctest::detail::TestSuite, [181](#)
 - doctest::TestCaseData, [176](#)
- m_passed
 - doctest::detail::Result, [148](#)
- m_scale
 - doctest::Approx, [55](#)
- m_severity
 - doctest::MessageData, [134](#)
- m_should_fail
 - doctest::detail::TestSuite, [181](#)
 - doctest::TestCaseData, [177](#)
- m_signature
 - doctest::detail::Subcase, [170](#)
- m_skip
 - doctest::detail::TestSuite, [181](#)
 - doctest::TestCaseData, [177](#)
- m_stream
 - doctest::detail::MessageBuilder, [133](#)
- m_string
 - doctest::MessageData, [134](#)
- m_template_id
 - doctest::detail::TestCase, [174](#)
- m_test
 - doctest::detail::TestCase, [174](#)
- m_test_case
 - doctest::AssertData, [57](#)
- m_test_suite
 - doctest::detail::TestSuite, [181](#)
 - doctest::TestCaseData, [177](#)
- m_threw
 - doctest::AssertData, [58](#)
- m_threw_as
 - doctest::AssertData, [58](#)
- m_timeout
 - doctest::detail::TestSuite, [181](#)
 - doctest::TestCaseData, [177](#)
- m_type
 - doctest::detail::TestCase, [175](#)
- m_value
 - doctest::Approx, [55](#)
- main
 - main.cpp, [209](#)
- main.cpp
 - main, [209](#)
- MakeContextScope
 - doctest::detail, [42](#)
- MESSAGE
 - doctest.h, [297](#)
- MessageBuilder
 - doctest::detail::MessageBuilder, [131](#)
- minimal
 - doctest::ContextOptions, [74](#)
- Motor, [134](#)
 - ~Motor, [136](#)
 - Motor, [136](#)
 - Tipo, [136](#)
- ne
 - doctest::detail::binaryAssertComparison, [45](#)
- need_to_destroy
 - doctest::detail::ContextScopeBase, [81](#)
- no_breaks
 - doctest::ContextOptions, [74](#)
- no_colors
 - doctest::ContextOptions, [74](#)

- no_debug_output
 - doctest::ContextOptions, 74
- no_exitcode
 - doctest::ContextOptions, 74
- no_intro
 - doctest::ContextOptions, 74
- no_line_numbers
 - doctest::ContextOptions, 74
- no_path_in_filenames
 - doctest::ContextOptions, 75
- no_run
 - doctest::ContextOptions, 75
- no_skip
 - doctest::ContextOptions, 75
- no_skipped_summary
 - doctest::ContextOptions, 75
- no_throw
 - doctest::ContextOptions, 75
- no_time_in_output
 - doctest::ContextOptions, 75
- no_version
 - doctest::ContextOptions, 75
- None
 - doctest::Color, 37
 - doctest::TestCaseFailureReason, 46
- notas-C6, 5
- notas-C7, 7
- npos
 - doctest::String, 165
- nullptr_t
 - std, 47
- num_data
 - doctest::QueryData, 143
- numAsserts
 - doctest::TestRunStats, 179
- numAssertsCurrentTest
 - doctest::CurrentTestCaseStats, 81
- numAssertsFailed
 - doctest::TestRunStats, 179
- numAssertsFailedCurrentTest
 - doctest::CurrentTestCaseStats, 82
- numTestCases
 - doctest::TestRunStats, 179
- numTestCasesFailed
 - doctest::TestRunStats, 179
- numTestCasesPassingFilters
 - doctest::TestRunStats, 179
- numTestSuitesPassingFilters
 - doctest::TestRunStats, 179
- OPERACIONAL
 - EstadoOperacional.hpp, 194
- operator bool
 - doctest::detail::Subcase, 170
 - doctest::IsNaN< F >, 121
- operator const String &
 - doctest::AssertData::StringContains, 166
- operator L
 - doctest::detail::Expression_lhs< L >, 95
- operator Result
 - doctest::detail::Expression_lhs< L >, 95
- operator!
 - doctest::IsNaN< F >, 121
- operator!=
 - doctest, 29
 - doctest::Approx, 53
- operator<
 - doctest, 29
 - doctest::Approx, 53, 54
 - doctest::detail::TestCase, 174
 - doctest::SubcaseSignature, 171
- operator<<
 - doctest::Color, 37
 - doctest::detail::ExpressionDecomposer, 96
 - doctest::detail::MessageBuilder, 132
 - doctest::String, 165
 - std, 48
- operator<=
 - doctest, 29
 - doctest::Approx, 54
- operator>
 - doctest, 30
 - doctest::Approx, 54
- operator>=
 - doctest, 30
 - doctest::Approx, 54, 55
- operator()
 - doctest::Approx, 53
 - doctest::detail::RelationalComparator< int, L, R >, 144
- operator+
 - doctest, 29
- operator+=
 - doctest::String, 163
- operator,
 - doctest::detail::MessageBuilder, 132
- operator=
 - doctest::Context, 69, 70
 - doctest::detail::ContextScope< L >, 79
 - doctest::detail::ContextScopeBase, 80, 81
 - doctest::detail::MessageBuilder, 132
 - doctest::detail::Subcase, 170
 - doctest::detail::TestCase, 174
 - doctest::String, 164
- operator==
 - doctest, 29, 30
 - doctest::Approx, 54
 - doctest::SubcaseSignature, 171
- operator[]
 - doctest::String, 164
- operator*
 - doctest::detail::MessageBuilder, 132
 - doctest::detail::TestCase, 174
 - doctest::detail::TestSuite, 180
- order_by
 - doctest::ContextOptions, 75
- ostream

- std, [47](#)
- out
 - doctest::ContextOptions, [76](#)
- Parametro, [136](#)
 - DiagnosticarParametro, [138](#)
 - GetDesc, [138](#)
 - GetEstado, [139](#)
 - GetHist, [139](#)
 - GetId, [139](#)
 - GetMax, [139](#)
 - GetMin, [140](#)
 - GetValorAtual, [140](#)
 - Parametro, [137](#), [138](#)
 - ResetarHistorico, [140](#)
 - ResetarLimites, [140](#)
 - SetDesc, [140](#)
 - SetMax, [141](#)
 - SetMin, [141](#)
 - SetValorAtual, [141](#)
 - ValorValido, [142](#)
- ProximoldConjunto
 - LinhaDeProducao, [126](#)
- ProximoldDisponivel
 - Sistema, [159](#)
- ProximoldEquipamento
 - Conjunto, [65](#)
- ProximoldParametro
 - Equipamento, [91](#)
- QuantidadeAlarmes
 - Conjunto, [65](#)
 - Equipamento, [91](#)
 - LinhaDeProducao, [127](#)
- QuantidadeConjuntos
 - LinhaDeProducao, [127](#)
- QuantidadeEquipamentos
 - Conjunto, [66](#)
- QuantidadeParametros
 - Equipamento, [91](#)
- quiet
 - doctest::ContextOptions, [76](#)
- rand_seed
 - doctest::ContextOptions, [76](#)
- react
 - doctest::detail::MessageBuilder, [133](#)
 - doctest::detail::ResultBuilder, [150](#)
- README.md, [202](#)
- Red
 - doctest::Color, [37](#)
- REGISTER_EXCEPTION_TRANSLATOR
 - doctest.h, [298](#)
- REGISTER_LISTENER
 - doctest.h, [298](#)
- REGISTER_REPORTER
 - doctest.h, [298](#)
- registerExceptionTranslator
 - doctest, [30](#)
- registerExceptionTranslatorImpl
 - doctest::detail, [42](#)
- registerReporter
 - doctest, [30](#)
- registerReporterImpl
 - doctest::detail, [42](#)
- Registrar
 - Log, [129](#)
- regTest
 - doctest::detail, [43](#)
- RemoverConjunto
 - LinhaDeProducao, [127](#)
- RemoverEquipamento
 - Conjunto, [66](#)
- RemoverLinha
 - Sistema, [159](#)
- RemoverParametro
 - Equipamento, [91](#)
- RemoverUsuario
 - Sistema, [160](#)
- report_query
 - doctest::IReporter, [108](#)
- reporterCreator
 - doctest::detail, [43](#)
- reporterCreatorFunc
 - doctest::detail, [40](#)
- REQUIRE
 - doctest.h, [298](#)
- REQUIRE_EQ
 - doctest.h, [298](#)
- REQUIRE_FALSE
 - doctest.h, [299](#)
- REQUIRE_FALSE_MESSAGE
 - doctest.h, [299](#)
- REQUIRE_GE
 - doctest.h, [299](#)
- REQUIRE_GT
 - doctest.h, [299](#)
- REQUIRE_LE
 - doctest.h, [299](#)
- REQUIRE_LT
 - doctest.h, [300](#)
- REQUIRE_MESSAGE
 - doctest.h, [300](#)
- REQUIRE_NE
 - doctest.h, [300](#)
- REQUIRE_NOTHROW
 - doctest.h, [300](#)
- REQUIRE_NOTHROW_MESSAGE
 - doctest.h, [300](#)
- REQUIRE_THROWS
 - doctest.h, [301](#)
- REQUIRE_THROWS_AS
 - doctest.h, [301](#)
- REQUIRE_THROWS_AS_MESSAGE
 - doctest.h, [301](#)
- REQUIRE_THROWS_MESSAGE
 - doctest.h, [301](#)

- REQUIRE_THROWS_WITH
 - doctest.h, [301](#)
- REQUIRE_THROWS_WITH_AS
 - doctest.h, [302](#)
- REQUIRE_THROWS_WITH_AS_MESSAGE
 - doctest.h, [302](#)
- REQUIRE_THROWS_WITH_MESSAGE
 - doctest.h, [302](#)
- REQUIRE_UNARY
 - doctest.h, [302](#)
- REQUIRE_UNARY_FALSE
 - doctest.h, [303](#)
- ResetarHistorico
 - Parametro, [140](#)
- ResetarLimites
 - Parametro, [140](#)
- Result
 - doctest::detail::Result, [148](#)
- ResultBuilder
 - doctest::detail::ResultBuilder, [149](#)
- rfind
 - doctest::String, [164](#)
- run
 - doctest::Context, [70](#)
- run_stats
 - doctest::QueryData, [143](#)
- SalvarAlteracoes
 - Sistema, [160](#)
- scale
 - doctest::Approx, [53](#)
- SCENARIO
 - doctest.h, [303](#)
- SCENARIO_CLASS
 - doctest.h, [303](#)
- SCENARIO_METHOD
 - doctest.h, [303](#)
- SCENARIO_TEMPLATE
 - doctest.h, [303](#)
- SCENARIO_TEMPLATE_DEFINE
 - doctest.h, [304](#)
- seconds
 - doctest::CurrentTestCaseStats, [82](#)
- Sensor, [151](#)
 - ~Sensor, [152](#)
 - Sensor, [152](#)
 - Tipo, [152](#)
- SessaoInvalida, [153](#)
- SetArquivoLog
 - Log, [130](#)
- setAsDefaultForAssertsOutOfTestCases
 - doctest::Context, [70](#)
- setAssertHandler
 - doctest::Context, [70](#)
- setCout
 - doctest::Context, [70](#)
- SetDesc
 - Alarme, [51](#)
 - Conjunto, [66](#)
 - Equipamento, [92](#)
 - LinhaDeProducao, [127](#)
 - Parametro, [140](#)
- SetEstado
 - Alarme, [51](#)
- SetLocal
 - LinhaDeProducao, [128](#)
- SetMax
 - Parametro, [141](#)
- SetMin
 - Parametro, [141](#)
- SetNome
 - Conjunto, [67](#)
 - LinhaDeProducao, [128](#)
- setOption
 - doctest::Context, [70](#)
- setResult
 - doctest::detail::ResultBuilder, [150](#)
- setTestSuite
 - doctest::detail, [43](#)
- SetValorAtual
 - Parametro, [141](#)
- SFINAE_OP
 - doctest.h, [304](#)
- shouldExit
 - doctest::Context, [70](#)
- ShouldHaveFailedAndDid
 - doctest::TestCaseFailureReason, [46](#)
- ShouldHaveFailedButDidnt
 - doctest::TestCaseFailureReason, [46](#)
- Sistema, [154](#)
 - AcessarLinha, [155](#)
 - AdicionarLinha, [155](#), [156](#)
 - AdicionarUsuario, [156](#)
 - CarregarSave, [157](#)
 - CarregarUltimoSave, [157](#)
 - ExibirLinhas, [157](#)
 - GetLinhas, [158](#)
 - GetUsuarioLogado, [158](#)
 - IdAtivo, [158](#)
 - Login, [158](#)
 - Logout, [159](#)
 - ProximoldDisponivel, [159](#)
 - RemoverLinha, [159](#)
 - RemoverUsuario, [160](#)
 - SalvarAlteracoes, [160](#)
 - Sistema, [155](#)
- Sistema de Tolerância de Falha em Fábricas, [1](#)
- size
 - doctest::String, [164](#)
- size_t
 - doctest, [27](#)
 - std, [47](#)
- size_type
 - doctest::String, [162](#)
- skipPathFromFilename
 - doctest, [30](#)
- src Directory Reference, [23](#)

- src/Alarme.cpp, 202
- src/Atuador.cpp, 202
- src/Conjunto.cpp, 203
- src/Dependencia.cpp, 204, 205
- src/Equipamento.cpp, 205
- src/LinhaDeProducao.cpp, 207
- src/Log.cpp, 208
- src/main.cpp, 209, 210
- src/Motor.cpp, 213
- src/Parametro.cpp, 214
- src/Sensor.cpp, 215
- src/Sistema.cpp, 216
- src/Usuario.cpp, 219
- src/Valvula.cpp, 220
- std, 47
 - istream, 47
 - nullptr_t, 47
 - operator<<, 48
 - ostream, 47
 - size_t, 47
- std::basic_istream< charT, traits >, 60
- std::basic_ostream< charT, traits >, 60
- std::char_traits< charT >, 61
- std::tuple< Types >, 183
- String
 - doctest::String, 162, 163
- string
 - doctest::Contains, 68
- StringContains
 - doctest::AssertData::StringContains, 166
- stringify
 - doctest::detail::ContextScope< L >, 79
 - doctest::IContextScope, 105
- stringifyBinaryExpr
 - doctest::detail, 43
- strip_file_prefixes
 - doctest::ContextOptions, 76
- SUBCASE
 - doctest.h, 304
- Subcase
 - doctest::detail::Subcase, 169, 170
- subcase_end
 - doctest::IReporter, 108
- subcase_filter_levels
 - doctest::ContextOptions, 76
- subcase_start
 - doctest::IReporter, 109
- substr
 - doctest::String, 164
- success
 - doctest::ContextOptions, 76
- TECNICO
 - Cargo.hpp, 191
- test_Alarme.cpp
 - TEST_CASE, 426, 427
- TEST_CASE
 - doctest.h, 304
 - test_Alarme.cpp, 426, 427
 - test_Conjunto.cpp, 429–432
 - test_Equipamento.cpp, 435–438
 - test_Excecoes.cpp, 441–444
 - test_LinhaDeProducao.cpp, 446–449
 - test_Log.cpp, 452, 453
 - test_Parametro.cpp, 455, 456
 - test_Sistema.cpp, 459–462
 - test_Usuario.cpp, 465
- TEST_CASE_CLASS
 - doctest.h, 304
- test_case_end
 - doctest::IReporter, 109
- test_case_exception
 - doctest::IReporter, 109
- TEST_CASE_FIXTURE
 - doctest.h, 305
- test_case_reenter
 - doctest::IReporter, 109
- test_case_skipped
 - doctest::IReporter, 109
- test_case_start
 - doctest::IReporter, 109
- TEST_CASE_TEMPLATE
 - doctest.h, 305
- TEST_CASE_TEMPLATE_APPLY
 - doctest.h, 305
- TEST_CASE_TEMPLATE_DEFINE
 - doctest.h, 305
- TEST_CASE_TEMPLATE_INSTANTIATE
 - doctest.h, 305
- TEST_CASE_TEMPLATE_INVOKE
 - doctest.h, 306
- test_Conjunto.cpp
 - TEST_CASE, 429–432
- test_Equipamento.cpp
 - TEST_CASE, 435–438
- test_Excecoes.cpp
 - TEST_CASE, 441–444
- test_LinhaDeProducao.cpp
 - TEST_CASE, 446–449
- test_Log.cpp
 - TEST_CASE, 452, 453
- test_main.cpp
 - DOCTEST_CONFIG_IMPLEMENT_WITH_MAIN, 454
- test_Parametro.cpp
 - TEST_CASE, 455, 456
- test_run_end
 - doctest::IReporter, 109
- test_run_start
 - doctest::IReporter, 109
- test_Sistema.cpp
 - TEST_CASE, 459–462
- TEST_SUITE
 - doctest.h, 306
- TEST_SUITE_BEGIN
 - doctest.h, 306
- TEST_SUITE_END

- doctest.h, 306
- test_Uuario.cpp
 - TEST_CASE, 465
- TestCase
 - doctest::detail, 44
 - doctest::detail::TestCase, 173
- TestCaseData
 - doctest, 34
- testCaseSuccess
 - doctest::CurrentTestCaseStats, 82
- tests Directory Reference, 24
- tests/doctest.h, 220, 312
- tests/test_Alarme.cpp, 426, 427
- tests/test_Conjunto.cpp, 428, 433
- tests/test_Equipamento.cpp, 435, 439
- tests/test_Excecoes.cpp, 440, 444
- tests/test_LinhaDeProducao.cpp, 446, 450
- tests/test_Log.cpp, 452, 453
- tests/test_main.cpp, 454
- tests/test_Parametro.cpp, 454, 457
- tests/test_Sistema.cpp, 458, 462
- tests/test_Uuario.cpp, 464, 466
- THEN
 - doctest.h, 306
- throwException
 - doctest::detail, 43
- Timeout
 - doctest::TestCaseFailureReason, 46
- Tipo
 - Atuador, 60
 - Equipamento, 92
 - Motor, 136
 - Sensor, 152
 - Valvula, 188
- tlssPop
 - doctest::detail, 43
- tlssPush
 - doctest::detail, 43
- TO_LVALUE
 - doctest.h, 307
- TooManyFailedAsserts
 - doctest::TestCaseFailureReason, 46
- toStream
 - doctest::detail, 44
- toString
 - doctest, 30–33
- translate
 - doctest::detail::ExceptionTranslator< T >, 94
 - doctest::detail::IExceptionTranslator, 107
- translateException
 - doctest::detail::ResultBuilder, 150
- type
 - doctest::detail::types::enable_if< true, T >, 85
 - doctest::detail::types::remove_const< const T >, 145
 - doctest::detail::types::remove_const< T >, 144
 - doctest::detail::types::remove_reference< T >, 146
 - doctest::detail::types::remove_reference< T & >, 146
 - doctest::detail::types::remove_reference< T && >, 147
 - doctest::detail::types::underlying_type< T >, 183
- TYPE_TO_STRING
 - doctest.h, 307
- TYPE_TO_STRING_AS
 - doctest.h, 307
- unary_assert
 - doctest::detail, 44
 - doctest::detail::ResultBuilder, 150
- Uuario, 184
 - GetCargo, 185
 - GetLogin, 185
 - GetSenha, 185
 - Uuario, 184
 - ValidarSenha, 185
- ValidarSenha
 - Uuario, 185
- ValorValido
 - Parametro, 142
- value
 - doctest::detail::should_stringify_as_underlying_type< T >, 154
 - doctest::detail::types::false_type, 98
 - doctest::detail::types::is_enum< T >, 113
 - doctest::detail::types::true_type, 182
 - doctest::IsNaN< F >, 121
- Valvula, 186
 - ~Valvula, 187
 - Tipo, 188
 - Valvula, 187
- version
 - doctest::ContextOptions, 76
- void_t
 - doctest::detail::types, 46
- WARN
 - doctest.h, 307
- WARN_EQ
 - doctest.h, 307
- WARN_FALSE
 - doctest.h, 308
- WARN_FALSE_MESSAGE
 - doctest.h, 308
- WARN_GE
 - doctest.h, 308
- WARN_GT
 - doctest.h, 308
- WARN_LE
 - doctest.h, 308
- WARN_LT
 - doctest.h, 309
- WARN_MESSAGE
 - doctest.h, 309
- WARN_NE
 - doctest.h, 309

- doctest.h, [309](#)
- WARN_NOTHROW
 - doctest.h, [309](#)
- WARN_NOTHROW_MESSAGE
 - doctest.h, [309](#)
- WARN_THROWS
 - doctest.h, [310](#)
- WARN_THROWS_AS
 - doctest.h, [310](#)
- WARN_THROWS_AS_MESSAGE
 - doctest.h, [310](#)
- WARN_THROWS_MESSAGE
 - doctest.h, [310](#)
- WARN_THROWS_WITH
 - doctest.h, [310](#)
- WARN_THROWS_WITH_AS
 - doctest.h, [311](#)
- WARN_THROWS_WITH_AS_MESSAGE
 - doctest.h, [311](#)
- WARN_THROWS_WITH_MESSAGE
 - doctest.h, [311](#)
- WARN_UNARY
 - doctest.h, [311](#)
- WARN_UNARY_FALSE
 - doctest.h, [312](#)
- WHEN
 - doctest.h, [312](#)
- White
 - doctest::Color, [37](#)
- Yellow
 - doctest::Color, [37](#)